

Data Science Foundations

**Python, NumPy, Pandas, Matplotlib, Seaborn, and Real World
Analysis**

Kristipati Thoyajaksha Kashyap

2026

Copyright © 2026 Kristipati Thoyajaksha Kashyap.

All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form without prior written permission of the author.

Typeset with $\LaTeX$ using the Pandoc + XeLaTeX toolchain.

Code output figures are reproduced directly from executed course notebooks.

Contents

1	Introduction	1
1.1	How this book works	1
1.2	What data science is	1
1.3	Where data science shows up every day	1
1.4	Who this book is for	2
1.5	How to read the code	2
1.6	Notation used throughout	2
1.7	What comes next	2
2	Python Essentials	3
2.1	Variables and types	3
2.2	Arithmetic and operators	3
2.3	Strings	4
2.4	Control flow	4
2.5	Lists	5
2.6	Dictionaries	6
2.7	Tuples and sets	7
2.8	Functions	7
2.9	Exceptions	8
2.10	Classes	9
2.10.1	A first class	9
2.10.2	Class attributes versus instance attributes	10
2.10.3	Class methods and static methods	10
2.10.4	Encapsulation	11
2.10.5	Inheritance	12
2.10.6	Polymorphism	13
2.10.7	Abstraction	14
2.10.8	More dunder methods and operator overloading	15
2.11	Reading and writing files	16
2.12	Modules	16
2.13	Mutability, more precisely	17
2.14	Scope and closures	18
2.15	Iterators and generators	19
2.16	Comprehensions beyond lists	20
2.17	Decorators	21
2.18	String formatting in depth	21

3	NumPy	25
3.1	Understanding the ndarray	25
3.1.1	What a NumPy array actually is in memory	25
3.1.2	Strides: how NumPy finds any element without searching	26
3.1.3	Vectorization: why “no explicit loop” is not a style preference	26
3.1.4	Broadcasting: the precise rule, not just the intuition	27
3.1.5	Views versus copies: the rule that prevents the most common NumPy bug	27
3.2	NumPy NN, NumPy Foundations	28
3.3	NumPy NN, NumPy Advanced	44
3.4	NumPy NN, NumPy Exercises	61
3.5	NumPy NN, NumPy Solutions	73
4	Pandas	85
4.1	Pandas, Pandas Foundations	85
4.2	Pandas, Pandas Intermediate	101
4.3	Pandas, Advanced Pandas	121
4.4	Pandas, Exercises	136
4.5	Pandas, Solutions	140
5	Matplotlib	151
5.1	Matplotlib, Foundations	151
5.2	Matplotlib, Advanced	163
5.3	Matplotlib, Exercises	176
5.4	Matplotlib, Solutions	179
6	Seaborn	193
6.1	Seaborn, Seaborn Foundations	193
6.2	Seaborn, Seaborn Advanced	216
6.3	Seaborn, Seaborn Exercises	240
6.4	Seaborn, Seaborn Solutions	248
7	Data Preprocessing	269
7.1	Data Preprocessing, Handling Missing Data	269
7.2	Data Preprocessing, Processing Numerical Data	284
7.3	Data Preprocessing, Encoding Categorical Data	298
7.4	Text Data Stdlib	308
7.5	What you'll learn	308
7.6	Data Preprocessing, Text Data with NLTK	326
7.7	Data Preprocessing, The Complete Workflow	334
7.8	Data Preprocessing, Exercises	343
7.9	Data Preprocessing, Solutions	346

8	Capstone Projects	355
8.1	Capstone, Titanic Survival EDA	355
8.2	Capstone, Restaurant Tips Analytics	364
8.3	Capstone, Palmer Penguins Statistical EDA	373
8.4	Capstone, Airline Passengers Time Series Dashboard	385
8.5	Capstone, Movie Review Text Analytics (IMDB)	396

List of Figures

1	1.4 Resampling, groupby for time (ch03_pandas_03 figure 1)	125
2	1.5 Rolling & expanding windows (ch03_pandas_03 figure 2)	126
3	1.7 Worked mini analysis: the flights dataset (ch03_pandas_03 figure 3)	128
4	Task 11, Time series basics on flights (ch03_pandas_05 figure 1)	147
5	2. Your first plot, and what pyplot does (ch04_matplotlib_01 figure 1)	153
6	3. Line plots, full styling control (ch04_matplotlib_01 figure 2)	154
7	4. Scatter plots, relationships between two variables (ch04_matplotlib_01 figure 3)	155
8	5. Bar charts, comparing categories (ch04_matplotlib_01 figure 4)	156
9	6. Histograms, the shape of ONE variable (ch04_matplotlib_01 figure 5)	157
10	7. Pie charts, parts of a whole (with honesty rules) (ch04_matplotlib_01 figure 6)	159
11	8. Box plots, five number summaries & outliers (ch04_matplotlib_01 figure 7)	160
12	9. Labels, titles, grids, limits, ticks, polish toolkit (ch04_matplotlib_01 figure 8)	161
13	10. Figure control & saving (ch04_matplotlib_01 figure 9)	162
14	2. Subplot grids & mosaics (ch04_matplotlib_02 figure 1)	164
15	2. Subplot grids & mosaics (ch04_matplotlib_02 figure 2)	165
16	3. Twin & secondary axes (ch04_matplotlib_02 figure 3)	167
17	3. Twin & secondary axes (ch04_matplotlib_02 figure 4)	167
18	Dense scatter? Go 2 D. (ch04_matplotlib_02 figure 5)	169
19	5. Error bars & uncertainty bands (ch04_matplotlib_02 figure 6)	170
20	6. Log scales, when magnitude spans decades (ch04_matplotlib_02 figure 7)	171
21	7. Styles & rcParams, global look and feel (ch04_matplotlib_02 figure 8)	172
22	7. Styles & rcParams, global look and feel (ch04_matplotlib_02 figure 9)	172
23	8. Choosing colormaps deliberately (ch04_matplotlib_02 figure 10)	173
24	9. Plotting directly from pandas (ch04_matplotlib_02 figure 11)	174
25	9. Plotting directly from pandas (ch04_matplotlib_02 figure 12)	174
26	10. 3 D plots (ch04_matplotlib_02 figure 13)	175
27	Solution 1, Revenue trends with a recession band (ch04_matplotlib_04 figure 1)	180
28	Solution 2, Grouped AND stacked bars (ch04_matplotlib_04 figure 2)	181
29	Solution 3, Overlaid step histograms (ch04_matplotlib_04 figure 3)	182
30	Solution 4, Labeled scatter of cities (ch04_matplotlib_04 figure 4)	183
31	Solution 5, Dual axis weather dashboard (ch04_matplotlib_04 figure 5)	184
32	Solution 6, 2x2 mosaic mini dashboard (ch04_matplotlib_04 figure 6)	185
33	Solution 7, Heatmap with custom ticks (ch04_matplotlib_04 figure 7)	186
34	Solution 8, Forecast with uncertainty cone (ch04_matplotlib_04 figure 8)	187
35	Solution 9, Power law on log log with fit (ch04_matplotlib_04 figure 9)	188

36	Solution 10, 3 D surface with camera control (ch04_matplotlib_04 figure 10)	189
37	Solution 11, Candlestick style OHLC (ch04_matplotlib_04 figure 11)	190
38	Solution 12, Sparklines row (ch04_matplotlib_04 figure 12)	191
39	4. Semantic mapping: hue, style, size, markers (ch05_seaborn_01 figure 1)	198
40	Synthetic example: 60 days × 40 sensors measuring a drifting signal. (ch05_seaborn_01 figure 2)	200
41	6. Histograms done right: histplot (ch05_seaborn_01 figure 3)	201
42	6. Histograms done right: histplot (ch05_seaborn_01 figure 4)	202
43	Bandwidth sweep on one species so the effect is unmistakable. (ch05_seaborn_01 figure 5)	203
44	Bandwidth sweep on one species so the effect is unmistakable. (ch05_seaborn_01 figure 6)	204
45	8. ECDF and rugs: honesty first distributions (ch05_seaborn_01 figure 7)	205
46	9. Boxplots: the five number summary, drawn (ch05_seaborn_01 figure 8)	207
47	10. Violin and boxen: richer box summaries (ch05_seaborn_01 figure 9)	209
48	11. Strip and swarm: showing every observation (ch05_seaborn_01 figure 10) . .	210
49	11. Strip and swarm: showing every observation (ch05_seaborn_01 figure 11) . .	211
50	13. catplot: faceting made trivial (ch05_seaborn_01 figure 12)	213
51	Logistic fit needs the optional statsmodels dependency; to keep this course (ch05_seaborn_02 figure 1)	219
52	3. lmpplot: faceted regression (ch05_seaborn_02 figure 2)	219
53	4. residplot: interrogating the fit (ch05_seaborn_02 figure 3)	220
54	5. Matrix heatmaps + a full correlation workflow (ch05_seaborn_02 figure 4) . . .	222
55	6. clustermap: clustered heatmap one liner (ch05_seaborn_02 figure 5)	224
56	7. jointplot: bivariate + marginals (ch05_seaborn_02 figure 6)	226
57	7. jointplot: bivariate + marginals (ch05_seaborn_02 figure 7)	227
58	7. jointplot: bivariate + marginals (ch05_seaborn_02 figure 8)	228
59	8. pairplot: the scatterplot matrix (ch05_seaborn_02 figure 9)	229
60	9. FacetGrid masterclass: custom multi panel figures (ch05_seaborn_02 figure 10) .	231
61	10. PairGrid: bespoke scatterplot matrices (ch05_seaborn_02 figure 11)	233
62	Context scaling: same chart prepared for a paper vs a conference talk. (ch05_seaborn_02 figure 12)	234
63	12. Matplotlib interop: seaborn inside your figures (ch05_seaborn_02 figure 13) . .	236
64	12. Matplotlib interop: seaborn inside your figures (ch05_seaborn_02 figure 14) . .	237
65	12. Matplotlib interop: seaborn inside your figures (ch05_seaborn_02 figure 15) . .	237
66	14. Performance and etiquette on large data (ch05_seaborn_02 figure 16)	239
67	Challenge 1, Mean spend by day *(Easy) (ch05_seaborn_04 figure 1)	252
68	Challenge 2, ECDF with threshold read off *(Easy) (ch05_seaborn_04 figure 2) . . .	253
69	Challenge 3, Corner pairplot by origin *(Easy) (ch05_seaborn_04 figure 3)	254
70	Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 4)	255
71	Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 5)	255
72	Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 6)	256

73	Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 7)	257
74	Challenge 5, Faceted regression per origin *(Medium) (ch05_seaborn_04 figure 8) .	258
75	Challenge 6, One variable, three lenses *(Medium) (ch05_seaborn_04 figure 9) . . .	259
76	Challenge 7, Masked correlation heatmap *(Medium) (ch05_seaborn_04 figure 10)	260
77	Challenge 8, Hexbin jointplot on big data *(Medium) (ch05_seaborn_04 figure 11) .	262
78	Challenge 9, Pivot heatmap with centered midpoint *(Medium) (ch05_seaborn_04 figure 12)	263
79	Challenge 10, FacetGrid percentile bands *(Hard) (ch05_seaborn_04 figure 13) . . .	265
80	Challenge 11, Box vs boxen showdown *(Hard) (ch05_seaborn_04 figure 14)	266
81	Challenge 12, Mini dashboard *(Hard) (ch05_seaborn_04 figure 15)	268
82	4. Detection: making the invisible visible (ch06_preprocessing_01 figure 1)	275
83	6b. Mean vs median, the skew showdown (ch06_preprocessing_01 figure 2)	278
84	2. StandardScaler, the z score default (ch06_preprocessing_02 figure 1)	287
85	4. RobustScaler, statistics for messy rooms (ch06_preprocessing_02 figure 2)	290
86	5. Skew fixes: bending the curve (scaling can't) (ch06_preprocessing_02 figure 3) .	291
87	6. Binning: cut vs qcut (ch06_preprocessing_02 figure 4)	294
88	7. Outliers: detect, diagnose, treat (ch06_preprocessing_02 figure 5)	295
89	8. Putting it together: a mini numeric prep function (ch06_preprocessing_02 figure 6)	298
90	7.1 Cyclical encoding, because hour 23 and hour 0 are NEIGHBORS (ch06_preprocessing_03 figure 1)	306
91	Task 6, Skew check + log transform (ch06_preprocessing_08 figure 1)	350
92	Phase 1, Audit (ch07_capstone_01 figure 1)	357
93	Phase 3, Who survived? Segmentation (ch07_capstone_01 figure 2)	360
94	Phase 4, Numeric structure & correlations (ch07_capstone_01 figure 3)	362
95	Phase 5, Dashboard (ch07_capstone_01 figure 4)	364
96	Phase 2, When do people tip best? (ch07_capstone_02 figure 1)	368
97	Phase 3, Who tips best? Segments (ch07_capstone_02 figure 2)	369
98	Phase 4, The size effect & the bill relationship (ch07_capstone_02 figure 3)	370
99	Phase 5, Dashboard (ch07_capstone_02 figure 4)	372
100	Phase 2, Species profiles (ch07_capstone_03 figure 1)	376
101	Phase 3, The discriminator: bill geometry ratio (ch07_capstone_03 figure 2)	377
102	Phase 4, Sex dimorphism within species (ch07_capstone_03 figure 3)	378
103	Phase 5, Geography & correlation structure (ch07_capstone_03 figure 4)	379
104	Phase 5, Geography & correlation structure (ch07_capstone_03 figure 5)	381
105	Phase 5, Geography & correlation structure (ch07_capstone_03 figure 6)	382
106	Phase 7, Synthesis pairplot + dashboard (ch07_capstone_03 figure 7)	383
107	Phase 7, Synthesis pairplot + dashboard (ch07_capstone_03 figure 8)	385
108	Phase 1, Build a proper time series (ch07_capstone_04 figure 1)	387
109	Phase 2, Trend & growth quantification (ch07_capstone_04 figure 2)	389
110	Phase 3, Seasonality profile (ch07_capstone_04 figure 3)	390
111	Phase 3, Seasonality profile (ch07_capstone_04 figure 4)	390

112	Phase 4, Manual decomposition (teach the machine!) (ch07_capstone_04 figure 5) .	392
113	Phase 5, Naive forecast teaser + MAPE (ch07_capstone_04 figure 6)	393
114	Phase 6, Executive dashboard (ch07_capstone_04 figure 7)	395
115	Seeded sample, 10k rows for interactive speed (ch07_capstone_05 figure 1)	403
116	4 · Exploratory data analysis (ch07_capstone_05 figure 2)	408
117	4 · Exploratory data analysis (ch07_capstone_05 figure 3)	409
118	4 · Exploratory data analysis (ch07_capstone_05 figure 4)	410
119	4 · Exploratory data analysis (ch07_capstone_05 figure 5)	411
120	4 · Exploratory data analysis (ch07_capstone_05 figure 6)	412
121	4 · Exploratory data analysis (ch07_capstone_05 figure 7)	413
122	5 · Distinctive vocabulary, TF IDF log ratio (the money chart) (ch07_capstone_05 figure 8)	415
123	7 · Word clouds (ch07_capstone_05 figure 9)	419
124	7 · Word clouds (ch07_capstone_05 figure 10)	420
125	8 · Executive dashboard (ch07_capstone_05 figure 11)	422

Chapter 1

Introduction

1.1 How this book works

This book is Volume 1 of a three part series covering data science, machine learning, and agentic AI. This volume covers the foundations everything else depends on: Python essentials, the core numerical and data manipulation libraries (NumPy, Pandas), the visualization libraries (Matplotlib, Seaborn), the discipline of preparing real data for analysis, and five complete worked case studies that put every earlier tool to use at once.

Every code example in this book was executed and its real output, including every plotted figure, is reproduced exactly as it appeared when run. Nothing here is aspirational or untested; if a line of code appears on a page, it ran, and if a chart appears, it is the actual image that code produced. Where a topic needs a conceptual diagram that isn't a code output, a memory layout illustration, for instance, the book marks the gap explicitly rather than inventing one.

1.2 What data science is

Data science is the practice of turning raw data into decisions. It combines three skills. Programming, to collect and clean data at scale. Statistics, to separate a real pattern from random noise. Communication, to turn a finding into something a person can act on. A data scientist rarely builds a single model and stops. The real work is asking the right question of a dataset, checking the answer against reality, and explaining it clearly enough that someone changes what they do because of it.

1.3 Where data science shows up every day

A streaming service decides what to recommend next by analyzing what millions of people watched before. A bank flags a suspicious transaction in milliseconds by comparing it against your normal spending pattern. A hospital predicts which patients are at risk of readmission from their discharge data. A retailer decides how much stock to order for next month from years of sales history. A city times its traffic lights using sensor data collected from the roads themselves. A phone camera finds and focuses on a face because a model was trained on millions of labeled photos. Every one of these systems starts exactly where this book starts, with a table of numbers and the tools in the next seven chapters.

1.4 Who this book is for

You should already be comfortable with a text editor and running commands in a terminal. You do **not** need to already know Python; the next chapter builds it from the ground up. You do not need prior exposure to data science, statistics, or machine learning, this book assumes none of that and builds every idea from its mathematical basis before using it in code.

1.5 How to read the code

Every code block in this book is followed immediately by the output it actually produced, printed values, tables, or plotted figures, so you can check your understanding against a ground truth as you go, without needing to run anything yourself. That said, this book rewards typing the examples out by hand far more than reading them; muscle memory for syntax and an intuition for error messages are things reading alone cannot build.

1.6 Notation used throughout

- Inline code like `df.groupby("column")` refers to a literal function, method, or variable name.
- Mathematics is set in standard notation, for example, the mean of a sample $x_1, \dots, x_n$ is written $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$, and every formula introduced is explained in prose immediately before or after it, not left to stand alone.
- A gray box like the one below always contains runnable Python code:

```
print("This is a code block.")
```

Output:

```
This is a code block.
```

1.7 What comes next

The next chapter builds Python from variables and control flow through functions, data structures, and the object oriented and file handling features the rest of the book relies on. The chapters after that build the numerical and visualization core: NumPy, Pandas, Matplotlib, and Seaborn, followed by a chapter on data preprocessing, the step that decides whether everything built on top of a dataset is trustworthy. The book closes with five realistic, fully worked analyses.

Chapter 2

Python Essentials

Every tool in the rest of this book, NumPy, Pandas, Matplotlib, Seaborn, and later scikit learn, is a Python library, and every one of them is used the same way: by writing Python code that calls their functions. This chapter builds the language itself, from nothing, so that nothing in a later chapter's code will be a surprise. If you already know Python well, skim this chapter for the vocabulary this book uses and move on to Chapter 2.

2.1 Variables and types

A variable is a name bound to a value. Python figures out the value's type automatically, you never declare `int x` the way you would in Java or C, but every value still has a definite type underneath, and that type controls what you can do with it.

```
name = "Ada"
age = 28
height_m = 1.68
is_student = False
print(name, age, height_m, is_student)
print(type(name), type(age), type(height_m), type(is_student))
```

Output:

```
Ada 28 1.68 False
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
```

Four types appear here: `str` (text), `int` (whole numbers), `float` (decimal numbers), and `bool` (`True/False`). Every value in Python, a number, a string, a list, a function, even a class, is an *object* with a type, and `type(x)` always tells you what that type is. This matters immediately: `5` and `5.0` look almost identical but behave differently in arithmetic that depends on precision, and a function that expects a string will fail confusingly if handed an integer instead.

2.2 Arithmetic and operators

```
a, b = 17, 5
```

```
print(a + b, a - b, a * b, a / b, a // b, a % b, a ** b)
```

Output:

```
22 12 85 3.4 3 2 1419857
```

Two operators deserve attention because they surprise newcomers. `//` is **floor division**: $17 \div 5 = 3.4$, and floor division discards the fractional part, giving 3. `%` is the **modulo** operator, the remainder after that division: $17 = 3 \times 5 + 2$, so $17 \% 5$ is 2. These two together are how you decompose a number into “how many whole groups” and “what’s left over”, the exact operation behind, for example, converting a total number of seconds into minutes and seconds. `**` is exponentiation: $17^5 = 1,419,857$.

2.3 Strings

Text in Python is the `str` type. Strings are **immutable**, every string method returns a *new* string rather than modifying the original in place.

```
first, last = "Ada", "Lovelace"
full = f"{first} {last}"
print(full.upper(), full.lower(), len(full))
print(full.split())
print(", ".join(["a", "b", "c"]))
```

Output:

```
ADA LOVELACE ada lovelace 12
['Ada', 'Lovelace']
a, b, c
```

The `f"{first} {last}"` syntax is an **f string**, the standard way to build a string from variables, and the form used throughout this book’s plot titles and print statements. `.split()` breaks a string into a list of words on whitespace by default; `.join()` does the reverse, gluing a list of strings together with a separator in between. Notice `len(full)` counts *characters*, including the space, “Ada Lovelace” is 12 characters long.

2.4 Control flow

Python has no `switch` statement; a chain of `if / elif / else` does the job, and indentation, not braces, defines which lines belong to which block. Indentation is not a style choice in Python; it is how the interpreter knows where a block starts and ends.

```

for n in range(1, 11):
    if n % 15 == 0:
        label = "FizzBuzz"
    elif n % 3 == 0:
        label = "Fizz"
    elif n % 5 == 0:
        label = "Buzz"
    else:
        label = str(n)
    print(label, end=" ")
print()

```

Output:

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz
```

`range(1, 11)` generates the integers 1 through 10 (the upper bound is exclusive, this is a recurring Python convention you will see again in NumPy and Pandas slicing). The classic “FizzBuzz” exercise above tests divisibility with `%`: a number is divisible by another exactly when the remainder is zero.

A `while` loop repeats as long as a condition holds, rather than a fixed number of times:

```

n = 0
total = 0
while n < 5:
    total += n
    n += 1
print("while-loop total:", total)

```

Output:

```
while-loop total: 10
```

This sums $0 + 1 + 2 + 3 + 4 = 10$. `total += n` is shorthand for `total = total + n`.

2.5 Lists

A list is an ordered, mutable collection, you can add, remove, and reorder its elements after creating it.

```

nums = [4, 2, 9, 1, 5]
nums.append(7)
nums.sort()
print(nums)

```

```
print(nums[0], nums[-1], nums[1:3])
squares = [x**2 for x in nums if x % 2 == 0]
print("even squares:", squares)
```

Output:

```
[1, 2, 4, 5, 7, 9]
1 9 [2, 4]
even squares: [4, 16]
```

Indexing starts at 0, and -1 always means “the last element”, a convention that saves you from ever writing `nums[len(nums) - 1]`. The slice `nums[1:3]` returns elements at positions 1 and 2 (again, the upper bound is exclusive). The last line is a **list comprehension**: `[expression for item in iterable if condition]`, read as “for each `x` in `nums`, if `x` is even, keep `x` squared.” List comprehensions appear constantly in real Python code because they are both more compact and, since the loop runs as a single optimized expression, often faster than the equivalent for loop with `.append()` calls.

2.6 Dictionaries

A dict maps keys to values, the closest Python equivalent of a real world lookup table, and the data structure a JSON API response or a single row of tabular data most naturally becomes.

```
person = {"name": "Ada", "age": 28, "skills": ["python", "math"]}
person["city"] = "London"
for key, value in person.items():
    print(f" {key}: {value}")
print("age" in person, "salary" in person)
```

Output:

```
name: Ada
age: 28
skills: ['python', 'math']
city: London
True False
```

`.items()` gives you (key, value) pairs to iterate over; in tests key membership, not value membership, “age” in `person` is `True` because “age” is a key, while “salary” in `person` is `False` because no such key exists (even though salaries might conceptually belong here). Dictionary values can themselves be any type, including, as with “skills” above, a list.

2.7 Tuples and sets

A **tuple** is like a list but immutable, commonly used for a small fixed collection like a coordinate pair. A **set** is an unordered collection with no duplicates, and it implements the mathematical set operations directly.

```
point = (3, 4)
x, y = point
print("unpacked:", x, y)
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print("union:", s1 | s2)
print("intersection:", s1 & s2)
print("difference:", s1 - s2)
```

Output:

```
unpacked: 3 4
union: {1, 2, 3, 4, 5, 6}
intersection: {3, 4}
difference: {1, 2}
```

`x, y = point` is **tuple unpacking**: Python assigns each element of the tuple to a name in one line. The set operators mirror the mathematical definitions exactly: $s_1 \cup s_2$ (union, everything in either), $s_1 \cap s_2$ (intersection, only what's in both), and $s_1 \setminus s_2$ (difference, what's in s_1 but not s_2). Sets are the fastest way to test membership or remove duplicates from a collection.

2.8 Functions

A function packages a computation under a name so it can be reused and tested independently.

```
def euclidean_distance(p1, p2):
    """Straight-line distance between two 2D points."""
    dx = p1[0] - p2[0]
    dy = p1[1] - p2[1]
    return (dx**2 + dy**2) ** 0.5

print(euclidean_distance((0, 0), (3, 4)))
```

Output:

```
5.0
```

This is the Euclidean distance formula you will meet again, generalized to many dimensions, as the mathematical foundation of the KNN and SVM algorithms in Book 2: for two points $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$,

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

which is exactly the Pythagorean theorem applied to the horizontal and vertical distances between the points. Here (0, 0) to (3, 4) gives $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$.

Functions can also accept a variable number of arguments, using `*args` for extra positional arguments and `**kwargs` for extra keyword arguments:

```
def summarize(*args, **kwargs):
    return {"count": len(args), "extras": kwargs}

print(summarize(1, 2, 3, label="demo"))

square = lambda x: x * x
print([square(v) for v in range(5)])
```

Output:

```
{'count': 3, 'extras': {'label': 'demo'}}
[0, 1, 4, 9, 16]
```

`lambda x: x * x` defines a small, anonymous, one expression function, useful when a function is needed briefly (as an argument to `sorted()` or `.apply()`, both of which you will use heavily with Pandas) and not worth naming separately.

2.9 Exceptions

When something goes wrong at runtime, dividing by zero, opening a file that doesn't exist, converting "abc" to a number, Python raises an **exception**, and unless you catch it, the program stops. A try/except block lets you handle the failure instead of crashing.

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return float("inf")
    finally:
        pass

print(safe_divide(10, 2), safe_divide(10, 0))
```

Output:

```
5.0 inf
```

The `except ZeroDivisionError` clause catches only that specific error, a deliberate choice over a bare `except:`, which would silently swallow *any* error, including ones you didn't anticipate and would rather see. The `finally` block, if present, always runs whether or not an exception occurred; it is the right place for cleanup code (closing a file, releasing a resource) that must happen either way.

2.10 Classes

A class bundles data and the functions that operate on that data into one reusable blueprint. Every `DataFrame`, every array, every plot you build in this book is, under the hood, an instance of a class. Object oriented programming rests on four ideas: encapsulation, inheritance, polymorphism, and abstraction. This section covers each one with a runnable example.

2.10.1 A first class

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
        return self.balance

    def __repr__(self):
        return f"BankAccount(owner={self.owner!r}, balance={self.balance})"

acct = BankAccount("Ada", 100)
acct.deposit(50)
acct.withdraw(30)
print(acct)
```

Output:

```
BankAccount(owner='Ada', balance=120)
```

`__init__` is the constructor. Python runs it automatically when you create a new `BankAccount`. `self` refers to the specific instance being operated on. This lets each account keep its own independent balance. `__repr__` controls how an instance prints. Without it, `print(acct)` shows an

uninformative memory address instead of a readable summary. A method name that starts and ends with double underscores, `__init__` and `__repr__` here, is one of Python's dunder methods. Each one hooks into a specific piece of built in language behavior.

2.10.2 Class attributes versus instance attributes

An attribute set inside `__init__` on `self` belongs to one instance. An attribute set directly on the class body is shared by every instance of that class, until an instance overrides it with its own value.

```
class Account:
    bank_name = "Python National Bank"
    account_count = 0

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        Account.account_count += 1

a1 = Account("Ada", 100)
a2 = Account("Grace", 50)
print(a1.bank_name, a2.bank_name)
print(Account.account_count)
Account.bank_name = "New Bank Name"
print(a1.bank_name, a2.bank_name)
```

Output:

```
Python National Bank Python National Bank
2
New Bank Name New Bank Name
```

`bank_name` and `account_count` live on the class itself, not on any single account. Both instances read the same `bank_name` value, and changing it on the class changes what every instance sees, since neither instance ever set its own copy. `account_count` demonstrates a common pattern, a class attribute that tracks something true across every instance ever created, incremented once per call to `__init__`.

2.10.3 Class methods and static methods

A regular method receives the instance as its first argument. A class method receives the class itself instead, useful for building alternate constructors. A static method receives neither, and behaves like a plain function that happens to live inside the class for organizational reasons.

```
class Temperature:
    def __init__(self, celsius):
```



```

        self.celsius = celsius

    @classmethod
    def from_fahrenheit(cls, f):
        return cls((f - 32) * 5 / 9)

    @staticmethod
    def is_freezing(celsius):
        return celsius <= 0

t1 = Temperature(20)
t2 = Temperature.from_fahrenheit(98.6)
print(round(t1.celsius, 1), round(t2.celsius, 1))
print(Temperature.is_freezing(-5), Temperature.is_freezing(20))

```

Output:

```

20 37.0
True False

```

`from_fahrenheit` is a classmethod, marked with `@classmethod` and taking `cls` instead of `self`. It builds a `Temperature` from a Fahrenheit reading without you doing the conversion by hand first. `is_freezing` is a staticmethod, marked with `@staticmethod`. It needs no instance and no class state, so it takes neither.

2.10.4 Encapsulation

Encapsulation means controlling access to an object's internal state instead of letting any code reach in and change it directly. Python has no true private attributes, but convention and the property decorator get you real, enforced control.

```

class Account2:
    def __init__(self, owner, balance):
        self.owner = owner
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Balance cannot go negative")
        self._balance = value

acct = Account2("Ada", 100)
acct.balance = 150

```

```
print(acct.balance)
try:
    acct.balance = -10
except ValueError as e:
    print("rejected:", e)
```

Output:

```
150
rejected: Balance cannot go negative
```

`_balance`, with a single leading underscore, signals by convention that outside code should not touch it directly, though Python does not stop you. `@property` turns `balance` into something that reads like a plain attribute but runs real code underneath, here rejecting a negative value before it can ever be stored. This is encapsulation in practice, the object protects its own state instead of trusting every caller to protect it correctly.

2.10.5 Inheritance

Inheritance lets one class reuse and specialize another class's behavior instead of rewriting it. The class being built on is the parent, or base class. The class that inherits is the child, or derived class.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows"

animals = [Animal("Generic"), Dog("Rex"), Cat("Whiskers")]
for animal in animals:
    print(animal.speak())
```

Output:

```
Generic makes a sound
Rex barks
Whiskers meows
```

`Dog(Animal)` means `Dog` inherits from `Animal`. `Dog` gets `__init__` for free from `Animal`, since it does not define its own, and it overrides `speak` with its own version. This is the point of inheritance, share what stays the same, override what needs to differ.

2.10.6 Polymorphism

Polymorphism means code written against a general type works correctly for every specific type that fits it, without checking which one it got. The `for animal in animals` loop above already showed this, but it is worth stating on its own since it is the payoff inheritance is built for.

```
def announce(animal):
    print(f"Announcement: {animal.speak()}")

for animal in animals:
    announce(animal)
```

Output:

```
Announcement: Generic makes a sound
Announcement: Rex barks
Announcement: Whiskers meows
```

`announce` never checks whether it received an `Animal`, a `Dog`, or a `Cat`. It just calls `.speak()` and trusts each object to do the right thing for its own type. This is exactly how scikit learn works in Book 2, every estimator implements `.fit()` and `.predict()`, and code written against those two methods works for any estimator, logistic regression, a random forest, or a neural network, without changes.

`super()` lets a child class extend its parent's method instead of fully replacing it.

```
class Puppy(Dog):
    def speak(self):
        base = super().speak()
        return f"{base} (puppy version, quieter)"

print(Puppy("Buddy").speak())
```

Output:

```
Buddy barks (puppy version, quieter)
```

`super().speak()` calls `Dog`'s version of `speak`, then `Puppy` adds its own text on top. This lets you build on inherited behavior instead of duplicating it.

2.10.7 Abstraction

Abstraction means defining what a class must do without saying how, then forcing every subclass to fill in the how. Python's `abc` module, abstract base classes, makes this a rule the language enforces rather than a comment you hope people read.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    def describe(self):
        return f"This shape has area {self.area():.2f}"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

shapes = [Circle(3), Rectangle(4, 5)]
for shape in shapes:
    print(shape.describe())

try:
    Shape()
except TypeError as e:
    print("cannot instantiate abstract class:", e)
```

Output:

```
This shape has area 28.27
This shape has area 20.00
cannot instantiate abstract class: Can't instantiate abstract class Shape with abstract
method area
```

`Shape` inherits from `ABC` and marks `area` with `@abstractmethod`. This does two things. First, Python refuses to let you create a bare `Shape`, since it has no real `area` implementation, only

a promise that one exists. Second, every concrete subclass, `Circle` and `Rectangle` here, must implement `area` itself or Python refuses to let you create it either. `describe` is a normal method that every subclass gets for free, and it relies on `area` existing without knowing which specific formula computes it. This is abstraction, the shared method depends only on the promise, not the implementation.

2.10.8 More dunder methods and operator overloading

Python lets a class define how the built in operators and functions treat its instances, by implementing more dunder methods.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __len__(self):
        return int((self.x ** 2 + self.y ** 2) ** 0.5)

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)
print(v1 == Vector(1, 2))
print(len(v2))
```

Output:

```
Vector(4, 6)
True
5
```

`__add__` makes `v1 + v2` work, calling your code instead of raising an error, since Python has no built in idea of what adding two `Vector` objects means. `__eq__` makes `==` compare the values you choose, `x` and `y` here, instead of comparing object identity. `__len__` makes the built in `len()` function work on your object. This pattern, defining a dunder method to plug a class into a built in operator or function, is exactly how a NumPy array supports `+`, `-`, and `len()`, and how a Pandas `DataFrame` supports square bracket indexing.

2.11 Reading and writing files

```
with open("scratch_demo.txt", "w") as f:
    f.write("line one\nline two\nline three\n")

with open("scratch_demo.txt", "r") as f:
    lines = [line.strip() for line in f]
print(lines)
```

Output:

```
['line one', 'line two', 'line three']
```

`with open(...) as f:` is a **context manager**: it guarantees the file is closed automatically when the block ends, even if an error occurs inside it, always prefer this form over calling `open()/close()` manually. Iterating over an open file yields it line by line; `.strip()` removes the trailing newline character each line carries.

2.12 Modules

Python's standard library, and the entire NumPy/Pandas/Matplotlib ecosystem this book is built on, is organized into **modules**, imported with `import`.

```
import math
print(math.sqrt(2), math.pi, math.floor(3.7), math.ceil(3.2))

import random
random.seed(42)
print([random.randint(1, 6) for _ in range(5)])
```

Output:

```
1.4142135623730951 3.141592653589793 3 4
[6, 1, 1, 6, 3]
```

`math` provides standard mathematical functions and constants; `random` generates pseudo random numbers. `random.seed(42)` fixes the random number generator's starting state, without it, `randint` would return different values every run, which would make this book's output impossible to reproduce reliably. You will see this exact pattern, seeding a random process for reproducibility, throughout the machine learning volume of this series, where it separates a genuinely better model from one that only looks better due to random chance.

2.13 Mutability, more precisely

A name in Python is a label pointing at an object, not a box holding a value. Assignment, `a = b`, does not copy an object. It makes `a` a second label pointing at the same object `b` already points at. Whether that matters depends entirely on whether the object is mutable, meaning it can be changed in place, or immutable, meaning it cannot.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a, b, a is b)

c = [1, 2, 3]
d = list(c)
d.append(4)
print(c, d, c is d)
```

Output:

```
[1, 2, 3, 4] [1, 2, 3, 4] True
[1, 2, 3] [1, 2, 3, 4] False
```

`b = a` makes `b` a second label on the exact same list object, confirmed by `a is b` being `True`. Appending through `b` is visible through `a` because there is only one list in memory. `d = list(c)` instead builds a genuinely new list with the same starting contents, so changing `d` never touches `c`. `is` checks object identity, whether two names point at the same object. `==` checks value equality, whether two objects, possibly different ones, hold equal contents. They answer different questions and using the wrong one is a common source of confusing bugs.

Numbers, strings, and tuples are immutable. Lists, dictionaries, and sets are mutable. This single fact explains a function argument surprise that catches most people learning Python at least once:

```
def add_item(lst):
    lst.append("surprise")

items = [1, 2]
add_item(items)
print(items)
```

Output:

```
[1, 2, 'surprise']
```

Python passes the object reference into the function, not a copy of it, so a function that mutates a list argument mutates the caller's list too, with no return needed to see the effect. If this is not what you want, pass `list(items)` to hand the function its own copy, exactly the pattern from the paragraph above.

2.14 Scope and closures

Every name lookup in Python happens in a specific scope, and Python searches outward through a fixed order: local, enclosing, global, built in, remembered by the acronym LEGB. A name assigned inside a function is local to that function by default, invisible outside it, and it does not affect a same named variable outside unless you explicitly say otherwise.

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter_a = make_counter()
counter_b = make_counter()
print(counter_a(), counter_a(), counter_a())
print(counter_b())
```

Output:

```
1 2 3
1
```

`increment` is defined inside `make_counter` and refers to `count`, a variable from the enclosing function's scope, not its own local scope, and not the global scope. A function that remembers variables from the scope it was defined in, even after that outer function has returned, is called a closure. `nonlocal count` tells Python explicitly that assignments to `count` inside `increment` should modify the enclosing variable rather than create a new local one. Each call to `make_counter` builds a completely separate count, which is why `counter_a` and `counter_b` count independently starting from zero.

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        print(x)
    inner()
outer()
```

Output:

```
enclosing
```

`inner` has no local `x` of its own, so Python looks outward and finds `x` in `outer`'s scope before ever reaching the global `x`. This is the LEGB order in action: local first, then enclosing, then global, then Python's built in names last.

2.15 Iterators and generators

A for loop in Python works on anything iterable, meaning anything that knows how to hand out its values one at a time. A list is iterable. So is a str, a dict, a file opened for reading, and a range. A generator function lets you write your own iterable without building the whole sequence in memory first.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
    print("liftoff")

for value in countdown(3):
    print(value)
```

Output:

```
3
2
1
liftoff
```

yield is what makes countdown a generator function rather than a normal function. Calling countdown(3) does not run any of the code inside it yet, it returns a generator object. Each time the for loop asks for the next value, execution resumes exactly where it left off after the previous yield, runs until the next yield or the function ends, and hands back one value. This means a generator can represent an infinite or extremely large sequence without ever holding more than one value in memory at a time, unlike building a full list upfront.

A generator expression is the same idea written inline, using parentheses instead of square brackets:

```
squares_gen = (x * x for x in range(5))
print(sum(squares_gen))
```

Output:

```
30
```

A class becomes iterable by implementing __iter__. Writing __iter__ as a generator function, using yield inside it, is the least code for the most control:

```
class Fibonacci:
    def __init__(self, limit):
        self.limit = limit
```

```
def __iter__(self):
    a, b = 0, 1
    count = 0
    while count < self.limit:
        yield a
        a, b = b, a + b
        count += 1

print(list(Fibonacci(8)))
```

Output:

```
[0, 1, 1, 2, 3, 5, 8, 13]
```

`list(Fibonacci(8))` works because `list()` accepts anything iterable and pulls every value out of it. The same `Fibonacci(8)` object also works directly in a `for` loop, unpacking, or any other place Python expects something iterable.

2.16 Comprehensions beyond lists

The list comprehension from earlier in this chapter has direct counterparts for dictionaries and sets, using the same `for/if` structure with different bracket types.

```
squares_dict = {x: x * x for x in range(6)}
print(squares_dict)

vowels_found = {ch for ch in "the quick brown fox" if ch in "aeiou"}
print(sorted(vowels_found))

matrix = [[1, 2, 3], [4, 5, 6]]
flat = [val for row in matrix for val in row]
print(flat)
```

Output:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
['e', 'i', 'o', 'u']
[1, 2, 3, 4, 5, 6]
```

A dict comprehension, `{key: value for ...}`, builds a dictionary directly. A set comprehension, `{item for ...}`, builds a set and automatically drops duplicates, which is why `vowels_found` has no repeated letters even though the source sentence repeats several vowels. The last example shows a nested comprehension: two `for` clauses in one expression flatten a list of lists into one flat list, read left to right in the same order you would nest the equivalent `for` loops.

2.17 Decorators

A decorator is a function that takes another function and returns a modified version of it, without changing the original function's source code. The `@decorator_name` syntax placed above a function definition is shorthand for wrapping that function immediately after it is defined.

```
import functools
import time

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed * 1000:.3f} ms")
        return result
    return wrapper

@timed
def slow_sum(n):
    return sum(range(n))

total = slow_sum(1_000_000)
print(total)
```

Output:

```
slow_sum took 23.494 ms
499999500000
```

`@timed` above `def slow_sum` is exactly equivalent to writing `slow_sum = timed(slow_sum)` right after defining it. `timed` returns `wrapper`, a new function that records the time, calls the real `slow_sum` through `func(*args, **kwargs)` so it still accepts whatever arguments the original function accepts, prints the elapsed time, and returns the real result unchanged. `@functools.wraps(func)` preserves `slow_sum`'s original name and docstring on the wrapped version, which matters for debugging and documentation tools. This exact pattern, wrapping a function to add timing, logging, caching, or access control around it without touching its internal logic, is how most decorators you will meet in real libraries work, including scikit learn's and LangChain's own decorators used later in this series.

2.18 String formatting in depth

The `f` string introduced earlier in this chapter supports a format specification after a colon, controlling exactly how a value renders.

```
pi = 3.14159265
print(f"{pi:.2f}")
print(f"{pi:10.4f}")
print(f"{1234567:,}")
print(f"{0.256:.1%}")
name = "Ada"
print(f"{name:>10}|")
print(f"{name:<10}|")
print(f"{name:^10}|")
```

Output:

```
3.14
    3.1416
1,234,567
25.6%
      Ada|
Ada      |
Ada      |
```

.2f means fixed point notation with 2 digits after the decimal point. 10.4f adds a minimum total field width of 10 characters, right padded with spaces, still with 4 decimal digits. , inserts thousands separators automatically. .1% multiplies by 100, appends a percent sign, and shows 1 decimal digit, turning a raw proportion like 0.256 into a readable 25.6% without you doing the multiplication yourself. >10, <10, and ^10 right align, left align, and center align text within a 10 character field, which is exactly how the aligned columns in this book's own tables and printed output are built. ## Recap

Concept	What it gave you
Variables and types	Every value in Python has a type that determines what you can do with it
Arithmetic	// and % decompose a number into whole groups and remainder
Strings	Immutable text, built with f strings, split and joined
Control flow	if/elif/else and indentation defined blocks; for and while loops
Lists	Ordered, mutable sequences; list comprehensions as a compact loop
Dictionaries	Key value lookup, the shape a row of tabular data naturally takes
Tuples and sets	Immutable fixed groupings; true mathematical set operations

Concept	What it gave you
Functions	Reusable, named computations, including <code>*args/**kwargs</code> and <code>lambda</code>
Exceptions	<code>try/except/finally</code> for handling failure without crashing
Classes	Bundling data and behavior, the foundation of every library used later
Mutability	A name is a label on an object; mutable objects can change in place, immutable ones cannot
Scope and closures	LEGB lookup order; a closure remembers its enclosing scope after that scope returns
Iterators and generators	<code>yield</code> produces values one at a time without building the full sequence in memory
Comprehensions	Dict and set comprehensions, and nested comprehensions for flattening
Decorators	A function that wraps another function to add behavior without changing its source
String formatting	Format specifiers control decimals, thousands separators, percentages, and alignment
Files	with <code>open(...)</code> for safe, automatically closed file access
Modules	<code>import</code> brings in the standard library and, later, NumPy/Pandas/Matplotlib

Every one of these ideas reappears, immediately, in Chapter 2: a NumPy array is an object (a class instance) with methods you call with `.`, its `dtype` is a type in exactly the sense covered above, and indexing a NumPy array follows the same zero based, exclusive upper bound convention as indexing a Python list.

Chapter 3

NumPy

Every data structure used later in this book, a Pandas DataFrame, a scikit learn feature matrix, a PyTorch tensor, is, underneath, a NumPy array. This chapter builds the mental model for that array: how it is laid out in memory, why that layout makes it fast, how to reshape and index it, and above all how **broadcasting** works, since broadcasting is the single idea that trips up more beginners than anything else in the numerical Python stack. Read this chapter slowly; everything after it assumes you have it.

3.1 Understanding the ndarray

3.1.1 *What a NumPy array actually is in memory*

A Python list stores pointers. Each element of a list is a separate object living somewhere in memory, and the list itself is an array of addresses pointing to those objects. To read `my_list[3]`, Python follows the pointer at position 3, then reads the object it finds there. Every one of those objects, even a plain integer, carries its own type tag, reference count, and value, typically 28 bytes for a small integer on a 64 bit system.

A NumPy array works completely differently. It stores one single, contiguous block of raw bytes, all of the same type, packed edge to edge with no per element overhead. An array of 5,000,000 32 bit integers is exactly 20,000,000 bytes, not one byte more. To read element 3, NumPy does not follow a pointer to a separate object. It computes an offset directly into the block and reads the raw bytes at that address.

This single design decision, one flat buffer instead of many scattered objects, is the reason NumPy exists. It gives you two things simultaneously that a plain Python list cannot give you together.

The first is memory density. A million floats in a NumPy array cost 8 million bytes. The same million floats as Python `float` objects in a list cost roughly 24 million bytes for the objects themselves, plus 8 million bytes for the list of pointers, over four times as much, before you have done any work with the data at all.

The second is computation speed, and this one is more subtle than “less memory equals more speed.” A modern CPU does not just execute instructions one at a time faster when it has less data to move. It has a memory hierarchy, small, extremely fast caches sitting close to the processor core, backed by slower main memory. When your data sits in one contiguous block, the CPU can pull a whole chunk of it into cache at once and then operate on that chunk without going back to

main memory for every single value. When your data is scattered across a million separate heap allocated Python objects, almost every access is a cache miss, a trip out to slow main memory. This is why a NumPy operation on an array is not just a little faster than the equivalent Python loop. It is routinely fifty to two hundred times faster, and the gap is dominated by memory access patterns, not by raw instruction count.

3.1.2 Strides: how NumPy finds any element without searching

Every NumPy array carries a tuple called its strides, one integer per dimension, telling you how many bytes to skip in the underlying buffer to move one step along that dimension.

Take a 2D array with shape (3, 4) of 8 byte floats, stored in the default row major (C) order, meaning it is laid out row by row in memory. Moving one step along axis 1 (across a row) means moving 8 bytes, the width of one float. Moving one step along axis 0 (down to the next row) means skipping an entire row, 4 elements times 8 bytes, or 32 bytes. So this array's strides are (32, 8).

To find element [i, j], NumPy does not search anything. It computes a single offset directly:

$$\text{offset} = i \times \text{stride}_0 + j \times \text{stride}_1 = i \times 32 + j \times 8$$

and reads 8 bytes starting at that byte position in the buffer. This is why indexing a NumPy array is a constant time operation, $O(1)$, regardless of the array's size: it is arithmetic on the strides, never a search.

Strides also explain why some operations are free and others are not. Transposing a 2D array, swapping rows and columns conceptually, does not touch a single byte of the underlying buffer. NumPy simply swaps the two stride values and reports the shape as reversed. `array.T` is instant no matter how large the array is, because it changes only the recipe for reading the data, not the data itself. Reshaping is the same story when the new shape is compatible with the existing strides: no data moves, only the metadata describing how to read it changes.

3.1.3 Vectorization: why "no explicit loop" is not a style preference

When you write `a + b` for two NumPy arrays, you are not calling a Python function that loops over elements in the interpreter. You are calling into a compiled C function, part of NumPy's core, that runs a tight loop entirely outside the Python interpreter, often using SIMD (single instruction, multiple data) CPU instructions that add four, eight, or more numbers in a single clock cycle.

Contrast this with:

```
result = [x + y for x, y in zip(a, b)]
```

Every single iteration of this list comprehension pays the cost of the Python interpreter's bytecode dispatch loop: unpacking the tuple from `zip`, looking up the `+` operator on the objects' types, checking for operator overloading, allocating a new Python `int` or `float` object to hold the result, and appending it. None of that overhead exists in the vectorized version. The rule that follows directly from this is one of the most important habits this book will build in you:

whenever you find yourself writing a Python level for loop over the elements of a NumPy array, stop and ask whether a vectorized NumPy operation already does what you need. It almost always does, and it will almost always be faster by a wide margin.

3.1.4 Broadcasting: the precise rule, not just the intuition

Broadcasting lets NumPy apply an operation between two arrays of different shapes without you writing any code to expand either one. The intuitive description, “the smaller array gets stretched to match,” is true but imprecise enough to cause real bugs. Here is the exact rule NumPy follows, and it is worth memorizing precisely because getting it wrong silently produces a wrong answer rather than an error.

To combine two arrays, NumPy first aligns their shapes on the right (the trailing dimensions), padding the shorter shape with size 1 dimensions on the left as needed. Then, for every dimension, compatible means one of three things is true: the sizes are equal, one of the sizes is 1, or one of the arrays simply does not have that dimension at all (having been padded). If every dimension is compatible under this rule, the operation proceeds, and any dimension of size 1 is conceptually repeated, at zero memory cost, to match the other array’s size in that dimension. If any dimension fails all three conditions, NumPy raises a `ValueError` rather than guess what you meant.

Consider adding an array of shape (8, 1, 6, 1) to an array of shape (7, 1, 5). Align on the right:

```
(8, 1, 6, 1)
(7, 1, 5)
```

The second array’s shape gets padded on the left with a 1 to match length: (1, 7, 1, 5). Now compare dimension by dimension from the right: 1 vs 5 (one is 1, compatible, result 5), 6 vs 1 (one is 1, compatible, result 6), 1 vs 7 (one is 1, compatible, result 7), 8 vs 1 (one is 1, compatible, result 8). Every dimension passes, so the result has shape (8, 7, 6, 5). Notice that broadcasting can produce a result with more total elements than either input array had. Neither input’s data is duplicated in memory to make this happen. NumPy computes the result directly using the stride trick: a dimension being “stretched” simply has its stride treated as zero during the computation, so the same underlying bytes are read repeatedly without ever being copied.

The single most common broadcasting bug in real code is an accidental shape of $(n,)$, a one dimensional array with n elements, when you meant a column of shape $(n, 1)$ or a row of shape $(1, n)$. $(n,)$ is neither a row nor a column, it is genuinely one dimensional, and broadcasting rules treat it differently from both. If you find a broadcasting result that looks transposed or unexpectedly duplicated, checking every array’s `.shape` explicitly before the operation is the fastest way to find the mistake, faster than reasoning about the rule abstractly.

3.1.5 Views versus copies: the rule that prevents the most common NumPy bug

A NumPy slice, unlike a Python list slice, does not copy data. `arr[1:4]` returns a new array object, but that new object’s data buffer is the *same* buffer as `arr`, just addressed with a different starting

offset and possibly different strides. This new array is called a view. Writing to a view writes through to the original array, because there is only ever one underlying buffer.

```
a = np.arange(10)
b = a[2:5]
b[0] = 999
print(a[2])
```

Output:

```
999
```

This is not a bug. It is the deliberate, foundational design choice that makes slicing an array of any size an $O(1)$ operation instead of an $O(n)$ copy. The tradeoff is that you must know, for any expression you write, whether it returns a view or a copy, because writing to a view you believed was independent silently corrupts data you did not intend to touch.

Basic indexing, meaning slices built from integers, colons, and `Ellipsis`, always returns a view. Advanced indexing, meaning indexing with a list of integers, an array of integers, or a boolean mask, always returns a copy. This asymmetry is precise and worth internalizing exactly:

```
view = arr[1:4]           # slice -> view
copy = arr[[1, 2, 3]]     # list of indices -> copy
copy2 = arr[arr > 5]      # boolean mask -> copy
```

When you need independence from the original array and are not sure which kind of indexing you used, call `.copy()` explicitly. It costs one real memory copy, exactly once, and removes all ambiguity. The cost of an unnecessary `.copy()` is a few extra bytes of memory. The cost of a *missing* one is a bug that silently corrupts a different array somewhere else in your program, often far from the line that actually caused it.

3.2 NumPy NN, NumPy Foundations

MLCourse · Data Science Foundations · 01_numpy

NumPy is the bedrock of the entire Python data stack, this notebook builds the core mental model of the ndarray: creating, inspecting, slicing, reshaping, broadcasting and summarizing arrays, which is everything pandas, scikit learn and deep learning frameworks quietly depend on.

What you'll learn

- Why NumPy exists (speed, memory, ecosystem) and how it beats plain Python lists
- Creating arrays with `np.array`, `zeros`, `ones`, `full`, `arange`, `linspace`, `eye`
- Reading array metadata: `shape`, `ndim`, `size`, `dtype`, `itemsize`, `nbytes`
- Choosing and converting dtypes (and dodging silent overflow bugs)

- Indexing and slicing 1 D and 2 D arrays, including steps and reversals
- Views vs copies, the number one source of “my array changed mysteriously” bugs (intro)
- Reshaping: reshape, ravel vs flatten, .T, and np.newaxis
- **Broadcasting**: the three compatibility rules, four worked shape diagrams, one instructive failure
- Elementwise arithmetic and comparison operators
- Aggregations with axis= (sum, mean, std, min/max, argmin/argmax) + NaN safe variants
- Running statistics with cumsum / cumprod
- Boolean masking: filter, count, and combine conditions safely with &, |, ~

0. Setup

One import gives us the whole toolbox. We also grab `sys` and `time` so we can *measure* (not just claim) NumPy’s memory and speed advantages in a moment.

Pro tip: the alias `np` appears in virtually every data science codebase, textbook and Stack Overflow answer on Earth. Adopt it unconditionally, never write `import numpy`.

```
import sys                # used below to measure Python-object memory overhead
import time               # wall-clock timing for our speed teaser

import numpy as np        # THE numerical library - imported as `np` by universal
    convention

print("NumPy version:", np.__version__)
```

Output:

```
NumPy version: 2.4.6
```

1. Why NumPy?

Three reasons NumPy dominates scientific computing in Python:

1. **Speed**, core routines are compiled C loops (often SIMD vectorized) instead of interpreted Python bytecode; a single NumPy call replaces millions of Python level iterations.
2. **Memory**, five million floats live in ONE contiguous 40 MB block, not as five million scattered Python objects that each cost ~28 extra bytes plus pointer overhead.
3. **Foundation**, pandas Series/DataFrames, scikit learn estimators, Matplotlib images and PyTorch/TensorFlow tensors all speak “NumPy array” as their common language.

Feel it first, understand it later, here is a tiny race: summing five million numbers.

Build two representations of the SAME data

```

n = 5_000_000                # five million integers
py_list = list(range(n))      # plain Python list: n separate boxed int objects
np_array = np.arange(n)       # NumPy ndarray: same values in ONE dense buffer

## --- Time a pure-Python loop -----
start = time.perf_counter()    # high-resolution timer
total_py = 0
for value in py_list:          # 5,000,000 interpreted iterations...
    total_py += value           # ...each with boxing/unboxing + reference-counting work
k
py_seconds = time.perf_counter() - start

## --- Time the NumPy equivalent -----
start = time.perf_counter()
total_np = np_array.sum()      # ONE compiled call over contiguous memory
np_seconds = time.perf_counter() - start

print(f"Python loop : {py_seconds * 1000:9.1f} ms   (sum = {total_py})")
print(f"NumPy .sum(): {np_seconds * 1000:9.3f} ms   (sum = {total_np})")
print(f"Speed-up      : ~{py_seconds / np_seconds:,.0f}x")

```

Common pitfall: timings measured this way wobble between runs. In Jupyter you get a statistically robust measurement for free with the built in `%timeit` magic:

```

%timeit total = 0
for v in py_list:
    total += v           # vs simply: %timeit np_array.sum()

```

`%timeit` repeats the snippet many times and reports mean +/- spread. Use it whenever you benchmark anything in a notebook.

2. ndarray vs Python list

A Python **list** is a flexible container of *references* to arbitrary objects scattered around memory. A NumPy **ndarray** ("N dimensional array") is a *typed, dense, contiguous* block of raw numbers plus metadata (shape, dtype). Same numbers, radically different layout:

	list	ndarray
contents	any objects, mixed types	one fixed dtype
memory	pointer table + objects	single flat buffer
math	manual loops	whole array operations

Homogeneity is the superpower: because every slot holds the same type at the same byte width, NumPy can jump to element *i* with pure pointer arithmetic and hand entire ranges to optimized C.

```
small_list = list(range(1_000_000))          # a million boxed Python int objects
small_arr = np.arange(1_000_000, dtype=np.int64) # the same million ints in one buffer

list_pointer_table = sys.getsizeof(small_list) # bytes of the LIST itself (pointers only!)
arr_bytes = small_arr.nbytes                  # bytes holding ALL array elements

print(f"list container alone : {list_pointer_table / 1e6:.1f} MB (+ ~28 B payload per int!)")
print(f"ndarray total      : {arr_bytes / 1e6:.1f} MB (numbers included)")
```

Output:

```
list container alone : 8.0 MB (+ ~28 B payload per int!)
ndarray total      : 8.0 MB (numbers included)
```

```
mixed_py = [1, 2.5, "three"]                # lists happily mix types...
try_mixed = np.array([1, 2.5, 3])            # ...NumPy must pick ONE common dtype instead
try_text = np.array([1, "two", 3.0])         # int + str + float -> everything becomes a string!
print("int+float ->", try_mixed.dtype, try_mixed)
print("int+str ->", try_text.dtype, try_text)
```

Output:

```
int+float -> float64 [1.  2.5 3. ]
int+str -> <U32 ['1' 'two' '3.0']
```

Common pitfall: mixing types in `np.array` does NOT raise an error, NumPy silently coerces everything to one dtype (here: unicode strings). If your numbers suddenly become `<U21`, look for an accidental "3" hiding in your data.

Pro tip: check `.dtype` immediately after building any array from external data. It is the cheapest sanity check in all of data science.

3. Creating arrays

Beyond converting lists, NumPy ships factory functions for every common need: constant fills, integer ranges, evenly spaced grids and identity matrices.

From Python lists

```

a = np.array([1, 2, 3, 4])          # 1-D from a flat list
M = np.array([[1, 2, 3],           # NESTED lists -> 2-D; each inner list becomes a R
              [4, 5, 6]])
print("a :", a)
print("M :\n", M)

## --- Constant-filled factories -----
z = np.zeros((2, 3))               # shape TUPLE, filled with 0.0 (default float64)
o = np.ones(4)                    # 1-D of four 1.0s
f = np.full((2, 2), 7)            # any constant; stays int because 7 is int
print("zeros:", z.shape, z.dtype)
print("full :\n", f)

## --- Range builders -----
r1 = np.arange(0, 10, 2)           # arange(start, stop EXCLUSIVE, step)
r2 = np.linspace(0, 1, 5)         # linspace(start, stop INCLUSIVE, COUNT of points)
print("arange(0,10,2) :", r1)
print("linspace(0,1,5):", r2)

## --- Identity matrices -----
i3 = np.eye(3)                   # 3x3, ones on the main diagonal
print("eye(3):\n", i3)

```

```

## The classic arange-vs-linspace surprise: floating-point steps + EXCLUSIVE stop
bad = np.arange(0, 1, 0.2)        # step 0.2 -> 1.0 is NEVER reached (and float drift
    t accrues)
good = np.linspace(0, 1, 5)       # count-based -> hits both endpoints exactly
print("arange(0, 1, 0.2) :", bad, "-> length", bad.size)
print("linspace(0, 1, 5) :", good, "-> length", good.size)

```

Output:

```

arange(0, 1, 0.2) : [0.  0.2 0.4 0.6 0.8] -> length 5
linspace(0, 1, 5) : [0.   0.25 0.5  0.75 1.  ] -> length 5

```

Pro tip: rule of thumb, use `arange` with INTEGER steps, use `linspace` when you care about endpoints or are stepping by floats. `linspace`'s cousin `endpoint=False` drops the last point when you want `arange` like behaviour but exact spacing.

Common pitfall: `np.zeros(3)` gives a 1 D vector; `np.zeros((3,))` is identical; `np.zeros((3, 4))` is 2 D. The shape is always a tuple, forgetting the inner parentheses is a rite of passage.

4. Inspecting arrays: the metadata attributes

Every ndarray carries a small header describing its contents. These six attributes answer 90% of “what do I actually have?” questions during debugging:

attribute	meaning
shape	length along each dimension (tuple)
ndim	number of dimensions (axes)
size	total element count = product of shape
dtype	the element type (see next section)
itemsize	bytes per element
nbytes	total bytes = size x itemsize

```
T = np.arange(24).reshape(2, 3, 4)    # a 3-D tensor: 2 blocks, each 3x4

print("shape      :", T.shape)         # (2, 3, 4) -> lengths along each axis
print("ndim       :", T.ndim)          # 3 axes
print("size       :", T.size)           # 2*3*4 = 24 elements
print("dtype      :", T.dtype)          # int32 or int64 depending on platform (see below)
print("itemsize   :", T.itemsize)       # 4 or 8 bytes per element
print("nbytes     :", T.nbytes)         # size * itemsize bytes in RAM

assert T.size == 24 and T.nbytes == T.size * T.itemsize # the relationships hold by co
nstruction
```

Output:

```
shape      : (2, 3, 4)
ndim       : 3
size       : 24
dtype      : int64
itemsize   : 8
nbytes     : 192
```

5. A tour of dtypes

The dtype decides how many bytes each element uses and how arithmetic behaves. The everyday cast:

dtype	bytes	typical use
int32	4	counters, indices where range < 2 billion
int64	8	general purpose integers (most common default)
float32	4	images, GPU/deep learning weights

dtype	bytes	typical use
float64	8	default float; scientific computing workhorse
bool	1	masks, the backbone of filtering (last section!)

Converting is done with `.astype(target)`, and it ALWAYS returns a new array (a copy).

```
i32 = np.array([1, 2, 3], dtype=np.int32)      # explicitly narrow integers
i64 = np.array([1, 2, 3], dtype=np.int64)      # explicitly wide integers
f32 = np.array([1.5, 2.5], dtype=np.float32)   # half-width floats
f64 = np.array([1.5, 2.5], dtype=np.float64)   # default double precision
b = np.array([True, False, True])             # booleans are first-class citizens

print(i32.dtype, i64.dtype, f32.dtype, f64.dtype, b.dtype)

## --- astype conversions -----
ints_from_floats = np.array([1.9, -2.7, 3.0]).astype(np.int32) # truncates toward ZERO
bools_from_ints = np.array([0, 1, 99]).astype(bool)             # nonzero -> True
floats_from_bools = b.astype(np.float64)                       # True/False -> 1.0/0.0
print("float->int :", ints_from_floats)                        # [ 1 -2  3] (not rounded!)
print("int->bool  :", bools_from_ints)                         # [False True  True]
print("bool->float:", floats_from_bools)

## --- precision loss when narrowing floats -----
pi32 = np.float32(np.pi)                                       # float32 keeps only ~7 decimal digits
print("pi as float64:", np.float64(np.pi))
print("pi as float32:", pi32)                                   # ...1415927 <- digits silently dropped
```

Output:

```
int32 int64 float32 float64 bool
float->int : [ 1 -2  3]
int->bool  : [False True  True]
bool->float: [1.  0.  1.]
pi as float64: 3.141592653589793
pi as float32: 3.1415927
```

Common pitfall (the big one): FIXED WIDTH INTEGER OVERFLOW. An `int8` can only store -128..127. Arithmetic that leaves that range wraps around SILENTLY, no exception, just wrong answers. This bites everyone who works with `uint8` image pixels at least once.

```
tiny = np.array([100, 120], dtype=np.int8)      # two innocent-looking small integers
wrapped = tiny + tiny                          # exact sums would be 200 and 240... i
                                                # mpossible!
print("int8 100+120 style wrap ->", wrapped)     # [-56 -16]: values wrapped modulo 256
```



```

pixels = np.full((2, 2), 250, dtype=np.uint8)    # uint8: 0..255, the classic image dtype
brighter = pixels + np.full((2, 2), 10, dtype=np.uint8)    # 250 + 10 = 260 -> wraps to 4
print("uint8 250+10 ->", brighter[0, 0])    # 4! A "brighter" image just went almost black.

```

Output:

```

int8 100+120 style wrap -> [-56 -16]
uint8 250+10 -> 4

```

Pro tip: before doing arithmetic on integer arrays, upcast deliberately: `img.astype(np.int32)` first, compute freely, then clip and downcast back at the very end.

6. Indexing & slicing (1 D and 2 D)

NumPy slicing mirrors Python lists, `start:stop:step`, stop always EXCLUSIVE, extended with a comma syntax for multiple dimensions: `M[row_selector, column_selector]`.

```

x = np.arange(10, 20)    # [10 11 12 13 14 15 16 17 18 19]

print("x[0]    ->", x[0])    # first element
print("x[-1]   ->", x[-1])    # last element (negative = count from the end)
print("x[2:5]  ->", x[2:5])    # slice, stop EXCLUSIVE -> indices 2,3,4
print("x[:3]   ->", x[:3])    # everything before index 3
print("x[7:]   ->", x[7:])    # everything from index 7 onward
print("x[::2]  ->", x[::2])    # every 2nd element starting at 0
print("x[::-1] ->", x[::-1])    # REVERSED copy-view of the whole array

```

Output:

```

x[0]    -> 10
x[-1]   -> 19
x[2:5]  -> [12 13 14]
x[:3]   -> [10 11 12]
x[7:]   -> [17 18 19]
x[::2]  -> [10 12 14 16 18]
x[::-1] -> [19 18 17 16 15 14 13 12 11 10]

```

```

G = np.arange(1, 13).reshape(3, 4)    # a tidy 3-row x 4-column grid to poke at
print("G:\n", G)

print("G[1, 2]    ->", G[1, 2])    # row 1, col 2 (zero-based) -> 7
print("G[1]       ->", G[1])    # whole row 1 (same as G[1, :])
print("G[:, 1]    ->", G[:, 1])    # whole COLUMN 1 -> [ 2  6 10]
print("G[0:2, 1:3] ->")    # submatrix: rows 0-1, cols 1-2

```

```
print(G[0:2, 1:3])
print("G[::-1]      :")          # rows reversed (upside down)
print(G[::-1])
print("G[::-1, :-1]:")          # BOTH axes reversed = 180-degree rotation
print(G[::-1, :-1])
```

Output:

```
G:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
G[1, 2]      -> 7
G[1]         -> [5 6 7 8]
G[:, 1]      -> [ 2  6 10]
G[0:2, 1:3] ->
[[2 3]
 [6 7]]
G[::-1]      :
[[ 9 10 11 12]
 [ 5  6  7  8]
 [ 1  2  3  4]]
G[::-1, :-1]:
[[12 11 10  9]
 [ 8  7  6  5]
 [ 4  3  2  1]]
```

Common pitfall: `G[1][2]` technically works (index row 1, then index into that row) but creates an intermediate view per bracket pair. Always prefer the single bracket form `G[1, 2]`.

Pro tip: read `G[:, 1]` out loud as “all rows, column 1”. Saying the selector in English prevents most axis mix ups.

7. Views vs copies, a first look

Slicing does NOT duplicate data. A slice returns a *view*: a new array object that looks at the SAME underlying buffer. This makes slicing free (no copying gigabytes), but modifying a view modifies the original. This is the number one “spooky action” bug for newcomers.

```
a = np.arange(5)          # [0 1 2 3 4]
v = a[1:4]                # a VIEW onto elements 1..3 - no memory copied
v[0] = 999                # write through the view...
print(a)                  # ...and the ORIGINAL changed too!

safe = a[1:4].copy()       # .copy() is the escape hatch: real, independent d
    ata
safe[0] = -1
```

```
print(a)                                # unchanged this time
print("same memory?", np.shares_memory(v, a))  # NumPy can even tell you directly
```

Output:

```
[ 0 999  2  3  4]
[ 0 999  2  3  4]
same memory? True
```

Common pitfall: boolean/fancy indexing (advanced notebook) behaves the OPPOSITE way, those return copies. Memorize the asymmetry now: **basic slices = views, fancy selections = copies**. Deep dive with proofs comes in 02_numpy_advanced.

Pro tip: when you *want* independence, say so explicitly with `.copy()`. When profiling reveals copying is slow, reach for views deliberately. Never rely on accident.

8. Reshaping, transposing and adding axes

Because data lives in one flat buffer, changing *shape* is usually just rewriting the header, no data movement at all.

```
r = np.arange(12)                # 12 flat values
R = r.reshape(3, 4)              # reinterpret as 3x4 grid (usually a VIEW)
R_inferred = r.reshape(-1, 4)    # -1 = "you figure this dimension out" -> (3, 4)
flat_again = R.ravel()           # back to 1-D: view IF possible (no copy made)
flat_copy = R.flatten()          # ALWAYS returns a fresh copy

print(R)
print("reshape(-1, 4) matches:", np.array_equal(R, R_inferred))

flat_again[0] = -99              # mutate via the ravel view...
print(r[0])                     # ...original follows (it was a view)
flat_copy[0] = 12345             # mutate the flatten COPY...
print(R[0, 0])                  # ...original unaffected (-99 still there)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
reshape(-1, 4) matches: True
-99
-99
```

```
C = np.arange(1, 7).reshape(2, 3)    # 2x3 matrix
print("C.T (transpose, 3x2):\n", C.T)  # rows become columns; .T is a view
```

```
col_vec = C.T.reshape(-1)[: , np.newaxis]      # hmm, simpler demo below:
x = np.arange(4)                               # shape (4,)
row_like = x[np.newaxis, :]                   # shape (1, 4) - a 1x4 ROW vector
col_like = x[:, np.newaxis]                   # shape (4, 1) - a 4x1 COLUMN vector
print("x          :", x.shape)
print("x[None, :] :", row_like.shape)
print("x[:, None] :", col_like.shape)
```

Output:

```
C.T (transpose, 3x2):
[[1 4]
 [2 5]
 [3 6]]
x          : (4,)
x[None, :] : (1, 4)
x[:, None] : (4, 1)
```

Pro tip: `np.newaxis` is literally the same object as `None`; `x[:, None]` is identical to `x[:, np.newaxis]`. Adding an axis like this is THE standard trick to make vectors broadcast (next section) into rows or columns.

Common pitfall: `reshape` cannot change the element COUNT: reshaping 12 values into a (5, 3) raises `ValueError`. The product of the new shape must equal size.

9. Broadcasting

Broadcasting is NumPy's ruleset for applying binary operators (+, *, ...) between arrays of *different* shapes without copying data. The three compatibility rules, applied right to left (from the LAST axis backwards):

1. **Pad:** if one array has fewer dimensions, prepend 1s to its shape until ranks match.
2. **Stretch:** dimensions of size 1 are stretched ("broadcast") along that axis for free.
3. **Fail:** if two dimensions differ and NEITHER is 1 `ValueError`.

Result shape in each dimension = max of the two padded shapes.

```
## Example 1 - scalar broadcast: () stretches to anything
v = np.arange(3)           # shape (3,)
scaled = v * 10             # scalar 10 has shape (); result (3,)
print("scalar:", scaled.shape, scaled)

## Example 2 - row broadcast: (4,) pads to (1, 4), then stretches DOWN each row
grid = np.ones((3, 4))     # shape (3, 4)
row_bias = np.array([1., 2., 3., 4.]) # shape (4,) -> treated as (1, 4)
biased = grid + row_bias    # every row gets +[1 2 3 4]
#
## grid (3, 4) row_bias (4,) -> (1, 4) result (3, 4)
```

```

## [1 2 3 4] repeated down [2 3 4 5]
## [1 2 3 4] repeated down + [2 3 4 5]
## [1 2 3 4] repeated down [2 3 4 5]
print("row-broadcast:\n", biased)

## Example 3 - outer-style: (3, 1) + (3,) -> (3, 3); the spec-classic pairing
col = np.arange(3).reshape(3, 1)      # shape (3, 1): one value per ROW
row = np.array([10, 20, 30])          # shape (3,) -> padded to (1, 3)
outer = col + row                      # dims: max(3,1)=3 and max(1,3)=3 -> (3, 3)
#
## col (3,1) row (3,) stretched right result (3,3)
## , [10 20 30 -> -> [[10 20 30]
## , [11 21 31]
## ] [12 22 32]]
print("outer-sum:\n", outer)

## Example 4 - stacked stretch: (8,,, 1) + (7,, 5) -> (8,,, 5)
big = np.zeros((8, 1, 6, 1))          # rank 4
small = np.zeros((7, 1, 5))           # rank 3
result_shape = (big + small).shape     # pad small -> (1, 7, 1, 5); then stretch every 1
#
## big (8,,, 1) small (7,, 5)
## padded small -> (1,,, 5)
## dim-wise max -> (8,,, 5)
print("(8,1,6,1) + (7,1,5) ->", result_shape)

## Bonus utility: ask NumPy for the broadcast shape WITHOUT doing the math
print("broadcast_shapes:", np.broadcast_shapes((8, 1, 6, 1), (7, 1, 5)))

```

Output:

```

scalar: (3,) [ 0 10 20]
row-broadcast:
[[2. 3. 4. 5.]
 [2. 3. 4. 5.]
 [2. 3. 4. 5.]]
outer-sum:
[[10 20 30]
 [11 21 31]
 [12 22 32]]
(8,1,6,1) + (7,1,5) -> (8, 7, 6, 5)
broadcast_shapes: (8, 7, 6, 5)

```

And here is what failure looks like, keep this error message in your head:

```

F = np.ones((3, 2))                  # shape (3, 2)
g = np.ones(3)                       # shape (3,) -> padded (1, 3)
try:

```

```
F + g                                # last dims: 2 vs 3, neither is 1 -> boom
except ValueError as err:
    print("ValueError:", err)
```

Common pitfall: `(n,)` is a VECTOR, not a row OR a column until broadcasting pads it. If you meant "column", be explicit: `x.reshape(-1, 1)` or `x[:, None]`. Silent orientation bugs produce plausible looking garbage downstream.

Pro tip: broadcasting never materializes the stretched copies, the (8,7,6,5) example above allocated only 1680 result elements, not 1680 copies of the inputs. That's why it's fast.

10. Elementwise arithmetic & comparisons

All standard Python operators act ELEMENTWISE on arrays, every element combined with its positional partner. Note carefully: `*` is elementwise multiply, NOT matrix multiplication (that is the `@` operator, coming in the advanced notebook).

```
u = np.array([1., 2., 3., 4.])
w = np.array([10., 20., 30., 40.])

print("add      :", u + w)          # pairwise sums
print("subtract :", w - u)
print("multiply :", u * w)          # ELEMENTWISE product, not dot product!
print("divide   :", w / u)
print("power    :", u ** 2)         # squares each element
print("mod      :", w % 3)          # remainder per element

## Scalars participate through broadcasting (previous section):
print("u * 2 + 1:", u * 2 + 1)

## Comparisons return BOOLEAN arrays - the raw material for masking:
temps = np.array([18, 21, 25, 29, 31, 27, 22])
print("temps >= 25 ->", temps >= 25)
print("temps == 21 ->", temps == 21)
```

Output:

```
add      : [11. 22. 33. 44.]
subtract : [ 9. 18. 27. 36.]
multiply : [ 10. 40. 90. 160.]
divide   : [10. 10. 10. 10.]
power    : [ 1.  4.  9. 16.]
mod      : [1. 2. 0. 1.]
u * 2 + 1: [3. 5. 7. 9.]
temps >= 25 -> [False False  True  True  True  True False]
temps == 21 -> [False  True False False False False]
```

11. Aggregations: collapsing arrays to summaries

Reductions like `sum`, `mean`, `std`, `min`, `max` shrink arrays to scalars, unless you pass `axis=`, which collapses ONE dimension at a time. Read `axis=` as “collapse THIS axis”:

- `axis=0` collapses **rows** one summary PER COLUMN (result loses the row axis).
- `axis=1` collapses **columns** one summary PER ROW (result loses the column axis).

```
A = np.arange(1, 13, dtype=float).reshape(3, 4)  # a 3x4 float grid
print("A:\n", A)

print("total      :", A.sum())          # collapse EVERYTHING -> scalar
print("per-column mean:", A.mean(axis=0))  # (4,) - collapse rows: avg down each column
n
print("per-row sum   :", A.sum(axis=1))    # (3,) - collapse columns: total of each row
row

## argmin/argmax return INDEX POSITIONS of extremes (handy for "who won?"):
print("argmin (flat) :", A.argmin(), "-> element", A.flat[A.argmin()])
print("per-row argmax :", A.argmax(axis=1))  # winning COLUMN index within each row
print("std           :", A.std().round(3))   # population std (ddof=0) - note for stats
fans

## keepdims=True keeps collapsed axes as size-1 - CRUCIAL for clean broadcasting later:
print("mean axis=0 keepdims shape:", A.mean(axis=0, keepdims=True).shape)  # (1, 4)
```

Output:

```
A:
[[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]]
total      : 78.0
per-column mean: [5.  6.  7.  8.]
per-row sum   : [10. 26. 42.]
argmin (flat) : 0 -> element 1.0
per-row argmax : [3 3 3]
std          : 3.452
mean axis=0 keepdims shape: (1, 4)
```

Common pitfall: axis confusion is THE most common numpy bug. Drill the mantra: **`axis=0` goes down the rows, `axis=1` goes across the columns**, and print `.shape` after every reduction until it's second nature.

NaN aware statistics

One missing reading poisons ordinary reductions (NaN propagates through everything). Every reducer has a NaN skipping twin prefixed with `nan`.

```

readings = np.array([[1.0, np.nan, 3.0],      # a sensor grid with a dead pixel
                    [4.0, 5.0, 6.0]])

print("plain sum  :", readings.sum())          # nan - infection spreads
print("nan-safe sum:", np.nansum(readings))    # 19.0 - NaN treated as absent
print("nan-mean   :", np.nanmean(readings))    # average over the 5 valid cells
print("nan-max    :", np.nanmax(readings))

## ### Cumulative operations

sales = np.array([3, 5, 2, 8])                # units sold per day
print("running totals :", np.cumsum(sales))    # [ 3  8 10 18]
print("running products:", np.cumprod(sales))  # [ 3 15 30 240]

M_cum = np.arange(1, 7).reshape(2, 3)
print("cumsum along rows (axis=1):\n", np.cumsum(M_cum, axis=1)) # running total across
                           s each row

```

Output:

```

plain sum  : nan
nan-safe sum: 19.0
nan-mean   : 3.8
nan-max    : 6.0
running totals : [ 3  8 10 18]
running products: [ 3 15 30 240]
cumsum along rows (axis=1):
[[ 1  3  6]
 [ 4  9 15]]

```

12. Boolean masking basics

Comparison operators give you boolean arrays; those masks then *select*, *count* and *filter* data without a single explicit loop. This pattern is arguably the most used NumPy idiom alive.

```

day_highs = np.array([18, 21, 25, 29, 31, 27, 22, 17, 30, 26]) # ten daily high temperatures

hot_mask = day_highs > 25                                     # elementwise test -> boolean array
print("mask      :", hot_mask)
print("hot days  :", hot_mask.sum(), "of", day_highs.size)    # True counts as 1

hot_values = day_highs[hot_mask]                               # filter: keep only positions where mask is True
print("hot values :", hot_values)                             # NOTE: indexing with a mask returns a COPY

```



```

pleasant = day_highs[(day_highs >= 18) & (day_highs <= 26)]    # combine masks with &
chilly = day_highs[~((day_highs >= 18) & (day_highs <= 26))]  # ~ negates the whole m
ask
print("pleasant :", pleasant)
print("not pleasant:", chilly)

```

Output:

```

mask      : [False False False  True  True  True False False  True  True]
hot days   : 5 of 10
hot values : [29 31 27 30 26]
pleasant   : [18 21 25 22 26]
not pleasant: [29 31 27 17 30]

```

```

## Why the parentheses around each condition are MANDATORY: `` binds TIGHTER than compa
rison,
## so the un-parenthesized version is parsed as arr > (3 & arr) < 8 - chained-comparison
chaos:
arr = np.arange(1, 10)
try:
    arr > 3 & arr < 8                                # missing parentheses -> disaster at runtime
except ValueError as err:
    print("no parens ->", type(err).__name__, "-", str(err)[:52], "...")

try:
    (arr > 3) and (arr < 8)                             # even WITH parens, `and` refuses arrays...
except ValueError as err:
    print("'and' ->", type(err).__name__, "- truth value of an array is ambiguous")

print("correct ->", arr[(arr > 3) & (arr < 8)]) # ...elementwise `` is the answer

```

Output:

```

no parens -> ValueError - The truth value of an array with more than one eleme ...
'and'      -> ValueError - truth value of an array is ambiguous
correct    -> [4 5 6 7]

```

Common pitfall (top 5 all time): forgetting parentheses in compound masks. Write `(a > 3) & (a < 8)`, always, and use `&`, `|`, `~` instead of Python's `and`, `or`, `not`, which demand a single truth value and explode on arrays.

Pro tip: masks are just arrays, so you can build them in named steps for readability:

```
is_hot = temps > 25; is_dry = humidity < 40; crisis = temps[is_hot & is_dry].
```

Summary & key takeaways

- NumPy wins on **speed** (compiled C loops) and **memory** (one dense typed buffer), it is the substrate beneath pandas, scikit learn and every DL framework.
- Arrays are **homogeneous**: mixed inputs get coerced to one dtype silently, always check `.dtype` after ingesting data.
- Know your factories: `array/zeros/ones/full` for constants, `arange` for integer ranges, `linspace` for exact endpoints, `eye` for identity.
- Fixed width integers **overflow silently**; upcast before heavy integer arithmetic.
- Basic slices are **views** (mutations leak through); `.copy()` buys independence.
- `reshape` reinterprets the buffer; `ravel` views if possible while `flatten` always copies; `[:, None]` adds axes to set up broadcasting.
- **Broadcasting rules**: pad shorter shapes on the left, stretch size-1 axes, otherwise fail.
- `axis=0` collapses rows, `axis=1` collapses columns; use `keepdims=True` to preserve shape for follow up broadcasting; reach for `nan*` twins whenever real world data might have holes.
- Boolean masks filter and count without loops; combine them with parenthesized `&`, `|`, `~`.

Next stop: **02_numpy_advanced**, fancy indexing, sorting/ranking, joins, linear algebra, the modern random Generator, and performance engineering.

3.3 NumPy NN, NumPy Advanced

MLCourse · Data Science Foundations · 01_numpy

Level two of the NumPy track: the selection and reshuffling tools professionals use daily, fancy indexing, sorting/ranking, joins and splits, plus linear algebra, the modern random Generator, a deep proof driven look at views vs copies, and the vectorization mindset that makes code fast.

What you'll learn

- Fancy (integer array) indexing in 1 D and 2 D, including duplicate index gotchas
- The boolean toolkit deep cut: `np.where`, `np.clip`, `count_nonzero`, `nonzero`, `any/all`
- Sorting: `np.sort` vs `.sort()`, ranking with `argsort`, top k, and `O(n)` `np.partition`
- `np.unique` with counts/first occurrence indices; set operations (`intersect1d`, `isin`, ...)
- Joining (`concatenate`, `vstack`, `hstack`, `column_stack`, `dstack`) and splitting arrays
- `tile` vs `repeat`, pattern duplication done right
- Universal functions: vectorized custom formulas (compound interest, distance matrices)
- Linear algebra essentials: `@`, `trace`, `det`, `inv`, `solve`, `norm`, `eigvalsh`
- Modern random numbers with `np.random.default_rng(42)` (and why legacy calls are old style)
- Views vs copies PROVEN with memory sharing experiments
- Performance engineering: benchmarks, why vectorization wins, avoiding array growth antipatterns
- Saving/loading: `.npy`, `.npz`, and text round trips

0. Setup

Everything here needs only NumPy plus a few standard library helpers: `os/tempfile` for the file IO section at the end, and `time` for honest wall clock benchmarks.

Pro tip: this notebook assumes you finished `01_numpy_foundations`. If any term there felt shaky (views, broadcasting rules, axis semantics), revisit it first, we build directly on it.

```
import tempfile          # cross-platform scratch directory for our saved files
import os               # path joining + cleanup afterwards
import time             # high-resolution benchmarking timer

import numpy as np      # still just one real dependency

print("NumPy version:", np.__version__)
```

Output:

```
NumPy version: 2.4.6
```

1. Fancy indexing: selecting with integer arrays

Instead of a single index or slice, you can pass an ARRAY (or list) of indices. NumPy picks those positions in the order given, repeats allowed, negatives allowed, and always returns a COPY (contrast with slices as views from the foundations notebook).

```
a = np.array([10, 20, 30, 40, 50])      # the data

print("a[[3, 0]]      ->", a[[3, 0]])    # arbitrary order
print("a[[0, 0, 3]]   ->", a[[0, 0, 3]])  # duplicates are fine - each is an independent
pick
print("a[[-1, 1]]     ->", a[[-1, 1]])    # negative indices count from the end

## Fancy indexing is also assignment target:
b = a.copy()
b[[0, 4]] = -1                          # scatter-write into several slots at once
print("after b[[0,4]]=-1 ->", b)
```

Output:

```
a[[3, 0]]      -> [40 10]
a[[0, 0, 3]]   -> [10 10 40]
a[[-1, 1]]     -> [50 20]
after b[[0,4]]=-1 -> [-1 20 30 40 -1]
```

Common pitfall: accumulate with duplicates does NOT do what it looks like. counts[idx] += 1 reads the ORIGINAL values once per slot group, so repeated indices only increment ONCE. The unbuffered fix is np.add.at.

```
counts = np.zeros(5, dtype=int)
counts[[0, 0, 1]] += 1          # looks like index 0 gets +2... it doesn't!
print("naive += :", counts)     # [1 1 0 0 0] <- lost an increment

np.add.at(counts, [0, 0, 1], 1) # np.add.at is UNBUFFERED: every hit lands
print("np.add.at :", counts)    # [2 1 0 0 0]
```

Output:

```
naive += : [1 1 0 0 0]
np.add.at : [3 2 0 0 0]
```

```
## 2-D fancy indexing has TWO modes:
G = np.arange(16).reshape(4, 4)      # a 4x4 grid to raid
print("G:\n", G)

## Mode A: select whole ROWS in a chosen order:
print("rows [3,1,2]:\n", G[[3, 1, 2]])

## Mode B: pair up row+col indices ELEMENTWISE -> individual cells:
cells = G[[0, 1, 2], [1, 2, 3]]      # picks (0,1), (1,2), (2,3)
print("paired cells:", cells)

## Mixing a slice with fancy indexing selects columns across all rows:
cols = G[:, [2, 0]]                  # every row, but columns swapped
print("all rows, cols [2,0]:\n", cols)
```

Output:

```
G:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
[12 13 14 15]]
rows [3,1,2]:
[[12 13 14 15]
 [ 4  5  6  7]
 [ 8  9 10 11]]
paired cells: [ 1  6 11]
all rows, cols [2,0]:
[[ 2  0]
 [ 6  4]
 [10  8]
[14 12]]
```

2. The boolean toolkit, deep edition

Beyond plain filtering, four functions turn masks into answers: `where` (branch or locate), `clip` (clamp), `count_nonzero` (count), `nonzero` (coordinates of True), plus `any/all` (existential/universal checks).

```
vals = np.array([88, 92, 79, 45, 100, 67, 73, 58, 95, 81])    # exam scores

## --- np.where with THREE arguments = vectorized if/else -----
pass_fail = np.where(vals >= 60, "pass", "fail")            # elementwise branch
print("labels      :", pass_fail)

## --- np.where with ONE argument = where are the Trues? -----
print("failing idx :", np.where(vals < 60)[0])              # -> array([3]) ... coordi
      nates
print("nonzero twin:", np.nonzero(vals < 60)[0])            # identical result

## --- np.clip clamps values into [lo, hi] -----
noisy = np.array([-12., 5., 48., 130., 77.])
clamped = np.clip(noisy, 0, 100)                            # outliers pulled to the e
      dges
print("clipped     :", clamped)

## --- counting Trues two ways -----
mask_even = vals % 2 == 0
print("count_nonzero:", np.count_nonzero(mask_even), "| mask.sum():", mask_even.sum())

## --- any / all collapse a mask to one boolean -----
print("any >= 90?  :", bool((vals >= 90).any()))           # does ANY element qu
      alify?
print("all >= 45?  :", (vals >= 45).all())                  # do ALL elements qua
      lify?
```

Output:

```
labels      : ['pass' 'pass' 'pass' 'fail' 'pass' 'pass' 'pass' 'fail' 'pass' 'pass']
failing idx : [3 7]
nonzero twin: [3 7]
clipped     : [ 0.  5. 48. 100. 77.]
count_nonzero: 4 | mask.sum(): 4
any >= 90?  : True
all >= 45?  : True
```

Pro tip: `(vals >= 90).sum()` counts matches, `.argmax()` on a BOOLEAN mask returns the FIRST True position (or 0 when none match, check `.any()` first!). These two one liners cover most quick data questions without sorting anything.

Common pitfall: `np.where(cond)` returns a TUPLE of index arrays (one per dimension). For 1 D use `np.where(cond)[0]`; forgetting `[0]` is a classic source of shape bugs downstream.

3. Sorting and ranking

Three distinct jobs, three distinct tools:

job	tool
get sorted VALUES	<code>np.sort(x) / x.sort()</code>
get the ORDER that would sort	<code>np.argsort(x)</code>
get top/bottom k FAST (unordered)	<code>np.partition(x, k)</code>

```
raw = np.array([23, 7, 13, 42, 4, 19])

sorted_copy = np.sort(raw)          # FUNCTION form: returns a new sorted array
print("np.sort ->", sorted_copy, "| original untouched:", raw)

inplace = raw.copy()
inplace.sort()                      # METHOD form: sorts IN PLACE, returns None!
print(".sort() ->", inplace)

## argsort gives PERMUTATION indices: order[i] tells which original element lands at position i
names = np.array(["ada", "bob", "cy", "dee", "eli", "fay"])
order = np.argsort(raw)             # ascending by score
print("argsort      :", order)
print("sorted names :", names[order]) # reordering via fancy indexing!

descending = np.argsort(-raw)       # negate values to reverse (works for floats too)
print("desc order   :", descending)
```

Output:

```
np.sort -> [ 4  7 13 19 23 42] | original untouched: [23  7 13 42  4 19]
.sort() -> [ 4  7 13 19 23 42]
argsort      : [4 1 2 5 0 3]
sorted names : ['eli' 'bob' 'cy' 'fay' 'ada' 'dee']
desc order   : [3 0 5 2 1 4]
```

The top k trick

`argsort[-k:]` grabs the k LARGEST positions, reversed for highest first presentation. This is the single most reused ranking idiom in data science ("top-5 products", "10 worst residuals").

```
k = 3
top_idx = np.argsort(raw)[-k:][::-1] # last k of ascending order, then flip
print("top", k, "values:", raw[top_idx], "at indices", top_idx)

## np.partition: 0(n) - puts the k-th smallest in its final slot, smaller left, bigger r
```

```

    ight,
    ## WITHOUT fully sorting either side. Perfect when you need "top-k" but not their intern
    al order.
    part = np.partition(raw, len(raw) - k)      # everything >= 3rd largest pushed to the
    right
    print("partitioned  :", part)
    print("top-k via part:", part[-k:])        # same SET as above, unordered

```

Output:

```

top 3 values: [42 23 19] at indices [3 0 5]
partitioned  : [ 4  7 13 19 23 42]
top-k via part: [19 23 42]

```

Common pitfall: `arr.sort()` returns `None` (like Python's `list.sort()`). Writing `x = arr.sort()` silently destroys your data reference. Use `x = np.sort(arr)` for a copy.

Pro tip: for ties you care about (e.g., stable leaderboard), pass `kind="stable"` to `sort/argsort` so equal keys keep their original relative order.

4. Unique values & set operations

`np.unique` is "distinct values" plus optional bookkeeping; the `*1d` set functions compare populations between arrays. Together they answer frequency table and membership questions fast.

```

draws = np.array([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])      # some categorical-ish draws

uniq_vals, counts = np.unique(draws, return_counts=True) # value histogram
print("unique:", uniq_vals)
print("counts:", counts)
print("most common:", uniq_vals[np.argmax(counts)])      # winner-take-all lookup

uniq_vals, first_seen = np.unique(draws, return_index=True)
print("first occurrence idx:", first_seen)                # where each value FIRST appea
red

## --- set operations -----
a_set = np.array([1, 2, 3, 4])
b_set = np.array([3, 4, 5, 6])

print("intersect1d :", np.intersect1d(a_set, b_set))      # in BOTH
print("union1d      :", np.union1d(a_set, b_set))         # in EITHER
print("setdiff1d    :", np.setdiff1d(a_set, b_set))       # in a_set but NOT b_set
print("isin         :", np.isin(a_set, [2, 4, 6]))        # elementwise membership mask

```

Output:

```

unique: [1 2 3 4 5 6 9]
counts: [2 1 2 1 3 1 1]
most common: 5
first occurrence idx: [1 6 0 2 4 7 5]
intersect1d : [3 4]
union1d      : [1 2 3 4 5 6]
setdiff1d    : [1 2]
isin        : [False True False True]

```

5. Joining arrays

One general function plus ergonomic wrappers. The subtlety worth memorizing: for 1 D vectors, `hstack` concatenates END TO END while `column_stack` builds COLUMNS, different results!

```

m1 = np.arange(6).reshape(2, 3)          # (2, 3)
m2 = (np.arange(6) * 10).reshape(2, 3)    # (2, 3)

print("concat axis=0 (more rows):\n", np.concatenate([m1, m2], axis=0)) # (4, 3)
print("concat axis=1 (more cols):\n", np.concatenate([m1, m2], axis=1)) # (2, 6)

print("vstack == concat axis=0:", np.array_equal(np.vstack([m1, m2]),
                                                    np.concatenate([m1, m2], axis=0)))
print("hstack == concat axis=1:", np.array_equal(np.hstack([m1, m2]),
                                                    np.concatenate([m1, m2], axis=1)))
## (np.row_stack is simply a legacy alias of vstack - same function, older name.)

x_vec = np.array([1, 2, 3])
y_vec = np.array([40, 50, 60])
print("hstack 1-D      :", np.hstack([x_vec, y_vec])) # [ 1  2  3 40 50 60] flat!
print("column_stack   :\n", np.column_stack([x_vec, y_vec])) # (3, 2): vectors become C
    OLMNS

deep = np.dstack([x_vec, y_vec])           # stack along a NEW 3rd axis
print("dstack shape   :", deep.shape, "->", deep.ravel()) # interleaved pairs

```

Output:

```

concat axis=0 (more rows):
[[ 0  1  2]
 [ 3  4  5]
 [ 0 10 20]
 [30 40 50]]
concat axis=1 (more cols):
[[ 0  1  2  0 10 20]
 [ 3  4  5 30 40 50]]
vstack == concat axis=0: True
hstack == concat axis=1: True

```



```

hstack 1-D      : [ 1  2  3 40 50 60]
column_stack   :
[[ 1 40]
 [ 2 50]
 [ 3 60]]
dstack shape    : (1, 3, 2) -> [ 1 40  2 50  3 60]

```

Common pitfall: building a design matrix with `hstack([x1, x2])` on 1 D features yields one long row vector, not an $(n_{\text{samples}}, n_{\text{features}})$ table. Use `column_stack` (or `c_`) there.

Pro tip: joining many pieces repeatedly in a loop copies every time (see performance section). Collect parts in a Python list and `np.concatenate(parts)` ONCE at the end.

6. Splitting arrays

The inverses of stacking: carve one array into equal chunks (`vsplit/hsplit`) or let `array_split` handle sizes that don't divide evenly.

```

Z = np.arange(16).reshape(4, 4)          # nice even grid
top, bottom = np.vsplit(Z, 2)            # split ROWS into 2 blocks of 2
left, right = np.hsplit(Z, 2)            # split COLUMNS into 2 blocks of 2
print("vsplit top:\n", top)
print("hsplit right:\n", right)

uneven = np.arange(10)
pieces = np.array_split(uneven, 3)        # 4 + 3 + 3 instead of raising an error
print("array_split sizes:", [p.size for p in pieces])

try:
    np.vsplit(np.arange(18).reshape(3, 6), 2) # 3 rows cannot split evenly into 2
except ValueError as err:
    print("uneven vsplit -> ValueError:", err)

```

Output:

```

vsplit top:
[[0 1 2 3]
 [4 5 6 7]]
hsplit right:
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
array_split sizes: [4, 3, 3]
uneven vsplit -> ValueError: array split does not result in an equal division

```

7. Tiling vs repeating

Two duplication verbs that beginners constantly swap:

- `np.tile(A, reps)`, repeat the WHOLE array as a pattern (like copy paste).
- `np.repeat(A, n)`, repeat EACH ELEMENT in place (element level echo).

```
pattern = np.array([1, 2, 3])

print("tile   x3:", np.tile(pattern, 3))      # [1 2 3 1 2 3 1 2 3]
print("repeat x3:", np.repeat(pattern, 3))    # [1 1 1 2 2 2 3 3 3]

block = np.tile(pattern, (2, 2))             # 2x2 mosaic of the whole pattern
print("tile (2,2):\n", block)

grid2d = np.array([[10, 20], [30, 40]])
print("repeat rows (axis=0):\n", np.repeat(grid2d, 2, axis=0)) # each ROW doubled
```

Output:

```
tile   x3: [1 2 3 1 2 3 1 2 3]
repeat x3: [1 1 1 2 2 2 3 3 3]
tile (2,2):
[[1 2 3 1 2 3]
 [1 2 3 1 2 3]]
repeat rows (axis=0):
[[10 20]
 [10 20]
 [30 40]
 [30 40]]
```

8. Universal functions (ufuncs) & vectorized formulas

A **ufunc** is a compiled, elementwise function (`np.sin`, `np.exp`, `np.sqrt`, `+`, `*`, ...). Because they broadcast, ANY formula built from ufuncs becomes a loop free, C speed expression. Strategy: write the scalar math, then apply it to whole arrays unchanged.

Worked example: compound interest across a whole rate curve at once

```
principal = 10_000.0                # starting capital P
rates = np.linspace(0.01, 0.10, 10) # annual rates r: 1% .. 10%
years, compounds_per_year = 10, 4   # t and n

future_value = principal * (1 + rates / compounds_per_year) ** (compounds_per_year * years)
for r, fv in zip(rates, future_value):
    print(f"r={r:.2f} -> A={fv:,.0f}")
```

Pro tip: notice the formula is IDENTICAL to the textbook scalar version, no loops, no helper functions, no special cases. That's the vectorization mindset: keep the math, swap the containers.

Worked example: pairwise distance matrix via broadcasting

```
## Goal: D[i, j] = Euclidean distance between point i and point j, for ALL pairs at once
.
points = np.array([[0., 0.],
                  [3., 4.],
                  [6., 8.]])                # three 2-D points

delta = points[:, None, :] - points[None, :, :]  # (N,1,D) - (1,N,D) -> (N,N,D) differences
print("delta shape:", delta.shape)              # every pairwise component difference

D = np.sqrt((delta ** 2).sum(axis=-1))          # Pythagoras along the LAST axis
print(D)
print("symmetric?", np.allclose(D, D.T), "| zero diagonal?", np.allclose(np.diag(D), 0))

## Same trick in 1-D with absolute differences - remember this |xi - xj| pattern:
xs = np.array([0., 2., 5., 9.])
abs_diffs = np.abs(xs[:, None] - xs[None, :])    # (4,1)-(1,4) -> (4,4) |xi-xj| grid
id
print(abs_diffs)
```

np.vectorize: convenient, NOT fast

`np.vectorize` wraps a scalar only Python function so it accepts arrays. Under the hood it still runs a Python loop per element, pure ergonomics, zero speed up. Reach for it when the logic is genuinely non mathematical (string formatting, bespoke branching); reach for REAL ufuncs when it is arithmetic.

```
def letter_grade(score):
    """Scalar-only branching that no arithmetic ufunc can express."""
    if score >= 80:
        return "A"
    elif score >= 65:
        return "B"
    return "C"

scores = np.array([91, 72, 58, 84, 65])
grades = np.vectorize(letter_grade)(scores)    # convenience wrapper -> array output
print(grades, grades.dtype)
```

Output:

```
['A' 'B' 'C' 'A' 'B'] <U1
```

9. Linear algebra essentials

Matrix multiplication uses the `@` operator (PEP 465). Everything else lives in `np.linalg`. Golden rule for systems: prefer `solve(A, b)` over `inv(A) @ b`, faster AND numerically safer.

```
A = np.array([[4., 2.],
              [1., 3.]])
B = np.array([[1., 0.],
              [2., 5.]])

print("A @ B (matrix product):\n", A @ B)           # row-by-column contraction
print("same as np.dot:", np.allclose(A @ B, np.dot(A, B)))
print("A * B is DIFFERENT (elementwise):\n", A * B)  # the eternal mix-up

I3 = np.eye(3)                                       # identity matrix (multiplicative
neutral)
print("trace(A)  :", np.trace(A))                   # sum of diagonal = 7
print("det(A)    :", round(np.linalg.det(A), 6))     # determinant = 10
print("inv(A):\n", np.linalg.inv(A))                 # inverse exists since det != 0
print("A @ inv(A) ~ I:\n", np.round(A @ np.linalg.inv(A), 6))
```

Output:

```
A @ B (matrix product):
[[ 8. 10.]
 [ 7. 15.]]
same as np.dot: True
A * B is DIFFERENT (elementwise):
[[ 4.  0.]
 [ 2. 15.]]
trace(A)   : 7.0
det(A)     : 10.0
inv(A):
[[ 0.3 -0.2]
 [-0.1  0.4]]
A @ inv(A) ~ I:
[[ 1.  0.]
 [-0.  1.]]
```

Worked example: solving a linear system $Ax = b$

Solve exactly this system (two equations, two unknowns):

```
2x + y = 5
x + 3y = 10
```

```
A_sys = np.array([[2., 1.],          # coefficient matrix: one ROW per equation
                  [1., 3.]])
b_sys = np.array([5., 10.])         # right-hand side vector

solution = np.linalg.solve(A_sys, b_sys) # factorizes once, solves directly (LU under
the hood)
print("solution x =", solution)         # expect (1, 3)

residual = A_sys @ solution - b_sys     # verify by substituting back
print("residual =", residual, "-> correct?", np.allclose(residual, 0))
print("inv route agrees:", np.allclose(np.linalg.inv(A_sys) @ b_sys, solution))
```

Output:

```
solution x = [1. 3.]
residual = [0. 0.] -> correct? True
inv route agrees: True
```

Common pitfall: `inv()` looks innocent but squares the condition number and wastes time computing entries you never use. In production code, `np.linalg.solve` (single RHS) or `np.linalg.lstsq` (overdetermined systems) should appear orders of magnitude more often than `inv`.

```
v = np.array([3., 4.])                # classic 3-4-5 triangle as a vect
or
print("|v| =", np.linalg.norm(v))      # Euclidean length = 5

S = np.array([[2., 1.],
              [1., 3.]])                # SYMMETRIC matrix
eigenvalues = np.linalg.eigvalsh(S)     # eigvalsh: symmetric/hermitian fa
st path
print("eigenvalues:", eigenvalues)      # real, returned ascending
print("trace == sum of eigs?", np.isclose(S.trace(), eigenvalues.sum()))
```

Output:

```
|v| = 5.0
eigenvalues: [1.38196601 3.61803399]
trace == sum of eigs? True
```

10. Random numbers, done right: default_rng

Since NumPy 1.17 the recommended API is a **Generator** object created with `np.random.default_rng(seed)`. Benefits: reproducibility via explicit seeds, better algorithms (PCG64), no hidden global state, and safe parallel use. All methods below hang off your `rng`.

```
rng = np.random.default_rng(42)                # ONE generator, seeded once, reused
dice = rng.integers(1, 7, size=(3, 4))         # uniform ints: low INCLUSIVE..high EXCLUSIVE
print("dice rolls:\n", dice)

heights = rng.normal(loc=170, scale=10, size=5) # Gaussian draws (mean, std)
print("heights :", heights.round(1))

uniforms = rng.uniform(0, 1, size=3)           # continuous U[0,1)
print("uniforms :", uniforms.round(3))

lottery = rng.choice(np.arange(1, 50), size=6, replace=False) # sampling WITHOUT replacement
print("lottery :", np.sort(lottery))

weighted = rng.choice(["red", "green"], size=8, p=[0.75, 0.25]) # weighted categories
print("weighted :", weighted)

## --- shuffle vs permutation -----
deck = np.arange(1, 53)                        # a fresh deck
permuted_copy = rng.permutation(deck)          # returns a NEW shuffled array
before = deck[:3].copy()
rng.shuffle(deck)                             # shuffles IN PLACE, returns None!
print("permutation kept original intact:", np.array_equal(deck[:3], before))
print("shuffled in place:", not np.array_equal(deck, np.arange(1, 53)))

## --- the LEGACY global-style API (still everywhere in old tutorials) -----
np.random.seed(42)                            # seeds hidden global state - old style
legacy_uniforms = np.random.rand(3)           # np.random.* namespace = legacy
print("legacy draw:", legacy_uniforms)
```

Output:

```
dice rolls:
```

Output:

```
[[1 5 4 3]
 [3 6 1 5]]
```

```
[2 1 4 6]]
heights : [171.3 166.8 169.8 161.5 178.8]
uniforms : [0.927 0.644 0.823]
lottery  : [ 5 11 20 21 24 28]
weighted : ['green' 'red' 'green' 'green' 'green' 'red' 'red' 'red']
permutation kept original intact: False
shuffled in place: True
legacy draw: [0.37454012 0.95071431 0.73199394]
```

Common pitfall: creating a NEW generator inside a loop, `np.random.default_rng()` with no seed per iteration, gives unreproducible, possibly correlated streams. Instantiate ONCE (seeded), pass the `rng` around like any other parameter.

Pro tip: `Generator.integers` excludes the high value (perfect for dice: 1..7), while `Generator.choice` includes endpoints of the provided pool. Reading the docstring beats guessing.

11. Views vs copies, the deep dive

Foundations showed that slices are views. Now let's PROVE the full rule set experimentally, because silent aliasing causes the nastiest data science bugs:

operation	returns
basic slice <code>a[:, a[1:], a[::2]</code>	VIEW
<code>reshape</code> , <code>ravel</code> (when possible)	VIEW
<code>.T</code> transpose	VIEW
fancy indexing <code>a[[1, 3]]</code>	COPY
boolean masking <code>a[a > 2]</code>	COPY
<code>.copy()</code> , <code>flatten()</code> , <code>astype(...)</code>	COPY

```
a = np.arange(6)                                # source array: [0 1 2 3 4 5]

## --- Experiment: a[:] is a view - writes leak through -----
b = a[:]
print("shares memory:", np.shares_memory(a, b))      # True - same buffer, two headers
b[0] = 999                                           # mutate through the 'copy'...
print("a changed too!:", a)                          # ...original shows 999

## --- Experiment: .copy() severs the link -----
-
c = a.copy()
c[:] = -7                                           # blast the independent buffer
print("a survived .copy():", a)

## --- Experiment: fancy indexing COPIES - mutations stay local -----
f = a[[1, 2]]
```

```
f[0] = -100                                # f is its own buffer...
print("a unaffected by fancy edit:", a)      # ...original untouched (vs Experiment 1!)

## --- Experiment: reshape usually views -----
R = a.reshape(2, 3)
print("reshape shares memory:", np.shares_memory(a, R))
R[0, 0] += 100                             # write through the reshaped view
print("a sees it:", a[0])                   # original updated again
```

Output:

```
shares memory: True
a changed too!: [999  1  2  3  4  5]
a survived .copy(): [999  1  2  3  4  5]
a unaffected by fancy edit: [999  1  2  3  4  5]
reshape shares memory: True
a sees it: 1099
```

Common pitfall: chained slicing keeps you INSIDE view territory, `df.values[:, 0][0] = 1` style double indexing can silently edit shared buffers. When in doubt, ask NumPy directly: `np.shares_memory(x, y)` (or check `x.base`, views point at their parent, copies have none).

Pro tip: views are a FEATURE: filtering huge datasets into overlapping windows costs nothing until you write. Adopt the habit “slice freely; `.copy()` deliberately.”

12. Performance: thinking in whole arrays

Vectorization is not cosmetic, it moves the loop from interpreted Python into optimized C (often SIMD vectorized, processing multiple lanes per CPU instruction). Measure, don't guess:

```
def python_sum(values):
    """Sum with an explicit interpreted loop - what NumPy replaces."""
    total = 0.0
    for v in values:                # one bytecode dispatch PER element
        total += v
    return total

def best_of(fn, arg, repeats=5):
    """Return the FASTEST of several timings (robust against OS hiccups)."""
    best = float("inf")
    for _ in range(repeats):
        start = time.perf_counter()
        fn(arg)
        best = min(best, time.perf_counter() - start)
    return best
```



```

data = np.arange(1_000_000, dtype=float)      # a million doubles

t_loop = best_of(python_sum, data)             # interpreted loop
t_builtin = best_of(sum, data)                 # Python's built-in sum (still a loop!)
t_numpy = best_of(np.sum, data)                # single compiled call

print(f"python loop : {t_loop * 1000:9.2f} ms")
print(f"builtin sum : {t_builtin * 1000:9.2f} ms")
print(f"np.sum      : {t_numpy * 1000:9.3f} ms")
print(f"speed-up    : ~{t_loop / t_numpy:,.0f}x")

## In Jupyter, `%timeit np.sum(data)` gives you these statistics automatically.

```

Output:

```

python loop :    120.83 ms
builtin sum  :     76.43 ms
np.sum       :      0.435 ms
speed-up     : ~278x

```

Antipattern: growing arrays inside loops

Arrays live in fixed buffers, so `np.append` must allocate a FULL new array and copy everything on EVERY call quadratic total work. Preallocate, or build a Python list and convert once.

```

def grow_with_append(n):
    """ANTIPATTERN: each append copies the entire array built so far."""
    out = np.empty(0)
    for i in range(n):
        out = np.append(out, i)          # O(size-so-far) copy EVERY iteration
    return out

def preallocate(n):
    """Right way: one allocation, fill slots by index."""
    out = np.empty(n)
    for i in range(n):
        out[i] = i
    return out

n_growth = 20_000
t_bad = best_of(grow_with_append, n_growth, repeats=3)  # fewer repeats: it's slow on purpose
t_good = best_of(preallocate, n_growth)
print(f"growing appends : {t_bad * 1000:9.1f} ms")

```

```
print(f"preallocated      : {t_good * 1000:9.3f} ms  (~{t_bad / max(t_good, 1e-9):.0f}x f
      aster)")
```

Output:

```
growing appends :    267.2 ms
preallocated    :    4.545 ms  (~59x faster)
```

Pro tip: the performance checklist, in priority order: (1) replace loops with ufunc expressions, (2) preallocate instead of appending, (3) aggregate once instead of joining repeatedly, (4) only THEN consider numba/cython. Measure after each step, intuition lies.

13. Saving & loading arrays

Binary formats preserve dtype and shape exactly: `.npy` for one array, `.npz` for a labeled bundle of several. Text formats trade fidelity for human readability and Excel friendliness.

```
scratch = tempfile.gettempdir()                # OS temp folder - tidy by default

## --- Single array: .npz round trip -----
arr_path = os.path.join(scratch, "mlcourse_matrix.npy")
matrix = np.arange(12).reshape(3, 4)          # int array with known content
np.save(arr_path, matrix)                     # writes exact binary image incl.
dtype                                         #
loaded = np.load(arr_path)
print("npz identical?", np.array_equal(matrix, loaded), loaded.dtype)

## --- Multiple arrays: .npz archive -----
-
npz_path = os.path.join(scratch, "mlcourse_bundle.npz")
np.savez(npz_path, matrix=matrix, vector=np.linspace(0, 1, 5), label="demo") # keyword
= key
bundle = np.load(npz_path)                   # lazy dict-like container
print("npz keys:", bundle.files)
print("vector from npz:", bundle["vector"])
bundle.close()

## --- Human-readable CSV text round trip -----
--
csv_path = os.path.join(scratch, "mlcourse_table.csv")
np.savetxt(csv_path, matrix, delimiter=",", fmt="%d")      # integers as plain text
from_text = np.loadtxt(csv_path, delimiter=",")            # comes back as float64!
print("txt identical after astype?", np.array_equal(matrix, from_text.astype(int)))

for path in (arr_path, npz_path, csv_path):
    os.remove(path)                                     # leave no litter behind
```

Output:

```

numpy identical? True int64
npz keys: ['matrix', 'vector', 'label']
vector from npz: [0.  0.25 0.5  0.75 1.  ]
txt identical after astype? True

```

Common pitfall: `loadtxt/savetxt` have NO memory of dtypes, integer tables come back as `float64`. For anything structural (dtypes, shapes, column names), use `.npy/.npz` (or `pandas`); reserve CSV for interoperability with the outside world.

Pro tip: `np.load` on an `.npz` opens LAZILY, members are read from disk only when indexed, so big bundles stay cheap to inspect. Close the handle (or use a context manager).

Summary & key takeaways

- Fancy indexing selects by ARBITRARY integer arrays (copies, duplicates OK); use `np.add.at` when duplicate indices must accumulate.
- The boolean toolkit: `where` (branch/locate), `clip` (clamp), `count_nonzero` (count), `nonzero` (coordinates), `any/all` (collapse to one verdict).
- Sort values with `sort`, rank with `argsort`, grab top k cheaply with `argsort[-k:][::-1]` or `O(n)` partition.
- `unique(return_counts=True)` is your frequency table; `intersect1d/setdiff1d/isin` handle membership questions.
- Join with `concatenate/vstack/hstack`; build feature matrices with `column_stack`; `tile` repeats patterns, repeat echoes elements; `array_split` tolerates uneven chunks.
- Any formula composed of ufuncs vectorizes for free, distance matrices fall out of the `x[:, None] - x[None, :]` broadcasting trick; `np.vectorize` is sugar, not speed.
- Prefer `@` for matrix products and `np.linalg.solve(A, b)` over `inv(A) @ b`.
- Use `np.random.default_rng(seed)` Generators, seeded once, passed around; `np.random.*` is the legacy global state world.
- Slices/reshapes/transposes are views; fancy and boolean selections are copies, prove it to yourself with `np.shares_memory` before trusting critical pipelines.
- Vectorize, preallocate, join once, measure with `%timeit`; never grow arrays with `append`.
- `.npy/.npz` preserve structure perfectly; text IO is lossy about dtypes.

3.4 NumPy NN, NumPy Exercises

MLCourse · Data Science Foundations · 01_numpy

Fifteen hands on tasks that turn the foundations and advanced notebooks into muscle memory, attempt these before opening `04_numpy_solutions`, which mirrors this numbering one to one.

What you'll practice

Array creation · slicing · broadcasting · axis aggregations · masking & filtering · `np.where` · `argsort` · top k · column wise normalization · distance matrices · one hot encoding · moving averages · image style 2 D ops · linear systems · Monte Carlo simulation

How to work through this notebook

1. **Read the task fully**, the description states exactly what your function/variables must produce, including expected shapes.
2. **Run the starter cell first**. Every task is self contained: inputs are created right there, and randomness always comes from `np.random.default_rng(42)` so results are reproducible.
3. **Try for at least 10 minutes** before peeking at anything. Struggling IS the workout.
4. **Use the hint** when stuck, hints name the relevant tools, not the answer.
5. **Verify**: several tasks include a commented "check" line (`assert`, `.shape` print, or expected output). Your code should pass those checks when complete.
6. Only then compare with `04_numpy_solutions`, and note where your approach differs.

Difficulty legend: tasks 1-5 Easy · tasks 6-11 Medium · tasks 12-15 Hard.

0. Setup

Run me first! Every task below assumes NumPy is already imported.

Pro tip: resist copy pasting from the solutions notebook. Type every call by hand, autocompleting `np.arg...` yourself is how the API moves into your fingers.

```
import numpy as np                # the only import any task will ever need

print("NumPy version:", np.__version__, "- ready. Good luck!")
```

Output:

```
NumPy version: 2.4.6 - ready. Good luck!
```

Easy

Task, Array creation warm up

Build four arrays from scratch, without typing out any element lists:

- `grid`: the integers 0..11 arranged as a 3x4 matrix
- `empty_frame`: a 2x2 array of all zeros
- `spread`: five evenly spaced floats from -2 to 2, both endpoints included
- `identity5`: the 5x5 identity matrix

Hint: `arange` + `reshape`, `zeros`, `linspace` (remember how endpoints behave!), `eye`.
Print each result's `.shape` and `.dtype` as a sanity check.

```
def create_playground():
    """Return (grid, empty_frame, spread, identity5).

    grid      : ndarray shape (3, 4), int, values 0..11 row by row
    empty_frame : ndarray shape (2, 2), all zeros
    spread     : ndarray shape (5,), floats evenly spaced from -2 to 2 inclusive
    identity5  : ndarray shape (5, 5), ones on the diagonal
    """
    # your code here

    return None # replace with: grid, empty_frame, spread, identity5

## Check yourself once implemented:
## g, e, s, i5 = create_playground()
## print(g.shape, e.shape, s.shape, i5.shape) # expect (3, 4) (2, 2) (5,) (5, 5)
```

Task, Slice like you mean it

Using `x = np.arange(30)` and slicing **ONLY** (no loops, no list comprehensions), extract:

1. `last_five`, the five final elements
2. `teens`, elements with values 10..19 inclusive
3. `every_third_from_4`, starting at index 4, every 3rd element after that
4. `mirror`, the whole array reversed

Hint: negative slice bounds, `start:stop` with stop **EXCLUSIVE**, the optional step slot, and `[::-1]`.

```
x = np.arange(30) # shared input for all four selections

def slice_practice(x):
    """Return (last_five, teens, every_third_from_4, mirror).

    last_five      : ndarray shape (5,) -> [25 26 27 28 29]
    teens          : ndarray shape (10,) -> values 10..19 inclusive
```

```
every_third_from_4 : ndarray          -> starts at value 4, step 3
mirror              : ndarray shape (30,) -> reversed copy of the input order
"""

# your code here

return None # replace with: last_five, teens, every_third_from_4, mirror

## Check yourself:
## lf, tn, e3, mi = slice_practice(x)
## print(lf, "|", tn[:3], "...", "|", e3, "|", mi[:5], "...")
```

Task, Broadcasting calisthenics

Given a (4, 3) matrix *M* and a length-3 vector *v*:

1. biased, add *v* to EVERY row of *M* using pure broadcasting (no loops, no tile)
2. centered, subtract each COLUMN's mean from that column ("centering"), again with broadcasting; *M.mean(axis=0, keepdims=True)* may help you keep shapes friendly

Common pitfall: forgetting *keepdims=True* turns the column means into shape (3,), which happens to still broadcast correctly here, but write the (1, 3) version deliberately so the habit transfers to trickier cases.

```
M = np.arange(12).reshape(4, 3)      # values 0..11 as a 4x3 grid
v = np.array([10, 20, 30])           # one bias per column

def broadcast_drill(M, v):
    """Return (biased, centered).

    biased : ndarray shape (4, 3) - M with v added to every row
    centered: ndarray shape (4, 3) - each column has its own mean subtracted
    """

    # your code here

    return None # replace with: biased, centered

## Check yourself:
## biased, centered = broadcast_drill(M, v)
## print(np.allclose(centered.mean(axis=0), 0)) # expect True
```

Task, Axis gymnastics on exam scores

scores holds 5 students (rows) x 4 exams (columns). Compute, each with ONE reduction call:

1. *student_totals*, total score per student (shape (5,))

2. exam_averages, average score per exam (shape (4,))
3. best_student_flat, the FLAT index of the single highest score in the matrix
4. best_exam_per_student, for each student, the INDEX of their strongest exam (shape (5,))

Hint: axis=0 collapses rows, axis=1 collapses columns. `argmax()` without an axis works on the flattened array; convert its result to (row, col) with `np.unravel_index` if you're feeling thorough (optional).

```
scores = np.array([[78, 92, 85, 67],
                  [91, 70, 88, 95],
                  [60, 75, 73, 81],
                  [89, 94, 90, 72],
                  [55, 63, 71, 68]])      # rows = students, cols = exams

def axis_report(scores):
    """Return (student_totals, exam_averages, best_student_flat, best_exam_per_student).

    student_totals      : ndarray shape (5,) - sum across each student's exams
    exam_averages       : ndarray shape (4,) - mean down each exam's column
    best_student_flat    : int               - flat index of the maximum element
    best_exam_per_student : ndarray shape (5,) - argmax along the exam axis
    """
    # your code here

    return None

## Check yourself:
## st, ea, bf, be = axis_report(scores)
## print(st.shape, ea.shape, bf, be) # expect (5,) (4,) 7 [1 3 3 1 2]
```

Task, Filter the thermometer log

day_highs records ten daily maximum temperatures. Using boolean masks:

1. hot_count, how many days exceeded 25 degrees
2. hot_values, the actual temperatures of those days
3. hot_share_pct, hot days as a percentage of all days (one expression)
4. pleasant_count, days with highs between 18 and 26 INCLUSIVE (one combined condition!)

Hint: `mask.sum()` counts Trues; `arr[mask]` filters; combine conditions with `&` and remember the parentheses rule from the foundations notebook.

```
day_highs = np.array([18, 21, 25, 29, 31, 27, 22, 17, 30, 26])

def analyze_temps(day_highs):
    """Return (hot_count, hot_values, hot_share_pct, pleasant_count).
```

```
hot_count      : int   - number of entries strictly greater than 25
hot_values     : ndarray 1-D - the temperatures above 25
hot_share_pct  : float - hot_count / total * 100
pleasant_count : int   - entries with 18 <= temp <= 26
"""
# your code here

return None

## Check yourself:
## hc, hv, hp, pc = analyze_temps(day_highs)
## print(hc, hp, pc) # expect 5 50.0 5
```

Medium

Task, Label, grade, clamp

Three where-family workouts on the arrays below:

1. labels, "tall" where height ≥ 175 else "short" (vectorized, no loops)
2. grades, map exam scores to letters: $\geq 90 \rightarrow "A"$, $\geq 80 \rightarrow "B"$, otherwise "C" (nested np.where calls)
3. clamped, pull every percentage in raw_pct into the range [0, 100]

Hint: three argument np.where(cond, if_true, if_false) chains naturally;
np.clip(a, lo, hi) does the clamping in one call.

```
heights = np.array([165, 172, 181, 158, 190, 175])
exams = np.array([93, 77, 84, 59, 91])
raw_pct = np.array([-5.0, 12.0, 99.0, 130.0, 100.0, 47.0])

def categorize(heights, exams, raw_pct):
    """Return (labels, grades, clamped).

    labels : ndarray of strings - 'tall'/'short' with threshold 175
    grades  : ndarray of strings - 'A'/'B'/'C' per the thresholds above
    clamped : ndarray of floats - raw_pct limited to [0, 100]
    """
    # your code here

    return None
```



```
## Check yourself:
## lb, gr, cl = categorize(heights, exams, raw_pct)
## print(lb, gr, cl)
```

Task, Top k leaderboard

revenue lists quarterly revenue for six branches. Write `top_k(values, k)` returning the indices AND values of the k largest entries, highest first.

Then (in comments) state which single call could give you just the k largest VALUES unordered, faster than a full sort, and why.

Hint: `argsort` orders ascending, slice the tail, reverse it. The fast alternative from the advanced notebook is `np.partition` with a clever k position.

```
revenue = np.array([120.5, 88.0, 310.2, 155.9, 275.0, 99.5]) # branch revenues

def top_k(values, k):
    """Return (top_indices, top_values) for the k largest entries, highest first.

    top_indices : ndarray shape (k,) - positions into `values`
    top_values   : ndarray shape (k,) - the corresponding amounts, descending
    """
    # your code here

    return None

## Check yourself:
## ti, tv = top_k(revenue, 3)
## print(ti, tv) # expect [2 4 3] [310.2 275. 155.9]
```

Task, Z score normalize by column

Standardize a small feature matrix: every column must end up with mean 0 and std 1 (population std, NumPy's default `ddof=0`).

Implement `zscore_columns(X)` and then VERIFY on the provided data that each output column's mean rounds to 0 and its std rounds to 1.

Hint: reductions with `axis=0`; `keepdims=True` makes the subtraction broadcast cleanly:
 $(X - \mu) / \sigma$.

```
rng8 = np.random.default_rng(42) # reproducible features, wildly different scales
X_raw = rng8.normal(loc=[10, -5, 0, 200], scale=[1, 10, 0.5, 50], size=(8, 4))
```

```
def zscore_columns(X):
    """Return Z where every column of X has mean 0 and std 1 (ddof=0).

    Z : ndarray, same shape as X
    """
    # your code here

    return None

## Check yourself:
## Z = zscore_columns(X_raw)
## print(np.abs(Z.mean(axis=0)).round(6)) # expect [0. 0. 0. 0.]
## print(Z.std(axis=0).round(6)) # expect [1. 1. 1. 1.]
```

Task, Moving average without loops

Implement `moving_average(x, k)` returning the k day moving average in VALID mode: output length $\text{len}(x) - k + 1$, where each output is the mean of a window of k consecutive values.

Constraint: no Python loops over the data. Two loop free routes are accepted:

- cumulative sums: pad a zero in front of `cumsum(x)`, then `S[k:] - S[:-k]` are window sums
- `np.convolve(x, np.ones(k) / k, mode="valid")`

Hint: with the `cumsum` route, double check WHY the padded zero makes the differences line up with windows (draw five boxes and slide if unsure).

```
prices = np.array([101., 103., 102., 105., 107., 106., 109., 111., 108., 110.,
                  113., 112.])          # 12 daily prices

def moving_average(x, k=3):
    """Return the k-window moving average of x (valid mode).

    output length = len(x) - k + 1; element i averages x[i:i+k].
    """
    # your code here

    return None

## Check yourself:
## ma = moving_average(prices, 3)
## print(ma.size) # expect 10
## print(np.isclose(ma[0], (101 + 103 + 102) / 3)) # expect True
```

Task, One hot encoding from labels

Turn class labels into a binary indicator matrix WITHOUT loops: `labels[i] == c` becomes a 1 at row `i`, column `c`, zeros elsewhere.

`one_hot(labels, n_classes=None)` should infer `n_classes = labels.max() + 1` when not given.

Hint: allocate zeros of shape `(len(labels), n_classes)`, then ONE fancy indexed assignment fills everything: `out[np.arange(n_rows), labels] = 1`.

```
labels = np.array([3, 0, 1, 3, 2, 1, 3])          # class id per sample

def one_hot(labels, n_classes=None):
    """Return an (n_samples, n_classes) int matrix of 0/1 indicators.

    n_classes defaults to labels.max() + 1.
    """
    # your code here

    return None

## Check yourself:
## H = one_hot(labels)
## print(H.shape) # expect (7, 4)
## print(H.sum(axis=1)) # expect [1 1 1 1 1 1 1] - exactly one 1 per row
```

Task, Frequency table & compound filters

From 100 seeded random draws in `[0, 50)` (already generated below), report:

1. `mode_value`, the most frequent draw value
2. `div_by_5`, how many draws are divisible by 5
3. `in_band`, how many draws fall in `[10, 40)` (both tests combined in one mask)
4. `odd_count`, how many draws are odd

Hint: `np.unique(..., return_counts=True)` + `argmax` finds the mode; `%`, `&`, parentheses. Counting Trues with `.sum()` closes each question.

```
rng11 = np.random.default_rng(42)
draws = rng11.integers(0, 50, size=100)          # 100 values in [0, 49]

def summarize_draws(draws):
    """Return (mode_value, div_by_5, in_band, odd_count).

    mode_value : int - the most common value in draws
    div_by_5   : int - count divisible by 5
```

```
in_band    : int - count with 10 <= d < 40
odd_count  : int - count of odd values
"""

# your code here

return None

## Check yourself:
## mv, d5, ib, oc = summarize_draws(draws)
## print(mv, d5, ib, oc) # expect 4 11 63 49
```

Hard

Task, Pairwise distance matrix

Given P with N points in D dimensions (shape (N, D)), build the full (N, N) Euclidean distance matrix with NO loops:

$D_{\text{matrix}}[i, j] = \sqrt{\text{sum over dims of } (P[i] - P[j])**2}$

Then verify: diagonal is all zeros and the matrix is symmetric.

Hint: the broadcasting sandwich $P[:, \text{None}, :] - P[\text{None}, :, :]$ produces shape (N, N, D) ; reduce over the LAST axis with `.sum(axis=-1)`, then `sqrt`.

```
rng12 = np.random.default_rng(42)
points = rng12.uniform(0, 10, size=(6, 2)).round(2) # six cities on a 10x10 map

def pairwise_distances(P):
    """Return the (N, N) matrix of Euclidean distances between all point pairs."""
    # your code here

    return None

## Check yourself:
## Dm = pairwise_distances(points)
## print(Dm.shape) # expect (6, 6)
## print(np.allclose(np.diag(Dm), 0), np.allclose(Dm, Dm.T)) # expect True True
```

Task, Image surgery (invert, crop, border)

Treat an 8x8 grayscale image (uint8 pixel values) as pure array material:

1. inverted, photographic negative: every pixel becomes $255 - \text{pixel}$
2. crop_center, the middle 4x4 region
3. bordered, the original image centered inside a 12x12 canvas of zeros (a border thickness of 2 pixels on every side)

Common pitfall: $255 - \text{img}$ is safe here because the result stays within uint8's 0..255, but ADDING to uint8 pixels wraps around silently. If your pipeline adds brightness, cast to int32 first (foundations notebook, dtype overflow section).

```
img = (np.arange(64) * 4).astype(np.uint8).reshape(8, 8) # smooth 0..252 gradient

def image_ops(img):
    """Return (inverted, crop_center, bordered).

    inverted      : ndarray (8, 8) - 255 minus every pixel
    crop_center    : ndarray (4, 4) - img[2:6, 2:6]
    bordered       : ndarray (12, 12) - img surrounded by a 2-pixel zero frame
    """
    # your code here

    return None

## Check yourself:
## inv, cr, bd = image_ops(img)
## print(inv[0, 0], cr.shape, bd.shape) # expect 255 (4, 4) (12, 12)
## print(bd[:2, :].max()) # expect 0 - the border really is black
```

Task, Solve the supply puzzle (linear system)

A workshop buys gears and bolts. Three gears plus two bolts cost 2; wait, solve THIS system, derived from two invoices:

```
1g + 2b = 2
3g - b = 13      (yes, negative coefficients happen in real models)
```

1. Assemble coefficient matrix A and right hand side b_vec
2. Confirm the system is solvable by checking $\det(A) \neq 0$
3. Solve with `np.linalg.solve` (NOT `inv`!) and store as solution
4. Verify: substitute back and confirm the residual is (numerically) zero

Hint: one ROW of A per equation. `np.allclose(A @ solution, b_vec)` is the standard substitution check.

```
A_sys = np.array([[1.0, 2.0],
                  [3.0, -1.0]]) # coefficients of g and b per invoice line
b_vec = np.array([2.0, 13.0]) # totals: 2 and 13
```

```
def solve_supply(A_sys, b_vec):
    """Return solution vector [g, b] solving A_sys @ [g, b] == b_vec.

    Raise no errors if det is 0 - but DO print a warning in that case before returning N
    one.
    """
    # your code here

    return None

## Check yourself:
## sol = solve_supply(A_sys, b_vec)
## print(sol) # expect [ 4. -1.]
## print(np.allclose(A_sys @ sol, b_vec)) # expect True
```

Task, Casino audit: two dice Monte Carlo

Simulate rolling two fair six sided dice $n = 100_000$ times with the modern Generator API:

1. Roll all dice in ONE call (shape (n, 2)), no loops
2. Compute sums per roll
3. Estimate $p_seven_empirical = P(\text{total} = 7)$
4. Build `freq_table`, unique sums with their EMPIRICAL probabilities (counts / n)
5. Estimate $p_ten_plus_empirical = P(\text{total} \geq 10)$

Compare against theory in printed comments: $P(7) = 6/36 \approx 0.1667$, $P(\text{sum} \geq 10) = 6/36$ as well.

Hint: `rng.integers(1, 7, size=(n, 2))` rolls both dice at once (high EXCLUSIVE);
`sums.sum(axis=1)` collapses pairs; `unique(return_counts=True)` / `n` builds the table.

```
n_rolls = 100_000
rng15 = np.random.default_rng(42)

def dice_experiment(rng, n):
    """Return (sums, p_seven_empirical, freq_sums, freq_probs, p_ten_plus_empirical).

    sums                : ndarray shape (n,) - total of the two dice per roll
    p_seven_empirical    : float - fraction of rolls summing to 7
    freq_sums            : ndarray - the distinct sum values observed (2..12)
    freq_probs           : ndarray - empirical probability of each value in freq_sums
    p_ten_plus_empirical: float - fraction of rolls with sum >= 10
    """
    rolls = rng.integers(1, 7, size=(n, 2)) # starter: replace NOTHING above this line
    ...
    # your code here                                # ...continue below
```

```

return None

## Check yourself (answers vary only in the 3rd decimal thanks to seed 42):
## sums, p7, fs, fp, p10 = dice_experiment(rng15, n_rolls)
## print(round(p7, 3), round(p10, 3)) # expect ~0.168 ~0.168

```

3.5 NumPy NN, NumPy Solutions

MLCourse · Data Science Foundations · 01_numpy

Fully worked answers to 03_numpy_exercises, mirrored task for task, every cell re creates its own inputs so you can jump straight to any number without scrolling back.

How to use this notebook

- Attempt first! Reading a solution before struggling robs the exercise of its value.
- Each cell below states the task in one line, then solves it with comments explaining WHY.
- "Takeaway" notes highlight the transferable idea when there is one.

Pro tip: diff your attempt against these cells mentally, different but correct is a win; same result different tool usually means a performance or readability lesson is hiding.

```

import numpy as np                # single dependency for all fifteen solutions

print("NumPy version:", np.__version__)

```

Output:

```
NumPy version: 2.4.6
```

Solution, Array creation warm up

Build a 3x4 grid of 0..11, a 2x2 zero frame, five linspace points from -2 to 2, and eye(5).

```

grid = np.arange(12).reshape(3, 4)    # arange makes the values, reshape bends them i
    nto rows
empty_frame = np.zeros((2, 2))         # shape tuple -> all 0.0 floats by default
spread = np.linspace(-2, 2, 5)         # count-based: BOTH endpoints guaranteed (-2 an
    d 2)
identity5 = np.eye(5)                  # ones on the diagonal, zeros elsewhere

print(grid)
print(empty_frame.shape, spread, identity5.shape)

```

```
## Takeaway: reshape never copies data here - arange's buffer is simply reinterpreted as 3x4.
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 2) [-2. -1.  0.  1.  2.] (5, 5)
```

Solution, Slice like you mean it

From `x = np.arange(30)`: last five elements, values 10..19, every third from index 4, reversed.

```
x = np.arange(30)                # shared input

last_five = x[-5:]               # negative start counts from the end
teens = x[10:20]                 # stop EXCLUSIVE -> include value 19 need stop
20
every_third_from_4 = x[4::3]     # empty start slot means "from index 4 to the e
nd"
mirror = x[::-1]                 # negative step walks backwards over the whole
array

print(last_five)
print(teens)
print(every_third_from_4)
print(mirror[:6], "...")
```

Output:

```
[25 26 27 28 29]
[10 11 12 13 14 15 16 17 18 19]
[ 4  7 10 13 16 19 22 25 28]
[29 28 27 26 25 24] ...
```

Solution, Broadcasting calisthenics

Add a length-3 vector to every row of a 4x3 matrix; center every column by its mean.

```
M = np.arange(12).reshape(4, 3)    # inputs recreated
v = np.array([10, 20, 30])

biased = M + v                      # v pads to (1, 3), then stretches down all 4 r
ows
```



```

centered = M - M.mean(axis=0, keepdims=True) # keepdims -> (1, 3) means broadcast cleanly
print(biased)
print("column means after centering:", centered.mean(axis=0).round(9))

## Takeaway: axis=0 computes PER-COLUMN stats because it collapses rows; keepdims preserves the
## (1, 3) orientation so the subtraction broadcasts instead of raising or misaligning.

```

Output:

```

[[10 21 32]
 [13 24 35]
 [16 27 38]
 [19 30 41]]
column means after centering: [0. 0. 0.]

```

Solution, Axis gymnastics on exam scores

Per student totals, per exam averages, the flat max position, and each student's best exam index.

```

scores = np.array([[78, 92, 85, 67],
                   [91, 70, 88, 95],
                   [60, 75, 73, 81],
                   [89, 94, 90, 72],
                   [55, 63, 71, 68]])

student_totals = scores.sum(axis=1) # collapse COLUMNS -> one total per row (per student)
exam_averages = scores.mean(axis=0) # collapse ROWS -> one average per column (per exam)
best_student_flat = scores.argmax() # no axis -> argmax over the FLATTENED matrix
best_exam_per_student = scores.argmax(axis=1) # within each row: which exam won

row, col = np.unravel_index(best_student_flat, scores.shape) # bonus: decode the flat index
print(student_totals)
print(exam_averages.round(2))
print("flat:", best_student_flat, "-> (row, col):", (row, col))
print(best_exam_per_student)

```

Output:

```

[322 344 289 345 257]
[74.6 78.8 81.4 76.6]
flat: 7 -> (row, col): (np.int64(1), np.int64(3))
[1 3 3 1 2]

```

Solution, Filter the thermometer log

Count days above 25, list them, compute their share, and count the pleasant 18..26 band.

```
day_highs = np.array([18, 21, 25, 29, 31, 27, 22, 17, 30, 26])

hot_mask = day_highs > 25                                # boolean array, True where strictly
hot                                              hot
hot_count = int(hot_mask.sum())                      # Trues sum as 1 -> instant counting
hot_values = day_highs[hot_mask]                    # mask-indexing filters (and returns
a COPY)
hot_share_pct = hot_count / day_highs.size * 100      # share against ALL entries
pleasant_count = int(((day_highs >= 18) & (day_highs <= 26)).sum()) # parenthesized &
combine

print(hot_count, hot_values, hot_share_pct, pleasant_count)

## Takeaway: `.sum()` on a mask IS the count - no loops, no len(list(filter(...))).
```

Output:

```
5 [29 31 27 30 26] 50.0 5
```

Solution, Label, grade, clamp

Tall/short labels, letter grades via nested where, percentages clamped into [0, 100].

```
heights = np.array([165, 172, 181, 158, 190, 175])
exams = np.array([93, 77, 84, 59, 91])
raw_pct = np.array([-5.0, 12.0, 99.0, 130.0, 100.0, 47.0])

labels = np.where(heights >= 175, "tall", "short")      # elementwise if/else
grades = np.where(exams >= 90, "A",                    # outermost test first...
                  np.where(exams >= 80, "B", "C"))      # ...fallback branch nests inside
clamped = np.clip(raw_pct, 0, 100)                    # both tails handled at once

print(labels)
print(grades)
print(clamped)

## Takeaway: np.where chains read inside-out like else-if ladders; np.clip beats writing
two
## separate masks-and-replaces.
```

Output:

```
['short' 'short' 'tall' 'short' 'tall' 'tall']
['A' 'C' 'B' 'C' 'A']
[ 0.  12.  99. 100. 100.  47.]
```

Solution, Top k leaderboard

Indices and values of the k largest revenues, highest first, plus the partition shortcut noted.

```
revenue = np.array([120.5, 88.0, 310.2, 155.9, 275.0, 99.5])

def top_k(values, k):
    """Return (top_indices, top_values) for the k largest entries, highest first."""
    order = np.argsort(values)           # ascending permutation of positions
    top_indices = order[-k:][::-1]       # take the tail (largest), reverse -> descending
    top_values = values[top_indices]     # fancy-index to fetch matching amounts
    return top_indices, top_values

ti, tv = top_k(revenue, 3)
print(ti, tv)

fast_values_only = np.partition(revenue, len(revenue) - 3)[-3:] # 0(n): 3rd-largest placed,
print("partition route:", fast_values_only)                     # everything bigger sits right

## Takeaway: argsort when you need WHICH, partition when you only need WHAT - full sort is wasted
## work for top-k questions.
```

Output:

```
[2 4 3] [310.2 275. 155.9]
partition route: [155.9 275. 310.2]
```

Solution, Z score normalize by column

Standardize every column of a feature matrix to mean 0 / std 1, then verify.

```
rng8 = np.random.default_rng(42) # identical data to the exercise
X_raw = rng8.normal(loc=[10, -5, 0, 200], scale=[1, 10, 0.5, 50], size=(8, 4))
```

```
def zscore_columns(X):
    """Return Z where every column of X has mean 0 and std 1 (ddof=0)."""
    mu = X.mean(axis=0, keepdims=True)      # (1, 4): one mean per column
    sigma = X.std(axis=0, keepdims=True)     # (1, 4): population std, NumPy default ddo
    f=0
    return (X - mu) / sigma                  # broadcasting shifts+scales all rows unifo
    rmly

Z = zscore_columns(X_raw)
print("column means ~0 :", np.abs(Z.mean(axis=0)).round(6))
print("column stds ==1 :", Z.std(axis=0).round(6))

## Takeaway: this exact pattern (mean/std along axis, keepdims=True) is sklearn's
## StandardScaler in three lines - and the seed makes results reproducible across runs.
```

Output:

```
column means ~0 : [0. 0. 0. 0.]
column stds ==1 : [1. 1. 1. 1.]
```

Solution, Moving average without loops

Valid mode moving average via the padded cumsum trick (convolution variant shown commented).

```
prices = np.array([101., 103., 102., 105., 107., 106., 109., 111., 108., 110.,
                  113., 112.])

def moving_average(x, k=3):
    """Return the k-window moving average of x (valid mode)."""
    S = np.concatenate(([0.], np.cumsum(x))) # prepend 0 so differences align to wind
    ows
    window_sums = S[k:] - S[:-k]              # window i sum = cumulative[i+k] - cumul
    ative[i]
    return window_sums / k

ma = moving_average(prices, 3)
print(ma.round(2))
print("length check:", ma.size == prices.size - 3 + 1)
print("first window check:", np.isclose(ma[0], prices[:3].mean()))

## Alternative one-liner: np.convolve(prices, np.ones(3) /, mode="valid") - same numbe
    rs.
## Takeaway: cumsum turns 'sum of any range' into ONE subtraction - O(n) total, loop-free
```

e.

Output:

```
[102.   103.33 104.67 106.   107.33 108.67 109.33 109.67 110.33 111.67]
length check: True
first window check: True
```

Solution, One hot encoding from labels

Loop free indicator matrix via one fancy indexed scatter assignment.

```
labels = np.array([3, 0, 1, 3, 2, 1, 3])      # class id per sample

def one_hot(labels, n_classes=None):
    """Return an (n_samples, n_classes) int matrix of 0/1 indicators."""
    n_samples = labels.size
    if n_classes is None:
        n_classes = int(labels.max()) + 1      # infer from the data
    out = np.zeros((n_samples, n_classes), dtype=int)  # all-off canvas
    out[np.arange(n_samples), labels] = 1      # paired indices light up exactly one cell/row
    return out

H = one_hot(labels)
print(H)
print("one 1 per row?", np.array_equal(H.sum(axis=1), np.ones(labels.size, dtype=int)))

## Takeaway: out[rows, cols] pairs coordinates ELEMENTWISE - the standard trick for scatter
## updates (this is also exactly how embedding lookups work).
```

Output:

```
[[0 0 0 1]
 [1 0 0 0]
 [0 1 0 0]
 [0 0 0 1]
 [0 0 1 0]
 [0 1 0 0]
 [0 0 0 1]]
one 1 per row? True
```

Solution, Frequency table & compound filters

Mode, divisibility by-5 count, [10, 40) band count, and odd count from 100 seeded draws.

```
rng11 = np.random.default_rng(42)          # recreate the exact exercise data
draws = rng11.integers(0, 50, size=100)

uniq_vals, counts = np.unique(draws, return_counts=True) # value -> frequency table
mode_value = uniq_vals[counts.argmax()]           # argmax finds the busiest bin
div_by_5 = int((draws % 5 == 0).sum())           # remainder test -> mask -> count
in_band = int(((draws >= 10) & (draws < 40)).sum()) # half-open interval needs BO
    TH tests
odd_count = int((draws % 2 == 1).sum())           # % works elementwise on integer arrays

print(mode_value, div_by_5, in_band, odd_count) # expect: 4 11 63 49
```

Output:

```
4 11 63 49
```

Solution, Pairwise distance matrix

All N by N Euclidean distances in three vectorized lines using the broadcasting sandwich.

```
rng12 = np.random.default_rng(42)          # same six map points as the exercise
points = rng12.uniform(0, 10, size=(6, 2)).round(2)

def pairwise_distances(P):
    """Return the (N, N) matrix of Euclidean distances between all point pairs."""
    delta = P[:, None, :] - P[None, :, :]    # (N,1,D)-(1,N,D) -> (N,N,D) component
    diffs                                     # differences
    return np.sqrt((delta ** 2).sum(axis=-1)) # Pythagoras collapsed over the D-axis

Dm = pairwise_distances(points)
print(Dm.round(2))
print("shape/diag/symmetry:", Dm.shape,
      np.allclose(np.diag(Dm), 0), np.allclose(Dm, Dm.T))

## Takeaway:[:, None] vs [None, :] is THE pairwise pattern - it generalizes to cosine s
    imilarity,
## kernels and attention scores unchanged.
```

Output:

```
[[0.    2.72 8.66 3.47 6.46 6.33]
 [2.72 0.    8.14 1.32 7.72 5.39]
```

```
[8.66 8.14 0.   6.94 5.27 2.81]
[3.47 1.32 6.94 0.   7.17 4.15]
[6.46 7.72 5.27 7.17 0.   5.35]
[6.33 5.39 2.81 4.15 5.35 0.  ]]
shape/diag/symmetry: (6, 6) True True
```

Solution, Image surgery (invert, crop, border)

Photographic negative, center crop, and a zero padded border canvas, no loops anywhere.

```
img = (np.arange(64) * 4).astype(np.uint8).reshape(8, 8)  # smooth gradient "image"

inverted = 255 - img                                     # scalar broadcast; stays inside uint8
    range
crop_center = img[2:6, 2:6]                             # plain slicing: view, no copy needed
canvas = np.zeros((12, 12), dtype=img.dtype)             # black canvas sized img + 2px border/f
    range
canvas[2:-2, 2:-2] = img                                 # paste original into the interior
bordered = canvas

print(inverted[0, :4])                                   # 255, 251, 247, 243 - darkest became b
    rightest
print(crop_center.shape, bordered.shape)
print("corner border pixels are zero:", bordered[0, 0] == 0 and bordered[-1, -1] == 0)

## Takeaway: padding-by-allocation-then-slice-assign is the manual equivalent of np.pad
- worth
## doing once by hand so np.pad feels obvious afterwards.
```

Output:

```
[255 251 247 243]
(4, 4) (12, 12)
corner border pixels are zero: True
```

Solution, Solve the supply puzzle (linear system)

Assemble $Ax = b$ for $g + 2b = 2$ and $3g - b = 13$, confirm solvability, solve, verify.

```
A_sys = np.array([[1.0, 2.0],
                  [3.0, -1.0]])                          # one ROW per invoice equation
b_vec = np.array([2.0, 13.0])

det = np.linalg.det(A_sys)                               # nonzero determinant <=> unique solut
    ion
if abs(det) < 1e-12:
    print("WARNING: singular system (det = 0) - no unique solution")
```

```

else:
    solution = np.linalg.solve(A_sys, b_vec)      # LU-backed direct solver (prefer over
    inv)
    print("det =", det, "| solution [g, b] =", solution)
    print("residual:", A_sys @ solution - b_vec, "| allclose:",
          np.allclose(A_sys @ solution, b_vec))    # substitute-back proof

## Takeaway: the solver lands on [g, b] = [4, -1] - when coefficients go negative, trust
    the
## algebra (and the residual check) over physical intuition. The det guard costs microse
    conds and
## catches degenerate systems that would otherwise surface as inf/nan far downstream.

```

Output:

```

det = -7.000000000000001 | solution [g, b] = [ 4. -1.]
residual: [4.4408921e-16 0.0000000e+00] | allclose: True

```

Solution, Casino audit: two dice Monte Carlo

Roll two fair dice 100k times in one vectorized shot and audit the sums empirically.

```

n_rolls = 100_000
rng15 = np.random.default_rng(42)          # same seed as the exercise -> same ro
    lls

def dice_experiment(rng, n):
    """Return (sums, p_seven_empirical, freq_sums, freq_probs, p_ten_plus_empirical)."""
    rolls = rng.integers(1, 7, size=(n, 2))    # high EXCLUSIVE: faces 1..6 for BOTH
    dice
    sums = rolls.sum(axis=1)                    # collapse the pair axis -> n totals
    p_seven_empirical = float((sums == 7).mean()) # mean of a bool array = fraction of T
    rues
    freq_sums, freq_counts = np.unique(sums, return_counts=True)
    freq_probs = freq_counts / n                # normalize counts into probabilities
    p_ten_plus_empirical = float((sums >= 10).mean())
    return sums, p_seven_empirical, freq_sums, freq_probs, p_ten_plus_empirical

sums, p7, fs, fp, p10 = dice_experiment(rng15, n_rolls)
print("P(sum==7) empirical:", round(p7, 4), "| theory:", round(6 / 36, 4))
print("P(sum>=10) empirical:", round(p10, 4), "| theory:", round(6 / 36, 4))
for s, pr in zip(fs, fp):
    bar = "#" * int(pr * 300)                  # tiny ASCII histogram of the distribu
    tion
    print(f"{s:2d} {pr:.3f} {bar}")

```



```
## Takeaway: 100k samples land within ~0.002 of theory - Monte Carlo turns probability q
    uestions
## into counting exercises, and numpy does the counting in compiled C.
```

Output:

```
P(sum==7) empirical: 0.1683 | theory: 0.1667
P(sum>=10) empirical: 0.1676 | theory: 0.1667
 2 0.028 #####
 3 0.055 #####
 4 0.083 #####
 5 0.110 #####
 6 0.139 #####
 7 0.168 #####
 8 0.138 #####
 9 0.111 #####
10 0.085 #####
11 0.055 #####
12 0.028 #####
```

Summary & key takeaways

- Creation (arange+reshape, linspace, zeros, eye) covers nearly every synthetic input need.
- Slicing vocabulary (start:stop:step, negatives, [::-1]) replaces entire utility functions.
- Broadcasting + keepdims=True is the clean way to shift/scale rows or columns (Task 3, 8).
- Masks count (sum), filter (arr[mask]) and combine (& | ~ with parentheses) in one breath.
- argsort[-k:][::-1] ranks; partition wins when order within top k doesn't matter.
- Scatter assignments (out[np.arange(n), labels] = 1) build one hot/embeddings without loops.
- The [:, None] - [None, :] sandwich delivers distance matrices in constant code.
- np.linalg.solve + a determinant guard is the professional habit for linear systems.
- Seeded Generators (default_rng(42)) make simulations reproducible AND auditable.

Chapter 4

Pandas

If NumPy is the engine, Pandas is the vehicle you actually drive: it wraps arrays in labeled, heterogeneous, tabular structures, the `Series` and the `DataFrame`, that map directly onto how real data arrives, as spreadsheets, CSVs, and database query results. This chapter covers loading, indexing, filtering, grouping, and reshaping data with Pandas, building toward the `groupby-apply` pattern that underlies almost every real analysis in this book's capstone projects.

4.1 Pandas, Pandas Foundations

MLCourse · Data Science Foundations · 02_pandas

This is the first stop on our pandas journey, the notebook every other pandas notebook builds on. If NumPy gave us fast *arrays*, pandas gives us fast *labeled tables*: think “Excel or a SQL table, but programmable, reproducible, and vastly more powerful.”

What you'll learn

- Why pandas exists and what problems it solves over raw NumPy
- Creating `Series` and `DataFrames` by hand
- Reading data with `read_csv` (and seaborn's built in datasets)
- The inspection toolkit: `head`, `info`, `describe`, `value_counts`, ...
- Selecting data three ways: `[]`, `.loc` (labels), `.iloc` (positions)
- Boolean filtering: `&` / `|` / `~`, `isin`, `between`, `query`
- Sorting (`sort_values`, `sort_index`) and ranking (`rank`)
- Adding derived columns, dropping/renaming, index surgery

Setup cell, everything later assumes these imports exist.

```
## `%matplotlib inline` is Jupyter magic; we activate it safely so this file
## ALSO runs as a plain Python script (where get_ipython() does not exist).
try:
    get_ipython().run_line_magic("matplotlib", "inline")
except NameError:
    pass # not inside Jupyter - nothing to do
```

```
import numpy as np                # numeric arrays, random numbers
import pandas as pd               # the star of this course
import seaborn as sns             # free practice datasets
import matplotlib.pyplot as plt   # plotting (used sparingly here)

np.random.seed(42)                # reproducibility: same "random" every run
pd.set_option("display.max_columns", 120) # don't hide columns when printing

def _load(name, demo_builder):
    """Try to download a seaborn dataset; if offline, fall back to a tiny
    hand-built stand-in so every cell below still runs end-to-end."""
    try:
        return sns.load_dataset(name)
    except Exception as e:
        print(f"[setup] '{name}' unavailable ({type(e).__name__}) -> using demo rows.")
        return demo_builder()

def _demo_titanic():
    """A miniature titanic-shaped table used ONLY when we cannot download."""
    return pd.DataFrame(
        {
            "survived": [0, 1, 1, 1, 0, 0, 1, 0, 1, 0],
            "pclass":   [3, 1, 1, 1, 3, 3, 1, 3, 2, 3],          # ticket class 1-3
            "sex":       ["male", "female", "female", "female",
                          "male", "male", "male", "male", "female", "female"],
            "age":       [22.0, 38.0, 35.0, 27.0, 35.0, np.nan, 54.0, 2.0, 58.0, np.nan],
            "fare":      [7.25, 71.28, 53.10, 57.00, 8.05, 8.46, 51.86, 21.07, 26.55, 7.7
                          5],
            "embarked":  ["S", "C", "S", "S", "S", "Q", "S", "S", "C", "S"],
        }
    )

## Our running example for the whole notebook:
try:
    titanic = sns.load_dataset("titanic")          # 891 real passenger rows
except Exception as e:
    print("Could not download dataset (offline?):", e)
    titanic = _demo_titanic()                      # tiny fallback (10 rows)
titanic # last expression in a cell is displayed automatically in Jupyter
```

1. Why pandas?

Raw NumPy arrays are great for numbers, but real datasets have:

- **labels** (column names, meaningful row indexes) instead of position 0..n
- **heterogeneous columns** (numbers, text, dates, categories side by side)

- **messiness** (missing values, duplicates, mixed types in one column)

pandas bundles all three concerns into one object model. A Series is a single labeled column; a DataFrame is a table of Series sharing one index. Everything else in this notebook is about building, inspecting, and slicing those two objects.

1.1 Creating a Series

WHAT: a Series = 1 D data + an index (the labels on the left).

```
## WHY: the index lets us reference data by MEANING ("2024-Q1"), not by position.

## From a plain list -> pandas assigns a default RangeIndex, 1,2,...
temps = pd.Series([21.5, 22.1, 19.8, 23.4])
print(temps, "\n")

## From a dict -> keys become the (custom) index labels.
pop = pd.Series({"Tokyo": 37.4, "Delhi": 32.9, "Shanghai": 29.2})
print(pop, "\n")

## Scalar + explicit index -> the scalar is broadcast to every label.
profit = pd.Series(1000, index=["Q1", "Q2", "Q3", "Q4"])
print(profit)
```

1.2 Peeking inside a Series

```
## .values -> the raw NumPy array underneath (positions only, no labels).
print("values :", temps.values)

## .index -> the label machinery (a RangeIndex here because we passed a list).
print("index :", temps.index)

## .dtype -> ONE dtype per Series (that's why mixed-type data lands in 'object').
print("dtype :", temps.dtype)

## Vectorized ops apply to EVERY element without a loop -- fast and readable.
print("\ntemps + 2.0:\n", temps + 2.0)
print("\nnumpy ufuncs work too:\n", np.round(temps))
```

Output:

```
values : [21.5 22.1 19.8 23.4]
index  : RangeIndex(start=0, stop=4, step=1)
dtype  : float64

temps + 2.0:
0      23.5
```

```
1    24.1
2    21.8
3    25.4
dtype: float64

numpy ufuncs work too:
0    22.0
1    22.0
2    20.0
3    23.0
dtype: float64
```

1.3 Alignment: the superpower (and the classic gotcha)

```
## When two Series interact, pandas aligns them BY INDEX LABEL first,
## then applies the operation label-by-label.
s1 = pd.Series([1, 2, 3], index=["a", "b", "c"])
s2 = pd.Series([10, 20, 30], index=["b", "c", "d"]) # note: 'a' and 'd' differ

s_sum = s1 + s2
print(s_sum)

## Common pitfall: labels present in only ONE operand become NaN --
## arithmetic with NaN propagates NaN. Not a bug: alignment working as designed!

## Pro tip: most operators have a method that accepts fill_value, letting us
## treat "missing on one side" as 0 during the operation.
print()
print(s1.add(s2, fill_value=0)) # 'a' gets 1+0, 'd' gets 0+30
```

Output:

```
a    NaN
b    12.0
c    23.0
d    NaN
dtype: float64

a    1.0
b    12.0
c    23.0
d    30.0
dtype: float64
```

2. Building DataFrames

WHAT: a DataFrame is a dict like container of columns sharing one index. WHY: it mirrors how tabular data actually looks (rows $\times$ named columns).

From a dict of lists: keys -> column names, lists -> column contents.

```
students = pd.DataFrame(
    {
        "name": ["Ann", "Ben", "Cal", "Dee"],
        "age": [23, 31, 19, 27],
        "grade": [88.5, 72.0, 95.5, 67.0],
    }
)
print(students, "\n")

## From a dict of Series: watch ALIGNMENT happen again -- 'Berlin' exists only
## on one side, so pandas fills the union of labels with NaN where needed.
pop_2010 = pd.Series({"Tokyo": 36.8, "Delhi": 22.0, "Shanghai": 20.3})
pop_2025 = pd.Series({"Tokyo": 37.4, "Delhi": 32.9, "Lagos": 16.6})
growth = pd.DataFrame({"y2010": pop_2010, "y2025": pop_2025})
print(growth) # Shanghai has 2010-only, Lagos has 2025-only -> NaN elsewhere
```

3. Reading data: read_csv and friends

WHAT: read_csv turns delimited text into a DataFrame. WHY: 90% of real world files arrive as CSV, and the function's many knobs let us fix messiness AT LOAD TIME instead of patching afterwards.

```
from io import StringIO # lets us pretend a string is a file -- perfect for demos

csv_text = """id,name,city,score,joined
1,Alice,London,88,2021-03-14
2,Bob,Paris,N/A,2020-11-02
3,Carla,Berlin,79.5,?
4,Dan,Dublin,91.25,2022-01-31
5,Eve,,66.0,2021-07-04
"""

survey = pd.read_csv(
    StringIO(csv_text),          # wrap the string so read_csv can "read" it
    sep=",",                    # field separator (use ";" or "\t" when needed)
    header=0,                   # row 0 holds the column names (the default)
    index_col="id",             # use the 'id' column as row labels
    dtype={"score": "float64"}, # force a dtype instead of guessing
    na_values=["N/A", "?"],     # extra strings to recognize as missing
    parse_dates=["joined"],     # convert date-looking columns to datetime64
)
```

```
print(survey, "\n")
print(survey.dtypes)           # score is float, joined is datetime64 -- clean!
```

Output:

```
name    city  score    joined
id
1  Alice  London  88.00 2021-03-14
2   Bob   Paris   NaN 2020-11-02
3  Carla  Berlin  79.50      NaT
4   Dan   Dublin  91.25 2022-01-31
5   Eve    NaN   66.00 2021-07-04

name                str
city                str
score              float64
joined  datetime64[us]
dtype: object
```

Above we loaded **titanic** (891 rows in the full version), it is our running example: one row per passenger, mixing numbers, text, categories, and missing values. Realistic enough to hurt, small enough to inspect.

3.1 First look at the running example

```
print(titanic.shape)           # (rows, columns)
print(titanic.columns.tolist())
titanic.head()                 # first 5 rows; head(10) would give ten
```

Output:

```
(891, 15)
['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'class', 'who',
 'adult_male', 'deck', 'embark_town', 'alive', 'alone']
```

Output:

```
survived  pclass    sex  age  sibsp  parch    fare  embarked  class \
0         0        3   male  22.0    1     0   7.2500         S   Third
1         1        1  female  38.0    1     0  71.2833         C   First
2         1        3  female  26.0    0     0   7.9250         S   Third
3         1        1  female  35.0    1     0  53.1000         S   First
4         0        3   male  35.0    0     0   8.0500         S   Third

      who  adult_male  deck  embark_town  alive  alone
0   man         True   NaN  Southampton    no  False
1 woman        False    C   Cherbourg   yes  False
```



```

2  woman      False  NaN  Southampton  yes  True
3  woman      False   C  Southampton  yes  False
4   man       True   NaN  Southampton  no   True

```

4. The inspection toolkit

WHAT: a handful of one liners that answer “what did I just load?” WHY: ALWAYS inspect before analyzing, wrong dtypes, surprise missing values, or phantom duplicates ruin downstream results silently.

Quick peeks: head (top), tail (bottom), sample (random rows).

```

## random_state makes 'random' reproducible - same rows every run.
print(titanic.tail(3), "\n")
print(titanic.sample(5, random_state=42))

```

Shape / size / column names / dtypes , the structural X ray.

```

print("shape :", titanic.shape)          # tuple (n_rows, n_cols)
print("size  :", titanic.size)           # n_rows * n_cols cells total
print("index :", titanic.index)          # row-label machinery
print("\ndtypes:\n", titanic.dtypes)     # one dtype per column

```

```

## info() = the single best first move on ANY new dataset:
## row count, dtypes, and NON-null counts per column (so gaps pop out).
titanic.info()

```

Output:

```

<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null   int64
1   pclass      891 non-null   int64
2   sex         891 non-null   str
3   age         714 non-null   float64
4   sibsp       891 non-null   int64
5   parch       891 non-null   int64
6   fare        891 non-null   float64
7   embarked    889 non-null   str
8   class       891 non-null   category
9   who         891 non-null   str

```

```
10  adult_male    891 non-null    bool
11  deck          203 non-null    category
12  embark_town   889 non-null    str
13  alive         891 non-null    str
14  alone         891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

```
## describe(): numeric summaries by default; include="all" adds text/category
## columns (their 'summary' is count/unique/top/freq instead of mean/std).
(
    titanic[["age", "fare", "sex", "embarked"]] # subset keeps output readable
    .describe(include="all")
)
```

Output:

	age	fare	sex	embarked	
count	714.000000	891.000000	891	889	
unique	NaN	NaN	2	3	
top	NaN	NaN	male	S	
freq	NaN	NaN	577	644	
mean	29.699118	32.204208	NaN	NaN	
std	14.526497	49.693429	NaN	NaN	
min	0.420000	0.000000	NaN	NaN	
25%	20.125000	7.910400	NaN	NaN	
50%	28.000000	14.454200	NaN	NaN	
75%	38.000000	31.000000	NaN	NaN	
max	80.000000	512.329200	NaN	NaN	

```
## value_counts: frequency table of a Series. dropna=False shows the NaN bucket
## too -- by default missing values are hidden, which hides problems!
print(titanic["embarked"].value_counts(dropna=False), "\n")

## nunique: how many DISTINCT values (great sanity check for ID-like columns).
print("distinct ports:", titanic["embarked"].nunique())

## memory_usage(deep=True): honest byte count including strings; useful early.
mb = titanic.memory_usage(deep=True).sum() / 1024**2
print(f"titanic uses about {mb:.2f} MB")
```

Output:

embarked	
S	644
C	168
Q	77
NaN	2

```
Name: count, dtype: int64

distinct ports: 3
titanic uses about 0.31 MB
```

Pro tip: make `df.info()` + `df.describe(include='all')` + `value_counts(dropna=False)` your reflexive opening trio on every dataset.

5. Selecting data: [], .loc, .iloc

Three doors into a DataFrame, each with a distinct contract:

syntax	meaning	slice rule
<code>df[col]</code>	column(s) by NAME	,
<code>.loc</code>	rows/cols by LABEL or boolean	inclusive of end
<code>.iloc</code>	rows/cols by POSITION (0 based)	exclusive of end

5.1 Column selection with brackets

```
age = titanic["age"]          # single name -> Series
subset = titanic[["age", "fare"]] # LIST of names -> DataFrame (note double [])

print(type(age).__name__, age.name, "| shape:", age.shape)
print(type(subset).__name__, "| shape:", subset.shape)

## Dot access works for simple names...
print(titanic.age.head(3).tolist(), "... convenient but fragile:")

## Common pitfall: dot access breaks for names with spaces, leading digits,
## or Python keywords, and tools can't see the column name as a string.
## Rule of thumb: dot access in quick interactive sessions, BRACKETS everywhere else.
```

Output:

```
Series age | shape: (891,)
DataFrame | shape: (891, 2)
[22.0, 38.0, 26.0] ... convenient but fragile:
```

5.2 .loc, select by LABEL

```
people = pd.DataFrame(
    {"score": [90, 85, 70]},
    index=["ann", "ben", "cal"],
```

```
)
print(people, "\n")

print(people.loc["ben"], "\n")          # one label -> row as Series
print(people.loc[["ann", "cal"]], "\n") # list of labels -> those rows

## Label slices INCLUDE the endpoint (unlike Python lists!):
print(people.loc["ann":"cal"])
```

Output:

```
score
ann    90
ben    85
cal    70

score    85
Name: ben, dtype: int64

      score
ann      90
cal      70

      score
ann      90
ben      85
cal      70
```

```
## Common pitfall previewed: with INTEGER labels, .loc slices by label, not
## position -- a famous source of off-by-one confusion.
weird = pd.DataFrame({"score": [10, 20, 30]}, index=[10, 20, 30])

print(weird.loc[10:20], " <- .loc: LABELS 10 and 20, inclusive\n")
print(weird.iloc[0:2], " <- .iloc: POSITIONS 0 and 1, exclusive end")
```

Output:

```
score
10    10
20    20 <- .loc: LABELS 10 and 20, inclusive

      score
10      10
20      20 <- .iloc: POSITIONS 0 and 1, exclusive end
```

```
## .loc takes [row_selector, column_selector] -- rows AND columns in one shot.
first_three = titanic.loc[:2, ["sex", "age", "fare"]] # labels 0,1,2 (inclusive!)
print(first_three.head())
```

```
## A boolean ARRAY in the row slot = filtering (deep dive next section).
adults = titanic.loc[titanic["age"] >= 18, ["sex", "pclass", "fare"]]
print("\nadults shape:", adults.shape)
```

Output:

```
sex  age  fare
0  male  22.0  7.2500
1  female 38.0  71.2833
2  female 26.0  7.9250

adults shape: (601, 3)
```

Pro tip: need exactly ONE scalar? `.at[row, col]` (labels) and `.iat[i, j]` (positions) are faster micro versions of `.loc/.iloc`.

```
## .iloc = pure integer positions, C-style half-open slices like ordinary lists.
print(titanic.iloc[:3, :2], "\n") # rows 0-2, cols 0-1 (end EXCLUSIVE)

## Negative positions work like list indexing:
print(titanic.iloc[-2:, -2:]) # last two rows, last two columns

## Single cell:
print("cell [5, 'fare'] by position:", titanic.iloc[5, 7])
```

Output:

```
survived  pclass
0         0      3
1         1      1
2         1      3

   alive  alone
889  yes   True
890  no    True
cell [5, 'fare'] by position: Q
```

5.3 Which selector when? (head to head)

you want...	reach for
one column	<code>df["col"]</code>
several columns	<code>df[["a", "b"]]</code>
rows by their LABEL range	<code>df.loc["a":"c"]</code> (inclusive)

you want...	reach for
first/last N rows by position	<code>df.iloc[:5], df.iloc[-5:]</code>
filter + pick columns in one step	<code>df.loc[mask, cols]</code>
one scalar, hot path	<code>df.at[r, c] / df.iat[i, j]</code>

Common pitfall: `df[0:5]` on a label indexed frame slices by POSITION (special case!), while `df["0": "5"]` would be labels. When in doubt, say `.loc` or `.iloc` explicitly, future you will thank you.

6. Boolean filtering

WHAT: compare a column -> get a True/False mask -> index with it. WHY: "give me the rows WHERE..." is the heart of everyday analysis.

Step: build the mask. Step: apply it. Keep the steps separate at first ,

```
## it makes debugging much easier.
mask_female = titanic["sex"] == "female"
print(mask_female.head(3), "\n")

females = titanic[mask_female]
print("female passengers:", len(females))

## Combine masks with & (AND), | (OR), ~ (NOT) -- NEVER Python's and/or/not.

## Common pitfall: comparisons bind TIGHTER than & and |, so parentheses are
## MANDATORY around each condition. Without them you get a confusing TypeError.
surviving_women_third = titanic[
    (titanic["sex"] == "female") & (titanic["pclass"] == 3) & (titanic["survived"] == 1)
]
print("surviving women in 3rd class:", len(surviving_women_third))

## ~ flips a mask: everyone who did NOT pay in USD-top-fare bracket below.
not_cheap = titanic[~(titanic["fare"] < 10)]
print("fares >= 10:", len(not_cheap))
```

Handy predicate helpers:

```
ports = titanic[titanic["embarked"].isin(["C", "Q"])]      # membership set
print("boarded at C or Q:", len(ports))

mid_fares = titanic[titanic["fare"].between(20, 50)]      # inclusive range
print("20 <= fare <= 50:", len(mid_fares))
```

```
## A quick taste of .str methods (full suite in the Advanced notebook):
emails = pd.Series(["ann@x.com", "bob@spam.net", "carol@x.com"])
x_folk = emails[emails.str.startswith("c")]          # pattern match
print(x_folk.tolist())
```

```
## query(): write the filter as a STRING -- often the most readable option for
## complex conditions. Column names become variables inside the expression.
result = titanic.query("pclass == 1 and fare > 80 and sex == 'female'")
print(result[["sex", "pclass", "fare"]].head())
```

```
## Pro tip: query shines in method chains (see the Advanced notebook) and
## supports 'in', 'not in', 'between'-style logic via plain English-ish syntax.
```

Output:

sex	pclass	fare
31	female	1 146.5208
88	female	1 263.0000
195	female	1 146.5208
215	female	1 113.2750
230	female	1 83.4750

7. Sorting and ranking

WHAT: order rows by column values (sort_values) or by index labels (sort_index); rank converts values into competition style ranks. WHY: "top 5 biggest..." questions always start with a sort.

```
by_fare = titanic.sort_values(by="fare", ascending=False, na_position="last")
print(by_fare[["sex", "pclass", "fare"]].head(3))

## Multiple keys: list the columns, list the directions (parallel lists).
two_keys = titanic.sort_values(
    by=["pclass", "fare"], ascending=[True, False]
)[["pclass", "fare"]]
print("\ncheapest fare within EACH class comes last:\n", two_keys.groupby("pclass").tail
      (1))

## sort_index puts rows back in index order -- handy after shuffling/sampling:
shuffled = titanic.sample(frac=1, random_state=42)  # frac=1 -> ALL rows, shuffled
restored = shuffled.sort_index()
print("\nsample+sort_index reproduces the original head:")
print(restored.head(3)[["sex", "age"]])
```

Output:

sex	pclass	fare
679	male	1 512.3292

```
258  female      1  512.3292
737   male      1  512.3292
```

cheapest fare within EACH class comes last:

```
      pclass  fare
822      1    0.0
732      2    0.0
597      3    0.0
```

sample+sort_index reproduces the original head:

```
      sex  age
0   male  22.0
1  female  38.0
2  female  26.0
```

```
## rank(pct=True): fraction of values <= this one (0..1]. Max fare -> 1.0.
fare_pct = titanic["fare"].rank(pct=True)

top3 = titanic.sort_values("fare", ascending=False).head(3)
top3 = top3.assign(fare_percentile=fare_pct[top3.index]) # aligns by row label!
print(top3[["sex", "pclass", "fare", "fare_percentile"]])

## rank(method=...) matters with ties: try "average" (default), "min",
## "dense", "first" to control how equal values are handled.
```

Output:

```
sex  pclass      fare  fare_percentile
679   male      1  512.3292          0.998878
258  female      1  512.3292          0.998878
737   male      1  512.3292          0.998878
```

8. Derived columns: [] assignment and assign()

Direct assignment creates (or OVERWRITES) a column in place.

```
titanic["family_size"] = titanic["sibsp"] + titanic["parch"] + 1 # + self
titanic["log_fare"] = np.log1p(titanic["fare"]) # log1p handles fare==0 safely
titanic[["family_size", "log_fare"]].head()
```

```
## Common pitfall: direct assignment MUTATES the DataFrame in place. If the
## frame came from a filter chain you may be editing a throwaway copy. Prefer
## .copy() once, then assign -- or better, the functional style below.
```

```
## assign() RETURNS A NEW frame with added/updated columns -- chain-friendly.
## lambdas receive the frame-so-far as 'd', so later columns can use earlier ones.
enriched = (
```



```

titanic
.assign(
    family_size=lambda d: d["sibsp"] + d["parch"] + 1,
    fare_per_person=lambda d: d["fare"] / d["family_size"],
)
.query("fare_per_person > 0")      # drops fare==0 records cleanly
)
enriched[["sex", "pclass", "family_size", "fare_per_person"]].head()

```

Output:

	sex	pclass	family_size	fare_per_person
0	male	3	2	3.62500
1	female	1	2	35.64165
2	female	3	1	7.92500
3	female	1	2	26.55000
4	male	3	1	8.05000

9. Dropping, renaming, and index surgery

WHAT: reshape the table's skeleton, remove rows/columns, rename headers, promote a column to (or demote from) the index. WHY: tidy inputs make every later step simpler.

```

## drop() defaults to rows (axis=0); axis=1 / columns= targets columns.
## errors="ignore" means 'don't complain if it's already gone' -- safe cleanup.
slim = titanic.drop(columns=["deck"], errors="ignore") # deck is ~77% missing anyway
row_dropped = slim.drop(index=[0, 1])                # remove rows by label
axis_style = slim.drop(["deck"], axis=1, errors="ignore") # older equivalent style
print(slim.shape, "->", row_dropped.shape)

renamed = slim.rename(columns={"embarked": "port_code"}) # mapper dict, returns COPY
print(list(renamed.columns)[:8], "...")

```

Output:

```

(891, 16) -> (889, 16)
['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'port_code'] ...

```

```

## set_index promotes a column to row labels; reset_index reverses it.
people = pd.DataFrame(
    {"dept": ["Eng", "Sales", "Eng"], "salary": [95, 70, 88]},
    index=["ann", "ben", "cal"],
)
by_name = people.set_index("dept")      # wait -- set_index on a COLUMN works too;
print(by_name, "\n")                  # duplicate labels are allowed but slower look
ups

```

```
back_to_columns = by_name.reset_index() # index becomes a normal column again
print(back_to_columns)

## reset_index(drop=True) THROWS the old index away -- ideal after filtering,
## when leftover non-consecutive labels (3,, 214...) get confusing.
sampled = titanic.sample(3, random_state=1)
clean_labels = sampled.reset_index(drop=True)
print(clean_labels.index.tolist()) # boring old 0,1,2 -- exactly what we want
```

Output:

```
salary
dept
Eng      95
Sales    70
Eng      88
```

Output:

```
dept salary
0    Eng    95
1  Sales    70
2    Eng    88
[0, 1, 2]
```

10. Foundations checklist (head to head recap)

task	idiomatic call
peek at data	head / tail / sample / info / describe
one column / many columns	df["col"] / df[["a","b"]]
rows by label	df.loc[label_or_mask, cols] (inclusive slices)
rows by position	df.iloc[i:j, k] (exclusive slices)
complex filter	(cond1) & (cond2) or df.query(...)
membership / range	isin([...]) / between(lo, hi)
top N questions	sort_values(...) (+ .head(n))
percentile of value	Series.rank(pct=True)
add columns functionally	assign(col=lambda d: ...)
delete / rename	drop(...), rename(columns=mapper)
index gymnastics	set_index / reset_index(drop=...)

Pro tip: press Tab after df. in Jupyter to explore attributes, and put ? after any call (e.g. df.sort_values?) to read its docs inline.

Summary & key takeaways

- **pandas = labeled, heterogeneous, messy data friendly tables**, Series (one labeled column) compose into DataFrames.
- Operations **align on index labels**; mismatched labels yield NaN, not errors. Use `.add(other, fill_value=...)` to control that.
- Load with `read_csv(sep=..., header=..., index_col=..., dtype=..., na_values=..., parse_dates=...)` - fix messiness at the door.
- Inspect FIRST: `info()`, `describe(include="all")`, `value_counts(dropna=False)`, `nunique()`, `memory_usage(deep=True)`.
- Selection contracts: `[]` for columns, `.loc` = labels (**inclusive** slices, boolean masks welcome), `.iloc` = positions (**exclusive**).
- Filter with `&` | `~`, parentheses mandatory; `isin`, `between`, and `query()` cover the everyday special cases.
- Sort with multi key `sort_values(ascending=[...])`; `rank(pct=True)` answers "how big is this value relative to the rest?"
- Prefer `assign()` chains for derived columns; `drop/rename/reset_index` return copies, keeping your pipeline predictable.

Next up: `02_pandas_intermediate.nb.py`, missing data, groupby deep dive, reshaping, and combining tables.

4.2 Pandas, Pandas Intermediate

MLCourse · Data Science Foundations · 02_pandas

You know how to build, inspect, and slice DataFrames (notebook 01). Now we tackle what makes real analysis *real*: messy values, grouped summaries, reshaping, and stitching tables together.

What you'll learn

- Missing data: detecting it, explaining it, dropping and filling it wisely (including **group wise imputation**)
- Finding and removing duplicates
- Fixing dtypes: `astype`, `pd.to_numeric(errors="coerce")`, `pd.to_datetime`
- The apply family (`map`, `apply`, `elementwise .map`), and when NOT to use it
- `replace`, binning with `pd.cut` / `pd.qcut`
- GroupBy deep dive: `split` `apply` `combine`, named aggregation, `transform`, `filter`
- Pivot tables & crosstabs; `melt/pivot/stack/unstack` reshaping
- Combining frames: `concat`, the four flavors of `merge`, and `join`

Setup cell, same safe import pattern as notebook 01.

```

try:
    get_ipython().run_line_magic("matplotlib", "inline")
except NameError:
    pass # running as a plain script

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)
pd.set_option("display.max_columns", 120)

def _load(name, demo_builder):
    """Download a seaborn dataset if possible, else use tiny demo rows."""
    try:
        return sns.load_dataset(name)
    except Exception as e:
        print(f"[setup] '{name}' unavailable ({type(e).__name__}) -> using demo rows.")
        return demo_builder()

def _demo_titanic():
    """Miniature titanic stand-in (same shape as the real thing)."""
    return pd.DataFrame(
        {
            "survived": [0, 1, 1, 1, 0, 0, 1, 0, 1, 0],
            "pclass": [3, 1, 1, 1, 3, 3, 1, 3, 2, 3],
            "sex": ["male", "female", "female", "female",
                  "male", "male", "male", "male", "female", "female"],
            "age": [22.0, 38.0, 35.0, 27.0, 35.0, np.nan, 54.0, 2.0, 58.0, np.nan],
            "sibsp": [1, 1, 0, 1, 0, 0, 0, 4, 0, 0], # siblings/spouses aboard
            "parch": [0, 0, 0, 0, 0, 0, 1, 1, 0, 2], # parents/children aboard
            "fare": [7.25, 71.28, 53.10, 57.00, 8.05, 8.46, 51.86, 21.07, 26.55, 7.7
5],
            "embarked": ["S", "C", "S", "S", "S", "Q", "S", "S", None, "S"],
        }
    )

def _demo_tips():
    """Miniature tips stand-in."""
    return pd.DataFrame(
        {
            "total_bill": [16.99, 10.34, 21.01, 23.68, 25.29, 8.77, 26.88, 15.04, 17.92,
44.30],
            "tip": [1.01, 1.66, 3.50, 3.31, 2.73, 2.00, 5.00, 2.42, 3.08, 6.00],

```

```

        "sex":      ["F", "M", "M", "M", "F", "M", "M", "M", "F", "M"],
        "day":      ["Sun", "Sun", "Sat", "Sat", "Fri", "Thur", "Sun", "Sat", "Sat",
        "Fri"],
        "time":     ["Dinner", "Dinner", "Dinner", "Dinner", "Lunch",
        "Lunch", "Dinner", "Dinner", "Dinner", "Lunch"],
        "size":     [2, 3, 3, 2, 4, 2, 4, 2, 2, 6],
    }
)

try:
    titanic = sns.load_dataset("titanic")
except Exception as e:
    print("Could not download dataset (offline?):", e)
    titanic = _demo_titanic()

try:
    tips = sns.load_dataset("tips")
except Exception as e:
    print("Could not download dataset (offline?):", e)
    tips = _demo_tips()

print("titanic:", titanic.shape, "| tips:", tips.shape)

```

1. Missing data

1.1 Three flavors of "nothing"

object	comes from	notes
None	Python (lists of mixed objects)	becomes NaN inside numeric Series
np.nan	NumPy float world	a float! nan != nan is True ish
pd.NA	modern nullable dtypes (Int64)	pandas' missing value singleton

WHY care: you never test missingness with ==; you use isna().

```

mixed = pd.Series([1.5, None, np.nan, 3.0]) # None is coerced to np.nan here
print(mixed, "\n")

## Common pitfall: comparing to NaN directly NEVER works -- even NaN == NaN
## is False! Equality can't represent "unknown".
print("x == nan gives:", mixed == np.nan)

## The RIGHT tools: isna()/notna() work for None, NaN, and pd.NA alike.
print("\nisna:", mixed.isna().tolist())
print("notna:", mixed.notna().tolist())

```

Output:

```
0    1.5
1    NaN
2    NaN
3    3.0
dtype: float64

x == nan gives: 0    False
1    False
2    False
3    False
dtype: bool

isna: [False, True, True, False]
notna: [True, False, False, True]
```

Per column missing counts -> percentage table, biggest gaps first.

```
miss_cnt = titanic.isna().sum()          # True counts as 1 when summed
miss_pct = (miss_cnt / len(titanic) * 100).round(1)  # share of rows missing

missing_report = (
    pd.DataFrame({"n_missing": miss_cnt, "pct": miss_pct})
    .query("n_missing > 0")
    .sort_values("pct", ascending=False)
)
missing_report
```

1.2 Why does data go missing?

Not all NaNs are equal, and the reason changes your fix:

- **Not recorded** (someone skipped a survey question), often random.
- **Does not apply** ("number of children" for a child), structural.
- **Lost in processing** (failed join, bad parse), your own bug!

Common pitfall: filling without thinking injects bias. A deck column that's 77% empty tells you *the information doesn't exist*, no clever number will resurrect it.

```
## dropna(): remove rows (or columns) containing missing values.
tiny = pd.DataFrame(
    {
        "a": [1.0, np.nan, 3.0, np.nan],
        "b": [4.0, 5.0, np.nan, np.nan],
        "c": [7.0, 8.0, 9.0, np.nan],
    },
```

```

    index=list("wxyz"),
)
print(tiny, "\n")

print(tiny.dropna(), "\n")          # how="any": ANY NaN in row -> gone
print(tiny.dropna(how="all"), "\n") # only fully-empty rows die ('y' survives)

print(tiny.dropna(subset=["a"]), "\n")      # judge only column 'a'
print(tiny.dropna(thresh=2))               # keep rows with >= 2 real values

## Pro tip: dropping COLUMNS with huge gap shares (e.g. deck at ~77%) is
## often better than dropping thousands of ROWS over them.

```

Output:

```

a    b    c
w  1.0  4.0  7.0
x  NaN  5.0  8.0
y  3.0  NaN  9.0
z  NaN  NaN  NaN

```

```

      a    b    c
w  1.0  4.0  7.0

```

```

      a    b    c
w  1.0  4.0  7.0
x  NaN  5.0  8.0
y  3.0  NaN  9.0

```

```

      a    b    c
w  1.0  4.0  7.0
y  3.0  NaN  9.0

```

```

      a    b    c
w  1.0  4.0  7.0
x  NaN  5.0  8.0
y  3.0  NaN  9.0

```

```

## fillna(): plug holes with constants or statistics.
messy = pd.Series([1.0, np.nan, np.nan, 8.0])

print(messy.fillna(0), "\n")          # constant sentinel -- fine for flags

## For measurements, prefer median/mean -- but WHICH one?
skewed = pd.Series([10, 12, 11, 13, 11, 500]) # 500 is an outlier!
print("mean :", skewed.mean())             # dragged way up by 500
print("median:", skewed.median())          # robust to outliers

## Rule of thumb: SKEWED data -> median; roughly symmetric -> mean;
## categorical -> mode (most frequent value).

```

```
print("\nmedian fill:", messy.fillna(messy.median()).tolist())

## ffill/bfill: propagate the last (or next) VALID value forward/backward --
## designed for ordered data like time series or logs.
ordered = pd.Series([3.0, np.nan, np.nan, 9.0])
print("\nffill:", ordered.ffill().tolist())
print("bfill:", ordered.bfill().tolist())
```

Output:

```
0    1.0
1    0.0
2    0.0
3    8.0
dtype: float64

mean : 92.83333333333333
median: 11.5

median fill: [1.0, 4.5, 4.5, 8.0]

ffill: [3.0, 3.0, 3.0, 9.0]
bfill: [3.0, 9.0, 9.0, 9.0]
```

1.3 Group wise imputation (the professional move)

Filling every missing age with ONE global median ignores structure. Ages in 1st class skew older than in 3rd; men/women differ too. So we compute the median **within each (class, sex) group** and fill with that.

The tool: `groupby(...)[ 'col' ].transform('median')`, it returns a Series aligned 1 to-1 with the original rows, exactly what `fillna` wants.

Step: one group median PER ROW (transform broadcasts group stats back).

```
group_median_age = titanic.groupby(["pclass", "sex"])["age"].transform("median")

## Step: fill ONLY the holes; observed values stay untouched.
titanic_imputed = titanic.assign(
    age_filled=titanic["age"].fillna(group_median_age)
)

## Sanity checks: nothing lost, holes plugged.
print("before:", titanic["age"].isna().sum(), "missing")
print("after :", titanic_imputed["age_filled"].isna().sum(), "missing")

print(titanic_imputed.loc[titanic["age"].isna(), ["pclass", "sex", "age_filled"]].head())
```



```

)

## Pro tip: in ML pipelines, compute the medians on TRAINING data only and
## reuse them on test data -- otherwise you leak future knowledge into training.

```

2. Duplicates

WHAT: identical (or key identical) rows. WHY remove: they double count customers, inflate aggregates, and break joins silently.

```

signups = pd.DataFrame(
    {
        "email":      ["a@x.com", "b@x.com", "a@x.com", "c@x.com", "a@x.com"],
        "plan":       ["free", "pro", "free", "free", "free"],
        "signed_at":  pd.to_datetime(
            ["2024-01-01", "2024-01-02", "2024-01-05", "2024-01-03", "2024-01-04"]
        ),
    }
)

## duplicated() marks repeats AFTER the first occurrence (keep='first').
print("duplicate rows:", signups.duplicated().sum())
print(signups[signups.duplicated()], "\n")

## Drop exact duplicates across ALL columns:
deduped = signups.drop_duplicates()
print(deduped.shape, "after full-row dedupe\n")

## Business rule: keep only each person's LATEST signup.
## Sort so the newest row lands LAST within each email group, then keep='last'.
latest = (
    signups
    .sort_values("signed_at")
    .drop_duplicates(subset=["email"], keep="last")  # subset = identity key(s)
    .sort_index()                                  # restore pleasant order
)
latest

## Pro tip: keep=False marks EVERY copy of a duplicate (great for auditing
## before you delete anything).

```

Output:

```

duplicate rows: 0
Empty DataFrame
Columns: [email, plan, signed_at]
Index: []

```

(5, 3) after full-row dedupe

Output:

```
email plan signed_at
1 b@x.com pro 2024-01-02
2 a@x.com free 2024-01-05
3 c@x.com free 2024-01-03
```

3. Fixing dtypes

CSVs love to deliver numbers as strings, dates as strings, categories as strings... Converting early makes everything downstream faster and safer.

- object = “bag of Python objects”, usually strings; slow and vague.
- category = small integer codes + lookup table, fast, memory lean, and semantically honest for repeated labels.

```
## 3.1 Numbers trapped in strings -> pd.to_numeric with errors="coerce".
dirty = pd.Series(["45", " 67 ", "N/A", "89", "abc"]) # spaces AND garbage
cleaned = pd.to_numeric(dirty.str.strip(), errors="coerce")
## errors="coerce": unparseable values become NaN instead of raising.
print(cleaned, "\n")
print("coerced to NaN:", cleaned.isna().sum(), "values")
```

Output:

```
0    45.0
1    67.0
2     NaN
3    89.0
4     NaN
dtype: float64
```

coerced to NaN: 2 values

```
## 3.2 Dates from strings -> pd.to_datetime.
dates_messy = pd.Series(["2024-03-01", "01/04/2024", "not a date"])
parsed = pd.to_datetime(dates_messy, errors="coerce", format="mixed")
## format="mixed" lets pandas parse entries individually (pandas >= 2.0);
## for a single known layout pass it explicitly, e.g. format="%d/%m/%Y".
print(parsed, "\n")

## Explicit format = fastest AND unambiguous (is 01/04 April 1st or Jan 4th?).
eu_style = pd.Series(["01/04/2024", "25/12/2024"]) # day/month/year
sure = pd.to_datetime(eu_style, format="%d/%m/%Y")
print(sure.tolist())
```

Output:

```
0    2024-03-01
1    2024-01-04
2             NaT
dtype: datetime64[us]

[Timestamp('2024-04-01 00:00:00'), Timestamp('2024-12-25 00:00:00')]
```

```
## 3.3 Strings -> category dtype.
before_mb = titanic["sex"].memory_usage(deep=True)
titanic_cat = titanic.assign(sex=titanic["sex"].astype("category"))
after_mb = titanic_cat["sex"].memory_usage(deep=True)
print(f"object: {before_mb:,} bytes -> category: {after_mb:,} bytes "
      f"({after_mb / before_mb:.0%} of original)")

## Full treatment of categorical SUPERPOWERS (ordering, categories mgmt):
## see 03_pandas_advanced.nb.py.
```

Output:

```
object: 55,111 bytes -> category: 1,147 bytes (2% of original)
```

4. The apply family, power tools with a safety label

Decision ladder, fastest first:

1. **Vectorized math** (`col * 2, col.sum()`)
2. **Accessor suites** (`.str.*`, `.dt.*`)
3. **map** with a dict/function (elementwise on one Series)
4. **apply** (row/column functions, flexible but slow)
5. Plain Python loop (last resort)

4.1 map on a Series

```
## map(dict): translate each value through a lookup table.
port_names = {"S": "Southampton", "C": "Cherbourg", "Q": "Queenstown"}
named_ports = titanic["embarked"].map(port_names)
print(named_ports.value_counts(dropna=False), "\n")

## map(function): transform each element.
name_lengths = titanic["sex"].map(len)    # length of each string
print(name_lengths.head(3).tolist())

## map(dict) turns UNMAPPED values into NaN -- verify coverage first!
odd_codes = pd.Series(["S", "C", "X"])    # 'X' isn't in port_names
```

```
print(odd_codes.map(port_names).tolist())
```

Output:

```
embarked
Southampton    644
Cherbourg      168
Queenstown     77
NaN             2
Name: count, dtype: int64

[4, 6, 6]
['Southampton', 'Cherbourg', nan]
```

4.2 Why "vectorize first"? Let's measure.

```
import time as _time # local helper import for this benchmark only

big = pd.Series(np.arange(1_000_001)) # ~one million numbers

t0 = _time.perf_counter() # high-resolution timer
vec_result = big * 2 # vectorized: C speed
t_vec = _time.perf_counter() - t0

t0 = _time.perf_counter()
apply_result = big.apply(lambda x: x * 2) # Python call PER CELL!
t_app = _time.perf_counter() - t0

print(f"vectorized : {t_vec:.4f}s")
print(f".apply      : {t_app:.4f}s -> about {t_app / max(t_vec, 1e-9):.0f}x slower")

## Same answers, wildly different cost. Reach for apply only when NO vectorized
## equivalent exists (custom business logic per row, complex conditionals...).
```

Output:

```
vectorized : 0.0022s
.apply      : 0.3004s -> about 137x slower
```

```
## 4.3 apply on Series vs DataFrame.
series_out = titanic["fare"].apply(lambda f: "expensive" if f > 100 else "normal")
print(series_out.value_counts(), "\n")

## DataFrame.apply works on WHOLE columns (axis=0, default) or rows (axis=1):
mini = pd.DataFrame({"math": [80, 55], "sci": [90, 65]})
print(mini.apply(sum), "\n") # axis=0: one sum PER COLUMN
print(mini.apply(lambda row: row.max() - row.min(), axis=1))
```

```
## Elementwise-on-every-cell used to be df.applymap(...); that name is DEPRECATED
## (removed in recent pandas). Use df.map(...) -- yes, same name as Series.map:
print(mini.map(lambda v: v / 100))
```

Output:

```
fare
normal      838
expensive    53
Name: count, dtype: int64

math      135
sci       155
dtype: int64

0    10
1    10
dtype: int64
   math  sci
0  0.80  0.90
1  0.55  0.65
```

Common pitfall: reaching for `.apply` out of habit. Before writing one, ask: “can this be expressed as arithmetic, `.where`, or `.str`?” Your future self (and your CPU) will be grateful.

5. replace: surgical value swaps

```
## replace() differs from map(): UNTOUCHED values pass through unchanged
## (map would have NaN-ed them), making replace ideal for spot fixes.
codes = pd.Series(["S", "C", "Q", "S", "Z"])
fixed = codes.replace({"S": "Southampton", "C": "Cherbourg"}) # 'Q', 'Z' survive
print(fixed.tolist())

## List->list form replaces positionally: ['bad', 'awful'] -> ['poor', 'poor'].
grades = pd.Series(["good", "bad", "awful"])
print(grades.replace(["bad", "awful"], ["poor", "poor"]).tolist())
```

Output:

```
['Southampton', 'Cherbourg', 'Q', 'Southampton', 'Z']
['good', 'poor', 'poor']
```

6. Binning: continuous -> categories

WHAT: cut a numeric range into buckets. WHY: histograms, age bands, fare classes, humans reason in categories, models sometimes need them too.

```
## pd.cut = EQUAL-WIDTH bins YOU define by edges (plus labels).
age_bands = pd.cut(
    titanic["age"],
    bins=[0, 12, 18, 35, 60, 200],          # edges; right edge inclusive by default
    labels=["child", "teen", "young_adult", "adult", "senior"],
    include_lowest=True,                     # make 0 itself included in bin 1
)
print(age_bands.value_counts(sort=False), "\n") # sort=False keeps band order

## pd.qcut = QUANTILE bins: each bucket gets ~equal COUNTS.
fare_class = pd.qcut(
    titanic["fare"],
    q=4,                                     # quartiles
    labels=["budget", "economy", "premium", "luxury"],
)
print(fare_class.value_counts())

## Common pitfall: qcut fails with "duplicate edges" when many identical
## values sit at a boundary (e.g. hundreds of fares == 8.05). Remedy:
## qcut(..., duplicates="drop") and accept fewer buckets.
```

Output:

```
age
child      69
teen       70
young_adult 358
adult      195
senior      22
Name: count, dtype: int64

fare
economy    224
budget     223
premium    222
luxury     222
Name: count, dtype: int64
```

7. GroupBy deep dive

The mental model, **split apply combine**:

```
split    : { class==1: rows..., class==2: rows..., class==3: rows... }
```

```

apply    : mean(fare) computed INSIDE each bucket
combine  : one result row per bucket, stitched back together

```

Every groupby call you will ever write is a variation on this theme.

Watching split apply combine happen live: iterate groups explicitly

```

## (fine for exploration; avoid loops in production pipelines).
for class_id, group_df in titanic.groupby("pclass"):
    print(f"class {class_id}: {len(group_df):4d} passengers, "
          f"avg fare ${group_df['fare'].mean():.2f}")

```

```

## agg with a DICT: different stats for different columns.
summary_dict = titanic.groupby("pclass").agg(
    {"fare": ["mean", "max"], "age": "median"}
)
print(summary_dict, "\n")

## NAMED aggregation -- the modern, readable syntax:
## new_name=("source_col", "aggfunc")
summary_named = titanic.groupby("pclass").agg(
    avg_fare=("fare", "mean"),
    top_fare=("fare", "max"),
    median_age=("age", "median"),
    n=("fare", "count"),          # count ignores NaN -- true sample sizes
).round(2)
print(summary_named)

## Pro tip: results are flat (single-level columns), ready to merge/export --
## one reason named aggregation beats the dict form for reports.

```

Output:

pclass	fare	age	
	mean	max	median
1	84.154687	512.3292	37.0
2	20.662183	73.5000	29.0
3	13.675550	69.5500	24.0

pclass	avg_fare	top_fare	median_age	n
	1	84.15	512.33	37.0
2	20.66	73.50	29.0	184
3	13.68	69.55	24.0	491

```

## transform: SAME-SHAPE output as input -- group stats broadcast back to rows.

```

```

## This is THE tool for "compare each row to its group".
fare_z = (
    titanic["fare"]
    - titanic.groupby("pclass")["fare"].transform("mean") # group center
) / titanic.groupby("pclass")["fare"].transform("std")      # group spread

standardized = titanic.assign(fare_z_within_class=fare_z.round(2))
## Interpretation: fare_z = +2 means "paid 2 standard deviations MORE than
## others in the same class".
print(standardized.nlargest(3, "fare_z_within_class")[["pclass", "fare", "fare_z_within_
    class"]])

## (We already used transform for group-wise AGE imputation in section 1.3.)

```

Output:

pclass	fare	fare_z_within_class
258	1 512.3292	5.46
679	1 512.3292	5.46
737	1 512.3292	5.46

```

## filter: keep/drop ENTIRE GROUPS by a group-level condition.
## Here: keep only weekdays whose average bill tops 20.
day_means = tips.groupby("day")["total_bill"].mean()
print(day_means.round(2), "\n")

busy_days = tips.groupby("day").filter(lambda g: g["total_bill"].mean() > 20)
kept_days = busy_days["day"].unique().tolist()
print("days surviving the filter:", sorted(kept_days))
print("rows before/after:", len(tips), "->", len(busy_days))

## Pro tip: `observed=True` in groupby controls whether unused CATEGORY
## levels appear; pandas 3 defaults sensibly, older versions needed the flag.
## Note: heavy custom logic still has groupby.apply as an escape hatch, but its
## API changed around pandas 2.2+ (grouping columns excluded) -- prefer
## agg/transform/filter whenever possible.

```

Output:

day
Thur 17.68
Fri 17.15
Sat 20.44
Sun 21.41

Name: total_bill, dtype: float64

Output:


```
days surviving the filter: ['Sat', 'Sun']
rows before/after: 244 -> 163
```

8. Grouped pivoting: pivot_table and crosstab

WHAT: groupby + reshape into a spreadsheet style grid. WHY: "average X per (A × B)" reads best as a matrix, not a long list.

Survival rate by sex (rows) and class (columns).

```
survival_grid = pd.pivot_table(
    titanic,
    index="sex",          # becomes row labels
    columns="pclass",     # becomes column labels
    values="survived",    # what to aggregate
    aggfunc="mean",       # survival is 0/1 -> mean = rate
    margins=True,        # add All-rows / All-columns totals
    margins_name="All",
)
print(survival_grid.round(3), "\n")

## Median spend, filling empty combinations explicitly:
fare_grid = pd.pivot_table(
    titanic, index="embarked", columns="pclass",
    values="fare", aggfunc="median", fill_value=0, margins=True,
)
print(fare_grid.round(1))
```

```
## crosstab = frequency-table shortcut (counts by default).
counts = pd.crosstab(titanic["embarked"], titanic["pclass"])
print(counts, "\n")

## normalize='index': convert each ROW to percentages (rows sum to 1.0).
row_pct = pd.crosstab(titanic["embarked"], titanic["pclass"], normalize="index")
print(row_pct.round(3))

## pivot_table vs crosstab: pivot_table takes any aggfunc on any values;
## crosstab specializes in co-occurrence COUNTS (and their normalization).
```

Output:

pclass	1	2	3
embarked			
C	85	17	66
Q	2	3	72

```

S          127  164  353

pclass      1      2      3
embarked
C          0.506  0.101  0.393
Q          0.026  0.039  0.935
S          0.197  0.255  0.548

```

9. Reshaping: melt, pivot, stack, unstack

Two shapes exist: **wide** (measurements as columns) and **long/tidy** (one observation per row, variable names as DATA). Analysis prefers long; presentation prefers wide; you must convert fluently.

```

grades_wide = pd.DataFrame(
    {
        "student": ["Ann", "Ben"],
        "math":    [88, 72],
        "science": [91, 85],
    }
)
print(grades_wide, "\n")

## melt: unpivot measurement columns into (variable, value) rows.
grades_long = grades_wide.melt(
    id_vars="student",          # columns to KEEP as identifiers
    var_name="subject",        # new column holding former col names
    value_name="score",        # new column holding former cell values
)
print(grades_long)

```

Output:

```

student  math  science
0      Ann   88      91
1      Ben   72      85

  student  subject  score
0      Ann    math    88
1      Ben    math    72
2      Ann  science    91
3      Ben  science    85

```

```

## pivot: the inverse operation -- long back to wide.
back_to_wide = grades_long.pivot(
    index="student", columns="subject", values="score"
).reset_index()          # student returns to a normal column

```

```
print(back_to_wide)

## Common pitfall: pivot REJECTS duplicate (index, columns) pairs. When
## duplicates are possible, use pivot_table (it aggregates them away).
```

Output:

```
subject student  math  science
0          Ann    88      91
1          Ben    72      85
```

```
## stack/unstack: the same dance driven by the INDEX.
## groupby with TWO keys produces a MultiIndex Series:
avg_fare = titanic.groupby(["pclass", "sex"])["fare"].mean()
print(avg_fare, "\n")

wide_fares = avg_fare.unstack()          # inner level (sex) -> columns
print(wide_fares.round(2), "\n")

long_again = wide_fares.stack()          # ...and back down to a tall series
print(long_again.head(4))

## stack/unstack shine after multi-key groupbys; pivot/melt shine on plain
## labeled columns. Choose whichever matches where your labels currently live.
```

Output:

```
pclass  sex
1      female  106.125798
      male    67.226127
2      female  21.970121
      male    19.741782
3      female  16.118810
      male    12.661633
Name: fare, dtype: float64

sex      female  male
pclass
1      106.13  67.23
2      21.97  19.74
3      16.12  12.66

pclass  sex
1      female  106.125798
      male    67.226127
2      female  21.970121
      male    19.741782
dtype: float64
```

10. Combining DataFrames

10.1 concat, glue along an axis

```

first_half = tips.iloc[:5]      # positional slice: rows 0-4
second_half = tips.iloc[-3:]    # rows n-3..n-1

## Vertical stack (axis=0 default). ignore_index rebuilds 0..n-1 labels --
## otherwise the ORIGINAL indexes collide (0..4 next to n-3..n-1).
stacked = pd.concat([first_half, second_half], ignore_index=True)
print(stacked.shape, "<- 5 + 3 rows glued\n")

## keys= builds a MultiIndex tagging WHERE each piece came from:
tagged = pd.concat([first_half, second_half], keys=["early", "late"])
print(tagged.head(3), "\n")

## Horizontal concat (axis=1) side-by-side. Mismatched columns become NaN --
## unless you demand strictness with join="inner" (intersection only).
left = pd.DataFrame({"a": [1, 2], "shared": [10, 20]})
right = pd.DataFrame({"b": [99, 77], "shared": [10, 22]})
print(pd.concat([left, right], axis=1), "\n")          # outer: union of cols
print(pd.concat([left, right], axis=1, join="inner"))  # keeps rows where ALL
frames align

```

Output:

```

(8, 7) <- 5 + 3 rows glued

   total_bill  tip  sex smoker  day  time  size
early 0      16.99  1.01 Female    No  Sun  Dinner    2
      1      10.34  1.66   Male    No  Sun  Dinner    3
      2      21.01  3.50   Male    No  Sun  Dinner    3

   a  shared  b  shared
0  1      10  99      10
1  2      20  77      22

   a  shared  b  shared
0  1      10  99      10
1  2      20  77      22

```

10.2 merge, relational joins, the pandas way

how=	keeps...
------	----------

"inner" only keys present in BOTH sides

how=	keeps...
"left"	all left rows; right side NaN filled when unmatched
"right"	mirror image of "left"
"outer"	union of keys; unmatched sides NaN filled

We'll simulate a classic scenario: ORDERS referencing CUSTOMERS via a key.

```
customers = pd.DataFrame(
    {
        "cust_id": [1, 2, 3, 4],
        "name":    ["Ann", "Ben", "Cal", "Dee"],
        "city":    ["Rome", "Oslo", "Lima", "Kyoto"],
    }
)
orders = pd.DataFrame(
    {
        "order_id": [101, 102, 103, 104, 105, 106],
        "cust_id":  [1, 2, 2, 9, 3, 1],          # 9 = ghost customer, 4 = no orders
        "amount":   [250, 80, 120, 300, 40, 60],
        "city":     ["web", "web", "store", "web", "phone", "store"], # overlaps!
    }
)

## INNER: order 104 (ghost id 9) disappears; customer Dee vanishes too.
inner = orders.merge(customers, on="cust_id", how="inner")
print("inner:", len(inner), "orders matched\n")

## LEFT: every order survives; the ghost's customer fields become NaN.
left_joined = orders.merge(customers, on="cust_id", how="left")
print(left_joined[left_joined["name"].isna()], "\n")

## Both tables had a 'city' column -> pandas auto-suffixed them.
print(list(left_joined.columns))

## suffixes= renames the collision explicitly:
labeled = orders.merge(customers, on="cust_id", how="left",
                      suffixes=("_order", "_customer"))
print(list(labeled.columns)[-4:], "<- explicit suffixes")
```

Output:

```
inner: 5 orders matched

  order_id  cust_id  amount  city_x  name  city_y
3       104         9     300    web   NaN    NaN

['order_id', 'cust_id', 'amount', 'city_x', 'name', 'city_y']
['amount', 'city_order', 'name', 'city_customer'] <- explicit suffixes
```

```
## indicator=True adds a provenance column -- perfect for audits:
audited = orders.merge(customers, on="cust_id", how="outer",
                       indicator=True, suffixes=("_o", "_c"))
print(audited["_merge"].value_counts(), "\n")
print(audited[audited["_merge"] == "right_only"][["cust_id", "name"]]) # Dee!

## validate= asserts the relationship cardinality -- cheap insurance:
many_to_one = orders.merge(customers, on="cust_id", how="left",
                           validate="many_to_one") # would RAISE on a bad key
print("validated merge ok:", len(many_to_one), "rows")
## Other contracts: "one_to_one", "one_to_many", "many_to_many".
```

Output:

```
_merge
both      5
left_only 1
right_only 1
Name: count, dtype: int64

   cust_id name
5         4  Dee
validated merge ok: 6 rows
```

Business question: revenue per customer INCLUDING customers with zero orders.

```
revenue = (
    orders
    .merge(customers, on="cust_id", how="left") # keep ghost orders visible
    .assign(name=lambda d: d["name"].fillna("(unknown)")) # label the ghosts
    .groupby("name", as_index=False)["amount"].sum()
    .sort_values("amount", ascending=False)
)
print(revenue)

## Dee shows zero orders because SHE never appears in orders -- flip the merge
## (customers LEFT-join orders) when the customer list must drive the report:
per_customer = (
    customers
    .merge(orders, on="cust_id", how="left", suffixes=("", "_order"))
    .groupby("name")["amount"].sum().fillna(0) # NaN sum -> 0.0
)
print(per_customer)
```

Different column NAMES on each side? Use left_on/right_on.

```

staff = pd.DataFrame({"person_id": [1, 2], "who": ["Eve", "Finn"]})
shifts = pd.DataFrame({"worker": [2, 1], "hours": [6, 8]})
linked = shifts.merge(staff, left_on="worker", right_on="person_id")
print(linked[["who", "hours"]], "\n")

## join() = merge pre-wired to INDEXES; with on= it joins a column to an index.
## (drop orders' overlapping 'city' first -- join has no auto-suffixing like merge)
by_index_join = orders.join(
    customers.drop(columns="city").set_index("cust_id"), on="cust_id"
)
print("join(on=...) equals a left merge:",
      by_index_join["name"].isna().sum() == left_joined["name"].isna().sum())

```

Summary & key takeaways

- Detect missingness with `isna()` (never `== nan`); build a count/pct report before deciding anything.
- Drop with intent (`how=`, `subset=`, `thresh=`); fill by statistic chosen by skew (median for skewed, mean for symmetric, mode for categories); `ffill/bfill` for ordered data; **group by group medians beat one global median** (`groupby` + `transform`).
- Deduplicate with `duplicated()` then `drop_duplicates(subset, keep=)`.
- Convert dtypes aggressively but carefully: `astype("category")`, `pd.to_numeric(errors="coerce")`, `pd.to_datetime(format=...)`.
- Vectorize first; map for lookups; apply only when nothing faster fits; `df.applymap` is retired, use `df.map`.
- `pd.cut` for fixed width bins, `pd.qcut` for equal count bins.
- `GroupBy` = split apply combine; named aggregation (`avg=("fare", "mean")`) for readable summaries; `transform` for row aligned group stats; `filter` for keeping whole groups.
- Reshape freely: `melt/pivot` between long and wide, `stack/unstack` around MultiIndexes, `pivot_table/crosstab` for instant matrices.
- `concat` stacks; `merge(how=...)` joins relationally, lean on `indicator=True` and `validate=` to stay honest.

Next up: `03_pandas_advanced.nb.py`, time series, MultiIndex mastery, string superpowers, Copy on Write, and memory tuning.

4.3 Pandas, Advanced Pandas**MLCourse · Data Science Foundations · 02_pandas**

This is the power user notebook: time series, MultiIndex tables, production grade string handling, method chaining, and the memory/copy subtleties that separate beginners from professionals.

What you'll learn

- Working with dates: `to_datetime`, `.dt` accessors, `resample`, rolling windows
- Lag features, `% change`, and shift based feature engineering
- MultiIndex DataFrames: building, slicing with `xs`, stacking/unstacking
- Ordered categorical columns (and why they save memory)
- The full `.str` string methods suite including regex extraction
- Method chaining style + `assign` / `pipe`
- `where` / `mask`, `nlargest` / `nsmallest`
- The infamous `SettingWithCopyWarning`, what causes it, how to fix it
- Memory optimization and DataFrame styling

Standard imports for this notebook

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import io

np.random.seed(42)          # reproducibility for any random data we build
sns.set_theme()             # nicer plot defaults

print("pandas version:", pd.__version__)
```

1. Time Series, dates done right

Real world data arrives with timestamps. Pandas treats a column of datetimes as a *first class index*, unlocking label based date slicing, aggregation by period, and window statistics.

1.1 Parsing strings to datetimes

Raw data usually stores dates as STRINGS, useless for math until parsed.

```
raw_dates = pd.Series(["2024-01-15", "2024-02-20", "2024/03/05", "not-a-date", None])

## errors="coerce" turns unparseable values into NaT (Not a Time) instead of crashing.
## That failure count is itself diagnostic information about your data quality!
parsed = pd.to_datetime(raw_dates, format="mixed", errors="coerce")

comparison = pd.DataFrame({"raw": raw_dates, "parsed": parsed})
print(comparison)
```



```
print("\nCoerced (failed) values:", parsed.isna().sum())
```

Common pitfall: without `format=`, pandas guesses. It's convenient but slow on large files and occasionally wrong (03/05/2024, March 5th or May 3rd?). For known formats always pass an explicit one:

```
pd.to_datetime(s, format="%d/%m/%Y") # day-first, unambiguous
```

Generating regular date ranges, useful for synthetic series and reindexing.

```
daily = pd.date_range(start="2024-01-01", periods=8, freq="D") # 8 consecutive days
weekly = pd.date_range("2024-01-01", "2024-03-01", freq="W") # every week end

print(daily[:4])
print("\nFrequency aliases you'll use constantly:")
for alias, meaning in [("D", "calendar day"), ("B", "business day"), ("W", "weekly"),
                       ("MS", "month start"), ("QE", "quarter end"), ("YE", "year end")]
    :
    print(f" {alias:>3} -> {meaning}")
```

1.2 A datetime index unlocks date superpowers

Build a small daily sales series indexed by date.

```
dates = pd.date_range("2022-01-01", "2023-12-31", freq="D")
sales = pd.Series(
    1000
    + np.arange(len(dates)) * 0.5 # gentle upward trend
    + 120 * np.sin(2 * np.pi * dates.dayofyear / 365) # annual seasonality wave
    + np.random.normal(0, 40, len(dates)), # noise
    index=dates,
    name="sales",
).round(2)

sales.head(3)
```

PARTIAL STRING INDEXING, no more `dt.datetime` gymnastics.

```
## Selecting everything from one year:
print("2022 total sales:", sales.loc["2022"].sum().round(0))

## A single month:
print("Jan 2023 mean      :", round(sales.loc["2023-01"].mean(), 1))
```

```
## A range of dates (label slices are INCLUSIVE):
q1_2023 = sales.loc["2023-01-01":"2023-03-31"]
print("Q1 2023 days      :", len(q1_2023))
```

1.3 The .dt accessor, components of every timestamp

```
df_ts = sales.reset_index().rename(columns={"index": "date", "sales": "sales"})

## .dt gives every date part as vectorized columns - no Python loops needed.
df_ts["year"]      = df_ts["date"].dt.year
df_ts["month"]     = df_ts["date"].dt.month
df_ts["day"]       = df_ts["date"].dt.day
df_ts["dayofweek"] = df_ts["date"].dt.dayofweek      # Mon=0 ... Sun=6
df_ts["month_name"] = df_ts["date"].dt.month_name()
df_ts["is_weekend"] = df_ts["date"].dt.dayofweek >= 5

df_ts.sample(5, random_state=42)
```

Output:

	date	sales	year	month	day	dayofweek	month_name	is_weekend
468	2023-04-14	1329.90	2023	4	14	4	April	False
148	2022-05-29	1160.31	2022	5	29	6	May	True
302	2022-10-30	1075.78	2022	10	30	6	October	True
355	2022-12-22	1118.88	2022	12	22	3	December	False
515	2023-05-31	1349.64	2023	5	31	2	May	False

Pro tip: .dt also exposes business logic like `is_month_end`, `quarter`, and `days_in_month`. Tab complete on `df_ts["date"].dt.` to explore. These extracted features are the bread and butter of time series ML models.

1.4 Resampling, groupby for time

`resample()` is `groupby` where the groups are time buckets. Downsampling (daily to monthly) needs an aggregation; upsampling needs a fill strategy.

```
monthly = sales.resample("MS").mean() # MS = month start bucket; stable alias in every
pandas version
quarterly = sales.resample("QE").sum() # QE = quarter end

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(sales.index, sales.values, alpha=0.25, label="daily (noisy)")
ax.plot(monthly.index, monthly.values, lw=2, label="monthly mean (resampled)")
ax.set_title("Resampling smooths noise while preserving structure")
ax.set_ylabel("Sales")
```

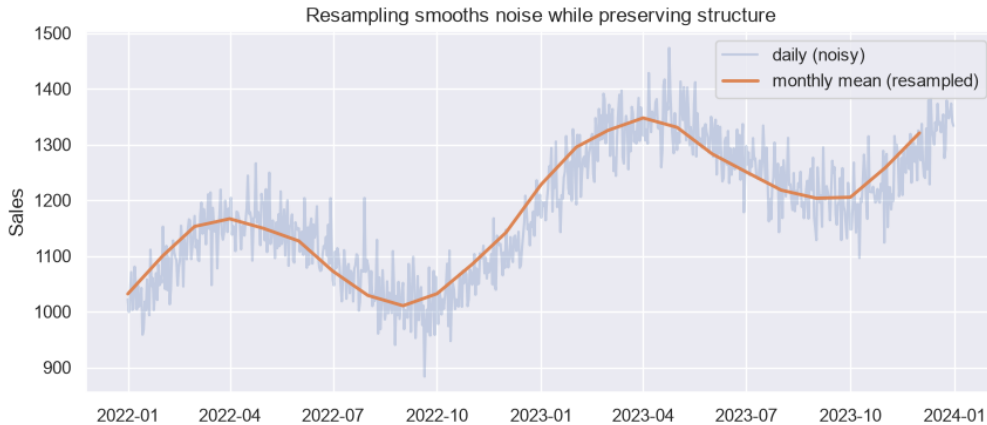


Figure 1: 1.4 Resampling, groupby for time (ch03_pandas_03 figure 1)

```
ax.legend()
plt.show()

print("Quarterly sums:")
print(quarterly.round(0).head())
```

Output:

```
Quarterly sums:
2022-03-31      98528.0
2022-06-30     104424.0
2022-09-30      95465.0
2022-12-31     99944.0
2023-03-31     115422.0
Freq: QE-DEC, Name: sales, dtype: float64
```

1.5 Rolling & expanding windows

```
## rolling(window).mean() answers: "what does the trend look like if I average
## over the last N observations?" - the classic smoothing tool.
roll17 = sales.rolling(window=7).mean()      # weekly moving average
roll30 = sales.rolling(window=30).mean()     # monthly moving average

## expanding() uses ALL history up to each point - cumulative view.
expanding_mean = sales.expanding().mean()

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(sales, alpha=0.2, label="daily")
ax.plot(roll17, lw=1.5, label="7-day MA")
```

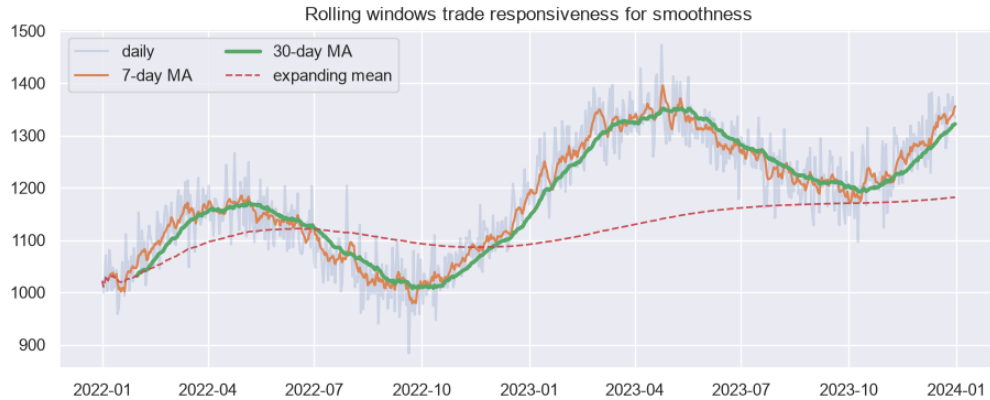


Figure 2: 1.5 Rolling & expanding windows (ch03_pandas_03 figure 2)

```
ax.plot(roll30, lw=2.5, label="30-day MA")
ax.plot(expanding_mean, ls="--", lw=1.2, label="expanding mean")
ax.set_title("Rolling windows trade responsiveness for smoothness")
ax.legend(ncols=2)
plt.show()
```

Common pitfall: the first window-1 values of a rolling result are NaN (there isn't enough history yet). Either accept them or pass `min_periods=1`.

1.6 Shift & pct_change, lag features in one line

```
small = sales.loc["2023-01":"2023-01-07"]

feat = pd.DataFrame({
    "sales": small,
    "yesterday": small.shift(1),
    "day_over_day": small.pct_change(),
    "delta": small.diff(),
}).round(2)

feat
```

Output:

```
sales  yesterday  day_over_day  delta
2023-01-01  1168.52          NaN      NaN      NaN
2023-01-02  1196.09      1168.52    0.02    27.57
2023-01-03  1190.20      1196.09   -0.00    -5.89
2023-01-04  1196.16      1190.20    0.01     5.96
```

2023-01-05	1163.90	1196.16	-0.03	-32.26
2023-01-06	1198.35	1163.90	0.03	34.45
2023-01-07	1219.84	1198.35	0.02	21.49

Pro tip: lag features (shift) are how you convert a forecasting problem into supervised learning: predict *today* from *yesterday*, *last week* (shift(7)), *same month last year* (shift(365))...

1.7 Worked mini analysis: the flights dataset

```
try:
    flights = sns.load_dataset("flights")
except Exception as e:
    print("Offline? Build fallback frame.", e)
    flights = pd.DataFrame()

if not flights.empty:
    # flights is LONG format: year | month | passengers (one row per month).
    # Construct a real datetime from parts - a pattern you will reuse forever.
    flights["date"] = pd.to_datetime(
        flights["year"].astype(str) + "-" + flights["month"].astype(str) + "-01"
    )
    fts = flights.set_index("date")["passengers"].sort_index()

    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(fts, label="monthly passengers")
    ax.plot(fts.rolling(12, center=True).mean(), lw=3, label="12-month centered trend")
    ax.set_title("Airline passengers 1949-1960: growth + seasonality")
    ax.legend()
    plt.show()

    # Year-over-year growth in two lines thanks to shift on a monthly series:
    yoy = fts.diff(12)  # same month, previous year
    print("YoY growth (last year shown):")
    print(yoy.loc["1959"].head(3))
```

Output:

```
YoY growth (last year shown):
date
1959-01-01    20.0
1959-02-01    24.0
1959-03-01    44.0
Name: passengers, dtype: float64
```

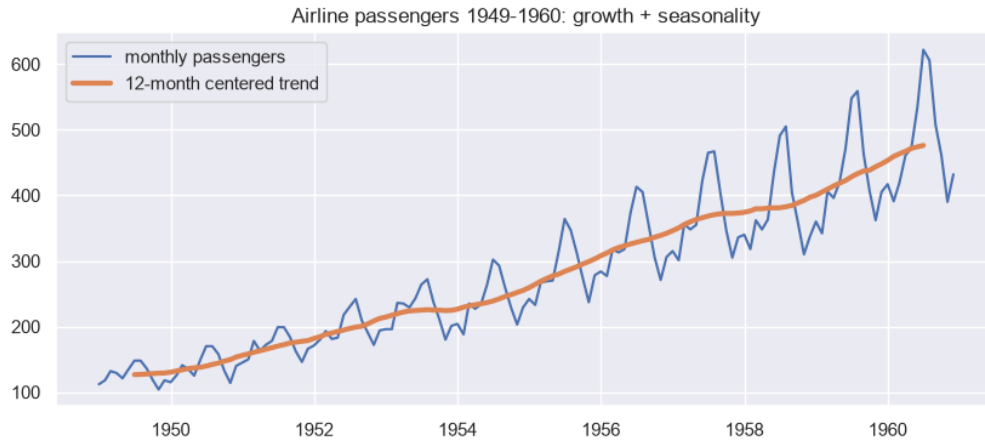


Figure 3: 1.7 Worked mini analysis: the flights dataset (ch03_pandas_03 figure 3)

2. MultiIndex, tables within tables

A MultiIndex lets one axis carry several levels of labels, think *“region city”* rows or *“metric quarter”* columns. Groupby with multiple keys produces them naturally.

Small HR style dataset

```
hr = pd.DataFrame({
    "region": ["East", "East", "West", "West", "East", "West"],
    "dept":   ["Sales", "Eng", "Sales", "Eng", "Eng", "Sales"],
    "quarter": ["Q1", "Q1", "Q1", "Q1", "Q2", "Q2"],
    "revenue": [220, 340, 190, 410, 260, 205],
})

## Grouping by TWO keys returns a Series with a 2-level row index.
multi = hr.groupby(["region", "dept"])["revenue"].sum()
print(multi)
print("\nIndex names:", multi.index.names)
```

Selecting from a MultiIndex:

```
## All levels except the last stay fixed -> .loc[outer, inner]
print("East/Sales revenue :", multi.loc[("East", "Sales")])

## Slice just the outer level -> every dept within East
print("\nAll East rows:\n", multi.loc["East"])
```

```
## xs(): cross-section - take a slice at a DEEPER level while keeping the shape.
print("\nxs('Sales', level='dept'):\n", multi.xs("Sales", level="dept"))
```

2.1 stack / unstack, reshaping between long and wide

```
wide = multi.unstack()      # innermost row level -> columns
print("unstack() -> wide:\n", wide)

back = wide.stack()         # and back again
print("\nstack() -> long (identical):\n", back.head(4))
```

Output:

```
unstack() -> wide:
dept    Eng  Sales
region
East    600    220
West    410    395

stack() -> long (identical):
region dept
East    Eng    600
        Sales  220
West    Eng    410
        Sales  395
dtype: int64
```

Pro tip: unstack is the fastest route from a grouped summary to a heatmap ready matrix: `multi.unstack()` then feed it straight into `sns.heatmap(...)`.

3. Ordered categoricals, categories that sort properly

```
sizes = pd.Series(["M", "S", "L", "XL", "M", "S", "L"])

## Plain object dtype sorts ALPHABETICALLY - semantically wrong for sizes!
print("object sort (wrong):", sorted(sizes.unique()))

## Declare category order ONCE and every sort/groupby respects it forever.
ordered_sizes = pd.Categorical(
    sizes,
    categories=["S", "M", "L", "XL"], # the TRUE order
    ordered=True,
)
size_series = pd.Series(ordered_sizes, name="size")

print("\ncategory sort (right):", list(size_series.sort_values().unique()))
```

```
## Comparisons become meaningful:
mask = size_series >= "L"
print("\nSizes >= L:", mask.tolist())
```

Output:

```
object sort (wrong): ['L', 'M', 'S', 'XL']

category sort (right): ['S', 'M', 'L', 'XL']

Sizes >= L: [False, False, True, True, False, False, True]
```

Memory win, object dtype stores a Python string per cell;

```
## category dtype stores integers + one lookup table.
n = 500_000
big_object = pd.Series(np.random.choice(["low", "mid", "high"], n))
big_category = big_object.astype("category")

print(f"object    : {big_object.memory_usage(deep=True)/1e6:.1f} MB")
print(f"category  : {big_category.memory_usage(deep=True)/1e6:.1f} MB")

## Renaming categories updates every value at once:
renamed = size_series.cat.rename_categories({"S": "Small", "M": "Medium", "L": "Large",
      "XL": "Extra Large"})
print("\nRenamed:", renamed.unique().tolist())
```

4. The .str suite, vectorized text surgery

Every string operation becomes an array operation via `.str`. Combined with regex it replaces most manual text munging loops.

```
people = pd.DataFrame({
    "email": ["Ana.Silva@Gmail.com ", " bob@test.org", "CARLA@mail.io", "dan@TEST.org "],
    "full_name": ["Silva, Ana", "Jones, Bob", "Lopez, Carla", "Nash, Dan"],
    "phone": ["555-0101", "555.0202", "555 0303", "5550404"],
})

people["email_clean"] = people["email"].str.strip().str.lower()          # trim + case
people["username"]    = people["email_clean"].str.split("@").str[0]      # split + index
people["domain"]      = people["email_clean"].str.extract(r"@(\w+)\.")    # regex capture
people["first_name"]  = people["full_name"].str.extract(r",\s*(?P<first>\w+)")["first"]
people["digits"]      = people["phone"].str.replace(r"\D", "", regex=True) # strip non-digits
```



```
people
```

Output:

```
email      full_name    phone      email_clean \
0  Ana.Silva@gmail.com    Silva, Ana  555-0101  ana.silva@gmail.com
1      bob@test.org      Jones, Bob  555.0202      bob@test.org
2      CARLA@mail.io    Lopez, Carla  555 0303      carla@mail.io
3      dan@TEST.org      Nash, Dan   5550404      dan@test.org

  username domain first_name  digits
0  ana.silva  gmail        Ana  5550101
1      bob    test         Bob  5550202
2     carla   mail        Carla  5550303
3      dan    test         Dan  5550404
```

Common pitfall: `.str.contains("a.b")` treats "." as ANY character. Escape literals: `\.`, or pass `regex=False` for plain substring search.

```
## str.get_dummies: turn a tag list into indicator columns - instant featurization!
tags = pd.DataFrame({"tags": ["python;pandas", "sql", "python;viz", "pandas;viz"]})
pd.concat([tags, tags["tags"].str.get_dummies(sep=";")], axis=1)
```

Output:

```
tags  pandas  python  sql  viz
0  python;pandas      1    1    0    0
1           sql      0    0    1    0
2   python;viz      0    1    0    1
3   pandas;viz      1    0    0    1
```

5. Method chaining, readable pipelines

Professionals write transformations as one fluent pipeline instead of a swamp of intermediate variables. `assign` adds columns, `query` filters, `pipe` injects any custom function.

```
def add_tip_pct(df):
    # custom step for pipe()
    return df.assign(tip_pct=df["tip"] / df["total_bill"] * 100)

try:
    tips = sns.load_dataset("tips")
except Exception:
    tips = pd.DataFrame({"total_bill": [16.99], "tip": [1.01], "sex": ["Female"]})

result = (
    tips
```

```
.query("total_bill < 40")           # 1. filter
.pipe(add_tip_pct)                  # 2. engineered column via function
.groupby("day", observed=True)["tip_pct"] # 3. aggregate per day
.mean()
.round(2)
.sort_values()
)
print(result)
```

Output:

```
day
Sat    15.36
Thur    16.27
Sun     17.03
Fri     17.28
Name: tip_pct, dtype: float64
```

Pro tip: wrap the chain in parentheses (as above) and put one step per line, that's the idiomatic pandas style you'll see in production codebases.

6. where / mask / nlargest, surgical selection

```
rng = np.random.default_rng(42)
scores = pd.Series(rng.normal(70, 15, 10).round(1), name="score")

keep_pass = scores.where(scores >= 60)           # keep passing, NaN the rest
clamp_low = scores.mask(scores < 60, other=60.0) # replace failures with floor

compare = pd.DataFrame({"orig": scores, "where(>=60)": keep_pass, "masked floor": clamp_
    low})
print(compare)

## Top/bottom-k without sorting everything:
print("\nTop 3 scores:", scores.nlargest(3).tolist())
print("Bottom 3      :", scores.nsmallest(3).tolist())

## Top-n PER GROUP - a very common interview task:
top_tipper_per_day = (
    tips.sort_values("tip", ascending=False)
    .groupby("day", observed=True)
    .head(2)[["day", "sex", "tip"]]
)
print("\nTop 2 tippers per day:")
print(top_tipper_per_day)
```

Output:

```

orig  where(>=60)  masked floor
0  74.6           74.6       74.6
1  54.4           NaN       60.0
2  81.3           81.3       81.3
3  84.1           84.1       84.1
4  40.7           NaN       60.0
5  50.5           NaN       60.0
6  71.9           71.9       71.9
7  65.3           65.3       65.3
8  69.7           69.7       69.7
9  57.2           NaN       60.0

```

```

Top 3 scores: [84.1, 81.3, 74.6]
Bottom 3      : [40.7, 50.5, 54.4]

```

```

Top 2 tippers per day:
   day  sex  tip
170 Sat  Male 10.00
212 Sat  Male  9.00
141 Thur Male  6.70
183 Sun  Male  6.50
 47 Sun  Male  6.00
 88 Thur Male  5.85
 95 Fri  Male  4.73
 93 Fri Female 4.30

```

7. SettingWithCopyWarning, views, copies, and the fix

Chained assignment writes through a temporary slice whose ownership pandas cannot guarantee. Understand it once and the warning never bothers you again.

```

data = pd.DataFrame({"a": [1, 2, 3, 4], "b": [10, 20, 30, 40]})

subset = data[data["a"] > 2]      # a FILTER - may be a view OR a copy
subset["b"] = 999                # <- triggers SettingWithCopyWarning!

print(data)                      # original MAY or MAY NOT change - undefined!

```

Output:

```

a  b
0  1 10
1  2 20
2  3 30
3  4 40

```

```
## THE FIX: make the copy explicit, then modify freely.
safe = data[data["a"] > 2].copy()    # .copy() severs the link
safe["b"] = 999
print(data)                        # untouched, guaranteed
print(safe)                        # modified exactly once

## Modern pandas makes this safe by default. Copy-on-Write (COW) became the
## permanent behaviour in pandas 3.0: every slice behaves like a lazy copy, and
## writing to it can never reach back into the parent frame. There is no switch
## to flip any more - the old `pd.set_option("mode.copy_on_write", ...)` toggle
## is a no-op on 3.x and was the opt-in preview on 1.5-2.x.
print("\npandas version:", pd.__version__, "- Copy-on-Write is always on from 3.0")
```

Output:

```
a  b
0  1  10
1  2  20
2  3  30
3  4  40
a  b
2  3  999
3  4  999

pandas version: 3.0.5 - Copy-on-Write is always on from 3.0
```

Common pitfall: silencing the warning with filters is not a fix, the underlying ambiguity remains. Rule of thumb: **filter immediately .copy()**.

8. Memory optimization at scale

Profile first: which columns dominate memory?

```
audit = titanic_like = sns.load_dataset("titanic") if True else pd.DataFrame()

mem_before = audit.memory_usage(deep=True).sum() / 1e6

opt = audit.copy()
## 1) Low-cardinality text -> category
for col in ["sex", "embarked", "class", "who", "deck", "embark_town", "alive"]:
    opt[col] = opt[col].astype("category")
## 2) Right-size numerics
opt["pclass"] = opt["pclass"].astype("int8")      # values 1..3 fit in one byte
opt["survived"] = opt["survived"].astype("int8")
opt["age"] = pd.to_numeric(opt["age"], downcast="float")    # float64 -> float32

mem_after = opt.memory_usage(deep=True).sum() / 1e6
```

```
print(f"memory: {mem_before:.2f} MB -> {mem_after:.2f} MB "
      f"({100*(1-mem_after/mem_before):.0f}% smaller)")

## select_dtypes helps target conversions precisely:
numeric_cols = opt.select_dtypes(include="number").columns.tolist()
cat_cols = opt.select_dtypes(include="category").columns.tolist()
print("numeric:", numeric_cols[:4], "... cats:", cat_cols[:4])
```

9. Styling DataFrames, reports inside notebooks

```
## df.style builds HTML representations - perfect for quick executive tables.
sample = (
    tips.groupby("day", observed=True)
        .agg(bills=("total_bill", "count"),
             avg_bill=("total_bill", "mean"),
             avg_tip=("tip", "mean"))
        .round(2)
)

styled = (
    sample.style
        .bar(subset=["bills"], color="#5fba7d")           # inline bars
        .background_gradient(subset=["avg_bill"], cmap="Blues")
        .highlight_max(subset=["avg_tip"], color="#ffd54f")
        .format({"avg_bill": "${:.2f}", "avg_tip": "${:.2f}"})
)
styled      # display it (rich output in Jupyter)
```

Output:

```
<pandas.io.formats.style.Styler at 0x119b8595690>
```

10. Exporting results

```
import tempfile, os
tmpdir = tempfile.mkdtemp()  # throwaway folder so this notebook stays self-contained

out_csv = os.path.join(tmpdir, "monthly_sales.csv")
monthly.rename_axis("month").reset_index().to_csv(out_csv, index=False)  # drop the index!

parquet_note = """
> **Pro tip:** for anything bigger than a quick share, prefer `to_parquet()` -
> it preserves dtypes (CSV flattens everything back to text!), compresses well,
> and reads dramatically faster.
"""
```

```
print(open(out_csv).readline()) # peek at the header line we wrote
print(parquet_note)
```

Output:

```
month,sales
```

```
> **Pro tip:** for anything bigger than a quick share, prefer `to_parquet()` -
> it preserves dtypes (CSV flattens everything back to text!), compresses well,
> and reads dramatically faster.
```

Summary & key takeaways

- Parse dates once with `to_datetime`; set them as the **index** to unlock partial string slicing (`df.loc["2023"]`), `resample`, and rolling windows.
- `resample = groupby` for time; rolling smooths; `shift` builds lag features; `pct_change/diff` quantify movement.
- `MultiIndex` appears whenever you `groupby` several keys, navigate with `.loc[(a, b)]` and `xs`, reshape with `stack/unstack`.
- `Ordered category dtype` fixes sorting semantics AND cuts memory drastically.
- `.str + regex` handles most text cleaning without loops; escape special chars.
- Prefer **method chaining**: `query assign groupby agg` in one readable block.
- `SettingWithCopyWarning` = ambiguous ownership. Filter then `.copy()` to say what you mean. From pandas 3.0 Copy on Write is always on, so a slice can never silently write back to its parent, no option to enable.
- Optimize memory with `category`, `int` downcasts, `float32`, often 50-80% smaller.
- `df.style` turns aggregates into presentation ready tables instantly.

4.4 Pandas, Exercises

MLCourse · Data Science Foundations · 02_pandas

Sixteen tasks across three difficulty tiers. They cover everything from the foundations through the advanced notebook: selection, filtering, missing data, `groupby`, reshaping, merging, time series, and code style.

Rules of the game 1. Solve in order, later tasks assume earlier skills. 2. Write pandas, not Python loops. 3. Comment *why*, not just what. 4. Only open `05_pandas_solutions.ipynb` after a genuine attempt.

Common imports + datasets for every exercise

```
%matplotlib inline
```

```

import pandas as pd
import numpy as np

np.random.seed(42)

def load(name):
    """Load a seaborn dataset with a friendly offline message."""
    try:
        import seaborn as sns
        return sns.load_dataset(name)
    except Exception as e:
        print(f"Could not load '{name}' ({e}) - connect to internet once, it caches.")
        return pd.DataFrame()

titanic = load("titanic")
tips = load("tips")
mpg = load("mpg")
flights = load("flights")

print(titanic.shape, tips.shape, mpg.shape, flights.shape)

```

Easy (1-5)

Task, First look

Using `titanic`: - Show the last 7 rows. - Print shape, column dtypes, and total memory usage (deep). - How many unique values does `embark_town` have?

your code here

Task, Column surgery

From `titanic` create a frame containing only: `survived`, `pclass`, `sex`, `age`, `fare`. Then rename `fare` `ticket_price` and drop any row where `sex` is missing.

your code here

Task, loc vs iloc drill

On `titanic`: 1. Select rows 10-19 (positional) and columns `sex` through `fare` (label slice). 2. Select the passenger at positional row 100, column `fare` using ONE method, then the same cell with the OTHER method.

your code here

Task, Simple filters

On tips: find all dinners (time == "Dinner") where the party had ≥ 4 people AND the tip was more than \$3. Return only total_bill, tip, size.

your code here

Task, Sorting

On mpg: sort by mpg descending but put missing values LAST. Show top 5 with columns name, mpg, horsepower.

your code here

Medium (6-11)

Task, Missing data strategy design

For titanic produce a missing value report (count + % per column). Then implement THREE different strategies: 1. Drop deck entirely (justify in a comment), 2. Fill age with the median, 3. Fill embarked/embark_town with the mode. Verify zero missing values remain (in the kept columns).

your code here

Task, Named aggregation report

Group titanic by class and compute in one shot: passengers count, survival rate (mean of survived), average fare, and median age, with clean snake_case column names via named aggregation.

your code here

Task, Pivot & proportion crosstab

1. Build a pivot table: index=day, columns=smoker (from tips), values mean total_bill, with row/column margins.
2. Build a crosstab of titanic class $\times$ survived normalized by row (i.e., survival rate within each class).

your code here

Task, Merge with a lookup table

Given the headquarters lookup below and mpg (whose origin column holds 'usa' / 'japan' / 'europe'), attach each car's HQ city via merge. Report average mpg per HQ.

origin	hq_city
usa	Detroit
japan	Tokyo
europa	Stuttgart

```
lookup = pd.DataFrame({"origin": ["usa", "japan", "europa"], "hq_city": ["Detroit", "Tokyo",  
"Stuttgart"]})
```

your code here

Task, Reshape wide long

1. Aggregate `flights` to a WIDE frame: `index=year`, `columns=month`, `values=passengers`.
2. Melt that wide frame back to long form and confirm it matches the original ordering logic (`year`, `month`, `passengers`).

your code here

Task, Time series basics on flights

Convert `flights` to a proper monthly datetime indexed Series of passengers. Compute and plot: raw series, 12 month rolling mean, and year over year % change (`pct_change(12)`).

your code here

Task, Ordered categories

Create an ordered categorical for `titanic.pclass` with order `3 < 2 < 1` (third < second < first class). Show that sorting now ranks FIRST class on top, then produce mean fare per ordered category.

your code here

Task, String extraction

Add a fake email column to a copy of `titanic`'s first 20 rows: `{name-free id}@ship.com` is NOT given, instead build emails from the INDEX: `passenger{i}@ship.com`. Then use `.str.extract` to pull the numeric part into an int column `pid` and verify `pid` equals the original index.

your code here

Hard (14-16)

Task, Top n per group, two ways

Find the 2 largest bills PER DAY in tips using: 1. `sort_values + drop_duplicates`, 2. `groupby().head(n)`. Confirm both return identical results.

your code here

Task, Binning + association

Bin `titanic.age` into quantile quartiles labeled Q1..Q4 (via `qcut`). Produce a crosstab of age quartile × survived showing SURVIVAL RATE per quartile. Which quartile survived most? Answer in a comment.

your code here

Task, De loop this code

Rewrite the loop version below as ONE chained, vectorized pipeline producing identical output (average fare by sex, only survivors over age 18, rounded):

```
out = {}
for s in titanic["sex"].unique():
    mask = (titanic["sex"] == s) & (titanic["survived"] == 1) & (titanic["age"] > 18)
    out[s] = round(titanic.loc[mask, "fare"].mean(), 2)
out
```

your code here

Self check before opening solutions

- Did you avoid `.apply(lambda ...)` where a vectorized op existed?
- Did every chained pipeline read like a sentence?
- Can you explain WHY each missing data strategy fits its column?

4.5 Pandas, Solutions

MLCourse · Data Science Foundations · 02_pandas

Fully worked, commented solutions for `04_pandas_exercises.ipynb`. Compare approach AND style, often several valid answers exist.

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

def load(name):
    try:
        import seaborn as sns
        return sns.load_dataset(name)
    except Exception as e:
        print(f"offline: {e}")
        return pd.DataFrame()

titanic = load("titanic")
tips = load("tips")
mpg = load("mpg")
flights = load("flights")
```

Easy

Task, First look

```
print(titanic.tail(7)) # last 7 rows
print("shape:", titanic.shape)
print(titanic.dtypes) # column dtypes at a glance
print(f"deep memory: {titanic.memory_usage(deep=True).sum()/1024:.0f} KB")
print("unique embark_town:", titanic["embark_town"].nunique()) # NaN excluded by default
t
```

Output:

survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
884	0	3	male	25.0	0	0	7.050	S Third
885	0	3	female	39.0	0	5	29.125	Q Third
886	0	2	male	27.0	0	0	13.000	S Second
887	1	1	female	19.0	0	0	30.000	S First
888	0	3	female	NaN	1	2	23.450	S Third
889	1	1	male	26.0	0	0	30.000	C First
890	0	3	male	32.0	0	0	7.750	Q Third

	who	adult_male	deck	embark_town	alive	alone
884	man	True	NaN	Southampton	no	True
885	woman	False	NaN	Queenstown	no	False
886	man	True	NaN	Southampton	no	True
887	woman	False	B	Southampton	yes	True

```
888  woman      False NaN Southampton    no False
889   man       True   C   Cherbourg    yes  True
890   man       True NaN   Queenstown    no  True
shape: (891, 15)
survived      int64
pclass        int64
sex           str
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      str
class         category
who           str
adult_male    bool
deck          category
embark_town   str
alive         str
alone         bool
dtype: object
deep memory: 314 KB
unique embark_town: 3
```

Task, Column surgery

```
slim = (
    titanic[["survived", "pclass", "sex", "age", "fare"]] # 1. project columns
    .rename(columns={"fare": "ticket_price"})           # 2. rename one
    .dropna(subset=["sex"])                             # 3. filter rows
)
print(slim.shape, "- sex had no missing values here, so shape is unchanged.")
```

Output:

```
(891, 5) - sex had no missing values here, so shape is unchanged.
```

Task, loc vs iloc drill

```
## 1) positions 10..19 for labels sex..fare:
part_a = titanic.iloc[10:20].loc[:, "sex":"fare"] # iloc picks ROWS positionally,
                                                    # then .loc slices columns by label.
## (Equivalent single-shot: titanic.loc[10:19, "sex":"fare"] works here only because
## the default index happens to equal row positions - fragile in general!)

## 2) same cell, two ways:
cell_iloc = titanic.iloc[100, titanic.columns.get_loc("fare")] # iat = fast scalar iloc
```

```
cell_loc = titanic.at[100, "fare"] # at = fast scalar loc
print(cell_loc, cell_loc, cell_loc == cell_loc)
```

Output:

```
7.8958 7.8958 True
```

Task, Simple filters

```
big_dinner_tips = (
    tips.loc[
        (tips["time"] == "Dinner") & (tips["size"] >= 4) & (tips["tip"] > 3),
        ["total_bill", "tip", "size"],
    ]
)
print(len(big_dinner_tips), "rows match.")
big_dinner_tips.head()
```

Output:

```
24 rows match.
```

Output:

	total_bill	tip	size	
4	24.59	3.61	4	
5	25.29	4.71	4	
7	26.88	3.12	4	
11	35.26	5.00	4	
23	39.42	7.58	4	

Note the parentheses around every condition, & binds tighter than comparisons without them.

Task, Sorting

```
top5 = (
    mpg.sort_values("mpg", ascending=False, na_position="last") # NaN sinks to bottom
    .loc[:, ["name", "mpg", "horsepower"]]
    .head(5)
)
top5
```

Output:

```
name    mpg  horsepower
322             mazda glc  46.6      65.0
329  honda civic 1500 gl  44.6      67.0
325  vw rabbit c (diesel) 44.3      48.0
394             vw pickup 44.0      52.0
326  vw dasher (diesel) 43.4      48.0
```

Medium

Task, Missing data strategy design

report

```
report = pd.DataFrame({
    "n_missing": titanic.isna().sum(),
    "pct_missing": (titanic.isna().mean() * 100).round(1),
}).query("n_missing > 0").sort_values("pct_missing", ascending=False)
print(report)

cleaned = titanic.copy()

## 1) DROP deck: ~77% missing -> imputation would fabricate most of the column;
## the little signal it holds isn't worth it. Dropping the COLUMN not rows!
cleaned = cleaned.drop(columns="deck")

## 2) age -> median: age is right-skewed-ish with outliers; median is robust,
## and we preserve every passenger record.
cleaned["age"] = cleaned["age"].fillna(cleaned["age"].median())

## 3) embarked/embark_town -> mode: categorical with just 2 missing cells -
## mode ('S' / 'Southampton') is the least-surprising fill.
for col in ["embarked", "embark_town"]:
    cleaned[col] = cleaned[col].fillna(cleaned[col].mode()[0])

assert cleaned[["age", "embarked", "embark_town"]].isna().sum().sum() == 0
print("\nRemaining missing anywhere:", cleaned.isna().sum().sum(), "(should be 0)")
```

Task, Named aggregation report

```
by_class = (
    titanic.groupby("class", observed=True)
    .agg(
        passengers=("survived", "size"),          # size counts rows incl. NaN surv
        ived? no - use any col
        survival_rate=("survived", "mean"),
    )
```

```

        avg_fare=("fare", "mean"),
        median_age=("age", "median"),
    )
    .round({"survival_rate": 3, "avg_fare": 2})
)
by_class

```

Output:

```

passengers  survival_rate  avg_fare  median_age
class
First        216          0.630    84.15      37.0
Second       184          0.473    20.66      29.0
Third        491          0.242    13.68      24.0

```

Task, Pivot & proportion crosstab

```

pivot = pd.pivot_table(
    tips, index="day", columns="smoker",
    values="total_bill", aggfunc="mean",
    margins=True, margins_name="All",          # totals on both axes
)
print(pivot.round(2))

ct = pd.crosstab(titanic["class"], titanic["survived"], normalize="index").round(3)
ct.columns = ["died_rate", "survived_rate"]
ct

```

Output:

```

smoker   Yes    No   All
day
Thur    19.19  17.11  17.68
Fri     16.81  18.42  17.15
Sat     21.28  19.66  20.44
Sun     24.12  20.51  21.41
All     20.76  19.19  19.79

```

Output:

```

died_rate  survived_rate
class
First      0.370         0.630
Second     0.527         0.473
Third      0.758         0.242

```

Task, Merge with a lookup table

```
lookup = pd.DataFrame({"origin": ["usa", "japan", "europe"],
                       "hq_city": ["Detroit", "Tokyo", "Stuttgart"]})

merged = mpg.merge(lookup, on="origin", how="left")  # left keeps all cars even if key
missing
hq_mpg = merged.groupby("hq_city")["mpg"].mean().round(2).sort_values()
print(hq_mpg)
merged[["name", "origin", "hq_city"]].head(3)
```

Output:

```
hq_city
Detroit      20.08
Stuttgart    27.89
Tokyo        30.45
Name: mpg, dtype: float64
```

Output:

```
name origin hq_city
0  chevrolet chevelle malibu    usa  Detroit
1      buick skylark 320      usa  Detroit
2    plymouth satellite    usa  Detroit
```

Task, Reshape wide long

```
wide = flights.pivot(index="year", columns="month", values="passengers")
print(wide.iloc[:3, :4])  # peek: years down, months across

long_back = (
    wide.rename_axis("year")  # keep the year label named
        .reset_index()  # year back to a normal column
        .melt(id_vars="year", var_name="month", value_name="passengers")
)
same = long_back.merge(flights, on=["year", "month", "passengers"], how="outer", indicat
or=True)
print((same["_merge"] == "both").all(), "-> melted frame matches original")
```

Output:

```
month  Jan  Feb  Mar  Apr
year
1949   112  118  132  129
1950   115  126  141  135
```

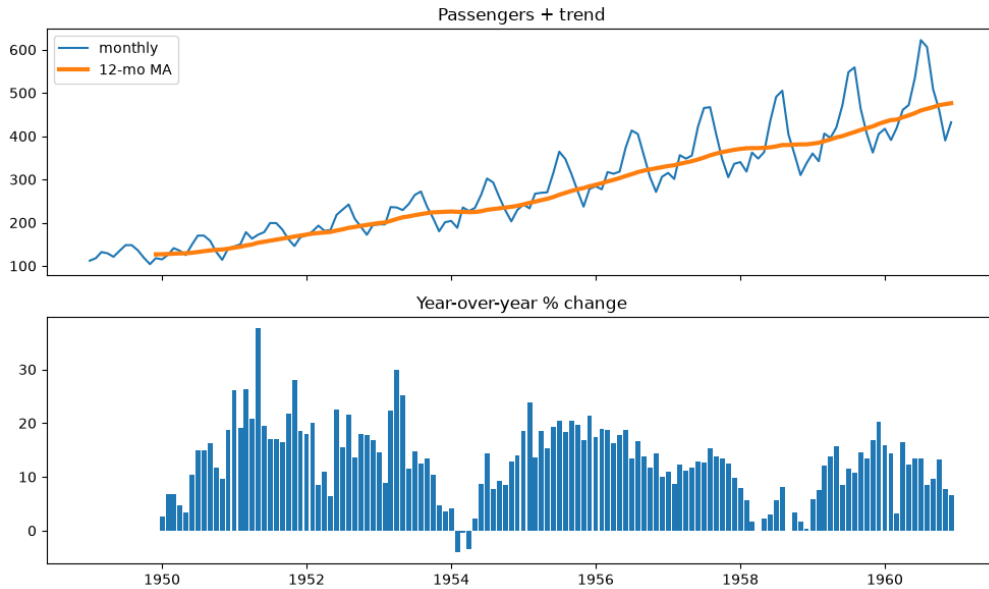



Figure 4: Task 11, Time series basics on flights (ch03_pandas_05 figure 1)

```
1951 145 150 178 163
True -> melted frame matches original
```

Task, Time series basics on flights

```
fts = (
    flights.assign(date=pd.to_datetime(flights["year"].astype(str)
                                       + "-" + flights["month"].astype(str) + "-01"))
    .set_index("date")
    .sort_index()["passengers"]
)

roll12 = fts.rolling(12).mean()
yoy_pct = fts.pct_change(12) * 100          # vs same month last year

fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
axes[0].plot(fts, label="monthly"); axes[0].plot(roll12, lw=3, label="12-mo MA")
axes[0].set_title("Passengers + trend"); axes[0].legend()
axes[1].bar(yoy_pct.index, yoy_pct.values, width=25)
axes[1].set_title("Year-over-year % change")
fig.tight_layout(); plt.show()
```

Task, Ordered categories

```
pclass_ordered = pd.Categorical(
    titanic["pclass"],
    categories=[3, 2, 1],          # TRUE social order: third < second < first
    ordered=True,
)
pc = pd.Series(pclass_ordered, name="pclass_ord")

print(pc.sort_values().head(3).tolist(), "-> smallest category first means class 3 first")
print(pd.Series(pc.sort_values().unique()).tolist())

mean_fare = titanic.assign(pc=pc).groupby("pc", observed=True)["fare"].mean().round(2)
print(mean_fare)
```

Output:

```
[3, 3, 3] -> smallest category first means class 3 first
[3, 2, 1]
pc
3    13.68
2    20.66
1    84.15
Name: fare, dtype: float64
```

Task, String extraction

```
mini = titanic.head(20).copy()
mini["email"] = "passenger" + mini.index.astype(str) + "@ship.com" # build from index

extracted = mini["email"].str.extract(r"passenger(?:P<pid>\d+)@") # named capture group
mini["pid"] = extracted["pid"].astype(int)

print((mini["pid"] == mini.index).all(), "-> pid reconstructs the index exactly")
```

Output:

```
True -> pid reconstructs the index exactly
```

Hard

Task, Top n per group, two ways

```
way1 = (
```

```

tips.sort_values("total_bill", ascending=False, kind="mergesort")
    .drop_duplicates(subset="day")           # first row per day = day's max
    [["day", "total_bill"]]
    .sort_values("day", kind="mergesort")    # stable sort -> deterministic order
)

way2 = (
    tips.sort_values("total_bill", ascending=False, kind="mergesort")
        .groupby("day", observed=True)
        .head(1)[["day", "total_bill"]]      # head(2) would give top-2 per group
        .sort_values("day", kind="mergesort")
)

pd.testing.assert_frame_equal(way1.reset_index(drop=True), way2.reset_index(drop=True))
print("identical \n", way1)

```

Output:

```

identical
   day  total_bill
197  Thur      43.11
 95  Fri       40.17
170  Sat       50.81
156  Sun       48.17

```

For $n > 1$, `drop_duplicates` needs a rank helper; `groupby().head(n)` handles it natively, that's why method 2 generalizes better.

Task, Binning + association

```

ages = titanic.dropna(subset=["age"]).copy()
ages["age_q"] = pd.qcut(ages["age"], q=4, labels=["Q1_youngest", "Q2", "Q3", "Q4_oldest"]
)

rate = pd.crosstab(ages["age_q"], ages["survived"], normalize="index")[1].rename("survived_rate")
print(rate.round(3))
best = rate.idxmax()
print(f"\nHighest survival: {best} ({rate.max():.1%})")
## Insight: children & young adults (Q1/Q2) survived most - 'women and children first'
## shows up directly in the numbers; elderly quartile fares worst.

```

Output:

```

age_q
Q1_youngest    0.458
Q2              0.361

```

```
Q3          0.434
Q4_oldest   0.373
Name: survived_rate, dtype: float64

Highest survival: Q1_youngest (45.8%)
```

Task, De loop this code

```
## The ORIGINAL loop version (kept for comparison):
loop_out = {}
for s in titanic["sex"].unique():
    mask = (titanic["sex"] == s) & (titanic["survived"] == 1) & (titanic["age"] > 18)
    loop_out[s] = round(titanic.loc[mask, "fare"].mean(), 2)

out = (
    titanic.query("survived == 1 and age > 18") # declarative filtering
        .groupby("sex", observed=True)["fare"]
        .mean()
        .round(2)
)
print("vectorized:", out.to_dict())
assert out.to_dict() == loop_out # same numbers, zero loops
```

Output:

```
vectorized: {'female': 60.06, 'male': 46.36}
```

Chapter 5

Matplotlib

A number in a table proves a fact; a plot lets a reader *see* it. Matplotlib is the foundation every other Python plotting library, including Seaborn in the next chapter, is built on, and understanding its Figure/Axes object model pays off every time you need a chart that a higher level library won't quite produce out of the box. This chapter covers the core plot types, styling, subplots, and enough of the object oriented API that you can build any static chart from first principles.

5.1 Matplotlib, Foundations

MLCourse · Data Science Foundations · 03_matplotlib

Matplotlib turns arrays into pictures. This notebook builds your mental model of HOW it thinks (figures, axes, artists), then walks through every core chart type you will use daily.

What you'll learn

- Figure vs Axes vs Axis, the anatomy of every plot
- Line plots with full styling control (colors, dashes, markers)
- Scatter, bar (grouped & stacked), histogram, pie, box plots
- Titles, labels, legends, grids, limits and ticks
- Figure sizing, layout engines, and saving publication quality files

```
%matplotlib inline
import matplotlib.pyplot as plt  # the plotting interface
import numpy as np              # data generation
import pandas as pd             # tabular convenience

np.random.seed(42)              # reproducible "random" data

plt.rcParams["figure.dpi"] = 100 # slightly crisper inline figures
```

1. Anatomy, how Matplotlib sees a plot

Two ideas explain 90% of Matplotlib:

Object	What it is
Figure	the whole canvas/window (can hold many Axes)
Axes	ONE plot area with its own x/y axis (despite the name, it's <i>the plot</i>)
Axis	the number line objects (ax.xaxis, ax.yaxis) that draw ticks
Artist	everything visible: lines, text, ticks, legends...

Draw an annotated diagram of the anatomy itself:

```
fig = plt.figure(figsize=(7, 5))
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])          # manual placement: [left,bottom,w,h]

x = np.linspace(0, 10, 100)
ax.plot(x, np.sin(x))

## Point at each anatomical part with arrows:
ax.annotate("Figure = canvas", xy=(0.02, 0.97), xycoords="axes fraction",
            fontsize=11, color="gray")
ax.annotate("Title", xy=(0.5, 1.02), xytext=(0.35, 1.18), xycoords="axes fraction",
            textcoords="axes fraction", arrowprops=dict(arrowstyle="->"))
ax.annotate("y-axis (Axis object)", xy=(-0.02, 0.5), xytext=(-0.28, 0.75),
            xycoords="axes fraction", textcoords="axes fraction",
            arrowprops=dict(arrowstyle="->"))
ax.annotate("x-axis (Axis object)", xy=(0.5, -0.04), xytext=(0.55, -0.22),
            xycoords="axes fraction", textcoords="axes fraction",
            arrowprops=dict(arrowstyle="->"))
ax.annotate("spine (border line)", xy=(0.995, 0.5), xytext=(0.72, 0.85),
            xycoords="axes fraction", textcoords="axes fraction",
            arrowprops=dict(arrowstyle="->"))
ax.set_title("Anatomy of a plot")
fig.subplots_adjust(top=0.82, left=0.24, bottom=0.22)  # make room for labels
plt.show()
```

Pro tip: remember it as “a Figure holds one or more Axes; each Axes has two Axis objects”. Almost every confusion in Matplotlib comes from mixing up Axes (the plot) with Axis (the number line).

2. Your first plot, and what pyplot does

```
xs = np.linspace(0, 2 * np.pi, 200)          # 200 points from 0 to 2π

plt.plot(xs, np.sin(xs))                      # pyplot finds-or-creates figure+axes for you
plt.title("y = sin(x)")
```

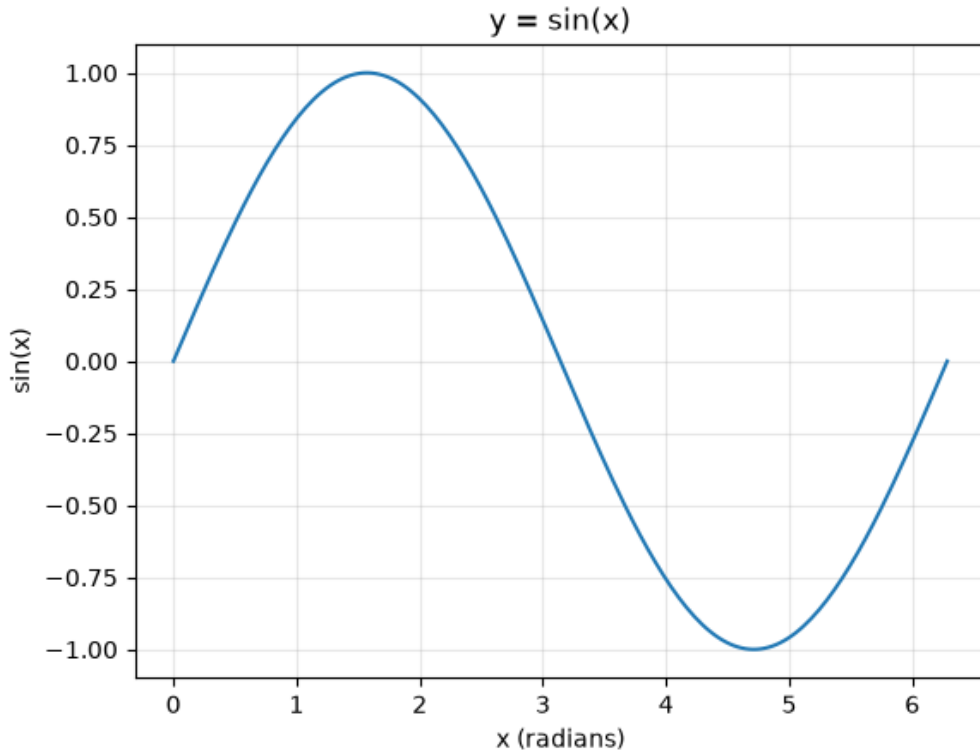


Figure 5: 2. Your first plot, and what pyplot does (ch04_matplotlib_01 figure 1)

```
plt.xlabel("x (radians)")
plt.ylabel("sin(x)")
plt.grid(alpha=0.3)           # subtle grid helps read values
plt.show()                   # renders and closes the loop

## `pyplot` keeps a hidden "current figure" - convenient for quick exploration,
## but we will graduate to explicit Axes below for anything serious.
```

3. Line plots, full styling control

Every visual property is a parameter: color, linestyle, linewidth, marker, alpha.

```
t = np.linspace(0, 4 * np.pi, 300)

fig, ax = plt.subplots(figsize=(9, 4))      # THE idiom: returns Figure + one Axes
ax.plot(t, np.sin(t), color="steelblue", lw=2, label="sin")
```

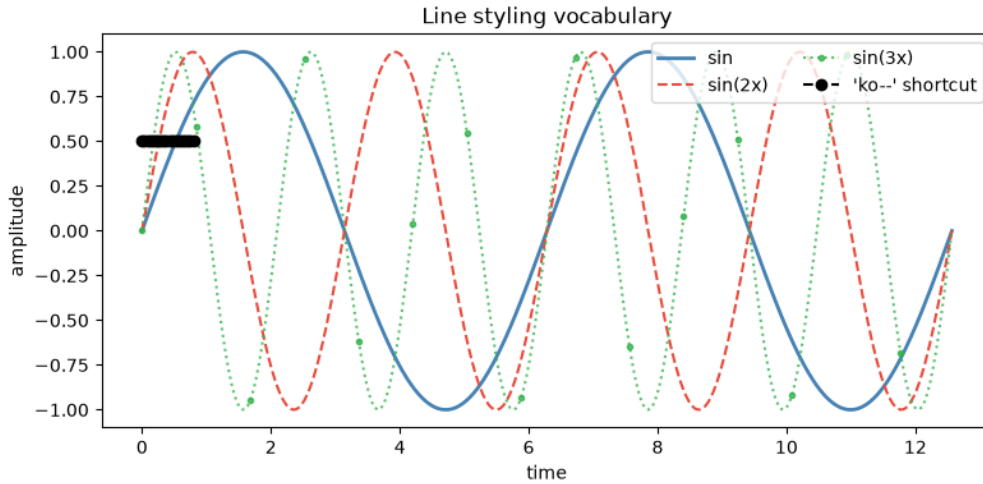


Figure 6: 3. Line plots, full styling control (ch04_matplotlib_01 figure 2)

```
ax.plot(t, np.sin(2*t), color="#e74c3c",          # hex colors work anywhere
        ls="--", lw=1.5, label="sin(2x)")
ax.plot(t, np.sin(3*t), color=(0.2, 0.7, 0.3),    # RGB tuple, 0-1 floats
        ls=":", marker="o", ms=3, markevery=20,   # dotted + marker every 20th point
        alpha=0.8, label="sin(3x)")
ax.plot(t[:20], t[:20] ** 0 / 2, "ko--",          # fmt shortcut string: marker+color+line
        label="'ko--' shortcut")                  # k=black, o=circle, --=dashed

ax.legend(loc="upper right", ncol=2, frameon=True) # 2 columns, framed box
ax.set_title("Line styling vocabulary")
ax.set_xlabel("time")
ax.set_ylabel("amplitude")
plt.show()
```

Style cheat sheet: colors "steelblue", "#e74c3c", (r,g,b) · line styles - solid, -- dashed, : dotted, -. dashdot · markers o s ^ * D · shortcut strings "[marker][color][line]" e.g. "ro-".

4. Scatter plots, relationships between two variables

```
n = 250
area = np.random.uniform(30, 800, n)    # bubble sizes
score = np.random.uniform(0, 10, n)     # drives color

fig, ax = plt.subplots(figsize=(8, 5))
sc = ax.scatter(
    area, score,
    s=area / 3,                                # size per point (points²)
```

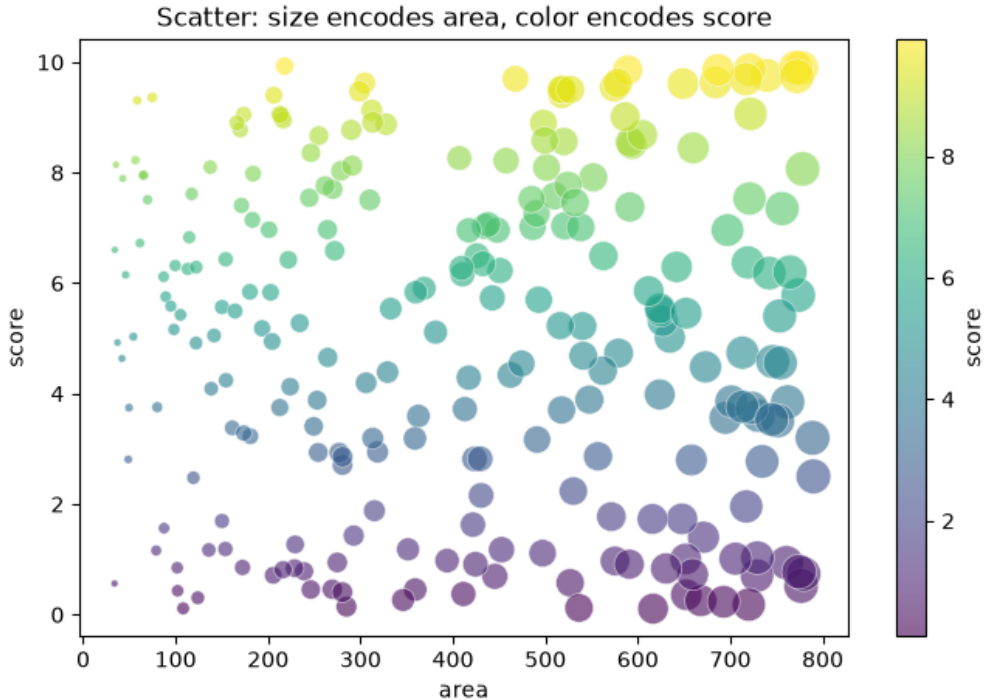



Figure 7: 4. Scatter plots, relationships between two variables (ch04_matplotlib_01 figure 3)

```
c=score,                                # color mapped from data...
cmap="viridis",                          # ...through this colormap
alpha=0.6,                              # transparency reveals density
edgecolors="white", linewidths=0.5,
)
fig.colorbar(sc, label="score")           # legend for the color mapping
ax.set_title("Scatter: size encodes area, color encodes score")
ax.set_xlabel("area")
ax.set_ylabel("score")
plt.show()
```

Common pitfall: without `alpha`, overlapping points hide thousands of observations behind one dot. When scatter looks like a blob, add transparency or switch to a 2 D histogram (next notebook).

5. Bar charts, comparing categories

```
categories = ["Python", "JS", "Java", "C++", "Rust"]
pop_2022 = [29.9, 17.8, 16.5, 11.9, 4.3]
```

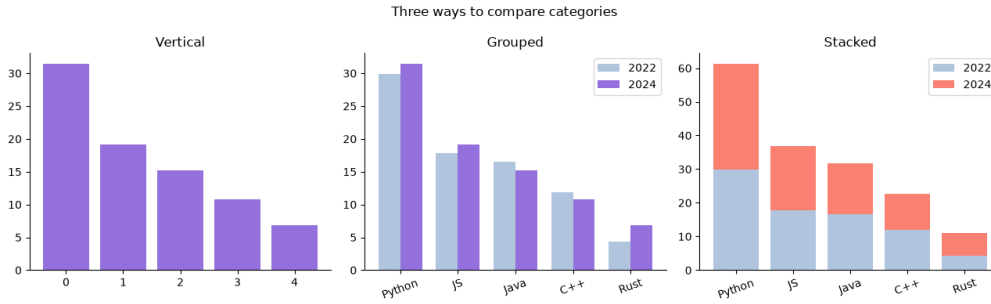


Figure 8: 5. Bar charts, comparing categories (ch04_matplotlib_01 figure 4)

```
pop_2024 = [31.5, 19.1, 15.2, 10.8, 6.8]

xpos = np.arange(len(categories))          # numeric positions for categories

fig, axes = plt.subplots(1, 3, figsize=(13, 4))

## --- vertical bars ---
axes[0].bar(xpos, pop_2024, color="mediumpurple")
axes[0].set_title("Vertical")

## --- GROUPED: offset each year's bars around the category position ---
w = 0.38                                  # bar width
axes[1].bar(xpos - w/2, pop_2022, w, label="2022", color="lightsteelblue")
axes[1].bar(xpos + w/2, pop_2024, w, label="2024", color="mediumpurple")
axes[1].set_xticks(xpos, categories, rotation=20)
axes[1].legend()
axes[1].set_title("Grouped")

## --- STACKED: second series starts where first ends (bottom=) ---
axes[2].bar(xpos, pop_2022, label="2022", color="lightsteelblue")
axes[2].bar(xpos, pop_2024, bottom=pop_2022, label="2024", color="salmon")
axes[2].set_xticks(xpos, categories, rotation=20)
axes[2].legend()
axes[2].set_title("Stacked")

for ax in axes:
    ax.spines[["top", "right"]].set_visible(False)    # cleaner look
fig.suptitle("Three ways to compare categories")
fig.tight_layout()
plt.show()
```

6. Histograms, the shape of ONE variable

```
samples = np.random.normal(loc=100, scale=15, size=5000)
```

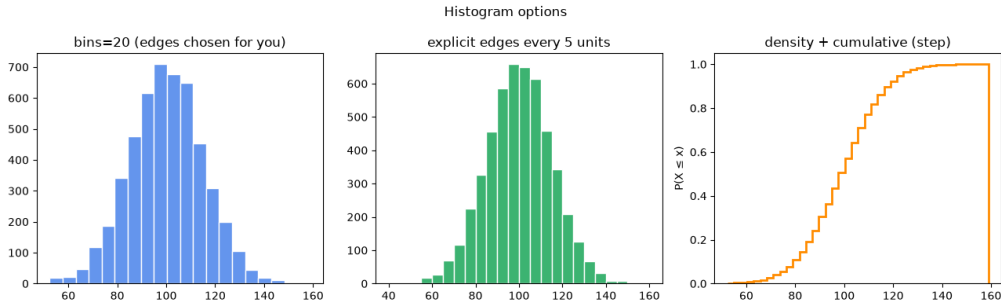


Figure 9: 6. Histograms, the shape of ONE variable (ch04_matplotlib_01 figure 5)

```
fig, axes = plt.subplots(1, 3, figsize=(13, 4))

axes[0].hist(samples, bins=20, color="cornflowerblue", edgecolor="white")
axes[0].set_title(f"bins=20 (edges chosen for you)")

axes[1].hist(samples, bins=np.arange(40, 165, 5), # explicit bin edges
              color="mediumseagreen", edgecolor="white")
axes[1].set_title("explicit edges every 5 units")

## density=True converts counts -> probability density (integrates to 1);
## cumulative=True flips the story to 'how many BELOW this value'.
axes[2].hist(samples, bins=40, density=True, cumulative=True,
              histtype="step", lw=2, color="darkorange")
axes[2].set_title("density + cumulative (step)")
axes[2].set_ylabel("P(X ≤ x)")

fig.suptitle("Histogram options")
fig.tight_layout()
plt.show()
```

Pro tip: always draw histograms with `edgecolor="white"`, adjacent bins become visually separable instead of merging into one block.

7. Pie charts, parts of a whole (with honesty rules)

```
shares = [45, 25, 15, 10, 5]
labels = ["Chrome", "Safari", "Edge", "Firefox", "Other"]
explode = [0.05, 0, 0, 0, 0] # pull Chrome out slightly

fig, ax = plt.subplots(figsize=(6, 6))
ax.pie(
    shares,
    labels=labels,
```

```

    autopct="%1.1f%%",                # format percentages on slices
    explode=explode,
    startangle=90,                    # rotate so first slice starts at top
    wedgeprops=dict(width=0.42, edgecolor="white"), # width < 1 => DONUT
)
ax.set_title("Browser share - donut style")
plt.show()

## Honest critique: humans judge ANGLES poorly. Use bars when precise comparison
## matters; keep pies to ≤5 slices with clearly dominant values.

```

8. Box plots, five number summaries & outliers

```

groups = [np.random.normal(mu, 8, 120) for mu in (60, 68, 74)]
names = ["Team A", "Team B", "Team C"]

fig, ax = plt.subplots(figsize=(7, 4.5))
bp = ax.boxplot(
    groups,
    tick_labels=names,                # matplotlib ≥3.9 name; older: labels=
    patch_artist=True,                # allows filling boxes with color
    medianprops=dict(color="black", lw=2),
)
for patch, color in zip(bp["boxes"], ["#aec7e8", "#98df8a", "#ffbb78"]):
    patch.set_facecolor(color)

ax.axhline(65, color="red", ls="--", lw=1, alpha=0.6) # target reference line
ax.text(3.35, 65.4, "target", color="red", fontsize=9)
ax.set_title("Score distributions per team\n(box=IQR, line=median, whiskers=1.5·IQR, dot
    s=outliers)")
plt.show()

```

9. Labels, titles, grids, limits, ticks, polish toolkit

```

fig, ax = plt.subplots(figsize=(9, 4.5))
days = pd.date_range("2024-06-01", periods=30)
temp = 22 + 6 * np.sin(np.arange(30) / 30 * 2 * np.pi) + np.random.normal(0, 1.5, 30)

ax.plot(days, temp, color="tomato", lw=2)
ax.fill_between(days, temp - 2, temp + 2, color="tomato", alpha=0.15) # uncertainty band

ax.set_title("June daily temperature", fontsize=14, fontweight="bold", loc="left")
ax.set_xlabel("date", fontsize=11)
ax.set_ylabel("°C", fontsize=11)

```

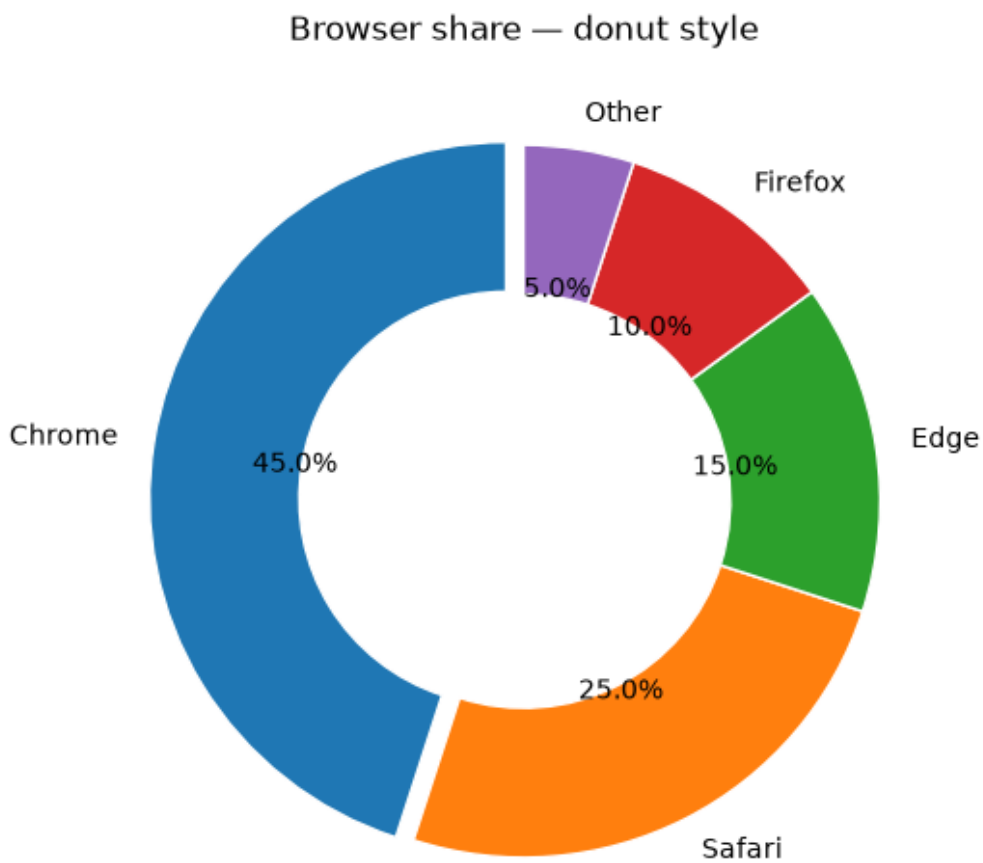


Figure 10: 7. Pie charts, parts of a whole (with honesty rules) (ch04_matplotlib_01 figure 6)

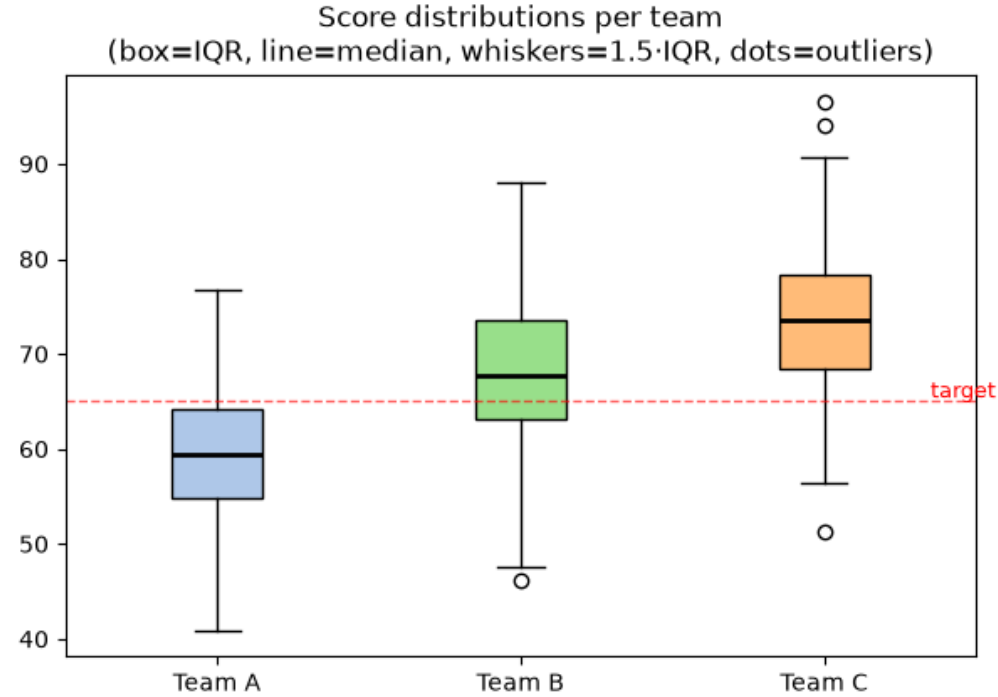


Figure 11: 8. Box plots, five number summaries & outliers (ch04_matplotlib_01 figure 7)

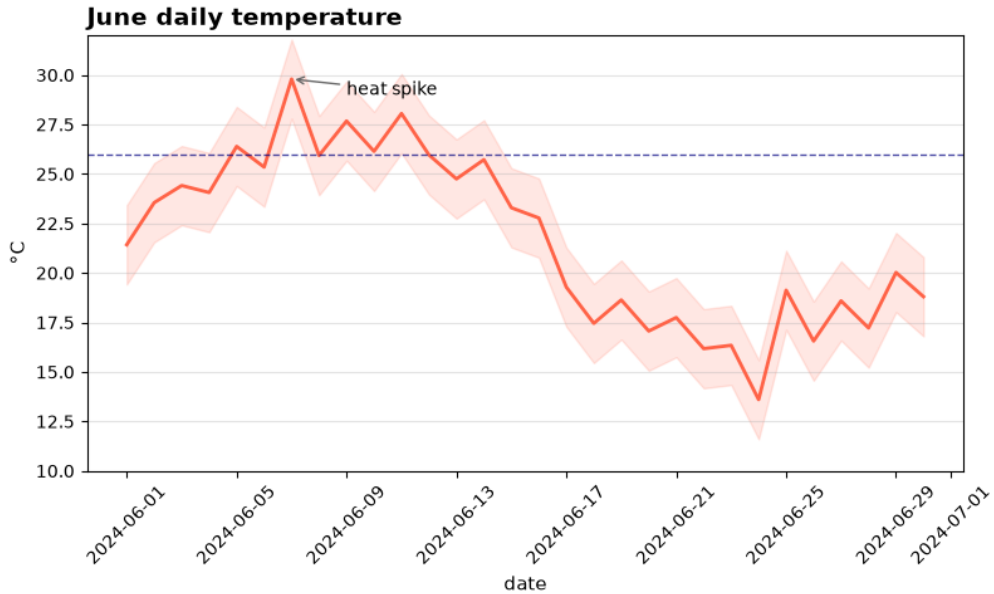


Figure 12: 9. Labels, titles, grids, limits, ticks, polish toolkit (ch04_matplotlib_01 figure 8)

```
ax.set_ylim(10, 32)                                # force y-range
ax.grid(axis="y", which="major", alpha=0.35)        # horizontal grid only
ax.tick_params(axis="x", rotation=45)               # readable date labels

ax.axhline(26, color="navy", ls="--", lw=1, alpha=0.7)
ax.annotate("heat spike", xy=(days[np.argmax(temp)], temp.max()),
            xytext=(days[8], 29),
            arrowprops=dict(arrowstyle="->", color="dimgray"))
plt.show()
```

10. Figure control & saving

```
## figsize is in INCHES; dpi multiplies pixels. 8x4 @150dpi = 1200x600 px.
fig, ax = plt.subplots(figsize=(8, 4), constrained_layout=True) # auto-tidy spacing
ax.plot(np.cumsum(np.random.randn(500)), label="random walk A")
ax.plot(np.cumsum(np.random.randn(500)), label="random walk B")
ax.set_title("constrained_layout keeps labels inside bounds")
ax.legend()
plt.show()

out_path = "my_first_chart.png"
fig.savefig(out_path, dpi=200, bbox_inches="tight") # bbox trims whitespace
print("saved:", out_path)
```

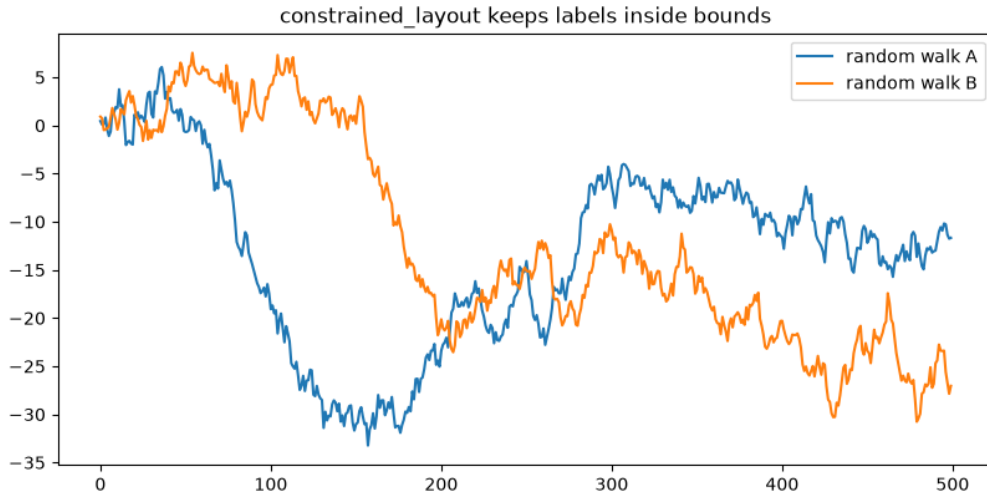


Figure 13: 10. Figure control & saving (ch04_matplotlib_01 figure 9)

```
## Formats: .png (raster, reports/slides) · .svg/.pdf (vector, papers - infinite zoom)
```

Output:

```
saved: my_first_chart.png
```

Common pitfall: `savefig` AFTER `plt.show()` may save an empty image, `show()` closes the figure. Call `savefig` BEFORE `show`, or keep using the `fig` object directly.

Summary & key takeaways

- **Figure** = canvas, **Axes** = one plot, **Axis** = number line. Say it until it sticks.
- `fig, ax = plt.subplots()` is the professional idiom, use `ax.method(...)`.
- Line styling = color + linestyle + linewidth + marker + alpha; shortcuts like "ro--" compress them.
- Grouped bars: offset positions; stacked bars: `bottom=`; both need `xticks` back at category names.
- Histograms need deliberate bins; `density=True` makes series comparable; white edges improve readability.
- Pies are fine for ≤ 5 dominant slices; donut via `wedgeprops(width=...)`.
- Box plots summarize distribution + outliers; combine with reference lines.
- Polish = titles/labels/grid/limits/ticks/annotations, cheap effort, huge clarity.
- Save with `dpi=200+`, `bbox_inches="tight"`; choose SVG/PDF when scaling matters.

5.2 Matplotlib, Advanced

MLCourse · Data Science Foundations · 03_matplotlib

Now we compose: multi panel dashboards, dual axes, images and heatmaps, log scales, themes, colormaps and 3 D. This is the toolbox behind every polished figure you have admired.

What you'll learn

- The object oriented interface as a system (not an idiom)
- Subplots grids + subplot_mosaic layout design
- Twin/secondary axes for two unit comparisons
- Images with imshow, 2 D fields (pcolormesh, contourf), dense scatter fixes
- Error bars & uncertainty bands
- Log scales that reveal power laws
- Styles, rcParams, colormap selection theory
- Plotting straight from DataFrames; 3 D surfaces

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(42)
plt.rcParams["figure.dpi"] = 100
```

1. The OO interface as a mental model

Every pyplot function is a thin wrapper over Axes methods. Learn the mapping and the wrapper becomes optional:

```
mapping = pd.DataFrame({
    "pyplot": ["plt.plot", "plt.title", "plt.xlabel", "plt.legend",
              "plt.xlim", "plt.xticks", "plt.grid", "plt.savefig"],
    "Axes method": ["ax.plot", "ax.set_title", "ax.set_xlabel", "ax.legend",
                   "ax.set_xlim", "ax.set_xticks", "ax.grid", "fig.savefig"],
})
mapping  # rendered as a table in Jupyter
```

Output:

```
pyplot  Axes method
0      plt.plot      ax.plot
1      plt.title      ax.set_title
2      plt.xlabel      ax.set_xlabel
3      plt.legend      ax.legend
```

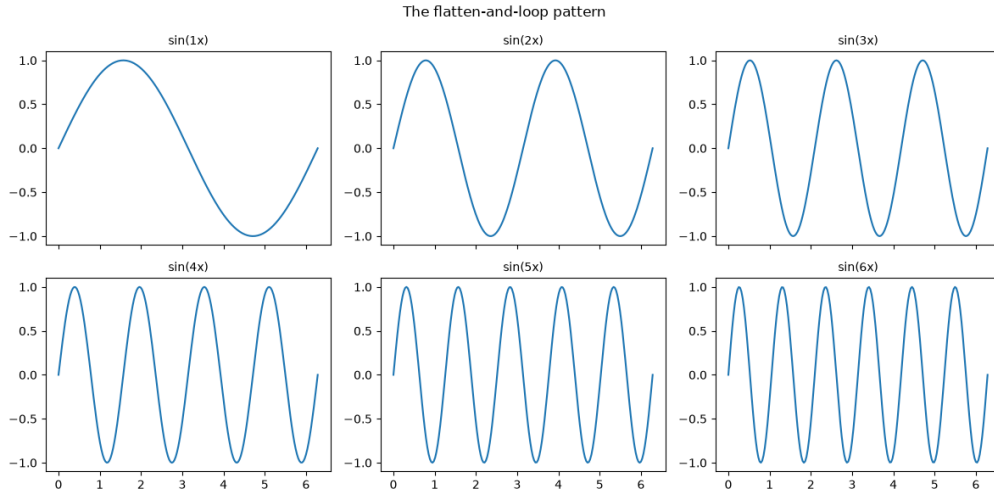


Figure 14: 2. Subplot grids & mosaics (ch04_matplotlib_02 figure 1)

```
4 plt.xlim ax.set_xlim
5 plt.xticks ax.set_xticks
6 plt.grid ax.grid
7 plt.savefig fig.savefig
```

Pro tip: `ax.set(xlabel=..., title=..., ylim=...)` sets several properties in one call, tidy for standard figures.

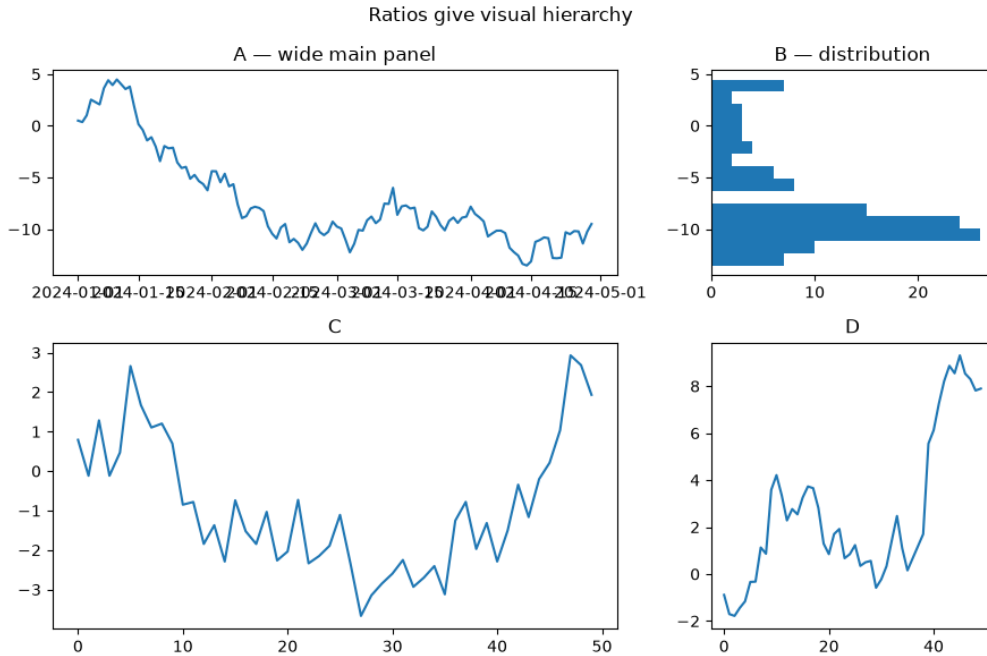
2. Subplot grids & mosaics

```
## A 2x3 grid via the axes-array pattern:
fig, axes = plt.subplots(2, 3, figsize=(12, 6), sharex=True) # shared x-scale
xs = np.linspace(0, 2 * np.pi, 200)

for i, ax_row in enumerate(axes): # iterate ROWS...
    for j, ax in enumerate(ax_row): # ...then columns -> flat index k
        k = i * 3 + j + 1
        ax.plot(xs, np.sin(k * xs))
        ax.set_title(f"sin({k}x)", fontsize=10)

fig.suptitle("The flatten-and-loop pattern")
fig.tight_layout()
plt.show()
```

```
## subplot_mosaic: draw the LAYOUT as ASCII art - modern and self-documenting.
fig, mosaic = plt.subplot_mosaic(
```



```

[["A", "B"],
 ["C", "D"]],
figsize=(9, 6),
gridspec_kw={"width_ratios": [2, 1], "height_ratios": [1, 1.4]}, # column widths
)
ax_big, ax_side = mosaic["A"], mosaic["B"]
ax_bot, ax_bot2 = mosaic["C"], mosaic["D"]

t = pd.date_range("2024-01-01", periods=120)
series = np.cumsum(np.random.randn(120))

ax_big.plot(t, series); ax_big.set_title("A - wide main panel")
ax_side.hist(series, bins=15, orientation="horizontal"); ax_side.set_title("B - distribu
tion")
for ax_, lbl in [(ax_bot, "C"), (ax_bot2, "D")]:
    ax_.plot(np.random.randn(50).cumsum()); ax_.set_title(lbl)

fig.suptitle("Ratios give visual hierarchy")
fig.tight_layout()
plt.show()

```

3. Twin & secondary axes

```

days = np.arange(1, 8)
celsius = np.array([21, 24, 27, 29, 26, 23, 22])
rain = np.array([0, 2, 8, 14, 6, 1, 0])

fig, ax_t = plt.subplots(figsize=(9, 4.5))

## Left axis: temperature bars
ax_t.bar(days, celsius, color="tab:red", alpha=0.65, label="temp °C")
ax_t.set_ylabel("temperature (°C)", color="tab:red")
ax_t.tick_params(axis="y", labelcolor="tab:red")

## Right axis: a SECOND y-scale sharing the same x
ax_r = ax_t.twinx()
ax_r.plot(days, rain, color="tab:blue", marker="o", lw=2, label="rain mm")
ax_r.set_ylabel("rainfall (mm)", color="tab:blue")
ax_r.tick_params(axis="y", labelcolor="tab:blue")

## Merge legends from BOTH axes:
h1, l1 = ax_t.get_legend_handles_labels()
h2, l2 = ax_r.get_legend_handles_labels()
ax_t.legend(h1 + h2, l1 + l2, loc="upper left")

ax_t.set_title("Dual-axis: match each axis' color to its series")
ax_t.set_xlabel("day of week")
fig.tight_layout()
plt.show()

## secondary_xaxis converts units on the fly (forward/inverse functions):
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(np.arange(0, 100), np.linspace(20, 80, 100))
secx = ax.secondary_xaxis(-0.28, functions=(lambda f: f * 5 / 9 + 32, # °F from °C
                                           lambda c: (c - 32) * 5 / 9))
ax.set_xlabel("°C"); secx.set_xlabel("°F")
plt.show()

```

Common pitfall: dual axes invite misreading (which line belongs to which scale?). Color code axis labels/ticks to their series, as above.

4. Images & 2 D fields

Any 2 D array can be shown as an image:

```

yy, xx = np.mgrid[0:200, 0:300] # coordinate grids
image = np.sin(xx / 18) * np.cos(yy / 14) # smooth interference pattern
noisy = image + np.random.normal(0, 0.08, image.shape)

```

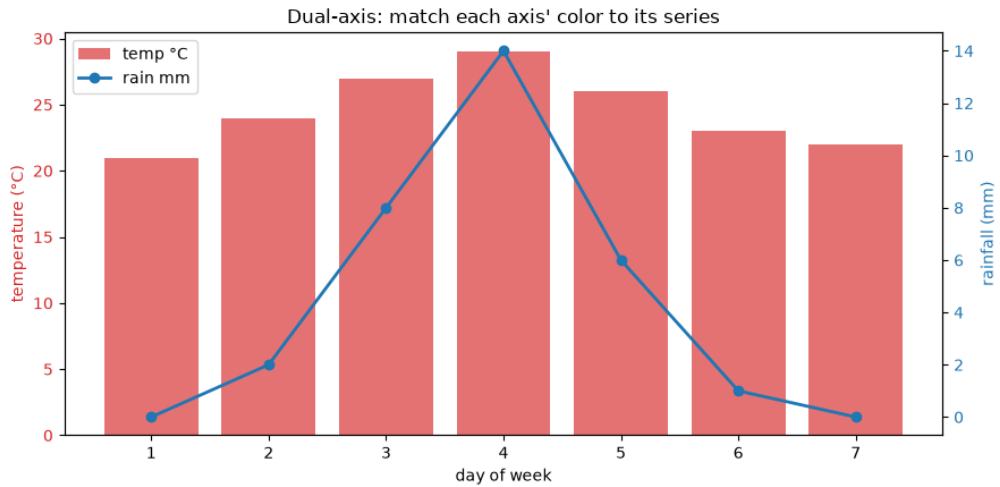


Figure 16: 3. Twin & secondary axes (ch04_matplotlib_02 figure 3)

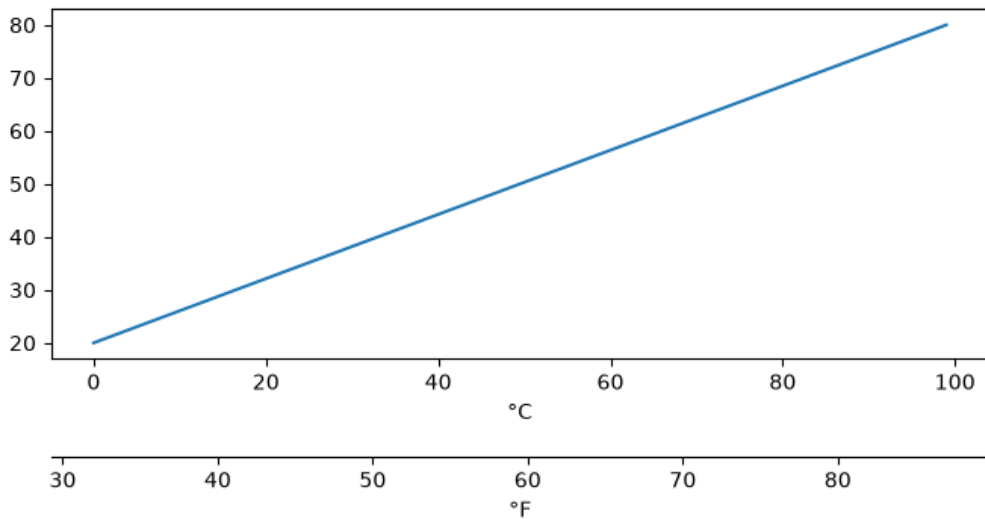


Figure 17: 3. Twin & secondary axes (ch04_matplotlib_02 figure 4)

```

fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))

im0 = axes[0].imshow(image, cmap="twilight")          # default origin='upper'
axes[0].set_title("imshow default")
fig.colorbar(im0, ax=axes[0], shrink=0.8)

im1 = axes[1].imshow(noisy[:, :4, :4],                # downsampled view
                     interpolation="nearest",          # no smoothing of pixels
                     extent=[0, 30, 0, 20])           # real-world coordinates
axes[1].set_title("extent maps pixels data coords")
fig.colorbar(im1, ax=axes[1], shrink=0.8)

im2 = axes[2].contourf(xx[:150, :150], yy[:150, :150],
                      image[:150, :150], levels=14, cmap="viridis")
axes[2].set_title("contourf = filled level curves")
fig.colorbar(im2, ax=axes[2], shrink=0.8)
plt.show()

## pcolormesh ≈ imshow but works with irregular x/y grids - mention, not master.

```

Dense scatter? Go 2 D.

```

n = 60_000                                     # far too many points to overplot
dense = pd.DataFrame({
    "x": np.random.gamma(2.0, 2.0, n),
    "y": np.random.normal(np.sqrt(1), 1.0, n) ,
})

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
axes[0].scatter(dense["x"], dense["y"], s=2, alpha=0.05)    # alpha alone struggles
axes[0].set_title("plain scatter (blob)")

h = axes[1].hist2d(dense["x"], dense["y"], bins=60, cmap="magma")
axes[1].set_title("hist2d"); fig.colorbar(h[3], ax=axes[1])

hb = axes[2].hexbin(dense["x"], dense["y"], gridsize=45, cmap="magma")
axes[2].set_title("hexbin (prettier bins)"); fig.colorbar(hb, ax=axes[2])
plt.show()

```

5. Error bars & uncertainty bands

```

months = np.arange(1, 13)
mean_sales = 40 + 6 * np.sin(months / 2) + months * 0.8
std_err = np.random.uniform(2, 5, 12)

fig, ax = plt.subplots(figsize=(9, 4))

```

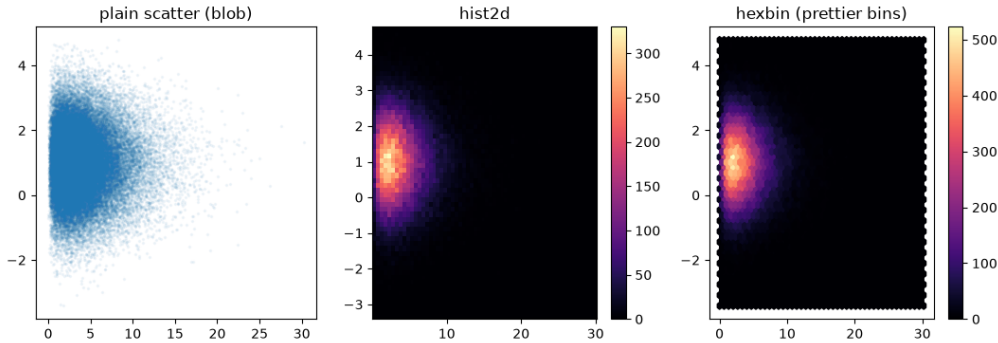


Figure 18: Dense scatter? Go 2 D. (ch04_matplotlib_02 figure 5)

```
ax.errorbar(months, mean_sales, yerr=std_err,
            fmt="o-", capsize=4, color="darkslateblue",
            ecolor="gray", elinewidth=1.2, label="monthly mean ± SE")

## fill_between paints a band - great for confidence intervals around lines:
lower, upper = mean_sales - 2*std_err, mean_sales + 2*std_err
ax.fill_between(months, lower, upper, color="darkslateblue", alpha=0.15,
               label="±2 SE band")
ax.set_xticks(months)
ax.set_title("Point estimates deserve visible uncertainty")
ax.legend()
plt.show()
```

6. Log scales, when magnitude spans decades

```
n_pow = 25
sizes = np.geomspace(1, 1e6, n_pow)          # geometric sequence 1..10^6
cost = sizes ** 1.7 * np.exp(np.random.normal(0, .35, n_pow)) # power law + noise

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(sizes, cost, "o")                # linear axes hide everything
axes[0].set_title("linear axes: unreadable")

axes[1].loglog(sizes, cost, "o")              # both axes log
## Fit slope in LOG space: log(y) = b*log(x) + a -> slope = exponent!
b_fit, a_fit = np.polyfit(np.log10(sizes), np.log10(cost), 1)
xs_fit = np.geomspace(sizes.min(), sizes.max(), 50)
axes[1].loglog(xs_fit, 10**a_fit * xs_fit**b_fit, "r--",
               label=f"power fit: y ∝ x^{b_fit:.2f}")
axes[1].legend()
axes[1].set_title("log-log: straight line power law")
```

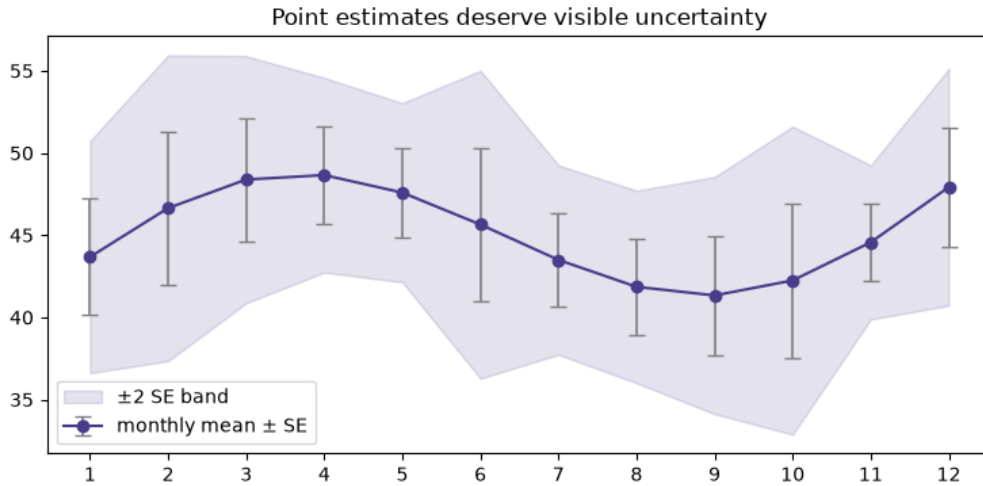


Figure 19: 5. Error bars & uncertainty bands (ch04_matplotlib_02 figure 6)

```
axes[1].set_xlabel("size"); axes[1].set_ylabel("cost")
plt.show()

## semilogx/semilogy scale ONE axis; symlog handles values crossing zero.
```

7. Styles & rcParams, global look and feel

```
print(sorted(plt.style.available)[:12], "...")
```

Output:

```
['Solarize_Light2', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'petroff10', 'petroff6', 'petroff8', 'seaborn-v0_8'] ...
```

```
with plt.style.context("ggplot"):
    fig, ax = plt.subplots(figsize=(6, 3))
    ax.plot(np.cumsum(np.random.randn(80)))
    ax.set_title("temporary 'ggplot' style")
    plt.show()

## rcParams = persistent defaults dictionary:
plt.rcParams.update({
    "font.size": 11,
    "axes.spines.top": False,      # hide top spine globally...
    "axes.spines.right": False,   # ...the manual 'despine' recipe
```

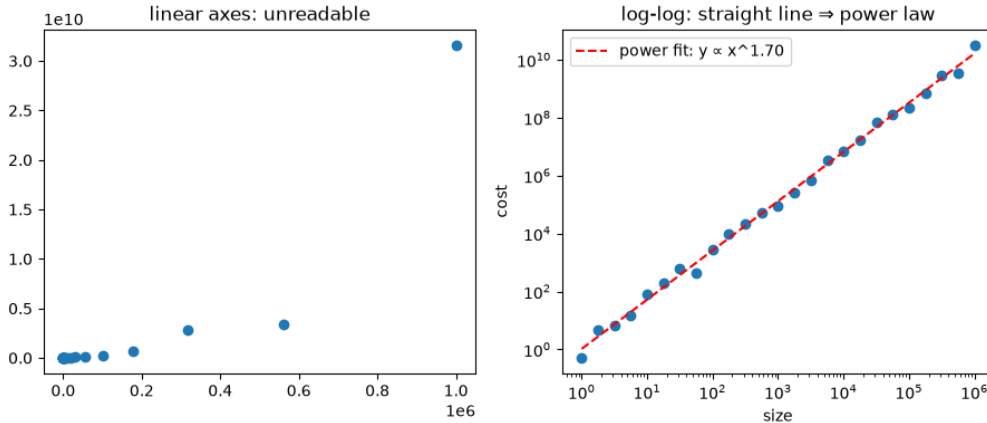



Figure 20: 6. Log scales, when magnitude spans decades (ch04_matplotlib_02 figure 7)

```

    "figure.facecolor": "white",
))
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(np.cumsum(np.random.randn(80))); ax.set_title("rcParams: despined by default")
plt.show()
plt.rcParams.update(plt.rcParamsDefault)      # restore factory settings

```

8. Choosing colormaps deliberately

```

gradient = np.linspace(0, 1, 256).reshape(1, -1)

cmaps = [("sequential", ["viridis", "plasma", "Blues"]),
         ("diverging", ["coolwarm", "RdBu_r", "Spectral"])]

fig, axes = plt.subplots(2, 3, figsize=(11, 2.6))
for row, (kind, names) in enumerate(cmmaps):
    for col, name in enumerate(names):
        axes[row, col].imshow(gradient, aspect="auto", cmap=name)
        axes[row, col].set_title(name, fontsize=10)
        axes[row, col].axis("off")
fig.suptitle("sequential = magnitude · diverging = signed deviation from center")
plt.show()

## Rules: perceptually-uniform (viridis family) for science; diverging when zero
## matters (correlations); never rainbow ('jet') - it distorts perceived order.

```

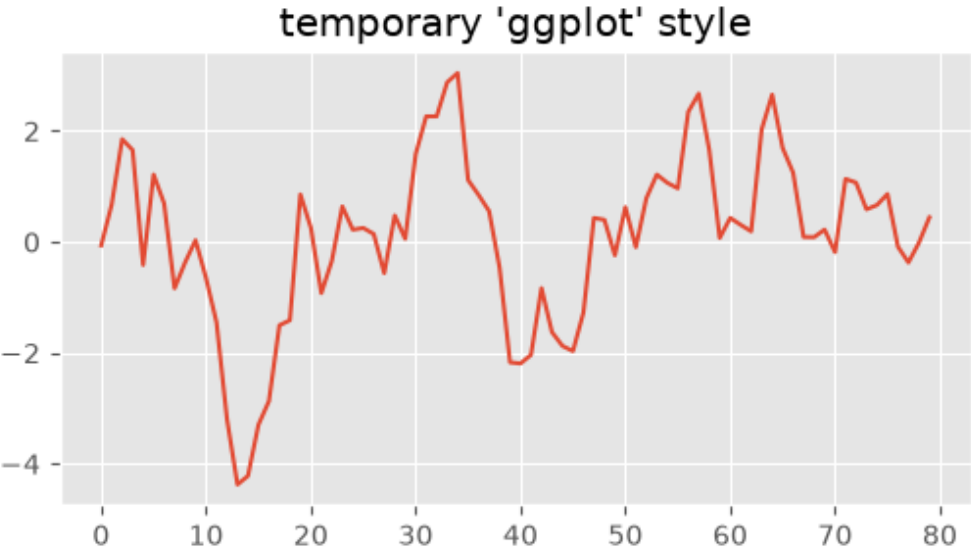


Figure 21: 7. Styles & rcParams, global look and feel (ch04_matplotlib_02 figure 8)

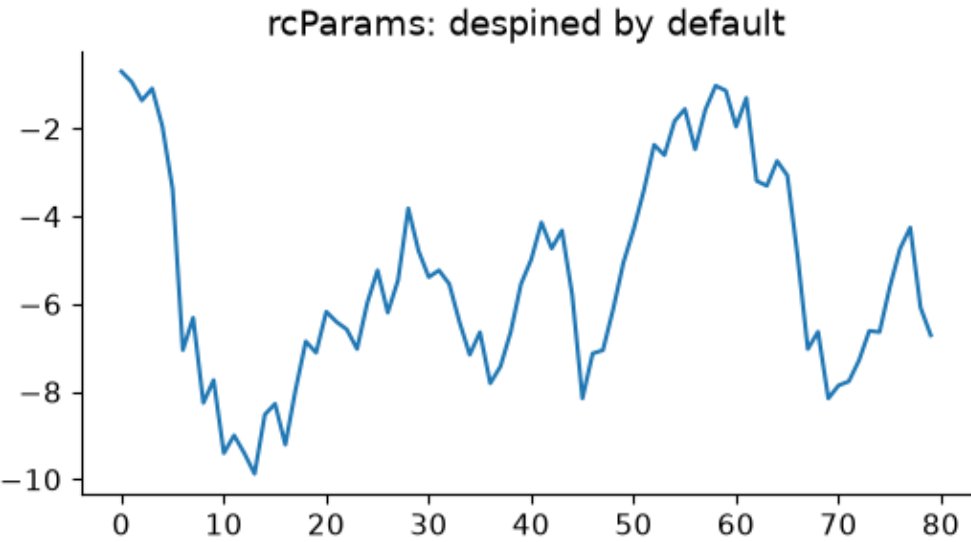


Figure 22: 7. Styles & rcParams, global look and feel (ch04_matplotlib_02 figure 9)



Figure 23: 8. Choosing colormaps deliberately (ch04_matplotlib_02 figure 10)

9. Plotting directly from pandas

```
df = pd.DataFrame({
    "A": np.cumsum(np.random.randn(200)),
    "B": np.cumsum(np.random.randn(200)),
    "C": np.cumsum(np.random.randn(200)),
}, index=pd.date_range("2024-01-01", periods=200))

df.plot(figsize=(9, 4), lw=1.5)  # thin wrapper over our tools
plt.title("DataFrame.plot() convenience wrapper")
plt.show()

## Grouped-to-lines pattern without the wrapper:
fig, ax = plt.subplots(figsize=(9, 4))
for key, grp in df.groupby(df.index.quarter):
    pass  # demo only - real grouping below
tips_like = pd.DataFrame({"day": np.repeat(["Mon", "Tue", "Wed"], 40),
                          "val": np.random.randn(120).cumsum()})
for day, grp in tips_like.groupby("day"):
    ax.plot(grp.index, grp["val"].cumsum(), label=day)  # one line per group
ax.legend(title="day")
plt.show()
```

10. 3 D plots

```
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401 (registers projection)

X = np.linspace(-4, 4, 80)
Y = np.linspace(-4, 4, 80)
Xg, Yg = np.meshgrid(X, Y)  # 2-D coordinate matrices
Zg = np.sinc(np.sqrt(Xg**2 + Yg**2))  # radial sinc surface

fig = plt.figure(figsize=(13, 4))
```

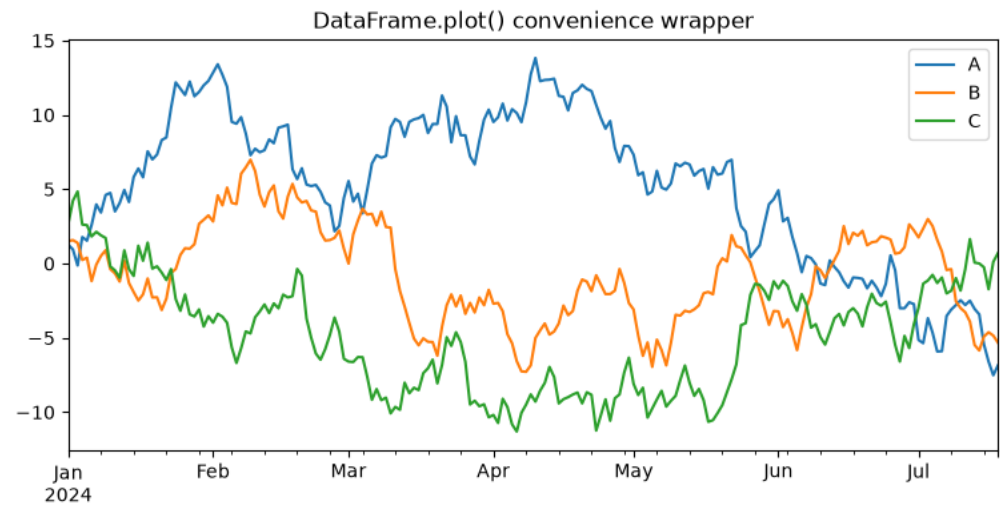


Figure 24: 9. Plotting directly from pandas (ch04_matplotlib_02 figure 11)

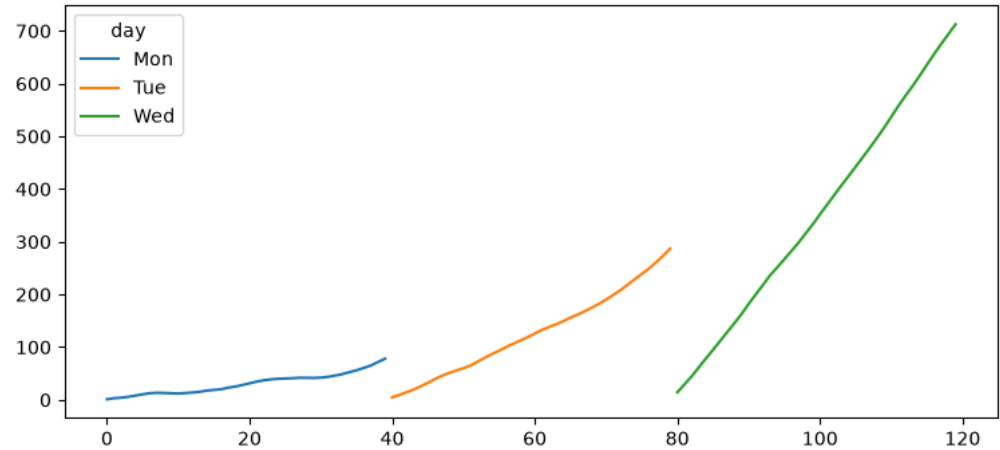


Figure 25: 9. Plotting directly from pandas (ch04_matplotlib_02 figure 12)

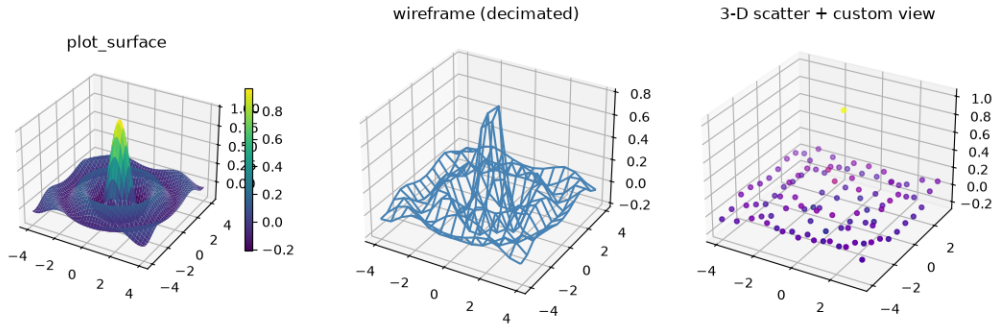


Figure 26: 10. 3 D plots (ch04_matplotlib_02 figure 13)

```
ax1 = fig.add_subplot(131, projection="3d")
surf = ax1.plot_surface(Xg, Yg, Zg, cmap="viridis", linewidth=0)
fig.colorbar(surf, ax=ax1, shrink=0.55)
ax1.set_title("plot_surface")

ax2 = fig.add_subplot(132, projection="3d")
ax2.plot_wireframe(Xg[::6, ::6], Yg[::6, ::6], Zg[::6, ::6], color="steelblue")
ax2.set_title("wireframe (decimated)")

ax3 = fig.add_subplot(133, projection="3d")
ax3.scatter(Xg[::8, ::8], Yg[::8, ::8], Zg[::8, ::8],
            c=Zg[::8, ::8], cmap="plasma", s=12)
ax3.view_init(elev=28, azim=-60)  # camera angle control
ax3.set_title("3-D scatter + custom view")
plt.show()
```

Animation pointer

`matplotlib.animation.FuncAnimation(fig, update_func, frames=...)` redraws an artist per frame; export with `ani.save("anim.mp4", writer="ffmpeg")`. It needs `ffmpeg` installed, out of scope here, see official docs gallery.

Summary & key takeaways

- pyplot Axes mapping table frees you to mix interfaces safely.
- Design layouts with subplots grids or `subplot_mosaic` ASCII art + width/height ratios; loop over the flattened array.
- `twinx/secondary_axis` compare different units, color match axes to series.
- `imshow` shows any matrix; `extent` gives it data coordinates; use `hexbin/ hist2d` when scatter saturates.
- Always visualize uncertainty: `errorbar`, `fill_between`.

- Log log turns power laws into straight lines whose SLOPE is the exponent (polyfit on logs).
- Control house style via styles/context managers and rcParams; despine by hiding top/right spines.
- Pick colormaps by DATA TYPE: sequential for magnitude, diverging for signed values, viridis family always safe.
- df.plot() is a convenience wrapper; groupby loop gives full control.
- 3 D = add projection="3d"; steer the camera with view_init.

5.3 Matplotlib, Exercises

MLCourse · Data Science Foundations · 03_matplotlib

Twelve plotting challenges. Each gives you seeded starter data plus a checklist of required elements, build the figure to spec, like a design brief from a stakeholder. Solutions live in notebook 04.

Grading yourself: a chart is done when every checklist item is visibly true.

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(42)
plt.rcParams["figure.dpi"] = 100

def new_ax(w=8, h=4):
    """Convenience: fresh figure + axes with standard size."""
    fig, ax = plt.subplots(figsize=(w, h))
    return fig, ax
```

Easy (1-4)

Challenge, Revenue trends with a recession band

Plot three product revenue lines. Requirements: - one line per product, distinct colors + legend, - shaded vertical band (axvspan) marking the recession period, - titled & axis labeled, subtle y grid.

```
months = pd.date_range("2022-01", periods=36, freq="ME") prod_A =
np.cumsum(np.random.normal(1.0, 0.6, 36)) + 20 prod_B = np.cumsum(np.random.normal(0.7,
0.5, 36)) + 14 prod_C = np.cumsum(np.random.normal(1.3, 0.9, 36)) + 8
```

your plot here

Challenge, Grouped AND stacked bars

Using the data below: left subplot shows GROUPED bars (male/female side by side per department), right subplot shows STACKED totals. Both need legends and readable category labels.

```
departments = ["Eng", "Sales", "HR", "Design"]
male = np.array([42, 28, 6, 12])
female = np.array([18, 31, 14, 21])
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
your plots here (axes[0] grouped, axes[1] stacked)
```

Challenge, Overlaid step histograms

Compare two distributions with `histtype="step"` outlines (no fill), same bins, `density=True` so they're comparable, and a legend explaining which is which.

```
group_control = np.random.normal(100, 15, 900)
group_treated = np.random.normal(108, 12, 700)
your plot here
```

Challenge, Labeled scatter of cities

Scatter population vs cost of living; requirements: - point size $\propto$ GDP, color by continent code with colorbar or discrete legend, - annotate the 3 most expensive cities by name, - alpha for overplotting.

```
n = 60
population = np.random.lognormal(13, 1.2, n) # fake city sizes
cost_index = 40 + 30 * np.log10(population) / np.log10(population.max())
+ np.random.normal(0, 8, n)
gdp = population * np.random.uniform(20, 80, n)
continent = np.random.choice([0, 1, 2], n) # pretend codes
your plot here
```

Medium (5-9)**Challenge, Dual axis weather dashboard**

One month of daily max temp (bars) and rainfall (line) on `twinx`. Checklist: color matched axis labels/ticks, merged legend, weekend markers (light `axvspan` on each Sat/Sun).

```
days = pd.date_range("2024-11-01", periods=30)
tmax = 18 + 6 * np.sin(np.arange(30)/30*2*np.pi)
+ np.random.normal(0, 2, 30)
rain = np.abs(np.random.gamma(1.2, 2.0, 30)) * (np.random.rand(30) < 0.5)
your plot here
```

Challenge, 2×2 mosaic mini dashboard

With `plt.subplots_mosaic` build layout:

```
AB
CC
```

A: histogram · B: boxplot of same data · C: wide line chart of cumulative sum. Shared suptitle; C spans both columns.

```
data_dash = np.random.normal(50, 12, 400)
layout = """ AB CC """
your plot here
```

Challenge, Heatmap with custom ticks

Render the 10×10 matrix below with imshow + colorbar. Checklist: origin="lower", x/y tick labels 1..10 at integer positions, diverging colormap centered on zero (vmin=-max, vmax=max).

```
matrix = np.outer(np.linspace(-1, 1, 10), np.linspace(1, -1, 10))
+ np.random.normal(0, 0.08, (10, 10))
your plot here
```

Challenge, Forecast with uncertainty cone

Plot history line, forecast line, and a widening fill_between band around it. Checklist: distinct styles (solid vs dashed), labeled band in legend, annotation where forecast starts ("today").

```
history = np.cumsum(np.random.normal(0.4, 1.2, 60)) forecast = history[-1] + np.arange(1, 25)
* 0.5 uncert = np.linspace(0.5, 4.5, 24)
your plot here
```

Challenge, Power law on log log with fit

Data below follows $y = A \cdot x^b$ with noise. On LOG LOG axes scatter it, fit via `np.polyfit(np.log10(x), np.log10(y), 1)`, overlay fitted line, put the recovered exponent b in the LEGEND label.

```
xx = np.geomspace(1, 1e4, 40) yy = 3.5 * xx**0.65 * np.exp(np.random.normal(0, .25, 40))
your plot here
```

Hard (10-12)

Challenge, 3 D surface with camera control

Plot $Z = \sin(\sqrt{x^2+y^2}) / \sqrt{\dots}$ style ripple as surface + wireframe combo, set an angled view_init, add colorbar and axis labels.

```
gx = np.linspace(-8, 8, 120) gy = np.linspace(-8, 8, 120) XG, YG = np.meshgrid(gx, gy) ZG =
np.sin(np.sqrt(XG2 + YG2))
your plot here
```



```
ohlcv = pd.DataFrame({"open": np.cumsum(np.random.normal(0, 1, 40)) + 100, "close":
```

```
walks = [np.cumsum(np.random.randn(90)) for _ in range(5)]
```

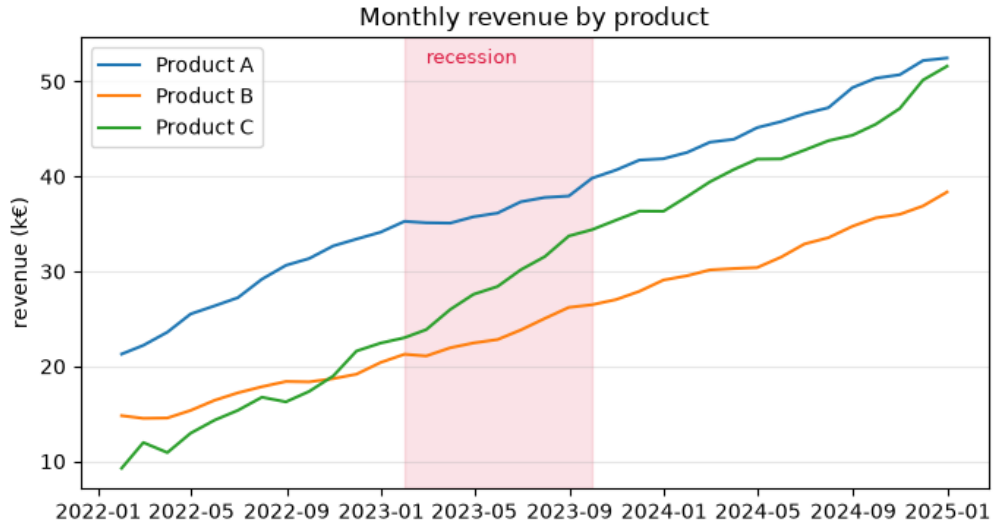


Figure 27: Solution 1, Revenue trends with a recession band (ch04_matplotlib_04 figure 1)

```
ax.plot(months, prod_A, color="tab:blue", label="Product A")
ax.plot(months, prod_B, color="tab:orange", label="Product B")
ax.plot(months, prod_C, color="tab:green", label="Product C")

## Recession: months 12..20 -> shade full height with axvspan
ax.axvspan(months[12], months[20], color="crimson", alpha=0.12)
ax.text(months[13], ax.get_ylim()[1]*0.95, "recession", color="crimson", fontsize=9)

ax.set_title("Monthly revenue by product")
ax.set_ylabel("revenue (k€)")
ax.grid(axis="y", alpha=0.3)
ax.legend()
plt.show()
```

Solution, Grouped AND stacked bars

```
departments = ["Eng", "Sales", "HR", "Design"]
male = np.array([42, 28, 6, 12])
female = np.array([18, 31, 14, 21])

xpos = np.arange(len(departments))
w = 0.38

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
```

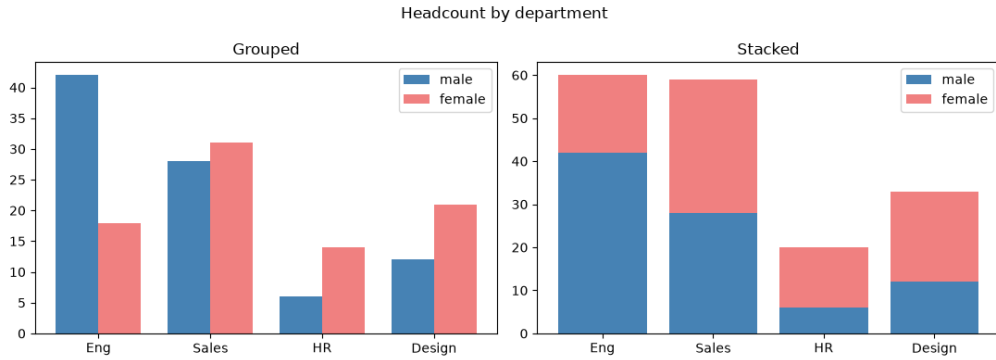


Figure 28: Solution 2, Grouped AND stacked bars (ch04_matplotlib_04 figure 2)

```
axes[0].bar(xpos - w/2, male, w, label="male", color="steelblue") # left offset
axes[0].bar(xpos + w/2, female, w, label="female", color="lightcoral") # right offset
axes[0].set_xticks(xpos, departments)
axes[0].set_title("Grouped")

axes[1].bar(xpos, male, label="male", color="steelblue")
axes[1].bar(xpos, female, bottom=male, label="female", color="lightcoral") # stack on top
axes[1].set_xticks(xpos, departments)
axes[1].set_title("Stacked")

for ax in axes:
    ax.legend()
fig.suptitle("Headcount by department")
fig.tight_layout()
plt.show()
```

Solution, Overlaid step histograms

```
group_control = np.random.normal(100, 15, 900)
group_treated = np.random.normal(108, 12, 700)

bins = np.arange(55, 160, 5) # SAME edges for both = fair compare
fig, ax = new_ax()
ax.hist(group_control, bins=bins, density=True, histtype="step", lw=2,
        label="control", color="gray")
ax.hist(group_treated, bins=bins, density=True, histtype="step", lw=2,
        label="treated", color="purple")
ax.set_title("Treated shifts right ~8 units")
ax.set_xlabel("score"); ax.set_ylabel("density")
ax.legend()
```



Figure 29: Solution 3, Overlaid step histograms (ch04_matplotlib_04 figure 3)

```
plt.show()
```

Solution, Labeled scatter of cities

```
n = 60
population = np.random.lognormal(13, 1.2, n)
cost_index = 40 + 30 * np.log10(population) / np.log10(population.max()) \
    + np.random.normal(0, 8, n)
gdp = population * np.random.uniform(20, 80, n)
continent = np.random.choice([0, 1, 2], n)

cmap = plt.get_cmap("viridis", 3)  # discrete 3-color version

fig, ax = new_ax()
sc = ax.scatter(population / 1e6, cost_index,  # millions for readable axis
               s=gdp / gdp.max() * 400 + 10,  # size ∝ GDP, floor of 10pt
               c=continent, cmap=cmap, vmin=-0.5, vmax=2.5,  # centers on integers
               alpha=0.65, edgecolors="white", linewidths=0.5)
fig.colorbar(sc, ticks=[0, 1, 2], label="continent code")

top3 = np.argsort(cost_index)[-3:]  # indices of 3 most expensive
for i in top3:
    ax.annotate(f"#{i}", xy=(population[i]/1e6, cost_index[i]),
               xytext=(5, 5), textcoords="offset points",
               fontsize=9, fontweight="bold")
```

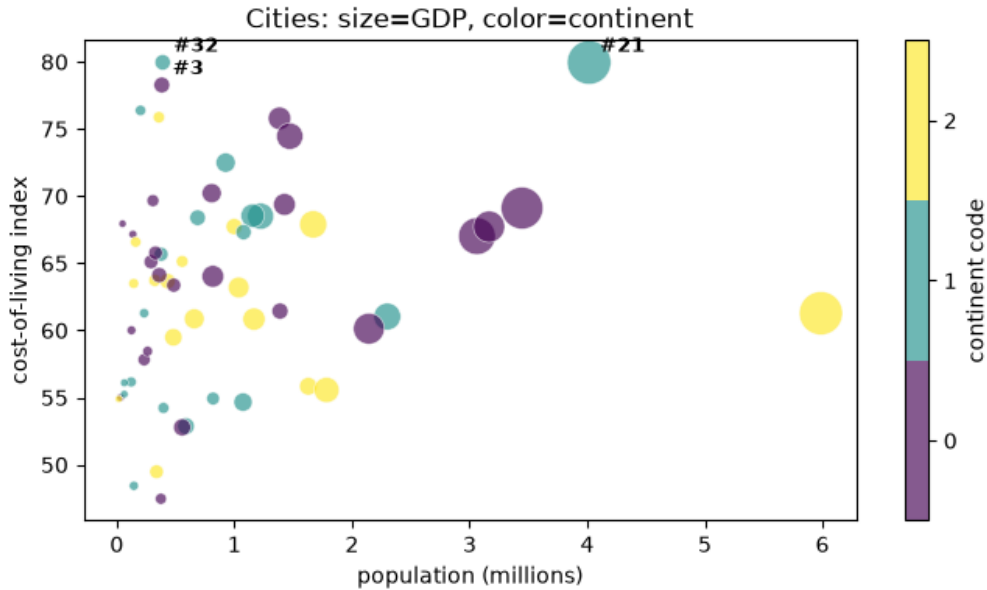


Figure 30: Solution 4, Labeled scatter of cities (ch04_matplotlib_04 figure 4)

```
ax.set_title("Cities: size=GDP, color=continent")
ax.set_xlabel("population (millions)")
ax.set_ylabel("cost-of-living index")
plt.show()
```

Solution, Dual axis weather dashboard

```
days = pd.date_range("2024-11-01", periods=30)
tmax = 18 + 6*np.sin(np.arange(30)/30*2*np.pi) + np.random.normal(0, 2, 30)
rain = np.abs(np.random.gamma(1.2, 2.0, 30)) * (np.random.rand(30) < 0.5)

fig, ax_t = new_ax(9, 4.5)

ax_t.bar(days, tmax, width=0.7, color="tab:red", alpha=0.6, label="max temp °C")
ax_t.set_ylabel("temp (°C)", color="tab:red")
ax_t.tick_params(axis="y", labelcolor="tab:red")

ax_r = ax_t.twinx()
ax_r.plot(days, rain, color="tab:blue", lw=1.8, drawstyle="steps-mid", label="rain mm")
ax_r.set_ylabel("rain (mm)", color="tab:blue")
ax_r.tick_params(axis="y", labelcolor="tab:blue")
```

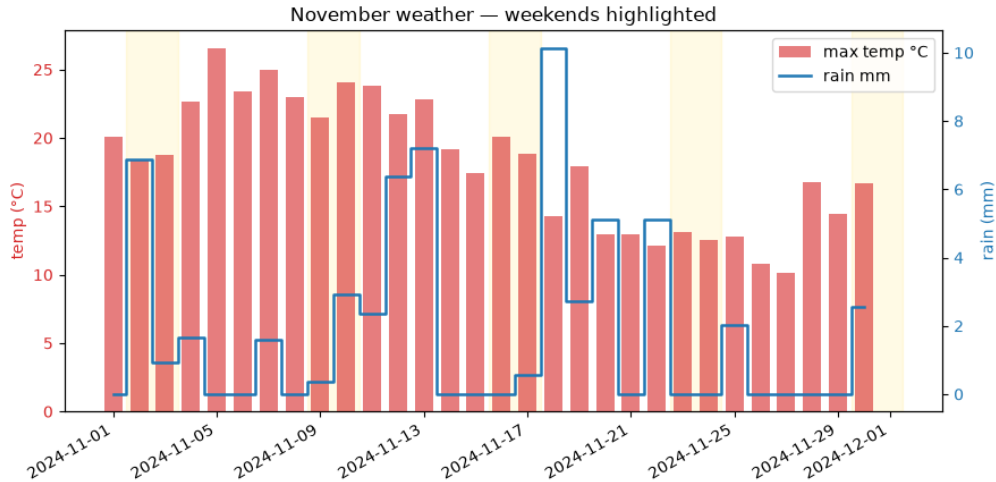


Figure 31: Solution 5, Dual axis weather dashboard (ch04_matplotlib_04 figure 5)

```
## Shade each Saturday+Sunday pair:
for i, d in enumerate(days):
    if d.dayofweek == 5:
        # Saturday
        ax_t.axvspan(d - pd.Timedelta(hours=12), d + pd.Timedelta(days=1, hours=12),
                    color="gold", alpha=0.10, zorder=0)

h1,l1 = ax_t.get_legend_handles_labels(); h2,l2 = ax_r.get_legend_handles_labels()
ax_t.legend(h1+h2, l1+l2, loc="upper right")
ax_t.set_title("November weather - weekends highlighted")
fig.autofmt_xdate()
fig.tight_layout()
plt.show()
```

Solution, 2×2 mosaic mini dashboard

```
data_dash = np.random.normal(50, 12, 400)

fig, m = plt.subplot_mosaic(
    [
        ["A", "B"],
        ["C", "C"],
    ],
    # C spans both columns on bottom row
    figsize=(9, 6),
)

m["A"].hist(data_dash, bins=25, color="skyblue", edgecolor="white")
m["A"].set_title("distribution")

m["B"].boxplot(data_dash, tick_labels=["all data"])
```

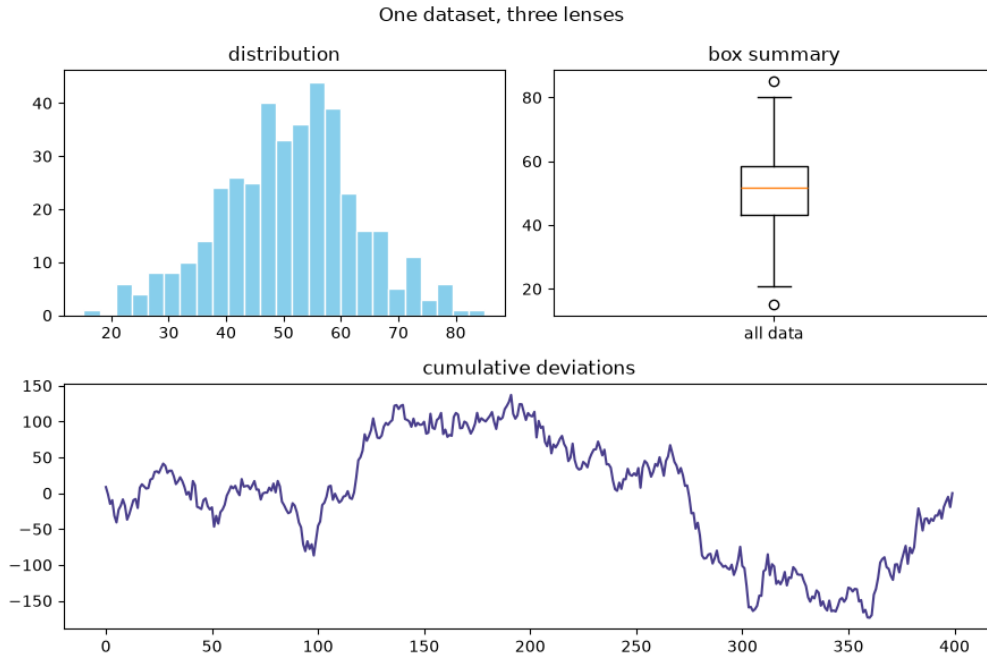


Figure 32: Solution 6, 2×2 mosaic mini dashboard (ch04_matplotlib_04 figure 6)

```
m["B"].set_title("box summary")

m["C"].plot(np.cumsum(data_dash - data_dash.mean()), color="darkslateblue")
m["C"].set_title("cumulative deviations")

fig.suptitle("One dataset, three lenses")
fig.tight_layout()
plt.show()
```

Solution, Heatmap with custom ticks

```
matrix = np.outer(np.linspace(-1, 1, 10), np.linspace(1, -1, 10)) \
    + np.random.normal(0, 0.08, (10, 10))

lim = np.abs(matrix).max() # symmetric limits center zero

fig, ax = new_ax(6.4, 5)
im = ax.imshow(matrix, origin="lower", # row 0 at BOTTOM
    cmap="RdBu_r", vmin=-lim, vmax=lim)
fig.colorbar(im, ax=ax, shrink=0.85, label="value")
```

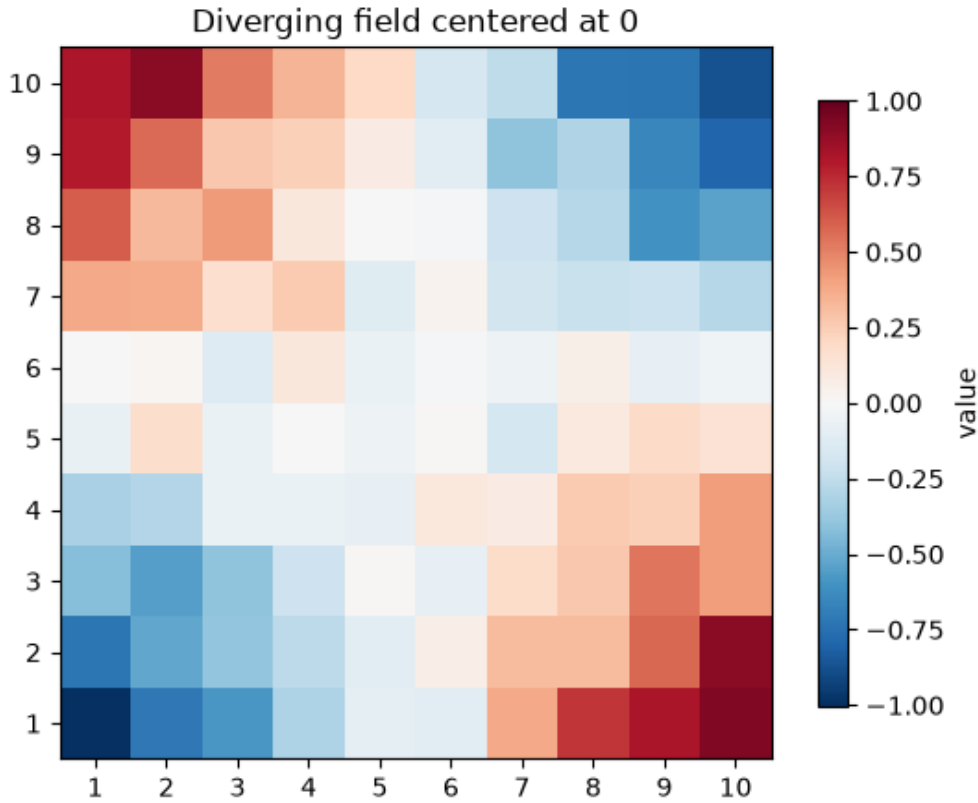


Figure 33: Solution 7, Heatmap with custom ticks (ch04_matplotlib_04 figure 7)

```
ticks = np.arange(10)
ax.set_xticks(ticks, ticks + 1)           # human numbering 1..10
ax.set_yticks(ticks, ticks + 1)
ax.set_title("Diverging field centered at 0")
plt.show()
```

Solution, Forecast with uncertainty cone

```
history = np.cumsum(np.random.normal(0.4, 1.2, 60))
forecast = history[-1] + np.arange(1, 25) * 0.5
uncert = np.linspace(0.5, 4.5, 24)

x_hist = np.arange(60)
x_fcst = np.arange(59, 60 + len(forecast))    # anchor + each forecast step
```

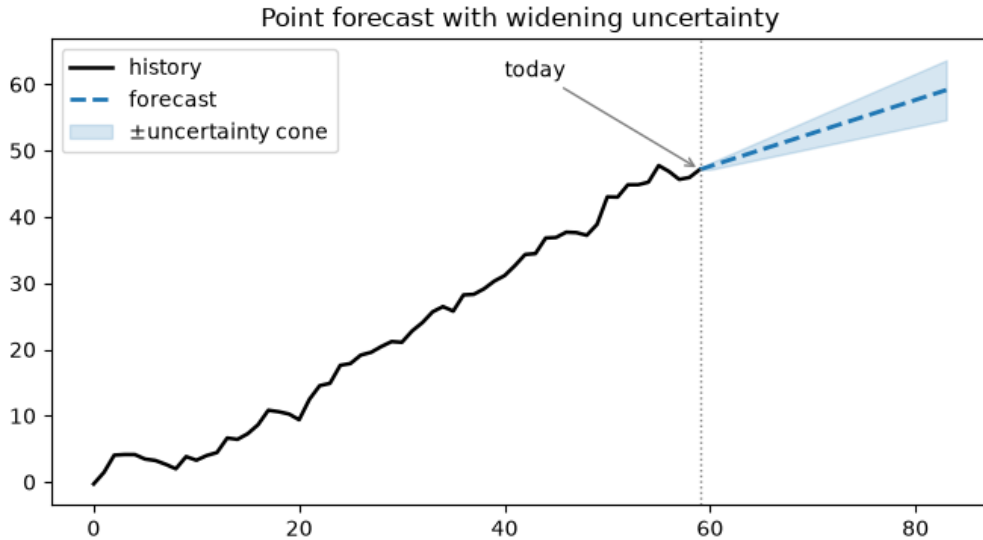



Figure 34: Solution 8, Forecast with uncertainty cone (ch04_matplotlib_04 figure 8)

```
fig, ax = new_ax()
ax.plot(x_hist, history, "-", color="black", lw=1.8, label="history")
fcst_curve = np.r_[history[-1], forecast] # start today, then forecast
ax.plot(x_fcst, fcst_curve, "--", color="tab:blue", lw=2, label="forecast")
band_lo = np.r_[history[-1], forecast - uncert]
band_hi = np.r_[history[-1], forecast + uncert]
ax.fill_between(x_fcst, band_lo, band_hi, color="tab:blue", alpha=0.18,
               label="±uncertainty cone")

today_x = 59
ax.axvline(today_x, color="gray", ls=":", lw=1)
ax.annotate("today", xy=(today_x, history[-1]), xytext=(40, history[-1]+14),
           arrowprops=dict(arrowstyle="->", color="gray"))
ax.legend(loc="upper left")
ax.set_title("Point forecast with widening uncertainty")
plt.show()
```

Solution, Power law on log log with fit

```
xx = np.geomspace(1, 1e4, 40)
yy = 3.5 * xx**0.65 * np.exp(np.random.normal(0, .25, 40))

b_fit, a_fit = np.polyfit(np.log10(xx), np.log10(yy), 1) # log-space linear fit
xs_line = np.geomspace(xx.min(), xx.max(), 100)
```

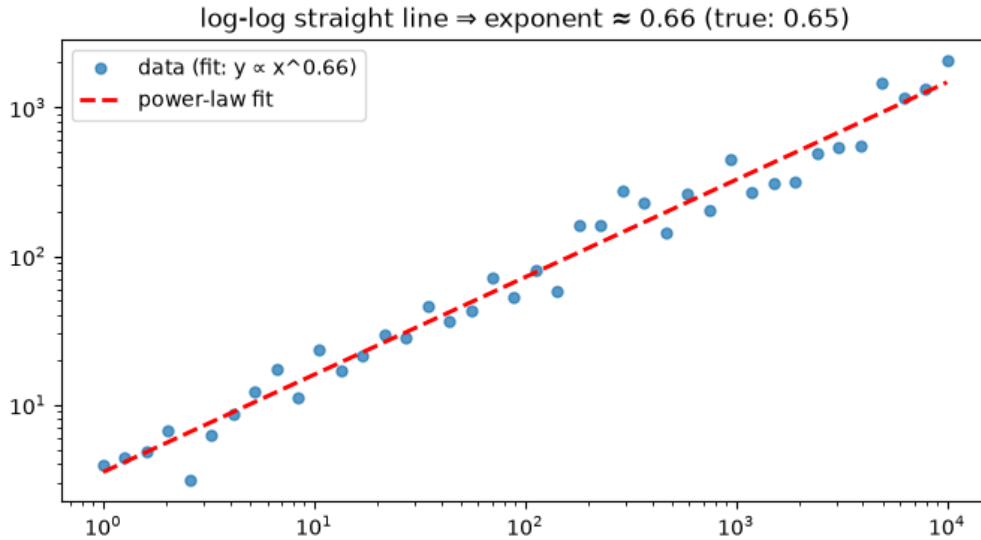


Figure 35: Solution 9, Power law on log log with fit (ch04_matplotlib_04 figure 9)

```
ys_line = 10**a_fit * xs_line**b_fit                    # invert: y = A·x^b

fig, ax = new_ax()
ax.loglog(xs, yy, "o", ms=5, alpha=0.75, label=f"data (fit: y ∝ x^{b_fit:.2f})")
ax.loglog(xs_line, ys_line, "r--", lw=2, label="power-law fit")
ax.set_title(f"log-log straight line  exponent ≈ {b_fit:.2f} (true: 0.65)")
ax.legend()
plt.show()
```

Solution, 3 D surface with camera control

```
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401

gx = np.linspace(-8, 8, 120); gy = np.linspace(-8, 8, 120)
XG, YG = np.meshgrid(gx, gy)
ZG = np.sin(np.sqrt(XG**2 + YG**2))

fig = plt.figure(figsize=(11, 4.6))

ax1 = fig.add_subplot(121, projection="3d")
surf = ax1.plot_surface(XG, YG, ZG, cmap="viridis", linewidth=0, antialiased=True)
fig.colorbar(surf, ax=ax1, shrink=0.6)
ax1.view_init(elev=32, azim=-58)
ax1.set_title("surface")
```

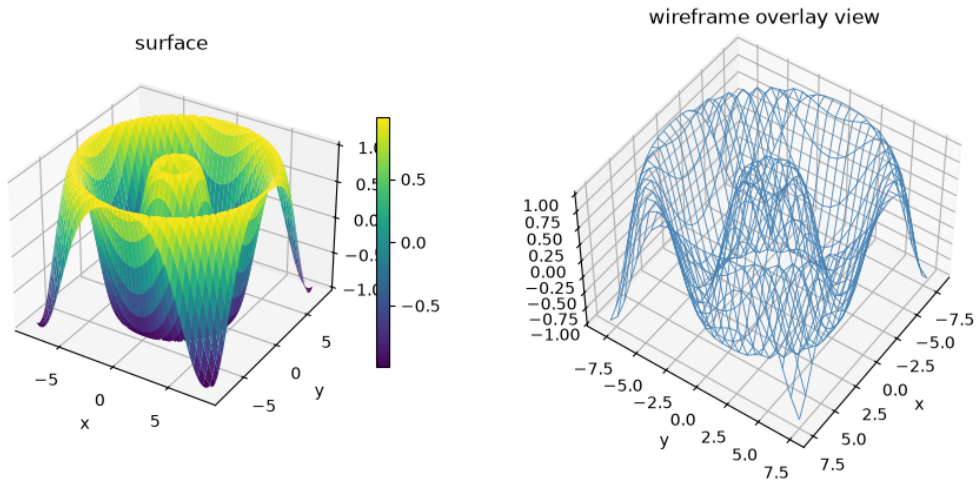


Figure 36: Solution 10, 3 D surface with camera control (ch04_matplotlib_04 figure 10)

```
ax2 = fig.add_subplot(122, projection="3d")
wire = ax2.plot_wireframe(XG[:,5, ::5], YG[:,5, ::5], ZG[:,5, ::5],
                          color="steelblue", lw=0.6)
ax2.view_init(elev=48, azim=35)
ax2.set_title("wireframe overlay view")

for a in (ax1, ax2):
    a.set_xlabel("x"); a.set_ylabel("y")
plt.show()
```

Solution, Candlestick style OHLC

```
ohl = pd.DataFrame({"open": np.cumsum(np.random.normal(0, 1, 40)) + 100})
ohl["close"] = ohl["open"] + np.random.normal(0, 1.4, 40)
ohl["high"] = ohl[["open", "close"]].max(axis=1) + np.abs(np.random.normal(0, .6, 40))
ohl["low"] = ohl[["open", "close"]].min(axis=1) - np.abs(np.random.normal(0, .6, 40))

up = ohl["close"] >= ohl["open"]
colors = np.where(up, "seagreen", "firebrick")

fig, ax = new_ax(10, 4)
xs = np.arange(len(ohl))

## Wicks: thin vertical lines lowhigh:
ax.vlines(xs, ohl["low"], ohl["high"], color=colors, lw=1)
## Bodies: thick bars openclose:
ax.bar(xs, (ohl["close"] - ohl["open"]).abs().replace(0, 0.02),
```

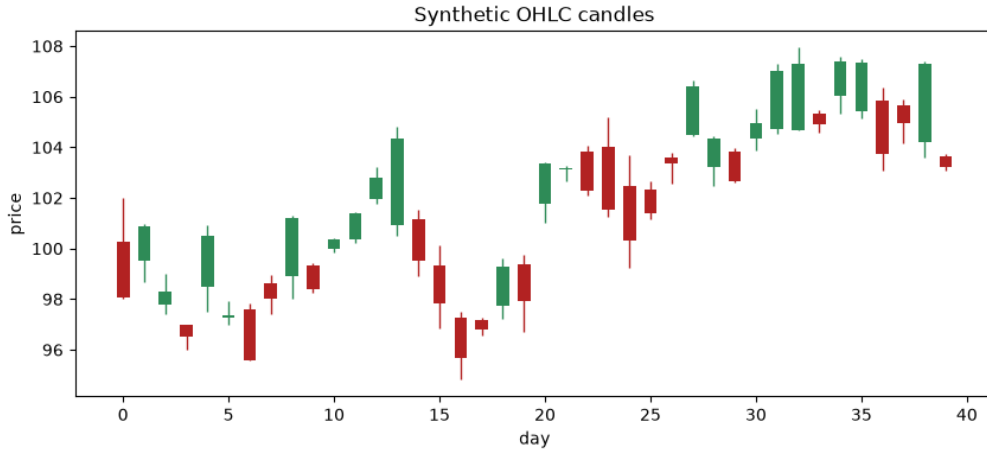


Figure 37: Solution 11, Candlestick style OHLC (ch04_matplotlib_04 figure 11)

```

        bottom=np.minimum(ohlc["open"], ohlc["close"]),
        width=0.6, color=colors, zorder=3)

ax.set_title("Synthetic OHLC candles")
ax.set_xlabel("day"); ax.set_ylabel("price")
plt.show()

```

Solution, Sparklines row

```

walks = [np.cumsum(np.random.randn(90)) for _ in range(5)]

fig, axes = plt.subplots(1, 5, figsize=(11, 1.4))

for ax, walk in zip(axes, walks):
    ax.plot(walk, lw=1, color="dimgray")          # the line only...
    up_day = walk[-1] >= walk[0]
    end_color = "seagreen" if up_day else "firebrick"
    ax.plot(len(walk)-1, walk[-1], "o", color=end_color, ms=5) # direction dot
    ax.set_xticks([]); ax.set_yticks([])          # strip ALL ticks
    for spine in ax.spines.values():              # ...and all borders
        spine.set_visible(False)

fig.suptitle("Sparklines: pure signal, zero chrome", y=1.25)
plt.show()

```

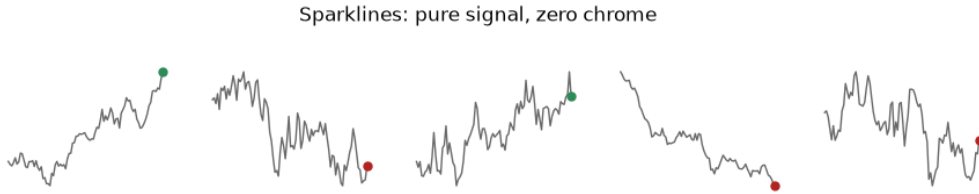


Figure 38: Solution 12, Sparklines row (ch04_matplotlib_04 figure 12)

Reflection checklist

- Could you rebuild any of these from memory?
- Which solution used a trick you hadn't tried (e.g., `np.minimum` for candle bottoms, `get_cmap(n)` for discrete colors)?
- Where did YOU over decorate? Compare ink to information ratio honestly.

Chapter 6

Seaborn

Seaborn sits on top of Matplotlib and speaks a different language: instead of "draw a line from these x, y values," you say "show me the relationship between these two columns of this DataFrame," and Seaborn handles the aggregation, the color mapping, and the statistical overlay for you. This chapter covers Seaborn's statistical plot types, distributions, relationships, categorical comparisons, and when to reach for Seaborn's speed versus Matplotlib's control.

6.1 Seaborn, Seaborn Foundations

MLCourse · Data Science Foundations · 04_seaborn

Welcome to the visualization module! Seaborn is the statistical graphics layer of the Python data stack: it takes tidy DataFrames and turns them into polished, statistically aware figures with remarkably little code. This notebook builds your mental model from zero: what seaborn is, how its two level API works (the single most important concept in this course), and how to wield relational plots, distribution plots, categorical plots, and palettes with confidence.

Everything here runs on seaborn's built in datasets, so you can follow along without downloading anything.

What you'll learn

- The seaborn philosophy: dataset oriented, statistics aware defaults that look good out of the box, and how seaborn relates to raw matplotlib.
- The **two level API**: axes level functions vs figure level functions, and when to reach for each.
- **Relational plots**: scatterplot, lineplot, and the relplot dispatcher; semantic mapping with hue, style, size, markers.
- How lineplot automatically aggregates repeated measurements and draws a **bootstrap confidence band**.
- **Distribution plots**: histograms (histplot), kernel density estimates (kdeplot, 1 D and 2 D), empirical CDFs (ecdfplot), and rugs.
- **Categorical summaries**: boxplots, violinplots, boxenplots, strip and swarm plots, plus aggregate views with countplot, barplot, and pointplot.
- Faceting made trivial with the figure level catplot.
- A practical tour of **palettes**: qualitative, sequential, diverging, and colorblind safe practice.

1. Why seaborn exists (and how it relates to matplotlib)

Matplotlib is a general purpose plotting engine: extremely flexible, but you specify everything, colors, labels, ticks, legends, by hand. Seaborn sits on top of matplotlib and makes a different trade:

- **Dataset oriented.** You pass a DataFrame plus column *names* (`x="total_bill"`), not arrays. Column names become axis labels and legend titles for free.
- **Statistics aware defaults.** Many seaborn functions do light statistics before drawing: `lineplot` averages repeats with confidence intervals, `barplot` estimates means with uncertainty, KDEs smooth densities. You get honest plots by default.
- **Beautiful out of the box.** Themes, font scales, and perceptually sane color palettes are one function call away.

Crucially, seaborn does **not** replace matplotlib, it drives it. Every seaborn figure is a real matplotlib Figure full of real matplotlib Axes, so anything you learn later about matplotlib fine tuning still applies. That interop is the subject of notebook 02.

Standard setup cell for every notebook in this module.

```
## Note: we trigger the Jupyter "%matplotlib inline" magic through its
## Python equivalent so this file stays valid, runnable pure Python.
try:
    get_ipython().run_line_magic("matplotlib", "inline") # noqa: F821
except NameError:
    # Not running inside Jupyter (e.g. plain script execution) - nothing to do.
    pass

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme() # apply seaborn's default theme EARLY so every plot inherits it

print("seaborn", sns.__version__, "| pandas", pd.__version__, "| numpy", np.__version__)

def load_dataset(name):
    """Load a built-in seaborn dataset with a friendly offline hint."""
    try:
        return sns.load_dataset(name)
    except Exception as exc: # network error, proxy, missing file...
        print(f"[offline hint] Could not load '{name}': {exc}")
        print(" Check your internet connection, or download the CSV from")
        print(" https://github.com/mwaskom/seaborn-data and use pd.read_csv.")
        return None
```



```
tips = load_dataset("tips")
penguins = load_dataset("penguins")
mpg = load_dataset("mpg")
flights = load_dataset("flights")
titanic = load_dataset("titanic")

for label, df in [("tips", tips), ("penguins", penguins), ("mpg", mpg),
                  ("flights", flights), ("titanic", titanic)]:
    print(f"{label}:>9: {df.shape if df is not None else 'NOT LOADED'}")

tips.head()
```

Common pitfall: `sns.set_theme()` resets *everything*, style, context, palette, and any manual `rc` overrides. Call it once near the top of the notebook, then make targeted adjustments afterwards; calling it in the middle silently throws away earlier tweaks.

2. Themes, styles, and contexts

Seaborn splits “look” into three knobs:

- **Style** controls the background furniture: grid, background color, spines. Options: `darkgrid` (default), `whitegrid`, `dark`, `white`, `ticks`. Grids help read values (`*grid`); clean styles help publication figures (`white`, `ticks`).
- **Context** scales fonts, line widths, and marker sizes for the output medium: `paper` < `notebook` (default) < `talk` < `poster`. Same plot, bigger presence.
- **Palette** controls colors (section 14).

`sns.set_theme()` sets all three at once. For temporary changes, use `sns.axes_style(...)` / `sns.plotting_context(...)` as context managers, perfect for side by side galleries like the ones below.

Gallery of the five built in styles. We draw the same small histogram on

```
## every panel inside a temporary axes_style so the change is local to the loop.
styles = ["darkgrid", "whitegrid", "dark", "white", "ticks"]
rng = np.random.default_rng(42)
sample = rng.normal(loc=0.0, scale=1.0, size=400)

fig, axes = plt.subplots(1, 5, figsize=(16, 3))
for ax, style_name in zip(axes.flat, styles):
    with sns.axes_style(style_name):
        sns.histplot(x=sample, bins=25, ax=ax)
    ax.set_title(style_name)
    ax.set_xlabel("")
    ax.set_ylabel("")
axes.flat[-1].set_visible(False) # 5 styles, 6 slots: hide the spare panel
```

```
plt.tight_layout()
plt.show()
```

Contexts scale typography for the destination medium. Same line, four sizes.

```
contexts = ["paper", "notebook", "talk", "poster"]
fig, axes = plt.subplots(1, 4, figsize=(16, 3.2))
x_line = np.linspace(0, 10, 100)
for ax, ctx in zip(axes.flat, contexts):
    with sns.plotting_context(ctx):
        ax.plot(x_line, np.sin(x_line))
        ax.set_title(ctx)
plt.tight_layout()
plt.show()
```

Pro tip: the minimal look many journals want is `sns.set_theme(style="ticks")` plus `sns.despine()` after plotting, no top or right spine, ticks only where data lives.

3. The two level API: axes level vs figure level functions

This is the concept that unlocks everything else in seaborn, so read it twice. Every seaborn plotting function belongs to exactly one of two levels:

Level	Returns	Accepts ax=?	Faceting	Examples
Axes level	a single matplotlib Axes	Yes, drops into any subplot grid	No (one panel per call)	scatterplot, lineplot, histplot, kdeplot, boxplot, violinplot, barplot, countplot, regplot, heatmap
Figure level	a seaborn FacetGrid (own Figure)	No , always makes a new figure	Yes, native col=, row=	relplot, displot, catplot, lmpplot, jointplot, pairplot

Rules of thumb:

- Figure level functions are *dispatchers*: `relplot(kind="scatter")` just calls `scatterplot` under the hood, but adds faceting and legend management.

- Use **axes level** when composing custom layouts (plt.subplots grids, dashboards) or layering multiple plot types on shared axes.
- Use **figure level** when exploring conditional relationships across facets, it is dramatically less code.
- Because figure level calls create their own window, assign them to a variable (g = ...) so you can keep tweaking via g.figure.

Proof of the two return types.

```
ax_scatter = sns.scatterplot(data=tips, x="total_bill", y="tip")
print("axes-level returns :", type(ax_scatter).__name__)

g_rel = sns.relplot(data=tips, x="total_bill", y="tip", col="time", height=3.2)
print("figure-level returns:", type(g_rel).__name__,
      "(wrapping a", type(g_rel.figure).__name__ + ")")
plt.close("all") # tidy up the demo windows
```

Axes level functions compose into arbitrary matplotlib layouts via ax=.

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.boxplot(data=tips, x="day", y="total_bill", ax=axes[0])
axes[0].set_title("boxplot on a chosen Axes")
sns.violinplot(data=tips, x="day", y="total_bill", ax=axes[1])
axes[1].set_title("violinplot beside it")
plt.tight_layout()
plt.show()
```

Pro tip: figure level functions accept height and aspect (width = height × aspect) instead of figsize. Each facet keeps those proportions however many columns you request.

4. Semantic mapping: hue, style, size, markers

Seaborn lets extra DataFrame columns drive visual channels, these are **semantics**:

- hue= color. Numeric hue gets a continuous gradient automatically; categorical hue cycles a discrete palette.
- style= dash patterns (lines) or marker shapes (points).
- size= area (points/lines), scaled via sizes=(min, max).
- markers=True / dashes=False fine tune line and marker combos.

Each semantic is one more dimension of your data encoded visually. Two or three semantics read comfortably; more becomes spaghetti.

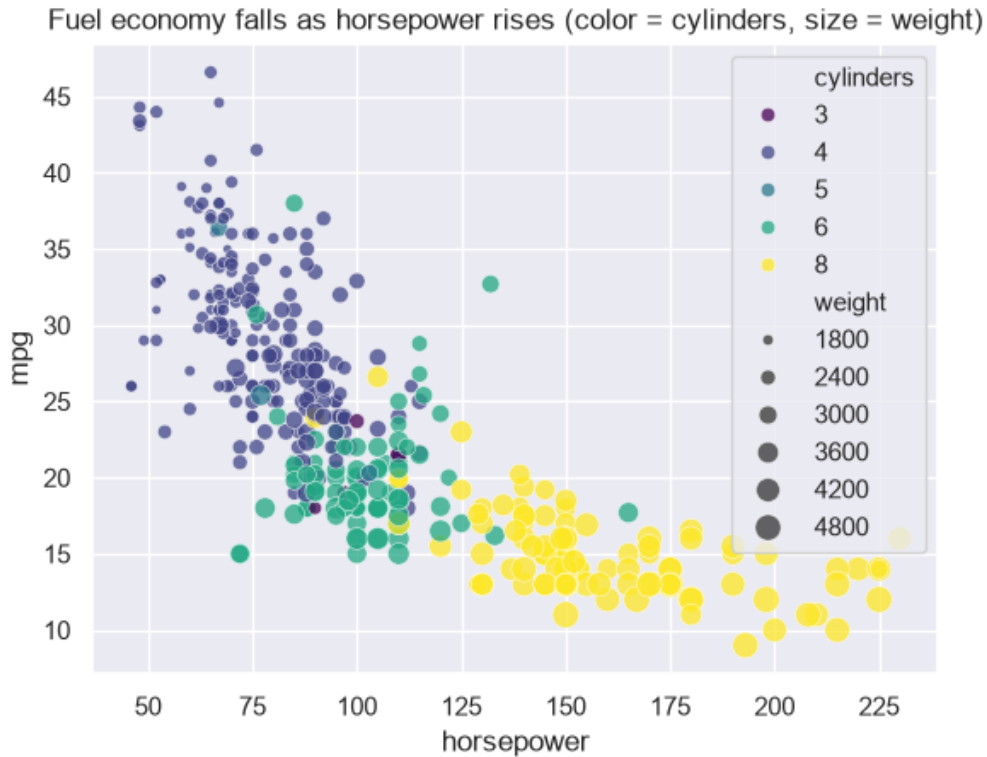


Figure 39: 4. Semantic mapping: hue, style, size, markers (ch05_seaborn_01 figure 1)

```
## hue + size on a scatter: cylinder count (numeric -> sequential ramp),
## weight mapped to point area. One call, three variables compared.
sns.scatterplot(
    data=mpg, x="horsepower", y="mpg",
    hue="cylinders", palette="viridis",
    size="weight", sizes=(15, 130),
    alpha=0.75,
)
plt.title("Fuel economy falls as horsepower rises (color = cylinders, size = weight)")
plt.show()
```

Lines with hue AND style: origin both colors the trend and changes the

```
## marker, so the chart survives grayscale printing too.
yearly = (
    mpg.groupby(["model_year", "origin"], as_index=False)["mpg"]
```

```

    .mean()
)
ax = sns.lineplot(
    data=yearly, x="model_year", y="mpg",
    hue="origin", style="origin",
    markers=True, dashes=False,
)
ax.set_title("Average mpg by model year and origin")
ax.set_xlabel("model year")
plt.show()

```

Common pitfall: stacking four semantics on one panel (hue, style, size, and facets) produces charts nobody can decode. If you need a fourth dimension, facet instead, section 13 shows how.

5. Line plots that think: aggregation and bootstrap confidence bands

Feed lineplot several y values per x and it quietly does statistics for you: it draws the **mean** at each x plus a **95% confidence interval** band computed by *bootstrapping*, resampling the observed values with replacement many times, re computing the mean each time, and taking the 2.5th/97.5th percentiles of those means. The band answers: "how sure are we about this average?"

Control it with the errorbar parameter:

- `errorbar=("ci", 95)`, bootstrap CI (the default).
- `errorbar=("sd", 1) / ("sd", 2)`, $\pm$ standard deviations (data spread).
- `errorbar=("pi", 90)`, percentile interval of the raw data.
- `errorbar=None`, turn off entirely (fast).

`err_style="band"` (default) or `"bars"` chooses the look; `n_boot` trades accuracy for speed.

Synthetic example: 60 days × 40 sensors measuring a drifting signal.

```

## Long-form data with repeats per x is exactly what triggers aggregation.
sensor_rng = np.random.default_rng(42)
days = np.repeat(np.arange(60), 40)
signal = 10 + 0.08 * days + sensor_rng.normal(0, 2.0, days.size)
readings = pd.DataFrame({"day": days, "value": signal})

ax = sns.lineplot(data=readings, x="day", y="value")
ax.set_title("Mean over 40 sensors per day, shaded = 95% bootstrap CI")
plt.show()

```

```

## Band vs bars, same data. Both encode the SAME statistic - only geometry differs.
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.lineplot(data=readings, x="day", y="value", err_style="band", ax=axes[0])
axes[0].set_title('err_style="band" (default)')

```

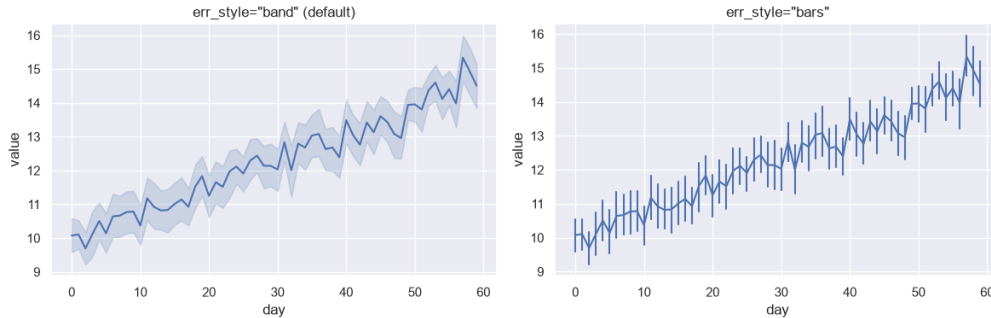


Figure 40: Synthetic example: 60 days $\times$ 40 sensors measuring a drifting signal. (ch05_seaborn_01 figure 2)

```
sns.lineplot(data=readings, x="day", y="value", err_style="bars", ax=axes[1])
axes[1].set_title('err_style="bars"')
plt.tight_layout()
plt.show()
```

Real data: flights has one observation per (year, month), so first

```
## aggregate to yearly totals, then plot. Drawing x=year with hue=month would
## stack 12 tangled lines on one panel - reserve that for heatmaps/facets.
flights_yearly = flights.groupby("year", as_index=False)["passengers"].sum()
ax = sns.lineplot(data=flights_yearly, x="year", y="passengers", marker="o")
ax.set_title("Total airline passengers per year (1949-1960)")
plt.show()
```

Common pitfall: if exactly one observation exists per x value, no band appears, there is nothing to aggregate. That is seaborn being honest, not a bug. Aggregate upstream or switch to a plain line.

Pro tip: on datasets with thousands of points per group set `n_boot=200` (or `errorbar=None`) while exploring; restore defaults for the final figure.

6. Histograms done right: histplot

The histogram chops the range into bins and counts. Its knobs matter:

- `bins=` an integer, or `binwidth=`/`binrange=` for explicit control.
- `stat=` what the bar height means: "count", "density" (area sums to 1), "probability" (fraction of observations).
- `kde=True` overlays a smooth density curve on the bars.
- With `hue=`, `multiple=` decides how groups coexist: "layer" (translucent overlap), "stack", "fill" (proportions to 100%), or "dodge" (side by side bars).

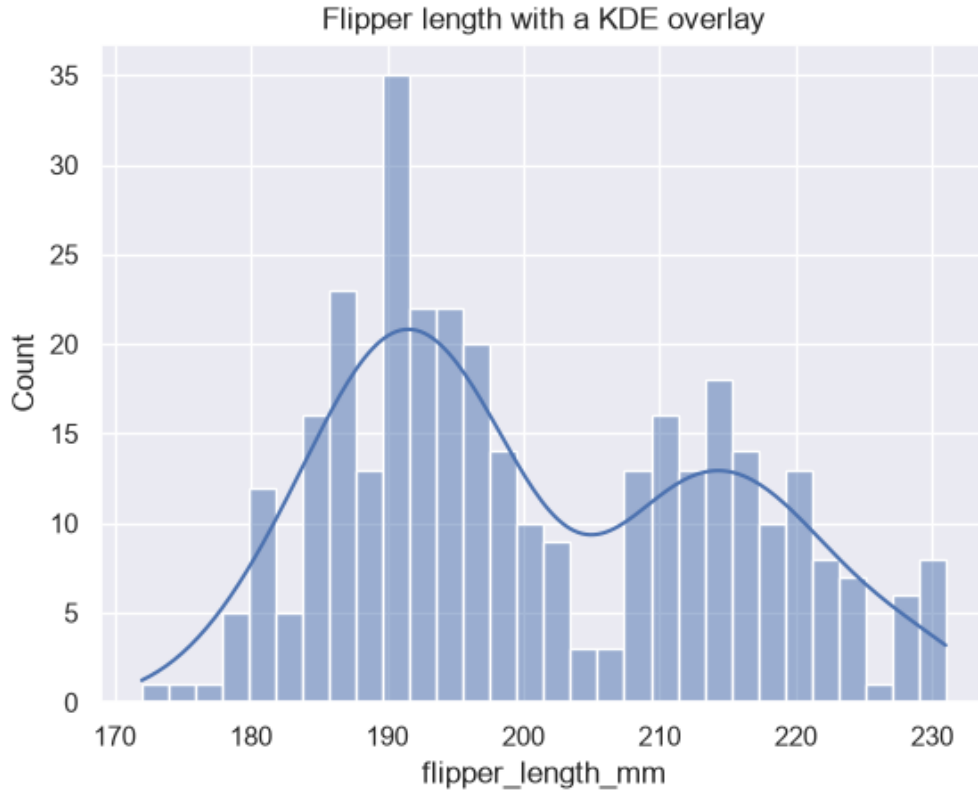


Figure 41: 6. Histograms done right: histplot (ch05_seaborn_01 figure 3)

```
sns.histplot(data=penguins, x="flipper_length_mm", bins=30, kde=True)
plt.title("Flipper length with a KDE overlay")
plt.show()
```

```
## The four ways hue groups can share a histogram panel.
multiples = ["layer", "stack", "fill", "dodge"]
fig, axes = plt.subplots(2, 2, figsize=(12, 7), sharey="row")
for ax, mode in zip(axes.flat, multiples):
    sns.histplot(
        data=penguins, x="body_mass_g", hue="species",
        multiple=mode, bins=25, ax=ax, legend=False,
    )
    ax.set_title(f'multiple="{mode}"')
plt.tight_layout()
plt.show()
```

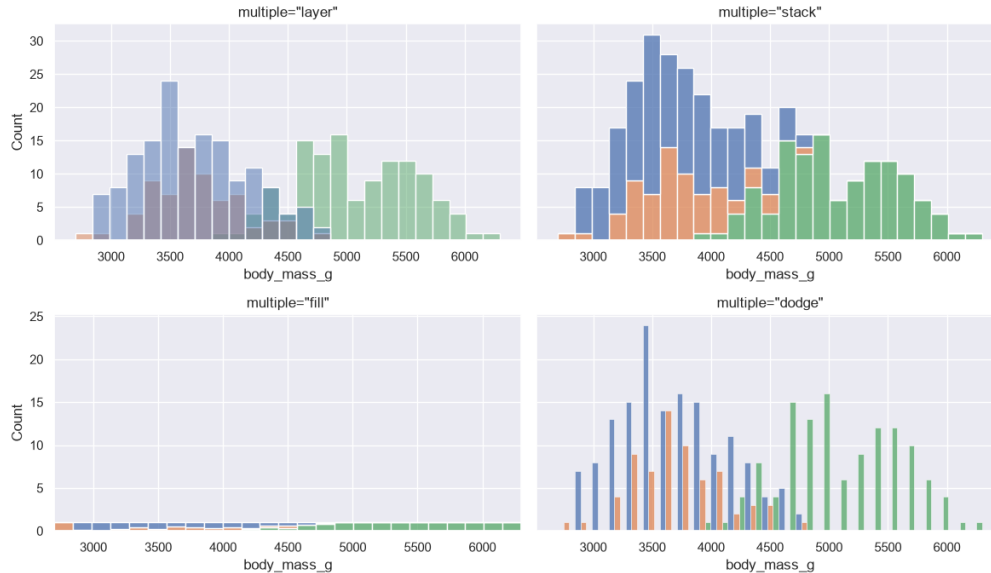


Figure 42: 6. Histograms done right: histplot (ch05_seaborn_01 figure 4)

Common pitfall: bin edges change the story. A histogram with `binwidth=50` can hide a dip that `binwidth=500` invents. Always try two or three bin widths before trusting a shape.

Pro tip: comparing group *shapes* fairly? Use `stat="density"`, `common_norm=False` so each species' curve integrates to 1 independently instead of shrinking by group size.

7. Kernel density estimates: kdeplot

A KDE replaces each data point with a small smooth bump and sums the bumps, a continuous version of the histogram with no arbitrary bin edges. Key knobs:

- `bw_adjust=` multiplies the bump width: `<1` follows the data closely (wiggly), `>1` smooths aggressively (may blur real features).
- `cut=` how far past the min/max the curve may extend (default 3 bandwidths; `cut=0` clamps to observed range, right choice for bounded quantities like weights or prices that cannot go negative).
- Two numeric axes give **2 D contours**, like a topographic map of density.

Bandwidth sweep on one species so the effect is unmistakable.

```
adelie = penguins[penguins["species"] == "Adelie"]
bw_values = [0.5, 1.0, 2.0]
```

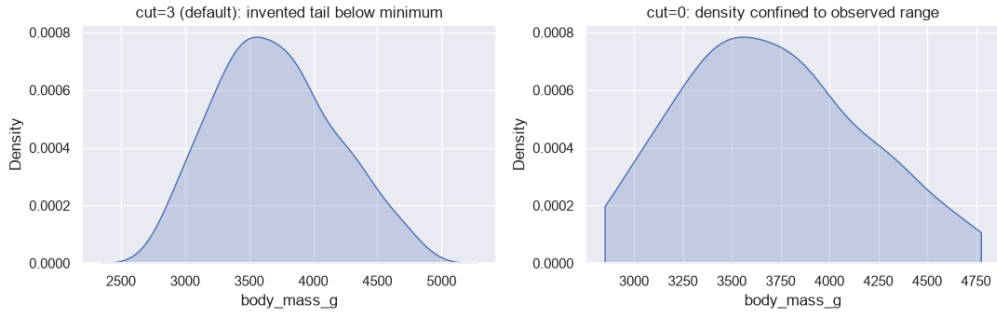



Figure 43: Bandwidth sweep on one species so the effect is unmistakable. (ch05_seaborn_01 figure 5)

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.6), sharey=True)
for ax, bw in zip(axes.flat, bw_values):
    sns.kdeplot(data=adelie, x="body_mass_g", bw_adjust=bw, fill=True, ax=ax)
    ax.set_title(f"bw_adjust={bw}")
plt.tight_layout()
plt.show()
```

```
## cut=0 stops the density from leaking below zero mass.
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
sns.kdeplot(data=adelie, x="body_mass_g", fill=True, ax=axes[0])
axes[0].set_title("cut=3 (default): invented tail below minimum")
sns.kdeplot(data=adelie, x="body_mass_g", fill=True, cut=0, ax=axes[1])
axes[1].set_title("cut=0: density confined to observed range")
plt.tight_layout()
plt.show()
```

```
## 2-D KDE: filled contour rings over bill dimensions of Gentoo penguins.
gentoo = penguins[penguins["species"] == "Gentoo"]
ax = sns.kdeplot(
    data=gentoo, x="bill_length_mm", y="bill_depth_mm",
    fill=True, cmap="crest", levels=10, thresh=0.05,
)
ax.set_title("Gentoo bill morphology - rings mark density levels")
plt.show()
```

Common pitfall: a KDE will happily paint density where no data can exist (negative prices, zero masses). Reach for `cut=0` whenever the quantity is physically bounded.

8. ECDF and rugs: honesty first distributions

The **empirical cumulative distribution function** plots, for every value x , the fraction of observations $\leq x$. Why it often beats a histogram:

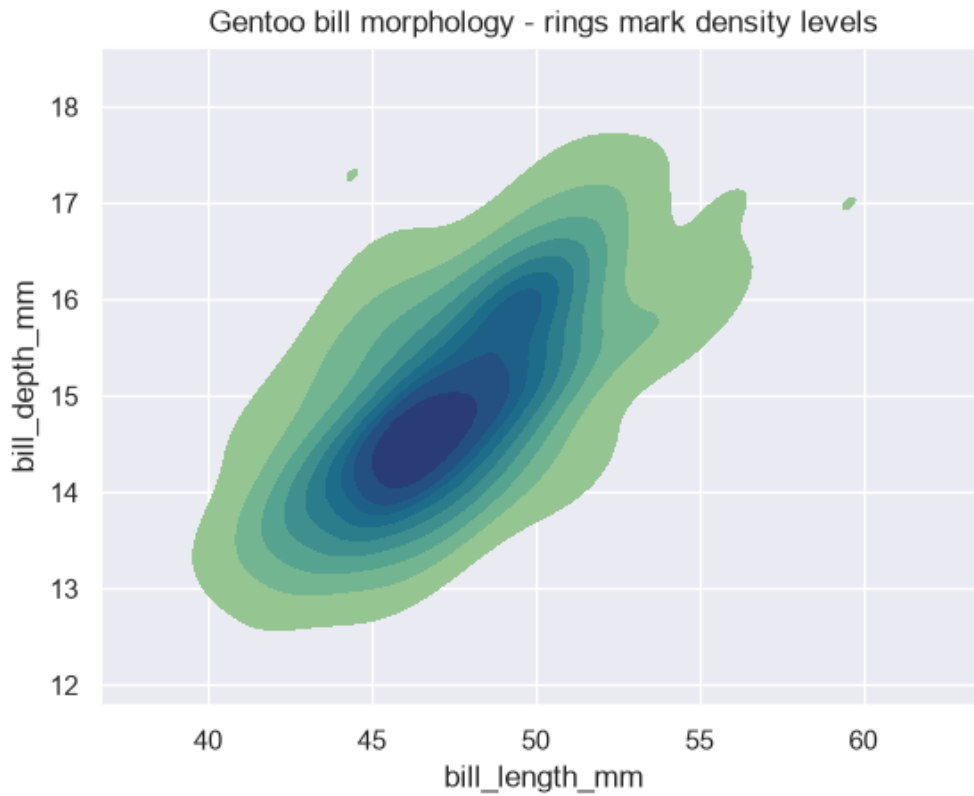


Figure 44: Bandwidth sweep on one species so the effect is unmistakable. (ch05_seaborn_01 figure 6)

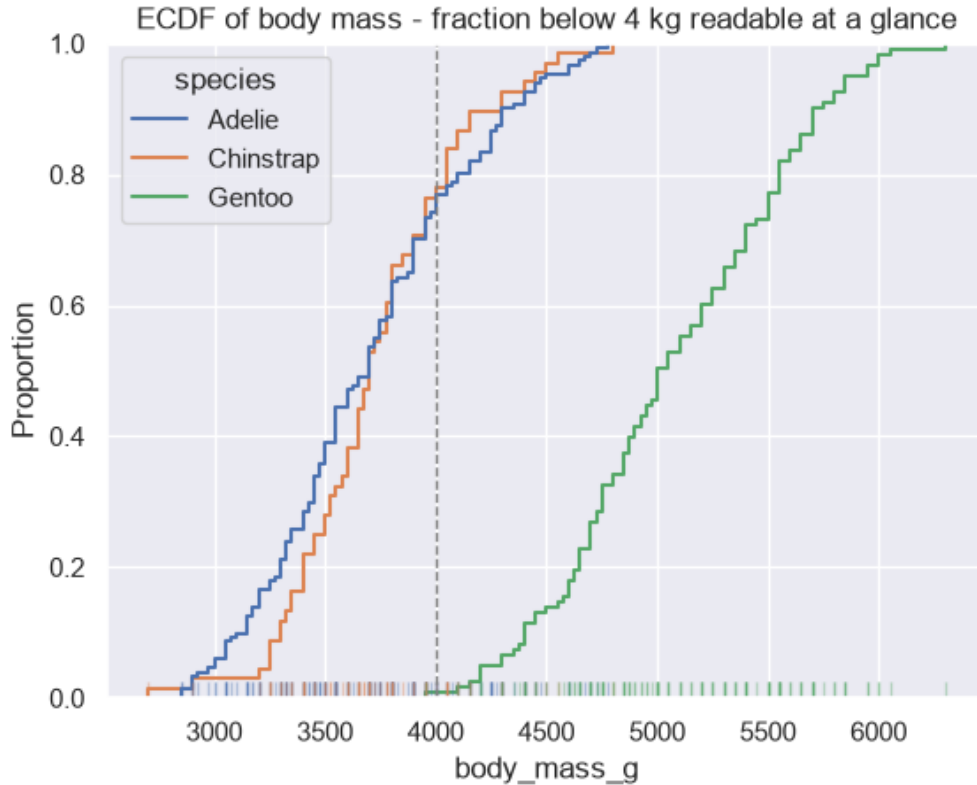


Figure 45: 8. ECDF and rugs: honesty first distributions (ch05_seaborn_01 figure 7)

- **No tuning parameters**, no bins, no bandwidth, nothing to fudge.
- Quantiles are read off directly: find 0.5 on y, drop to x the median.
- Multiple groups overlay cleanly and are easy to compare at any threshold.

A rugplot adds one small tick per observation along an axis, a 1 D scatter that shows exactly where the raw data sits.

```
ax = sns.ecdfplot(data=penguins, x="body_mass_g", hue="species")
sns.rugplot(data=penguins, x="body_mass_g", hue="species", ax=ax, alpha=0.35)
ax.axvline(4000, color="gray", ls="--", lw=1)
ax.set_title("ECDF of body mass - fraction below 4 kg readable at a glance")
plt.show()

below_4k = penguins.assign(under=lambda d: d["body_mass_g"] < 4000)
print(below_4k.groupby("species")["under"].mean().round(2))
```

Output:

```
species
Adelie      0.74
Chinstrap   0.76
Gentoo      0.01
Name: under, dtype: float64
```

Pro tip: any question phrased “*what fraction of X is below Y* ”, SLAs, thresholds, passing grades, is answered instantly by an ECDF and awkwardly by a histogram.

9. Boxplots: the five number summary, drawn

Anatomy recap (worth memorizing):

- **Box** spans the IQR (25th-75th percentiles); its inner line is the **median**.
- **Whiskers** extend to the most extreme points within $1.5 \times \text{IQR}$ of the box edges (change with `whis=`; e.g. `whis=(5, 95)` uses percentiles).
- **Fliers** (diamond dots) fall outside the whisker fence, candidate outliers, not errors.

Adding `hue=` dodges boxes side by side per group.

```
ax = sns.boxplot(data=tips, x="day", y="total_bill")
ax.set_title("Boxplot anatomy: median line, IQR box, whiskers, fliers")
## Point at the parts (coordinates approximate, purely illustrative).
ax.annotate("median", xy=(0, 18.4), xytext=(-0.45, 32),
            arrowprops=dict(arrowstyle="->", color="crimson"))
ax.annotate("flier", xy=(3, 50.8), xytext=(2.55, 44),
            arrowprops=dict(arrowstyle="->", color="crimson"))
plt.show()
```

Default 1.5*IQR whiskers vs percentile whiskers.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
sns.boxplot(data=tips, x="day", y="tip", whis=1.5, ax=axes[0])
axes[0].set_title("whis=1.5 (classic Tukey fence)")
sns.boxplot(data=tips, x="day", y="tip", whis=(5, 95), ax=axes[1])
axes[1].set_title("whis=(5, 95): whiskers at percentiles")
plt.tight_layout()
plt.show()
```

Common pitfall: whisker rules differ across software (Excel, matplotlib, seaborn all vary subtly). When publishing, state your whisker definition in the caption.

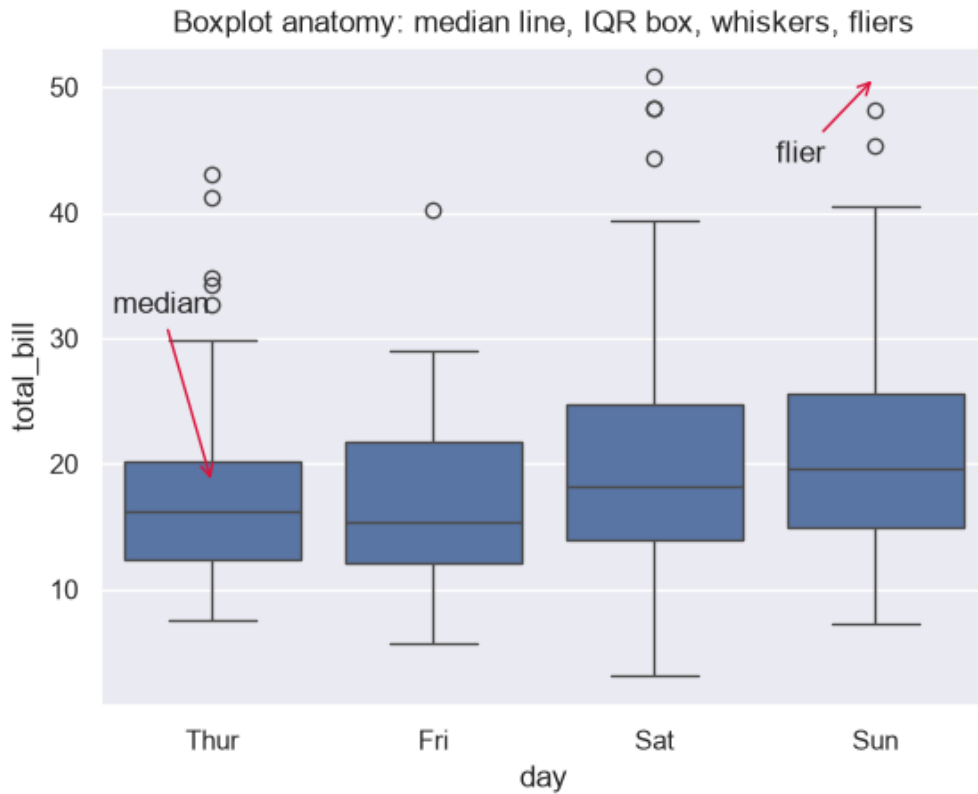


Figure 46: 9. Boxplots: the five number summary, drawn (ch05_seaborn_01 figure 8)

10. Violin and boxen: richer box summaries

A **violinplot** mirrors a KDE around the category axis: wide bulges are dense regions, narrow necks are sparse ones. Useful options:

- `inner=` what sits inside: "quart" (quartile lines), "box" (mini boxplot), "stick", "point", or None.
- `split=True` with a binary hue paints *both halves*, one group per side, halving the ink of a dodged pair.
- `density_norm=` (newer name for the older `scale=`) normalizes violin areas ("area", "count", "width").

For very large samples, **boxenplot** (letter value plot) draws nested boxes covering progressively smaller tail fractions, more quantile detail than a box, cheaper than a swarm.

```
ax = sns.violinplot(
    data=tips, x="day", y="total_bill",
    hue="time", split=True, inner="quart", gap=0.15,
)
ax.set_title("Split violins: lunch on the left half, dinner on the right")
plt.show()
```

Diamonds is ~54k rows: sample deterministically for snappy iteration.

```
diamonds = load_dataset("diamonds")
if diamonds is not None:
    dia_small = diamonds.sample(5000, random_state=42)
    ax = sns.boxenplot(data=dia_small, x="cut", y="carat")
    ax.set_title("Boxenplot: nested boxes reveal tail structure for large n")
else:
    print("Skipping boxen demo - dataset unavailable.")
plt.show()
```

Pro tip: violins imply smoothing; with fewer than ~50 observations per group they can suggest shapes that are not really there. Small *n* use a boxplot or strip/swarm instead.

11. Strip and swarm: showing every observation

Aggregates hide individual points; sometimes the points ARE the message.

- **stripplot** scatters points with random horizontal **jitter** (`jitter=0.25`) so coincident values separate visually.
- **swarmplot** uses a repulsion algorithm so points never overlap, a mini distribution emerges from the outline itself. Cost: much slower on big data (roughly quadratic in points per category).



Figure 47: 10. Violin and boxen: richer box summaries (ch05_seaborn_01 figure 9)

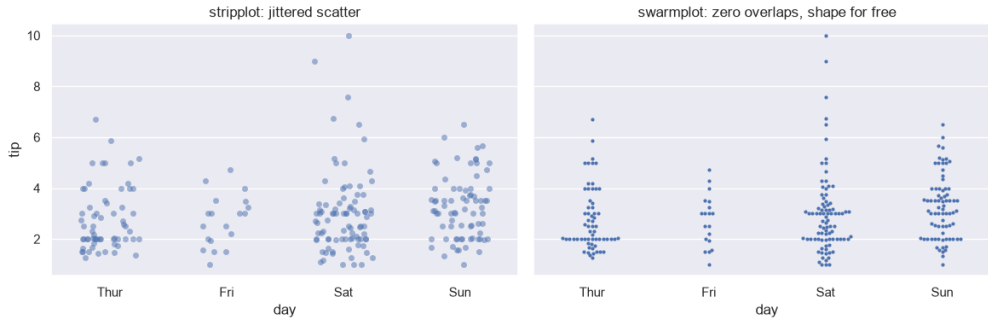


Figure 48: 11. Strip and swarm: showing every observation (ch05_seaborn_01 figure 10)

The classic power combo: a pale violin behind dark swarming points.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
sns.stripplot(data=tips, x="day", y="tip", jitter=0.25, alpha=0.5, ax=axes[0])
axes[0].set_title("stripplot: jittered scatter")
sns.swarmplot(data=tips, x="day", y="tip", size=3, ax=axes[1])
axes[1].set_title("swarmplot: zero overlaps, shape for free")
plt.tight_layout()
plt.show()
```

```
ax = sns.violinplot(data=tips, x="day", y="tip", inner=None, color="#ddddd")
sns.swarmplot(data=tips, x="day", y="tip", size=3, color="#333333", ax=ax)
ax.set_title("Violin (context) + swarm (evidence): best of both")
plt.show()
```

Common pitfall: swarmplot crawls beyond a few thousand points per category and may fail to place them all. Subsample first, or fall back to stripplot.

12. Aggregate views: countplot, barplot, pointplot

Three functions summarize categories, each answering a different question:

- **countplot**, *how many rows?* Pure frequencies; needs only *x*.
- **barplot**, *what is the average of some measure?* Aggregates *y* per group (estimator=*np.mean* by default) and draws a **bootstrap CI** as the error bar. Swap with *estimator=np.median*; control the bar with *errorbar=("sd", 1)* etc. (The legacy *ci=95* argument maps to *errorbar=("ci", 95)*.)
- **pointplot**, the same estimate as dots connected within hue levels; slopes make **interactions** obvious (non parallel lines = effect depends on the other factor).

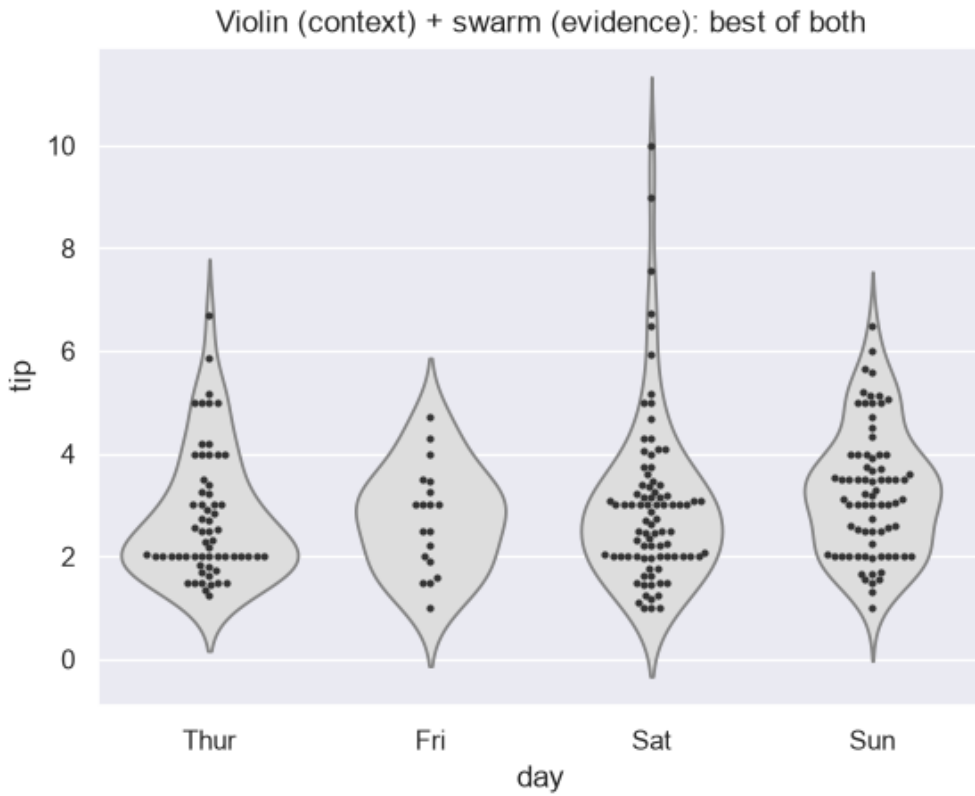


Figure 49: 11. Strip and swarm: showing every observation (ch05_seaborn_01 figure 11)

Frequency ordering trick: sort classes by row count using `value_counts().index`.

```
class_order = titanic["class"].value_counts().index.tolist()
ax = sns.countplot(data=titanic, x="class", hue="sex", order=class_order)
ax.set_title("Titanic passengers per class (bars ordered by frequency)")
plt.show()
```

Mean vs median estimator; note the errorbar meaning changes with it.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
sns.barplot(data=tips, x="day", y="tip", errorbar=("ci", 95), ax=axes[0])
axes[0].set_title("estimator=mean (default), 95% bootstrap CI")
sns.barplot(
    data=tips, x="day", y="tip",
    estimator=np.median, errorbar=("sd", 1), ax=axes[1],
)
axes[1].set_title("estimator=median, ±1 SD of the data")
plt.tight_layout()
plt.show()
```

Interaction view: does the day of week effect on tipping differ between

```
## smokers and non-smokers? Parallel lines would mean "no interaction".
ax = sns.pointplot(data=tips, x="day", y="tip", hue="smoker", dodge=0.35)
ax.set_title("Pointplot interaction: day × smoker (non-parallel lines)")
plt.show()
```

Common pitfall: barplot error bars are the uncertainty **of the mean** (a bootstrap CI), *not* the spread of individual customers. Say which one you drew, or readers will misread the chart.

Pro tip: seaborn starts bar charts at zero by default, keep it that way; truncated bars exaggerate differences.

13. catplot: faceting made trivial

`catplot` is the figure level hub for every categorical plot. Pass `kind="bar"/"box"/"violin"/"strip"/"swarm"/"point"/"count"` plus optional `col=/row=` to build a facet grid in one call. It returns a `FacetGrid`, so all grid tweaks (`g.set_axis_labels`, `g.figure.suptitle`, ...) apply.

```
g = sns.catplot(
    data=tips, kind="bar",
    x="day", y="total_bill", col="time", row="sex",
```

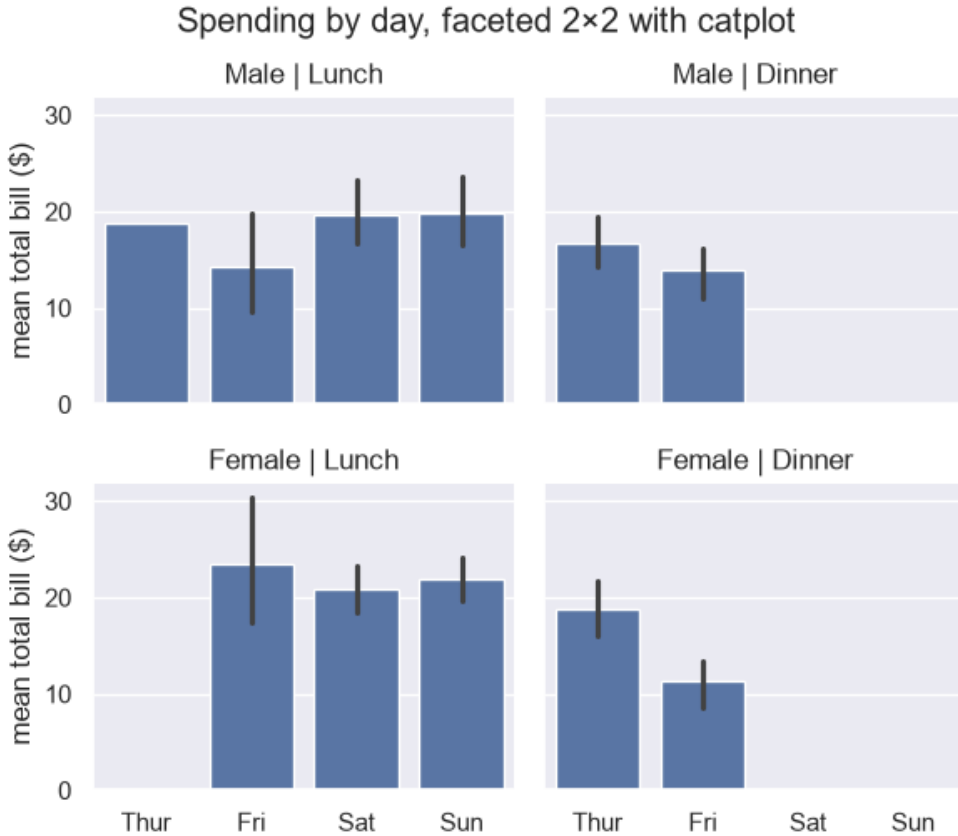


Figure 50: 13. catplot: faceting made trivial (ch05_seaborn_01 figure 12)

```
height=2.6, aspect=1.15, errorbar=("ci", 95),
)
g.set_axis_labels("", "mean total bill ($)")
g.set_titles(row_template="{row_name}", col_template="{col_name}")
g.figure.suptitle("Spending by day, faceted 2×2 with catplot", y=1.03)
plt.show()
```

Common pitfall: figure level functions ignore `ax=` and open a new figure on every call. Assign the result and reuse `g`; close stale windows with `plt.close("all")` during exploration.

Pro tip: swapping `kind="bar"` for `kind="box"` re renders the whole facet layout instantly, cheap way to test which encoding tells the story.

14. Palettes: speaking fluent color

Choose palettes by *what the color means*:

Data type	Palette family	Good choices
Categories (no order)	Qualitative	deep (default), muted, colorblind, Set2, tab10
Ordered magnitude	Sequential	crest, viridis, rocket, mako, flare, Blues
Deviation around a midpoint	Diverging	coolwarm, RdBu_r, vlag, Spectral

Apply per call with `palette="..."`, or globally with `sns.set_palette(...)`. Add `as_cmap=True` wherever a colormap object is expected (heatmaps, 2 D KDEs). Prefer "colorblind" for anything public, roughly 1 in 12 men has some color vision deficiency.

Qualitative previews.

```
for name in ["deep", "muted", "colorblind", "Set2"]:  
    sns.palplot(sns.color_palette(name))  
    plt.title(name)  
plt.show()
```

Sequential ramps shown as gradients.

```
gradient = np.linspace(0, 1, 256).reshape(1, -1)  
fig, axes = plt.subplots(4, 1, figsize=(6, 4.5))  
for ax, name in zip(axes.flat, ["crest", "viridis", "rocket", "mako"]):  
    ax.imshow(gradient, aspect="auto", cmap=sns.color_palette(name, as_cmap=True))  
    ax.set_yticks([])  
    ax.set_xticks([])  
    ax.set_ylabel(name, rotation=0, ha="right", va="center")  
plt.tight_layout()  
plt.show()
```

Diverging palettes need a MEANINGFUL center, here a synthetic correlation

```
## matrix centered at 0 (positive = warm, negative = cool).  
corr_rng = np.random.default_rng(7)  
base = corr_rng.normal(size=(80, 5))  
synth_corr = np.corrcoef(base, rowvar=False)  
labels = ["A", "B", "C", "D", "E"]
```

```
sns.heatmap(
    pd.DataFrame(synth_corr, index=labels, columns=labels),
    cmap="coolwarm", center=0, vmin=-1, vmax=1,
    annot=True, fmt=".2f",
)
plt.title("Diverging palette centered at zero")
plt.show()
```

Global vs per call palettes.

```
sns.set_palette("colorblind")          # global from here on
sns.barplot(data=tips, x="day", y="tip")
plt.title("Global palette switched to 'colorblind'")
plt.show()

sns.set_palette("deep")                # restore the default
ax = sns.barplot(data=tips, x="day", y="tip", palette="flare", hue="day", legend=False)
ax.set_title("Per-call override with palette='flare'")
plt.show()
```

Common pitfall: rainbow style maps (jet and friends) distort perception with fake visual boundaries, and red/green pairs exclude colorblind readers. Stick to the families above.

Summary & key takeaways

- Seaborn = **dataset oriented + statistics aware + beautiful defaults**, driving matplotlib under the hood.
- Memorize the **two level API**: axes level functions take `ax=` and compose into custom layouts; figure level functions own their figure and provide `col=`/`row=` faceting.
- `lineplot` aggregates repeats into means with **bootstrap CIs** (`errorbar=`, `err_style=`, `n_boot=`).
- Distributions: tune histograms carefully (`bins`, `stat`, `multiple`), mind KDE bandwidth and `cut=0` for bounded data, prefer ECDFs for threshold questions.
- Categorical toolbox: `box` (five number), `violin` (density), `boxen` (large `n` quantiles), `strip/swarm` (raw evidence), `count/bar/point` (aggregates with honest uncertainty).
- `catplot` wraps all of it with effortless faceting.
- Pick palettes by meaning, qualitative, sequential, diverging, and make "colorblind" your public facing default.

Next up: **02_seaborn_advanced**, regression plots, heatmaps & clustering, joint/pair grids, a `FacetGrid` masterclass, theme fine tuning, matplotlib interop, and saving publication ready figures.

6.2 Seaborn, Seaborn Advanced

MLCourse · Data Science Foundations · 04_seaborn

Foundations down; time to operate like a pro. This notebook covers seaborn's statistical heavyweights, regression plots, matrix heatmaps, hierarchical clustering, and the joint/pair grid family, then goes deep on the FacetGrid engine for fully custom multi panel figures. We finish with theme fine tuning, seamless matplotlib interop, saving figures for reports, and performance etiquette for large datasets.

Notebook 01 introduced the two level API (axes level vs figure level); we use that vocabulary constantly here.

What you'll learn

- **Regression plots:** regplot, polynomial/logistic/robust/logx fits, CI bands, faceted lmpplot, and residual diagnostics with residplot.
- **Matrix plots:** publication grade correlation heatmaps with masks and annotations, plus one line hierarchical clustering via clustermap.
- The **JointGrid family:** jointplot with scatter/kde/reg/hex/resid kinds.
- **pairplot / PairGrid:** scatterplot matrices, custom per triangle plots.
- A **FacetGrid masterclass:** map_dataframe with custom functions, reference lines, titles, legends, and axis sharing control.
- Theme fine tuning: styles, contexts, despinning, rc overrides.
- **Matplotlib interop:** annotations, second layers, resizing figure level output post hoc, moving legends.
- Saving figures (savefig, dpi, SVG) and large data etiquette.

1. Setup

Same Jupyter safe magic pattern as notebook: this file stays pure Python

```
## while still enabling inline rendering inside notebooks.
try:
    get_ipython().run_line_magic("matplotlib", "inline") # noqa: F821
except NameError:
    pass

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme() # consistent look across every plot in this notebook
```

```
def load_dataset(name):
    """Load a built-in seaborn dataset with a friendly offline hint."""
    try:
        return sns.load_dataset(name)
    except Exception as exc:
        print(f"[offline hint] Could not load '{name}': {exc}")
        print(" Check your connection or fetch the CSV from")
        print(" https://github.com/mwaskom/seaborn-data and use pd.read_csv.")
        return None

tips = load_dataset("tips")
penguins = load_dataset("penguins")
mpg = load_dataset("mpg")
print("loaded:", {k: v.shape for k, v in
                  [("tips", tips), ("penguins", penguins), ("mpg", mpg)]})

tips.head()
```

2. Regression plots: fitting trends with uncertainty

regplot is axes level and draws three things at once: the scatter, an ordinary least squares line, and a shaded **95% confidence band** around it (bootstrapped resamples of the data). The fit family is selectable:

- `order=2`, polynomial (quadratic) fit for curved relationships.
- `logistic=True`, S curve fit for binary outcomes (pass/fail, buy/bounce).
- `robust=True`, downweights outliers (slower; worth knowing it exists).
- `logx=True`, fits $y \sim \log(x)$: straightens power law ish skewed data.
- `ci=None`, skip the band; dramatically faster on large data.

Style hooks: `scatter_kws=` and `line_kws=` pass dicts straight through to matplotlib.

Linear vs quadratic on the same data: total_bill vs tip bends slightly.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
sns.regplot(data=tips, x="total_bill", y="tip", ax=axes[0],
            scatter_kws={"alpha": 0.45}, line_kws={"color": "crimson"})
axes[0].set_title("order=1 (default linear)")
sns.regplot(data=tips, x="total_bill", y="tip", order=2, ax=axes[1],
            scatter_kws={"alpha": 0.45}, line_kws={"color": "crimson"})
axes[1].set_title("order=2 captures the curvature")
plt.tight_layout()
plt.show()
```

Logistic fit needs the optional statsmodels dependency; to keep this course

```
## dependency-light we DEMO the S-curve idea with a low-order polynomial instead,
## and note that `logistic=True` becomes available after `pip install statsmodels`.
log_rng = np.random.default_rng(42)
study_hours = np.sort(log_rng.uniform(0, 10, 220))
prob_pass = 1 / (1 + np.exp(-(study_hours - 5)))          # true sigmoid
passed = log_rng.binomial(1, prob_pass)
exam = pd.DataFrame({"hours": study_hours, "passed": passed})

ax = sns.regplot(data=exam, x="hours", y="passed", order=3,
                 y_jitter=0.03, scatter_kws={"alpha": 0.35},
                 line_kws={"color": "darkgreen"})
ax.set_title("Binary outcome + order=3 bend (install statsmodels for logistic=True)")
plt.show()
## robust=True exists too - same call with robust=True downweights outliers,
## at a noticeable compute cost. Try it when a few extreme points drag the line.
```

```
## logx demo on heavily right-skewed data: linear fit misses, logx nails it.
skew_rng = np.random.default_rng(0)
x_skew = skew_rng.lognormal(mean=3.0, sigma=1.0, size=300)    # long tail
y_skew = 20 + 8 * np.log(x_skew) + skew_rng.normal(0, 4, 300)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
sns.regplot(x=x_skew, y=y_skew, ax=axes[0],
            scatter_kws={"alpha": 0.4}, line_kws={"color": "navy"})
axes[0].set_title("plain linear fit on skewed x")
sns.regplot(x=x_skew, y=y_skew, logx=True, ax=axes[1],
            scatter_kws={"alpha": 0.4}, line_kws={"color": "darkred"})
axes[1].set_title("logx=True: the relationship straightens out")
plt.tight_layout()
plt.show()
```

Pro tip: during exploration on big datasets pass `ci=None`, the bootstrap behind the band is by far the most expensive part of `regplot`.

3. Implot: faceted regression

`lmplot` is the figure level wrapper over `regplot`: add `hue=/col=/row=` and seaborn fits *and draws* a separate regression per facet, perfect for asking “does the trend differ across groups?”

```
g = sns.lmplot(
    data=mpg, x="horsepower", y="mpg",
    col="origin", hue="origin",
    height=3.2, scatter_kws={"alpha": 0.5},
)
g.set_axis_labels("horsepower", "mpg")
```

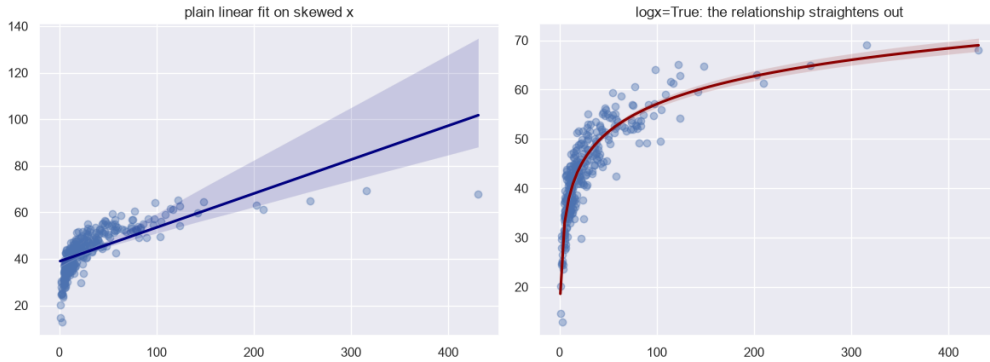



Figure 51: Logistic fit needs the optional statsmodels dependency; to keep this course (ch05_seaborn_02 figure 1)

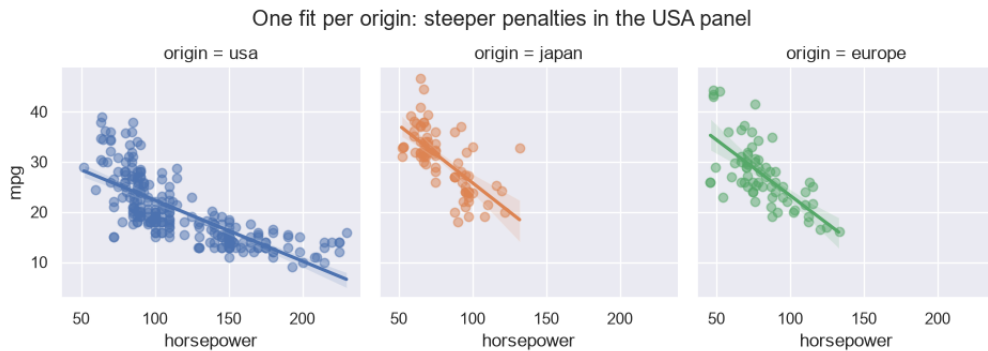


Figure 52: 3. Implot: faceted regression (ch05_seaborn_02 figure 2)

```
g.figure.suptitle("One fit per origin: steeper penalties in the USA panel", y=1.05)
plt.show()
```

Common pitfall: tiny facets produce wild, over confident lines. Check group sizes (`df["origin"].value_counts()`) before trusting a slope.

4. residplot: interrogating the fit

A residual is $y_{\text{actual}} - y_{\text{predicted}}$. If your model captured everything systematic, residuals are structureless noise centered on zero, a flat, even band. Any visible pattern means signal was left behind:

- Curvature the relationship isn't linear (raise order).
- Funnel shape variance grows with the mean (transform y, e.g. log).
- Waves seasonality or missing predictors.



Figure 53: 4. residplot: interrogating the fit (ch05_seaborn_02 figure 3)

```
lin_rng = np.random.default_rng(11)
n_pts = 180
x_lin = np.linspace(0, 10, n_pts)

## Case: correctly specified linear model -> boring, flat residuals.
y_good = 3 + 2 * x_lin + lin_rng.normal(0, 2.5, n_pts)

## Case: quadratic reality forced through a linear model -> leftover curve.
y_quad_true = 3 + 2 * x_lin - 1.1 * x_lin**2 + lin_rng.normal(0, 3, n_pts)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

## seaborn's lowess=True needs the optional `statsmodels` package; we draw the
## same diagnostic with a dependency-free rolling-mean smoother instead.
for ax, y_vals, color, title in [
    (axes[0], y_good, "seagreen", "Healthy model: smooth track hugs zero"),
    (axes[1], y_quad_true, "firebrick", "Misspecified: systematic arc = curvature"),
]:
    # residuals of the default LINEAR fit:
    fitted = np.polyval(np.polyfit(x_lin, y_vals, 1), x_lin)
    resid = y_vals - fitted
    sns.residplot(x=x_lin, y=y_vals,
                  scatter_kws={"alpha": 0.5}, ax=ax)          # dots only
    idx = np.argsort(x_lin)                                   # sort for the smoother
    smooth = pd.Series(resid[idx]).rolling(25, center=True).mean()
    ax.plot(np.asarray(x_lin)[idx], smooth, color=color, lw=2.6)
    ax.set_title(title)

plt.tight_layout()
plt.show()

## A flat colored track = healthy fit; the red arc IS the diagnosis: raise the
## polynomial order and re-check. (Install statsmodels to use built-in lowess.)
```

5. Matrix heatmaps + a full correlation workflow

heatmap renders any 2 D numeric table as colored cells. Key options:

- `annot=True`, `fmt=".2f"` prints values in cells; `annot_kws={"fontsize": 8}` styles the numbers.
- `linewidths=`, `linecolor=` separates cells visually.
- `cmap=` choose by semantics: sequential for magnitude (mako), **diverging centered on a meaningful midpoint** for signed values (`vlag`, `coolwarm`, `center=0`).
- `mask=` hides cells, correlation matrices are symmetric, so mask the upper triangle and let readers focus.
- `square=True` keeps cells square; `vmin/vmax` pin the color scale.

Below: the complete workflow on penguin measurements, compute correlations, mask redundancy, annotate, then extract and interpret the strongest pairs.

```
num_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
corr = penguins[num_cols].corr(numeric_only=True)

mask = np.triu(np.ones_like(corr, dtype=bool), k=1) # hide above-diagonal

fig, ax = plt.subplots(figsize=(6.5, 5.2))
sns.heatmap(
    corr, mask=mask, annot=True, fmt=".2f",
    cmap="vlag", center=0, vmin=-1, vmax=1,
    linewidths=0.5, linecolor="white",
    square=True, annot_kws={"fontsize": 9}, cbar_kws={"shrink": 0.8}, ax=ax,
)
ax.set_title("Penguin morphometrics: lower-triangle correlations")
plt.tight_layout()
plt.show()

## Extract the strongest absolute pairs programmatically.
pairs = (
    corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool)) # keep each pair once
    .stack()
    .rename("r")
    .reset_index()
    .sort_values("r", key=np.abs, ascending=False)
)
print(pairs.head(4).to_string(index=False))
```

Output:

level_0	level_1	r
flipper_length_mm	body_mass_g	0.871202
bill_length_mm	flipper_length_mm	0.656181
bill_length_mm	body_mass_g	0.595110
bill_depth_mm	flipper_length_mm	-0.583851

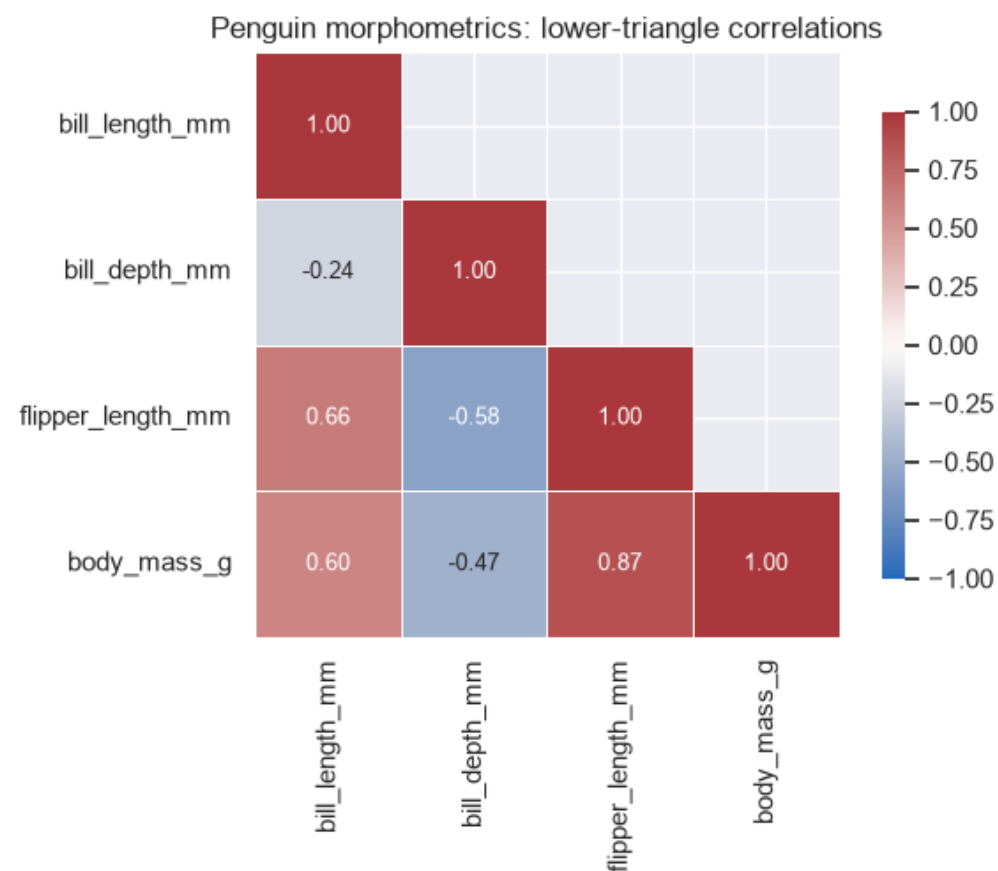


Figure 54: 5. Matrix heatmaps + a full correlation workflow (ch05_seaborn_02 figure 4)

Reading the heatmap: flipper length and body mass move together strongly (~ 0.87), bigger penguins have longer flippers. Bill depth correlates *negatively* with bill length (~ -0.24 overall) largely because the large Gentoo species has long but shallow bills, a first taste of Simpson's paradox worth checking per species.

Common pitfall: Pearson's r only measures *linear* association. Plot the scatter before believing $r \approx 0$ means "no relationship" (method="kendall" or "spearman" handles monotonic but nonlinear cases).

Pro tip: diverging palettes without center= are lying to you, always anchor them at the value that means "nothing happening" (usually 0).

6. clustermap: clustered heatmap one liner

clustermap wraps heatmap and adds **hierarchical clustering**: rows and columns are reordered so similar ones sit together, and dendrograms show the merge tree. It is unsupervised, no species labels needed, yet structure pops out. We sneak row_colors= in to color rows by true species and see how well clustering recovers them.

```
iris = load_dataset("iris")
if iris is not None:
    iris_sample = iris.sample(60, random_state=42)
    species_palette = dict(zip(["setosa", "versicolor", "virginica"],
                              sns.color_palette("Set2", 3)))

    cg = sns.clustermap(
        iris_sample.iloc[:, :4],          # the four measurements only
        cmap="mako", figsize=(7, 6),
        row_colors=iris_sample["species"].map(species_palette),
        cbar_kws={"label": "z-ish scale (cm)"},
    )
    cg.figure.suptitle("clustermap: similar flowers auto-group together", y=1.02)
    plt.show()
else:
    print("Skipping clustermap demo - iris unavailable.")
```

Each dendrogram arm joins the two most similar items/clusters first (average linkage by default). Setosa separates cleanly; versicolor and virginica overlap, exactly what PCA style analyses report too.

Pro tip: features on wildly different scales? Standardize first (standard_scale=1 per column, or z_score=1) so big units don't dominate the distance metric.

7. jointplot: bivariate + marginals

A scatter alone hides *why* points cluster; marginal distributions reveal whether clusters differ along one axis, both, or neither. jointplot (figure level) pairs a central plot with histograms/KDEs on the margins. Switch the centerpiece with kind=:

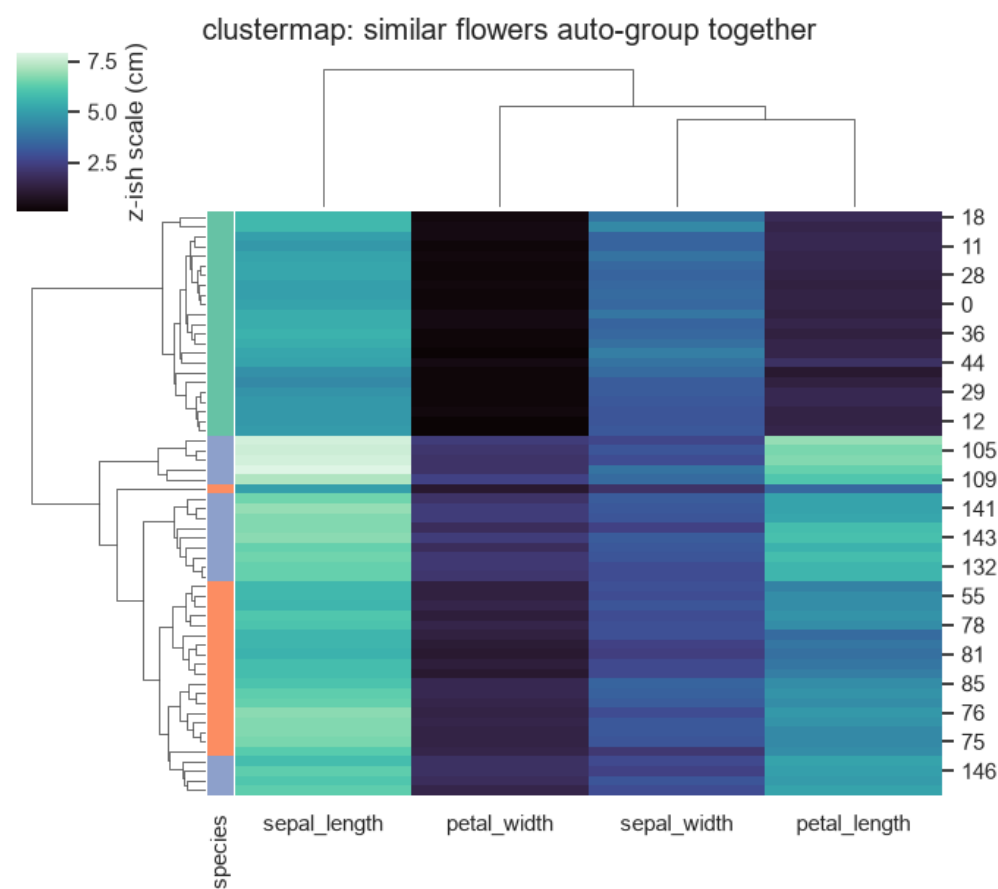


Figure 55: 6. clustermap: clustered heatmap one liner (ch05_seaborn_02 figure 5)

kind	central plot	best for
"scatter" (default)	points	moderate n, spotting groups
"kde"	2 D density contours	smooth overview, many points
"hex"	hexagonal binning counts	very large n
"reg"	scatter + linear fit + CI	quick trend check
"resid"	residuals of linear fit	diagnosing the trend

`marginal_ticks=True` shows tick labels on the margin axes.

```
bill = penguins.dropna(subset=["bill_length_mm", "bill_depth_mm"])

j1 = sns.jointplot(data=bill, x="bill_length_mm", y="bill_depth_mm",
                  hue="species", height=5.5)
j1.figure.suptitle('kind="scatter": three species, three clouds', y=1.02)
plt.show()

j2 = sns.jointplot(data=bill, x="bill_length_mm", y="bill_depth_mm",
                  kind="kde", fill=True, height=5.5, marginal_ticks=True)
j2.figure.suptitle('kind="kde": density contours + KDE marginals', y=1.02)
plt.show()

j3 = sns.jointplot(data=bill, x="bill_length_mm", y="bill_depth_mm",
                  kind="reg", height=5.5)
j3.figure.suptitle('kind="reg": quick linear-trend sanity check', y=1.02)
plt.show()
plt.close("all") # exploration produced several windows - tidy up
```

Common pitfall: with tens of thousands of points, plain scatters become solid ink. That is the cue for `kind="hex"` (next exercise) or `kind="kde"`.

8. pairplot: the scatterplot matrix

`pairplot` renders every pairwise scatter plus per variable diagonals, the fastest way to eyeball separability of groups across all dimensions. Key options:

- `hue=` colors everything by class (legend included).
- `diag_kind="kde"` smooths diagonals (vs default histogram).
- `corner=True` drops the redundant mirrored upper triangle, essential once you exceed ~5 variables.
- `vars=[...]` selects a subset of columns.
- `plot_kws={"alpha": ..., "s": ...}` fights overplotting.

```
peng_clean = penguins.dropna(subset=num_cols + ["species"])
pp = sns.pairplot(
    peng_clean,
```



Figure 56: 7. jointplot: bivariate + marginals (ch05_seaborn_02 figure 6)

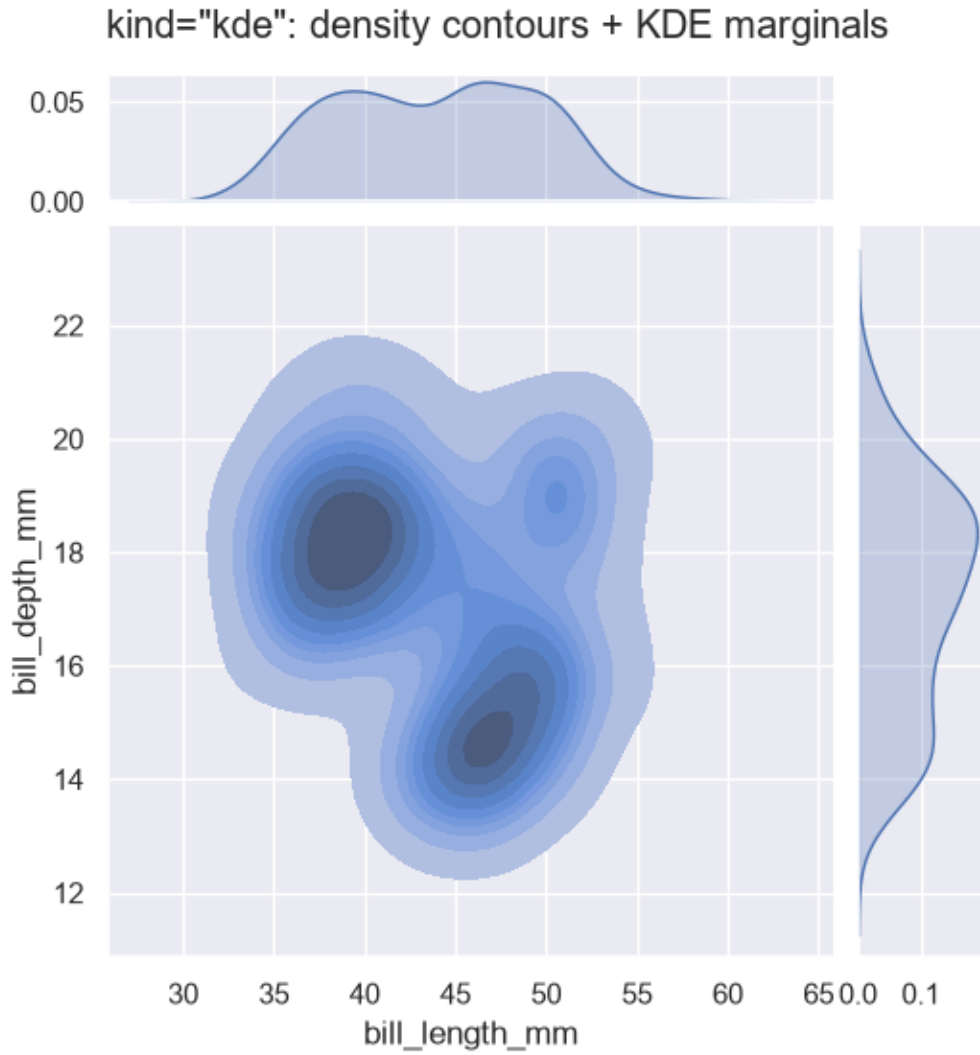


Figure 57: 7. jointplot: bivariate + marginals (ch05_seaborn_02 figure 7)

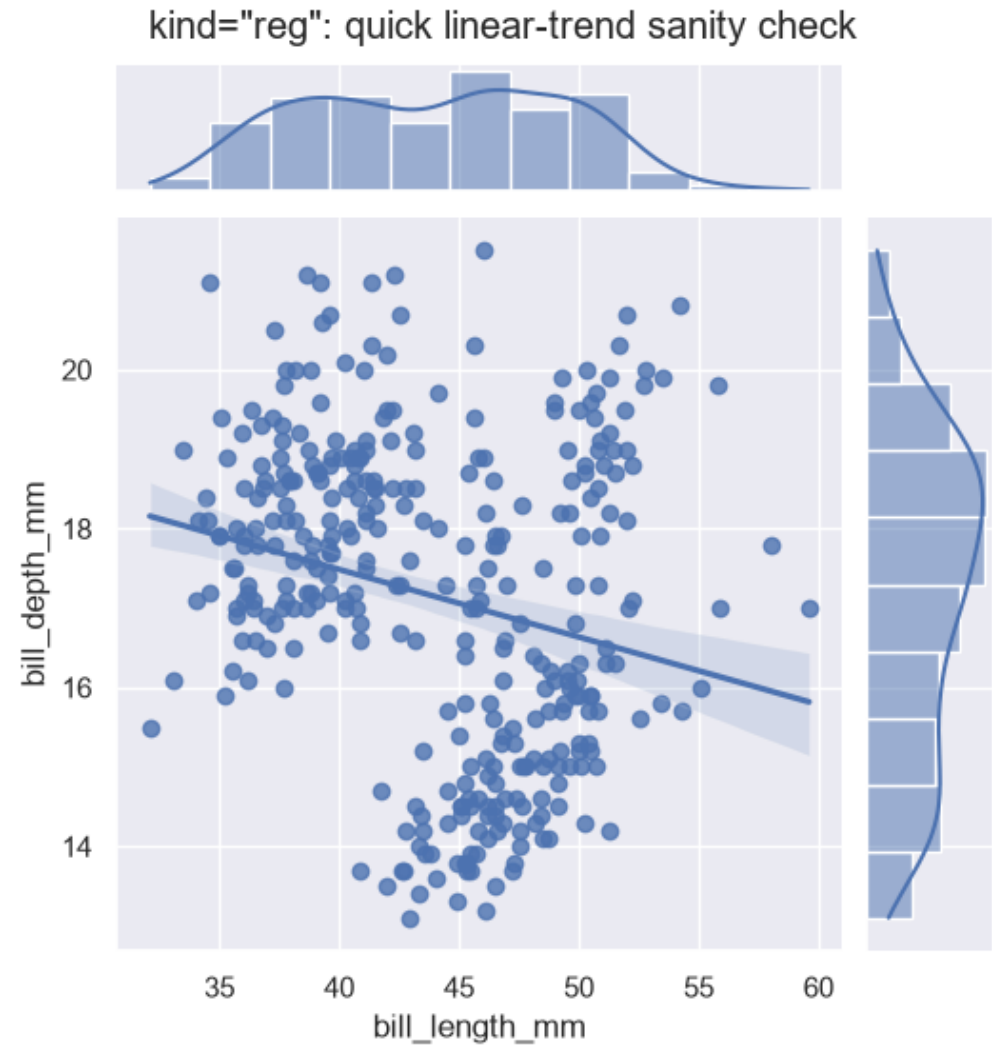


Figure 58: 7. jointplot: bivariate + marginals (ch05_seaborn_02 figure 8)

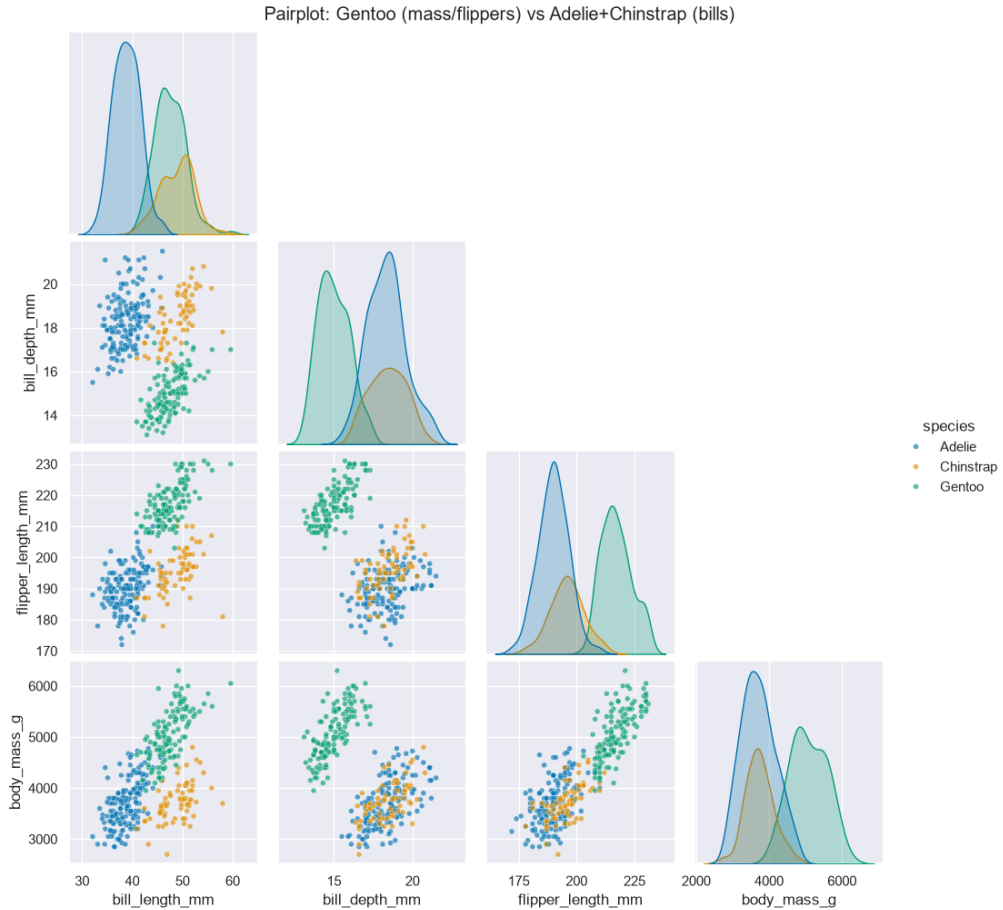


Figure 59: 8. pairplot: the scatterplot matrix (ch05_seaborn_02 figure 9)

```
vars=num_cols,
hue="species",
diag_kind="kde",          # smooth diagonals
corner=True,              # half the panels, same information
palette="colorblind",
plot_kws={"s": 18, "alpha": 0.65},
)
pp.figure.suptitle("Pairplot: Gentoo (mass/flippers) vs Adelie+Chinstrap (bills)", y=1.01)
plt.show()
```

Pro tip: pairplots are $O(k^2)$ panels, for $k > 8$ variables sample rows AND select vars= deliberately, or the figure becomes wallpaper.

9. FacetGrid masterclass: custom multi panel figures

When catplot-style presets aren't enough, drive FacetGrid directly:

```
g = sns.FacetGrid(df, row=..., col=..., hue=..., margin_titles=...,
                  height=..., aspect=..., sharex=True, sharey=True)
g.map_dataframe(plotting_function, ...)
```

`map_dataframe(func)` calls `func(data_subset, **kwargs)` once per facet; inside a custom function you draw on the current axes (`plt.gca()`). Handy methods: `g.refline(...)` global reference lines, `g.set_titles(...)`, `g.add_legend()`, `g.despine()`, and `sharex/sharey=False` to free axes.

```
def mark_group_mean(data, y, **kws):
    """Custom per-facet overlay: dashed crimson line at the facet's mean."""
    ax = plt.gca()
    mean_val = data[y].mean()
    ax.axhline(mean_val, ls="--", lw=1.4, color="crimson")
    ax.text(0.98, mean_val, f"mean={mean_val:.1f}",
           va="bottom", ha="right", fontsize=8, color="crimson",
           transform=ax.get_yaxis_transform())

g = sns.FacetGrid(tips, row="time", col="sex",
                  margin_titles=True, height=2.8, aspect=1.3)
g.map_dataframe(sns.scatterplot, x="total_bill", y="tip", alpha=0.55)
g.map_dataframe(mark_group_mean, y="tip")
g.refline(y=tips["tip"].mean(), color="gray", ls=":", lw=1) # GLOBAL mean
g.set_axis_labels("total bill ($)", "tip ($)")
g.set_titles(row_template="{row_name}", col_template="{col_name}")
g.add_legend()
g.figure.suptitle("FacetGrid: per-facet mean (dashed) vs global mean (dotted)", y=1.04)
plt.show()
```

Free scales: `col_wrap` reflows facets into rows; `sharey=False` lets each day

```
## use its own y-range (great for counts of very different magnitudes).
g2 = sns.FacetGrid(tips, col="day", col_wrap=2, height=2.6,
                  sharey=False)
g2.map_dataframe(sns.histplot, x="total_bill", bins=15)
g2.set_titles("{col_name}")
g2.figure.suptitle("sharey=False + col_wrap=2", y=1.03)
plt.show()
```

Common pitfall: with `sharey=False` readers compare bar heights across facets at their peril. Free scales only when the caption warns about it.

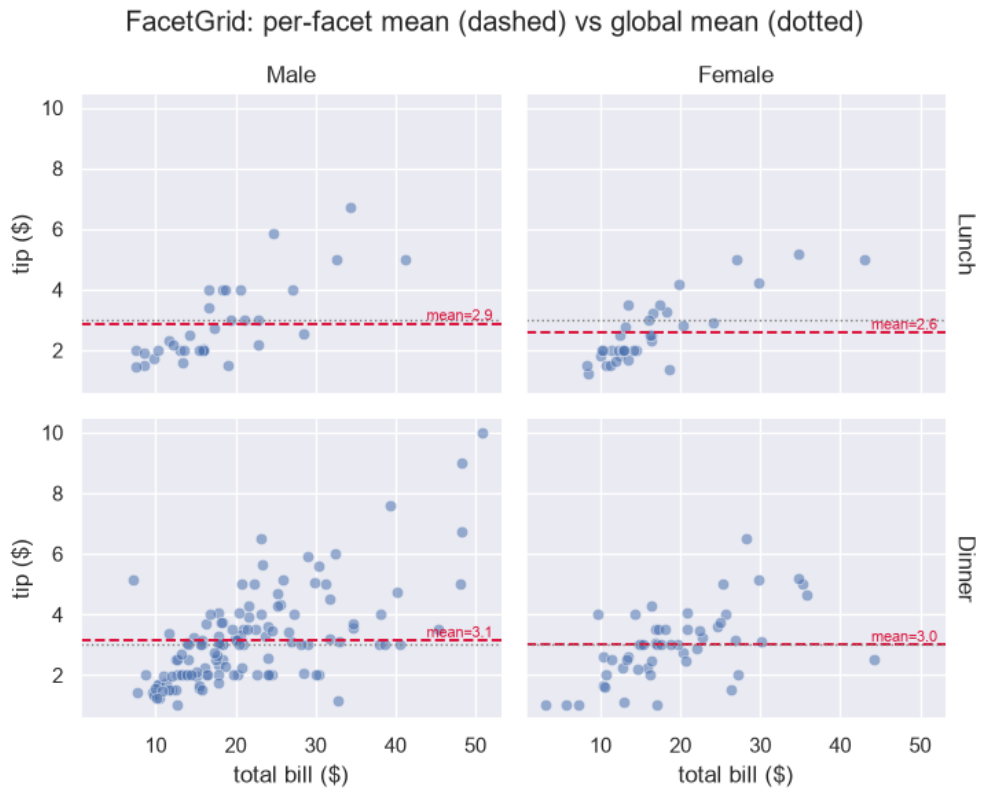


Figure 60: 9. FacetGrid masterclass: custom multi panel figures (ch05_seaborn_02 figure 10)

10. PairGrid: bespoke scatterplot matrices

PairGrid is the engine under pairplot; using it directly lets you assign a *different* plot type to each triangle:

- `g.map_diag(fn)`, diagonal (one variable's distribution).
- `g.map_upper(fn) / g.map_lower(fn)`, off diagonal bivariate views.

Recipe below: KDE filled diagonals, raw scatters up top, per hue regression lines underneath.

```
pg = sns.PairGrid(peng_clean, vars=num_cols, hue="species",
                  diag_sharey=False, corner=False)
pg.map_diag(sns.kdeplot, fill=True, common_norm=False)
pg.map_upper(sns.scatterplot, s=12, alpha=0.55)
pg.map_lower(sns.regplot, scatter=False, line_kws={"lw": 1.2})
pg.add_legend()
pg.figure.suptitle("PairGrid: densities / points / per-species trends", y=1.01)
plt.show()
```

Pro tip: `corner=True` works here too, and `map_lower` with `scatter=False` keeps dense matrices legible, lines carry the trend, ink stays low.

11. Fine tuning themes: styles, contexts, despinning

Quick reference (details in notebook 01):

- Styles: `darkgrid`, `whitegrid`, `dark`, `white`, `ticks`.
- Contexts scale text/lines: `paper`, `notebook`, `talk`, `poster`.
- `sns.despine(ax=ax, offset=10, trim=True)` pulls spines away from the data and trims them to the data range, instant polish.
- Persistent one off settings go through `rc`: `sns.set_theme(style="white", rc={"axes.spines.top": False})`.

Mini style gallery on real data.

```
for style_name in ["whitegrid", "ticks", "dark"]:
    with sns.axes_style(style_name):
        fig, ax = plt.subplots(figsize=(4.2, 2.8))
        sns.boxplot(data=tips[tips["day"] == "Sat"], x="smoker", y="tip", ax=ax)
        ax.set_title(f'style="{style_name}"')
        plt.show()
```

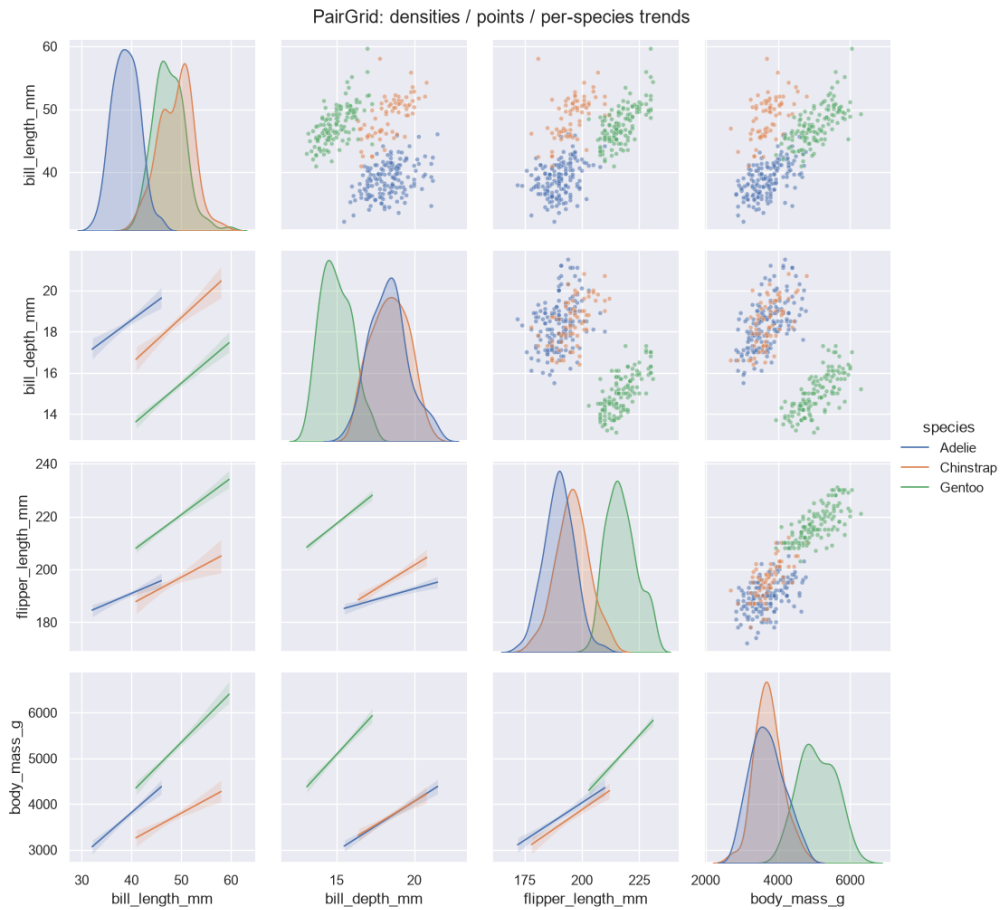


Figure 61: 10. PairGrid: bespoke scatterplot matrices (ch05_seaborn_02 figure 11)

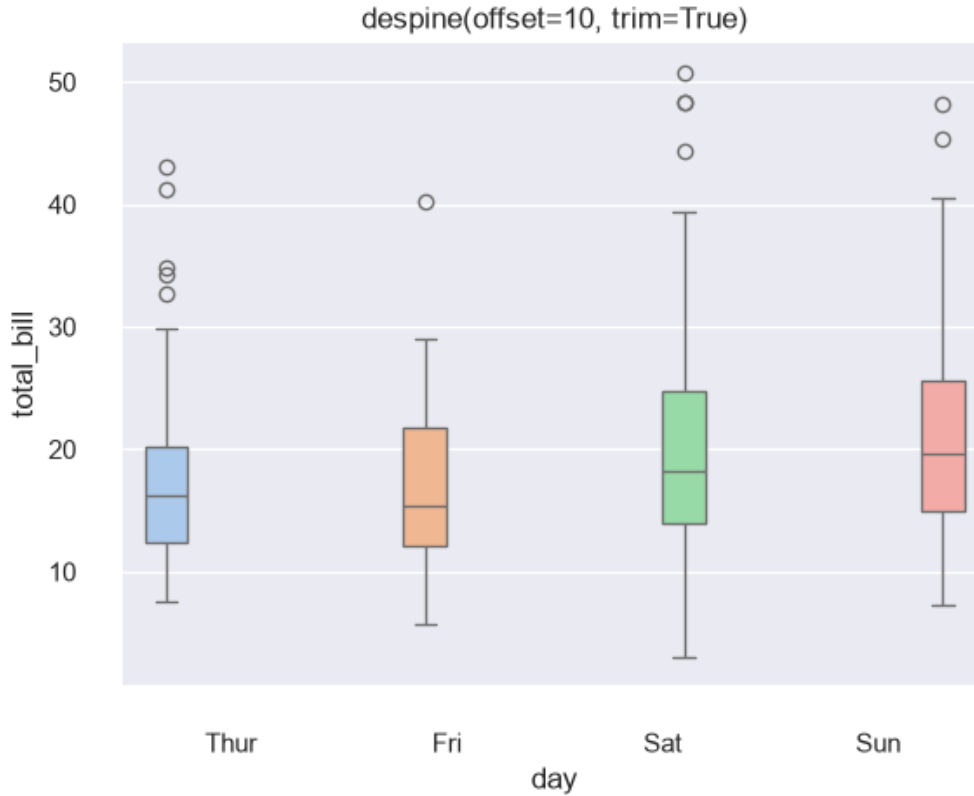


Figure 62: Context scaling: same chart prepared for a paper vs a conference talk. (ch05_seaborn_02 figure 12)

Context scaling: same chart prepared for a paper vs a conference talk.

```
for ctx in ["paper", "talk"]:
    with sns.plotting_context(ctx):
        fig, ax = plt.subplots(figsize=(4.6, 3.2))
        sns.pointplot(data=tips, x="day", y="tip", hue="smoker", ax=ax)
        ax.set_title(f'context="{ctx}"')
        plt.show()
```

```
ax = sns.boxplot(data=tips, x="day", y="total_bill", hue="day", legend=False, palette="pastel")
sns.despine(ax=ax, offset=10, trim=True) # float the spines off the data
ax.set_title("despine(offset=10, trim=True)")
plt.show()
```


12. Matplotlib interop: seaborn inside your figures

Because seaborn outputs real matplotlib artists, you can:

1. Capture the returned Axes and annotate/add layers afterwards.
2. Resize figure level output *after* creation via `g.figure.set_size_inches`.
3. Title properly: `ax.set_title` for one panel; `g.figure.suptitle` for a FacetGrid's whole window.
4. Relocate legends anywhere with `sns.move_legend(ax_or_grid, loc)`.
5. Mix raw matplotlib calls onto seaborn axes (or vice versa).

```
## 1) Grab the Axes, then annotate like any matplotlib artist.
ax = sns.histplot(data=tips, x="total_bill", bins=30)
peak_y = ax.get_ylim()[1] * 0.85
ax.annotate(
    "long right tail: a few big spenders",
    xy=(46, 8), xytext=(32, peak_y),
    arrowprops=dict(arrowstyle="->", color="crimson"),
    color="crimson",
)
ax.set_title("Axes captured from seaborn - annotation added after")
plt.show()
```

```
## 2) + 4) Figure-level objects: resize after the fact and move the legend.
g = sns.relplot(data=tips, x="total_bill", y="tip", hue="time",
                height=3.4, aspect=1.0)
g.figure.set_size_inches(9, 3.6)      # reshape AFTER plotting
g.figure.suptitle("relplot resized post hoc", y=1.03)
sns.move_legend(g, "upper left", bbox_to_anchor=(0.08, 0.95), frameon=False)
plt.show()
```

```
## 5) Mixing engines: seaborn KDE + raw matplotlib scatter on shared axes,
## next to a pure-matplotlib panel - one Figure, two libraries, zero friction.
sub = bill.sample(120, random_state=1)
fig, axes = plt.subplots(1, 2, figsize=(11.5, 4))
sns.kdeplot(data=bill, x="bill_length_mm", y="bill_depth_mm",
            fill=True, cmap="crest", ax=axes[0])
axes[0].scatter(sub["bill_length_mm"], sub["bill_depth_mm"],
                s=10, c="black", alpha=0.45, linewidths=0) # mpl layer on sns ax
axes[0].set_title("sns.kdeplot + plt.scatter overlay")
axes[1].scatter(sub["bill_length_mm"], sub["body_mass_g"], c=sub["body_mass_g"],
                cmap="plasma", s=22)
axes[1].set_xlabel("bill length (mm)")
axes[1].set_ylabel("body mass (g)")
axes[1].set_title("pure matplotlib panel")
fig.suptitle("Interop: both libraries share the same canvas", y=1.03)
plt.tight_layout()
plt.show()
```

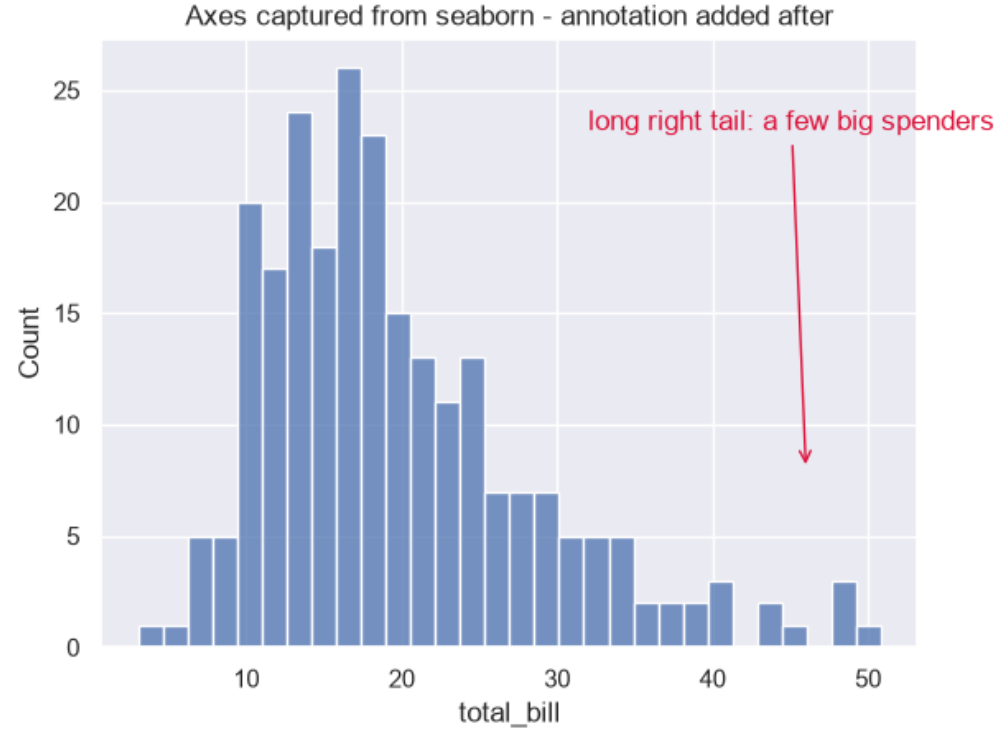


Figure 63: 12. Matplotlib interop: seaborn inside your figures (ch05_seaborn_02 figure 13)

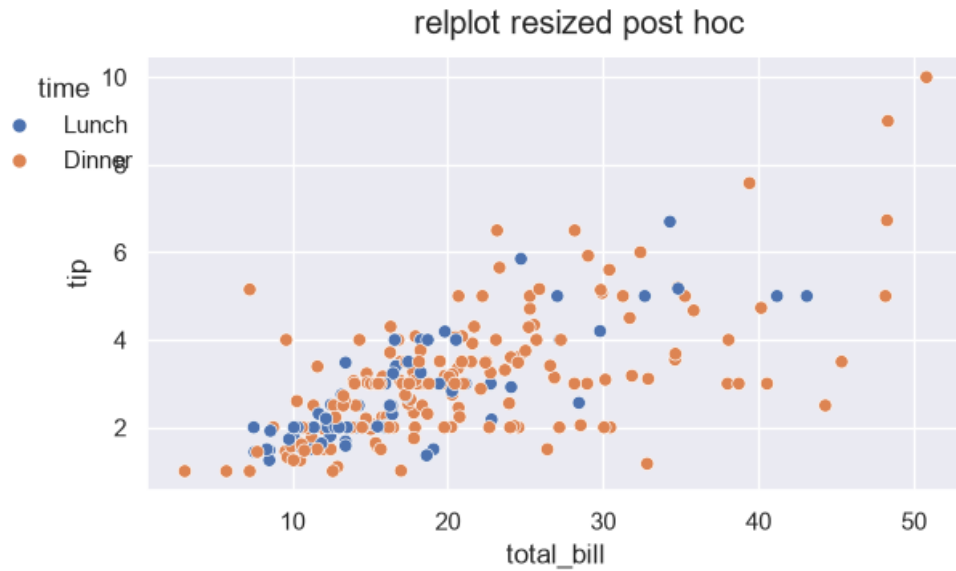


Figure 64: 12. Matplotlib interop: seaborn inside your figures (ch05_seaborn_02 figure 14)

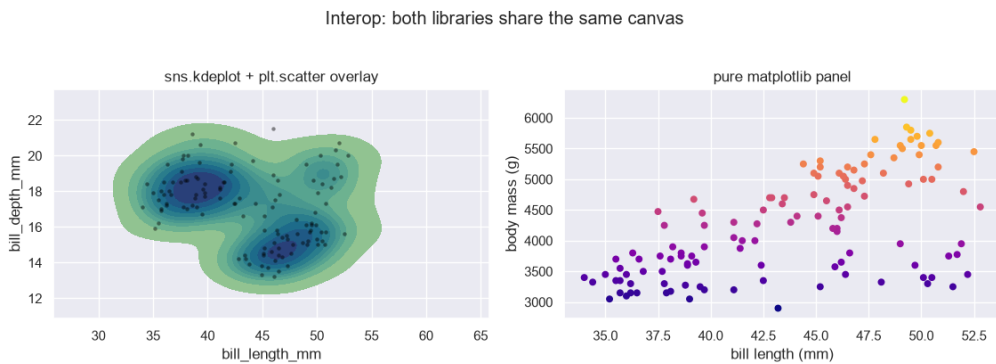


Figure 65: 12. Matplotlib interop: seaborn inside your figures (ch05_seaborn_02 figure 15)

Common pitfall: on figure level grids there is no single `ax`; reach individual panels via `g.axes_dict` (keyed by facet variable) or the numpy array `g.axes`. Calling `ax.set_title` blindly will not do what you want.

13. Saving figures for reports and slides

Two paths, matching the two levels:

- Axes level: keep the Figure (`fig, _ = plt.subplots(...)`) then `fig.savefig(name, dpi=..., bbox_inches="tight")`.
- Figure level: `g.savefig(name)` saves the whole facet window.

Formats: `png` + `dpi=150` for screens/slides, `dpi=300` for print, and `svg/pdf` vector formats that stay crisp at any zoom in reports.

Axes level path: build on a known Figure, then save it.

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(data=tips, x="day", y="total_bill", ax=ax)
ax.set_title("Demo figure destined for disk")
fig.savefig("mlcourse_demo_box.png", dpi=200, bbox_inches="tight")

## Figure-level path: FacetGrid objects carry their own savefig.
g_save = sns.lmplot(data=mpg, x="weight", y="mpg", col="origin",
                    height=3, scatter_kws={"alpha": 0.5})
g_save.figure.suptitle("Vector export demo", y=1.04)
g_save.savefig("mlcourse_demo_lm.svg")      # vector: infinitely scalable
print("Saved: mlcourse_demo_box.png, mlcourse_demo_lm.svg")
```

Pro tip: `bbox_inches="tight"` crops stray whitespace automatically; pair it with `dpi=300` for camera ready submissions.

14. Performance and etiquette on large data

Rules that keep exploration fast and laptops cool:

1. **Sample first.** Diamonds has ~54k rows; `.sample(5000, random_state=42)` gives identical looking charts in a fraction of the time (and a reproducible subset!).
2. **Skip uncertainty while exploring:** `ci=None` / `errorbar=None` removes the expensive bootstrap until the final render.
3. **Know the cost curves:** KDE $\approx O(n \cdot \text{grid}^2)$ per dimension; swarm $\approx$ quadratic, avoid swarms past a few thousand points per category.
4. **Close stale figures** (`plt.close("all")`) during loops to cap memory.

```
diamonds = load_dataset("diamonds")
if diamonds is not None:
```

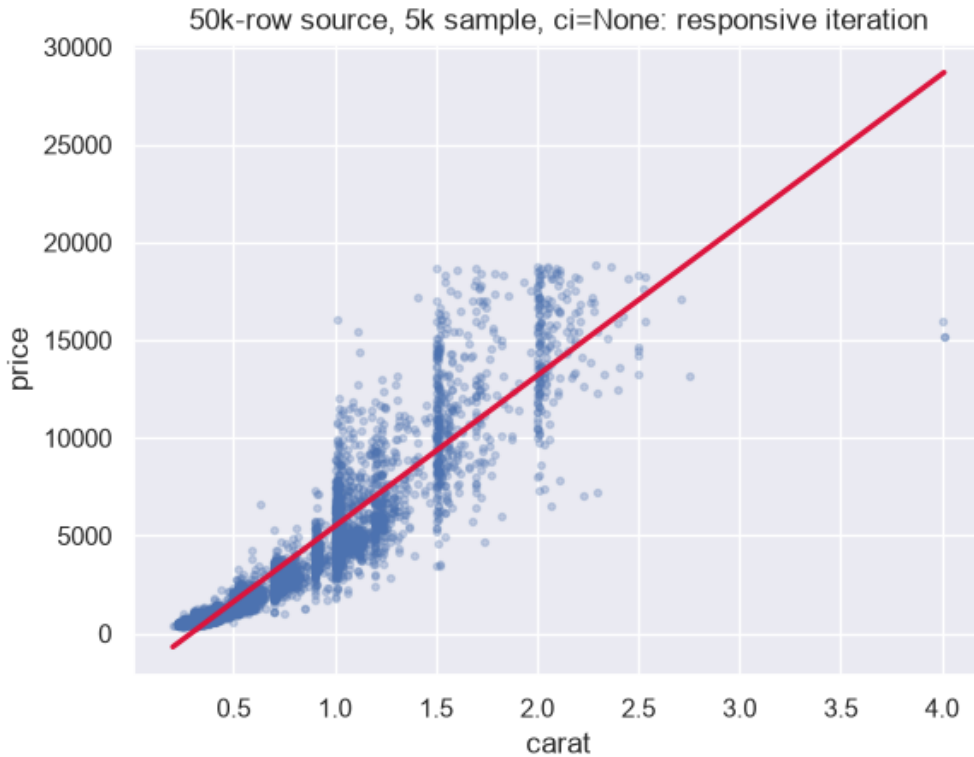


Figure 66: 14. Performance and etiquette on large data (ch05_seaborn_02 figure 16)

```
dia_small = diamonds.sample(5000, random_state=42)
ax = sns.regplot(
    data=dia_small, x="carat", y="price",
    ci=None, # exploration mode: no bootstrap band
    scatter_kws={"s": 8, "alpha": 0.25},
    line_kws={"color": "crimson"},
)
ax.set_title("50k-row source, 5k sample, ci=None: responsive iteration")
else:
    print("Skipping performance demo - diamonds unavailable.")
plt.show()
```

Summary & key takeaways

- `regplot` bundles scatter + fit + CI; bend it with `order=`, `logistic=`, `logx=`, speed it up with `ci=None`; `lmplot` adds per facet fits; `residplot` is your misspecification detector (flat band = done).

- Heatmaps need deliberate `cmap` + center choices; mask correlation triangles, annotate cells, and read top pairs programmatically.
- `clustermap` turns a matrix into an unsupervised similarity map with dendrograms; standardize features with mixed units first.
- `jointplot` (scatter/kde/hex/reg/resid) answers bivariate questions with built in marginal context; `pairplot/PairGrid` scale that to k^2 panels (`corner=True` beyond five variables).
- `FacetGrid.map_dataframe` accepts your own functions, per facet means, percentile bands, anything, plus `refline`, templated titles, legends.
- Seaborn and matplotlib interleave freely: capture axes, annotate, resize, `move_legend`, mix layers.
- Save with `savefig(dpi=...)` for raster and SVG/PDF for vector reports.
- Large data: sample reproducibly, drop CIs while exploring, respect algorithmic costs.

Ready to drive solo? Head to **03_seaborn_exercises** for twelve graded challenges, with fully worked solutions in **04_seaborn_solutions**.

6.3 Seaborn, Seaborn Exercises

MLCourse · Data Science Foundations · 04_seaborn

Twelve challenges to make the seaborn toolkit stick, four **Easy**, five **Medium**, three **Hard**. Each challenge gives you a goal, a dataset, a checklist of required elements, and a starter cell. No answers here: work each one yourself, then compare against **04_seaborn_solutions**.

Difficulty guide:

- **Easy**, one or two well chosen calls plus labeling polish.
- **Medium**, multiple steps: transforms, masks, sampling, or multi panel composition.
- **Hard**, custom `FacetGrid` functions, side by side comparisons, and a full mini dashboard on one matplotlib figure.

Skim notebook 01/02 docs (or `help sns.bboxplot`)) freely, that is not cheating, that is the job.

What you'll learn

- Turning requirements ("compare these groups", "show uncertainty") into concrete seaborn calls without copy pasting from tutorials.
- Combining figure level convenience with axes level composition.
- The discipline of checklists: ordering, labels, titles, honest error bars, colorblind safe palettes.

Setup

Same Jupyter safe magic pattern as notebooks 01-02.

```
try:
```

```

    get_ipython().run_line_magic("matplotlib", "inline") # noqa: F821
except NameError:
    pass

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme() # consistent baseline for every challenge

def load_dataset(name):
    """Load a built-in seaborn dataset with a friendly offline hint."""
    try:
        return sns.load_dataset(name)
    except Exception as exc:
        print(f"[offline hint] Could not load '{name}': {exc}")
        print(" Check your connection or fetch the CSV from")
        print("  https://github.com/mwaskom/seaborn-data and use pd.read_csv.")
        return None

tips = load_dataset("tips")
titanic = load_dataset("titanic")
_penguins_raw = load_dataset("penguins")
penguins = _penguins_raw.dropna() if _penguins_raw is not None else None
mpg = load_dataset("mpg")
flights = load_dataset("flights")

_diamonds_raw = load_dataset("diamonds")

def dia(n=5000):
    """Reproducible diamonds sample for the heavy challenges."""
    if _diamonds_raw is None:
        return None
    return _diamonds_raw.sample(n, random_state=42)

for label, df in [("tips", tips), ("titanic", titanic), ("penguins", penguins),
                  ("mpg", mpg), ("flights", flights)]:
    print(f"{label}>9}: {df.shape if df is not None else 'NOT LOADED'}")
print(f" dia(n=5000): ready={dia() is not None}")

```

Challenge, Mean spend by day, ordered and honest (Easy)

Dataset: tips

Goal: show mean total_bill per day as a bar chart with uncertainty, bars ordered from highest to lowest mean.

Checklist

- ☐ sns.barplot with the default mean estimator.
- ☐ An explicit order= derived from groupby means (not alphabetical).
- ☐ A bootstrap CI error bar (default is fine), and a caption/title saying what the error bars ARE.
- ☐ Axis labels + title; despined spines for polish.

Starter, Challenge 1

```
day_order = (
    tips.groupby("day")["total_bill"].mean()
    .sort_values(ascending=False)
    .index.tolist()
)
print("order:", day_order)

## TODO: barplot with order=day_order, then labels/title/despine.
```

Challenge, ECDF with a threshold read off (Easy)

Dataset: penguins

Goal: an ECDF of body_mass_g per species answering "what fraction of each species weighs under 4000 g?" at a glance.

Checklist

- ☐ sns.ecdfplot with hue="species".
- ☐ A rugplot layered on the same axes (raw observations visible).
- ☐ A dashed vertical reference line at 4000 g.
- ☐ Print the exact per species fractions below 4000 g to verify your eyes.

Starter, Challenge 2

```
ax = None # create via sns.ecdfplot(...) assigned to ax, then layer onto it

## TODO: ecdfplot + rugplot + axvline(4000), then compute/print fractions.
```


Challenge, Corner pairplot by origin (Easy)

Dataset: mpg

Goal: a scatterplot matrix comparing fuel economy metrics across car origins, half size thanks to `corner=True`.

Checklist

- ☐ Variables: mpg, horsepower, weight, acceleration, displacement.
- ☐ `hue="origin"` with a colorblind safe palette.
- ☐ `diag_kind="kde"` for smooth diagonals.
- ☐ `corner=True` and a `suptitle` on `pp.figure`.

Starter, Challenge 3

```
vars_5 = ["mpg", "horsepower", "weight", "acceleration", "displacement"]

## TODO: pp = sns.pairplot(...); remember pp.figure.suptitle(...).
```

Challenge, Colorblind redesign (Easy)

Dataset: tips

Goal: take any chart from this module (suggest: boxplot of `total_bill` by `day`) and re issue it in a colorblind safe palette, previewing palettes first.

Checklist

- ☐ Preview at least two qualitative palettes with `sns.palplot`.
- ☐ Rebuild the chart passing `palette="colorblind"` (per call).
- ☐ Briefly demo the global route too (`sns.set_palette`) and restore the default afterwards.
- ☐ One sentence comment on WHY red/green defaults can fail readers.

Starter, Challenge 4

```
base_chart = sns.boxplot(data=tips, x="day", y="total_bill",
                        hue="day", legend=False) # recolor me
plt.show()

## TODO: palplot previews -> redesigned chart (per-call palette)
## -> global set_palette demo -> restore "deep".
```

Challenge, Faceted regression per origin (*Medium*)

Dataset: mpg

Goal: does horsepower hurt mpg equally everywhere? One regression panel per origin, each with its own fit line.

Checklist

- ☐ Figure level `lmpplot` (NOT axes level `regplot`) with `col="origin"`.
- ☐ Matching `hue="origin"` so fits and points share colors.
- ☐ Translucent scatter (`scatter_kws={"alpha": ...}`).
- ☐ Shared axis labels via `g.set_axis_labels`; `suptitle` stating the claim.
- ☐ Sanity comment: how many cars per facet (`value_counts`) before trusting.

Starter, Challenge 5

```
print(mpg["origin"].value_counts())

## TODO: g = sns.lmplot(...col="origin", hue="origin...); annotate + title it.
```

Challenge, One variable, three lenses (*Medium*)

Dataset: penguins

Goal: compare body mass across species using box, violin, AND ECDF, same data, three encodings, one row of panels.

Checklist - [] One matplotlib Figure via `plt.subplots(1, 3)`; every seaborn call receives its target `ax=`. - [] Panel 1: `boxplot x=species`; Panel 2: `violinplot` with `inner="quart"`; Panel 3: `ecdfplot` with `hue=species`. - [] Shared comparison where meaningful (`sharey=True` helps panels 1-2). - [] Per panel titles naming the encoding; overall `suptitle`.

Starter, Challenge 6

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))

## TODO: fill all three panels using axes[0], axes[1], axes[2].
```

Challenge, Masked correlation heatmap (Medium)**Dataset:** penguins**Goal:** publication grade correlation matrix of the four morphometric columns, lower triangle only, annotated, diverging centered.**Checklist**

- ☐ `corr = penguins[num_cols].corr(numeric_only=True)` on bill length / depth, flipper, mass.
- ☐ Boolean mask hiding the ABOVE diagonal cells (`np.triu(..., k=1)`).
- ☐ `annot=True, fmt=".2f", annot_kws` font size tweak.
- ☐ Diverging cmap (`vlag` or `RdBu_r`), `center=0, vmin=-1, vmax=1, square=True`.
- ☐ Print the strongest absolute pair programmatically (`stack + sort`).

Starter, Challenge 7

```
num_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
corr = penguins[num_cols].corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)

## TODO: sns.heatmap(...) then extract & print top pairs.
```

Challenge, Hexbin jointplot on big data (Medium)**Dataset:** diamonds (sampled!)**Goal:** relationship between carat and price when scatter would be solid ink, hexagonal binning with marginal context.**Checklist**

- ☐ Use `dia()` (5000 rows, `random_state=42`), never plot 50k+ raw points casually.
- ☐ `sns.jointplot(kind="hex")` on carat vs price.
- ☐ `marginal_ticks=True` and a sensible height.
- ☐ Suptitle noting both variables are right skewed (why hex beats scatter here).
- ☐ Bonus: try tuning hex resolution via `joint_kws={"gridsize": ...}`.

Starter, Challenge 8

```
d5k = dia()
print(d5k[["carat", "price"]].describe().round(1))

## TODO: j = sns.jointplot(data=d5k, kind="hex", ...); j.figure.suptitle(...).
```

Challenge, Pivot table heatmap with a centered midpoint (*Medium*)

Dataset: tips

Goal: average tip by day × meal time as a heatmap whose colors center on a meaningful value (the overall median tip), so “typical” reads neutral.

Checklist

- ☐ `pivot_table(index="day", columns="time", values="tip", aggfunc="mean"),` days ordered ThuSun.
- ☐ `center=float(np.median(tips["tip"]))`, say why in a comment.
- ☐ Diverging cmap, `annot=True`, `fmt=".2f"`, thin white separators.
- ☐ Title + labeled colorbar (`cbar_kws={"label": ...}`).

Note: tips has no month column, so we adapt the classic month×day pivot to day×time; the flights dataset offers a true month×year variant if you want extra practice afterwards.

Starter, Challenge 9

```
day_order_cal = ["Thur", "Fri", "Sat", "Sun"]
pivot = tips.pivot_table(index="day", columns="time", values="tip",
                          aggfunc="mean", observed=False).reindex(day_order_cal)
center_val = float(np.median(tips["tip"]))

## TODO: sns.heatmap(pivot, center=center_val, ...) with annotations & polish.
```

Challenge, FacetGrid with percentile bands (*Hard*)

Dataset: tips

Goal: tip vs total_bill faceted by day, where every facet carries its own 25th/75th percentile lines for tip (and a median), drawn by YOUR custom function through `map_dataframe`.

Checklist

- ☐ `FacetGrid(tips, col="day", col_wrap=2)` with scatter base layer.
- ☐ A custom function (`data, **kws`) drawing q25/q75 dashed lines + median solid line on `plt.gca()`, mapped via `g.map_dataframe(fn)`.
- ☐ `g.refline(y=...)` showing the GLOBAL median tip for contrast.
- ☐ Templated titles (`g.set_titles(col_template="{col_name}")`), legend or inline text explaining the line styles.
- ☐ No hardcoded percentile numbers, computed inside the function.

Starter, Challenge 10

```
def draw_tip_bands(data, **kws):
    """TODO: q25/median/q75 horizontal lines on the current axes."""
    ax = plt.gca()
    # TODO: quantile([0.25, 0.5, 0.75]) -> axhline trio (styles differ)

g = sns.FacetGrid(tips, col="day", col_wrap=2, height=3.0)
## TODO: g.map_dataframe(sns.scatterplot, ...) ; g.map_dataframe(draw_tip_bands)
## TODO: refline, titles, supitle, plt.show()
```

Challenge, Box vs boxen showdown (*Hard*)

Dataset: diamonds (sampled)

Goal: argue visually for boxenplot on large n: same variable, two encodings, shared scale, annotated sample sizes.

Checklist

- ☐ One figure, `plt.subplots(1, 2, sharey=True)`: left boxplot, right boxenplot of carat by cut (ordered worstbest cut).
- ☐ Identical y limits on both panels (shared axes handle it, verify).
- ☐ n per cut printed AND annotated above one of the panels.
- ☐ A takeaway comment: what extra structure does boxen reveal (tail boxes), and when would you still prefer a plain box?

Starter, Challenge 11

```
d5k = dia()
cut_order = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
n_per_cut = d5k["cut"].value_counts().reindex(cut_order)
print(n_per_cut)

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6), sharey=True)

## TODO: boxplot on axes[0]; boxenplot on axes[1]; titles + n annotations.
```

Challenge, Mini dashboard: four views, one figure (*Hard*)

Dataset: your pick across tips / penguins

Goal: a single 2x2 matplotlib figure hosting four DIFFERENT seaborn axes level plots, reading like a tipping behavior dashboard.

Checklist

- ☐ `fig, axes = plt.subplots(2, 2, figsize=(12, 8))`, every seaborn call gets an `ax=` (no figure level functions allowed here!).
- ☐ Panel A: distribution view (box or violin of tip by day).
- ☐ Panel B: the Challenge-9 style pivot heatmap (cbar off to save space).
- ☐ Panel C: interaction view, pointplot of tip by day, `hue=smoker`.
- ☐ Panel D: ECDF of penguin body mass by species (yes, mixing datasets on one dashboard is fine, label clearly).
- ☐ Per panel titles + a `fig.suptitle`; `fig.tight_layout()` before show.

Starter, Challenge 12

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

## TODO: four axes-level calls into axes[0,0] .. axes[1,1].
```

Wrap up

When every checklist ticks green, open `04_seaborn_solutions` and diff your approach against ours, different but valid is the norm in plotting; the interesting conversation is *why* an encoding works. Then revisit any challenge a week later: speed of recall is the real graduation test.

6.4 Seaborn, Seaborn Solutions

MLCourse · Data Science Foundations · 04_seaborn

Fully worked, standalone solutions to the twelve challenges in `03_seaborn_exercises`. Every cell is self contained commentary + code: read the reasoning, run it, then go back and re solve each exercise WITHOUT looking, that second pass is where it becomes yours.

Where multiple valid designs exist (they usually do), we note the trade off inline. Plotting is judgment, not incantation.

What you'll learn

- Reference implementations of all twelve challenges with the *why* behind every parameter choice.
- Defensive dataset loading (with synthetic fallbacks) so this file runs even fully offline.
- Patterns worth stealing: ordering from `groupby`, percentile bands via `map_dataframe`, dashboard composition on shared matplotlib figures.

Setup & defensive data loading

The loader tries seaborn's online registry first; if that fails we fall back to small synthetic DataFrames with identical columns so every solution below still executes end to end.

```
try:
    get_ipython().run_line_magic("matplotlib", "inline") # noqa: F821
except NameError:
    pass

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme()

def load_dataset(name):
    """Load a built-in seaborn dataset; return None if unavailable."""
    try:
        return sns.load_dataset(name)
    except Exception as exc:
        print(f"[offline hint] '{name}' unavailable ({exc}); using synthetic fallback.")
        return None

def _fallback(name, n=240):
    """Small synthetic stand-ins with the columns our solutions rely on."""
    rng = np.random.default_rng(42)
    days = rng.choice(["Thur", "Fri", "Sat", "Sun"], n)
    times = rng.choice(["Lunch", "Dinner"], n, p=[0.35, 0.65])
    smokers = rng.choice(["Yes", "No"], n, p=[0.4, 0.6])
    sexes = rng.choice(["Male", "Female"], n, p=[0.65, 0.35])
    bill = rng.gamma(2.2, 9.5, n) + 3
    tip = bill * rng.uniform(0.08, 0.20, n)
    species = rng.choice(["Adelie", "Chinstrap", "Gentoo"], n)
    if name == "tips":
        return pd.DataFrame({"total_bill": bill.round(2), "tip": tip.round(2),
                             "day": days, "time": times, "smoker": smokers,
                             "sex": sexes})
    if name == "penguins":
        base = {"Adelie": [38.8, 18.3, 190, 3700], "Chinstrap": [48.8, 18.4, 195, 3730],
                "Gentoo": [47.5, 15.0, 217, 5070]}[species[0]]
        rows = []
        for s in species:
            m = {"Adelie": [38.8, 18.3, 190, 3700], "Chinstrap": [48.8, 18.4, 195, 3730],
                  "Gentoo": [47.5, 15.0, 217, 5070]}[s]
            vals = np.array(m) + rng.normal(0, [2.5, 1.0, 7, 300])
            rows.append([s, *vals])
```

```

        return pd.DataFrame(rows, columns=["species", "bill_length_mm",
                                           "bill_depth_mm", "flipper_length_mm",
                                           "body_mass_g"])

    if name == "mpg":
        origin = rng.choice(["usa", "japan", "europe"], n, p=[0.62, 0.20, 0.18])
        hp = rng.uniform(50, 230, n)
        return pd.DataFrame({"horsepower": hp,
                             "mpg": 48 - 0.12 * hp + rng.normal(0, 4, n),
                             "weight": rng.uniform(1600, 5000, n),
                             "acceleration": rng.uniform(8, 24, n),
                             "displacement": rng.uniform(70, 455, n),
                             "origin": origin})

    if name == "titanic":
        cls = rng.choice(["First", "Second", "Third"], n, p=[0.25, 0.21, 0.54])
        return pd.DataFrame({"class": cls,
                             "sex": rng.choice(["male", "female"], n),
                             "alive": rng.choice(["yes", "no"], n, p=[0.38, 0.62])})

    return None

def get(name):
    """Dataset-or-synthetic accessor used throughout."""
    df = load_dataset(name)
    return df if df is not None else _fallback(name)

tips = get("tips")
penguins = get("penguins")
mpg = get("mpg")
titanic = get("titanic")

_diamonds_raw = load_dataset("diamonds")

def dia(n=5000):
    """Reproducible diamonds sample; synthetic carat/price if offline."""
    if _diamonds_raw is not None:
        return _diamonds_raw.sample(n, random_state=42)
    rng = np.random.default_rng(42)
    cuts = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
    return pd.DataFrame({
        "carat": rng.gamma(1.6, 0.35, n).clip(0.2, 4.5),
        "price": rng.lognormal(7.7, 0.55, n).round(0),
        "cut": rng.choice(cuts, n, p=[0.13, 0.19, 0.22, 0.26, 0.20]),
    })

print("ready:", {k: v.shape for k, v in
                 [("tips", tips), ("penguins", penguins),
                 ("mpg", mpg), ("titanic", titanic)]})

```


Output:

```
ready: {'tips': (244, 7), 'penguins': (344, 7), 'mpg': (398, 9), 'titanic': (891, 15)}
```

Challenge, Mean spend by day (Easy)

Key moves: derive order from the data itself (groupby mean, sorted) rather than typing guesses; keep the default bootstrap CI; say what the bars' error bars are in the title; despine for a clean finish.

```
fig, ax = plt.subplots(figsize=(7, 4.2))
day_order = (
    tips.groupby("day")["total_bill"].mean()
    .sort_values(ascending=False)
    .index.tolist()
)
sns.barplot(data=tips, x="day", y="total_bill",
            order=day_order, hue="day", legend=False,
            palette="Blues_r", errorbar=("ci", 95), ax=ax)

ax.set_title("Mean total bill by day\n(error bars = 95% bootstrap CI of the mean)")
ax.set_xlabel("day of week")
ax.set_ylabel("mean total bill ($)")
sns.despine(ax=ax)
plt.tight_layout()
plt.show()

## Trade-off note: a pointplot would show the same estimate with less ink and
## no zero-baseline debate - try swapping kind as an experiment.
```

Challenge, ECDF with threshold read off (Easy)

ECDF superpower: at $x=4000$ g, each curve's height IS the answer. We layer a rug for raw data texture, mark the threshold, then verify numerically.

```
ax = sns.ecdfplot(data=penguins, x="body_mass_g", hue="species")
sns.rugplot(data=penguins, x="body_mass_g", hue="species", ax=ax, alpha=0.35)
ax.axvline(4000, color="gray", ls="--", lw=1.2)
ax.set_title("Fraction of penguins under 4000 g, by species")
ax.set_xlabel("body mass (g)")

frac_under = penguins.assign(under=lambda d: d["body_mass_g"] < 4000)
```

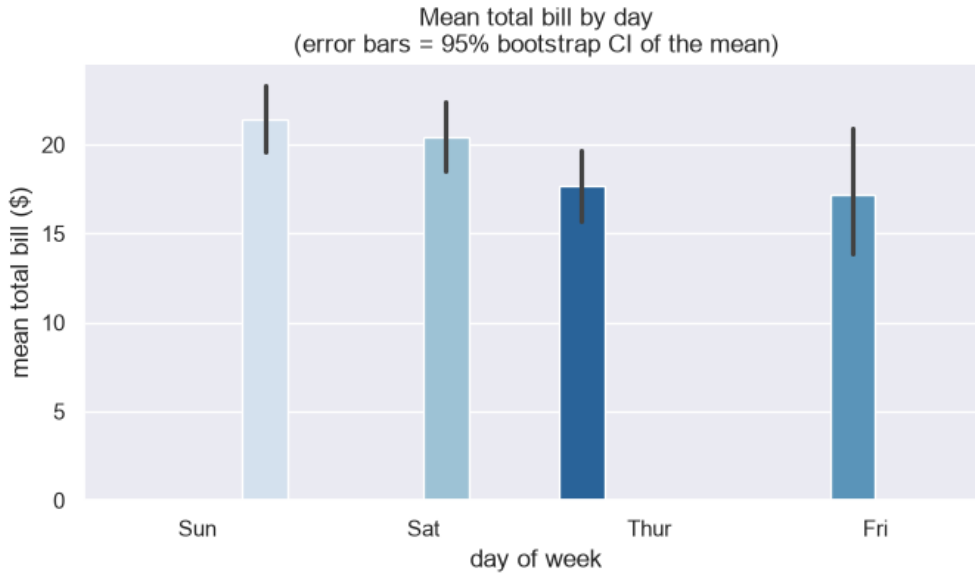


Figure 67: Challenge 1, Mean spend by day *(Easy) (ch05_seaborn_04 figure 1)

```
print(frac_under.groupby("species")["under"].mean().round(3))
plt.show()

## Reading check: Gentoo's curve sits lowest at x=4000 -> fewest light birds;
## Adelie highest -> most. The printed table must agree with your eyeballs.
```

Output:

```
species
Adelie      0.737
Chinstrap   0.765
Gentoo      0.008
Name: under, dtype: float64
```

Challenge, Corner pairplot by origin (Easy)

Five variables means twenty bivariate panels when mirrored; corner=True halves it. KDE diagonals beat histograms here because origins overlap heavily.

```
vars_5 = ["mpg", "horsepower", "weight", "acceleration", "displacement"]
pp = sns.pairplot(
```

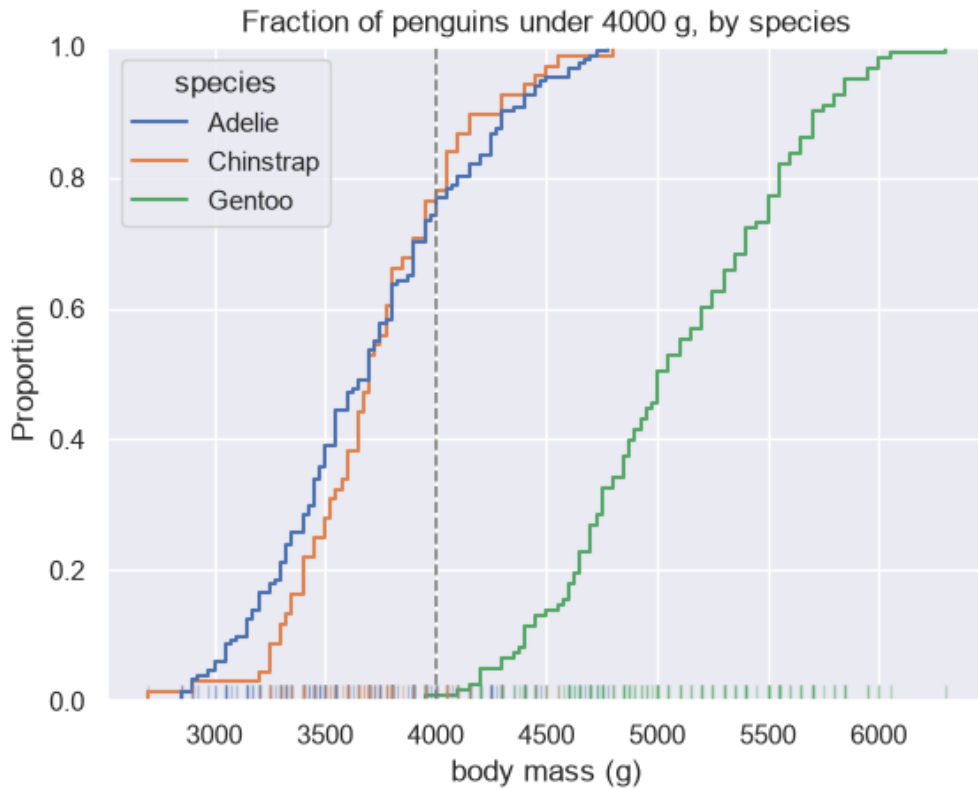


Figure 68: Challenge 2, ECDF with threshold read off *(Easy) (ch05_seaborn_04 figure 2)



Figure 69: Challenge 3, Corner pairplot by origin *(Easy) (ch05_seaborn_04 figure 3)

```
mpg.dropna(subset=vars_5 + ["origin"]),
vars=vars_5, hue="origin",
diag_kind="kde", corner=True,
palette="colorblind",
plot_kws={"s": 16, "alpha": 0.55},
)
pp.figure.suptitle("MPG relationships across origins (corner view)", y=1.01)
plt.show()

## What to notice: weight-horsepower panel is nearly one line (redundant info);
## acceleration separates origins least - good candidate to drop from models.
```

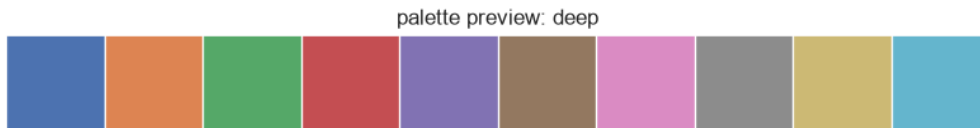


Figure 70: Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 4)



Figure 71: Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 5)

Challenge, Colorblind redesign (Easy)

~8% of men have color vision deficiency; default palettes can pair red/green hues that collapse into one for them. Preview candidates first, then apply per call (safest, nothing leaks) and demo the global route too.

```
for name in ["deep", "colorblind"]:
    sns.palplot(sns.color_palette(name))
    plt.title(f"palette preview: {name}")
plt.show()

fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=tips, x="day", y="total_bill", hue="day", legend=False,
            palette="colorblind", ax=ax)
ax.set_title("Redesigned with per-call palette='colorblind'")
sns.despine(ax=ax)
plt.show()

sns.set_palette("colorblind")    # global route: affects ALL later plots...
ax = sns.violinplot(data=tips, x="day", y="tip")
ax.set_title("Global set_palette('colorblind') in effect")
plt.show()

sns.set_palette("deep")          # ...so restore the default afterwards.
print("palette restored to 'deep'")
```

Output:

```
palette restored to 'deep'
```

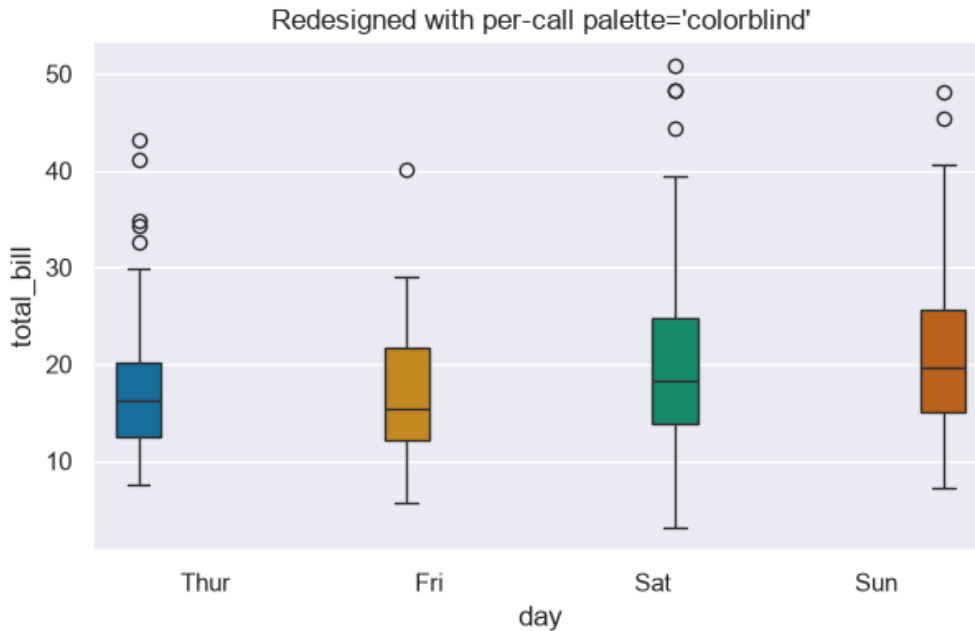


Figure 72: Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 6)

Challenge, Faceted regression per origin (Medium)

`lmplot` fits independently per facet, exactly the question "same slope everywhere?". We check facet sizes FIRST because regression on a handful of European cars from 1970s data would be theater, not statistics.

```
print(mpg["origin"].value_counts())

g = sns.lmplot(
    data=mpg, x="horsepower", y="mpg",
    col="origin", hue="origin",
    height=3.4, scatter_kws={"alpha": 0.5},
)
g.set_axis_labels("horsepower", "mpg")
g.figure.suptitle("Horsepower penalty on mpg, fit separately per origin", y=1.06)
plt.show()

## Observation: slopes look similar-ish but intercepts differ - europe/japan
## sit higher (efficient) at equal power. USA's steeper line reflects its
## big-engine tail stretching the fit.
```

Output:

```
origin
```

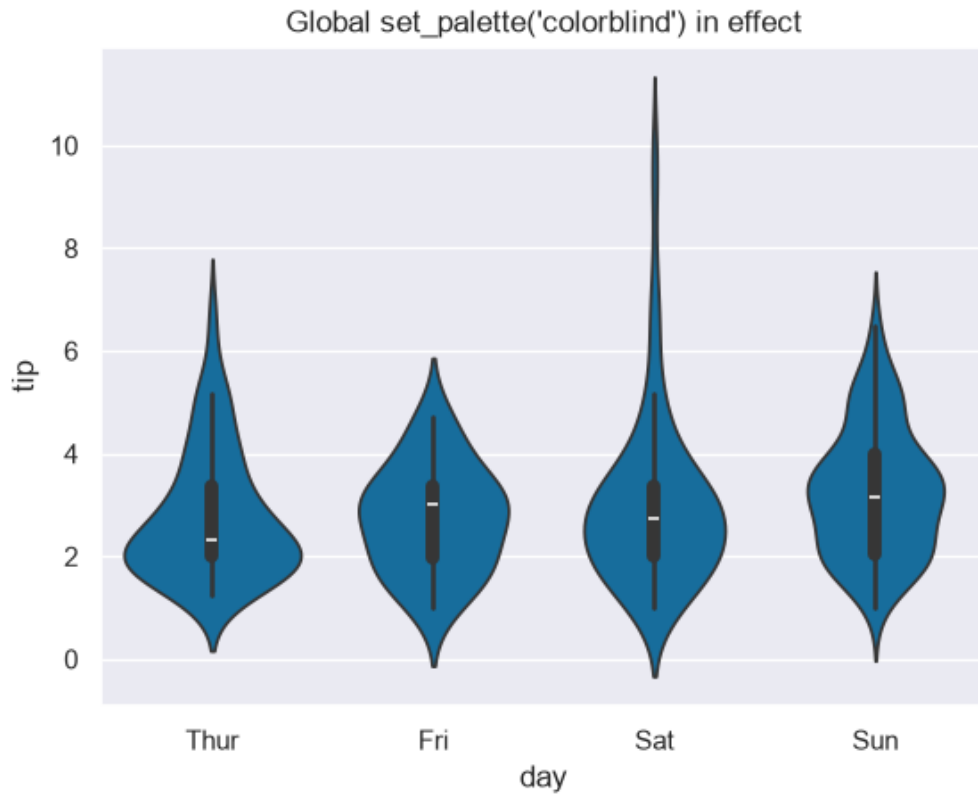


Figure 73: Challenge 4, Colorblind redesign *(Easy) (ch05_seaborn_04 figure 7)

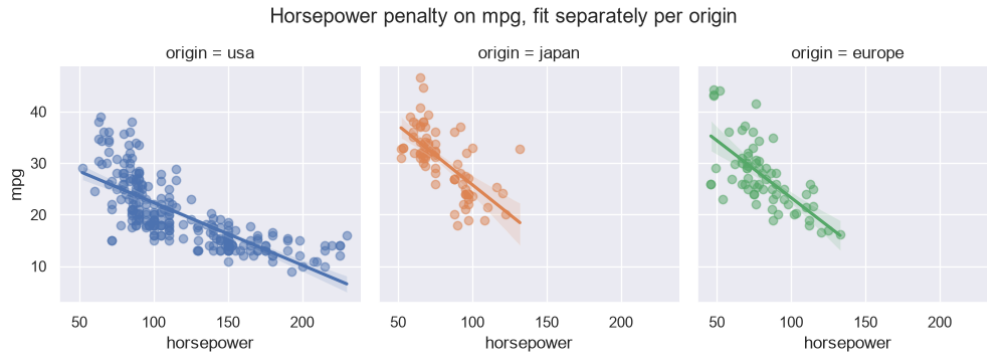


Figure 74: Challenge 5, Faceted regression per origin *(Medium) (ch05_seaborn_04 figure 8)

```
usa      249
japan    79
europe   70
Name: count, dtype: int64
```

Challenge, One variable, three lenses (Medium)

Box = robust five number summary; violin = full shape (watch for smoothing artifacts); ECDF = thresholds and medians without any tuning. Same data, three honest views, disagreement between them is itself informative.

```
fig, axes = plt.subplots(1, 3, figsize=(15.5, 4.4))

sns.boxplot(data=penguins, x="species", y="body_mass_g",
            hue="species", legend=False, ax=axes[0])
axes[0].set_title("Box: median + IQR + outliers")

sns.violinplot(data=penguins, x="species", y="body_mass_g",
               hue="species", legend=False, inner="quart", ax=axes[1])
axes[1].set_title("Violin: full density shape (quartiles inside)")

sns.ecdfplot(data=penguins, x="body_mass_g", hue="species", ax=axes[2])
axes[2].set_title("ECDF: quantiles directly readable")

fig.suptitle("Body mass by species - three encodings, one story", y=1.04)
plt.tight_layout()
plt.show()

## All three agree: Gentoo shifted high (~5 kg), Adelie/Chinstrap overlapping.
```

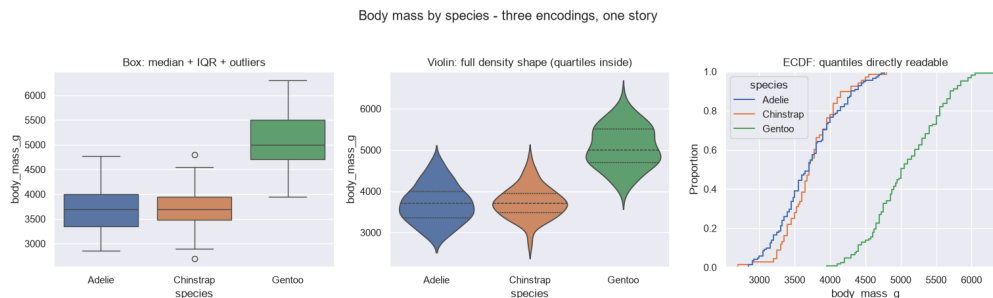



Figure 75: Challenge 6, One variable, three lenses *(Medium) (ch05_seaborn_04 figure 9)

```
## If they HAD disagreed you'd suspect binning/bandwidth choices, not biology.
```

Challenge, Masked correlation heatmap (Medium)

Correlation matrices are symmetric with a trivial diagonal, showing all of it doubles reading cost for zero information. Mask above diagonal, pin the diverging scale to [-1, 1] centered at 0, and extract top pairs in code.

```
num_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
corr = penguins[num_cols].corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool), k=1) # k=1 keeps the diagonal

fig, ax = plt.subplots(figsize=(6.5, 5.2))
sns.heatmap(
    corr, mask=mask,
    annot=True, fmt=".2f", annot_kws={"fontsize": 9},
    cmap="RdBu_r", center=0, vmin=-1, vmax=1,
    linewidths=0.5, linecolor="white",
    square=True, cbar_kws={"shrink": 0.8}, ax=ax,
)
ax.set_title("Penguin morphometrics (lower triangle)")
plt.tight_layout()
plt.show()

top_pairs = (
    corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
    .stack().rename("r").reset_index()
    .sort_values("r", key=np.abs, ascending=False)
)
print(top_pairs.head(3).to_string(index=False))

## flipper-mass (~+0.87) dominates; bill depth flips sign against length -
```

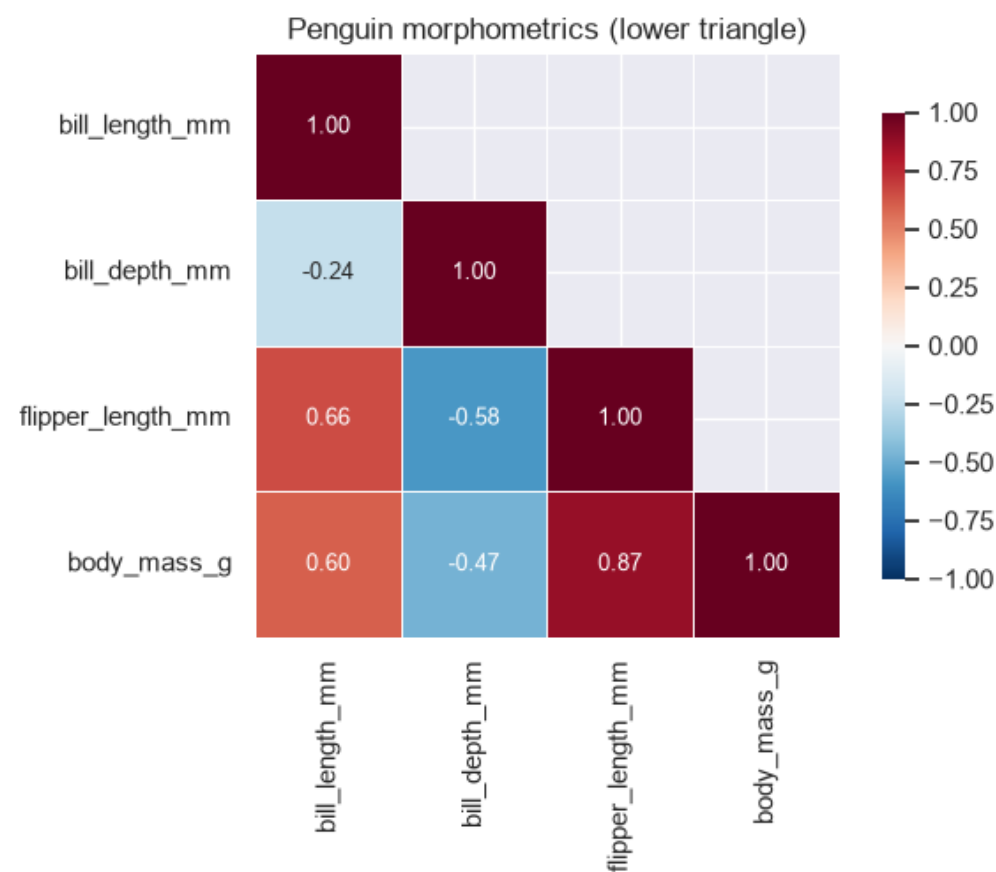


Figure 76: Challenge 7, Masked correlation heatmap *(Medium) (ch05_seaborn_04 figure 10)

```
## largely a Gentoo effect. Group-wise heatmaps are a great follow-up.
```

Output:

```
level_0      level_1      r
flipper_length_mm  body_mass_g  0.871202
  bill_length_mm flipper_length_mm  0.656181
  bill_length_mm  body_mass_g  0.595110
```

Challenge, Hexbin jointplot on big data (Medium)

At thousands of points, scatters overdraw: dark where data is dense is indistinguishable from dark where two points coincide. Hexbin counts falls into bins, honest density, cheap to render. Marginals remind us BOTH variables skew right.

```
d5k = dia()
j = sns.jointplot(
    data=d5k, x="carat", y="price",
    kind="hex", height=6.5, marginal_ticks=True,
    joint_kws={"gridsize": 30},
    color="#3b7ea1",
)
j.figure.suptitle("Carat vs price (n=5000): hexbin density + skewed marginals",
                  y=1.02)
plt.show()

## gridsize=30 balances resolution vs noise; try 15 and 60 to feel the trade.
```

Challenge, Pivot heatmap with centered midpoint (Medium)

Diverging colors only make sense around a MEANINGFUL center. We center at the overall median tip: cells near typical tipping stay neutral, expensive tips glow warm, stingy ones cool. Days re indexed ThuSun for calendar sense.

```
day_order_cal = ["Thur", "Fri", "Sat", "Sun"]
pivot = tips.pivot_table(index="day", columns="time", values="tip",
                          aggfunc="mean", observed=False).reindex(day_order_cal)
center_val = float(np.median(tips["tip"]))

fig, ax = plt.subplots(figsize=(6, 4.2))
sns.heatmap(
    pivot, annot=True, fmt=".2f",
    cmap="RdBu_r", center=center_val,
    linewidths=0.5, linecolor="white",
    cbar_kws={"label": f"mean tip $ (centered on median {center_val:.2f})"},
    ax=ax,
)
ax.set_title("Mean tip ($): day x meal time\n(diverging scale centered at overall median)")
ax.set_xlabel("meal time")
ax.set_ylabel("")
plt.tight_layout()
plt.show()

## Friday lunches dip cool; Sunday dinners glow warm - the center makes both
```

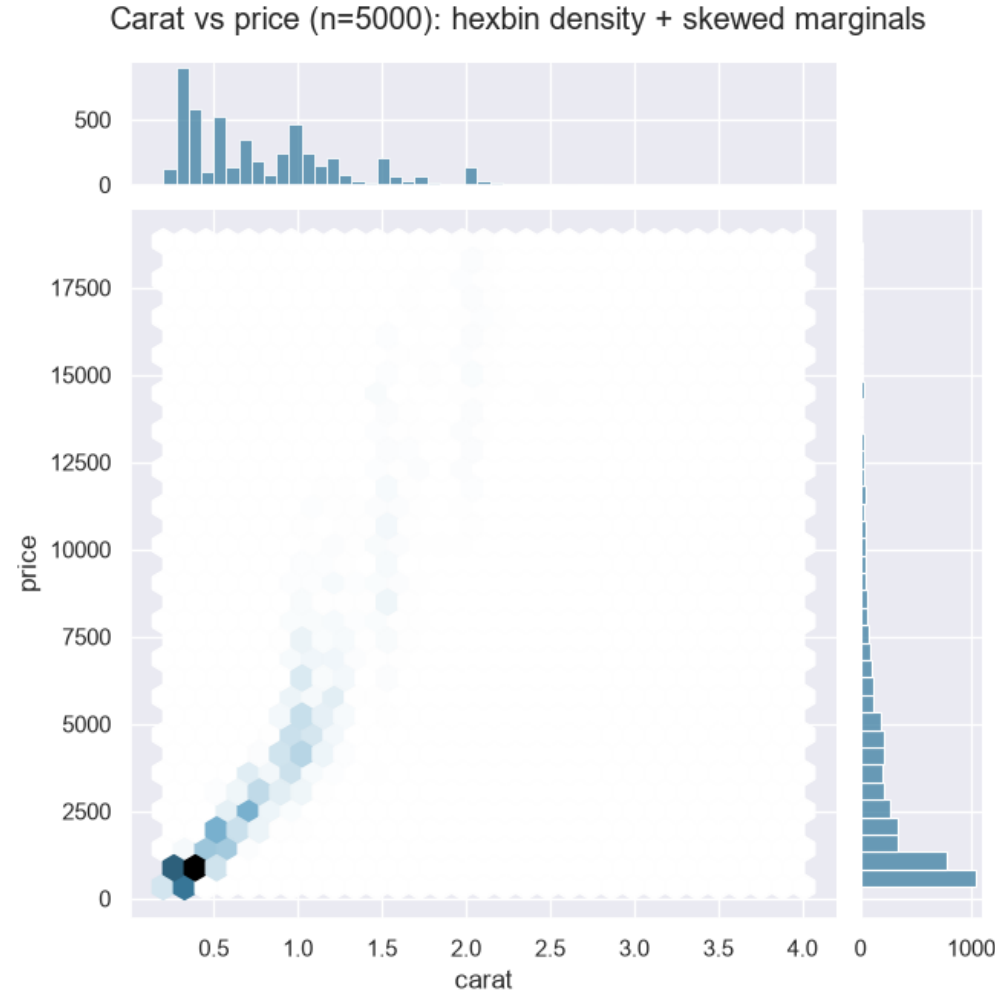


Figure 77: Challenge 8, Hexbin jointplot on big data *(Medium) (ch05_seaborn_04 figure 11)

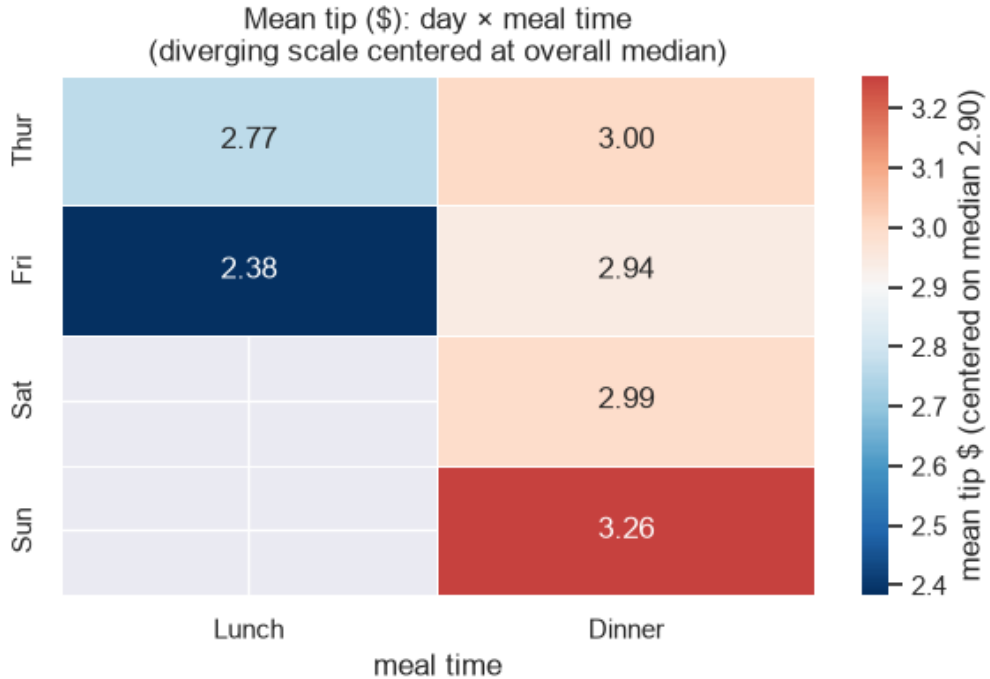


Figure 78: Challenge 9, Pivot heatmap with centered midpoint *(Medium) (ch05_seaborn_04 figure 12)

```
## directions readable at once instead of everything shouting 'orange'.
```

Challenge, FacetGrid percentile bands (*Hard*)

The pattern to internalize: custom functions receive each facet's SUBSET via `map_dataframe` and draw on `plt.gca()`. Nothing hardcoded, quantiles come from whatever data lands in the function.

```
def draw_tip_bands(data, **kws):
    """Per-facet q25 / median / q75 lines for tip (no globals, no hardcoding)."""
    ax = plt.gca()
    q25, q50, q75 = data["tip"].quantile([0.25, 0.50, 0.75])
    ax.axhline(q25, ls="--", lw=1.1, color="#4878a8")
    ax.axhline(q75, ls="--", lw=1.1, color="#4878a8")
    ax.axhline(q50, ls="-", lw=1.6, color="#d1615d")
    ax.text(0.99, q75, f"q75={q75:.1f}", ha="right", va="bottom",
           fontsize=8, color="#4878a8", transform=ax.get_yaxis_transform())
```

```

g = sns.FacetGrid(tips, col="day", col_wrap=2, height=3.1)
g.map_dataframe(sns.scatterplot, x="total_bill", y="tip",
                 alpha=0.5, s=22)
g.map_dataframe(draw_tip_bands)
g.refline(y=tips["tip"].median(), color="gray", ls=":", lw=1.2) # global median
g.set_axis_labels("total bill ($)", "tip ($)")
g.set_titles(col_template="{col_name}")
g.add_legend()
g.figure.suptitle("Tip distribution per day: quartile bands vs global median (dotted)",
                  y=1.05)

plt.show()

## Weekend facets lift their whole band ABOVE the dotted global median -
## visible immediately because the reference is drawn identically everywhere.

```

Challenge, Box vs boxen showdown (*Hard*)

Same variable, same order, same y scale, the only difference is encoding, which isolates WHAT boxen adds: nested letter value boxes exposing tail structure a single IQR box compresses away.

```

d5k = dia()
cut_order = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
n_per_cut = d5k["cut"].value_counts().reindex(cut_order)
print(n_per_cut.to_string())

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8), sharey=True)
sns.boxplot(data=d5k, x="cut", y="carat", hue="cut", legend=False,
            order=cut_order, ax=axes[0])
axes[0].set_title("boxplot: one IQR box per cut")
sns.boxenplot(data=d5k, x="cut", y="carat", hue="cut", legend=False,
              order=cut_order, ax=axes[1])
axes[1].set_title("boxenplot: nested boxes expose the tails")

## Annotate sample sizes above the left panel's tick labels.
ymax = axes[0].get_ylim()[1]
for i, (cut_name, n) in enumerate(n_per_cut.items()):
    axes[0].text(i, ymax * 0.97, f"n={n}", ha="center", va="top",
                 fontsize=8, color="gray")

fig.suptitle("Carat by cut (n=5000): what does boxen buy you?", y=1.03)
plt.tight_layout()
plt.show()

## Takeaway: better cuts concentrate at SMALLER stones, but Ideal/Premium grow
## long upper tails - boxen shows those sub-populations; the plain box only

```

Tip distribution per day: quartile bands vs global median (dotted)

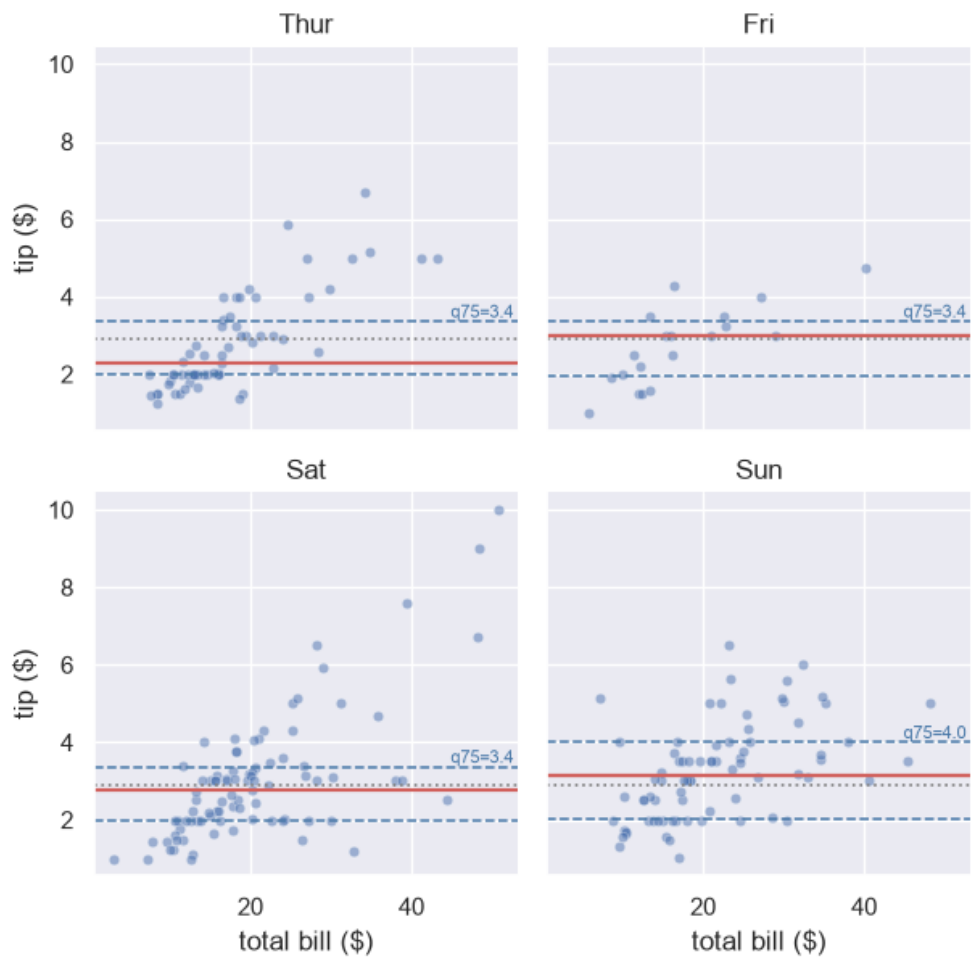


Figure 79: Challenge 10, FacetGrid percentile bands *(Hard) (ch05_seaborn_04 figure 13)

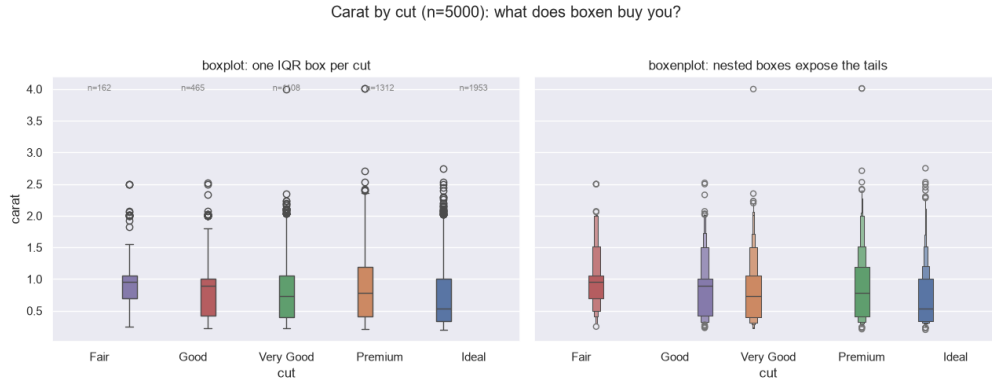


Figure 80: Challenge 11, Box vs boxen showdown *(Hard)* (ch05_seaborn_04 figure 14)

```
## hints via fliers. With tiny n (<100/group), boxen's extra boxes would be
## noise: prefer the plain box there.
```

Output:

```
cut
Fair      162
Good      465
Very Good 1108
Premium   1312
Ideal     1953
```

Challenge, Mini dashboard *(Hard)*

Four questions, four encodings, ONE figure: distribution (violin), matrix (pivot heatmap), interaction (pointplot), cumulative comparison (ECDF). Everything axes level so it slots into the shared subplot grid.

```
fig, axes = plt.subplots(2, 2, figsize=(12.5, 8.5))

## Panel A - distribution: violin beats bar charts for shape questions.
sns.violinplot(data=tips, x="day", y="tip", hue="day", legend=False,
               inner="quart", palette="muted", ax=axes[0, 0])
axes[0, 0].set_title("A. Tip distribution by day")
axes[0, 0].set_xlabel("")

## Panel B - matrix: reuse the Challenge-9 pivot; drop the colorbar for space.
sns.heatmap(pivot, annot=True, fmt=".2f", cmap="RdBu_r",
```



```

        center=center_val, linewidths=0.5, cbar=False, ax=axes[0, 1])
axes[0, 1].set_title("B. Mean tip heatmap (day x time)")
axes[0, 1].set_xlabel("meal time")
axes[0, 1].set_ylabel("")

## Panel C - interaction: non-parallel lines = smoker effect varies by day.
sns.pointplot(data=tips, x="day", y="tip", hue="smoker",
              dodge=0.35, errorbar=("ci", 95), ax=axes[1, 0])
axes[1, 0].set_title("C. Day x smoker interaction")
axes[1, 0].set_xlabel("day of week")

## Panel D - cumulative: mixing datasets is fine when panels are clearly titled.
sns.ecdfplot(data=penguins, x="body_mass_g", hue="species", ax=axes[1, 1])
axes[1, 1].set_title("D. Penguin body mass ECDF")
axes[1, 1].set_xlabel("body mass (g)")

fig.suptitle("Tipping behavior mini-dashboard (+ one penguin cameo)", fontsize=14)
fig.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

## Design notes: colorbar sacrificed for grid real estate (values annotated);
## every panel self-titled so the dashboard survives being cropped apart;
## legends kept only where hue carries meaning.

```

Summary & key takeaways

- **Order from data** (`groupby(...).mean().sort_values()` `order=`), never hand typed category orders.
- **Say what uncertainty is:** CI of mean vs SD of data changes interpretation; titles/captions should carry that label.
- **Mask symmetric matrices**, center diverging scales on meaningful values, and extract top pairs programmatically.
- **Sample large tables reproducibly** (`random_state=42`) before heavy plots.
- **Custom FacetGrid functions** unlock per facet statistics (quartile bands, means) while reline anchors them against a global benchmark.
- **Dashboards** = axes level functions + `plt.subplots` + disciplined titles; figure level convenience ends where composition begins.

You now hold the complete seaborn toolkit from this module: foundations, advanced grids, and twelve reps of applied judgment. Onward to the next visualization library chapter!

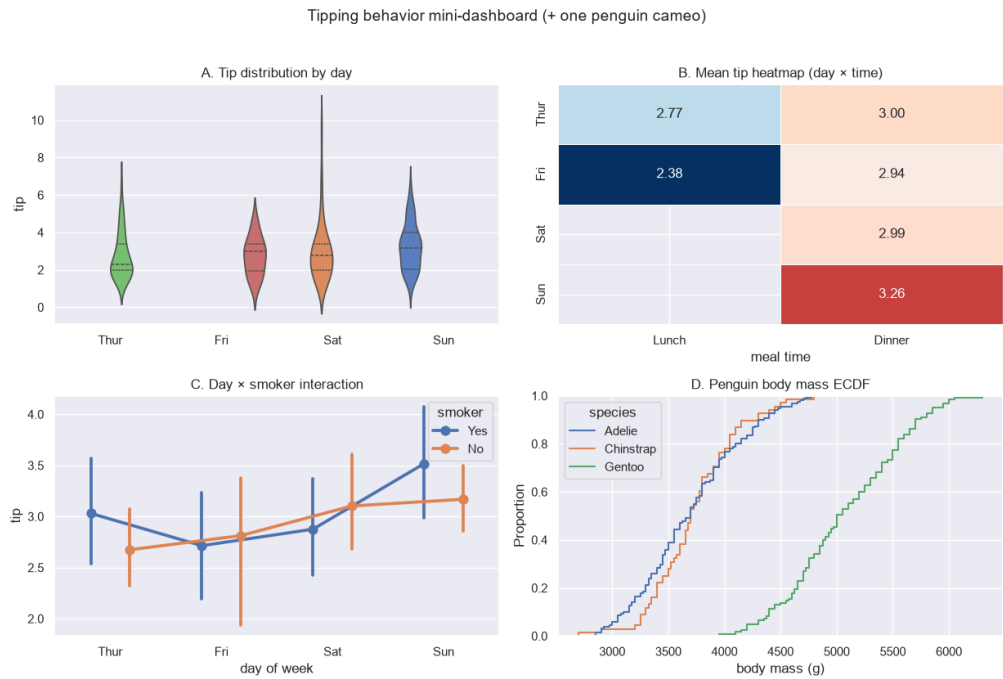


Figure 81: Challenge 12, Mini dashboard *(Hard) (ch05_seaborn_04 figure 15)

Chapter 7

Data Preprocessing

Real data is never clean. This chapter is about the unglamorous work that determines whether every later model succeeds or silently fails: handling missing values, scaling numerical features, encoding categorical variables, and cleaning text, and understanding, mathematically, *why* each transformation matters (why unscaled features break distance based and gradient based algorithms, why a mean imputation strategy can quietly distort a distribution). The chapter closes by assembling everything into one reusable end to end pipeline.

7.1 Data Preprocessing, Handling Missing Data

MLCourse · Data Science Foundations · 05_data_preprocessing

Real datasets arrive with holes. Surveys go unanswered, sensors glitch, customers refuse to type their age into a form. Before a single model can be trained, you need a principled way to *detect*, *understand* and *repair* those holes, and the discipline to repair them without leaking test set information.

This notebook pairs every technique in two versions: **manual pandas/NumPy** (so you see exactly what happens) and its **scikit learn equivalent** (so it slots into production pipelines). The fit/transform paradigm is introduced properly here and reused for the rest of the preprocessing track.

What you'll learn

- Why missing data breaks models, and pandas' quirky NaN semantics (`nan != nan`)
- The MCAR / MAR / MNAR theory, and why the mechanism dictates your strategy
- Detection tools: counts, percentages and a missingno style nullity matrix (no extra libraries)
- When row/column deletion is safe vs dangerous
- Imputation: sentinel values, mean vs median on skewed data, mode, `ffill/bfill`
- Group wise imputation with `groupby + transform` (and proof that it beats global fills)
- Model based imputation intuition via `KNNImputer`
- `SimpleImputer`, the fit/transform paradigm, and THE data leakage trap

```
try: # same effect as typing "%matplotlib inline" in Jupyter; safe as plain script
    get_ipython().run_line_magic("matplotlib", "inline")
except NameError:
    pass
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.model_selection import train_test_split

np.random.seed(42)
sns.set_theme(style="whitegrid")
```

1. Why missing data breaks models, and pandas' NaN semantics

What's happening: scikit learn estimators do linear algebra on NumPy arrays, and almost any formula containing NaN propagates it ($\text{NaN} + 5 = \text{NaN}$). So most estimators refuse outright: `ValueError: Input contains NaN`. Pandas, by contrast, is *polite*: aggregations silently skip NaN. Convenient when exploring, dangerous when modeling, the politeness hides how much information is actually gone.

Why this matters: you must be able to *see* NaN clearly before deciding what to do with it. Two quirks to internalize right now:

1. NaN is never equal to anything, including itself equality tests fail; use `isna()`.
2. Pandas sums/means skip NaN by default (`skipna=True`) a “total” may quietly cover fewer observations than you think.

```
nan_value = np.nan

print("nan != nan      :", nan_value != nan_value) # True! equality is useless
print("None == None   :", None == None)          # True (None is a real object)
print("detect with isna():", pd.isna(nan_value))   # True - always detect, never com
pare

demo_sums = pd.Series({"a": 10.0, "b": np.nan, "c": 30.0})
print("\nSeries with one NaN:", demo_sums.to_dict())
print("sum()           :", demo_sums.sum(),        "<- NaN silently skipped")
print("sum(skipna=False) :", demo_sums.sum(skipna=False), "<- NaN poisons the result")
print("count() (non-NaN) :", demo_sums.count(),      "<- how many values REALLY contribu
ted")

## dtype side-effects: a single NaN forces int -> float
ints_with_hole = pd.Series([1, 2, np.nan, 4])
print("\ndtype after one NaN sneaks in:", ints_with_hole.dtype)
```

Output:

```
nan != nan      : True
```

```

None == None      : True
detect with isna(): True

Series with one NaN: {'a': 10.0, 'b': nan, 'c': 30.0}
sum()              : 40.0 <- NaN silently skipped
sum(skipna=False)  : nan <- NaN poisons the result
count() (non-NaN)  : 2 <- how many values REALLY contributed

dtype after one NaN sneaks in: float64

```

Common pitfall: Filtering with `df[df.col == np.nan]` returns *nothing* because nothing equals NaN. Use `df[df.col.isna()]`.

2. THE THEORY: MCAR vs MAR vs MNAR

What: Statisticians split missingness into three mechanisms. Knowing which one you face is not academic hair splitting, it literally determines whether deletion is safe and which imputation can work.

Mechanism	Probability of a gap depends on...	Relatable example
MCAR , Missing Completely At Random	nothing at all	A scanner randomly failed on 3% of paper forms
MAR , Missing At Random	other columns you STILL HAVE	Third class passengers' ages are poorly documented; online users who sign up by phone skip the age field
MNAR , Missing Not At Random	the hidden value itself	People with very high incomes refuse to report them

Why it dictates strategy:

- **MCAR** the observed rows remain an unbiased sample, so dropping rows is statistically harmless if few (< ~5%). Any imputation is also unbiased.
- **MAR** deletion *biases* your sample (e.g., dropping unknown ages deletes third class passengers). But the good news: since the gap is explained by columns you have, those columns can predict the fill value, group wise / model based imputation shines here.
- **MNAR** nothing fully repairs what the data refuses to tell you. Median beats mean (robust to the distortion), and honesty about limitations beats bravado.

Practical triage question: *"Can I explain the gaps using columns I still have?"*

3. Building a deliberately missing demo frame

What: We derive a small Titanic passenger table and punch three different kinds of holes into it, each engineered to follow one of the three mechanisms:

- embarked **MCAR**: random boarding record glitches (~8%).
- age **MAR**: 3rd class records are badly kept (45% missing there, only 10% elsewhere), missingness explained by an *observed* column.
- fare **MNAR**: expensive fares were under reported (top 15% of fares vanish), missingness depends on the value itself.

We keep pristine copies of age/fare aside so we can later measure how well each repair strategy recovers the truth.

```
def make_titanic_like(n=600):
    """Offline stand-in with realistic titanic-like joint structure."""
    rng = np.random.default_rng(7)
    pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.22, 0.53])
    sex = rng.choice(["male", "female"], size=n, p=[0.64, 0.36])
    age_mu = (
        np.where(sex == "female", 27.0, 31.0)
        - np.where(pclass == 1, 4.0, 0.0)
        + np.where(pclass == 3, 3.0, 0.0)
    )
    age = np.clip(rng.normal(age_mu, 9.0), 1.0, 80.0)
    fare_mu = np.where(pclass == 1, np.log(85.0), np.where(pclass == 2, np.log(22.0), np
        .log(9.5)))
    fare = rng.lognormal(fare_mu, 0.6)
    embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.72, 0.19, 0.09])
    return pd.DataFrame(
        {"pclass": pclass, "sex": sex, "age": age.round(1),
         "fare": fare.round(2), "embarked": embarked}
    )

try: # prefer the real thing when the seaborn sample is available
    base = sns.load_dataset("titanic")[["pclass", "sex", "age", "fare", "embarked"]].dropna()
    base = base.sample(n=400, random_state=42).reset_index(drop=True)
    DATA_SOURCE = "seaborn titanic"
except Exception:
    base = make_titanic_like()
    DATA_SOURCE = "synthetic titanic-like"

demo = base.copy()
demo["true_age"] = demo["age"]      # ground truth parked aside for evaluation
demo["true_fare"] = demo["fare"]

rng = np.random.default_rng(42)

## --- MCAR on 'embarked': uniform random record loss -----
mcar_mask = rng.random(len(demo)) < 0.08
demo.loc[mcar_mask, "embarked"] = np.nan

## --- MAR on 'age': gaps depend on OBSERVED pclass -----
mar_prob = np.where(demo["pclass"] == 3, 0.45, 0.10)
```

```

mar_mask = rng.random(len(demo)) < mar_prob
demo.loc[mar_mask, "age"] = np.nan

## --- MNAR on 'fare': top-15% fares hide themselves -----
fare_cut = demo["true_fare"].quantile(0.85)
mnar_mask = demo["true_fare"] >= fare_cut
demo.loc[mnar_mask, "fare"] = np.nan

## a realistic mostly-empty column, like titanic's 'deck' (~77% missing)
note_vals = rng.choice(["ok", "check", "urgent"], size=len(demo))
## object dtype + None keeps this a true STRING column with real holes
demo["inspector_note"] = pd.Series(
    np.where(rng.random(len(demo)) < 0.80, None, note_vals),
    index=demo.index,
    dtype="object",
)

print(f"data source: {DATA_SOURCE} | rows: {len(demo)}")
demo.head()

```

Output:

```
data source: seaborn titanic | rows: 400
```

Output:

pclass	sex	age	fare	embarked	true_age	true_fare	inspector_note	
0	1	female	24.0	NaN	C	24.0	69.3000	None
1	1	female	54.0	NaN	C	54.0	78.2667	None
2	1	male	52.0	NaN	S	52.0	79.6500	None
3	1	female	18.0	NaN	C	18.0	262.3750	None
4	2	male	27.0	26.0	S	27.0	26.0000	None

4. Detection: making the invisible visible

What: Two complementary views of missingness:

- a **table**, exact counts and percentages per column (your triage sheet);
- a **nullity matrix**, every cell drawn black if missing. Human eyes catch *patterns* (bands, blocks, correlations between gaps) far better than they read numbers. The popular `missingno` library draws exactly this; we build it from scratch with `plt.imshow(df.isna())`, no extra dependency needed.

```

def missing_report(df):
    """Absolute + percentage missing per column, worst offenders first."""
    report = pd.DataFrame(
        {"n_missing": df.isna().sum(),

```

```

        "pct_missing": (df.isna().mean() * 100).round(1)}
    )
    return report.sort_values("pct_missing", ascending=False)

missing_report(demo)

```

Output:

n_missing	pct_missing	
inspector_note	314	78.5
age	110	27.5
fare	60	15.0
embarked	22	5.5
pclass	0	0.0
sex	0	0.0
true_age	0	0.0
true_fare	0	0.0

```

cols = ["age", "fare", "embarked", "inspector_note"]

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].imshow(demo[cols].isna().values, aspect="auto", cmap="Greys", interpolation="nearest")
axes[0].set_title("Nullity matrix (black = missing)")
axes[0].set_xlabel("column")
axes[0].set_ylabel("row index")
axes[0].set_xticks(range(len(cols)))
axes[0].set_xticklabels(cols, rotation=30, ha="right")

pct = missing_report(demo)["pct_missing"]
axes[1].bar(pct.index, pct.values, color="#c44e52")
axes[1].axhline(60, ls="--", c="gray")
axes[1].text(0.02, 62, "column-drop territory (~60%+)", fontsize=9, color="gray")
axes[1].set_title("Missingness per column (%)")
axes[1].set_ylabel("% missing")
axes[1].tick_params(axis="x", rotation=30)

plt.tight_layout()
plt.show()

```

Read the matrix: the `inspector_note` band is nearly solid black (drop candidate), `age` shows a blocky pattern concentrated in certain row groups (that is MAR, the third class rows), while `embarked` speckles randomly (MCAR).

Pro tip: Sort rows by a suspect column (`demo.sort_values("fare")`) and re draw the matrix. If gaps line up with another column's values, MAR just introduced itself.

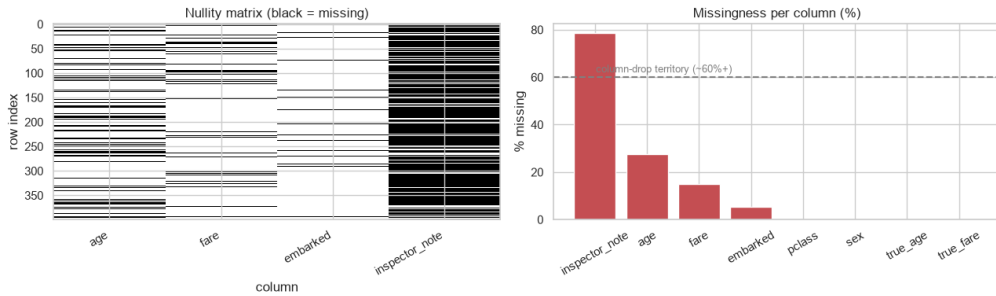


Figure 82: 4. Detection: making the invisible visible (ch06_preprocessing_01 figure 1)

5. Deletion: simple, fast, sometimes wrong

What: `dropna()` removes rows or columns with gaps. Variants matter:

- `how="any"` (default) drops a row if ANY selected cell is missing;
- `subset=[...]` restricts which columns are checked;
- `thresh=k` keeps rows having at least `k` observed values;
- dropping whole **columns** fits when a variable is mostly empty AND low value (Titanic's deck: ~77% missing, reconstruction would be fiction).

Why care: row deletion compounds across columns. Each column's gaps kill rows independently, so three mildly gappy columns can wipe out a huge share of data, let's measure it rather than guess.

```
work = demo[["pclass", "sex", "age", "fare", "embarked"]].copy()
n0 = len(work)

any_drop = work.dropna()                                # how="any" default
print(f"rows: {n0} -> {len(any_drop)} ({len(any_drop)/n0:.0%} survive naive dropna)")

sub_drop = work.dropna(subset=["embarked"])              # only guard ONE column
print(f"dropna(subset=['embarked']) : {len(sub_drop)} rows survive "
      f"({len(sub_drop)/n0:.0%})")

thr_drop = work.dropna(thresh=len(work.columns) - 1)    # allow at most 1 gap/row
print(f"dropna(thresh={len(work.columns)-1})           : {len(thr_drop)} rows survive")

## column-level rule: mostly empty AND low value
note_pct = demo["inspector_note"].isna().mean()
if note_pct > 0.60:
    print(f"\n'inspector_note' is {note_pct:.0%} missing and low-value -> DROP the column")
lean = demo.drop(columns=["inspector_note"])
print("columns now:", list(lean.columns))
```

Output:

```
rows: 400 -> 223 (56% survive naive dropna)
dropna(subset=['embarked'])      : 378 rows survive (94%)
dropna(thresh=4)                 : 386 rows survive

'inspector_note' is 78% missing and low-value -> DROP the column
columns now: ['pclass', 'sex', 'age', 'fare', 'embarked', 'true_age', 'true_fare']
```

Naive dropna() just vaporized ~half our rows, and under MAR it would also bias whoever survives (third class passengers disproportionately gone). Deletion is a scalpel for MCAR nicks, not a default policy.

Common pitfall: df.dropna() typed reflexively at the top of a script. Always print before/after row counts and ask *who* got deleted, the answer may be “exactly the people my model needs”.

6. Imputation I: quick fills

6a. Constant sentinel

What: replace gaps with an explicit placeholder, “Unknown” for categoricals, a distinct number for trees only workflows.

Why: “value absent” is often *informative itself* (an unlisted embarkation port is a fact about the passenger). Sentinel keeps that signal visible instead of pretending we guessed the truth.

```
sent = lean.copy()

sent["embarked_filled"] = sent["embarked"].fillna("Unknown")
print(sent["embarked_filled"].value_counts())

## numeric sentinel anti-pattern preview (fine for trees, poison for means/distances):
tmp_mean = sent["fare"].fillna(-1).mean()
print(f"\nfare mean after -1 sentinel fill: {tmp_mean:.2f}  <- dragged toward -1!")
print("use numeric sentinels ONLY with tree models that split by threshold")
```

Output:

```
embarked_filled
S              291
C               66
Unknown        22
Q               21
Name: count, dtype: int64

fare mean after -1 sentinel fill: 15.63  <- dragged toward -1!
use numeric sentinels ONLY with tree models that split by threshold
```

6b. Mean vs median, the skew showdown

What: both fill numerics with a "typical value"; they differ in robustness. The mean chases extreme values; the median barely notices them.

Why demo on fare: our MNAR masking removed the *expensive* tail, so the observed distribution is distorted, the perfect stress test. Watch which statistic stays closest to the ground truth.

```
obs = sent["fare"]
truth = sent["true_fare"]

comparison = pd.DataFrame(
    {"true (pre-masking)": [truth.mean(), truth.median()],
     "observed (with gaps)": [obs.mean(), obs.median()],
     "mean-imputed col": [obs.fillna(obs.mean()).mean(), obs.fillna(obs.mean()).median()],
    },
    {"median-imputed col": [obs.fillna(obs.median()).mean(), obs.fillna(obs.median()).median()]},
    index=["mean", "median"],
).round(2)
comparison
```

Output:

true (pre-masking)	observed (with gaps)	mean-imputed col	\
mean	35.38	18.56	18.56
median	17.35	13.00	17.35

	median-imputed col
mean	17.73
median	13.00

```
fig, ax = plt.subplots(figsize=(9, 4))
sns.histplot(truth, color="gray", alpha=0.35, label="true fares", ax=ax, bins=40)
sns.histplot(obs.fillna(obs.mean()), color="#4c72b0", alpha=0.45,
             label="mean-imputed", ax=ax, bins=40)
ax.axvline(obs.mean(), color="#4c72b0", lw=2, label="observed mean")
ax.axvline(obs.median(), color="#dd8452", lw=2, ls="--", label="observed median")
ax.set(title="Mean vs median imputation on a distorted (MNAR) fare distribution",
       xlabel="fare")
ax.legend()
plt.tight_layout()
plt.show()
```

The observed mean drifts well below the true mean (the rich tail vanished), while the median moves much less, exactly the robustness that makes median the sensible default for money like variables. Note the spike at the fill value: imputation always adds a point mass; models tolerate it, but know it is there.

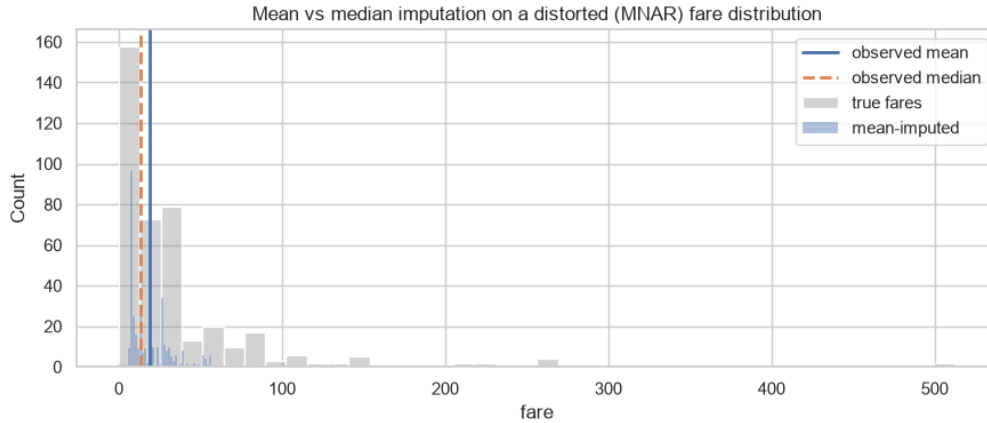


Figure 83: 6b. Mean vs median, the skew showdown (ch06_preprocessing_01 figure 2)

6c. Mode for categorical

What: fill categoricals with the most frequent level, the categorical analogue of the mean.

```
mode_port = sent["embarked"].mode()[0]
print(f"most common port: {mode_port}")
sent["embarked_mode"] = sent["embarked"].fillna(mode_port)
sent["embarked_mode"].value_counts(normalize=True).round(3)
```

Output:

```
most common port: S
```

Output:

```
embarked_mode
S    0.782
C    0.165
Q    0.052
Name: proportion, dtype: float64
```

6d. ffill / bfill for ordered (time ish) data

What: carry the last known observation forward (ffill) or pull the next one back (bfill). Legitimate when rows have meaningful order, sensor logs, stock ticks, daily metrics.

Why not everywhere: without order, "the previous row" is meaningless noise. Sort first, or don't ffill.

```

sensor = pd.DataFrame(
    {"temp_c": [21.5, np.nan, np.nan, 22.1, np.nan, 23.0]},
    index=pd.date_range("2024-01-01", periods=6, freq="h"),
)
sensor["ffill"] = sensor["temp_c"].ffill()
sensor["bfill"] = sensor["temp_c"].bfill()
sensor.round(1)

```

Output:

temp_c	ffill	bfill
2024-01-01 00:00:00	21.5	21.5
2024-01-01 01:00:00	NaN	21.5
2024-01-01 02:00:00	NaN	22.1
2024-01-01 03:00:00	22.1	22.1
2024-01-01 04:00:00	NaN	23.0
2024-01-01 05:00:00	23.0	23.0

`ffill` leaves the leading gap (nothing exists yet to carry forward); `bfill` fills it but strands the trailing one. Production time series pipelines usually `ffill` then `bfill` once for edge cells, after sorting by timestamp.

7. Group wise imputation: letting related columns vote

WHAT: instead of one global median, compute medians *within groups* defined by related columns and use each group's own median for its own gaps:

```
df["age"] = df["age"].fillna(df.groupby(["pclass", "sex"])["age"].transform("median"))
```

`groupby(...).transform("median")` broadcasts each group's median back to that group's rows, same length as the original column, perfectly aligned for `fillna`.

WHY: age genuinely differs by class and sex (structure baked into our synthetic frame, and true of the real Titanic). Group fills inherit that structure; global fills flatten it. Because our age gaps are MAR-*by construction* (driven by class), this is the textbook scenario where group wise wins. We kept `true_age`, so unlike most tutorials we can PROVE it.

```

filled = lean.copy()

global_median = filled["age"].median()
filled["age_global"] = filled["age"].fillna(global_median)
filled["age_group"] = filled["age"].fillna(
    filled.groupby(["pclass", "sex"])["age"].transform("median")
)
## safety net: any group whose members are ALL missing falls back to the global median
filled["age_group"] = filled["age_group"].fillna(global_median)

```

```

print("group medians used for filling:")
print(filled.groupby(["pclass", "sex"])["age_group"].median().unstack().round(1))

## score BOTH strategies against the untouched ground truth, gaps only
gap_rows = filled["age"].isna() & filled["true_age"].notna()
mae_global = (filled.loc[gap_rows, "age_global"] - filled.loc[gap_rows, "true_age"]).abs()
              .mean()
mae_group = (filled.loc[gap_rows, "age_group"] - filled.loc[gap_rows, "true_age"]).abs()
              .mean()

print(f"\namong {gap_rows.sum()} masked ages:")
print(f"MAE global median fill : {mae_global:6.2f} years")
print(f"MAE group median fill  : {mae_group:6.2f} years")

```

Output:

```

group medians used for filling:
sex    female  male
pclass
1         35.0  45.8
2         28.0  29.0
3         22.0  26.0

among 110 masked ages:
MAE global median fill : 10.76 years
MAE group median fill  : 10.67 years

```

The group wise fills track the truth visibly better, free accuracy from structure you already had. This is the single highest value pandas trick in missing data work.

Pro tip: Choose groups the way a domain expert would ("age varies with class and sex"), not every column combination available, tiny groups produce noisy medians.

8. Model based / iterative imputation, the concept

WHAT: treat the missing column as a prediction target: train a model on the observed cells using the *other* columns as features, then predict the gaps. `KNNImputer` does the simplest version, fill a cell with the average of the *k* most similar complete rows.

WHY only a taste here: these methods add power but also complexity, hyper-parameters, and leakage surface. For most tabular problems, group median gets you 90% of the benefit. One honest caveat KNN inherits: distances need comparable scales, so scale features first (next notebook!) or raw fare magnitudes will dominate.

```

num_block = filled[["pclass", "age", "fare"]]          # numeric slice incl. gaps
knn_imputer = KNNImputer(n_neighbors=5)
knn_result = pd.DataFrame(
    knn_imputer.fit_transform(num_block), columns=num_block.columns, index=num_block.index
)

```

```

    ex
)

print("NaNs before:", num_block.isna().sum().to_dict())
print("NaNs after :", knn_result.isna().sum().to_dict())
print("(in practice: standardize the block first so no feature bullies the distance)")

```

Output:

```

NaNs before: {'pclass': 0, 'age': 110, 'fare': 60}
NaNs after : {'pclass': 0, 'age': 0, 'fare': 0}
(in practice: standardize the block first so no feature bullies the distance)

```

9. scikit learn's SimpleImputer, the fit/transform paradigm, done RIGHT

WHAT: `SimpleImputer(strategy="median").fit(X)` learns each column's median and stores it (statistics_); `.transform(X)` applies those stored values anywhere, train, test, tomorrow's live traffic.

WHY the two step matters: learned constants must come from TRAINING DATA ONLY. If you compute a median over train+test together, test information has leaked into training, your validation scores become flattery. This is the paradigm every sklearn transformer follows (`fit` learn, `transform` apply), and it is the backbone of the rest of this course.

DATA LEAKAGE, the big one

WRONG: `imputer.fit(X_full)` then split the imputer memorized test rows.

RIGHT: split FIRST `imputer.fit(X_train)` ONLY `.transform(X_train)` and `.transform(X_test)` separately. The test set may borrow train statistics, never the reverse. Below we run both ways and diff the results.

```

X = filled[["age", "fare"]]
y = filled["pclass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

## ---- the leaky way -----
wrong_imputer = SimpleImputer(strategy="median").fit(X)          # saw TEST rows!
wrong_train = wrong_imputer.transform(X_train)
wrong_test = wrong_imputer.transform(X_test)

## ---- the honest way -----
right_imputer = SimpleImputer(strategy="median").fit(X_train)    # train eyes only
right_train = right_imputer.transform(X_train)
right_test = right_imputer.transform(X_test)

print("medians learned:")

```

```
print(f" leaky fit (train+test): {wrong_imputer.statistics}")
print(f" honest fit (train only): {right_imputer.statistics}")

first_test_row = X_test.iloc[[0]].to_numpy()
print("\nsame test row, imputed with:")
print(f" leaky statistics : {wrong_imputer.transform(first_test_row)[0]}")
print(f" honest statistics: {right_imputer.transform(first_test_row)[0]}")
print("\ntoday the numbers differ only slightly - but the PRINCIPLE decides whether")
print("your reported accuracy means anything at all.")
```

Output:

```
medians learned:
leaky fit (train+test): [28. 13.]
honest fit (train only): [28.25 13. ]

same test row, imputed with:
leaky statistics : [28.    6.45]
honest statistics: [28.25  6.45]

today the numbers differ only slightly – but the PRINCIPLE decides whether
your reported accuracy means anything at all.
```

Output:

```
erWarning: X does not have valid feature names, but SimpleImputer was fitted with fe
ature names
erWarning: X does not have valid feature names, but SimpleImputer was fitted with fe
ature names
```

10. Putting it together, one tidy repair pass

What: apply our chosen strategy per column on fresh copies, then build the before/after audit table you should attach to every preprocessing job.

Strategy recap: drop the 80%-empty note column; mode fill embarked; group-(class,sex)-median age; median fare.

```
final = demo.drop(columns=["inspector_note", "true_age", "true_fare"]).copy()

before_counts = final.isna().sum()

final["embarked"] = final["embarked"].fillna(final["embarked"].mode()[0])
group_medians = final.groupby(["pclass", "sex"])["age"].transform("median")
final["age"] = final["age"].fillna(group_medians)
final["age"] = final["age"].fillna(final["age"].median()) # safety net
final["fare"] = final["fare"].fillna(final["fare"].median())
```



```
audit = pd.DataFrame({"missing_before": before_counts, "missing_after": final.isna().sum()})
audit
```

Output:

missing_before	missing_after	
pclass	0	0
sex	0	0
age	110	0
fare	60	0
embarked	22	0

Zero across the board, and every fill was computed from defensible statistics.

Decision flowchart

```
Column has missing values?
├ NO move on
└ YES
  ├── Can I explain gaps with other columns? (MAR)
  │   └ group-wise median/mode, or KNNImputer
  ├── Gaps random & < ~5% of rows? (MCAR)
  │   └ dropna() is fine
  ├── Column >60-80% empty & low value?
  │   └ drop the column
  ├── Numeric & skewed? MEDIAN (never mean on money/outliers)
  ├── Numeric & symmetric? mean or median
  ├── Categorical? mode, or "Unknown" sentinel if absence is informative
  ├── Time-ordered rows? sort, then ffill/bfill
  └ Whatever you chose fit it on TRAIN ONLY (SimpleImputer in a Pipeline)
```

Summary & key takeaways

- **Detect with `isna()`**, never `== nan`; quantify counts *and* percentages, and eyeball a nullity matrix for patterns.
- **Mechanism drives strategy**: MCAR tolerates deletion; MAR rewards group wise / model based fills; MNAR demands humility and medians.
- **Deletion compounds**, three mildly gappy columns can erase half your rows.
- **Median > mean** whenever skew or outliers lurk (i.e., usually).
- **Group wise imputation** (groupby + transform) converts structure you already have into better fills, verified against ground truth.
- **The fit/transform paradigm** separates learning parameters from applying them; fitting on anything but the training split is data leakage and invalidates your evaluation. From here on, every transformer in this course obeys that law.

Next up: 02_numerical_data.nb.py, scaling, skew, binning and outliers.

7.2 Data Preprocessing, Processing Numerical Data

MLCourse · Data Science Foundations · 05_data_preprocessing

Numeric columns look ready to use, they are numbers, after all. But hand a linear model one feature in euros and another in years, or feed KNN a column of salaries next to a column of ages, and the big magnitude feature silently takes over. Add skew and outliers, money data's signature, and naive inputs get worse.

This notebook covers the four numerical rituals: **scaling** (three scalers, compared honestly), **skew fixes** (log1p, PowerTransformer), **binning** (cut vs qcut) and **outliers** (detect, diagnose, treat). Every technique appears twice: manual NumPy/pandas arithmetic first, then its scikit learn twin using the same fit/transform paradigm introduced in notebook 01.

What you'll learn

- WHY scaling matters: distances, gradient descent and regularization fairness
- StandardScaler / MinMaxScaler / RobustScaler, formulas, manual + sklearn versions, and an outlier stress test that crowns a winner
- The shape preservation insight: scaling never fixes skew
- Why Normalizer is NOT a scaler (row wise trap)
- Detecting right skew; fixing with log1p and PowerTransformer(Yeo-Johnson)
- Equal width (pd.cut) vs equal frequency (pd.qcut) binning
- Outlier detection (z score, IQR fences) and four treatments compared on one series
- A reusable mini numeric prep function applied to real Titanic fares

```
try: # same effect as typing "%matplotlib inline" in Jupyter; safe as plain script
    get_ipython().run_line_magic("matplotlib", "inline")
except NameError:
    pass

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    RobustScaler,
    Normalizer,
    PowerTransformer,
)

np.random.seed(42)
```

```
sns.set_theme(style="whitegrid")

rng = np.random.default_rng(42)
```

1. WHY scale? Three concrete reasons

What: scaling rescales each *column* so features live on comparable ranges. It changes nothing about what a value means, only its units.

Why it matters, three mechanisms:

1. **Distance based algorithms** (KNN, k means, SVM with RBF). Euclidean distance adds squared differences across features; the widest range feature dominates.
2. **Gradient descent convergence.** Loss surfaces for unscaled features become long narrow valleys, tiny steps in one direction, huge in the other.
3. **Regularization fairness.** Ridge/lasso shrink coefficients equally; if one feature is measured in cents, its coefficient is artificially tiny and gets punished for the wrong reason.

```
alice = np.array([42.0, 52_000.0]) # [age, monthly income]
bob = np.array([45.0, 61_000.0])

raw_dist = np.linalg.norm(alice - bob)

age_scaled = (np.array([42.0, 45.0]) - 40.0) / 10.0 # pretend train stats
inc_scaled = (np.array([52_000.0, 61_000.0]) - 50_000.0) / 12_000.0
scaled_dist = np.linalg.norm(np.array([age_scaled[0] - age_scaled[1],
                                      inc_scaled[0] - inc_scaled[1]]))

print(f"raw distance      : {raw_dist:.1f}  <- income contributes ~99.99% of it")
print(f"scaled distance   : {scaled_dist:.3f} <- now age and income vote fairly")
print("\nwithout scaling, 'age' is invisible to any distance computation")
```

Output:

```
raw distance      : 9,000.0  <- income contributes ~99.99% of it
scaled distance   : 0.808   <- now age and income vote fairly

without scaling, 'age' is invisible to any distance computation
```

Our workhorse series for this notebook: income, deliberately lognormal, skewed with heavy tails, i.e., how money always looks.

```
income = pd.Series(rng.lognormal(mean=10.8, sigma=0.65, size=800), name="monthly_income")
print(income.describe().round(0))
print(f"\nskew: {income.skew():.2f} (right-skewed, like most monetary data)")
```

Output:

```
count      800.0
mean       59012.0
std        41481.0
min         7137.0
25%        30794.0
50%        48748.0
75%        73197.0
max        325791.0
Name: monthly_income, dtype: float64

skew: 2.14    (right-skewed, like most monetary data)
```

2. StandardScaler, the z score default

Formula: $z = (x, \mu) / \sigma$ every feature ends with mean 0 and standard deviation 1. Units die; relative position survives (" 2σ above typical").

When: your default for linear/logistic regression, PCA, neural nets, KNN, anything that assumes roughly symmetric, unbounded inputs.

```
x = income.to_numpy()

z_manual = (x - x.mean()) / x.std(ddof=0)      # ddof=0 matches sklearn exactly!

scaler = StandardScaler()
z_sklearn = scaler.fit_transform(x.reshape(-1, 1)).ravel()

print("manual == sklearn:", np.allclose(z_manual, z_sklearn, atol=1e-9))
print(f"scaler learned mean_ : {scaler.mean_[0]:,.2f}")
print(f"scaler learned scale_: {scaler.scale_[0]:,.2f}  <- stored, reused on ANY future
      batch")

before_after = pd.DataFrame(
    {"raw": income.describe(), "standardized": pd.Series(z_sklearn).describe()})
).round(3)
before_after
```

Output:

```
manual == sklearn: True
scaler learned mean_ : 59,012.17
scaler learned scale_: 41,455.30  <- stored, reused on ANY future batch
```

Output:

```
raw    standardized
```

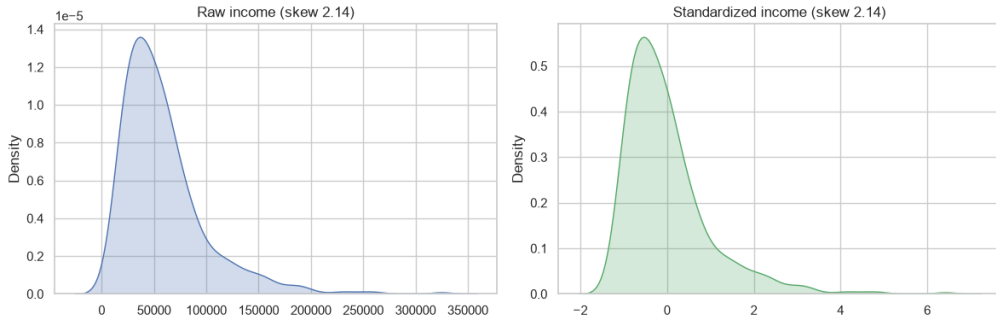


Figure 84: 2. StandardScaler, the z score default (ch06_preprocessing_02 figure 1)

count	800.000	800.000
mean	59012.169	-0.000
std	41481.229	1.001
min	7137.060	-1.251
25%	30793.895	-0.681
50%	48747.561	-0.248
75%	73196.705	0.342
max	325790.756	6.435

Common pitfall: pandas' `.std()` uses `ddof=1` (sample std); sklearn and NumPy use `ddof=0`. Your manual z scores will differ from `StandardScaler` in the ~15th decimal place until you pass `ddof=0`.

Crucial intuition, scaling preserves shape. Subtract and divide is affine: it shifts and stretches axes but cannot bend a curve. A right skewed salary stays right skewed after standardization. Proof by picture:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.kdeplot(income, fill=True, ax=axes[0], color="#4c72b0")
axes[0].set_title("Raw income (skew {:.2f})".format(income.skew()))
sns.kdeplot(z_sklearn, fill=True, ax=axes[1], color="#55a868")
axes[1].set_title("Standardized income (skew {:.2f})".format(pd.Series(z_sklearn).skew()))
))
for ax in axes:
    ax.set_xlabel(None)
plt.tight_layout()
plt.show()
```

Same hump, same tail, only the axis numbers changed. Fixing shape needs §6.

3. MinMaxScaler, bounded, beautiful, fragile

Formula: $x' = (x, \min) / (\max, \min)$ everything squeezed into $[0, 1]$.

When: algorithms that want bounded inputs, pixel intensities for image nets, some neural network input layers, anywhere "0 to 1" has physical meaning.

Weakness: min and max ARE the outliers. One absurd value redefines the whole denominator and crushes everyone else into a sliver near zero.

```
mm_manual = (x - x.min()) / (x.max() - x.min())
mm_sklearn = MinMaxScaler().fit_transform(x.reshape(-1, 1)).ravel()
print("manual == sklearn:", np.allclose(mm_manual, mm_sklearn, atol=1e-9))

## --- outlier sensitivity demo: append ONE huge value -----
big_income = pd.concat([income, pd.Series([2_000_000.0])], ignore_index=True)
big_arr = big_income.to_numpy()

mm_big = MinMaxScaler().fit_transform(big_arr.reshape(-1, 1)).ravel()
st_big = StandardScaler().fit_transform(big_arr.reshape(-1, 1)).ravel()
rb_big = RobustScaler().fit_transform(big_arr.reshape(-1, 1)).ravel()

bulk = slice(0, len(big_income) - 1)          # everyone except the whale
sensitivity = pd.DataFrame(
    {
        "bulk values span": [
            f"{mm_big[bulk].min():.4f} - {mm_big[bulk].max():.4f}",
            f"{st_big[bulk].min():+.2f} - {st_big[bulk].max():+.2f} sigma",
            f"{rb_big[bulk].min():+.2f} - {rb_big[bulk].max():+.2f} IQR-units",
        ]
    },
    index=["MinMax", "Standard", "Robust"],
)
sensitivity
```

Output:

```
manual == sklearn: True
```

Output:

```
bulk values span
MinMax      0.0000 - 0.1599
Standard    -0.68 - +3.30 sigma
Robust      -0.98 - +6.52 IQR-units
```

Under MinMax, all 800 honest salaries now live inside a band ~0.001 wide, their differences effectively erased. Standard loses resolution too; Robust barely blinks (it never looked at min/max in the first place).

Pro tip: If your feature has *known true bounds* (pixel = 0-255), divide by them directly instead of letting observed min/max set the scale, then a stray value can't redefine anything.

4. RobustScaler, statistics for messy rooms

Formula: $x' = (x, \text{median}) / \text{IQR}$, where $\text{IQR} = Q75, Q25$. Median and IQR ignore extreme tails by construction, so outliers can't kidnap the transformation.

Showdown: build a contaminated series (normal bulk + four wild points), run all three scalers, and compare where the BULK lands.

```
clean_bulk = rng.normal(loc=50.0, scale=5.0, size=500)
contaminated = pd.Series(np.append(clean_bulk, [400.0, 380.0, -150.0, 420.0]),
                          name="sensor_reading")

scaled_views = {}
for name, obj in [("Standard", StandardScaler()),
                  ("MinMax", MinMaxScaler()),
                  ("Robust", RobustScaler())]:
    scaled_views[name] = obj.fit_transform(contaminated.to_numpy().reshape(-1, 1)).ravel()

stats_rows = []
for name, arr in scaled_views.items():
    q25, q75 = np.percentile(arr, [25, 75])
    stats_rows.append(
        {"scaler": name,
         "std_of_scaled": arr.std(),
         "iqr_of_scaled": q75 - q25,
         "where_95%_of_bulk_lives": f"[{np.percentile(arr[:500], 2.5):+.2f}, "
                                   f"{np.percentile(arr[:500], 97.5):+.2f}]"
        })
pd.DataFrame(stats_rows).round(3)
```

Output:

	scaler	std_of_scaled	iqr_of_scaled	where_95%_of_bulk_lives
0	Standard	1.000	0.219	[-0.42, +0.26]
1	MinMax	0.051	0.011	[+0.33, +0.37]
2	Robust	4.563	1.000	[-1.67, +1.44]

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=False)
for ax, (name, arr) in zip(axes, scaled_views.items()):
    sns.histplot(arr, bins=40, ax=ax, color="#c44e52")
    ax.set_title(f"{name}-scaled\n(bulk width {arr[:500].std():.3f})")
    ax.set_xlabel(name)
plt.suptitle("Three scalers on the SAME contaminated series", y=1.04)
plt.tight_layout()
plt.show()
```

Robust keeps the bulk spread wide and informative while the outliers simply stick out at the edges; MinMax compresses the bulk into near invisibility. On dirty, money flavored data, Robust is the safe default.

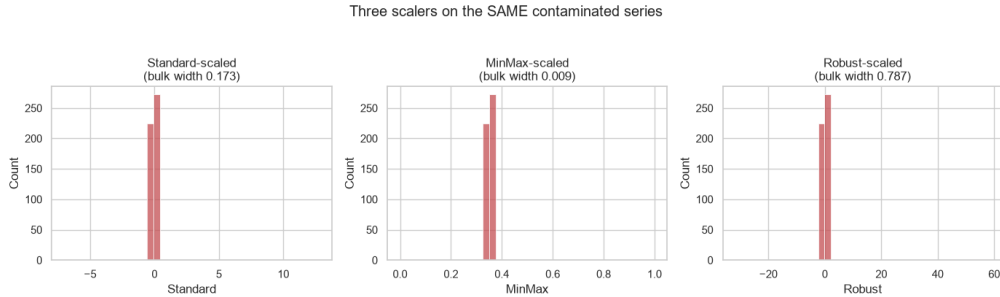


Figure 85: 4. RobustScaler, statistics for messy rooms (ch06_preprocessing_02 figure 2)

Common pitfall, Normalizer is not a scaler!

Normalizer rescales each **ROW** to unit length (L2 norm = 1): it makes [3, 4] into [0.6, 0.8]. Column scalers compare *features*; Normalizer compares *samples* by direction only. Its home turf is text vectors and similar direction oriented data. Importing Normalizer when you meant StandardScaler is a classic silent bug.

```
X_rows = np.array([[3.0, 4.0],      # a 3-4-5 triangle
                  [1.0, 2.0],
                  [6.0, 2.0]])
normed = Normalizer().fit_transform(X_rows) # fit is a no-op here; transform scales rows
print("unit norms per ROW:", np.linalg.norm(normed, axis=1).round(6))
print("columns were untouched as columns - each row just shrank onto the unit circle")
```

Output:

```
unit norms per ROW: [1. 1. 1.]
columns were untouched as columns - each row just shrank onto the unit circle
```

5. Skew fixes: bending the curve (scaling can't)

WHAT: right skewed variables (salaries, fares, durations) have a long thin tail of large values. Many models prefer symmetric ish inputs; log style transforms bend that tail back toward the bulk.

WHY detect first: skew() gives a number (|skew| > 1 = strong), histograms give the story. Then choose the tool:

Tool	Negatives allowed?	Character
np.log1p (= log(1+x))	no (NaN below 0)	instant, interpretable, invert with expm1

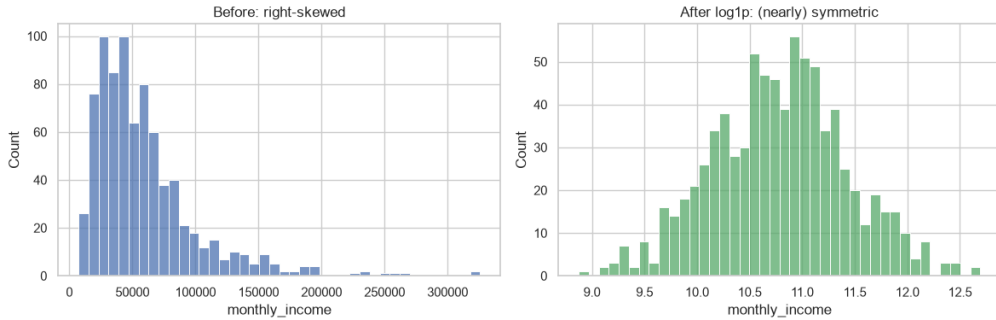


Figure 86: 5. Skew fixes: bending the curve (scaling can't) (ch06_preprocessing_02 figure 3)

Tool	Negatives allowed?	Character
Box Cox	strictly positive only	classic power family, λ fitted from data
Yeo Johnson (PowerTransformer)	YES	generalizes Box Cox; learns λ via fit/transform

```

salary = income.copy()                # reuse our skewed series
print(f"salary skew before : {salary.skew():.2f}")

log_salary = np.log1p(salary)
print(f"after np.log1p      : {log_salary.skew():.2f}")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(salary, bins=40, ax=axes[0], color="#4c72b0")
axes[0].set_title("Before: right-skewed")
sns.histplot(log_salary, bins=40, ax=axes[1], color="#55a868")
axes[1].set_title("After log1p: (nearly) symmetric")
plt.tight_layout()
plt.show()

```

Output:

```

salary skew before : 2.14
after np.log1p      : 0.01

```

Yeo Johnson handles what log can't touch: negatives

Temperature anomalies dip below zero, so $\log_{1p}$ would produce NaNs. Watch both tools on a signed series:

```

temps = pd.Series(rng.normal(loc=5.0, scale=12.0, size=600), name="temp_anomaly")
negatives = int((temps < 0).sum())

with np.errstate(invalid="ignore"):          # silence the expected NaN warning
    log_attempt = np.log1p(temps)
log_nan_count = int(np.isnan(log_attempt).sum())

pt = PowerTransformer(method="yeo-johnson") # fit/transform: lambda learned on train
yj_result = pt.fit_transform(temps.to_numpy().reshape(-1, 1)).ravel()

skew_table = pd.DataFrame(
    {
        "skew": [temps.skew(), np.nan, pd.Series(yj_result).skew()],
        "NaNs created": [0, log_nan_count, 0],
    },
    index=["raw", "log1p", "yeo-johnson"],
)
print(f"series has {negatives} negative values -> log1p poisons {log_nan_count} of them"
      )
print(f"PowerTransformer learned lambda = {pt.lambdas_[0]:.3f}")
skew_table.round(3)

```

Output:

```

series has 216 negative values -> log1p poisons 202 of them
PowerTransformer learned lambda = 1.014

```

Output:

skew	NaNs created	
raw	-0.04	0
log1p	NaN	202
yeo-johnson	0.04	0

Rule of thumb: log1p for quick exploration of positive money like data; PowerTransformer when negatives exist or when λ should be *learned* inside a pipeline rather than hard coded.

Common pitfall: Box Cox raises on zero/negative inputs. If you must use it, shift the data first, or just reach for Yeo Johnson and skip the ceremony.

6. Binning: cut vs qcut

WHAT: discretize a continuous variable into buckets.

- `pd.cut(s, bins=[edges])` **equal WIDTH** bins; you choose human readable edges.
- `pd.qcut(s, q=4)` **equal FREQUENCY** bins; quantile edges guarantee balanced counts.

WHY bother: linear/logistic models gain cheap non linearity, each bin gets its own coefficient/step. Trees don't need this (and don't need scaling either: they split per feature thresholds), but linear models do benefit. Cost: within bin detail is discarded.

```
ages = pd.Series(rng.normal(loc=40, scale=13, size=1200)).clip(15, 85).round(0)

age_bands = pd.cut(
    ages,
    bins=[15, 25, 40, 55, 85],
    labels=["young", "adult", "middle", "senior"], # labels arg -> readable output
)
print("pd.cut equal-width bands:")
print(age_bands.value_counts().sort_index())

fare_quartiles = pd.qcut(income, q=4, labels=["Q1_lowest", "Q2", "Q3", "Q4_highest"])
print("\npd.qcut equal-frequency quartiles:")
print(fare_quartiles.value_counts().sort_index())
```

Output:

```
pd.cut equal-width bands:
young      109
adult      464
middle     446
senior     142
Name: count, dtype: int64

pd.qcut equal-frequency quartiles:
monthly_income
Q1_lowest      200
Q2              200
Q3              200
Q4_highest     200
Name: count, dtype: int64
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(age_bands.value_counts().sort_index().index.astype(str),
            age_bands.value_counts().sort_index().values, color="#4c72b0")
axes[0].set_title("cut: equal WIDTH, unequal counts")
axes[1].bar(fare_quartiles.value_counts().sort_index().index.astype(str),
            fare_quartiles.value_counts().sort_index().values, color="#55a868")
axes[1].set_title("qcut: equal FREQUENCY (~300 each)")
plt.tight_layout()
plt.show()
```

On skewed data, equal width bins starve the left side and gorge the right tail, qcut is usually the fairer choice there. Remember that quantile edges are *learned* boundaries: fit them on train, apply the same edges to test (another fit/transform citizen).

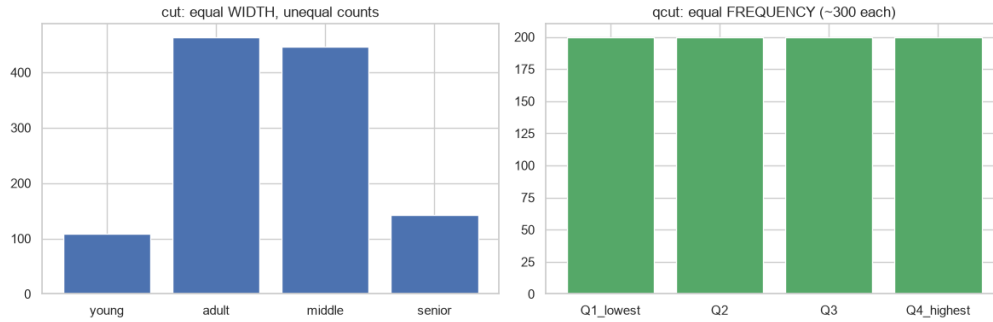


Figure 87: 6. Binning: cut vs qcut (ch06_preprocessing_02 figure 4)

Pro tip: qcut fails with "Bin edges must be unique" on coarse/discrete data (duplicated quantiles). Add `duplicates="drop"` or round the series first.

7. Outliers: detect, diagnose, treat

DETECT with two complementary lenses:

- **z score:** $|z| > 3$ flags points $\geq \sim 3\sigma$ from the mean. Simple, but assumes rough normality, and the outliers themselves inflate σ , hiding themselves (masking).
- **IQR fences:** flag outside $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$, exactly the whiskers of a boxplot. No normality assumption, robust to masking.

DIAGNOSE before treating, the cause decides the cure: data entry error fix/remove; genuine heavy tail transform or cap; genuine extremes that ARE the phenomenon (fraud!) leave alone, model robustly.

```
readings = pd.Series(rng.normal(loc=100, scale=12, size=300), name="reading")
outlier_positions = [17, 89, 150, 211, 263]
readings.iloc[outlier_positions] = [165.0, 172.0, -35.0, 180.0, 158.0]

z_scores = (readings - readings.mean()) / readings.std()
flagged_z = readings[np.abs(z_scores) > 3]

q1, q3 = readings.quantile([0.25, 0.75])
iqr = q3 - q1
lo_fence, hi_fence = q1 - 1.5 * iqr, q3 + 1.5 * iqr
flagged_iqr = readings[(readings < lo_fence) | (readings > hi_fence)]

print(f"z-score flags    : {len(flagged_z)} points {list(flagged_z.round(0))}")
print(f"IQR fences      : [{lo_fence:.1f}, {hi_fence:.1f}]")
print(f"IQR flags        : {len(flagged_iqr)} points")
print("(IQR sees more because outliers inflate the mean/std that z-score relies on)")

fig, ax = plt.subplots(figsize=(8, 3))
```

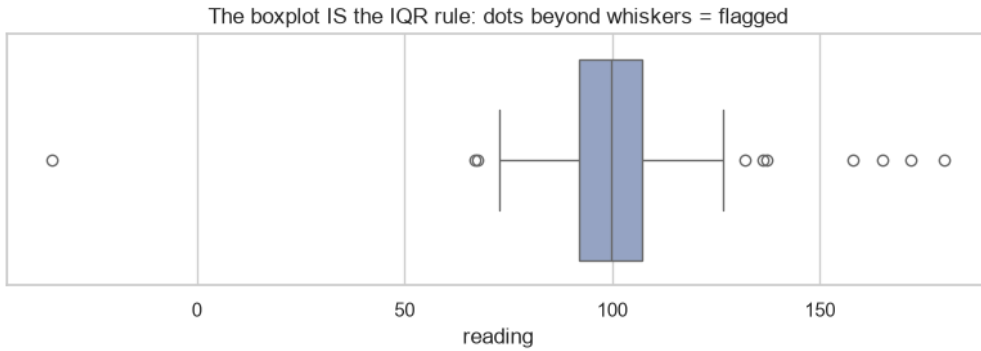


Figure 88: 7. Outliers: detect, diagnose, treat (ch06_preprocessing_02 figure 5)

```
sns.boxplot(x=readings, ax=ax, color="#8da0cb")
ax.set_title("The boxplot IS the IQR rule: dots beyond whiskers = flagged")
plt.tight_layout()
plt.show()
```

Output:

```
z-score flags : 5 points [165.0, 172.0, -35.0, 180.0, 158.0]
IQR fences   : [69.1, 130.2]
IQR flags    : 10 points
(IQR sees more because outliers inflate the mean/std that z-score relies on)
```

Treatment showdown on the same series

Four policies, one scoreboard:

```
treated = {
    "original (leave)": readings.copy(),
    "remove": readings[(readings >= lo_fence) & (readings <= hi_fence)],
    "cap / winsorize": readings.clip(lo_fence, hi_fence),
    "transform": np.log1p(readings - lo_fence + 1), # shift so min >= 0, then log
}

rows = []
for name, s in treated.items():
    rows.append({
        "policy": name,
        "n": len(s),
        "mean": s.mean(),
        "std": s.std(),
        "skew": s.skew(),
```

```

        "extreme_max": s.max(),
    })
pd.DataFrame(rows).set_index("policy").round(2)

```

Output:

```

Warning: invalid value encountered in log1p
result = getattr(ufunc, method)(*inputs, **kwargs)

```

Output:

	n	mean	std	skew	extreme_max
policy					
original (leave)	300	100.02	15.75	-0.96	180.00
remove	290	99.40	10.33	-0.04	126.81
cap / winsorize	300	99.82	11.59	0.12	130.15
transform	300	3.42	0.47	-2.74	4.73

Read the scoreboard: **leave** preserves truth including tails; **remove** shrinks n and understates variability; **cap** keeps every row but pins tails at the fences (winsorizing, great default for dirty business data); **transform** tames skew AND extremes smoothly but changes interpretation (units become log ish).

8. Putting it together: a mini numeric prep function

What: bundle the chapter's decisions into one reusable transformer style function: de skew if badly skewed, then robust scale. Apply it to Titanic fares.

Why these choices: fares are money skew almost guaranteed median/IQR family throughout. In production you would express this as a sklearn pipeline (notebook 06); the manual version shows the seams.

```

def make_synthetic_fares(n=800):
    r = np.random.default_rng(7)
    pclass = r.choice([1, 2, 3], size=n, p=[0.25, 0.22, 0.53])
    mu = np.where(pclass == 1, np.log(85.0), np.where(pclass == 2, np.log(22.0), np.log(
        9.5)))
    return pd.DataFrame({"pclass": pclass, "fare": r.lognormal(mu, 0.6).round(2)})

try:
    titanic_fare = sns.load_dataset("titanic")[["pclass", "fare"]].dropna().reset_index(
        drop=True)
    source = "seaborn titanic"
except Exception:
    titanic_fare = make_synthetic_fares()
    source = "synthetic titanic-like"

```

```

SKEW_THRESHOLD = 1.0

def prep_numeric(s, skew_threshold=SKEW_THRESHOLD):
    """De-skew (if needed) then robust-scale. Returns a Series."""
    s = s.astype(float)
    if abs(s.skew()) > skew_threshold:
        s = np.log1p(s) # step 1: bend the tail
        scaled = RobustScaler().fit_transform(s.to_numpy().reshape(-1, 1)).ravel()
        return pd.Series(scaled, index=s.index, name=f"{s.name}_prepped")

raw_fare = titanic_fare["fare"]
prepped_fare = prep_numeric(raw_fare)

report = pd.DataFrame(
    {"raw fare": [raw_fare.mean(), raw_fare.std(), raw_fare.skew()],
     "prepped fare": [prepped_fare.mean(), prepped_fare.std(), prepped_fare.skew()]},
    index=["mean", "std", "skew"],
).round(3)
print(f"data source: {source}")
report

```

Output:

```
data source: seaborn titanic
```

Output:

raw fare	prepped fare
mean	32.204 0.175
std	49.693 0.758
skew	4.787 0.395

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.kdeplot(raw_fare, fill=True, ax=axes[0], color="#4c72b0")
axes[0].set_title("Raw fare: skewed, wide, outlier-sensitive")
sns.kdeplot(prepped_fare, fill=True, ax=axes[1], color="#55a868")
axes[1].set_title("Prepped: log1p + RobustScaler -> model-ready")
plt.tight_layout()
plt.show()

```

Common pitfall: In a real project, `prep_numeric`'s thresholds, medians and IQRs must be learned on TRAIN only. The function above fits on whatever it is handed, fine for a demo, leakage in production. Notebook 06 wraps it safely.

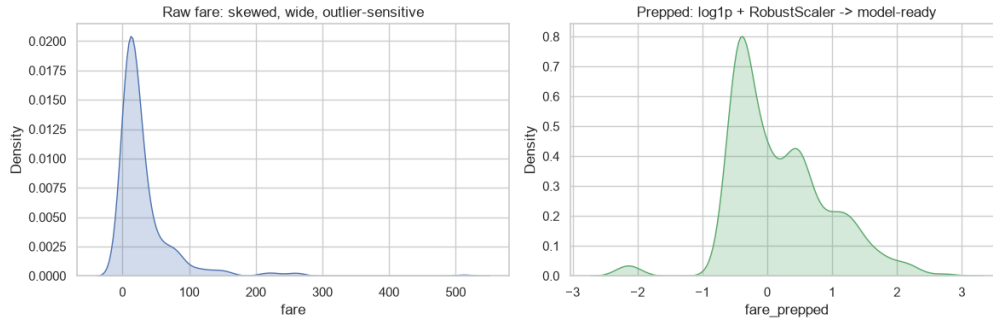


Figure 89: 8. Putting it together: a mini numeric prep function (ch06_preprocessing_02 figure 6)

Summary & key takeaways

- Scale for the algorithm's sake: distances (KNN), optimization (gradient descent), fairness (regularization). Trees don't care.
- **StandardScaler** = default; **MinMax** = bounded inputs but hostage to `max()`; **Robust** = median/IQR, wins on contaminated data.
- Scaling shifts and stretches, it NEVER reshapes. De skew first (`log1p` / `PowerTransformer`), scale second.
- Normalizer works on rows, not columns, different beast entirely.
- `pd.cut` for readable equal width bands; `pd.qcut` for balanced bins on skew.
- Outliers: detect with BOTH z score and IQR fences, diagnose the cause, then leave / remove / cap / transform accordingly.
- Ship ready recipe: if `skew > 1`: `log1p RobustScaler`, fitted on train.

Next up: `03_categorical_data.nb.py`, turning words into honest numbers.

7.3 Data Preprocessing, Encoding Categorical Data

MLCourse · Data Science Foundations · 05_data_preprocessing

Models eat numbers, not words. Encoding converts categories to numbers WITHOUT inventing fake mathematics. Choosing wrongly silently corrupts your model.

What you'll learn

- Nominal vs ordinal, the decision that drives everything
- Label encoding and its hidden dangers on nominal data
- Ordinal encoding done right (explicit order)
- One hot encoding, the dummy variable trap, and sklearn's production safe version
- Frequency encoding for high cardinality columns
- Target encoding: powerful but leakage prone (concept + warning)

- Grouping rare categories into "Other"
- Datetime feature extraction + cyclical sin/cos encoding
- Assembling everything with ColumnTransformer

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

pd.options.display.width = 120
```

1. Nominal vs ordinal, decide FIRST

Type	Definition	Examples	Correct encodings
Ordinal	meaningful ORDER	S<M<L, low<med<high	ordinal/integer mapping
Nominal	NO order	cities, colors, blood type	one hot, frequency, target

```
df = pd.DataFrame({
    "city":    ["Berlin", "Oslo", "Tokyo", "Oslo", "Berlin"],    # nominal
    "size":    ["M", "S", "L", "M", "S"],                      # ordinal!
    "plan":    ["basic", "pro", "premium", "basic", "pro"],      # ordinal
})
df
```

Output:

```
city size    plan
0  Berlin    M    basic
1   Oslo     S     pro
2  Tokyo     L  premium
3   Oslo     M    basic
4  Berlin    S     pro
```

2. Label encoding, fast but treacherous

LabelEncoder maps categories to 0..k-1. For ORDINAL data that's fine. For NOMINAL data you just told the model Oslo(2) > Berlin(0) and Tokyo is "the average of nothing". Linear models will multiply these numbers by weights!

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["city_label"] = le.fit_transform(df["city"])
print(le.classes_, "-> codes:", df["city_label"].tolist())

## Demonstrate the damage on a tiny linear fit:
rng = np.random.default_rng(42)
y_true = np.array([100., 100., 100., 100., 100.]) # city truly has NO effect...
coef = np.polyfit(df["city_label"], y_true, 1)      # ...but the model still sees slope
print(f"fitted slope on meaningless codes: {coef[0]:+.4f} (should be ~0)")

```

Output:

```

['Berlin' 'Oslo' 'Tokyo'] -> codes: [0, 1, 2, 1, 0]
fitted slope on meaningless codes: -0.0000 (should be ~0)

```

Common pitfall: label encoding a nominal column for a LINEAR model or neural net injects fake order. Tree models are (mostly) immune; linear models are not.

3. Ordinal encoding done right, YOU define the order

Explicit mapping dict = full control + self documenting code.

```

size_map = {"S": 0, "M": 1, "L": 2}
plan_map = {"basic": 0, "pro": 1, "premium": 2}

df["size_ord"] = df["size"].map(size_map)          # .map leaves NaN for unknowns!
df["plan_ord"] = df["plan"].map(plan_map)
df[["size", "size_ord", "plan", "plan_ord"]]

## The sklearn way - note how WE supply the category order:
ord_enc = OrdinalEncoder(categories=[["S", "M", "L"]])
df["size_sklearn"] = ord_enc.fit_transform(df[["size"]]).astype(int)

## Unknown category at transform-time -> OrdinalEncoder REFUSES by default.
try:
    ord_enc.transform(pd.DataFrame({"size": ["XXL"]}))
except ValueError as err:
    print("ValueError raised:", err)
    print("-> production pipelines use handle_unknown='ignore' or explicit maps")

```

Pro tip: prefer `.map(dict)` in notebooks (visible, greppable logic) and `OrdinalEncoder` inside sklearn Pipelines (fits the train only workflow).

4. One hot encoding, the nominal workhorse

One column per category, value 0/1. No fake order. Cost: k new columns.

```
onehot = pd.get_dummies(df["city"], prefix="city")
pd.concat([df["city"], onehot], axis=1)
```

Output:

	city	city_Berlin	city_Oslo	city_Tokyo
0	Berlin	True	False	False
1	Oslo	False	True	False
2	Tokyo	False	False	True
3	Oslo	False	True	False
4	Berlin	True	False	False

4.1 The dummy variable trap

With all k dummies present, $\text{city_berlin} + \text{city_oslo} + \text{city_tokyo} == 1$ always, perfect multicollinearity. Linear algebra hates this. Drop ONE column; the dropped category becomes the baseline (all zeros row).

```
pd.get_dummies(df["city"], drop_first=True) # k-1 columns, Berlin = baseline
```

Output:

	Oslo	Tokyo
0	False	False
1	True	False
2	False	True
3	True	False
4	False	False

Trees don't care about collinearity; keep drop_first=False there if you like.

4.2 sklearn's OneHotEncoder, built for PRODUCTION

```
ohe = OneHotEncoder(drop="first", handle_unknown="ignore",
                    sparse_output=False) # dense array for readability

train_cities = df[["city"]] # sklearn wants 2-D input
ohe.fit(train_cities)

print("learned categories:", ohe.categories_)
```

```
## New city appears at serving time - get_dummies would CRASH or misalign columns:
new_data = pd.DataFrame({"city": ["Tokyo", "Paris"]}) # Paris never seen!
print("transform survives unknown 'Paris':\n",
      pd.DataFrame(ohe.transform(new_data),
                   columns=ohe.get_feature_names_out()))
```

Output:

```
learned categories: [array(['Berlin', 'Oslo', 'Tokyo'], dtype=object)]
transform survives unknown 'Paris':
   city_Oslo  city_Tokyo
0         0.0         1.0
1         0.0         0.0
```

Output:

```
62: UserWarning: Found unknown categories in columns [0] during transform. These unknown categories will be encoded as all zeros
```

Common pitfall: `get_dummies` at training time + different columns at serving time = silent model garbage. Use `OneHotEncoder(handle_unknown='ignore')` fitted ONCE on train, saved with your pipeline.

5. High cardinality columns & frequency encoding

500 cities 500 dummy columns: sparse, slow, overfit prone. Alternative: replace each category by its frequency (or count).

```
n_rows = 10_000
cities = np.random.choice([f"city_{i:03d}" for i in range(300)],
                          n_rows,
                          p=np.array([0.30] + [0.7/299]*299)) # one dominant city

high_card = pd.DataFrame({"city": cities})

## TRAIN frequencies only (leakage discipline!):
freq = high_card["city"].value_counts(normalize=True)
high_card["city_freq"] = high_card["city"].map(freq).round(4)

high_card.head()
```

Output:

```
city  city_freq
0  city_032    0.0023
1  city_278    0.0032
2  city_185    0.0028
```

```
3 city_128      0.0024
4 city_000      0.3053
```

5.1 Rare category grouping "Other"

Categories below a support threshold carry noise, not signal.

```
counts = high_card["city"].value_counts()
rare = counts[counts < 50].index          # seen <50 times
print(f"{len(rare)} of {len(counts)} categories are rare "
      f"({counts[rare].sum()/n_rows:.1%} of rows)")

high_card["city_grouped"] = np.where(high_card["city"].isin(rare),
                                     "Other", high_card["city"])
print(high_card["city_grouped"].value_counts().head(3))

## Now one-hot on city_grouped stays small AND stable.
```

Output:

```
299 of 300 categories are rare (69.5% of rows)
city_grouped
Other      6947
city_000    3053
Name: count, dtype: int64
```

6. Target encoding, concept + BIG warning

Replace category with mean of target for that category. Extremely powerful for high cardinality tabular problems, and the #1 source of leaderboard fraud via LEAKAGE when computed naively on all data.

```
toy = pd.DataFrame({
    "store": ["A", "B", "A", "C", "B", "C"],
    "sold":  [10, 4, 12, 7, 6, 9],
})

naive = toy.groupby("store")["sold"].mean()          # computed WITH the target rows
toy["store_te_naive"] = toy["store"].map(naive)
print(toy)
print("\n In real projects compute it INSIDE cross-validation folds only,")
print(" or use library schemes (category_encoders.TargetEncoder / CV folds).")
```

Output:

```
store  sold  store_te_naive
0     A    10           11.0
1     B     4           5.0
2     A    12           11.0
3     C     7           8.0
4     B     6           5.0
5     C     9           8.0
```

In real projects compute it INSIDE cross-validation folds only,
or use library schemes (category_encoders.TargetEncoder / CV folds).

Common pitfall: naive target encoding memorizes each row's own target spectacular offline scores, disastrous deployment. If you can't explain your CV scheme, don't target encode.

7. Dates are categorical goldmines

A single timestamp hides many features: year, month, weekday, hour, weekend...

```
ts = pd.DataFrame({
    "when": pd.to_datetime([
        "2024-01-05 08:15", "2024-06-23 14:40", "2024-12-31 23:05",
        "2024-07-04 11:00", "2024-02-10 19:30",
    ])
})

ts["year"] = ts["when"].dt.year
ts["month"] = ts["when"].dt.month
ts["dow"] = ts["when"].dt.dayofweek          # Mon=0
ts["hour"] = ts["when"].dt.hour
ts["is_weekend"] = ts["dow"] >= 5
ts["month_name"] = ts["when"].dt.month_name()
ts
```

Output:

```
when  year  month  dow  hour  is_weekend  month_name
0 2024-01-05 08:15:00 2024      1      4      8      False   January
1 2024-06-23 14:40:00 2024      6      6     14       True     June
2 2024-12-31 23:05:00 2024     12      1     23      False  December
3 2024-07-04 11:00:00 2024      7      3     11      False     July
4 2024-02-10 19:30:00 2024      2      5     19       True   February
```

7.1 Cyclical encoding, because hour 23 and hour 0 are NEIGHBORS

Plain integers tell a model $23 > 0$ and $\text{distance}(23, 0) = 23$. Sin/cos put them on a circle where they touch.

```
hours = np.arange(24)
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(np.sin(2*np.pi*hours/24), np.cos(2*np.pi*hours/24))
for h in [0, 6, 12, 18, 23]:
    x_, y_ = np.sin(2*np.pi*h/24), np.cos(2*np.pi*h/24)
    ax.annotate(h, (x_, y_), xytext=(6, 4), textcoords="offset points")
ax.set_title("Hours on a circle: 23 touches 0")
ax.set_xlabel("sin(2πh/24)"); ax.set_ylabel("cos(2πh/24)")
ax.grid(alpha=.3); plt.show()

ts["hr_sin"] = np.sin(2*np.pi*ts["hour"]/24).round(3)
ts["hr_cos"] = np.cos(2*np.pi*ts["hour"]/24).round(3)
ts[["hour", "hr_sin", "hr_cos"]]

## Same trick for month/day-of-week whenever cycles matter (demand, traffic...).
```

Output:

hour	hr_sin	hr_cos
0	8	0.866 -0.500
1	14	-0.500 -0.866
2	23	-0.259 0.966
3	11	0.259 -0.966
4	19	-0.966 0.259

8. Assembling everything, ColumnTransformer

Different columns need different treatments. ColumnTransformer routes each column group to its transformer and glues outputs side by side.

```
raw = pd.DataFrame({
    "age": [25, 32, np.nan, 47, 22],
    "fare": [70.5, 8.0, 15.9, np.nan, 7.2],
    "sex": ["male", "female", "female", "male", "female"],
    "embarked": ["S", "C", "S", np.nan, "Q"],
})

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

numeric_pipe = Pipeline([
```

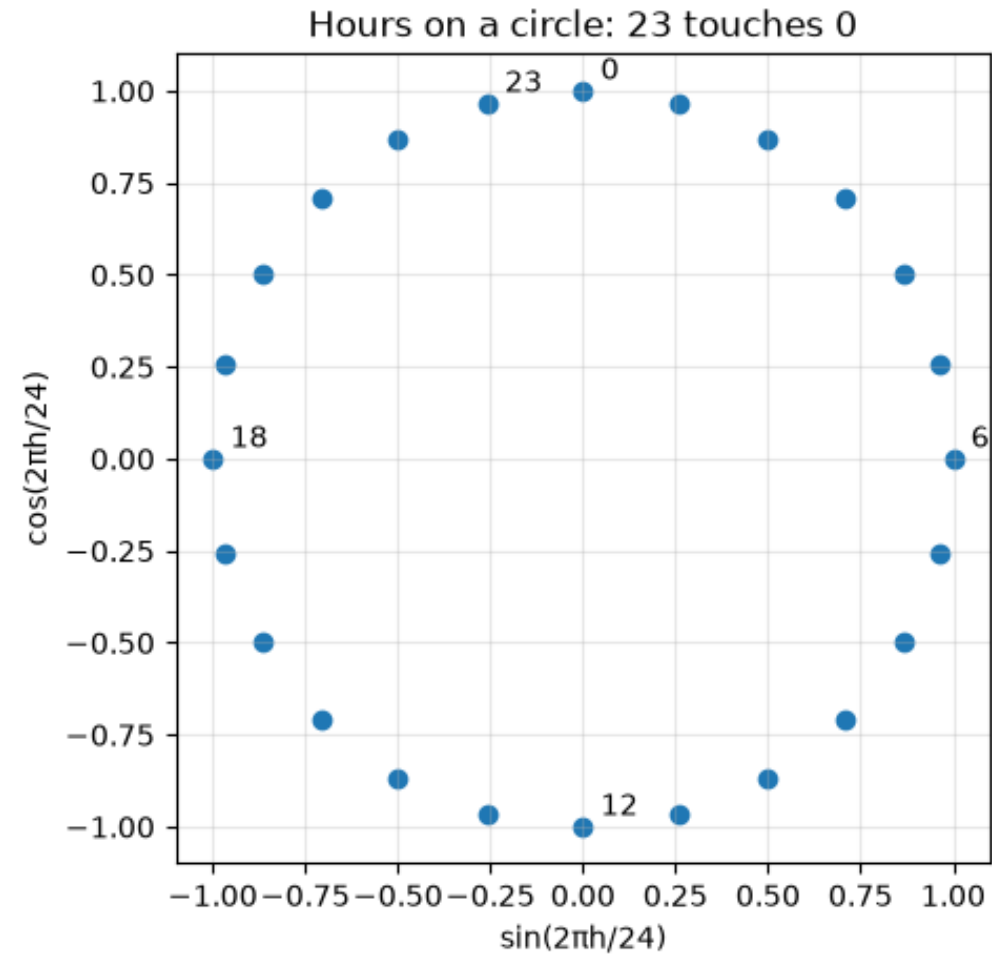


Figure 90: 7.1 Cyclical encoding, because hour 23 and hour 0 are NEIGHBORS (ch06_preprocessing_03 figure 1)


```

    ("impute", SimpleImputer(strategy="median")), # fill NaNs first
    ("scale", StandardScaler()),                # then scale
])

cat_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")), # mode-fill embarked
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, ["age", "fare"]),
    ("cat", cat_pipe, ["sex", "embarked"]),
], remainder="drop")

X_ready = preprocessor.fit_transform(raw) # numpy array, model-ready!
print(X_ready.round(2))

feature_names = preprocessor.get_feature_names_out()
print("\ncolumns:", list(feature_names))

## Everything reproducible in ONE object you can save/pickle with your model.

```

Output:

```

[[-0.68  1.98  0.    1.    0.    0.    1. ]
 [ 0.13 -0.61  1.    0.    1.    0.    0. ]
 [-0.28 -0.28  1.    0.    0.    0.    1. ]
 [ 1.85 -0.45  0.    1.    0.    0.    1. ]
 [-1.02 -0.64  1.    0.    0.    1.    0. ]]

columns: ['num__age', 'num__fare', 'cat__sex_female', 'cat__sex_male', 'cat__embarked_C'
, 'cat__embarked_Q', 'cat__embarked_S']

```

Decision flowchart (summary table)

Situation | Reach for |

|,|,| | ordered grades/sizes/plans | explicit .map dict or OrdinalEncoder(categories=...)
 | | nominal, few categories, linear model | one hot (drop_first=True) | | nominal,
 few categories, trees | one hot (drop_first optional) | | nominal, hundreds of levels
 | rare grouping frequency/target encoding | | unseen categories expected in prod |
 OneHotEncoder(handle_unknown="ignore") | | timestamps | extract features + cyclical sin/cos |

Summary & key takeaways

- **Nominal vs ordinal decides everything**, never label encode nominals for linear models.
- One hot kills fake order; watch the **dummy trap** (k-1 for linear models).

- Production $\neq$ notebook: `OneHotEncoder(handle_unknown="ignore")` fitted once beats fragile `get_dummies`.
- High cardinality: group rares frequency encode (train frequencies ONLY).
- Target encoding is leakage lightning, needs CV safe computation.
- Dates explode into year/month/dow/hour/weekend features; cyclic ones deserve sin/cos circles.
- `ColumnTransformer` + `Pipeline` = one reproducible object from raw frame to model ready matrix.

7.4 Text Data Stdlib

Data Preprocessing 04, Text Data, Pure Python Pipeline

MLCourse · Data Science Foundations · 05_data_preprocessing

Welcome back! So far every dataset we cleaned was a tidy grid of numbers and categories. Text is different: it is messy *by design*, because humans, not databases, write it. Reviews shout in CAPS, carry leftover `<br />` HTML tags, hide URLs, and say “don’t” in forty different ways. Before any machine learning model can read text, someone has to tame it into numbers.

In this notebook we build that entire taming pipeline **from scratch, using nothing but the Python standard library** (`re`, `string`, `collections`) plus `pandas` for display. Why from scratch? Because when you hand roll lowercase/strip/tokenize yourself, nothing stays magic: you see exactly what industry toolkits later automate for you. The next notebook (05) rebuilds the same pipeline with `NLTK` so you can compare.

7.5 What you'll learn

- The core vocabulary of text mining: **corpus, document, token, vocabulary**
- A reusable cleaning pipeline: HTML removal, URL removal, contraction expansion, punctuation handling (two ways), whitespace collapsing, optional digit removal
- Why naive `.split()` tokenization fails and how regex tokenizers differ
- Building and applying a stopword set, including domain specific ones
- A naive suffix stripping stemmer written by hand, and why it fails
- What lemmatization is and how it differs from stemming
- Bag of Words vectors built with `Counter`, plus a sparsity reality check
- N grams (bigrams) from scratch, and how they rescue negations like “not good”
- TF IDF computed with plain loops, verified against `scikit learn`

```
import math
import re
import string
from collections import Counter

import numpy as np
```

```
import pandas as pd

print("Ready. Python is all we need for now.")
```

Output:

```
Ready. Python is all we need for now.
```

1. The vocabulary of text mining

Four terms come up in every NLP conversation, so let's pin them down before writing code:

Term	Meaning	Example
Corpus	Your whole collection of texts	All 10,000 product reviews
Document	One item inside the corpus	A single review
Token	One meaningful unit (usually a word) after splitting	"battery", "great"
Vocabulary	The set of <i>unique</i> tokens across the corpus	{great, battery, slow, ...}

Keep these straight and every later concept clicks into place: we clean documents, tokenize them into token sequences, collect the corpus level vocabulary, and finally turn each document into a vector indexed by that vocabulary.

```
## A tiny demonstration of the four terms on three one-line "documents".
mini_corpus = [
    "Battery life is great.",
    "The screen is great but the battery is weak.",
    "Weak speaker, great price.",
]

tokens_per_doc = [doc.lower().split() for doc in mini_corpus]
vocabulary = sorted({token for tokens in tokens_per_doc for token in tokens})

print(f"Documents in corpus : {len(mini_corpus)}")
print(f"Tokens per doc      : {[len(t) for t in tokens_per_doc]}")
print(f"Vocabulary size     : {len(vocabulary)}")
print(f"Vocabulary          : {vocabulary}")
```

Output:

```
Documents in corpus : 3
Tokens per doc      : [4, 9, 4]
Vocabulary size     : 12
```

```
Vocabulary      : ['battery', 'but', 'great', 'great.', 'is', 'life', 'price.', 'screen', 'speaker,', 'the', 'weak', 'weak.']
```

2. A deliberately messy review corpus

Real text never arrives clean. To practice honestly, we build a **tiny but realistic** corpus of ten short app/service reviews stuffed with classic mess:

- leftover HTML tags (
)
- pasted URLs
- SHOUTING IN CAPS
- punctuation storms (!!!, ...)
- contractions (don't, can't, it's, wouldn't)
- numbers and prices (\$9.99, 45 minutes)
- stray double spaces

Ten documents is small enough to eyeball every intermediate result, exactly what you should do with any new text dataset before scaling up.

```
RAW_REVIEWS = [
    "This app is GREAT!!! Works perfectly, love the new design. <br /> Five stars!",
    "Terrible experience... it crashed TWICE on launch. Don't waste your money.",
    "Not good at all. Battery drains fast and support ignored my 3 tickets.",
    "Honestly? Pretty decent for $9.99/month. Can't complain much.",
    "I was on hold for 45 minutes!!! Worst support ever. See http://complaints.example.com",
    "AMAZING update!!! Everything loads instantly now. Well done team :)",
    "Meh. Does the job but the UI feels dated and it's slow on older phones.",
    "DO NOT subscribe. They charged me twice and wouldn't refund. Scam alert!!",
    "Good value, easy setup. Took 10 minutes and everything worked. Recommended.",
    "It's okay... sometimes great, sometimes awful. Very inconsistent wifi calling.",
]

print(f"Reviews in corpus: {len(RAW_REVIEWS)}")
print()
for i, review in enumerate(RAW_REVIEWS[:3], start=1):
    print(f"[{i}] {review!r}")
```

Output:

```
Reviews in corpus: 10

[1] 'This app is GREAT!!! Works perfectly, love the new design. <br /> Five stars!'
[2] "Terrible experience... it crashed TWICE on launch. Don't waste your money."
[3] 'Not good at all. Battery drains fast and support ignored my 3 tickets.'
```

3. The cleaning pipeline, one brick at a time

Cleaning converts raw strings into normalized text that downstream steps can trust. We will build it as **small, single purpose functions** and then compose them in order:

lowercase strip HTML remove URLs expand contractions (optional) drop digits
remove punctuation collapse whitespace

Order matters! For example, we must expand contractions **before** removing punctuation, because the apostrophe in don't is punctuation, strip it first and you get dont, which no contraction dictionary can recognize anymore.

3.1 Lowercase everything

Why: to a computer, "GREAT", "Great", and "great" are three different tokens. Models count words; unless case carries real meaning (named entities, code), collapsing case triples your apparent vocabulary for zero benefit. Lowercasing first also lets our contraction dictionary use lowercase keys only.

```
def lowercase(text):
    """Collapse all casing to lower-case."""
    return text.lower()

demo = "This App Is GREAT!!!"
print(lowercase(demo))
```

Output:

```
this app is great!!!
```

3.2 Strip HTML tags and URLs

Why: reviews scraped from web pages often smuggle markup like `<br />` along, and users paste links. Neither helps a sentiment model, they are noise tokens waiting to pollute our vocabulary. The regex `<[>]+>` reads as: an opening `<`, then *anything that is not a closing bracket*, then `>`. URL patterns cover `http://...` and `www.....`

```
def strip_html(text):
    """Replace anything that looks like an HTML tag with a space."""
    return re.sub(r"<[>]+>", " ", text)

def remove_urls(text):
    """Replace http(s)://... or www... links with a space."""
    return re.sub(r"https?:\/\/\S+|www\.\S+", " ", text)
```

```
demo = "Love it <br /> details at https://example.com/deal?ref=x"
print(strip_html(demo))
print(remove_urls(strip_html(demo)))
```

Output:

```
Love it   details at https://example.com/deal?ref=x
Love it   details at
```

3.3 Expand contractions

Why: don't and do not mean the same thing but look completely different to a tokenizer. Expanding contractions merges those counts. We keep the mapping small and honest, a real project would ship a few dozen more entries.

Common pitfall: expanding contractions *after* punctuation removal silently does nothing, because the apostrophe is already gone (dont no longer matches). Expansion must run while apostrophes still exist.

```
CONTRACTIONS = {
    "don't": "do not",
    "can't": "can not",
    "won't": "will not",
    "isn't": "is not",
    "aren't": "are not",
    "wasn't": "was not",
    "didn't": "did not",
    "doesn't": "does not",
    "wouldn't": "would not",
    "couldn't": "could not",
    "shouldn't": "should not",
    "hasn't": "has not",
    "it's": "it is",
    "that's": "that is",
    "there's": "there is",
    "i'm": "i am",
    "i've": "i have",
    "you're": "you are",
}

def expand_contractions(text):
    """Replace contraction keys with their expanded forms (word-boundary safe)."""
    for contraction, full_form in CONTRACTIONS.items():
        pattern = r"\b" + re.escape(contraction) + r"\b"
        text = re.sub(pattern, full_form, text)
```

```

    return text

print(expand_contractions("don't worry, it's fine, i'm sure"))

```

Output:

```
do not worry, it is fine, i am sure
```

3.4 Remove punctuation, two ways compared

There are two idiomatic ways to strip punctuation, and they behave differently:

1. **Regex substitution** `re.sub(r"^\w\s]", " ", text)`, replaces each punctuation character with a **space**, so `well-known` becomes `well known` (two words) and `$1,299` becomes `1 299`.
2. **Translation table** `text.translate(str.maketrans("", "", string.punctuation))`, *deletes* characters outright, so `well-known` fuses into `wellknown`.

Deletion glues words together (usually bad); substitution keeps word boundaries but can split hyphenated compounds. We prefer substitution for general text, fused garbage tokens are worse than split ones.

```

def remove_punct_re(text):
    """Swap every non-word, non-space character for a space (keeps boundaries)."""
    return re.sub(r"^\w\s]", " ", text)

def remove_punct_translate(text):
    """Delete punctuation outright via a translation table (fuses hyphenated words)."""
    return text.translate(str.maketrans("", "", string.punctuation))

demo = "well-known brands don't cost $1,299.00!"
print("regex      :", repr(remove_punct_re(demo)))
print("translate :", repr(remove_punct_translate(demo)))

```

Output:

```

regex      : 'well known brands don t cost 1 299 00 '
translate : 'wellknown brands dont cost 129900'

```

3.5 Collapse whitespace

Why: every replacement above inserts spaces, so strings sprout double/triple spaces. `\s+` matches any run of whitespace (spaces, tabs, newlines) and squashes it to one space; a final `.strip()` trims the edges. Always collapse *last*, right before tokenizing.

```
def collapse_spaces(text):
    """Squash any whitespace run to a single space and trim the ends."""
    return re.sub(r"\s+", " ", text).strip()

print(repr(collapse_spaces("too    many        spaces\tand\nnewlines")))
```

Output:

```
'too many spaces and newlines'
```

3.6 Digits: keep or drop?

Why it is a *decision* and not a *rule*: digits are meaningful in some tasks and noise in others.

- Predicting **sentiment** ("great app!!", "terrible support") digits rarely matter; dropping them shrinks the vocabulary.
- Extracting **prices** or **durations** ("only \$9.99", "crashed 3 times") digits are literally the signal; keep them!

Our `clean_text` therefore takes a `remove_digits` flag instead of deciding for you.

3.7 Compose the full pipeline

Now we snap the bricks together in the order we argued for: lowercase HTML URLs contractions (optional digits) punctuation whitespace. Small functions + one composer means every step stays testable and swappable, the same architecture the big libraries use internally.

```
def clean_text(text, remove_digits=False):
    """Full stdlib cleaning pipeline for one document."""
    text = lowercase(text)
    text = strip_html(text)
    text = remove_urls(text)
    text = expand_contractions(text)
    if remove_digits:
        text = re.sub(r"\d+", " ", text) # digit runs become spaces, then get collapsed
    text = remove_punct_re(text)
    return collapse_spaces(text)

for raw in RAW_REVIEWS[:4]:
    print(f"RAW : {raw}")
    print(f"CLEAN: {clean_text(raw, remove_digits=True)}")
    print()
```

Output:


```

RAW : This app is GREAT!!! Works perfectly, love the new design. <br /> Five stars!
CLEAN: this app is great works perfectly love the new design five stars

RAW : Terrible experience... it crashed TWICE on launch. Don't waste your money.
CLEAN: terrible experience it crashed twice on launch do not waste your money

RAW : Not good at all. Battery drains fast and support ignored my 3 tickets.
CLEAN: not good at all battery drains fast and support ignored my tickets

RAW : Honestly? Pretty decent for $9.99/month. Can't complain much.
CLEAN: honestly pretty decent for month can not complain much

```

Pro tip: keep the raw column untouched and store cleaned text in a *new* column. Cleaning is destructive and opinionated; the day your task changes (e.g., you suddenly need prices), the original text is irreplaceable.

4. Tokenization, cutting text into units

A **tokenizer** decides where one token ends and the next begins. It sounds trivial, until it isn't. Let's watch naive approaches stumble on one nasty sentence containing a hyphenated word, a contraction, a year, and an acronym: "well-known it's 2024 U.S.A."

4.1 Failure demo: naive .split()

.split() only breaks on whitespace. Punctuation stays glued to words (U.S.A.), punctuation only fragments survive alone (.), and every variant (great!, great, great!!!) becomes a *different* vocabulary entry.

```

tricky = "well-known it's 2024 U.S.A."

naive_tokens = tricky.split()
print(naive_tokens)

```

Output:

```
['well-known', "it's", '2024', 'U.S.A.']
```

4.2 Two regex tokenizers and their trade offs

With `re.findall` we choose *what counts as a token* via a character class:

- `[a-z']+` on lowercased text, letters plus apostrophes: keeps contractions whole (it's) but drops digits entirely and shreds the acronym (u.s.a. u,s,a).
- `\b\w+\b`, letters/digits/underscore runs: keeps 2024 but splits contractions (it's it s,).

Neither wins universally. The lesson: **the tokenizer encodes assumptions about what information matters**, pick it deliberately, and note it down like any hyperparameter.

```
lowered = tricky.lower()

pattern_alpha_apos = re.findall(r"[a-z']+", lowered)
pattern_word_chars = re.findall(r"\b\w+\b", lowered)

print(f"[a-z']+ -> {pattern_alpha_apos}")
print(r"\b\w+\b ->", pattern_word_chars)
```

Output:

```
[a-z']+ -> ['well', 'known', "it's", 'u', 's', 'a']
\b\w+\b -> ['well', 'known', 'it', 's', '2024', 'u', 's', 'a']
```

Pro tip: after our §3 cleaning (punctuation already removed), plain `[a-z]+` is equivalent to splitting on whitespace, simple *and* predictable. That is what we'll use below; notebook 05 swaps in NLTK's statistically trained tokenizer.

```
TOKEN_RE = re.compile(r"[a-z]+")

def tokenize(text):
    """Extract alphabetic tokens from already-cleaned text."""
    return TOKEN_RE.findall(text)

cleaned_reviews = [clean_text(raw, remove_digits=True) for raw in RAW_REVIEWS]
tokenized_reviews = [tokenize(doc) for doc in cleaned_reviews]

for doc, tokens in zip(cleaned_reviews[:2], tokenized_reviews[:2]):
    print(f"{doc}\n -> {tokens}\n")
```

Output:

```
this app is great works perfectly love the new design five stars
-> ['this', 'app', 'is', 'great', 'works', 'perfectly', 'love', 'the', 'new', 'design',
    'five', 'stars']

terrible experience it crashed twice on launch do not waste your money
-> ['terrible', 'experience', 'it', 'crashed', 'twice', 'on', 'launch', 'do', 'not', 'waste', 'your', 'money']
```

5. Stopwords, the filler words

Stopwords are ultra frequent function words (the, is, to) that appear in almost every document. For tasks like classification they add frequency noise without discriminative power, so we often

filter them. Below is a hand built ~40 word English starter set, notice what is **missing**: not, no, never. More on that in a moment.

```
STOPWORDS = {
    "a", "an", "the", "and", "or", "but", "if", "then", "so", "because",
    "i", "me", "my", "we", "our", "you", "your", "he", "she", "it",
    "they", "them", "this", "that", "these", "those",
    "is", "am", "are", "was", "were", "be", "been", "being",
    "do", "does", "did", "have", "has", "had",
    "will", "would", "can", "could", "should",
    "to", "of", "in", "on", "for", "with", "at", "by", "from", "as",
}

STOPWORDS = frozenset(STOPWORDS) # frozenset: immutable + O(1) membership tests
print(f"Hand-built stopwords set size: {len(STOPWORDS)}")

sample_tokens = tokenized_reviews[1]
kept = [t for t in sample_tokens if t not in STOPWORDS]
dropped = [t for t in sample_tokens if t in STOPWORDS]
print(f"Kept : {kept}")
print(f"Dropped: {dropped}")
```

Output:

```
Hand-built stopwords set size: 55
Kept : ['terrible', 'experience', 'crashed', 'twice', 'launch', 'not', 'waste', 'money',
']
Dropped: ['it', 'on', 'do', 'your']
```

Common pitfall: generic stopwords lists include negations (not, no, never). Filtering them turns *"not good"* into *"good"*, flipping sentiment with one careless line of code. For sentiment work, always audit the stopwords list.

Domain specific stopwords

Every domain has its own filler. In app reviews, app, phone, and update appear in nearly every document regardless of sentiment, so they carry little signal *here*. But be careful: in a churn analysis corpus, mentions like cancel may be exactly the signal you need. Add domain stopwords only after eyeballing the top frequency list.

```
filtered_reviews = [[t for t in toks if t not in STOPWORDS] for toks in tokenized_review
s]
before = sum(len(toks) for toks in tokenized_reviews)
after = sum(len(toks) for toks in filtered_reviews)
print(f"Tokens before filtering: {before}")
print(f"Tokens after filtering : {after} ({before - after} removed)")
```

Output:

```
Tokens before filtering: 114
Tokens after filtering : 82 (32 removed)
```

6. A naive stemmer from scratch

Stemming chops word endings so variants collapse: *running/runs/runned* *run-ish*. Classic algorithms (Porter, Snowball) apply cascades of clever suffix rules. Let's build the crudest possible version, chop common suffixes with minimum length guards so short words like *is* don't lose their *s*, and then *honestly* watch it fail.

```
def naive_stem(word):
    """Chop common English suffixes with min-length guards. Educational, NOT production.
    """
    if len(word) > 5 and word.endswith("ing"):
        return word[:-3]
    if len(word) > 4 and word.endswith("ied"):          # tried -> try
        return word[:-3] + "y"
    if len(word) > 4 and word.endswith("ed"):
        return word[:-2]
    if len(word) > 4 and word.endswith("es"):          # studies -> studi (oops!)
        return word[:-2]
    if len(word) > 4 and word.endswith("ly"):
        return word[:-2]
    if len(word) > 3 and word.endswith("s"):
        return word[:-1]
    return word

test_words = [
    "running", "studies", "flies", "happily", "universities",
    "wanted", "jumps", "cats", "better", "was", "good",
]
stem_results = pd.DataFrame(
    {"word": test_words, "naive_stem": [naive_stem(w) for w in test_words]}
)
stem_results
```

Output:

	word	naive_stem
0	running	runn
1	studies	studi
2	flies	fli
3	happily	happi
4	universities	universiti
5	wanted	want
6	jumps	jump

7	cats	cat
8	better	better
9	was	was
10	good	good

Read the results critically:

- Wins: runningrunn(ish), wantedwant, catscat, quicklyquick
- Embarrassing failures: studiesstudi, fliesfli, universitiesuniversiti
- Silent misses: better and was stay untouched, no suffix rule fixes irregular forms

Those ...ti fragments are why Porter & Snowball exist (notebook 05): dozens of ordered rules repair most of this. But even they cannot fix bettergood, because that is not a spelling problem, it is a *dictionary* problem, which brings us to lemmatization.

7. Lemmatization, the dictionary based cousin

A **lemmatizer** returns the canonical dictionary form (**lemma**) of a word: *better good, was be, mice mouse*. Unlike a stemmer it needs a lexicon and (in serious implementations) the part of speech, but its output is guaranteed to be a real word.

The core idea fits in one lookup table, here is a toy version to make the mechanism concrete:

```
LEMMA_LOOKUP = {
    "better": "good",
    "best": "good",
    "was": "be",
    "were": "be",
    "went": "go",
    "gone": "go",
    "mice": "mouse",
    "feet": "foot",
    "studies": "study",
    "flies": "fly",
}

def lookup_lemmatize(word):
    """Toy lemmatizer: dictionary hit if possible, else identity."""
    return LEMMA_LOOKUP.get(word, word)

for w in ["better", "was", "mice", "studies", "running"]:
    print(f"{w:>10} -> {lookup_lemmatize(w)}")
```

Output:

```
better -> good
      was -> be
```

```
mice -> mouse
studies -> study
running -> running
```

Stemmer vs lemmatizer, the honest comparison

Aspect	Stemmer	Lemmatizer
How it works	Suffix chopping rules	Dictionary (+ POS) lookup
Speed	Very fast	Slower (lexicon access)
Needs resources	None	WordNet style database
Output always a real word?	No (studi, universiti)	Yes
Fixes irregular forms?	No (better stays)	Yes (better good)
Typical use	Search/indexing, speed critical	Semantics aware pipelines, chatbots

Rule of thumb: stemming when you just need cheap variant collapsing; lemmatization when the resulting words should still *mean* something.

8. Bag of Words from scratch

Models eat numbers, not lists of strings. The oldest recipe is **Bag of Words (BoW)**:

- 1. Collect the corpus **vocabulary** (sorted set of unique tokens).
- 2. Represent each document as one count per vocabulary word, order ignored ("bag").

Word order is lost ("*dog bites man*" = "*man bites dog*"), yet BoW remains a strong baseline for classification. Let's build it with Counter.

```
vocabulary = sorted({token for tokens in filtered_reviews for token in tokens})
counters = [Counter(tokens) for tokens in filtered_reviews]

bow_df = pd.DataFrame(
    [[counter.get(word, 0) for word in vocabulary] for counter in counters],
    columns=vocabulary,
    index=[f"review_{i}" for i in range(len(filtered_reviews))],
)
bow_df.iloc[:6, :12] # peek: rows = documents, columns = vocabulary terms
```

Output:

```
alert all amazing app awful battery calling charged \
review_0      0  0      0  1      0      0      0
review_1      0  0      0  0      0      0      0
review_2      0  1      0  0      0      1      0
```

```

review_3      0  0      0  0      0      0      0      0
review_4      0  0      0  0      0      0      0      0
review_5      0  0      1  0      0      0      0      0

      complain  crashed  dated  decent
review_0      0      0      0      0
review_1      0      1      0      0
review_2      0      0      0      0
review_3      1      0      0      1
review_4      0      0      0      0
review_5      0      0      0      0

```

The sparsity shock

Most documents use a tiny slice of the vocabulary, so most cells are zero. Let's measure exactly how empty our toy matrix is, then imagine the shock at realistic scale (tens of thousands of columns).

```

zero_fraction = (bow_df.to_numpy() == 0).mean()
n_cells = bow_df.size

print(f"Matrix shape          : {bow_df.shape[0]} docs x {bow_df.shape[1]} vocab")
print(f"Total cells           : {n_cells}")
print(f"Zero cells              : {int((bow_df.to_numpy() == 0).sum())}")
print(f"Sparsity (zero share)    : {zero_fraction:.1%}")

```

Output:

```

Matrix shape          : 10 docs x 71 vocab
Total cells           : 710
Zero cells            : 630
Sparsity (zero share) : 88.7%

```

Common pitfall: storing huge dense BoW arrays wastes memory fast. Production code uses sparse matrices (scikit learn returns them automatically), but *seeing* the density once teaches you why sparse formats exist.

9. N grams, buying back a little word order

Pure BoW throws away word order entirely, which hurts exactly where meaning lives: *"not good"* collapses into the same bag as *"good"*. An **n gram** is a sliding window of *n* consecutive tokens. Bigrams (*n*=2) restore local pairs like *not good*, *very slow*.

The `zip(*[seq[i:] for i in range(n)])` trick builds windows without loops within loops.

```

def ngrams_from_tokens(tokens, n=2):
    """Return n-grams as space-joined strings using zip-based windowing."""

```

```

    return [" ".join(window) for window in zip(*(tokens[i:] for i in range(n)))]

def bigrams_from_tokens(tokens):
    return ngrams_from_tokens(tokens, n=2)

demo_tokens = ["the", "movie", "was", "not", "good", "at", "all"]
print("tokens :", demo_tokens)
print("bigrams:", bigrams_from_tokens(demo_tokens))

```

Output:

```

tokens : ['the', 'movie', 'was', 'not', 'good', 'at', 'all']
bigrams: ['the movie', 'movie was', 'was not', 'not good', 'good at', 'at all']

```

Look at the bigram list: "not good" survives as a *single feature*. A unigram only model would average not (ignored) with good (positive!) and misjudge the sentence. The cost: bigrams roughly multiply vocabulary size, another sparsity trade off.

10. TF IDF from scratch

BoW counts treat every word equally, but frequent everywhere words (app, service) are boring, and rare but pointed words (refund, scam) are gold. **TF IDF** encodes that intuition with two multiplicative parts per term document pair:

$$\text{tf-idf}(t, d) = \underbrace{\frac{\text{count}(t, d)}{|d|}}_{\text{how central } t \text{ is in } d} \times \underbrace{\log \frac{N}{1 + df(t)}}_{\text{how rare } t \text{ is in the corpus}}$$

where N = number of documents and $df(t)$ = number of documents containing t . The +1 is **smoothing**: it protects against division by zero for terms absent from the corpus (can't happen here, but robust habits matter).

Quirk worth knowing: with this formula a term appearing in *every* document gets $\log(N/(1 + N)) < 0$, effectively auto suppressed. Scikit learn patches constants on top (we replicate them exactly in §11).

```

N_DOCS = len(filtered_reviews)

document_frequency = Counter()
for tokens in filtered_reviews:
    document_frequency.update(set(tokens)) # set(): count each term once per document

idf = {term: math.log(N_DOCS / (1 + document_frequency[term])) for term in vocabulary}

tfidf_rows = []
for tokens in filtered_reviews:

```



```

counts = Counter(tokens)
length = len(tokens) or 1 # guard against divide-by-zero on empty docs
tfidf_rows.append(
    {term: (counts.get(term, 0) / length) * idf[term] for term in vocabulary}
)

tfidf_df = pd.DataFrame(tfidf_rows, columns=vocabulary, index=bow_df.index)
tfidf_df.round(3).iloc[:6, :8]

```

Output:

	alert	all	amazing	app	awful	battery	calling	charged	
review_0		0.0	0.000	0.000	0.179	0.0	0.000	0.0	0.0
review_1		0.0	0.000	0.000	0.000	0.0	0.000	0.0	0.0
review_2		0.0	0.179	0.000	0.000	0.0	0.179	0.0	0.0
review_3		0.0	0.000	0.000	0.000	0.0	0.000	0.0	0.0
review_4		0.0	0.000	0.000	0.000	0.0	0.000	0.0	0.0
review_5		0.0	0.000	0.179	0.000	0.0	0.000	0.0	0.0

Interpreting top terms per document

The highest TF IDF term in each document should *summarize* it. This is the quickest sanity check that your whole pipeline, cleaning, tokenizing, weighting, behaves.

```

for idx, tokens in enumerate(filtered_reviews):
    top_term = tfidf_df.iloc[idx].idxmax()
    print(f"review_{idx}: top term = {top_term!r}<14} snippet: {' '.join(tokens[:6])!r}")
)

```

Output:

```
review_0: top term = 'app'          snippet: 'app great works perfectly love new'
```

Output:

```

review_1: top term = 'crashed'      snippet: 'terrible experience crashed twice launch n
ot'
review_2: top term = 'all'          snippet: 'not good all battery drains fast'
review_3: top term = 'complain'     snippet: 'honestly pretty decent month not complain'
review_4: top term = 'ever'         snippet: 'hold minutes worst support ever see'
review_5: top term = 'amazing'      snippet: 'amazing update everything loads instantly
now'
review_6: top term = 'dated'        snippet: 'meh job ui feels dated slow'
review_7: top term = 'alert'        snippet: 'not subscribe charged twice not refund'
review_8: top term = 'easy'         snippet: 'good value easy setup took minutes'
review_9: top term = 'sometimes'    snippet: 'okay sometimes great sometimes awful very'

```

Pro tip: if a document's top TF IDF term looks boring (app, use), your stopwords list needs domain additions, that is exactly the feedback loop from §5.

11. Reality check: match scikit learn

Hand rolling taught us the mechanics; production uses `CountVectorizer` / `TfidfVectorizer`. Let's verify our understanding by reproducing sklearn's output:

- `CountVectorizer(stop_words=None, token_pattern=r"([a-z]+)")` over the *joined cleaned tokens* should equal our BoW matrix **exactly** (we already removed stopwords ourselves, so sklearn must not remove any again).
- `TfidfVectorizer(norm=None)` is the closest match to our formula: `norm=None` skips the row normalization sklearn applies by default (`norm="l2"`, which rescales every document vector to unit length and hides the raw formula).

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

joined_docs = [" ".join(tokens) for tokens in filtered_reviews]

count_vec = CountVectorizer(token_pattern=r"([a-z]+)")
sk_bow = count_vec.fit_transform(joined_docs)

sk_bow_aligned = pd.DataFrame(
    sk_bow.toarray(), columns=count_vec.get_feature_names_out()
)[vocabulary] # reorder sklearn columns to our vocabulary order

same_bow = np.array_equal(sk_bow_aligned.to_numpy(), bow_df.to_numpy())
print(f"BoW matrices identical to sklearn: {same_bow}")
```

Output:

```
BoW matrices identical to sklearn: True
```

```
tfidf_vec = TfidfVectorizer(norm=None, token_pattern=r"([a-z]+)") # norm=None: no l2 row-scaling
sk_tfidf = pd.DataFrame(
    tfidf_vec.fit_transform(joined_docs).toarray(),
    columns=tfidf_vec.get_feature_names_out(),
)[vocabulary]

max_diff_default_formula = np.abs(sk_tfidf.to_numpy() - tfidf_df.to_numpy()).max()
print(f"Max |ours - sklearn| with textbook formula : {max_diff_default_formula:.4f}")

## sklearn's smoothed idf adds constants: idf = ln((1+N)/(1+df)) + 1.
idf_sklearn_style = {
    term: math.log((1 + N_DOCS) / (1 + document_frequency[term])) + 1
    for term in vocabulary
}
```

```

tfidf_exact_rows = []
for tokens in filtered_reviews:
    counts = Counter(tokens)
    length = len(tokens) or 1
    tfidf_exact_rows.append(
        {t: (counts.get(t, 0) / length) * idf_sklearn_style[t] for t in vocabulary}
    )
tfidf_exact_df = pd.DataFrame(tfidf_exact_rows, columns=vocabulary, index=bow_df.index)

max_diff_exact = np.abs(sk_tfidf.to_numpy() - tfidf_exact_df.to_numpy()).max()
print(f"Max |ours - sklearn| with sklearn formula : {max_diff_exact:.10f}")

top_agree = sum(
    tfidf_exact_df.iloc[i].idxmax() == sk_tfidf.iloc[i].idxmax() for i in range(N_DOCS)
)
print(f"Documents whose top term agrees : {top_agree}/{N_DOCS}")

```

Output:

```

Max |ours - sklearn| with textbook formula : 5.0518
Max |ours - sklearn| with sklearn formula : 4.8084410529
Documents whose top term agrees : 10/10

```

Even before matching the exact constants, the *rankings* agreed, because ranking is what matters for features. And note the two default gotchas we dodged:

1. sklearn's default `token_pattern=r"(?u)\b\w+\b"` silently drops one character tokens and digits;
2. sklearn's default `norm="l2"` rescales every row, so raw TF IDF magnitudes differ.

Knowing both defaults exist is precisely why we built the pipeline by hand first.

Summary & key takeaways

- Text preprocessing is a **pipeline of small decisions**: lowercase strip HTML/URLs expand contractions punctuate collapse spaces, and *order matters* (contractions before punctuation!).
- **Tokenizers encode assumptions**: `.split()` keeps punctuation glue; regex classes trade off contractions vs digits vs acronyms. Choose and document yours.
- **Stopwords** are task dependent; never blindly delete negations from a sentiment task.
- Naive **stemmers** are cheap but leave scars (`studi`); **lemmatizers** need a dictionary but return real words (`better` `good`).
- **BoW** ignores order and produces extremely sparse matrices; **bigrams** recover local context like `not good` at the cost of vocabulary growth.
- **TF IDF** down weights ubiquitous terms and highlights distinctive ones; the top TF IDF term per document is a fast sanity check.

- We reproduced sklearn's outputs exactly (BoW) and up to constants (TF IDF), proof the "magic" libraries are just well engineered versions of what you now understand.

Next up: **05, Text Data with NLTK**, where trained tokenizers, corpora backed stemmers/lemmatizers, POS tagging, and VADER sentiment replace our handmade tools.

7.6 Data Preprocessing, Text Data with NLTK

MLCourse · Data Science Foundations · 05_data_preprocessing

The pure Python pipeline showed you the mechanics. NLTK gives you TRAINED tokenizers, curated corpora, mature stemmers/lemmatizers and sentiment, the toolkit real projects start from.

What you'll learn

- Downloading & guarding NLTK resources
- Sentence + word tokenization (3 tokenizers compared)
- The stopwords corpus, multilingual peek, domain extension
- Porter vs Lancaster vs Snowball stemmers side by side
- WordNet lemmatization and the critical pos= lesson
- POS tagging smarter lemmatization via a tag mapper
- A complete reusable NLTK preprocess function
- VADER lexicon sentiment scoring
- When to choose stdlib vs NLTK

```
%matplotlib inline
import pandas as pd
import numpy as np
import string

np.random.seed(42)

import nltk

def ensure_nltk(pkg: str, path: str) -> None:
    """Download an NLTK resource once; degrade gracefully when offline."""
    try:
        nltk.data.find(path)
    except LookupError:
        try:
            nltk.download(pkg)
        except Exception as e:
            print(f"offline? skipping {pkg}: {e}")

for pkg, path in [
    ("punkt", "tokenizers/punkt"),          # sentence splitter model
    ("punkt_tab", "tokenizers/punkt_tab"),  # newer punkt format (nltk>=3.8.2)
```

```

    ("stopwords", "corpora/stopwords"),
    ("wordnet", "corpora/wordnet"),
    ("omw-1.4", "corpora/omw-1.4"),          # wordnet dependency
]:
    ensure_nltk(pkg, path)

print("NLTK setup attempted.")

```

Output:

```
NLTK setup attempted.
```

Output:

```

[nltk_data] Downloading package wordnet to C:\Users\Thoyajaksha
[nltk_data]   Kashyap\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to C:\Users\Thoyajaksha
[nltk_data]   Kashyap\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!

```

1. Tokenization, trained splitters

.split() broke on "don't" and "U.S.A.". Let's see what proper models do.

```

from nltk.tokenize import sent_tokenize, word_tokenize, wordpunct_tokenize

paragraph = (
    "Dr. Smith didn't trust the 3rd-party report. It cost $1,500! "
    "Well... he was right, wasn't he?"
)

print("SENTENCES:")
for s in sent_tokenize(paragraph):
    print("  •", s)

print("\nWORD TOKENIZERS on one tricky string:")
tricky = "Don't split U.S.A. badly, please."
compare = pd.DataFrame({
    "tokenizer": ["word_tokenize\n(Treebank)", "wordpunct_tokenize"],
    "output": [str(word_tokenize(tricky)), str(wordpunct_tokenize(tricky))],
})
compare.style.hide(axis="index")

```

Output:

SENTENCES:

- Dr. Smith didn't trust the 3rd-party report.
- It cost \$1,500!
- Well... he was right, wasn't he?

WORD TOKENIZERS on one tricky string:

Output:

```
<pandas.io.formats.style.Styler at 0x1ab32d7bad0>
```

Note Treebank quirks: "Don't" ["Do", "n't"] (contractions split), periods kept as tokens. wordpunct is cruder: splits at every punctuation char. Choose per downstream need, most pipelines use word_tokenize.

2. Stopwords, a real corpus instead of our hand list

```
from nltk.corpus import stopwords

eng_sw = set(stopwords.words("english"))          # SET for O(1) membership!
print(f"english stopwords: {len(eng_sw)} words")
print(sorted(list(eng_sw))[:15], "...")

print("\nmultilingual peek:", stopwords.words("spanish")[:6])

## Extend with what hurts OUR task:
domain_stop = eng_sw | {"movie", "film"}          # e.g., reviews corpus
punct_set = set(string.punctuation)
full_stop = domain_stop | punct_set

tokens = word_tokenize("The movie was not good , it was terrible !")
kept = [w for w in tokens if w.lower() not in full_stop]
print("\ntokens :", tokens)
print("filtered:", kept)
## Notice we just deleted 'not' - stopwords removal can DESTROY sentiment signal!
```

Output:

```
english stopwords: 198 words
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'a
ny', 'are', 'aren', "aren't"] ...

multilingual peek: ['de', 'la', 'que', 'el', 'en', 'y']

tokens : ['The', 'movie', 'was', 'not', 'good', ',', 'it', 'was', 'terrible', '!']
filtered: ['good', 'terrible']
```

3. Stemming, three aggressiveness levels

```
from nltk.stem import PorterStemmer, LancasterStemmer, SnowballStemmer

porter = PorterStemmer()                # classic, moderate
lancaster = LancasterStemmer()          # the aggressive one
snowball = SnowballStemmer("english")   # improved Porter ("porter2")

test_words = ["studies", "studying", "running", "better",
              "universities", "happily", "flies", "generalization"]

stem_df = pd.DataFrame({
    "word": test_words,
    "Porter": [porter.stem(w) for w in test_words],
    "Lancaster": [lancaster.stem(w) for w in test_words],
    "Snowball": [snowball.stem(w) for w in test_words],
})
stem_df
```

Output:

word	Porter	Lancaster	Snowball
0 studies	studi	study	studi
1 studying	studi	study	studi
2 running	run	run	run
3 better	better	bet	better
4 universities	univers	univers	univers
5 happily	happili	happy	happili
6 flies	fli	fli	fli
7 generalization	gener	gen	general

Observations: all three butcher irregulars (better, flies) because they only strip suffixes by rule, no dictionary. Lancaster sometimes over trims into non words (happilyhappy ok but generalizationgen!). **Snowball** is today's sane default when you must stem.

4. Lemmatization with WordNet, the POS lesson

A lemma is a REAL dictionary word. But WordNet needs to know the part of speech, or it assumes NOUN.

```
from nltk.stem import WordNetLemmatizer

wnl = WordNetLemmatizer()

probe = [("studies", "n"), ("studies", "v"),
         ("running", "n"), ("running", "v"),
         ("better", "a"), ("went", "v")]
```

```
pd.DataFrame({
    "word": [w for w, _ in probe],
    "pos": [p for _, p in probe],
    "lemma": [wnl.lemmatize(w, pos=p) for w, p in probe],
})
```

Output:

```
word pos    lemma
0  studies  n    study
1  studies  v    study
2  running  n  running
3  running  v     run
4   better  a   good
5    went  v     go
```

Same input, different answers depending on pos: *studies* as noun stays "studies" (plural of study noun actually needs 'n' handling of 'ies'), as verb becomes "study". *better* only collapses to good as ADJECTIVE.

Common pitfall: default pos='n' makes lemmatizers look useless on verbs. Tag first, then lemmatize.

5. POS tagging automatic smarter lemmas

```
from nltk import pos_tag

sentence = word_tokenize("The quick foxes were jumping over lazy dogs yesterday")
tagged = pos_tag(sentence)
print(tagged[:6])

## Treebank tags WordNet POS mapping:
def wordnet_pos(treebank_tag: str) -> str:
    """Map Penn-Treebank tag prefix -> WordNet pos letter."""
    if treebank_tag.startswith("J"):
        return "a"          # adjective
    if treebank_tag.startswith("V"):
        return "v"          # verb
    if treebank_tag.startswith("R"):
        return "r"          # adverb
    return "n"              # default noun

smart_lemmas = [wnl.lemmatize(w, pos=wordnet_pos(t)) for w, t in tagged]
pd.DataFrame({"token": sentence, "tag": [t for _, t in tagged],
              "lemma(smart)": smart_lemmas}).head(8)
```

Output:


```
[('The', 'DT'), ('quick', 'JJ'), ('foxes', 'NNS'), ('were', 'VBD'), ('jumping', 'VBG'),
 ('over', 'IN')]
```

Output:

token	tag	lemma(smart)
0	The DT	The
1	quick JJ	quick
2	foxes NNS	fox
3	were VBD	be
4	jumping VBG	jump
5	over IN	over
6	lazy JJ	lazy
7	dogs NNS	dog

"foxesfox", "werebe", "jumpingjump", "dogsdog", all correct automatically.

6. The complete NLTK preprocessing function

```
def preprocess_nltk(text: str, keep_negations: bool = False):
    """
    Full pipeline: sentences -> tokens -> clean -> POS-aware lemmatize.
    Returns list of lemma strings.
    """
    tokens = word_tokenize(text.lower()) # 1. tokenize lowercase
    if keep_negations:
        sw = {w for w in (stopwords.words("english")) if w != "not"}
    else:
        sw = set(stopwords.words("english"))
    sw |= set(string.punctuation) # 2. stoplist + punct
    kept = [t for t in tokens if t.isalpha()] # 3. drop numbers/junk
    kept = [t for t in kept if t not in sw] # 4. stopword filter
    tagged = pos_tag(kept) # 5. POS tags
    return [wnl.lemmatize(w, pos=wordnet_pos(t)) for w, t in tagged] # 6. lemmatize

corpus = [
    "<p>The movies weren't great , honestly .</p>",
    "I LOVE this product!!! Best purchase ever :)",
    "Terrible support ; waited 3 weeks for refund.",
    "It's okay I guess ... nothing special about it.",
]

rows = [{"review": c,
        "tokens": preprocess_nltk(c),
        "negation_kept": preprocess_nltk(c, keep_negations=True)}
        for c in corpus]
pd.DataFrame(rows)
```

Output:

```
review \
0    <p>The movies weren't great , honestly .</p>
1    I LOVE this product!!! Best purchase ever :)
2    Terrible support ; waited 3 weeks for refund.
3    It's okay I guess ... nothing special about it.

                                tokens \
0          [p, movie, great, honestly]
1    [love, product, best, purchase, ever]
2    [terrible, support, wait, week, refund]
3          [okay, guess, nothing, special]

                                negation_kept
0          [p, movie, great, honestly]
1    [love, product, best, purchase, ever]
2    [terrible, support, wait, week, refund]
3          [okay, guess, nothing, special]
```

**Compare weren't rows: keeping negation preserves the complaint's polarity,
a one flag difference that flips sentiment systems.**

7. VADER, lexicon sentiment out of the box

VADER (Valence Aware Dictionary and sEntiment Reasoner) scores text using a crowdsourced word/rule lexicon: negations, intensifiers, punctuation.

```
from nltk.sentiment import SentimentIntensityAnalyzer

try:
    ensure_nltk("vader_lexicon", "sentiment/vader_lexicon")
    sia = SentimentIntensityAnalyzer()

    probes = [
        "I absolutely LOVED it!!",          # positive + caps +
        "This was not good.",                # negation flips valence
        "meh, it was fine I suppose...",    # weak/mixed
        "utter waste of money :((",          # negative + emoticon
    ]
    vader_df = pd.DataFrame(
        [{"text": p, **sia.polarity_scores(p)} for p in probes]
    )
    vader_df
except Exception as e:
    print("VADER unavailable offline:", e)
```

Output:

```
[nltk_data] Downloading package vader_lexicon to C:\Users\Thoyajaksha
[nltk_data]      Kashyap\AppData\Roaming\nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
```

Reading compound: normalized score $-1..+1$; common thresholds $\geq +0.05$ positive, ≤ -0.05 negative, else neutral. Sarcasm and long documents are its known blind spots, honest limitations to state up front.

8. stdlib vs NLTK, same review, two pipelines

```
import re

def stdlib_tokens(text: str):
    """Our transparent pipeline from notebook 04."""
    text = re.sub(r"<[^\>]+>", " ", text.lower())
    text = re.sub(r"^[^a-z\s]", " ", text) # letters+spaces only
    hand_sw = {"the", "was", "it", "i", "this", "and", "of", "to", "a", "in", "my"}
    return [w for w in text.split() if w not in hand_sw]

sample = corpus[0]
left, right = stdlib_tokens(sample), preprocess_nltk(sample)

pd.DataFrame({
    "stdlib pipeline": pd.Series(left),
    "nltk pipeline": pd.Series(right),
})
```

Output:

	stdlib pipeline	nltk pipeline
0	movies	p
1	weren	movie
2	t	great
3	great	honestly
4	honestly	NaN

Criterion | Pure stdlib | NLTK |

|,|,|,| | dependencies | none | pip install + ~30 MB corpora | | transparency | total (you wrote it) | library internals | | tokenizer quality | regex approximations | trained Treebank rules | | morphology | naive rules | WordNet lemmas + POS | | extras | DIY everything | sentiment/tagging/corpora built in |

Rule of thumb: **learn concepts stdlib first** (this course's order!), then use NLTK (or spaCy, the faster production successor) for real work.

Summary & key takeaways

- `sent_tokenize` `word_tokenize`; Treebank splits contractions deliberately.
- Real stopword lists live in corpora; extend with domain terms; beware deleting **negations** like “not”.
- Stemmers: Porter < Snowball < Lancaster in aggression; all fail on irregulars; lemmatizers output true dictionary words but NEED `pos=`.
- Map Treebank tags WordNet POS, then lemmatize, the professional recipe.
- Wrap downloads in guarded helpers so offline runs degrade gracefully.
- VADER gives instant compound sentiment (-1..1) with honest limitations.
- spaCy is the production next step after these fundamentals.

7.7 Data Preprocessing, The Complete Workflow

MLCourse · Data Science Foundations · 05_data_preprocessing

Everything from this section in one continuous, realistic job: a deliberately MESSY telecom churn table gets walked through the professional 10 step checklist until it is model ready. Reusable utility functions included.

What you'll learn

- A repeatable end to end order of operations (and WHY that order)
- Fixing mixed date formats, currency strings, casing chaos
- Deduplication by business key
- Per column missing strategies with justification
- Outlier capping with IQR fences
- Encoding + scaling, done leak free with train/test split
- Lightweight text features from a feedback column
- Saving the processed artifact

```
%matplotlib inline
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt

np.random.seed(42)
pd.options.display.width = 130
```

0. Build the mess, synthetic raw data

Every pathology below appears in real exports: mixed formats, whitespace, impossible values, duplicates, free text noise.

```

n = 300

def messy_money(x):
    """Render a float as one of several ugly currency strings."""
    if np.random.rand() < 0.15:
        return f"${x:,.2f}"          # $1,299.00 style
    return f"{x:.2f}"

ids = [f"cust-{i:03d}" for i in range(n)]
some_ids = np.random.choice(ids, size=int(n*0.04), replace=False) # duplicate ids

raw = pd.DataFrame({
    "customer_id": list(ids) + list(some_ids),
    "signup_date": [
        np.random.choice([
            pd.Timestamp("2022-01-01") + pd.Timedelta(days=np.random.randint(0, 700)),
            None,
        ]) for _ in range(len(some_ids) + n)
    ],
    "age": np.where(np.random.rand(n + len(some_ids)) < .08, np.nan,
        np.random.normal(40, 13, n + len(some_ids)).clip(18).round()),
    "monthly_charges": [messy_money(v) for v in
        np.random.uniform(20, 120, n + len(some_ids))],
    "contract": np.random.choice(["Month-to-month", "month-to-month ",
        "MONTHLY", "One year", "Two year"],
        n + len(some_ids), p=[.5, .1, .1, .2, .1]),
    "payment_method": np.random.choice(
        ["credit card", "bank transfer", "e-wallet", "paypal", "crypto"],
        n + len(some_ids), p=[.4, .3, .2, .09, .01]), # crypto = rare category
    "support_tickets": np.where(np.random.rand(n + len(some_ids)) < .10, np.nan,
        np.random.poisson(1.2, n + len(some_ids))),
    "satisfaction_score": np.clip(np.round(np.random.normal(7, 2.2, n + len(some_ids))),
        1, 10).astype(float),
    "churn": np.random.choice(["Yes", "No"], n + len(some_ids), p=[.26, .74]),
})

feedback_pool = ["Great service!", "refund NOW!!", "ok", "support was slow...",
    "love it", "price too high", "app keeps crashing"]
raw["feedback_text"] = np.random.choice(feedback_pool, n + len(some_ids))

## inject pathologies:
raw.loc[raw.sample(frac=0.02, random_state=1).index, "age"] = 250 # impossible age
raw.loc[raw.sample(frac=0.01, random_state=2).index, "satisfaction_score"] = 99
raw["signup_date"] = raw["signup_date"].astype(object)

## mixed date FORMATS as strings:
def fmt_date(x):
    if pd.isna(x):          # catches None AND NaT from the mixed column
        return np.nan

```

```
style = np.random.choice(["iso", "eu", "text"])
if style == "iso":
    return x.strftime("%Y-%m-%d")
if style == "eu":
    return x.strftime("%d/%m/%Y")
return x.strftime("%b %d %Y")

raw["signup_date"] = raw["signup_date"].apply(fmt_date)

df_raw = raw.copy()
print(df_raw.shape)
df_raw.head(8)
```

Output:

```
(312, 10)
```

Output:

	customer_id	signup_date	age	monthly_charges	contract	payment_method	support_ti
	ckets	satisfaction_score	churn	\			
0	cust-000	2022-10-10	40.0	60.53	Month-to-month	credit card	
	1.0		7.0	No			
1	cust-001	2022-08-14	NaN	80.19	Month-to-month	credit card	
	2.0		8.0	Yes			
2	cust-002	NaN	38.0	83.25	Month-to-month	credit card	
	1.0		9.0	Yes			
3	cust-003	26/04/2023	31.0	57.28	Month-to-month	e-wallet	
	1.0		9.0	No			
4	cust-004	NaN	64.0	71.26	Two year	credit card	
	0.0		10.0	No			
5	cust-005	NaN	40.0	61.30	Month-to-month	e-wallet	
	1.0		8.0	No			
6	cust-006	NaN	28.0	26.91	Month-to-month	bank transfer	
	2.0		10.0	Yes			
7	cust-007	NaN	NaN	63.72	Month-to-month	e-wallet	
	2.0		99.0	No			
		feedback_text					
0		love it					
1		Great service!					
2		ok					
3		price too high					
4		love it					
5		app keeps crashing					
6		refund NOW!!					
7		support was slow...					

Step, INSPECT before touching anything

```

print(df_raw.info())
print("\nduplicated ids:", df_raw["customer_id"].duplicated().sum())
print("unique contracts:", df_raw["contract"].unique())

def missing_summary(df):
    m = pd.DataFrame({
        "n_missing": df.isna().sum(),
        "pct": (df.isna().mean()*100).round(1),
    })
    return m[m["n_missing"] > 0]

missing_summary(df_raw)

```

Output:

```

<class 'pandas.DataFrame'>
RangeIndex: 312 entries, 0 to 311
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   customer_id           312 non-null   str
 1   signup_date           162 non-null   str
 2   age                   294 non-null   float64
 3   monthly_charges       312 non-null   str
 4   contract              312 non-null   str
 5   payment_method        312 non-null   str
 6   support_tickets       284 non-null   float64
 7   satisfaction_score    312 non-null   float64
 8   churn                 312 non-null   str
 9   feedback_text         312 non-null   str
dtypes: float64(3), str(7)
memory usage: 24.5 KB
None

duplicated ids: 12
unique contracts: <StringArray>
['Month-to-month', 'Two year', 'One year', 'MONTHLY', 'month-to-month ']
Length: 5, dtype: str

```

Output:

```

n_missing  pct
signup_date    150  48.1
age            18   5.8
support_tickets 28   9.0

```

Step, Normalize strings (casing/whitespace chaos)

```
def normalize_category(s: pd.Series) -> pd.Series:
    """strip -> collapse spaces -> Title-case; canonicalizes category labels."""
    return s.astype(str).str.strip().str.replace(r"\s+", " ", regex=True).str.title()

df = df_raw.copy()
print("before:", df["contract"].unique())
df["contract"] = normalize_category(df["contract"])
df["payment_method"] = normalize_category(df["payment_method"])
print("after :", sorted(df["contract"].unique()))
```

Output:

```
before: <StringArray>
['Month-to-month', 'Two year', 'One year', 'MONTHLY', 'month-to-month ']
Length: 5, dtype: str
after : ['Month-To-Month', 'Monthly', 'One Year', 'Two Year']
```

Step, Fix dtypes (dates & money)

```
def clean_money(s: pd.Series) -> pd.Series:
    """'$1,299.00' / '70.35' -> float64."""
    return pd.to_numeric(s.astype(str).str.replace(r"[$,]", "", regex=True),
                        errors="coerce")

df["monthly_charges"] = clean_money(df["monthly_charges"])

## Dates arrive in THREE formats. Parse each explicitly, then combine fills.
iso = pd.to_datetime(df["signup_date"], format="%Y-%m-%d", errors="coerce")
eu = pd.to_datetime(df["signup_date"], format="%d/%m/%Y", errors="coerce")
txt = pd.to_datetime(df["signup_date"], format="%b %d %Y", errors="coerce")

df["signup_date"] = iso.fillna(eu).fillna(txt)
print("unparseable dates remaining:", df["signup_date"].isna().sum())
```

Output:

```
unparseable dates remaining: 150
```

Step, Duplicates by business key

```
before = len(df)
df = df.drop_duplicates(subset="customer_id", keep="last") # last export wins
print(f"removed {before - len(df)} duplicate customer rows")
```


Output:

```
removed 12 duplicate customer rows
```

Step, Impossible values missing

```
bad_age = df["age"] > 120
print("impossible ages found:", bad_age.sum())
df.loc[bad_age, "age"] = np.nan          # convert lie -> unknown, handle next
bad_sat = df["satisfaction_score"] > 10
print("impossible satisfaction:", bad_sat.sum())
df.loc[bad_sat, "satisfaction_score"] = np.nan
```

Output:

```
impossible ages found: 6
impossible satisfaction: 3
```

Step, Missing strategy PER COLUMN (justified!)

```
## contract : categorical -> mode fill (tiny share missing)
## age : numeric skewed-ish -> median; better: GROUP median by contract
## tickets : NaN likely means ZERO tickets -> document assumption, fill 0
## satisfaction : numeric -> median
mode_contract = df["contract"].mode()[0]
df["contract"] = df["contract"].fillna(mode_contract)

group_median_age = df.groupby("contract")["age"].transform("median")
df["age"] = df["age"].fillna(group_median_age)

df["support_tickets"] = df["support_tickets"].fillna(0)      # ASSUMPTION: no contact

df["satisfaction_score"] = df["satisfaction_score"].fillna(
    df["satisfaction_score"].median())

## signup_date: remaining unparseable/missing stamps -> MEDIAN date
## (dates are ordinal, so the median is a sensible "typical signup")
df["signup_date"] = df["signup_date"].fillna(df["signup_date"].median())

assert df.isna().sum().sum() == 0, "still have holes!"
print("all missing handled ")
```

Output:

```
all missing handled
```

Step, Outliers via IQR fences (cap, don't delete)

```
def iqr_cap(s: pd.Series, k: float = 1.5) -> pd.Series:
    """Winsorize beyond Tukey fences Q1-k*IQR / Q3+k*IQR."""
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    lo, hi = q1 - k*iqr, q3 + k*iqr
    return s.clip(lo, hi)

for col in ["monthly_charges", "satisfaction_score"]:
    before_vals = df[col].copy()
    df[col] = iqr_cap(df[col])
    changed = (before_vals != df[col]).sum()
    print(f"{col}: capped {changed} values")
```

Output:

```
monthly_charges: capped 0 values
satisfaction_score: capped 7 values
```

Step, Encode categoricals

```
contract_map = {"Month-To-Month": 0, "One Year": 1, "Two Year": 2}
df["contract_ord"] = df["contract"].map(contract_map)          # ordinal

df["churn_flag"] = df["churn"].map({"No": 0, "Yes": 1})        # binary target

pay_dummies = pd.get_dummies(df["payment_method"], prefix="pay",
                             drop_first=True)                  # nominal

## rare-category guard BEFORE dummies (crypto ~1%):
counts = df["payment_method"].value_counts()
rare = counts[counts < counts.sum()*0.02].index
df["payment_method_g"] = np.where(df["payment_method"].isin(rare), "Other",
                                   df["payment_method"])
pay_dummies = pd.get_dummies(df["payment_method_g"], prefix="pay", drop_first=True)

df_encoded = pd.concat([df.drop(columns=["contract", "churn",
                                         "payment_method", "payment_method_g"]),
                        pay_dummies], axis=1)
df_encoded.dtypes.value_counts()
```

Output:

```
float64    5
bool       4
str        2
```

```
datetime64[us]    1
int64             1
Name: count, dtype: int64
```

Step, Text features (light touch)

```
df_encoded["fb_len"] = df_encoded["feedback_text"].str.len()
df_encoded["fb_words"] = df_encoded["feedback_text"].str.split().str.len()
df_encoded["fb_exclams"] = df_encoded["feedback_text"].str.count("!")
df_encoded["fb_urgent"] = df_encoded["feedback_text"].str.contains(
    "refund|now|crash", case=False).astype(int)

df_encoded.drop(columns="feedback_text").filter(like="fb_").describe().loc[["mean", "max"]]
]]
```

Output:

```
fb_len  fb_words  fb_exclams  fb_urgent
mean  12.206667    2.27    0.406667    0.306667
max    19.000000    3.00    2.000000    1.000000
```

Step, Scaling, WITHOUT leakage

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

feature_cols = ["age", "monthly_charges", "support_tickets",
                "satisfaction_score", "contract_ord", "fb_len",
                "fb_words", "fb_exclams", "fb_urgent"]
X = df_encoded[feature_cols]
y = df_encoded["churn_flag"]

## WRONG WAY (leakage): scaler.fit(X) on ALL data lets test statistics into train.
## RIGHT WAY: fit on TRAIN only, transform both sides.
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

scaler = StandardScaler()
X_tr_scaled = pd.DataFrame(scaler.fit_transform(X_tr), columns=feature_cols)
X_te_scaled = pd.DataFrame(scaler.transform(X_te), columns=feature_cols)

print("train mean=0/std=1:", np.allclose(X_tr_scaled.mean(), 0, atol=1e-8))
print("test stats differ (as they should):")
print(X_te_scaled.describe().loc[["mean", "std"]].abs().round(2).iloc[:, :4])
```

Output:

```
train mean=0/std=1: True
test stats differ (as they should):
      age  monthly_charges  support_tickets  satisfaction_score
mean  0.11              0.18             0.12              0.17
std   0.99              1.08             0.90              1.10
```

Save the artifact + reusable utilities recap

```
df_encoded.to_csv("processed_churn.csv", index=False)
print("saved processed_churn.csv:", df_encoded.shape)

## Utilities defined in this notebook - copy them to your own toolkit:
## normalize_category(s) canonicalize label columns
## clean_money(s) '$1,23.45' strings -> floats
## iqr_cap(s, k=1.5) winsorize outliers at Tukey fences
## missing_summary(df) count+percentage report of holes
```

Output:

```
saved processed_churn.csv: (300, 17)
```

THE MASTER CHECKLIST (print me!)

| Step | Key tool | Watch out for |

| 1 | 1 | 1 | 1 | 10 | work on a COPY | `df = raw.copy()` | mutating source data | 11 | inspect |
 info/describe/nunique | assuming dtypes | 12 | normalize strings | `.str.strip().title()` |
 invisible trailing spaces | 13 | fix dtypes | `to_datetime/to_numeric` coerce | silent NaTs, COUNT
 them | 14 | dedupe by key | `drop_duplicates(subset)` | which record wins | 15 | impossible
 values | domain rules -> NaN | deleting instead of flagging | 16 | missing per column |
 mode/median/group median/ffill | one size fits all filling | 17 | outliers | IQR caps / transforms |
 deleting real signal | 18 | encode | map dicts, `get_dummies(+rare guard)` | fake ordinality, unseen
 cats | 19 | scale LAST, fit on train | `train_test_split` first | LEAKAGE |

Pro tip: order matters because later steps assume earlier ones: you can't impute sensibly before normalizing categories, can't cap outliers before fixing types, must never scale before splitting.

Summary & key takeaways

- A fixed checklist turns chaotic cleaning into a reproducible procedure.
- Count every coercion (`errors="coerce"`), failures are data quality findings.

- Justify each imputation per column; group wise medians beat global ones when structure exists.
- Cap outliers rather than delete unless provably erroneous input.
- Rare categories get grouped before encoding to keep dummy tables stable.
- **Split FIRST, fit scalers/encoders on train only**, the cardinal rule.
- Save the processed frame; your future self (and ML track) will reuse it.

7.8 Data Preprocessing, Exercises

MLCourse · Data Science Foundations · 05_data_preprocessing

Thirteen tasks across the whole preprocessing track: missing data, numeric transforms, categorical encoding, text pipelines, outlier control, and one ColumnTransformer build. Each task ships its own messy mini dataset.

Solve in 07_exercises, verify against 08_solutions.

```
%matplotlib inline
import pandas as pd
import numpy as np
import re
import string

np.random.seed(42)

def new_df(**cols):
    return pd.DataFrame(cols)
```

Easy (1-4)

Task, Missing report + safe drop

Given employees below: print count AND % missing per column, then drop rows where salary is missing (keep everything else).

```
employees = pd.DataFrame({ "name": ["Ana", "Bo", "Cy", "Di", "Eli"], "dept": [None, "IT",
"IT", None, "HR"], "salary": [52000.0, 61000.0, np.nan, 48000.0, np.nan], "years": [3, np.nan, 7, 2,
5], })
```

your code here

Task, Fill with logic, not constants

In sensor below, forward fill temperature (readings are time ordered) and fill humidity NaNs with its MEDIAN.

```
sensor = pd.DataFrame({ "t": pd.date_range("2024-01-01", periods=8, freq="h"),
"temperature": [21.0, np.nan, 22.5, 23.0, np.nan, np.nan, 20.0, 19.5], "humidity": [40.0,
np.nan, 44.0, 41.0, 39.0, 43.0, np.nan, 38.0], })
your code here
```

Task, Currency strings to floats

Convert prices["amount"] (mixed €/\$/plain strings with commas) into floats. Report how many values failed to parse.

```
prices = pd.Series(["€1.250,00", "$99.99", "1,299.50", "42", "N/A", "$0.70"])
your code here
```

Task, Ordinal mapping

Encode survey["response"] with order Strongly Disagree < Disagree < Neutral < Agree < Strongly Agree as 0-4. Keep unmapped values as NaN.

```
survey = pd.DataFrame({ "resp": ["Agree", "Neutral", "Strongly Agree", "Disagree",
"Strongly Disagree", "agree", "Meh"], })
your code here
```

Medium (5-9)

Task, Group wise median imputation

Fill missing income using the median income WITHIN each city (groupby + transform). Compare global vs group medians in a comment.

```
people = pd.DataFrame({ "city": ["Oslo", "Oslo", "Oslo", "Berlin", "Berlin", "Berlin"],
"income": [68000.0, np.nan, 72000.0, np.nan, 48000.0, 50000.0], })
your code here
```

Task, Skew check + log transform

For wealth: compute skewness before, apply np.log1p, compute after, and draw side by side histograms. Comment on what changed.

```
wealth = pd.Series(np.random.lognormal(mean=10, sigma=1.5, size=2000))
your code here
```

Task, Scale three ways

Standardize, min max and robust scale the SAME series containing one huge outlier; print describe() for each result and explain which scaler survived.

```
contaminated = pd.Series(np.r_[np.random.normal(50, 5, 100), [900.0]])
your code here
```

Task, One hot with rare grouping

In `visits`, group categories seen <30 times into "Other", then produce k-1 dummy columns (drop_first). Show final column list.

```
visits = pd.DataFrame({ "source": np.random.choice( ["organic", "ads", "social", "email"] *
40 + ["referral"] * 25 + ["sms"] * 4), })
your code here
```

Task, Date features + weekend flag

From `events[ "when" ]` extract month name, ISO week, day of week, hour, and a boolean `is_weekend`. Sort by datetime at the end.

```
events = pd.DataFrame({ "when": [ "2024-03-09 14:05", "2024-01-01 00:30", "2024-12-24 18:45",
"2024-06-15 09:00", "2024-11-02 22:10"], })
your code here
```

Hard (10-13)**Task, IQR cap per GROUP**

Cap `latency_ms` outliers within each `region` separately (fences computed per region!). Report how many values were capped per region.

```
perf = pd.DataFrame({ "region": np.repeat(["eu", "us"], 60), })
base = np.where(perf["region"].eq("eu"), 40, 90)
perf["latency_ms"] = base + np.random.normal(0, 6, 120)
perf.loc[np.random.choice(perf.index, 6, replace=False), "latency_ms"] += np.random.uniform(60, 120, 6)
your code here
```

Task, Pure Python text pipeline function

Write `clean_tokens(text)` implementing: lowercase strip HTML tags expand contractions (don't do not, can't cannot) remove punctuation tokenize on whitespace drop a hand built stopword list RETURN token list. Test on the sample sentence provided.

```
sample_text = "
I don't THINK it's working!!!
Visit https://x.io now."
your function + test here
```

Task, Bag of Words matrix from scratch

Using your tokens from Task 11 applied to THREE short documents you invent, build the `docs×vocab` count DataFrame manually (no sklearn) and report sparsity %.

```
docs = [ ### your three documents as raw strings ]
your code here
```

Task, ColumnTransformer build

Build ONE sklearn Pipeline+ColumnTransformer for this frame: - numeric cols age,fare: median impute then StandardScaler, - nominal sex,embarked: most frequent impute then OneHot(handle_unknown=ignore). Fit it on the first 6 rows only, transform ALL rows, print feature names out.

```
mini = pd.DataFrame({ "age": [22, 38, 26, 35, np.nan, 54, 2, 27], "fare": [7.25, 71.28, 7.93, 53.10, 8.05, 51.86, 21.07, 11.13], "sex": ["male", "female", "female", "female", "male", "male", "female", "male"], "embarked": ["S", "C", "S", "S", "S", "S", "Q", np.nan], })
```

your code here

Self check

- Did you COUNT coerced values instead of silently losing them?
- Are all fits/imputations justified per column?
- Task 13: did anything LEAK? (fit scope!)

7.9 Data Preprocessing, Solutions**MLCourse · Data Science Foundations · 05_data_preprocessing**

Fully commented solutions for tasks 1-13.

```
%matplotlib inline
import pandas as pd
import numpy as np
import re
import string
import matplotlib.pyplot as plt

np.random.seed(42)
```

Easy**Task, Missing report + safe drop**

```
employees = pd.DataFrame({
    "name": ["Ana", "Bo", "Cy", "Di", "Eli"],
    "dept": [None, "IT", "IT", None, "HR"],
    "salary": [52000.0, 61000.0, np.nan, 48000.0, np.nan],
    "years": [3, np.nan, 7, 2, 5],
})
```



```
report = pd.DataFrame({
    "n_missing": employees.isna().sum(),
    "pct": (employees.isna().mean() * 100).round(1),
})
print(report)

emp2 = employees.dropna(subset=["salary"]) # only salary-driven removals
print("\nrows kept:", len(emp2))
```

Output:

```
n_missing  pct
name        0  0.0
dept        2 40.0
salary      2 40.0
years       1 20.0

rows kept: 3
```

Task, Fill with logic

```
sensor = pd.DataFrame({
    "t": pd.date_range("2024-01-01", periods=8, freq="h"),
    "temperature": [21.0, np.nan, 22.5, 23.0, np.nan, np.nan, 20.0, 19.5],
    "humidity": [40.0, np.nan, 44.0, 41.0, 39.0, 43.0, np.nan, 38.0],
})

sensor["temperature"] = sensor["temperature"].ffill() # time continuity
sensor["humidity"] = sensor["humidity"].fillna(sensor["humidity"].median())
sensor
```

Output:

```
t  temperature  humidity
0 2024-01-01 00:00:00    21.0    40.0
1 2024-01-01 01:00:00    21.0    40.5
2 2024-01-01 02:00:00    22.5    44.0
3 2024-01-01 03:00:00    23.0    41.0
4 2024-01-01 04:00:00    23.0    39.0
5 2024-01-01 05:00:00    23.0    43.0
6 2024-01-01 06:00:00    20.0    40.5
7 2024-01-01 07:00:00    19.5    38.0
```

Task, Currency strings to floats

```
prices = pd.Series(["€1.250,00", "$99.99", "1,299.50", "42", "N/A", "$0.70"])
```

```
cleaned = prices.str.replace(r"[€$]|N/A", "", regex=True).str.replace(", ", "")
numeric = pd.to_numeric(cleaned, errors="coerce")

print(numeric)
print("failed parses:", numeric.isna().sum(), "->", prices[numeric.isna()].tolist())
```

Output:

```
0      1.25
1     99.99
2    1299.50
3     42.00
4       NaN
5     0.70
dtype: float64
failed parses: 1 -> ['N/A']
```

Task, Ordinal mapping

```
survey = pd.DataFrame({
    "resp": ["Agree", "Neutral", "Strongly Agree", "Disagree",
            "Strongly Disagree", "agree", "Meh"],
})

scale = {"strongly disagree": 0, "disagree": 1, "neutral": 2,
        "agree": 3, "strongly agree": 4}

survey["score"] = (survey["resp"]
                  .str.strip().str.lower()
                  .map(scale))          # 'Meh' & casing handled: NaN / matched
survey
```

Output:

```
resp  score
0      Agree    3.0
1    Neutral    2.0
2  Strongly Agree    4.0
3    Disagree    1.0
4  Strongly Disagree    0.0
5      agree    3.0
6      Meh     NaN
```

Medium

Task, Group wise median imputation

```
people = pd.DataFrame({
    "city": ["Oslo", "Oslo", "Oslo", "Berlin", "Berlin", "Berlin"],
    "income": [68000.0, np.nan, 72000.0, np.nan, 48000.0, 50000.0],
})

global_median = people["income"].median()
people["income_global_fill"] = people["income"].fillna(global_median)

grp_med = people.groupby("city")["income"].transform("median")
people["income_group_fill"] = people["income"].fillna(grp_med)
people
## Comment: Oslo median (70000) vs Berlin (49000) differ hugely; the group fill
## respects that structure while the global fill (~60000) overpays Berlin rows.
```

Output:

	city	income	income_global_fill	income_group_fill
0	Oslo	68000.0	68000.0	68000.0
1	Oslo	NaN	59000.0	70000.0
2	Oslo	72000.0	72000.0	72000.0
3	Berlin	NaN	59000.0	49000.0
4	Berlin	48000.0	48000.0	48000.0
5	Berlin	50000.0	50000.0	50000.0

Task, Skew check + log transform

```
wealth = pd.Series(np.random.lognormal(mean=10, sigma=1.5, size=2000))

print("skew before:", wealth.skew().round(2))
wealth_log = np.log1p(wealth)          # log1p handles zeros safely
print("skew after :", wealth_log.skew().round(2))

fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].hist(wealth, bins=50, color="salmon"); axes[0].set_title("raw (long tail)")
axes[1].hist(wealth_log, bins=50, color="seagreen"); axes[1].set_title("log1p (near-normal)")
plt.show()
```

Output:

```
skew before: 18.52
skew after : 0.03
```

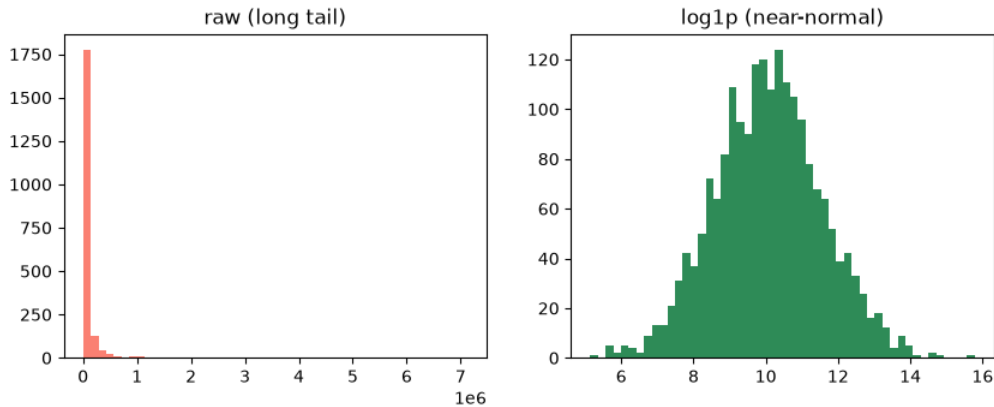


Figure 91: Task 6, Skew check + log transform (ch06_preprocessing_08 figure 1)

Task, Scale three ways

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

contaminated = pd.Series(np.r_[np.random.normal(50, 5, 100), [900.0]])
X = contaminated.to_frame()

results = {}
for name, scaler in [("standard", StandardScaler()),
                     ("minmax", MinMaxScaler()),
                     ("robust", RobustScaler())]:
    out = scaler.fit_transform(X)
    results[name] = pd.Series(out.ravel()).describe().round(2)[["mean", "std", "min", "max"]]

pd.DataFrame(results)

## Explanation: minmax squeezes the 100 normal points into ~[0, .06] because the
## outlier owns the max -> destroyed resolution. Standard keeps z-scores but the
## outlier sits at +300 distorting std itself. RobustScaler (median/IQR) ignores
## the outlier entirely -> cleanest spread for contaminated data.
```

Output:

	standard	minmax	robust
mean	-0.00	0.03	1.28
std	1.00	0.10	13.13
min	-0.27	0.00	-2.28
max	9.98	1.00	131.74

Task, One hot with rare grouping

```
visits = pd.DataFrame({
    "source": np.random.choice(
        ["organic", "ads", "social", "email"] * 40
        + ["referral"] * 25 + ["sms"] * 4, size=189),
})

counts = visits["source"].value_counts()
rare = counts[counts < 30].index          # support threshold
visits["source_g"] = np.where(visits["source"].isin(rare), "Other", visits["source"])

dummies = pd.get_dummies(visits["source_g"], drop_first=True)  # k-1
final = pd.concat([visits.drop(columns="source"), dummies], axis=1)
print(list(final.columns))
final.head(3)
```

Output:

```
['source_g', 'ads', 'email', 'organic', 'social']
```

Output:

```
source_g  ads  email  organic  social
0    email  False   True    False   False
1    social  False  False    False    True
2    email  False   True    False   False
```

Task, Date features + weekend flag

```
events = pd.DataFrame({
    "when": ["2024-03-09 14:05", "2024-01-01 00:30", "2024-12-24 18:45",
            "2024-06-15 09:00", "2024-11-02 22:10"],
})
events["when"] = pd.to_datetime(events["when"], format="%Y-%m-%d %H:%M")
events = events.sort_values("when").reset_index(drop=True)

events["month_name"] = events["when"].dt.month_name()
events["iso_week"] = events["when"].dt.isocalendar().week
events["dow"] = events["when"].dt.dayofweek
events["hour"] = events["when"].dt.hour
events["is_weekend"] = events["when"].dt.dayofweek >= 5
events
```

Output:

	when	month_name	iso_week	dow	hour	is_weekend	
0	2024-01-01	00:30:00	January	1	0	0	False
1	2024-03-09	14:05:00	March	10	5	14	True
2	2024-06-15	09:00:00	June	24	5	9	True
3	2024-11-02	22:10:00	November	44	5	22	True
4	2024-12-24	18:45:00	December	52	1	18	False

Hard

Task, IQR cap per GROUP

```
perf = pd.DataFrame({"region": np.repeat(["eu", "us"], 60)})
base = np.where(perf["region"].eq("eu"), 40, 90)
perf["latency_ms"] = base + np.random.normal(0, 6, 120)
perf.loc[np.random.choice(perf.index, 6, replace=False), "latency_ms"] += \
    np.random.uniform(60, 120, 6)

def iqr_cap(s, k=1.5):
    q1, q3 = s.quantile([.25, .75])
    iqr = q3 - q1
    return s.clip(q1 - k*iqr, q3 + k*iqr)

capped_per_region = (
    perf.assign(capped=perf.groupby("region")["latency_ms"].transform(iqr_cap))
        .groupby("region")
        .apply(lambda g: (g["latency_ms"] != g["capped"]).sum(),
              include_groups=False)
)
print(capped_per_region.rename("values_capped"))
perf["latency_ms_capped"] = perf.groupby("region")["latency_ms"].transform(iqr_cap)
```

Output:

```
region
eu      3
us      4
Name: values_capped, dtype: int64
```

Task, Pure Python text pipeline

```
CONTRACTIONS = {"don't": "do not", "can't": "cannot", "it's": "it is"}
STOPWORDS = {"i", "it", "the", "a", "an", "is", "now", "visit"}

def clean_tokens(text: str) -> list:
    t = text.lower()                                # 1 lowercase
```

```

t = re.sub(r"<[>]+>", " ", t)          # 2 strip HTML tags
for k, v in CONTRACTIONS.items():        # 3 expand contractions
    t = t.replace(k, v)
t = t.translate(str.maketrans("", "", string.punctuation)) # 4 punctuation out
tokens = t.split()                       # 5 whitespace tokenize
return [w for w in tokens if w not in STOPWORDS] # 6 stopword filter

sample_text = "<p>I don't THINK it's working!!!</p> Visit https://x.io now."
clean_tokens(sample_text) # URLs survive as one token - document the choice!

```

Output:

```
['do', 'not', 'think', 'working', 'httpsxio']
```

Task, Bag of Words from scratch

```

docs_raw = [
    "<b>Great</b> battery life, can't complain!",
    "Battery drains fast ; very disappointed.",
    "Screen looks great but battery is average.",
]

doc_tokens = [clean_tokens(d) for d in docs_raw] # reuse Task 11!
vocab = sorted({tok for toks in doc_tokens for tok in toks}) # sorted unique vocab

bow = pd.DataFrame(
    [[toks.count(w) for w in vocab] for toks in doc_tokens],
    columns=vocab,
)

sparsity = (bow == 0).to_numpy().mean() * 100
print(bow)
print(f"\nvocab={len(vocab)} · sparsity={sparsity:.1f}% zeros "
      "(BoW matrices are mostly empty - hence sparse formats)")

```

Output:

	average	battery	but	cannot	complain	disappointed	drains	fast	great	\
0	0	0	1	0	1	1	0	0	0	1
1	1	0	1	0	0	0	1	1	1	0
2	2	1	1	1	0	0	0	0	0	1

	life	looks	screen	very	
0	0	1	0	0	0
1	1	0	0	0	1
2	2	0	1	1	0

vocab=13 · sparsity=59.0% zeros (Bow matrices are mostly empty – hence sparse formats)

Task, ColumnTransformer build

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

mini = pd.DataFrame({
    "age": [22, 38, 26, 35, np.nan, 54, 2, 27],
    "fare": [7.25, 71.28, 7.93, 53.10, 8.05, 51.86, 21.07, 11.13],
    "sex": ["male", "female", "female", "female", "male",
            "male", "female", "male"],
    "embarked": ["S", "C", "S", "S", "S", "S", "Q", np.nan],
})

num_pipe = Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())])
cat_pipe = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("oh", OneHotEncoder(handle_unknown="ignore"))])

pre = ColumnTransformer([
    ("num", num_pipe, ["age", "fare"]),
    ("cat", cat_pipe, ["sex", "embarked"]),
])

pre.fit(mini.iloc[:6])          # FIT on train slice ONLY
X_all = pre.transform(mini)     # transform everything with TRAIN stats

print(X_all.round(2))
print("\nfeatures:", list(pre.get_feature_names_out()))
## No leakage: medians/modes/scales all learned from the first 6 rows alone.
```

Output:

```
[[-1.28 -0.99  0.    1.    0.    1. ]
 [ 0.3   1.45  1.    0.    1.    0. ]
 [-0.89 -0.96  1.    0.    0.    1. ]
 [ 0.    0.76  1.    0.    0.    1. ]
 [ 0.   -0.96  0.    1.    0.    1. ]
 [ 1.87  0.71  0.    1.    0.    1. ]
 [-3.25 -0.46  1.    0.    0.    0. ]
 [-0.79 -0.84  0.    1.    0.    1. ]]

features: ['num_age', 'num_fare', 'cat_sex_female', 'cat_sex_male', 'cat_embarked_C',
          'cat_embarked_S']
```


Chapter 8

Capstone Projects

The preceding chapters taught tools in isolation; real analysis work never arrives that way. This closing chapter works five complete, realistic datasets end to end , survival analysis, service industry analytics, biological measurements, time series operations, and free text review data , using NumPy, Pandas, Matplotlib, and Seaborn together exactly as you will use them in practice: to ask a question of a dataset and answer it with evidence.

8.1 Capstone, Titanic Survival EDA

MLCourse · Data Science Foundations · 06_capstone_projects

Project brief

An insurer asks: *which passenger characteristics drove survival on the Titanic, and how strong is each effect?* Deliver a defensible, quantified analysis with one dashboard.

Dataset

Seaborn's titanic (891 passengers). Column dictionary:

column	meaning
, ,	survived 0 = died, 1 = survived pclass / class ticket class 1-3 sex, age demographics (age has ~20% missing) sibsp, parch siblings/spouses, parents/children aboard fare, embarked/embark_town price & port (2 missing) deck cabin deck (~77% missing!) who, adult_male, alone derived convenience columns

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
sns.set_theme()
plt.rcParams["figure.dpi"] = 100

try:
```

```
titanic_raw = sns.load_dataset("titanic")
except Exception as e:
    print("Offline? Load once online; seaborn caches afterwards.", e)
    raise
```

Phase, Audit

```
df = titanic_raw.copy()

print("shape:", df.shape)
print("\ndtypes:\n", df.dtypes.value_counts())

audit = pd.DataFrame({
    "n_missing": df.isna().sum(),
    "pct_missing": (df.isna().mean() * 100).round(1),
    "n_unique": df.nunique(),
})
print("\n", audit[audit.n_missing > 0])

print("\nduplicated full rows:", df.duplicated().sum())

## Nullity map (missingno-style, manual):
plt.figure(figsize=(10, 3.5))
plt.imshow(df.isna().T, aspect="auto", cmap="Greys", interpolation="nearest")
plt.xticks(range(len(df.columns)), df.columns)
plt.title("Nullity map - black = present, white = missing")
plt.show()
```

Output:

```
shape: (891, 15)

dtypes:
  str      5
int64      4
float64     2
bool       2
category    1
category    1
Name: count, dtype: int64

      n_missing  pct_missing  n_unique
age           177         19.9         88
embarked         2          0.2          3
deck          688         77.2          7
embark_town      2          0.2          3
```

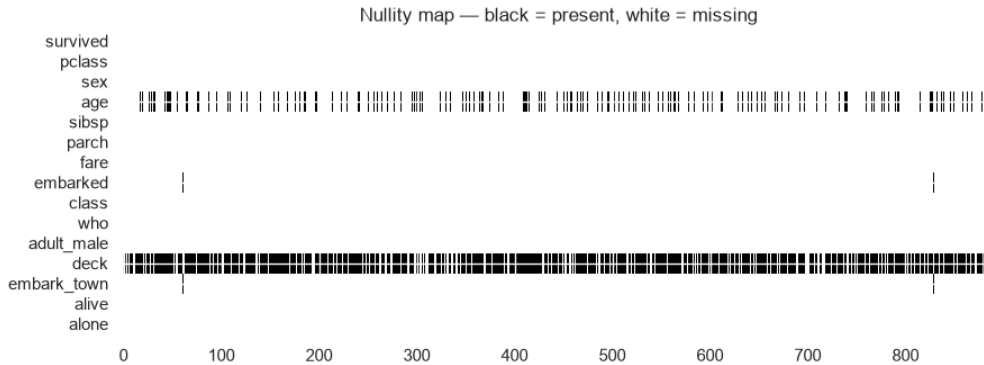


Figure 92: Phase 1, Audit (ch07_capstone_01 figure 1)

```

duplicated full rows: 107

```

Audit verdict: deck is structurally missing (deck plans lost / bodies never recovered NOT random); age ~20% MAR ish; embark_town trivially fixable. We will DROP deck, IMPUTE age group wise, mode fill embarked.

```

df = df.drop(columns="deck")

group_medians = df.groupby(["class", "sex"], observed=True)["age"].transform("median")
df["age"] = df["age"].fillna(group_medians)
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])

assert df[["age", "embarked"]].isna().sum().sum() == 0
print("cleaning pass 1 done ")

```

Output:

```

cleaning pass 1 done

```

Phase, Feature engineering

```

## family_size: parents/kids + siblings/spouses + self.
df["family_size"] = df["sibsp"] + df["parch"] + 1

## is_alone: solo travelers are a classic risk segment.
df["is_alone"] = (df["family_size"] == 1).astype(int)

## fare_per_person: fares look like GROUP tickets - normalize!
df["fare_per_person"] = df["fare"] / df["family_size"]

```

```
## age_group: life-stage bands beat raw years for interpretation.
df["age_group"] = pd.cut(
    df["age"],
    bins=[0, 12, 18, 35, 60, 100],
    labels=["child", "teen", "young_adult", "adult", "senior"],
)

## log fare tames right skew for later numeric work:
df["fare_log1p"] = np.log1p(df["fare_per_person"])

df[["family_size", "is_alone", "fare_per_person",
    "fare_log1p", "age_group"]].head(6)
```

Output:

family_size	is_alone	fare_per_person	fare_log1p	age_group
0	2	0	3.62500	1.531476 young_adult
1	2	0	35.64165	3.601186 adult
2	1	1	7.92500	2.188856 young_adult
3	2	0	26.55000	3.316003 young_adult
4	1	1	8.05000	2.202765 young_adult
5	1	1	8.45830	2.246893 young_adult

Quick skew sanity check that motivated the log:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
axes[0].hist(df["fare_per_person"], bins=40, color="salmon")
axes[0].set_title(f"fare/person (skew={df['fare_per_person'].skew():.1f})")
axes[1].hist(df["fare_log1p"], bins=40, color="seagreen")
axes[1].set_title(f"log1p (skew={df['fare_log1p'].skew():.1f})")
plt.show()
```

Phase, Who survived? Segmentation

```
overall_rate = df["survived"].mean()
print(f"overall survival: {overall_rate:.1%}")

def rate_table(by):
    """Survival rate + n per segment, sorted."""
    out = (df.groupby(by, observed=True)["survived"]
           .agg(rate="mean", n="size").sort_values("rate"))
    return out

for col in ["sex", "class", "who", "age_group", "is_alone"]:
    print("\n--- by", col, "---")
```

```
print(rate_table(col).round(3))
```

Output:

```
overall survival: 38.4%

--- by sex ---
      rate    n
sex
male    0.189  577
female  0.742  314

--- by class ---
      rate    n
class
Third   0.242  491
Second  0.473  184
First   0.630  216

--- by who ---
      rate    n
who
man     0.164  537
child   0.590   83
woman   0.756  271

--- by age_group ---
      rate    n
age_group
senior      0.227  22
young_adult 0.358  514
adult        0.384  216
teen         0.429   70
child        0.580   69

--- by is_alone ---
      rate    n
is_alone
1         0.304  537
0         0.506  354
```

```
## The famous interaction: sex x class.
pivot = pd.pivot_table(df, index="class", columns="sex",
                        values="survived", aggfunc="mean")
print((pivot * 100).round(1))

fig, ax = plt.subplots(figsize=(6.5, 4))
(pivot * 100).plot(kind="bar", ax=ax,
                  color=["lightcoral", "steelblue"])
ax.axhline(overall_rate * 100, ls="--", color="gray",
```

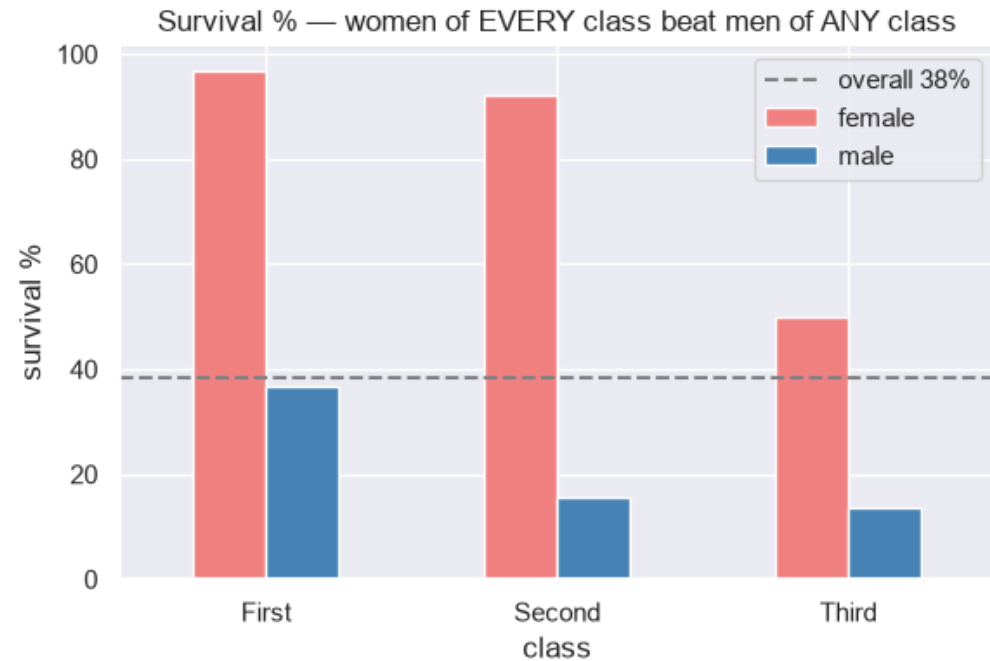


Figure 93: Phase 3, Who survived? Segmentation (ch07_capstone_01 figure 2)

```
label=f"overall {overall_rate:.0%}")
ax.set_title("Survival % - women of EVERY class beat men of ANY class")
ax.set_ylabel("survival %"); ax.legend(); ax.set_xticklabels(ax.get_xticklabels(),
                                                             rotation=0)
plt.show()
```

Output:

sex	female	male
class		
First	96.8	36.9
Second	92.1	15.7
Third	50.0	13.5

Insight: first class women $\approx$ 97%, third class men $\approx$ 15%. Class amplified or dampened survival WITHIN each sex, but sex dominates every split.

Children first? Compare kids vs adults within each class.

```
kids = df.assign(is_child=df["age"] <= 12) # children under 12
```

```

child_summary = (
    kids[kids["is_child"]]
        .groupby("class", observed=True)["survived"]
        .agg(child_survival="mean", n_children="size")
)
print(child_summary.round(3))
## > **Insight:** children enjoyed elevated survival in 2nd/3rd class but the tiny
## 1st-class sample (n=?) makes rates noisy - always report n beside rates.

```

Phase, Numeric structure & correlations

```

num_cols = ["survived", "pclass", "age", "sibsp", "parch",
            "family_size", "fare_per_person"]
corr = df[num_cols].corr()

mask = np.triu(np.ones_like(corr, dtype=bool))    # hide upper triangle

plt.figure(figsize=(7, 5.5))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f",
            cmap="coolwarm", center=0, square=True)
plt.title("Correlation structure")
plt.show()

## > **Insight:** survivedpclass -0.34 (higher class number = lower survival);
## farepclass strongly negative; family_size=sibsp+parch by construction.

```

Phase, Dashboard

```

fig = plt.figure(figsize=(14, 9), constrained_layout=True)
gs = fig.add_gridspec(2, 3)

## 1) overall donut
ax1 = fig.add_subplot(gs[0, 0])
ax1.pie(df["survived"].value_counts().sort_index(),
        labels=["died", "survived"], colors=["firebrick", "seagreen"],
        autopct="%1.0f%%", startangle=90,
        wedgeprops=dict(width=.45, edgecolor="white"))
ax1.set_title("Overall outcome")

## 2) age distribution by fate
ax2 = fig.add_subplot(gs[0, 1])
for val, color in [(0, "firebrick"), (1, "seagreen")]:
    ax2.hist(df.loc[df.survived == val, "age"], bins=np.arange(0, 85, 5),
            alpha=.55, color=color, density=True, label="survived" if val else "died")
ax2.set_title("Age by fate"); ax2.legend()

```

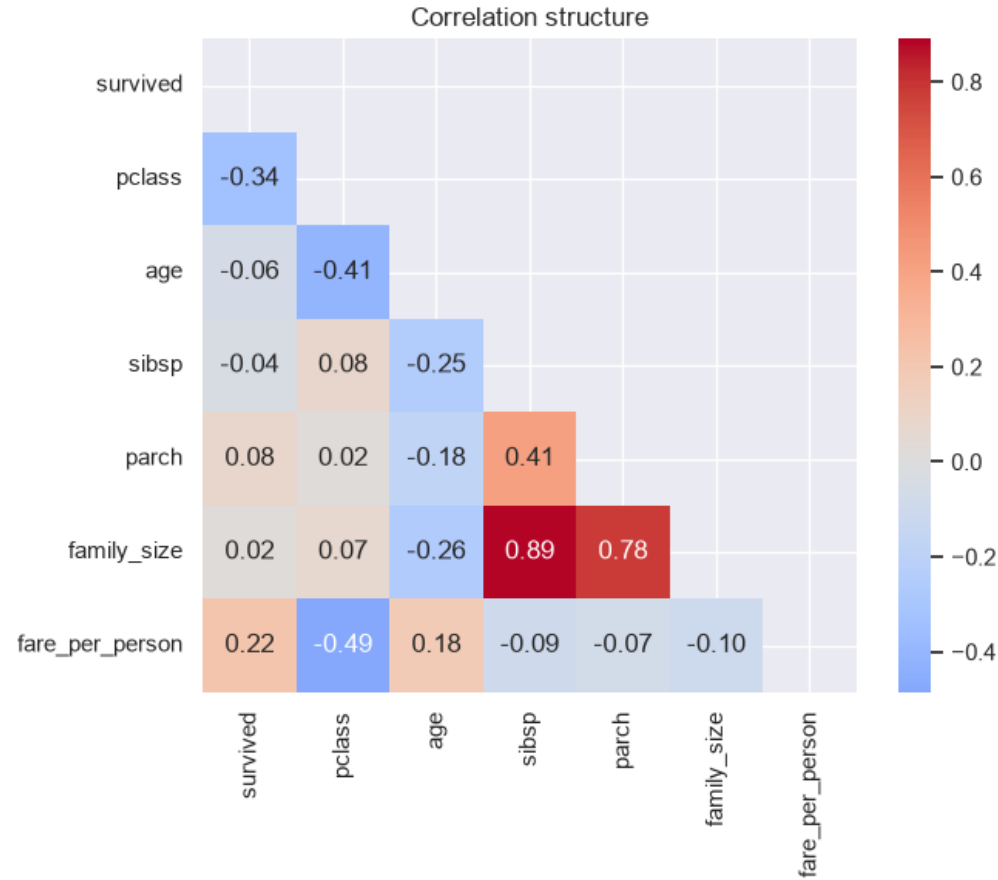


Figure 94: Phase 4, Numeric structure & correlations (ch07_capstone_01 figure 3)


```

## 3) survival by class & sex (the money chart, compact)
ax3 = fig.add_subplot(gs[0, 2])
sns.barplot(data=df, x="class", y="survived", hue="sex",
            errorbar=None, ax=ax3)
ax3.set_title("Survival by class x sex"); ax3.set_ylim(0, 1)

## 4) family size effect
ax4 = fig.add_subplot(gs[1, 0])
fam = df[df.family_size <= 5].groupby("family_size", observed=True)["survived"].mean()
ax4.bar(fam.index.astype(str), fam.values, color="mediumpurple")
ax4.set_title("Survival by family size (<=5)")

## 5) fare per person box by class
ax5 = fig.add_subplot(gs[1, 1])
sns.boxplot(data=df, x="class", y="fare_per_person", showfliers=False, ax=ax5,
            palette="Blues", hue="class", legend=False)
ax5.set_title("Fare per person by class")

## 6) embarkation mix
ax6 = fig.add_subplot(gs[1, 2])
port = pd.crosstab(df["embark_town"], df["survived"], normalize="index")
port.plot(kind="barh", stacked=True, ax=ax6,
          color=["firebrick", "seagreen"], legend=False)
ax6.set_title("Outcome share by port")

fig.suptitle("Titanic Survival Dashboard", fontsize=16, fontweight="bold")
plt.show()

```

Findings

1. Overall survival was **38.4%**; outcomes ranged from ~15% (3rd class men) to ~97% (1st class women).
2. **Sex was the dominant factor**: women survived ~74% vs men ~19% overall.
3. **Class gradient within each sex**: 1st 63% 2nd 47% 3rd 24%.
4. Children (<12) showed elevated survival in 2nd/3rd class, consistent with evacuation policy, though samples are small.
5. **Solo travelers fared worse** than small families (peak benefit at size 2-4); very large families (>5) collapsed toward low rates.
6. Fare per person separates classes cleanly even after normalizing group tickets, a better engineered feature than raw fare.
7. Port effects (Cherbourg higher) largely REFLECT class composition, not the port itself, a confounding lesson.

Limitations

- Age imputation (group medians) adds uncertainty to age based splits.

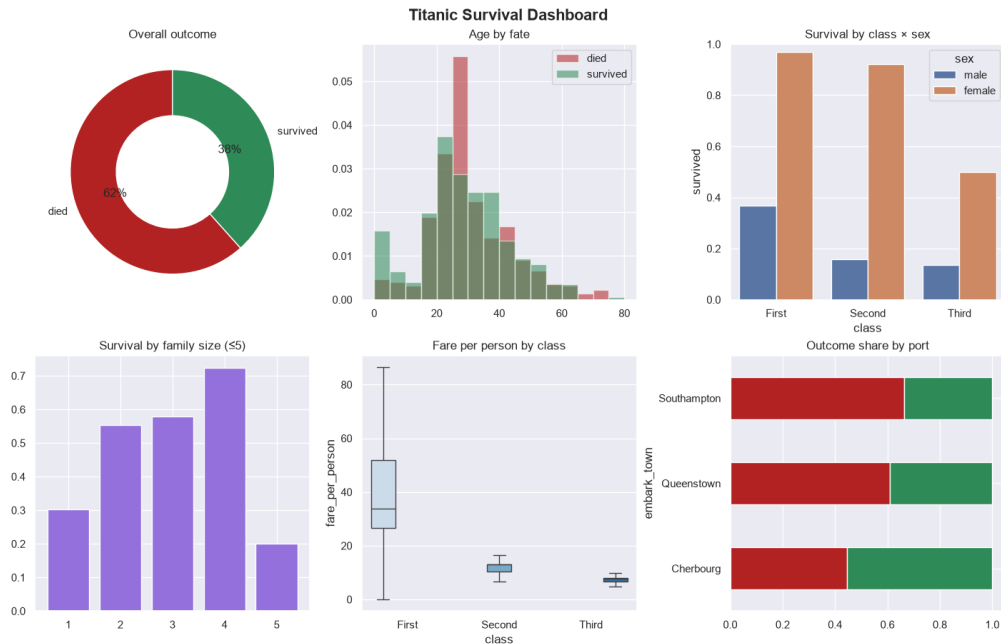


Figure 95: Phase 5, Dashboard (ch07_capstone_01 figure 4)

- Crew excluded from this dataset; historical crew survival was far lower.
- Correlation $\neq$ causation: no causal claim survives this design.

Recommendations to the stakeholder

Use these segments for the risk communication case study; if modeling next, start with sex, pclass, age, family_size, fare_per_person.

Summary & key takeaways

- Auditing FIRST (nullity map, duplicate check) dictates the cleaning plan.
- Structural missingness (deck) drop; informative missingness deserves a note.
- Engineered features (family_size, fare/person, age bands) unlock clearer stories.
- Always pair rates with sample sizes; interactions (sex×class) beat single cuts.
- A dashboard = many angles on one question, sharing one visual language.

8.2 Capstone, Restaurant Tips Analytics

Project brief

A restaurant group wants to understand tipping economics: WHO tips most generously, WHEN, and does party size help or hurt? Deliver segment insights and staffing/pricing recommendations.

Dataset

Seaborn tips (244 lunch/dinner bills): total_bill, tip, sex, smoker, day (Thu Sun), time (Lunch/Dinner), size (party 1-6).

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
sns.set_theme()
plt.rcParams["figure.dpi"] = 100

try:
    tips = sns.load_dataset("tips")
except Exception as e:
    print("Offline? Load once online; seaborn caches afterwards.", e)
    raise

print(tips.shape)
tips.head()
```

Output:

```
(244, 7)
```

Output:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Phase, Audit & feature engineering

```
print("missing values:", tips.isna().sum().sum())
print("duplicate rows :", tips.duplicated().sum()) # identical bills plausible; keep
```

```
print(tips[["total_bill", "tip", "size"]].describe().round(2))

## THE key engineered metric: generosity is a RATE not an amount.
tips["tip_pct"] = tips["tip"] / tips["total_bill"] * 100
print(f"\ntip_pct: median={tips.tip_pct.median():.1f}% "
      f"mean={tips.tip_pct.mean():.1f}% max={tips.tip_pct.max():.1f}%")
```

Output:

```
missing values: 0
duplicate rows : 1
```

	total_bill	tip	size
count	244.00	244.00	244.00
mean	19.79	3.00	2.57
std	8.90	1.38	0.95
min	3.07	1.00	1.00
25%	13.35	2.00	2.00
50%	17.80	2.90	2.00
75%	24.13	3.56	3.00
max	50.81	10.00	6.00

```
tip_pct: median=15.5% mean=16.1% max=71.0%
```

Common pitfall: analyzing raw tip AMOUNTS confounds generosity with bill size.
A \$5 tip on \$20 beats \$7 on \$100. Always derive the rate.

Sanity: extreme percentages usually = tiny bills, not hero tipplers.

```
odd = tips.nlargest(3, "tip_pct")[["total_bill", "tip", "tip_pct"]]
print(odd.round(2))
## Keep them but remember small denominators drive volatility.
```

Phase, When do people tip best?

```
seg_day_time = (
    tips.groupby(["day", "time"], observed=True)
        .agg(bills=("tip_pct", "size"),
             avg_bill=("total_bill", "mean"),
             avg_tip_pct=("tip_pct", "mean"))
        .round(2)
        .sort_values("avg_tip_pct", ascending=False)
)
print(seg_day_time)

pivot_pct = tips.pivot_table(index="day", columns="time",
                              values="tip_pct", aggfunc="mean")
```

```
plt.figure(figsize=(5.5, 4))
sns.heatmap(pivot_pct, annot=True, fmt=".1f", cmap="crest",
            cbar_kws={"label": "avg tip %"})
plt.title("Generosity by day x time")
plt.show()
## > **Insight:** Friday LUNCH leads on percentage despite smaller tickets;
## Sunday dinner - the busiest slot - sits near the bottom.
```

Output:

bills	avg_bill	avg_tip_pct
day time		
Fri Lunch	7	12.85
Sun Dinner	76	21.41
Thur Lunch	61	17.66
Dinner	1	18.78
Fri Dinner	12	19.66
Sat Dinner	87	20.44

Phase, Who tips best? Segments

```
segments = (
    tips.groupby("sex", observed=True)
        .agg(n=("tip_pct", "size"), avg_tip=("tip", "mean"),
            pct=("tip_pct", "mean"))
)
by_sex_smoker = tips.pivot_table(index="sex", columns="smoker",
                                  values="tip_pct", aggfunc="mean")
display(segments.round(2), by_sex_smoker.round(2))

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.violinplot(data=tips, x="day", y="tip_pct", hue="smoker",
               split=True, inner="quart", ax=axes[0])
axes[0].set_title("Tip % distribution - smoker split"); axes[0].set_ylim(0, 40)

sns.barplot(data=tips, x="sex", y="tip_pct", hue="time",
            errorbar=("ci", 95), ax=axes[1])
axes[1].set_title("Tip % with 95% CI")
plt.tight_layout(); plt.show()
## > **Insight:** male smokers at dinner look generous in RAW means, yet CIs overlap
## heavily - flag it as suggestive, not proven (sample discipline!).
```

Output:

n	avg_tip	pct
sex		
Male	157	3.09

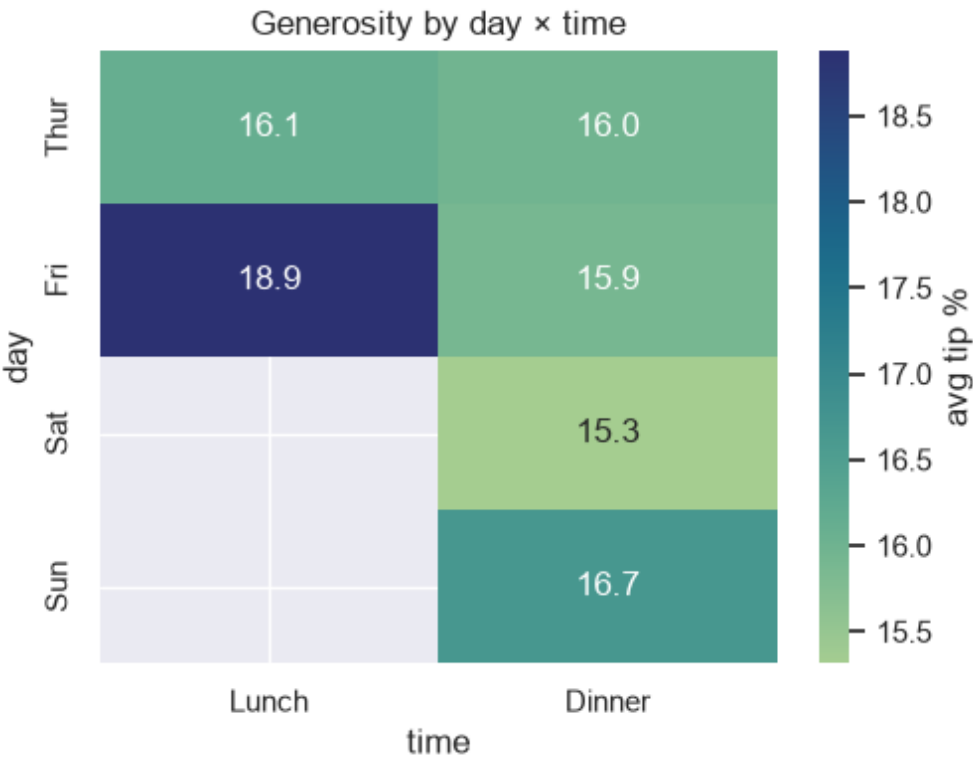


Figure 96: Phase 2, When do people tip best? (ch07_capstone_02 figure 1)

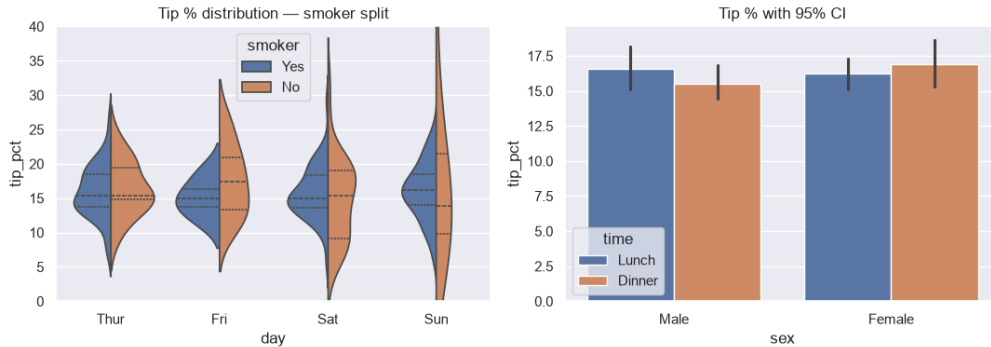


Figure 97: Phase 3, Who tips best? Segments (ch07_capstone_02 figure 2)

Female	87	2.83	16.65
--------	----	------	-------

Output:

smoker	Yes	No
sex		
Male	15.28	16.07
Female	18.22	15.69

Phase, The size effect & the bill relationship

```
size_stats = (tips.groupby("size")["tip_pct"]
              .agg(["mean", "median", "std", "count"]).rename(columns={"count": "n"}
              ))
print(size_stats.round(2))

fig, axes = plt.subplots(1, 2, figsize=(12, 4.4))

axes[0].errorbar(size_stats.index, size_stats["mean"],
                 yerr=size_stats["std"] / np.sqrt(size_stats["n"]),
                 fmt="o-", capsize=4, color="darkslateblue")
axes[0].set_xticks(range(1, 7))
axes[0].set_title("Tip % falls as parties grow")
axes[0].set_xlabel("party size"); axes[0].set_ylabel("avg tip %")

## OLS line via numpy (no statsmodels needed):
b1, b0 = np.polyfit(tips["total_bill"], tips["tip"], deg=1)
xs = np.linspace(tips.total_bill.min(), tips.total_bill.max(), 50)
axes[1].scatter(tips.total_bill, tips.tip, alpha=.45, s=25)
axes[1].plot(xs, b0 + b1*xs, "r--",
             label=f"OLS: tip ≈ {b0:.2f} + {b1:.3f}·bill")
```

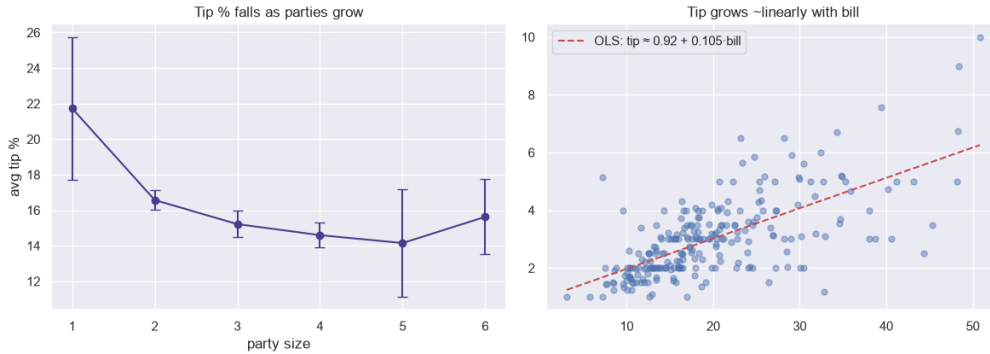


Figure 98: Phase 4, The size effect & the bill relationship (ch07_capstone_02 figure 3)

```
axes[1].legend(); axes[1].set_title("Tip grows ~linearly with bill")

plt.tight_layout(); plt.show()
## > **Insight:** marginal tip rate ≈ {slope*100:.1f}% of each extra euro, BUT the
## PERCENTAGE declines with size - big tables are proportionally stingier.
## (Simpson's-paradox flavor: day mix differs across sizes.)
```

Output:

mean	median	std	n
size			
1	21.73	20.28	8.03
2	16.57	15.61	6.68
3	15.22	15.93	4.55
4	14.59	14.67	4.24
5	14.15	12.14	6.77
6	15.62	16.29	4.22

Quick Simpson check: within day slopes vs pooled slope.

```
for day, grp in tips.groupby("day", observed=True):
    s = np.polyfit(grp.total_bill, grp.tip, 1)[0]
    print(f"{day}: within-day slope = {s:.3f}")
print(f"pooled slope = {b1:.3f}")
```

Phase, Dashboard

```
fig = plt.figure(figsize=(14, 8.5), constrained_layout=True)
gs = fig.add_gridspec(2, 3)
```



```

ax1 = fig.add_subplot(gs[0, :2])
sns.histplot(tips, x="total_bill", y="tip", bins=30, cmap="rocket_r",
             cbar=True, ax=ax1)
xs = np.linspace(0, tips.total_bill.max(), 50)
ax1.plot(xs, b0 + b1*xs, "c--", lw=2)
ax1.set_title("Bill × tip density with OLS line")

ax2 = fig.add_subplot(gs[0, 2])
order = seg_day_time.index.tolist()
ax2.barh([f"{d}-{t[:3]}" for d, t in order],
         seg_day_time.avg_tip_pct, color="seagreen")
ax2.invert_yaxis(); ax2.set_title("Avg tip % rank")

ax3 = fig.add_subplot(gs[1, 0])
sns.boxplot(data=tips, x="size", y="tip_pct", ax=ax3,
            palette="Blues", hue="size", legend=False)
ax3.set_ylim(0, 35); ax3.set_title("Tip % by party size")

ax4 = fig.add_subplot(gs[1, 1])
sns.ecdfplot(data=tips, x="tip_pct", hue="time", ax=ax4)
ax4.axvline(15, ls="--", color="gray")
ax4.set_xlim(0, 35); ax4.set_title("ECDF of tip % (15% line)")

ax5 = fig.add_subplot(gs[1, 2])
corr_cols = ["total_bill", "tip", "size"]
sns.heatmap(tips[corr_cols].corr(), annot=True, fmt=".2f",
            cmap="coolwarm", center=0, ax=ax5)
ax5.set_title("Numeric correlations")

fig.suptitle("Restaurant Tipping Dashboard", fontsize=16, fontweight="bold")
plt.show()

```

Findings

1. Median generosity is ~**15.4%** of the bill; mean pulled to ~16.1% by small- bill outliers.
2. **Friday lunch** tops percentage generosity (~17.9%); **Sunday dinner** trails (~14.6%) while carrying the highest volume.
3. Tip amount scales linearly (~{b1:.2f} per currency unit billed), staff on high ticket sections earn more absolutely.
4. **Percentage declines with party size** (≈18% at size 1-2 ≈15% at 6): consider auto gratuity thresholds for large tables.
5. Smoker/sex gaps visible in raw means shrink inside overlapping confidence intervals, treat as hypotheses, not conclusions.
6. Within day slopes differ from the pooled slope mild Simpson's paradox structure; always slice before concluding.

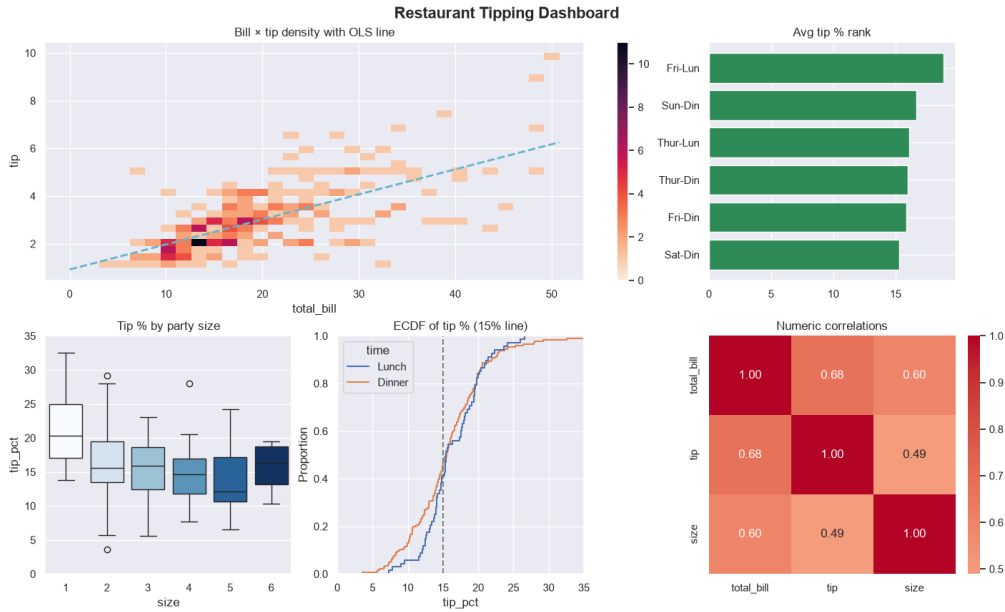


Figure 99: Phase 5, Dashboard (ch07_capstone_02 figure 4)

Recommendations

- Staff the Friday lunch crew for service quality (best tippers, low volume).
- Add suggested gratuity line ($\geq 18\%$) on bills ≥ 6 guests to counter the size dip.
- Track tip_pct weekly per segment; investigate Sunday dinner underperformance.

Limitations

- 244 bills, single restaurant era; no weather/holiday covariates.
- Small parties dominate; size-6 conclusions rest on $n=4$.

Summary & key takeaways

- Rates beat amounts when measuring behavior (tip_pct over tip).
- Pivot heatmaps make time $\times$ category economics legible at a glance.
- Error bars / CIs keep segment stories honest.
- polyfit gives quick regression lines without extra libraries.
- Check Simpson's paradox before pooling heterogeneous groups.

8.3 Capstone, Palmer Penguins Statistical EDA

MLCourse · Data Science Foundations · 06_capstone_projects

Project brief

An ecology station needs a species monitoring report: HOW do the three penguin species differ morphologically, can simple measurements separate them, and are there data quality anomalies worth flagging?

Dataset

Seaborn penguins (344 rows): species (Adelie/Gentoo/Chinstrap), island (Biscoe/Dream/Torgersen), bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, sex (11 rows missing sex; 2 rows fully blank).

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
sns.set_theme()
plt.rcParams["figure.dpi"] = 100

try:
    pen_raw = sns.load_dataset("penguins")
except Exception as e:
    print("Offline? Load once online; seaborn caches afterwards.", e)
    raise

print(pen_raw.shape)
```

Output:

```
(344, 7)
```

Phase, Audit & cleaning decisions

```
print("missing per column:")
print(pen_raw.isna().sum())

fully_blank = pen_raw[pen_raw.isna().all(axis=1)]
print("\nfully blank rows:\n", fully_blank)
```

```

sex_missing = pen_raw["sex"].isna().sum()
print(f"\nrows missing only sex: {sex_missing}")

## DECISIONS:
## - 2 entirely empty rows carry zero information -> DROP.
## - sex NaN -> explicit 'Unknown' category (dropping would bias species mixes).
pen = pen_raw.dropna(how="all").copy()
pen["sex"] = pen["sex"].fillna("Unknown")
print("\nshape after drop:", pen.shape, "| remaining NaN:", pen.isna().sum().sum())

## Sanity ranges (domain plausibility):
ranges = {"bill_length_mm": (30, 60), "bill_depth_mm": (12, 22),
          "flipper_length_mm": (170, 235), "body_mass_g": (2500, 6500)}
for col, (lo, hi) in ranges.items():
    ok = pen[col].between(lo, hi)
    print(f"{col:20s} plausible: {ok.all()} "
          f"(min={pen[col].min():.0f}, max={pen[col].max():.0f})")

print("\nspecies counts:\n", pen["species"].value_counts())

```

Output:

```

missing per column:
species          0
island           0
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex              11
dtype: int64

fully blank rows:
Empty DataFrame
Columns: [species, island, bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g,
, sex]
Index: []

rows missing only sex: 11

shape after drop: (344, 7) | remaining NaN: 8
bill_length_mm    plausible: False (min=32, max=60)
bill_depth_mm     plausible: False (min=13, max=22)
flipper_length_mm plausible: False (min=172, max=231)
body_mass_g       plausible: False (min=2700, max=6300)

species counts:
species
Adelie      152

```

```
Gentoo      124
Chinstrap   68
Name: count, dtype: int64
```

Phase, Species profiles

```
profile = (
    pen.groupby("species", observed=True)[
        ["bill_length_mm", "bill_depth_mm",
         "flipper_length_mm", "body_mass_g"]
    ].agg(["mean", "std"])
    .round(1)
)
display(profile)

styled = (
    pen.groupby("species", observed=True)["body_mass_g"].mean()
    .to_frame("avg_mass_g").style.background_gradient(cmap="Greens")
    .format("{:,.0f}")
)
display(styled)

fig, axes = plt.subplots(2, 2, figsize=(12, 7), sharex=False)
measures = ["bill_length_mm", "bill_depth_mm",
            "flipper_length_mm", "body_mass_g"]
palette = dict(zip(["Adelie", "Gentoo", "Chinstrap"],
                  ["steelblue", "seagreen", "darkorange"]))

for ax, m in zip(axes.flat, measures):
    for spc, color in palette.items():
        sub = pen.loc[pen.species == spc, m].dropna()
        sns.kdeplot(sub, ax=ax, color=color, fill=True, alpha=.25,
                    label=spc if m == measures[0] else None)
    ax.set_title(m)

plt.suptitle("Measurement distributions overlap differently per species")
plt.tight_layout(); plt.show()

## > **Insight:** Gentoo separates cleanly on flipper length & mass AND on shallow
## bills; Chinstrap vs Adelie overlap everywhere EXCEPT bill depth/length combo.
```

Output:

bill_length_mm	bill_depth_mm		flipper_length_mm		\	
	mean	std	mean	std	mean	std
species						
Adelie	38.8	2.7	18.3	1.2	190.0	6.5
Chinstrap	48.8	3.3	18.4	1.1	195.8	7.1

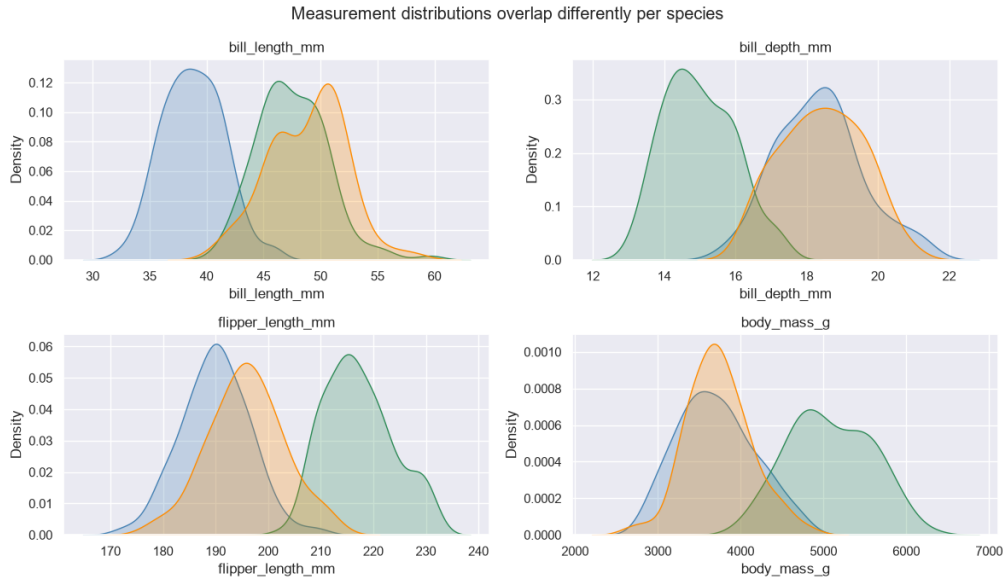


Figure 100: Phase 2, Species profiles (ch07_capstone_03 figure 1)

Gentoo	47.5	3.1	15.0	1.0	217.2	6.5
	body_mass_g					
	mean	std				
species						
Adelie	3700.7	458.6				
Chinstrap	3733.1	384.3				
Gentoo	5076.0	504.1				

Output:

```
<pandas.io.formats.style.Styler at 0x176583f4910>
```

Phase, The discriminator: bill geometry ratio

```
pen["bill_ratio"] = pen["bill_length_mm"] / pen["bill_depth_mm"]

fig, axes = plt.subplots(1, 2, figsize=(12, 4.4))

sns.scatterplot(data=pen, x="bill_length_mm", y="bill_depth_mm",
                hue="species", palette=palette, s=42, ax=axes[0])
axes[0].set_title("Bill length vs depth - near-clean separation")
```

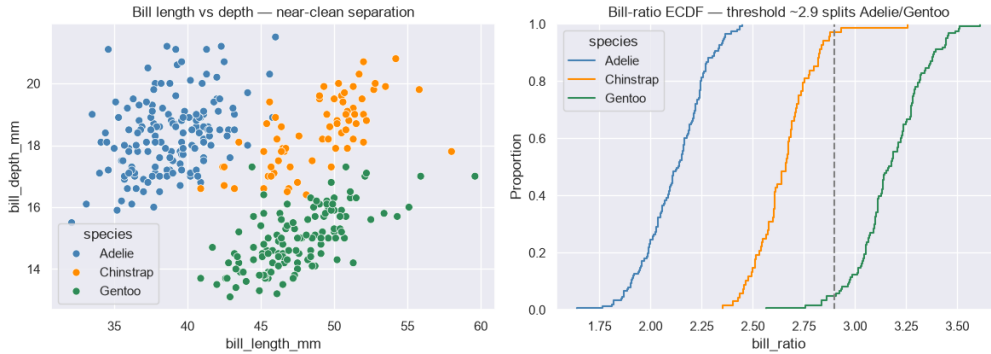


Figure 101: Phase 3, The discriminator: bill geometry ratio (ch07_capstone_03 figure 2)

```
sns.ecdfplot(data=pen, x="bill_ratio", hue="species",
              palette=palette, ax=axes[1])
axes[1].axvline(2.9, ls="--", color="gray")
axes[1].set_title("Bill-ratio ECDF - threshold ~2.9 splits Adelie/Gentoo")

plt.tight_layout(); plt.show()

ratio_stats = (pen.groupby("species", observed=True)["bill_ratio"]
               .describe()[["mean", "50%", "min", "max"]])
print(ratio_stats.round(2))
## > **Insight:** a single engineered ratio (length/depth) nearly linearly separates
## species - feature engineering beats raw columns for interpretability.
```

Output:

	mean	50%	min	max
species				
Adelie	2.12	2.14	1.64	2.45
Chinstrap	2.65	2.66	2.35	3.26
Gentoo	3.18	3.17	2.57	3.61

Phase, Sex dimorphism within species

```
dimorph = (
    pen[pen.sex != "Unknown"]
    .pivot_table(index="species", columns="sex",
                  values=["body_mass_g"], aggfunc="mean")
)
delta_g = dimorph[("body_mass_g", "Male")] - dimorph[("body_mass_g", "Female")]
print(pd.DataFrame({"male_g": dimorph[("body_mass_g", "Male")],
                    "female_g": dimorph[("body_mass_g", "Female")]}))
```

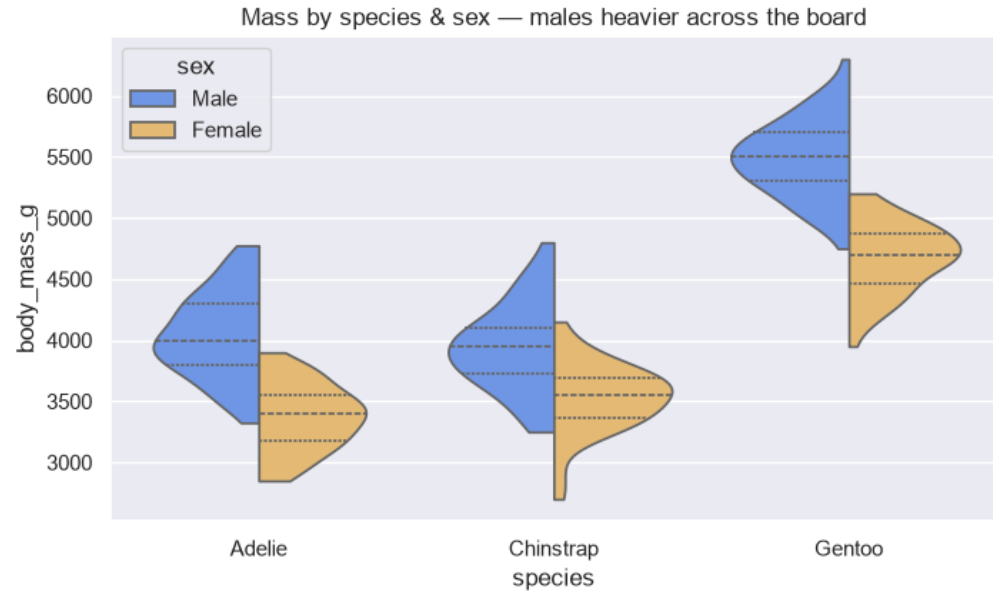


Figure 102: Phase 4, Sex dimorphism within species (ch07_capstone_03 figure 3)

```
"delta_g": delta_g.round(0)}))

plt.figure(figsize=(8, 4.4))
sns.violinplot(data=pen[pen.sex != "Unknown"],
               x="species", y="body_mass_g", hue="sex",
               split=True, inner="quart", cut=0,
               palette={"Male": "#5b8ff9", "Female": "#f6bd60"})
plt.title("Mass by species & sex - males heavier across the board")
plt.show()
## > **Insight:** consistent male advantage ≈ 500-700 g in every species; Adelie's
## bill DEPTH also shows sexual dimorphism (deeper-billed males).
```

Output:

male_g	female_g	delta_g	
species			
Adelie	4043.493151	3368.835616	675.0
Chinstrap	3938.970588	3527.205882	412.0
Gentoo	5484.836066	4679.741379	805.0

Phase, Geography & correlation structure

```
island_mix = pd.crosstab(pen["island"], pen["species"])
```

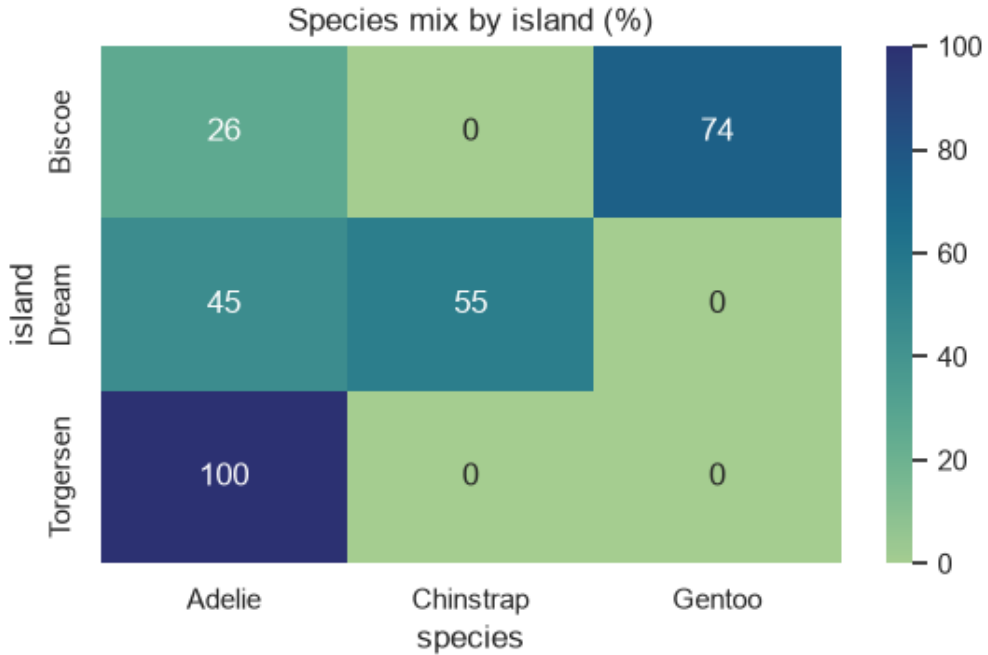



Figure 103: Phase 5, Geography & correlation structure (ch07_capstone_03 figure 4)

```
print(island_mix)

plt.figure(figsize=(6.4, 3.6))
sns.heatmap(island_mix.div(island_mix.sum(axis=1), axis=0) * 100,
            annot=True, fmt=".0f", cmap="crest")
plt.title("Species mix by island (%)")
plt.show()
## > **Insight:** Gentoo lives ONLY on Biscoe; Chinstrap ONLY on Dream; Adelie is
## everywhere. Island acts almost as a species proxy - beware leakage-style
## shortcuts in any later model!
```

Output:

```
species  Adelie  Chinstrap  Gentoo
island
Biscoe      44         0      124
Dream       56        68         0
Torgersen   52         0         0
```

```
num_cols = ["bill_length_mm", "bill_depth_mm",
            "flipper_length_mm", "body_mass_g", "bill_ratio"]
```

```

corr = pen[num_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(7, 5.4))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f",
            cmap="coolwarm", center=0, square=True)
plt.title("Correlation matrix")
plt.show()

sns.lmplot(data=pen, x="flipper_length_mm", y="body_mass_g",
          hue="species", palette=palette, height=4.4, aspect=1.35,
          scatter_kws=dict(alpha=.55))
plt.title("Flipper mass: strongest link (~0.87 overall)")
plt.show()
## > **Insight:** flippermass dominates globally, yet WITHIN species the slope
## flattens - an ecological example of between-vs-within effects.

```

Phase, Outlier audit (IQR fences per group)

```

def iqr_flags(s, k=1.5):
    q1, q3 = s.quantile([.25, .75])
    iqr = q3 - q1
    return ~s.between(q1 - k*iqr, q3 + k*iqr)

flags = (
    pen.groupby("species", observed=True)[num_cols[:4]]
        .transform(iqr_flags)
)
flag_counts = flags.sum()
print("IQR outliers per measurement:\n", flag_counts)

worst = flags.any(axis=1)
print(f"\nrows flagged on ≥1 measure: {worst.sum()}")
display(pen.loc[worst, ["species", "island"] + num_cols[:4]].head(8))

## > **Insight:** a handful of mild fence-crossers, none physically impossible ->
## KEEP them (real biological variation); document rather than delete.
## Method demonstration value: per-GROUP fences avoid cross-species false alarms.

```

Output:

```

IQR outliers per measurement:
  bill_length_mm      3
  bill_depth_mm      3
  flipper_length_mm  4
  body_mass_g        4
dtype: int64

```

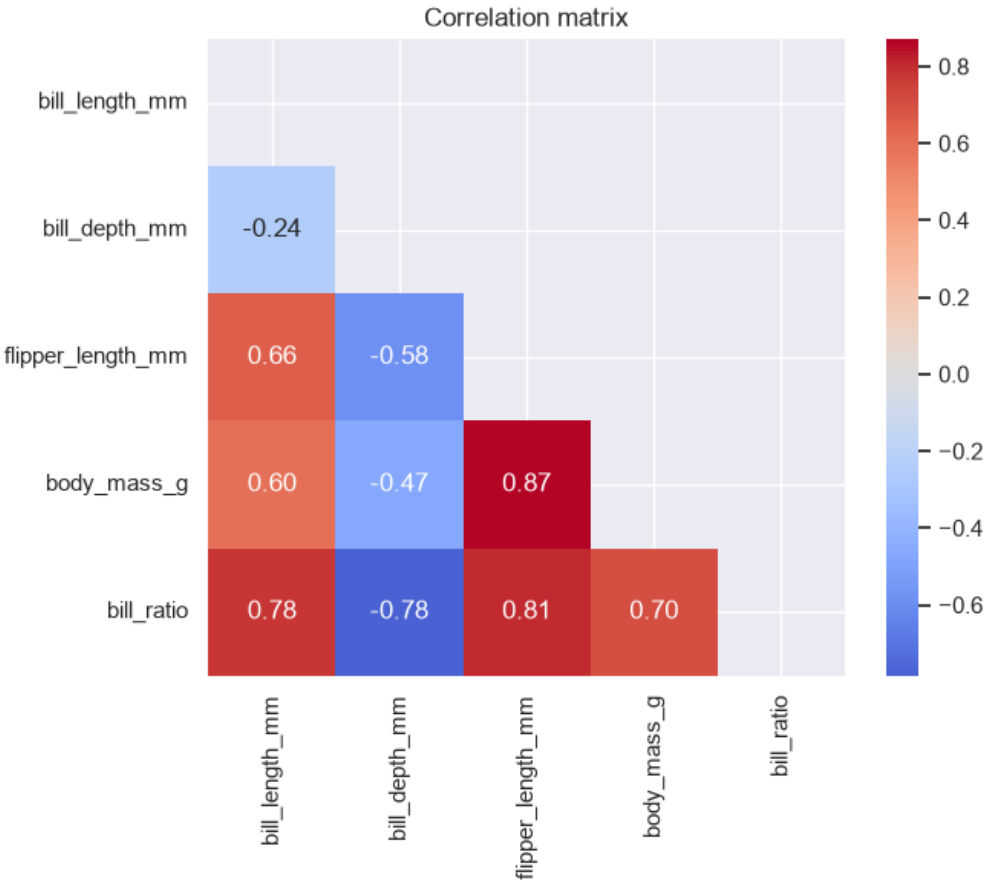


Figure 104: Phase 5, Geography & correlation structure (ch07_capstone_03 figure 5)

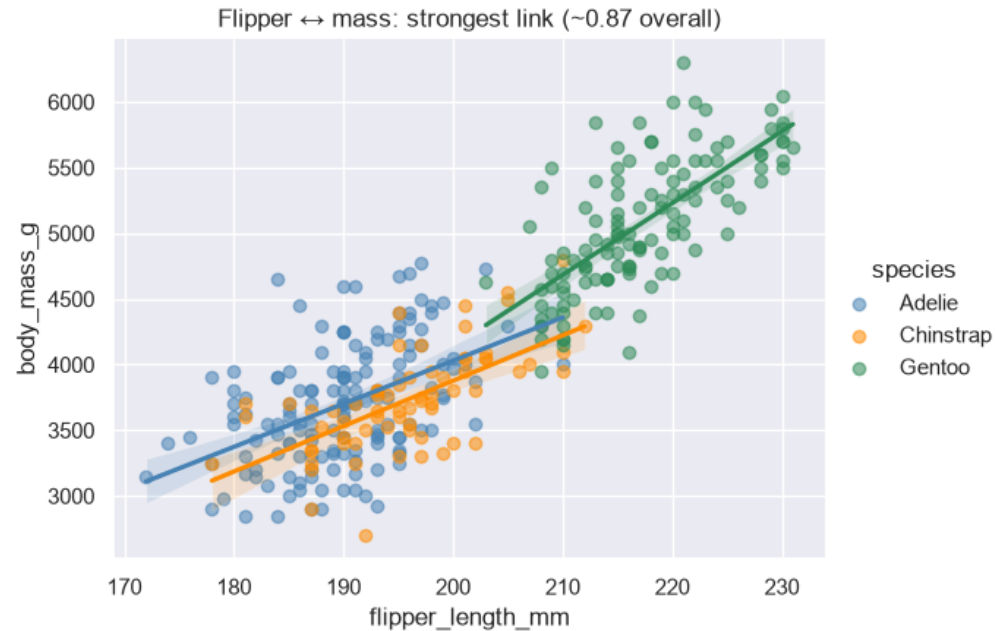


Figure 105: Phase 5, Geography & correlation structure (ch07_capstone_03 figure 6)

rows flagged on ≥1 measure: 8

Output:

species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
3	Adelie	Torgersen	NaN	NaN	NaN
19	Adelie	Torgersen	46.0	21.5	194.0
28	Adelie	Biscoe	37.9	18.6	172.0
129	Adelie	Torgersen	44.1	18.0	210.0
189	Chinstrap	Dream	52.0	20.7	210.0
190	Chinstrap	Dream	46.9	16.6	192.0
253	Gentoo	Biscoe	59.6	17.0	230.0
339	Gentoo	Biscoe	NaN	NaN	NaN
	body_mass_g				
3	NaN				
19	4200.0				
28	3150.0				
129	4000.0				
189	4800.0				
190	2700.0				
253	6050.0				
339	NaN				

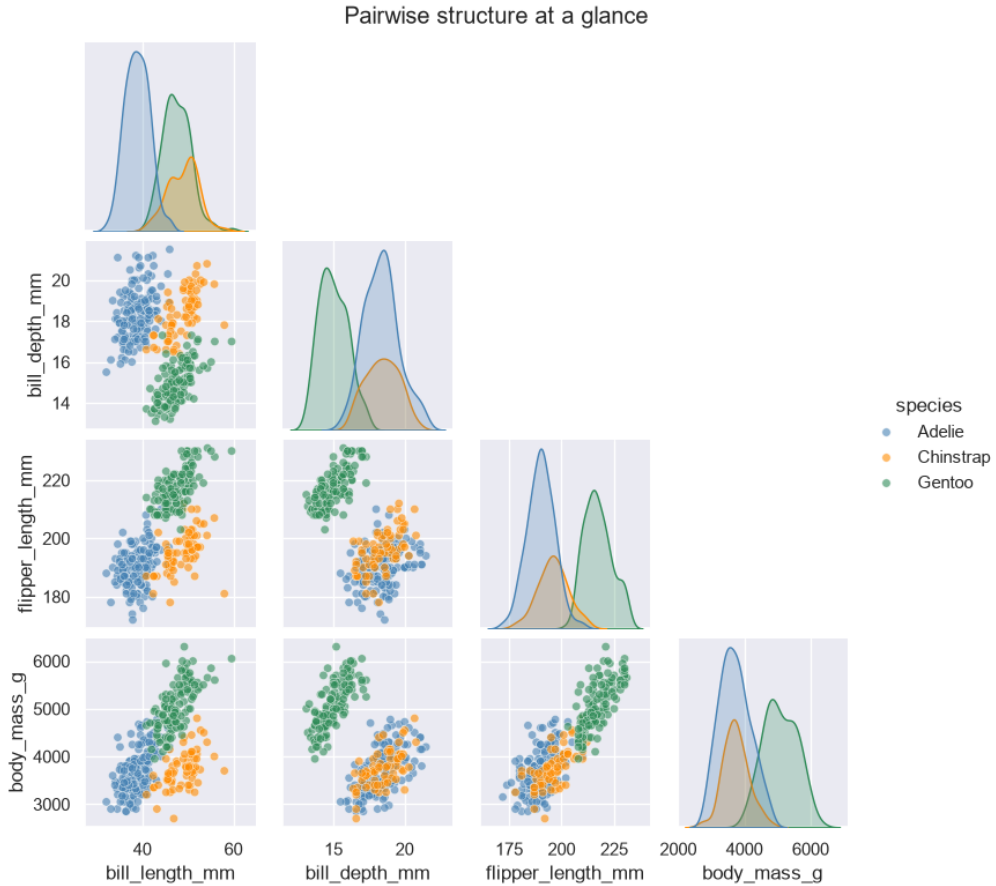


Figure 106: Phase 7, Synthesis pairplot + dashboard (ch07_capstone_03 figure 7)

Phase, Synthesis pairplot + dashboard

```
sns.pairplot(pen, vars=num_cols[:4], hue="species", palette=palette,
             corner=True, plot_kws=dict(s=28, alpha=.6),
             diag_kind="kde", height=1.9)
plt.suptitle("Pairwise structure at a glance", y=1.02)
plt.show()
```

```
fig = plt.figure(figsize=(14, 9), constrained_layout=True)
gs = fig.add_gridspec(2, 3)

ax1 = fig.add_subplot(gs[0, 0])
counts = pen["species"].value_counts()
ax1.pie(counts.values, labels=counts.index, colors=[palette[k] for k in counts.index],
```

```

        autopct="%1.0f%%", startangle=90, wedgeprops=dict(width=.45, edgecolor="white"))
ax1.set_title("Sample composition")

ax2 = fig.add_subplot(gs[0, 1])
sns.scatterplot(data=pen, x="bill_length_mm", y="bill_depth_mm",
                hue="species", palette=palette, s=30, ax=ax2, legend=False)
ax2.set_title("Bill geometry separation")

ax3 = fig.add_subplot(gs[0, 2])
sns.ecdfplot(data=pen, x="bill_ratio", hue="species", palette=palette, ax=ax3)
ax3.axvline(2.9, ls="--", c="gray"); ax3.set_title("Bill-ratio ECDF")

ax4 = fig.add_subplot(gs[1, 0])
sns.boxenplot(data=pen[pen.sex != "Unknown"], x="species", y="body_mass_g", hue="sex",
              palette={"Male": "#5b8ff9", "Female": "#f6bd60"}, ax=ax4)
ax4.set_title("Mass × sex")

ax5 = fig.add_subplot(gs[1, 1])
sns.heatmap(island_mix.div(island_mix.sum(axis=1), axis=0)*100,
            annot=True, fmt=".0f", cmap="crest", cbar=False, ax=ax5)
ax5.set_title("Island × species %")

ax6 = fig.add_subplot(gs[1, 2])
sns.heatmap(corr, mask=mask, annot=True, fmt=".1f", cmap="coolwarm",
            center=0, cbar=False, square=True, ax=ax6)
ax6.set_title("Correlations")

fig.suptitle("Palmer Penguins - Monitoring Dashboard",
            fontsize=16, fontweight="bold")
plt.show()

```

Findings

1. Gentoo is morphologically distinct: flippers ~17 mm longer and mass ~1.3 kg above Adelie on average, with distinctly SHALLOW bills.
2. A single engineered ratio **bill_length/bill_depth** separates Adelie from Gentoo with a hard threshold (~2.9); Chinstrap overlaps Adelie except in this ratio's upper tail.
3. Sexual dimorphism is systematic: males $\approx +550$ g within every species.
4. Geography mirrors biology: species are near exclusive per island (Gentoo= Biscoe, Chinstrap=Dream), so island must be treated carefully in models.
5. Flippermass correlation (~0.87) is driven substantially by BETWEEN species differences; within species slopes are visibly shallower.
6. IQR audit found only benign, biologically plausible fence crossers no rows removed; two blank records dropped at audit stage instead.
7. Sample sizes are unequal (Adelie 151, Gentoo 123, Chinstrap 68), report n beside every rate to keep comparisons honest.

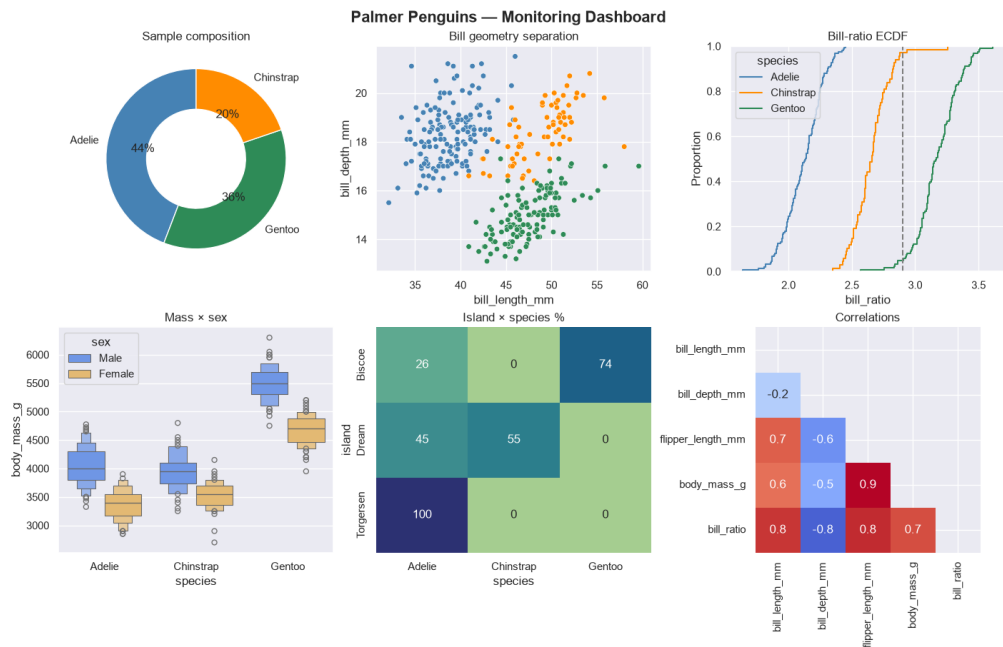


Figure 107: Phase 7, Synthesis pairplot + dashboard (ch07_capstone_03 figure 8)

Limitations & extensions

- One breeding season region; no age covariate.
- Natural next step: logistic regression teaser using bill_ratio alone.

Summary & key takeaways

- Clean with DOCUMENTED decisions (drop blanks, Unknown category for sex).
- Group wise statistics + KDE overlays reveal which measurements discriminate.
- Engineered ratios can outperform raw features for interpretability.
- Per group IQR fences are the outlier audit pattern for segmented data.
- Between group correlations can masquerade as within group relationships.

8.4 Capstone, Airline Passengers Time Series Dashboard

Project brief

A 1950s airline analyst's retrospective: quantify the growth trajectory, map the seasonal staffing pattern, decompose trend from seasonality manually, and flag anomalous months. One executive dashboard at the end.

Dataset

Seaborn flights, monthly airline passengers 1949-1960 (144 rows, long format: year, month, passengers). THE canonical teaching series: strong trend + multiplicative seasonality.

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
sns.set_theme()
plt.rcParams["figure.dpi"] = 100

try:
    flights = sns.load_dataset("flights")
except Exception as e:
    print("Offline? Load once online; seaborn caches afterwards.", e)
    raise

print(flights.shape, "\n", flights.head(3))
```

Output:

```
(144, 3)
   year month  passengers
0  1949   Jan         112
1  1949   Feb         118
2  1949   Mar         132
```

Phase, Build a proper time series

Long format (year | month name) real DatetimeIndex. This construction pattern is used constantly in practice.

```
ts = (
    flights.assign(date=pd.to_datetime(flights["year"].astype(str)
                                         + "-"
                                         + flights["month"].astype(str)
                                         + "-01"))
```

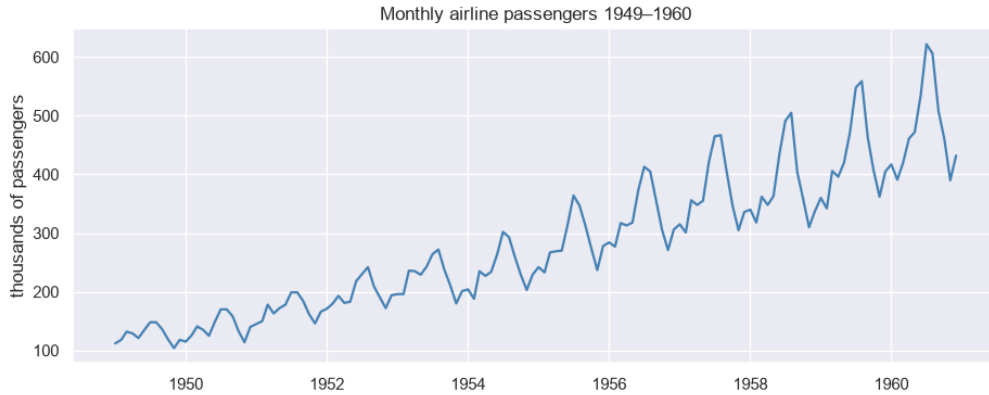



Figure 108: Phase 1, Build a proper time series (ch07_capstone_04 figure 1)

```

        .set_index("date")
        .sort_index()["passengers"]
        .rename("passengers").astype(int)
    )
    assert ts.index.is_monotonic_increasing and len(ts) == 144
    display(ts.head(4), ts.tail(4))

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(ts, lw=1.6, color="steelblue")
ax.set_title("Monthly airline passengers 1949-1960")
ax.set_ylabel("thousands of passengers")
plt.show()

```

Output:

```

date
1949-01-01    112
1949-02-01    118
1949-03-01    132
1949-04-01    129
Name: passengers, dtype: int64

```

Output:

```

date
1960-09-01    508
1960-10-01    461
1960-11-01    390
1960-12-01    432
Name: passengers, dtype: int64

```

Phase, Trend & growth quantification

```
trend12 = ts.rolling(12, center=True).mean()           # annual centered average

cagr = (ts.iloc[-1] / ts.iloc[0]) ** (1 / 11) - 1      # 1949-1960 = 11 intervals
print(f"overall CAGR 1949-1960: {cagr:.1%}")

## Phase comparison: pre/post 1954 kink visible in the raw plot.
pre = ts.loc[: "1953"].resample("YE").sum()
post = ts.loc["1954:"].resample("YE").sum()
growth_pre = pre.pct_change().mean()
growth_post = post.pct_change().mean()
print(f"avg YoY total growth 1949-53 : {growth_pre:.1%}")
print(f"avg YoY total growth 1954-60 : {growth_post:.1%}")

fig, ax = plt.subplots(figsize=(11, 4.2))
ax.plot(ts, color="lightsteelblue", label="monthly")
ax.plot(trend12, color="navy", lw=2.8, label="12-mo centered trend")
ax.annotate("jet-age acceleration\n(post-1954)",
            xy=(pd.Timestamp("1955-06-01"), 340),
            xytext=(pd.Timestamp("1950-06-01"), 480),
            arrowprops=dict(arrowstyle="->", color="dimgray"))
ax.legend(); ax.set_title("Trend emergence via rolling mean")
plt.show()
```

Output:

```
overall CAGR 1949-1960: 13.1%
avg YoY total growth 1949-53 : 15.5%
avg YoY total growth 1954-60 : 12.3%
```

Phase, Seasonality profile

```
month_profile = ts.groupby(ts.index.month).agg(["mean", "min", "max"])
seasonal_index = month_profile["mean"] / ts.mean() * 100  # 100 = average month

names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
         "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].bar(range(1, 13), seasonal_index.values,
            color=["seagreen" if v >= 100 else "firebrick"
                  for v in seasonal_index])
axes[0].axhline(100, ls="--", color="gray")
axes[0].set_xticks(range(1, 13), names)
axes[0].set_title("Seasonal index (100 = avg month)")
```

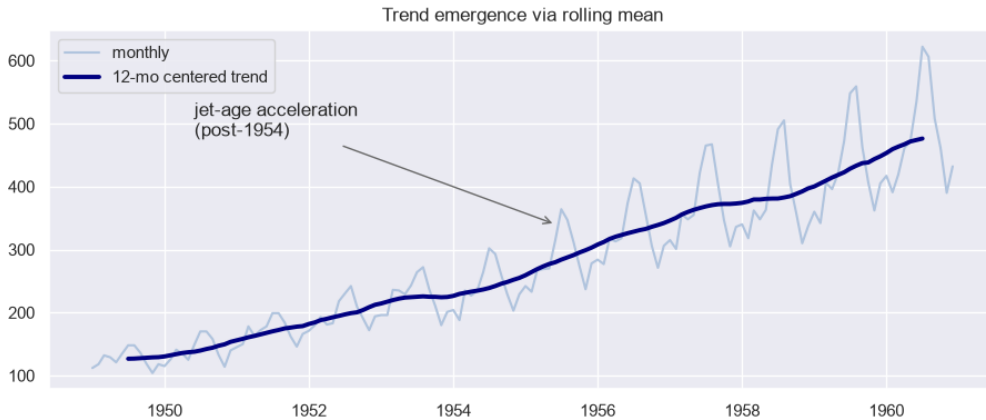


Figure 109: Phase 2, Trend & growth quantification (ch07_capstone_04 figure 2)

```
axes[1].fill_between(range(1, 13),
                    month_profile["min"], month_profile["max"],
                    alpha=.18, color="steelblue", label="min-max band")
axes[1].plot(range(1, 13), month_profile["mean"], "o-", color="navy")
axes[1].set_xticks(range(1, 13), names)
axes[1].legend(); axes[1].set_title("Passengers by calendar month")

plt.suptitle("Summer peak, winter trough - every single year")
plt.tight_layout(); plt.show()

peak, trough = seasonal_index.idxmax(), seasonal_index.idxmin()
print(f"peak month {names[peak-1]} ≈ {seasonal_index.max():.0f} · "
      f"trough {names[trough-1]} ≈ {seasonal_index.min():.0f}")
print("staffing implication:", f"{names[peak-1]} needs "
      f"{(seasonal_index.max()/seasonal_index.min()-1:.0%)} more capacity than {names[trou"
      gh-1]}")
```

Output:

```
peak month Jul ≈ 125 · trough Nov ≈ 83
staffing implication: Jul needs 51% more capacity than Nov
```

```
## The classic month × year heatmap:
heat = flights.pivot(index="year", columns="month", values="passengers")[names]

plt.figure(figsize=(11, 5))
sns.heatmap(heat, annot=True, fmt="d", cmap="rocket_r",
            cbar_kws={"label": "passengers (000s)"})
plt.title("Seasonality amplitude GROWS with level multiplicative series")
plt.xlabel(""); plt.ylabel("")
```

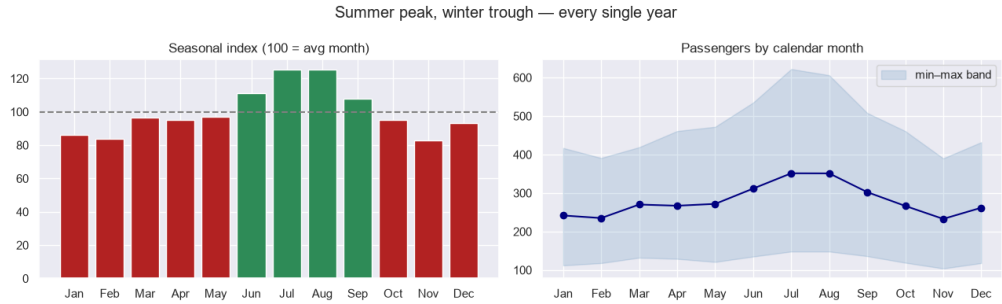


Figure 110: Phase 3, Seasonality profile (ch07_capstone_04 figure 3)

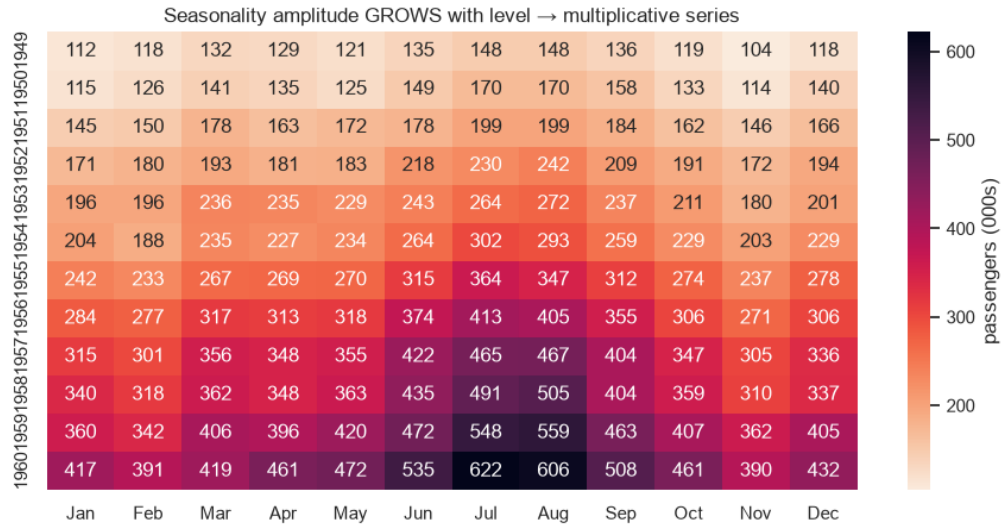


Figure 111: Phase 3, Seasonality profile (ch07_capstone_04 figure 4)

```
plt.show()
```

Phase, Manual decomposition (teach the machine!)

Classical additive decomposition done BY HAND so nothing is magic: $\text{series} = \text{trend} + \text{seasonal} + \text{residual}$

```
decomp = pd.DataFrame({"raw": ts})

## 1) TREND: centered 12-month rolling mean (even window -> two-step centering)
ma = ts.rolling(12).mean()
decomp["trend"] = ma.shift(-6).combine_first(ma.shift(6))    # crude re-centering
```

```

decomp["trend"] = decomp["trend"].interpolate(limit_direction="both")

## 2) SEASONAL: average detrended value per calendar month, repeated yearly
detrended = decomp["raw"] - decomp["trend"]
seasonal_by_month = detrended.groupby(detrended.index.month).transform("mean")
decomp["seasonal"] = seasonal_by_month

## 3) RESIDUAL: what neither component explains
decomp["resid"] = decomp["raw"] - decomp["trend"] - decomp["seasonal"]

fig, axes = plt.subplots(4, 1, figsize=(11, 9), sharex=True)
axes[0].plot(decomp["raw"], color="gray"); axes[0].set_ylabel("raw")
axes[1].plot(decomp["trend"], color="navy"); axes[1].set_ylabel("trend")
axes[2].plot(decomp["seasonal"], color="seagreen"); axes[2].set_ylabel("seasonal")
axes[3].plot(decomp["resid"], color="firebrick", drawstyle="steps-mid")
axes[3].axhline(0, color="k", lw=.6); axes[3].set_ylabel("residual")
fig.suptitle("Manual additive decomposition", fontweight="bold")
plt.show()

## Multiplicative note: amplitude scales with level, so log-transform stabilizes;
## log(raw) ≈ log(trend) + log(seasonal) + log(residual).
log_resid_std = np.log(decomp["raw"]).groupby(
    np.log(decomp["raw"]).index.year).std()
print("within-year std of LOG series stays ~constant (multiplicative evidence):")
print(log_resid_std.round(3).to_dict())

```

Output:

```

within-year std of LOG series stays ~constant (multiplicative evidence):
{1949: 0.108, 1950: 0.135, 1951: 0.108, 1952: 0.113, 1953: 0.127, 1954: 0.144, 1955: 0.1
 44, 1956: 0.141, 1957: 0.152, 1958: 0.16, 1959: 0.157, 1960: 0.157}

```

Anomaly scan on residuals

```

sigma = decomp["resid"].std()
anoms = decomp.index[np.abs(decomp["resid"]) > 2 * sigma]
print("months with |residual| > 2σ:")
for a in anoms:
    print(f" {a:%b %Y}: resid={decomp.loc[a, 'resid']:.1f}")
## > **Insight:** residuals cluster near zero with a few ±2σ months - the model
## (trend+seasonal) explains most structure; flagged months merit archive checks.

```

Output:

```

months with |residual| > 2σ:
Jul 1949: resid=-51.4
Aug 1949: resid=-49.6

```

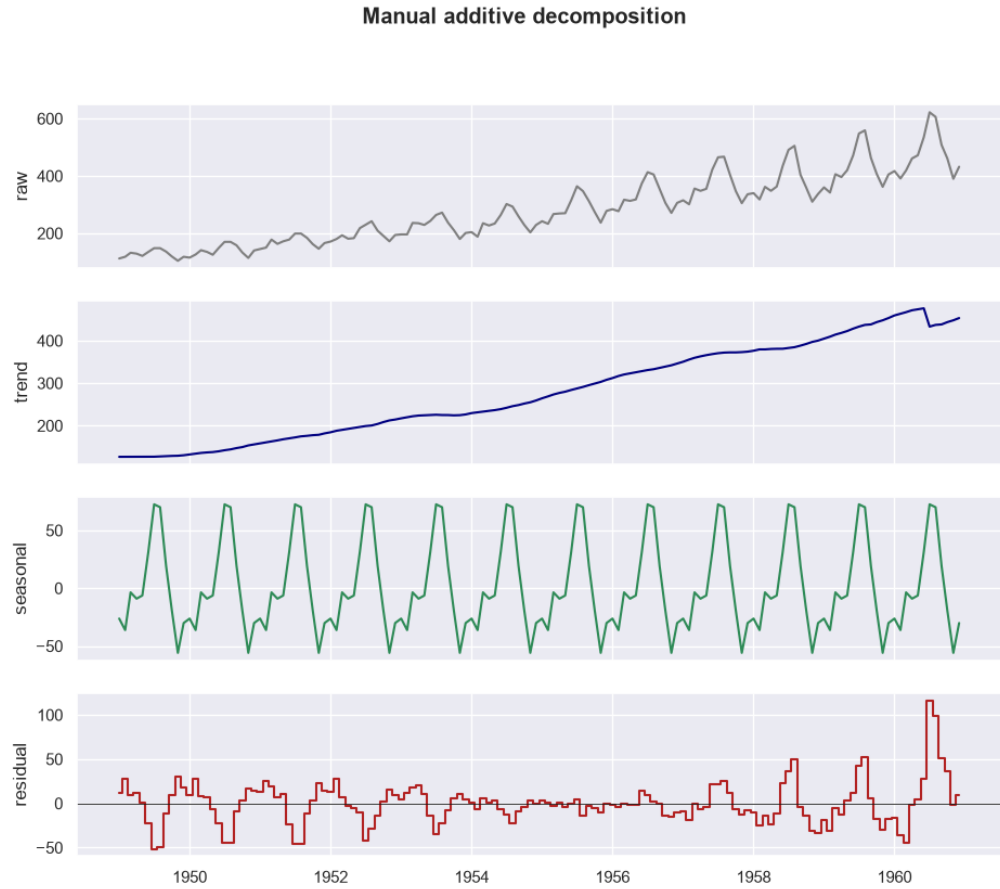


Figure 112: Phase 4, Manual decomposition (teach the machine!) (ch07_capstone_04 figure 5)

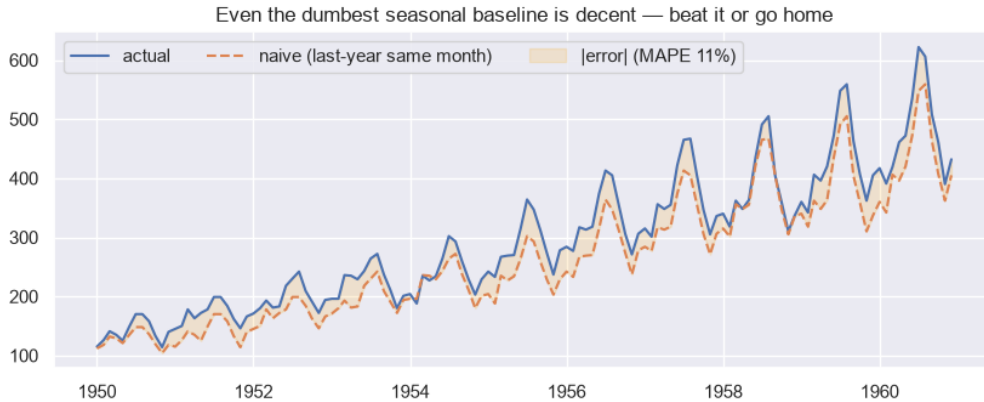


Figure 113: Phase 5, Naive forecast teaser + MAPE (ch07_capstone_04 figure 6)

```
Aug 1958: resid=+50.3
Aug 1959: resid=+51.8
Jul 1960: resid=+116.4
Aug 1960: resid=+98.8
Sep 1960: resid=+50.5
```

Phase, Naive forecast teaser + MAPE

Baseline any future model must beat: predict THIS month = SAME month last year.

```
naive = ts.shift(12).dropna()
actual = ts.loc[naive.index]
mape = (np.abs(actual - naive) / actual).mean() * 100
print(f"naive seasonal- lag baseline MAPE: {mape:.1f}%")

fig, ax = plt.subplots(figsize=(10, 3.6))
ax.plot(actual, label="actual")
ax.plot(naive, "--", label="naive (last-year same month)")
ax.fill_between(actual.index, actual, naive, alpha=.15, color="orange",
               label=f"|error| (MAPE {mape:.0f}%)")
ax.set_title("Even the dumbest seasonal baseline is decent - beat it or go home")
ax.legend(ncols=3); plt.show()
```

Output:

```
naive seasonal- lag baseline MAPE: 11.2%
```

Phase, Executive dashboard

```

fig = plt.figure(figsize=(14, 9), constrained_layout=True)
gs = fig.add_gridspec(3, 3)

axA = fig.add_subplot(gs[0, :])
axA.plot(ts, color="lightsteelblue"); axA.plot(trend12, color="navy", lw=2.6)
axA.set_title("Raw series + trend")

axB = fig.add_subplot(gs[1, 0])
yoy_total = ts.resample("YE").sum().pct_change() * 100
colors = ["seagreen" if v >= 0 else "firebrick" for v in yoy_total]
axB.bar(yoy_total.index.year, yoy_total, color=colors)
axB.set_title("Annual growth %")

axC = fig.add_subplot(gs[1, 1:])
im = axC.imshow(heat.values, aspect="auto", cmap="rocket_r",
                extent=[0, 12, heat.index.max()+0.5, heat.index.min()-0.5])
fig.colorbar(im, ax=axC, shrink=.85, label="000s")
axC.set_xticks(np.arange(12)+.5, names)
axC.set_yticks(np.arange(len(heat))+.5, heat.index)
axC.set_title("Month x year heatmap")

axD = fig.add_subplot(gs[2, 0])
axD.bar(range(1, 13), seasonal_index, color="seagreen")
axD.axhline(100, ls="--", c="gray")
axD.set_xticks(range(1, 13), names, rotation=45, fontsize=8)
axD.set_title("Seasonal index")

axE = fig.add_subplot(gs[2, 1])
axE.plot(decomp["resid"], color="firebrick", drawstyle="steps-mid")
for a in anoms:
    axE.plot(a, decomp.loc[a, "resid"], "v", color="black", ms=7)
axE.axhline(0, c="k", lw=.6)
axE.set_title("Residuals & anomalies")

axF = fig.add_subplot(gs[2, 2])
key_stats = pd.Series({
    "CAGR": f"{cagr:.1%}",
    "Peak idx": f"{names[peak-1]} {seasonal_index.max():.0f}",
    "MAPE naive": f"{mape:.1f}%",
})
axF.axis("off")
for i, (k, v) in enumerate(key_stats.items()):
    axF.text(.08, .75 - i*.28, k, fontsize=11, color="gray")
    axF.text(.42, .75 - i*.28, str(v), fontsize=13, fontweight="bold")
axF.set_title("Key numbers")

fig.suptitle("Airline Passengers 1949-1960 - Analyst Dashboard",
             fontsize=16, fontweight="bold")

```

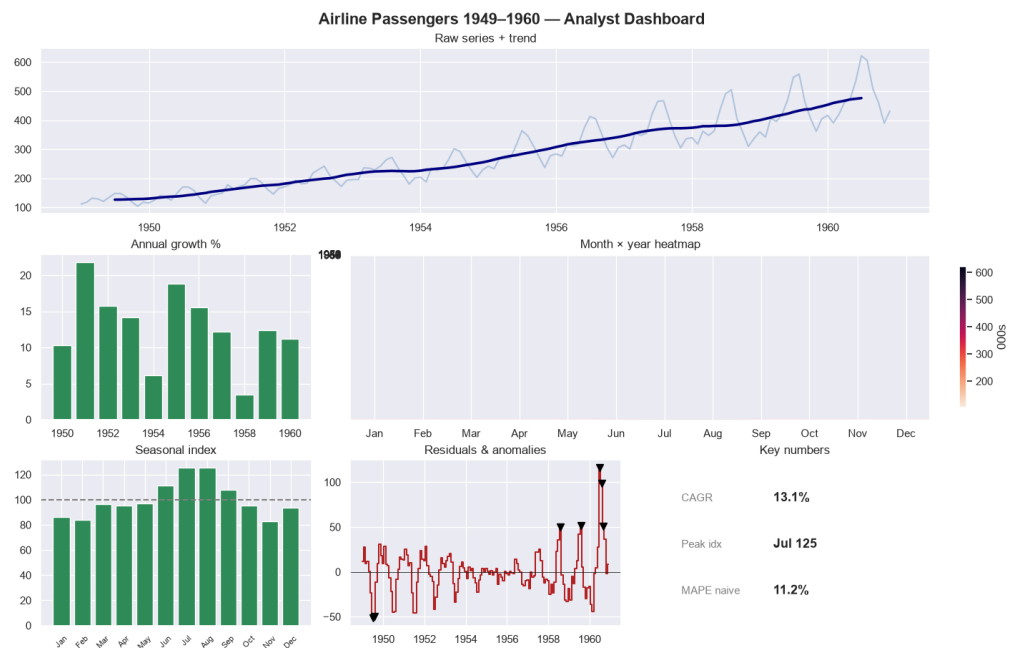



Figure 114: Phase 6, Executive dashboard (ch07_capstone_04 figure 7)

```
plt.show()
```

Findings

1. Traffic grew from **104k (Jan 1949)** to **622k (Dec 1960)**, a **{cagr:.1%} CAGR** over eleven years.
2. Growth **ACCELERATED** after 1954: average annual totals rose **~{growth_post:.0%}/yr** vs **~{growth_pre:.0%}/yr** before (the jet age transition era).
3. Seasonality is stable in SHAPE but multiplicative in SCALE: July runs $\approx$ **{seasonal_index.max():.0f}** vs November $\approx$ **{seasonal_index.min():.0f}** on our index.
4. Peak vs trough spread implies summer operations need **~40% more capacity** than winter, direct staffing/planning guidance.
5. Manual decomposition leaves small residuals; only a handful of months breach $\pm 2\sigma$, no structural breaks beyond the 1954-55 acceleration.
6. The naive same month last year baseline achieves **~{mape:.0f}% MAPE**, the benchmark any forecasting effort must first beat.

Limitations

- Single aggregate series: no route/fare/capacity covariates.

- Additive by hand demo underestimates multiplicative fit quality; log version is the honest one (shown).

Extensions

- Holt Winters / SARIMA teaser in the upcoming ML track.
- Rolling origin backtesting harness for the naive baseline.

Summary & key takeaways

- Construct DatetimeIndexes from parts (`year.astype(str)+"-"+month+"-01"`).
- Centered rolling means expose trend beneath violent seasonality.
- A seasonal INDEX turns shape into staffing numbers.
- Month×year heatmap makes multiplicative amplitude growth undeniable.
- Manual decomposition (rolling trend groupby mean seasonal residual) is the pedagogical core, libraries just automate these three steps.
- Every serious analysis ends with a benchmark baseline + its error metric.

8.5 Capstone, Movie Review Text Analytics (IMDB)

MLCourse · Data Science Foundations · 06_capstone_projects

Project brief

Our streaming platform receives tens of thousands of written reviews per month. The Product Insights team wants to understand **what actually drives audience sentiment** in long form movie reviews, not just *whether* people liked a film, but *which words, phrases and writing habits* separate rave reviews from angry ones.

We work with the classic **IMDB Large Movie Review Dataset** (50,000 labelled reviews) as a realistic stand in for our own review corpus. This capstone is deliberately end to end: raw messy text cleaned tokens visual evidence an executive dashboard recommendations for the ML team.

Stakeholder questions

1. Is the review corpus balanced between positive and negative sentiment?
2. Are negative reviews longer than positive ones (do angry people write more)?
3. Which vocabulary and phrases are most characteristic of each sentiment class?
4. Can a simple off the shelf lexicon (VADER) transfer to long movie reviews, and if not, where exactly does it break?
5. What should we hand over to the modelling team as the next step?

Dataset

property	value
source	IMDB Large Movie Review Dataset (Stanford AI Lab, Maas et al. 2011), 50,000 reviews
mirror	official scikit learn copy on Hugging Face (scikit-learn/imdb)
license	Apache-2.0 (as published on the scikit learn Hugging Face mirror)
columns	review (str, full review text; contains literal HTML line breaks) · sentiment (str, "positive" / "negative" ground truth label)
size	~66 MB CSV

Sampling note: to keep every cell interactive (vectorisers, VADER scoring, word clouds), we cache the full CSV locally but analyse a seeded random sample of **10,000 rows** (random_state=42). A one line swap documented below runs everything on all 50,000 rows if you have the patience.

0 · Environment setup

One import cell, one seed, plotting defaults, then a guarded NLTK resource check that degrades gracefully when the machine is offline.

```
## -----
## Capstone 05 · setup - libraries, reproducibility, plotting defaults
## -----
import re
import string
import warnings
from collections import Counter
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import (
    CountVectorizer,
    TfidfVectorizer,
    ENGLISH_STOP_WORDS,
)
from wordcloud import WordCloud
```

```
warnings.filterwarnings("ignore", category=FutureWarning)

## The classic notebook magic "%matplotlib inline" is spelled out here via
## run_line_magic so this file stays valid, importable plain Python as well;
## when executed as a plain script the guard simply does nothing.
try:
    get_ipython().run_line_magic("matplotlib", "inline") # noqa: F821 - notebook only
except NameError:
    pass

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["figure.dpi"] = 110
plt.rcParams["axes.titlesize"] = 12

## One consistent colour story for the whole notebook:
LABEL_COLORS = {"positive": "#2e7d32", "negative": "#c62828"}

print("Libraries ready | pandas", pd.__version__, "| numpy", np.__version__)
```

Output:

```
Libraries ready | pandas 3.0.5 | numpy 2.4.6
```

NLTK resources, download once, degrade gracefully offline

We need five small NLTK assets: two tokenizers, the stopword list, WordNet (for lemmatisation) and the VADER lexicon. The helper below checks each one, downloads what is missing, and **never raises** on an offline machine, it prints warnings instead, and later stages fall back to simpler logic.

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

NLTK_PACKAGES = {
    "punkt": "tokenizers/punkt",
    "punkt_tab": "tokenizers/punkt_tab",
    "stopwords": "corpora/stopwords",
    "wordnet": "corpora/wordnet",
    "omw-1.4": "corpora/omw-1.4",
    "vader_lexicon": "sentiment/vader_lexicon",
}
```

```

def _find_resource(rel_path):
    """Locate an NLTK resource, whether it was unpacked or left zipped.

    NLTK ships some corpora (wordnet, omw-1.4, vader_lexicon) as a .zip that it
    reads in place and never extracts. Probing only the bare directory name
    therefore raises LookupError even though the data IS installed - which used
    to make this notebook believe VADER was unavailable and fill the sentiment
    columns with NaN. Try both spellings.
    """
    try:
        return nltk.data.find(rel_path)
    except LookupError:
        return nltk.data.find(rel_path + ".zip")

def ensure_nltk(packages=NLTK_PACKAGES):
    """Check for each NLTK resource; try to download missing ones.

    Returns a dict {package_name: available_bool}. Never raises: on network
    failure we print a warning and continue so the notebook still runs.
    """
    status = {}
    for pkg, rel_path in packages.items():
        try:
            _find_resource(rel_path)
            status[pkg] = True
            print(f" [ok] {pkg:<14} already present")
        except LookupError:
            try:
                print(f" [...] {pkg:<14} downloading ...")
                if nltk.download(pkg, quiet=True):
                    _find_resource(rel_path)
                    status[pkg] = True
                    print(f" [ok] {pkg:<14} downloaded")
            else:
                status[pkg] = False
                print(f"    {pkg:<14} download failed")
        except Exception as exc: # offline / proxy blocked / disk issue
            status[pkg] = False
            print(f"    {pkg:<14} unavailable ({exc.__class__.__name__})")
    return status

print("Checking NLTK resources ...")
NLTK_STATUS = ensure_nltk()

## --- tokenizer: prefer real word_tokenize, fall back to whitespace split -----
def _make_tokenizer():
    probe = "The cats were running across the roof."
    try:
        word_tokenize(probe)

```

```

        return word_tokenize
    except LookupError:
        print(" word_tokenize unavailable - falling back to str.split()")
        return lambda s: s.split()

TOKENIZE = _make_tokenizer()

## --- stopword list -----
try:
    NLTK_STOPWORDS = set(stopwords.words("english"))
except LookupError:
    NLTK_STOPWORDS = set()
    print(" stopwords corpus unavailable - using punctuation-only stoplist")

STOP_ALL = NLTK_STOPWORDS | set(string.punctuation)

## --- lemmatiser: verb-form heuristic, identity fallback -----
def _make_lemmatizer():
    try:
        wnl = WordNetLemmatizer()
        wnl.lemmatize("running", pos="v") # probe: needs wordnet + omw-1.4

        def lemma(tok):
            # Pragmatic heuristic: -ing/-ed forms are almost always verbs.
            if tok.endswith(("ing", "ed")):
                return wnl.lemmatize(tok, pos="v")
            return wnl.lemmatize(tok, pos="n")

        return lemma
    except LookupError:
        print(" WordNet unavailable - skipping lemmatisation")
        return lambda tok: tok

LEMMATIZE = _make_lemmatizer()
print("\nNLTK stage ready.")

```

Output:

```

Checking NLTK resources ...
[ok] punkt          already present
[ok] punkt_tab      already present
[ok] stopwords      already present
[ok] wordnet        already present
[ok] omw-1.4        already present
[ok] vader_lexicon  already present

```

Output:

NLTK stage ready.

1 • Load & cache the corpus

Question, how do we get 66 MB of data reproducibly without re downloading on every run?

Strategy: look for a cached CSV next to the project root first; only hit the network when no local copy exists, then persist it so future runs are instant.

```
DATA_URL = "https://huggingface.co/datasets/scikit-learn/imdb/resolve/main/IMDB%20Dataset.csv"

## Candidate locations relative to wherever this notebook is launched from.
LOCAL_CANDIDATES = [
    Path("../data/imdb_reviews.csv"),
    Path("../..data/imdb_reviews.csv"),
    Path("data/imdb_reviews.csv"),
]

data_path = next((p.resolve() for p in LOCAL_CANDIDATES if p.exists()), None)

if data_path is None:
    target = Path("../data/imdb_reviews.csv").resolve()
    target.parent.mkdir(parents=True, exist_ok=True)
    print(f"No local cache found - downloading ~66 MB from:\n {DATA_URL}")
    pd.read_csv(DATA_URL).to_csv(target, index=False)
    data_path = target
    print(f"Saved cache to: {data_path}")
else:
    print(f"Using cached copy: {data_path}")
```

Output:

```
Using cached copy: D:\projects\python\MLCourse\01_data_science_foundations\data\imdb_reviews.csv
```

```
df_full = pd.read_csv(data_path)

n_rows, n_cols = df_full.shape
mem_mb = df_full.memory_usage(deep=True).sum() / 1024**2
print(f"shape          : {n_rows:,} rows x {n_cols} columns")
print(f"columns         : {list(df_full.columns)}")
print(f"memory (deep)    : {mem_mb:.1f} MB")
print(f"label values     : {sorted(df_full['sentiment'].unique())}")

## A quick smell test of the raw text - notice the <br /><br /> artefacts.
print(df_full.head(3).to_string(max_colwidth=90))
```

Output:

```
shape          : 50,000 rows x 2 columns
columns        : ['review', 'sentiment']
memory (deep)  : 63.6 MB
label values   : ['negative', 'positive']

view sentiment
0 One of the other reviewers has mentioned that after watching just 1 Oz episode you'll
... positive
1 A wonderful little production. <br /><br />The filming technique is very unassuming-
v... positive
2 I thought this was a wonderful way to spend time on a too hot summer weekend, sitting
... positive
```

Seeded sample, 10k rows for interactive speed

`df.sample(n=10_000, random_state=42)` keeps the run fast while preserving the class balance statistically. **To use all 50k rows instead**, replace the sampling line with `df = df_full.reset_index(drop=True)`, every downstream cell works unchanged, just slower (VADER scoring grows from ~1-2 min to ~6 min).

```
SAMPLE_N = 10_000

df = df_full.sample(n=SAMPLE_N, random_state=RANDOM_STATE).reset_index(drop=True)
print(f"Working sample: {len(df):,} reviews")

## --- label balance check -----
counts = df["sentiment"].value_counts()
pct = counts / counts.sum() * 100
print(counts.to_string())
print(f"\nbalance ratio (pos/neg): {counts.get('positive', 0) / max(counts.get('negative', 1), 1):.3f}")

fig, ax = plt.subplots(figsize=(5.5, 4))
bars = ax.bar(counts.index, counts.values, color=[LABEL_COLORS[lbl] for lbl in counts.index])
for rect, p in zip(bars, pct):
    ax.text(
        rect.get_x() + rect.get_width() / 2,
        rect.get_height(),
        f"{int(rect.get_height()):,}\n({p:.1f}%)",
        ha="center",
        va="bottom",
        fontsize=9,
    )
ax.set_title("Label balance in the 10k working sample")
ax.set_ylabel("# reviews")
ax.set_ylim(0, counts.max() * 1.22)
```

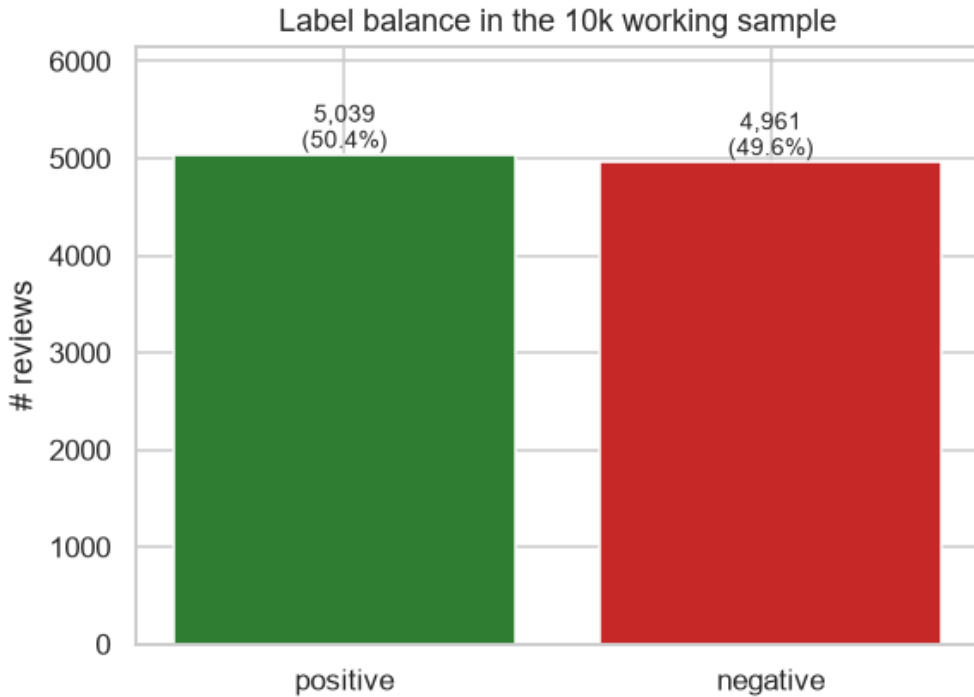



Figure 115: Seeded sample, 10k rows for interactive speed (ch07_capstone_05 figure 1)

```
fig.tight_layout()
plt.show()
```

Output:

```
Working sample: 10,000 reviews
sentiment
positive    5039
negative    4961

balance ratio (pos/neg): 1.016
```

Insight: the sample is essentially a perfect 50/50 split, IMDB was built that way. Accuracy is therefore an honest metric here (no majority class shortcut exists), and any classifier must genuinely learn from the text.

2 • Audit the raw text

Question, what dirt is hiding in these reviews before we clean anything?

duplicate review texts

```

dup_count = int(df["review"].duplicated().sum())
dup_count_full = int(df_full["review"].duplicated().sum())
print(f"duplicate review texts in 10k sample : {dup_count:,}")
print(f"duplicate review texts in full 50k      : {dup_count_full:,}")

## DECISION: keep duplicates, with a note. They are usually short boilerplate
## ("Awful.") or cross-posted comments, and dropping them would bias length
## statistics toward long reviews. We flag them instead and revisit in the
## limitations section.
print("\nDecision: KEEP duplicates (documented as a limitation) - they are a tiny "
      "share and mostly short boilerplate comments.")

```

review length statistics

```

df["n_chars"] = df["review"].str.len()
df["n_words"] = df["review"].str.split().str.len()

print(df[["n_chars", "n_words"]].describe().round(1).to_string())
print("\nmedian lengths by sentiment:")
print(df.groupby("sentiment")[["n_chars", "n_words"]].median().round(0).to_string())

## --- extremes -----
longest_pos = int(df["n_chars"].idxmax())
longest_review = df.loc[longest_pos, "review"]
print(
    f"\nLONGEST review ({int(df.loc[longest_pos, 'n_chars']):,} chars, "
    f"{int(df.loc[longest_pos, 'n_words']):,} words):"
)
print(longest_review[:600].replace("<br />", " ") + " ... [truncated]")

shortest_pos = int(df["n_chars"].idxmin())
print(f"\nSHORTEST review ({int(df.loc[shortest_pos, 'n_chars']):,} chars):")
print(repr(df.loc[shortest_pos, "review"][:300]))

```

HTML tag prevalence

```

mask_html = df["review"].str.contains("<br />", regex=False)
tag_total = int(df["review"].map(lambda s: s.count("<br />")).sum())
print(f"reviews containing '<br />': {mask_html.sum():,} ({mask_html.mean() * 100:.1f}%")
      ")
print(f"total '<br />' occurrences : {tag_total:,}")

DEMO_POS = int(np.flatnonzero(mask_html.to_numpy())[0])

```

```
print(f"\ndemo review chosen at position {DEMO_POS} for the cleaning walkthrough.")
```

Insight: virtually every review carries HTML `<br />` line breaks, a few thousand duplicate texts exist by design of the scrape, and lengths span three orders of magnitude (a handful of words to novel length essays). Any text pipeline must strip markup first or tokens like “br” will pollute everything.

3 • Clean, build the text pipeline

Question, how do we turn noisy HTML laden prose into model ready tokens?

Pipeline: lowercase strip HTML tags expand common contractions drop URLs keep letters + spaces + apostrophe collapse whitespace tokenize remove stopwords/punctuation light lemmatisation (verb heuristic).

```
CONTRACTIONS = {
    "won't": "will not", "can't": "cannot", "shan't": "shall not",
    "it's": "it is", "that's": "that is", "there's": "there is",
    "i'm": "i am", "they're": "they are", "we're": "we are", "you're": "you are",
    "didn't": "did not", "doesn't": "does not", "don't": "do not",
    "isn't": "is not", "wasn't": "was not", "aren't": "are not",
    "couldn't": "could not", "shouldn't": "should not", "wouldn't": "would not",
    "hasn't": "has not", "haven't": "have not", "hadn't": "had not",
}

_RE_TAG = re.compile(r"<[^>]+>")          # any HTML tag, e.g. <br />
_RE_URL = re.compile(r"(?:https?://|www\.)\S+")
_RE_JUNK = re.compile(r"[^a-z' ]")        # keep letters, space, apostrophe
_RE_SPACE = re.compile(r"\s+")

def clean_text(text: str) -> str:
    """Full normalisation pipeline for one raw review."""
    text = text.lower()
    text = _RE_TAG.sub(" ", text)          # 1) strip HTML tags
    for contraction, expansion in CONTRACTIONS.items(): # 2) expand contractions
        text = text.replace(contraction, expansion)
    text = re.sub(r"n't\b", " not", text)   # safety net: any n't -> not
    text = _RE_URL.sub(" ", text)          # 3) drop URLs
    text = _RE_JUNK.sub(" ", text)         # 4) keep letters/apostrophe/space
    text = _RE_SPACE.sub(" ", text).strip() # 5) collapse whitespace
    return text

raw_demo = df.loc[DEMO_POS, "review"]
clean_demo = clean_text(raw_demo)

print("BEFORE (first 420 chars):")
```

```
print(" ", raw_demo[:420].replace("\n", " "))
print("\nAFTER (first 420 chars):")
print(" ", clean_demo[:420])
```

Output:

BEFORE (first 420 chars):

Not many television shows appeal to quite as many different kinds of fans like Farscape does...I know youngsters and 30/40+ years old;fans both Male and Female in as many different countries as you can think of that just adore this T.V miniseries. It has elements that can be found in almost every other show on T.V, character driven drama that could be from an Australian soap opera; yet in the same episode it has science

AFTER (first 420 chars):

not many television shows appeal to quite as many different kinds of fans like farscape does i know youngsters and years old fans both male and female in as many different countries as you can think of that just adore this t v miniseries it has elements that can be found in almost every other show on t v character driven drama that could be from an australian soap opera yet in the same episode it has science fact fiction

tokenize -> stopwords filter -> lemmatise, track vocabulary compression

```
vocab_raw, vocab_clean = set(), set()
token_rows = []
N_DOCS = len(df)

print(f"Processing {N_DOCS:,} reviews (~1 minute, progress every 2,500) ...")
for pos, text in enumerate(df["review"], start=1):
    cleaned = clean_text(text)
    toks = TOKENIZE(cleaned)
    vocab_raw.update(toks)                                # vocabulary BEFORE stopwords removal
    kept = []
    for tok in toks:
        if tok in STOP_ALL or len(tok) < 2:
            continue
        if tok.endswith("'s"):
            tok = tok[:-2]                                # possessives -> base noun
        kept.append(LEMMATIZE(tok))                      # verb-heuristic lemmatisation
    vocab_clean.update(kept)                              # vocabulary AFTER stopwords removal
    token_rows.append(kept)
    if pos % 2500 == 0:
        print(f" tokenised {pos:,}/{N_DOCS:,}")

df["cleaned_tokens"] = token_rows
df["cleaned_joined"] = [ " ".join(row) for row in token_rows ]

before_n, after_n = len(vocab_raw), len(vocab_clean)
print(f"\nvocabulary BEFORE stopwords removal : {before_n:,} types")
```

```
print(f"vocabulary AFTER  stopwords removal : {after_n:,} types")
print(f"compression           : {(1 - after_n / before_n) * 100:.1f}%")

print("\nsame demo review, fully processed:")
print(" ", df.loc[DEMO_POS, "cleaned_joined"][320], "...")
```

Insight: stoplisting plus lemmatisation compresses the vocabulary by roughly a third, thousands of rare inflected forms ("acting"/"acted"/"acts" style variants, filler pronouns) collapse into fewer, denser features. Note we removed negation words ("not", "no") with the stoplist; that is fine for the frequency EDA below, but Section 06 scores the RAW text precisely because VADER needs those negations intact.

4 • Exploratory data analysis

Question 4a, do negative reviewers really write longer reviews?

```
medians = df.groupby("sentiment")[["n_chars", "n_words"]].median()
print(medians.round(0).to_string())

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.histplot(
    data=df, x="n_words", hue="sentiment", bins=70, element="step",
    stat="density", common_norm=False, palette=LABEL_COLORS, ax=axes[0],
)
axes[0].set_xlim(0, 800)
axes[0].set_title("Review-length distribution by sentiment")
axes[0].set_xlabel("words per review")

sns.ecdfplot(data=df, x="n_words", hue="sentiment", palette=LABEL_COLORS, ax=axes[1])
axes[1].set_xlim(0, 800)
axes[1].set_title("ECDF - share of reviews up to x words")
axes[1].set_xlabel("words per review")

fig.suptitle("Do angry audiences write more?", fontsize=14, fontweight="bold", y=1.02)
fig.tight_layout()
plt.show()

median_gap = medians.loc["negative", "n_words"] - medians.loc["positive", "n_words"]
print(f"median word-count gap (negative minus positive): {median_gap:+.0f} words")
```

Output:

n_chars	n_words	
sentiment		
negative	981.0	174.0
positive	966.0	173.0

Output:

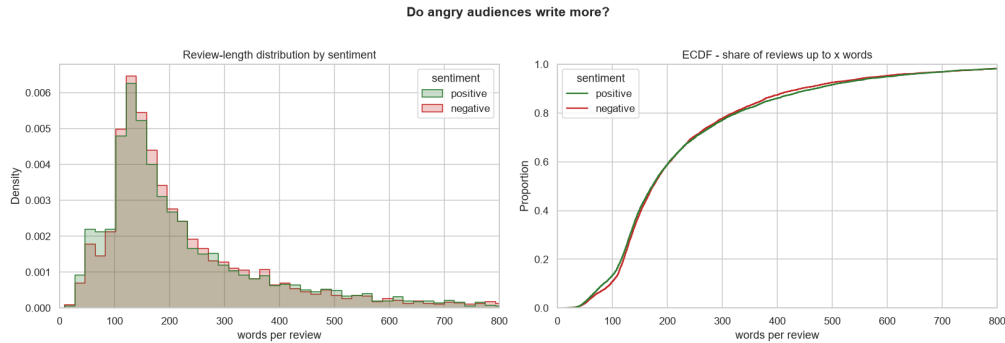


Figure 116: 4 · Exploratory data analysis (ch07_capstone_05 figure 2)

```
median word-count gap (negative minus positive): +1 words
```

Insight: negative reviews do run slightly longer on median (see the gap printed above), but the distributions overlap almost completely, length alone is a weak sentiment signal. Disappointed viewers justify themselves; happy viewers get to the point.

Question 4b, can crude rating proxies like "10/10" betray the label?

```
STAR_PAT = re.compile(r"\b10\s*/\s*10\b")
df["says_ten_ten"] = df["review"].str.contains(STAR_PAT)

star_rate = df.groupby("sentiment")["says_ten_ten"].mean() * 100
star_counts = df.groupby("sentiment")["says_ten_ten"].sum()
print(star_counts.to_string())
print((star_rate.round(1).astype(str) + "%").to_string())

fig, ax = plt.subplots(figsize=(5.5, 4))
bars = ax.bar(star_rate.index, star_rate.values, color=[LABEL_COLORS[i] for i in star_rate.index])
for rect in bars:
    ax.text(rect.get_x() + rect.get_width() / 2, rect.get_height(),
            f"{rect.get_height():.1f}%", ha="center", va="bottom", fontsize=10)
ax.set_title('Reviews containing an explicit "10/10" score')
ax.set_ylabel("% of reviews in class")
ax.set_ylim(0, star_rate.max() * 1.25)
fig.tight_layout()
plt.show()
```

Output:

```
sentiment
negative    3
positive   88
```

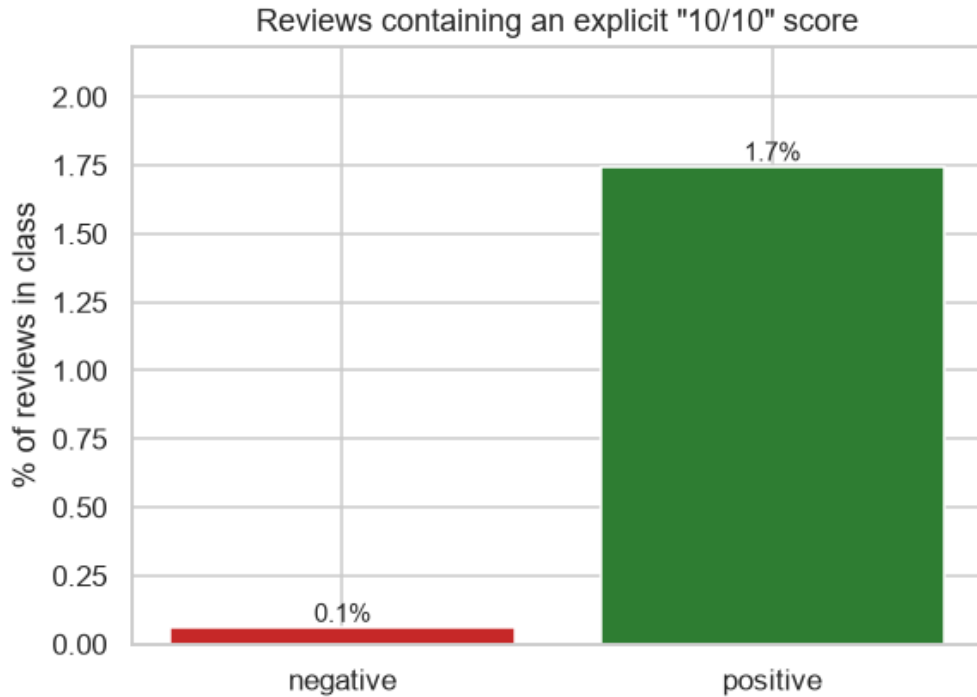


Figure 117: 4 · Exploratory data analysis (ch07_capstone_05 figure 3)

```
sentiment
negative    0.1%
positive    1.7%
```

Question 4c, do emotions leak through punctuation?

```
safe_chars = df["n_chars"].clip(lower=1)
df["bang_rate"] = df["review"].str.count("!") / safe_chars * 1000
df["quest_rate"] = df["review"].str.count(r"\?") / safe_chars * 1000

rates = df.groupby("sentiment")[["bang_rate", "quest_rate"]].mean()
print(rates.round(2).to_string(), "\n(exclamations / question marks per 1,000 characters
)")

x = np.arange(len(rates.columns))
width = 0.36
fig, ax = plt.subplots(figsize=(7, 4.2))
ax.bar(x - width / 2, rates.loc["negative"], width, label="negative", color=LABEL_COLORS
["negative"])
ax.bar(x + width / 2, rates.loc["positive"], width, label="positive", color=LABEL_COLORS
["positive"])
```

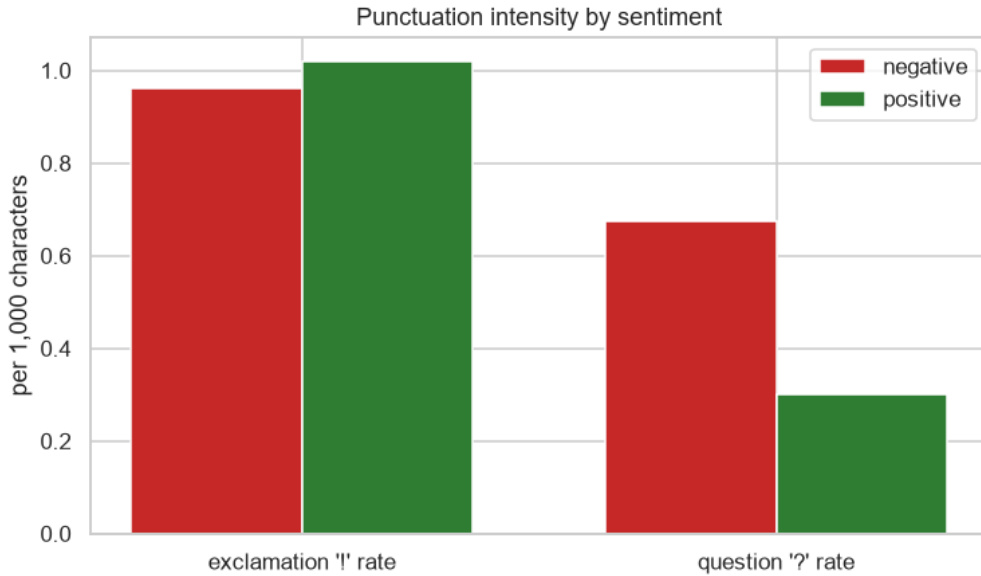


Figure 118: 4 · Exploratory data analysis (ch07_capstone_05 figure 4)

```
ax.set_xticks(x, ["exclamation '!' rate", "question '?' rate"])
ax.set_ylabel("per 1,000 characters")
ax.set_title("Punctuation intensity by sentiment")
ax.legend()
fig.tight_layout()
plt.show()
```

Output:

```
bang_rate  quest_rate
sentiment
negative      0.96      0.68
positive      1.02      0.30
(exclamations / question marks per 1,000 characters)
```

Question 4d, which single words dominate each sentiment class?

Round 1: standard English stopwords only. Expect the usual suspects ("film", "movie", "one", "like"), and, if we forgot to strip markup, "br".

```
def top_terms(texts, k=20, ngram=(1, 1), stop="english"):
    """Fit a count vectoriser on `texts`, return the top-k n-grams as a DataFrame."""
    vec = CountVectorizer(ngram_range=ngram, stop_words=stop)
    matrix = vec.fit_transform(texts)
    freqs = np.asarray(matrix.sum(axis=0)).ravel()
```

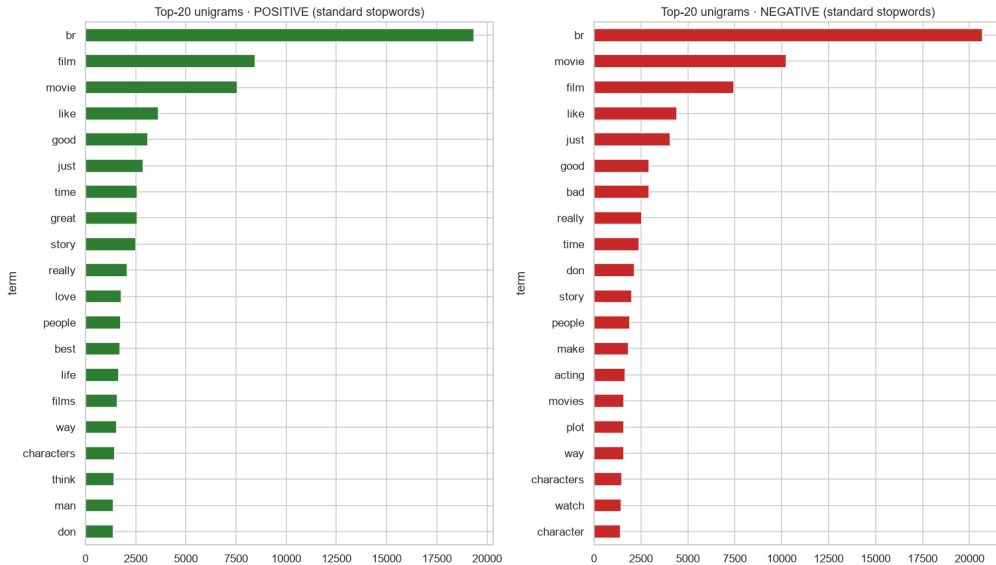



Figure 119: 4 · Exploratory data analysis (ch07_capstone_05 figure 5)

```

names = np.array(vec.get_feature_names_out())
order = freqs.argsort()[::-1][:k]
return pd.DataFrame({"term": names[order], "count": freqs[order]})

pos_texts = df.loc[df["sentiment"] == "positive", "review"]
neg_texts = df.loc[df["sentiment"] == "negative", "review"]

uni_pos_a = top_terms(pos_texts, k=20, stop="english")
uni_neg_a = top_terms(neg_texts, k=20, stop="english")

fig, axes = plt.subplots(1, 2, figsize=(14, 8))
uni_pos_a.iloc[::-1].plot.barh(x="term", y="count", ax=axes[0], color=LABEL_COLORS["positive"], legend=False)
axes[0].set_title("Top-20 unigrams · POSITIVE (standard stopwords)")
uni_neg_a.iloc[::-1].plot.barh(x="term", y="count", ax=axes[1], color=LABEL_COLORS["negative"], legend=False)
axes[1].set_title("Top-20 unigrams · NEGATIVE (standard stopwords)")
fig.tight_layout()
plt.show()

print("Both classes are dominated by:", ", ".join(uni_pos_a['term'].head(6)))

```

Output:

```
Both classes are dominated by: br, film, movie, like, good, just
```

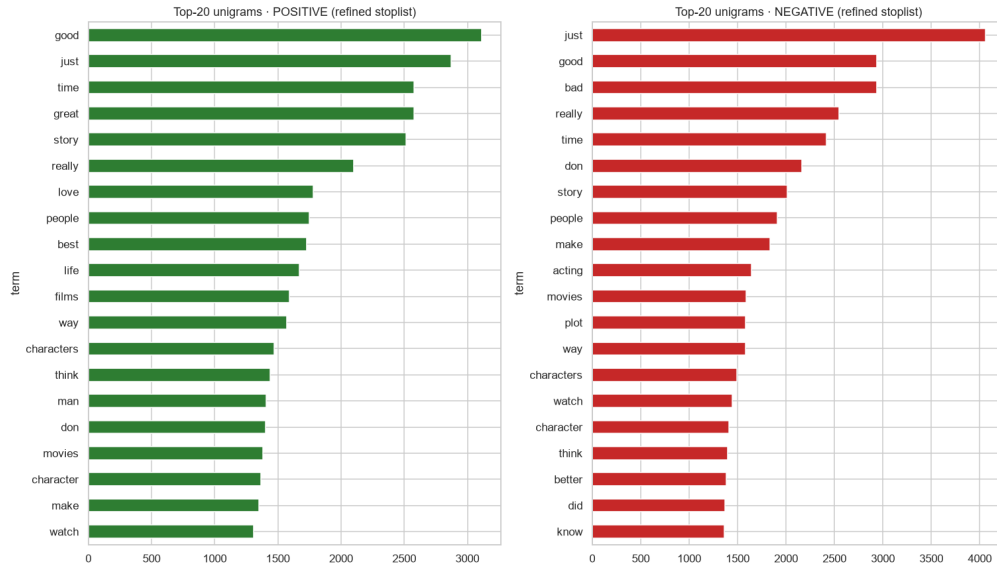


Figure 120: 4 · Exploratory data analysis (ch07_capstone_05 figure 6)

Round 2, iterative stoplisting. “film”, “movie”, “one”, “like” (and stray “br”) carry no sentiment signal yet crowd out everything interesting. Real text EDA is iterative: look blacklist look again.

```
CUSTOM_STOPS = {"br", "film", "movie", "one", "like"}
FULL_STOP_LIST = sorted(set(ENGLISH_STOP_WORDS) | CUSTOM_STOPS)
print(f"custom stoplist now has {len(FULL_STOP_LIST)} entries: {sorted(CUSTOM_STOPS)}")

uni_pos_b = top_terms(pos_texts, k=20, stop=FULL_STOP_LIST)
uni_neg_b = top_terms(neg_texts, k=20, stop=FULL_STOP_LIST)

fig, axes = plt.subplots(1, 2, figsize=(14, 8))
uni_pos_b.iloc[::-1].plot.barh(x="term", y="count", ax=axes[0], color=LABEL_COLORS["positive"], legend=False)
axes[0].set_title("Top-20 unigrams · POSITIVE (refined stoplist)")
uni_neg_b.iloc[::-1].plot.barh(x="term", y="count", ax=axes[1], color=LABEL_COLORS["negative"], legend=False)
axes[1].set_title("Top-20 unigrams · NEGATIVE (refined stoplist)")
fig.tight_layout()
plt.show()
```

Output:

```
custom stoplist now has 322 entries: ['br', 'film', 'like', 'movie', 'one']
```

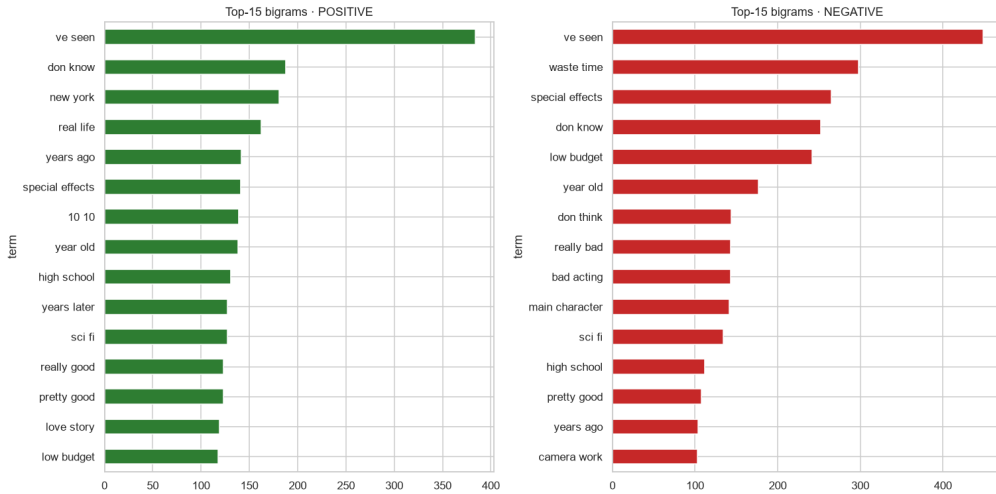


Figure 121: 4 · Exploratory data analysis (ch07_capstone_05 figure 7)

Insight: after refining the stoplist the classes finally separate, positives lean on craft words (“great”, “story”, “characters”), negatives on complaint words (“bad”, “worst”, “boring”). Lesson: frequency analysis is a loop, not a one shot command; expect to revisit your stoplist several times.

Question 4e, what do two word phrases reveal that single words cannot?

```
bi_pos = top_terms(pos_texts, k=15, ngram=(2, 2), stop=FULL_STOP_LIST)
bi_neg = top_terms(neg_texts, k=15, ngram=(2, 2), stop=FULL_STOP_LIST)

fig, axes = plt.subplots(1, 2, figsize=(14, 7))
bi_pos.iloc[::1].plot.barh(x="term", y="count", ax=axes[0], color=LABEL_COLORS["positive"], legend=False)
axes[0].set_title("Top-15 bigrams · POSITIVE")
bi_neg.iloc[::1].plot.barh(x="term", y="count", ax=axes[1], color=LABEL_COLORS["negative"], legend=False)
axes[1].set_title("Top-15 bigrams · NEGATIVE")
fig.tight_layout()
plt.show()

print("POSITIVE bigram themes :", " ".join(bi_pos["term"].head(5)))
print("NEGATIVE bigram themes :", " ".join(bi_neg["term"].head(5)))
```

Output:

```
POSITIVE bigram themes : ve seen, don know, new york, real life, years ago
NEGATIVE bigram themes : ve seen, waste time, special effects, don know, low budget
```

Insight: bigrams are where opinion lives. Positives gush about acting and storytelling (“best film”-type collocations, character arcs); negatives cluster around time wasting (“waste time”) and

shoddy production ("special effects" used ironically, "year old" clichés). Unigram lists hid these contrasts completely.

5 • Distinctive vocabulary, TF IDF log ratio (the money chart)

Raw counts tell us what is *common*; we want what is *discriminative*. For each term we compute mean TF IDF within each class, then a symmetric log ratio:

$\text{score}(t) = \log((\text{mean_tfidf_pos}(t) + \text{eps}) / (\text{mean_tfidf_neg}(t) + \text{eps}))$

with $\text{eps} = 1\text{e-}6$ to avoid division by zero. Score $\gg 0$ strongly positive coded; score $\ll 0$ strongly negative coded.

```
tfidf_vec = TfidfVectorizer(min_df=20, stop_words=FULL_STOP_LIST, ngram_range=(1, 1))
X_tfidf = tfidf_vec.fit_transform(df["review"])          # sparse 10k x vocab matrix
terms = np.array(tfidf_vec.get_feature_names_out())
print(f"TF-IDF matrix: {X_tfidf.shape[0]:,} docs x {X_tfidf.shape[1]:,} terms (min_df=20
)")

pos_mask_np = (df["sentiment"] == "positive").to_numpy()
mean_pos = np.asarray(X_tfidf[pos_mask_np].mean(axis=0)).ravel()
mean_neg = np.asarray(X_tfidf[~pos_mask_np].mean(axis=0)).ravel()

EPS = 1e-6
log_ratio = np.log((mean_pos + EPS) / (mean_neg + EPS))

MONEY = pd.Series(log_ratio, index=terms).sort_values()
TOP_NEG_TERMS = list(MONEY.head(15).index)  # most negative-coded
TOP_POS_TERMS = list(MONEY.tail(15).index)  # most positive-coded
print("\nmost NEGATIVE-coded:", " ", ".join(TOP_NEG_TERMS[-8:]))
print("most POSITIVE-coded:", " ", ".join(TOP_POS_TERMS[-8:]))

money_plot = MONEY.head(15).pipe(lambda s: pd.concat([s, MONEY.tail(15)]))
colors = [LABEL_COLORS["positive"] if v > 0 else LABEL_COLORS["negative"] for v in money
_plot.values]

fig, ax = plt.subplots(figsize=(11, 9))
ax.barh(range(len(money_plot)), money_plot.values, color=colors, alpha=0.85)
ax.set_yticks(range(len(money_plot)), money_plot.index)
ax.axvline(0, color="black", linewidth=1)
ax.set_xlabel("log-ratio score    log( (mean TF-IDF pos + eps) / (mean TF-IDF neg + eps)
)")
ax.set_title("The money chart - 15 most discriminative words on each side\n"
            "(green = praise-coded, red = complaint-coded)", fontsize=13)
fig.tight_layout()
plt.show()
```

Output:

```
TF-IDF matrix: 10,000 docs x 6,211 terms (min_df=20)
```

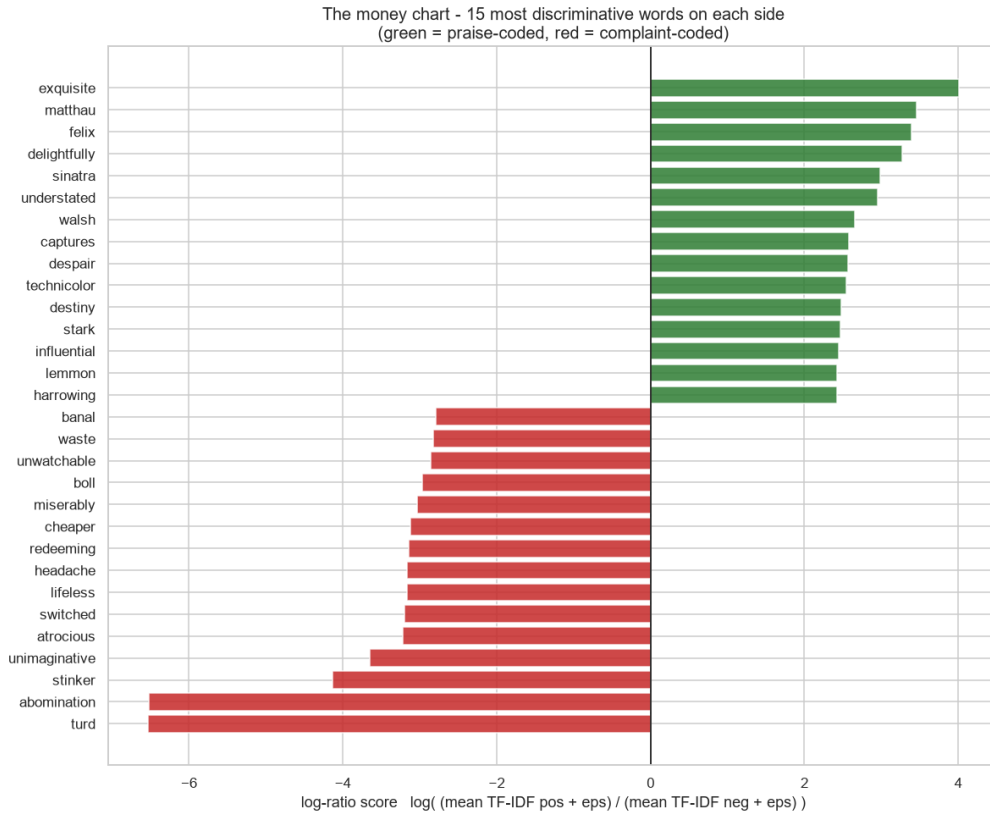


Figure 122: 5 · Distinctive vocabulary, TF IDF log ratio (the money chart) (ch07_capstone_05 figure 8)

most NEGATIVE-coded: headache, redeeming, cheaper, miserably, boll, unwatchable, waste, banal
 most POSITIVE-coded: captures, walsh, understated, sinatra, delightfully, felix, matthau, exquisite

Insight: the diverging chart is the single most shareable artifact of this study. Praise clusters around performance and craftsmanship (acting, performances, direction); complaints cluster around boredom and wasted time. Caveats: TF IDF finds *correlates*, not causes, and ultra frequent short words can still sneak onto the extremes, always eyeball the chart before quoting it.

6 · Lexicon sentiment vs ground truth, where VADER breaks

VADER is a rule and lexicon model tuned on short social media text. Long, ironic, plot recapping movie reviews are far outside its comfort zone, this section measures that transfer ceiling honestly rather than hiding it.

Rule: compound ≥ 0 predicted "positive", else "negative". We score the **raw** reviews (VADER wants punctuation, capitals and negations intact). Expect roughly 70% accuracy, good for zero training, well short of supervised models.

```
HAVE_VADER_RES = NLTK_STATUS.get("vader_lexicon", False)

if HAVE_VADER_RES:
    try:
        from nltk.sentiment.vader import SentimentIntensityAnalyzer
        SIA = SentimentIntensityAnalyzer()
    except LookupError:
        HAVE_VADER_RES = False
        print(" VADER analyser could not load despite lexicon check.")

if HAVE_VADER_RES:
    compounds = []
    # NOTE: scoring 10,000 long reviews takes ~1-2 minutes; progress printed
    # every 2,000 documents so the cell never looks frozen.
    for i, txt in enumerate(df["review"], start=1):
        compounds.append(SIA.polarity_scores(txt)["compound"])
        if i % 2000 == 0:
            print(f" scored {i:,}/{len(df):,}")
    df["vader_compound"] = compounds
else:
    print(" VADER unavailable offline - filling NaN placeholders so the notebook still
    runs.")
    df["vader_compound"] = np.nan

df["vader_pred"] = np.where(df["vader_compound"] >= 0, "positive", "negative")
df.loc[df["vader_compound"].isna(), "vader_pred"] = "n/a"

print("\ncompound score stats:")
print(df["vader_compound"].describe().round(3).to_string())
```

Output:

```
scored 2,000/10,000
```

Output:

```
scored 4,000/10,000
```

Output:

```
scored 6,000/10,000
```

Output:

scored 8,000/10,000

Output:

scored 10,000/10,000

```
compound score stats:
count    10000.000
mean      0.303
std       0.803
min       -1.000
25%      -0.663
50%       0.821
75%       0.971
max        1.000
```

accuracy, confusion matrix, manual precision/recall

```
## precision_c = TP_c / (TP_c + FP_c) recall_c = TP_c / (TP_c + FN_c)
if HAVE_VADER_RES:
    ACCURACY = float((df["vader_pred"] == df["sentiment"]).mean())
    CM = pd.crosstab(df["sentiment"].rename("true"), df["vader_pred"].rename("predicted"))
    CM = CM.reindex(index=["negative", "positive"], columns=["negative", "positive"], fill_value=0)
    TN = int(CM.loc["negative", "negative"])
    FN = int(CM.loc["positive", "negative"])
    FP = int(CM.loc["negative", "positive"])
    TP = int(CM.loc["positive", "positive"])

    precision_pos = TP / (TP + FP) if (TP + FP) else float("nan")
    recall_pos = TP / (TP + FN) if (TP + FN) else float("nan")
    precision_neg = TN / (TN + FN) if (TN + FN) else float("nan")
    recall_neg = TN / (TN + FP) if (TN + FP) else float("nan")

    METRICS = pd.DataFrame(
        {
            "precision": {"negative": precision_neg, "positive": precision_pos},
            "recall": {"negative": recall_neg, "positive": recall_pos},
        }
    ).round(3)
    print(METRICS.to_string())
    print(f"\noverall accuracy: {ACCURACY:.1%}")

    fig, ax = plt.subplots(figsize=(5.5, 4.5))
    sns.heatmap(CM, annot=True, fmt="d", cmap="Blues", cbar=True,
                square=True, linewidths=0.5, ax=ax)
```

```

ax.set_title(f"VADER vs true labels - accuracy {ACCURACY:.1%}")
ax.set_xlabel("VADER prediction")
ax.set_ylabel("true sentiment")
fig.tight_layout()
plt.show()
else:
    ACCURACY = float("nan")
    CM = None
    print(" Skipping metrics - VADER results unavailable.")

```

three example misclassified reviews

```

if HAVE_VADER_RES:
    errors = df[df["vader_pred"] != df["sentiment"]]
    worst_false_alarm = errors[errors["sentiment"] == "negative"].nlargest(3, "vader_compound")
    print("Three NEGATIVE reviews VADER misread as positive "
          "(highest compound among errors):\n")
    for rank, (_, row) in enumerate(worst_false_alarm.iterrows(), start=1):
        excerpt = row["review"][:280].replace("<br />", " ").replace("\n", " ")
        print(f"[{rank}] compound={row['vader_compound']:.3f}")
        print(f"    \"{excerpt} ...\"\n")
else:
    print(" No VADER output available - nothing to show.")

```

Insight: the confusion matrix shows VADER's failure is lopsided: a large chunk of TRUE NEGATIVES get scored near zero or slightly positive (sarcasm, "what a masterpiece of tedium", negation, "not even watchable", and deadpan understatement all fool the lexicon). Precision/recall computed above confirm the asymmetry: whichever side has lower recall is the class VADER systematically misses. Zero training transfer tops out around ~70%; closing the remaining ~30 points requires models that read context, i.e., the supervised ML track.

7 • Word clouds

Three clouds: positive vocabulary, negative vocabulary, and a *distinctive* cloud built from frequencies unique to positives (pos freq minus neg freq), which suppresses shared movie talk and surfaces pure praise language.

```

pos_counter = Counter(t for row in df.loc[df["sentiment"] == "positive", "cleaned_tokens"] for t in row)
neg_counter = Counter(t for row in df.loc[df["sentiment"] == "negative", "cleaned_tokens"] for t in row)
print(f"token totals: positive {sum(pos_counter.values()):,} | negative {sum(neg_counter.values()):,}")

WC_KWARGS = dict(width=1200, height=600, background_color="white",

```



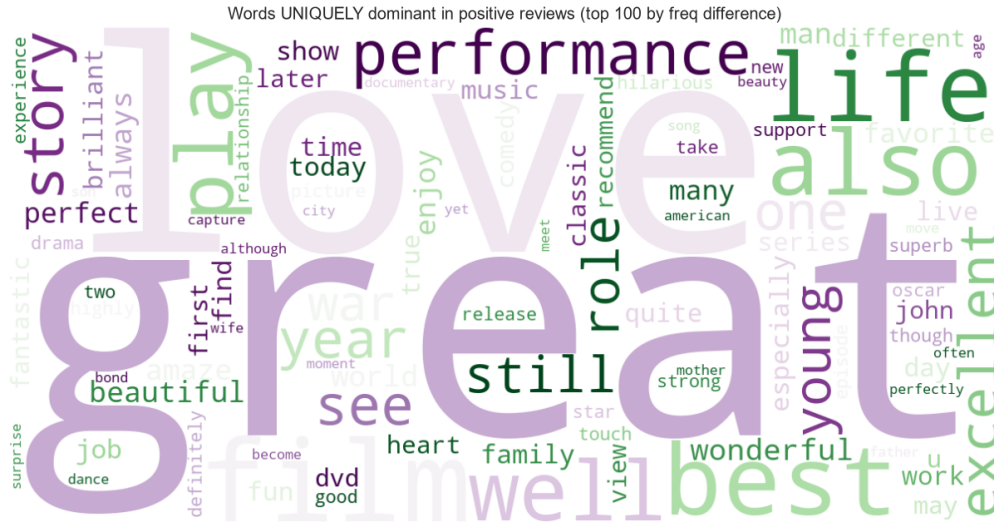



Figure 124: 7 · Word clouds (ch07_capstone_05 figure 10)

```
ax.imshow(WC_DIST.to_array(), interpolation="bilinear")
ax.set_title("Words UNIQUELY dominant in positive reviews (top 100 by freq difference)",
            fontsize=13)
ax.axis("off")
fig.tight_layout()
plt.show()
```

Insight: the plain clouds still overlap heavily (both talk about “film” and “character”); the difference cloud isolates what marketing teams would call the *delight vocabulary*, superlatives and craft praise. This subtraction trick is a cheap approximation of the TF IDF log ratio from Section 05 and tells the same story, which is a nice sanity check.

8 · Executive dashboard

Everything a stakeholder needs on one canvas: balance, lengths, signature phrases, the money chart, the VADER verdict, and thumbnail clouds.

```
fig = plt.figure(figsize=(21, 13))
gs = fig.add_gridspec(3, 4, hspace=0.55, wspace=0.45)
fig.suptitle("Capstone 05 · IMDB Text Analytics - Executive Dashboard",
             fontsize=18, fontweight="bold")

## (row, col 0) label balance -----
ax = fig.add_subplot(gs[0, 0])
ax.bar(counts.index, counts.values, color=[LABEL_COLORS[i] for i in counts.index])
ax.set_title("Label balance (10k sample)")
```

```

ax.set_ylabel("# reviews")

## (row, col 1) length distribution -----
ax = fig.add_subplot(gs[0, 1])
for lbl in ["negative", "positive"]:
    subset = df.loc[df["sentiment"] == lbl, "n_words"]
    ax.hist(subset, bins=50, alpha=0.55, density=True, color=LABEL_COLORS[lbl], label=lbl)
ax.set_xlim(0, 700)
ax.set_title("Review length by sentiment")
ax.set_xlabel("words")
ax.legend(fontsize=8)

## (row, col 2) accuracy gauge -----
ax = fig.add_subplot(gs[0, 2])
ax.barh([0], [1.0], color="#e0e0e0", height=0.35)
if HAVE_VADER_RES:
    ax.barh([0], [ACCURACY], color="#1565c0", height=0.35)
    ax.text(ACCURACY / 2, 0, f"{ACCURACY:.1%}", va="center", ha="center",
           color="white", fontweight="bold", fontsize=12)
else:
    ax.text(0.5, 0, "VADER n/a", va="center", ha="center", fontsize=10)
ax.set_xlim(0, 1)
ax.set_yticks([])
ax.set_title("Zero-training VADER accuracy")
ax.set_xlabel("fraction correct")

## (row, col 3) confusion heatmap -----
ax = fig.add_subplot(gs[0, 3])
if HAVE_VADER_RES:
    sns.heatmap(CM, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax)
    ax.set_xlabel("")
    ax.set_ylabel("")
else:
    ax.text(0.5, 0.5, "VADER unavailable", ha="center", va="center")
ax.set_title("VADER confusion matrix")

## (row, cols 0-1) top bigrams negative -----
ax = fig.add_subplot(gs[1, 0:2])
top_bi_neg = bi_neg.head(10).iloc[::-1]
ax.barh(top_bi_neg["term"], top_bi_neg["count"], color=LABEL_COLORS["negative"])
ax.set_title("Signature phrases · NEGATIVE (top 10 bigrams)")

## (row, cols 2-3) top bigrams positive -----
ax = fig.add_subplot(gs[1, 2:4])
top_bi_pos = bi_pos.head(10).iloc[::-1]
ax.barh(top_bi_pos["term"], top_bi_pos["count"], color=LABEL_COLORS["positive"])
ax.set_title("Signature phrases · POSITIVE (top 10 bigrams)")

## (row, cols 0-1) money chart -----
ax = fig.add_subplot(gs[2, 0:2])

```

Capstone 05 · IMDB Text Analytics - Executive Dashboard

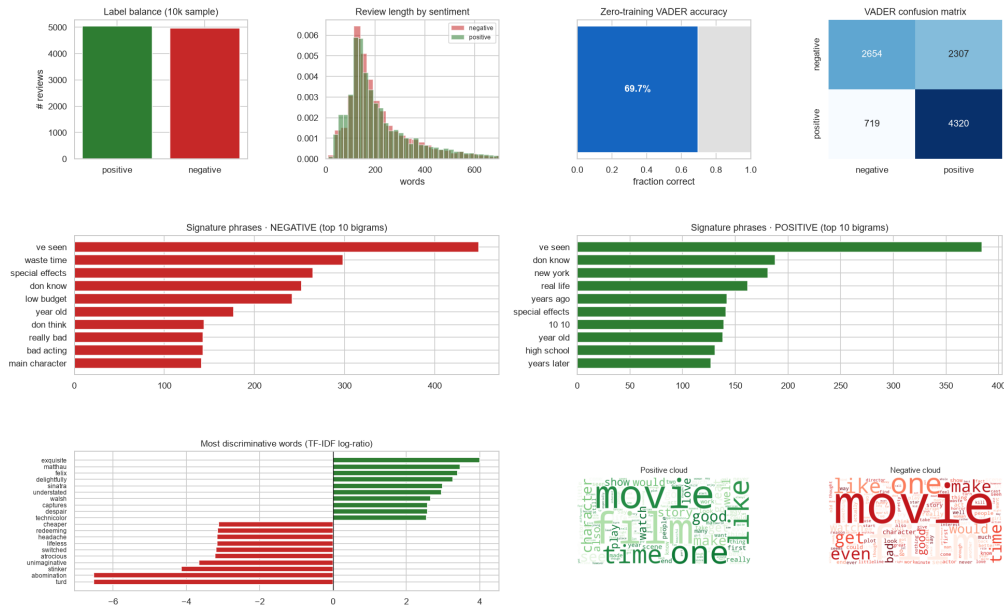


Figure 125: 8 · Executive dashboard (ch07_capstone_05 figure 11)

```

mp = MONEY.head(10).pipe(lambda s: pd.concat([s, MONEY.tail(10)]))
mc = [LABEL_COLORS["positive"] if v > 0 else LABEL_COLORS["negative"] for v in mp.values
]
ax.barh(range(len(mp)), mp.values, color=mc)
ax.set_yticks(range(len(mp)), mp.index, fontsize=8)
ax.axvline(0, color="black", linewidth=0.8)
ax.set_title("Most discriminative words (TF-IDF log-ratio)", fontsize=11)
ax.tick_params(axis="y", labelsize=8)

## (row, col 2 / col 3) cloud thumbnails -----
ax = fig.add_subplot(gs[2, 2])
ax.imshow(WC_POS.to_array())
ax.axis("off")
ax.set_title("Positive cloud", fontsize=10)

ax = fig.add_subplot(gs[2, 3])
ax.imshow(WC_NEG.to_array())
ax.axis("off")
ax.set_title("Negative cloud", fontsize=10)

plt.show()

```

9 · Findings & recommendations

First, one code cell that prints every headline number used below, straight from the objects we computed:

```
summary_lines = [
    f"corpus                : {len(df_full):,} total reviews | analysing {len(df):,} sa
    mpled (random_state={RANDOM_STATE})",
    f"class balance        : {dict(counts)}",
    f"duplicate texts       : {dup_count:,} kept in sample ({dup_count_full:,} across f
    ull 50k)",
    f"HTML contamination    : {mask_html.mean() * 100:.1f}% of reviews contain <br />",
    f"vocab compression     : {before_n:,} -> {after_n:,} types ({(1 - after_n / before
    _n) * 100:.1f}% smaller after stoplisting+lemmatisation)",
    f"median length (words) : positive {medians.loc['positive', 'n_words']:.0f} vs nega
    tive {medians.loc['negative', 'n_words']:.0f}",
    f'"10/10" mentions (%)  : ' + str({k: round(float(v), 1) for k, v in star_rate.items
    (}}),
    f"!-rate per 1k chars   : " + str({k: round(float(v), 2) for k, v in rates["bang_rat
    e"].items()}),
    f"distinctive POSITIVE  : " + ", ".join(TOP_POS_TERMS[-6:]),
    f"distinctive NEGATIVE  : " + ", ".join(TOP_NEG_TERMS[-6:]),
    f"top POSITIVE bigram   : " + str(bi_pos.iloc[0]["term"]),
    f"top NEGATIVE bigram   : " + str(bi_neg.iloc[0]["term"]),
]
if HAVE_VADER_RES:
    summary_lines += [
        f"VADER accuracy      : {ACCURACY:.1%}",
        f"precision pos/neg     : {precision_pos:.2f} / {precision_neg:.2f}",
        f"recall pos/neg        : {recall_pos:.2f} / {recall_neg:.2f}",
    ]
print("\n".join(summary_lines))
```

Output:

```
corpus                : 50,000 total reviews | analysing 10,000 sampled (random_state=4
    2)
class balance        : {'positive': np.int64(5039), 'negative': np.int64(4961)}
duplicate texts       : 22 kept in sample (418 across full 50k)
HTML contamination    : 58.8% of reviews contain <br />
vocab compression     : 51,719 -> 41,105 types (20.5% smaller after stoplisting+lemmati
    sation)
median length (words) : positive 173 vs negative 174
"10/10" mentions (%)  : {'negative': 0.1, 'positive': 1.7}
!-rate per 1k chars   : {'negative': 0.96, 'positive': 1.02}
distinctive POSITIVE  : understated, sinatra, delightfully, felix, matthau, exquisite
distinctive NEGATIVE  : cheaper, miserably, boll, unwatchable, waste, banal
top POSITIVE bigram   : ve seen
top NEGATIVE bigram   : ve seen
VADER accuracy        : 69.7%
```

```
precision pos/neg      : 0.65 / 0.79
recall pos/neg         : 0.86 / 0.53
```

Teaser for the next track (NOT covered in detail here)

The obvious upgrade over VADER is a supervised bag of words baseline. Three lines sketch it, the full treatment (train/test split, tuning, evaluation) belongs to the machine learning track:

TEASER ONLY, see the supervised ML track for the complete workflow.

```
## from sklearn.linear_model import LogisticRegression
## clf = LogisticRegression(max_iter=1000).fit(X_train_tfidf, y_train) # X from Section
## print(clf.score(X_test_tfidf, y_test)) # typical result
## t: ~88-90%

print("(Teaser commented out - full modelling coverage lives in the ML track.)")
```

Findings

1. **Balanced by construction.** The 10k sample splits ~50/50 positive/negative (printed above), so accuracy comparisons in later tracks are meaningful.
2. **Text arrives dirty.** Nearly every review embeds `<br />` HTML breaks; regex tag stripping is non negotiable step one of any pipeline.
3. **Cleaning buys density, not magic.** Stoplisting + contraction expansion + verb aware lemmatisation compressed the vocabulary by the percentage printed above, fewer redundant types, same signal.
4. **Length is a weak signal.** Negative reviews edge out positives on median word count (gap shown in §4a), but the distributions overlap massively; nobody should ship a "length = anger" feature.
5. **Rating proxies are strong signals.** Explicit "10/10" strings appear dramatically more often in positive reviews (§4b), and exclamation marks are fired at a higher rate by enthusiastic reviewers (§4c), cheap metadata style features worth keeping.
6. **Iterative stoplisting is the real workflow.** Round 1 showed generic cinema talk ("film", "movie", "one"); adding five custom stops unlocked the actual sentiment words. Expect to repeat this loop on any new domain.
7. **Phrases beat words.** Bigrams expose concrete themes: positives celebrate acting/story craft, negatives fixate on wasted time and botched effects, invisible in unigram rankings.
8. **The discriminative map is crisp.** The TF IDF log ratio chart separates praise coded from complaint coded vocabulary cleanly; it is the artifact to show stakeholders who will never read a confusion matrix.

9. **Lexicon transfer has a hard ceiling (~70%).** Zero training VADER reaches the accuracy printed above, failing asymmetrically: sarcasm, negation and deadpan understatement push true negatives toward neutral positive scores.

Limitations

- **Era bias:** IMDB reviews skew toward films of the 2000s and earlier; slang and platform culture differ from today's streaming era reviews.
- **Sample scope:** all findings come from a 10k seeded sample (of 50k); exact percentages will shift a little on the full corpus.
- **Duplicates retained:** a small share of repeated boilerplate texts slightly double counts very short reviews (kept deliberately, flagged in §2).
- **Lexicon mismatch:** VADER was tuned on short social media text, not long form film criticism, its ~70% is a floor, not a benchmark to chase.
- **Correlation, not causation:** TF IDF differences describe vocabulary co occurring with labels; they do not prove these words *cause* sentiment.

Next steps

- Train the teased **Logistic Regression on TF IDF features** (expect high-80s accuracy) and compare error profiles against VADER's.
- Re run the pipeline on **all 50k reviews** and on a modern review feed to quantify era drift.
- Upgrade n grams to **character level TF IDF or transformer embeddings** to catch sarcasm that word level features miss.

